

LeetMastery

Study-Order Edition

Python Only · Priority-Grouped ·

Difficulty-First · 1×1 Portrait

331 questions · 9 rounds · Interview Approach
High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (43) ↗

Arrays & Hashing (6) ↗

☐ ● #1 Two Sum ↗

☐ ● ★ #163 Missing Ranges ↗

☐ ● ★ #346 Moving Average from Data Stream ↗

☐ ● ★ #760 Find Anagram Mappings ↗

☐ ● ★ #1426 Counting Elements ↗

☐ ● #1534 Count Good Triplets ↗

String (3) ↗

☐ ● #383 Ransom Note ↗

☐ ● ★ #734 Sentence Similarity ↗

☐ ● ★ #1165 Single Row Keyboard ↗

Two Pointers (5) ↗

☐ ● #88 Merge Sorted Array ↗

☐ ● #125 Valid Palindrome ↗

☐ ● #283 Move Zeroes ↗

☐ ● ★ #408 Valid Word Abbreviation ↗

☐ ● #977 Squares of a Sorted Array ↗

Sorting (6) ↗

☐ ● #169 Majority Element ↗

☐ ● #217 Contains Duplicate ↗

☐ ● #242 Valid Anagram ↗

☐ ● ★ #252 Meeting Rooms ↗

☐ ● ★ #1086 Largest Unique Number ↗

☐ ● #1122 Relative Sort Array ↗

Binary Search (5) ↗

☐ ● #35 Search Insert Position ↗

☐ ● #69 Sqrt(x) ↗

☐ ● #278 First Bad Version ↗

☐ ● #374 Guess Number Higher or Lower ↗

☐ ● #704 Binary Search ↗

Matrix (5) ↗

☐ ● ★ #422 Valid Word Square ↗

☐ ● #566 Reshape the Matrix ↗

- ☐ ● #766 Toeplitz Matrix ↗
- ☐ ● #867 Transpose Matrix ↗
- ☐ ● #2373 Largest Local Values in a Matrix ↗
- Trees & BST (11)** ↗
- ☐ ● #100 Same Tree ↗
- ☐ ● #101 Symmetric Tree ↗
- ☐ ● #104 Maximum Depth of Binary Tree ↗
- ☐ ● #108 Convert Sorted Array to Binary Search Tree ↗
- ☐ ● #110 Balanced Binary Tree ↗
- ☐ ● #112 Path Sum ↗
- ☐ ● #226 Invert Binary Tree ↗
- ☐ ● ★ #270 Closest Binary Search Tree Value ↗
- ☐ ● #501 Find Mode in Binary Search Tree ↗
- ☐ ● #543 Diameter of Binary Tree ↗
- ☐ ● #572 Subtree of Another Tree ↗
- DFS (2)** ↗
- ☐ ● #463 Island Perimeter ↗

☐ ● #733 Flood Fill ↗

Round 2 | High | M Medium (116) ↗

Arrays & Hashing (11) ↗

☐ ● #57 Insert Interval ↗

☐ ● #238 Product of Array Except Self ↗

☐ ● #281 Zigzag Iterator ↗

☐ ● #307 Range Sum Query – Mutable ↗

☐ ● #454 4Sum II ↗

☐ ● #525 Contiguous Array ↗

☐ ● #560 Subarray Sum Equals K ↗

☐ ● #1010 Pairs of Songs With Total Durations
Divisible by 60 ↗

☐ ● ★ #1272 Remove Interval ↗

☐ ● ★ #1429 First Unique Number ↗

☐ ● #3159 Find Occurrences of an Element in
an Array ↗

String (5) ↗

☐ ● #8 String to Integer (atoi) ↗

☐ ● ★ #249 Group Shifted Strings ↗

☐ ● ★ #271 Encode and Decode Strings ↗

☐ ● #635 Design Log Storage System ↗

☐ ● #1138 Alphabet Board Path ↗

Two Pointers (14) ↗

☐ ● #11 Container With Most Water ↗

☐ ● #15 3Sum ↗

☐ ● #16 3Sum Closest ↗

☐ ● #31 Next Permutation ↗

☐ ● #75 Sort Colors ↗

☐ ● ★ #161 One Edit Distance ↗

☐ ● #167 Two Sum II – Input Array Is Sorted ↗

☐ ● ★ #186 Reverse Words in a String II ↗

☐ ● #189 Rotate Array ↗

☐ ● #532 K-diff Pairs in an Array ↗

☐ ● #923 3Sum With Multiplicity ↗

☐ ● #986 Interval List Intersections ↗

☐ ● ★ #1055 Shortest Way to Form String ↗

☐ ● #2563 Count the Number of Fair Pairs ↗

Sliding Window (8) ↗

- ☐ ● #3 Longest Substring Without Repeating Characters ↗
- ☐ ● ★ #159 Longest Substring with At Most Two Distinct Characters ↗
- ☐ ● ★ #340 Longest Substring with At Most K Distinct Characters ↗
- ☐ ● #424 Longest Repeating Character Replacement ↗
- ☐ ● #438 Find All Anagrams in a String ↗
- ☐ ● ★ #487 Max Consecutive Ones II ↗
- ☐ ● ★ #1100 Find K-Length Substrings with No Repeated Characters ↗
- ☐ ● #1248 Count Number of Nice Subarrays ↗

Sorting (3) ↗

- ☐ ● #49 Group Anagrams ↗
- ☐ ● #56 Merge Intervals ↗
- ☐ ● ★ #1152 Analyze User Website Visit Pattern ↗

Binary Search (12) ↗

- ☐ ● #33 Search in Rotated Sorted Array ↗

- ☐ ● #34 Find First and Last Position of Element in Sorted Array ↗
- ☐ ● #153 Find Minimum in Rotated Sorted Array ↗
- ☐ ● #300 Longest Increasing Subsequence ↗
- ☐ ● #362 Design Hit Counter ↗
- ☐ ● #528 Random Pick with Weight ↗
- ☐ ● #852 Peak Index in a Mountain Array ↗
- ☐ ● #981 Time Based Key-Value Store ↗
- ☐ ● #1011 Capacity to Ship Packages Within D Days ↗
- ☐ ● ★ #1060 Missing Element in Sorted Array ↗
- ☐ ● #1062 Longest Repeating Substring ↗
- ☐ ● ★ #1533 Find the Index of the Large Integer ↗
- Matrix (14)** ↗
- ☐ ● #36 Valid Sudoku ↗
- ☐ ● #48 Rotate Image ↗
- ☐ ● #54 Spiral Matrix ↗
- ☐ ● #59 Spiral Matrix II ↗

- ☐ ● #64 Minimum Path Sum ↗
- ☐ ● #73 Set Matrix Zeroes ↗
- ☐ ● #74 Search a 2D Matrix ↗
- ☐ ● #221 Maximal Square ↗
- ☐ ● ★ #311 Sparse Matrix Multiplication ↗
- ☐ ● #498 Diagonal Traverse ↗
- ☐ ● ★ #531 Lonely Pixel I ↗
- ☐ ● ★ #723 Candy Crush ↗
- ☐ ● #835 Image Overlap ↗
- ☐ ● ★ #1198 Find Smallest Common Element in All Rows ↗

Trees & BST (24) ↗

- ☐ ● #98 Validate Binary Search Tree ↗
- ☐ ● #102 Binary Tree Level Order Traversal ↗
- ☐ ● #103 Binary Tree Zigzag Level Order Traversal ↗
- ☐ ● #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- ☐ ● #199 Binary Tree Right Side View ↗

- ☐ ● #230 Kth Smallest Element in a BST ↗
- ☐ ● #235 Lowest Common Ancestor of a Binary Search Tree ↗
- ☐ ● #236 Lowest Common Ancestor of a Binary Tree ↗
- ☐ ● ★ #250 Count Univalue Subtrees ↗
- ☐ ● #285 Inorder Successor in BST ↗
- ☐ ● ★ #298 Binary Tree Longest Consecutive Sequence ↗
- ☐ ● ★ #314 Binary Tree Vertical Order Traversal ↗
- ☐ ● ★ #333 Largest BST Subtree ↗
- ☐ ● ★ #366 Find Leaves of Binary Tree ↗
- ☐ ● #437 Path Sum III ↗
- ☐ ● ★ #545 Boundary of Binary Tree ↗
- ☐ ● ★ #549 Binary Tree Longest Consecutive Sequence II ↗
- ☐ ● ★ #582 Kill Process ↗
- ☐ ● #662 Maximum Width of Binary Tree ↗

- ☐ ● #863 All Nodes Distance K in Binary Tree ↗
- ☐ ● ★ #1120 Maximum Average Subtree ↗
- ☐ ● ★ #1490 Clone N-ary Tree ↗
- ☐ ● ★ #1522 Diameter of N-Ary Tree ↗
- ☐ ● ★ #1650 Lowest Common Ancestor of a Binary Tree III ↗

DFS (16) ↗

- ☐ ● #130 Surrounded Regions ↗
- ☐ ● #133 Clone Graph ↗
- ☐ ● #200 Number of Islands ↗
- ☐ ● #207 Course Schedule ↗
- ☐ ● #210 Course Schedule II ↗
- ☐ ● ★ #261 Graph Valid Tree ↗
- ☐ ● #310 Minimum Height Trees ↗
- ☐ ● ★ #323 Number of Connected Components in an Undirected Graph ↗
- ☐ ● #417 Pacific Atlantic Water Flow ↗
- ☐ ● ★ #490 The Maze ↗

☐ ● ★ #694 Number of Distinct Islands ↗

☐ ● #695 Max Area of Island ↗

☐ ● #721 Accounts Merge ↗

☐ ● #785 Is Graph Bipartite? ↗

☐ ● ★ #1236 Web Crawler ↗

☐ ● #1242 Web Crawler Multithreaded ↗

Graphs (3) ↗

☐ ● #128 Longest Consecutive Sequence ↗

☐ ● ★ #277 Find the Celebrity ↗

☐ ● ★ #1136 Parallel Courses ↗

BFS (6) ↗

☐ ● ★ #286 Walls and Gates ↗

☐ ● #322 Coin Change ↗

☐ ● #542 01 Matrix ↗

☐ ● #994 Rotting Oranges ↗

☐ ● ★ #1197 Minimum Knight Moves ↗

☐ ● #1730 Shortest Path to Get Food ↗

Round 3 | High | H Hard (23) ↗

Arrays & Hashing (1) ↗

☐ ● #41 First Missing Positive ↗

Sliding Window (1) ↗

☐ ● #76 Minimum Window Substring ↗

Binary Search (7) ↗

☐ ● #4 Median of Two Sorted Arrays ↗

☐ ● #315 Count of Smaller Numbers After Self ↗

☐ ● #410 Split Array Largest Sum ↗

☐ ● #668 Kth Smallest Number in
Multiplication Table ↗

☐ ● #710 Random Pick with Blacklist ↗

☐ ● #774 Minimize Max Distance to Gas Station ↗

☐ ● #1235 Maximum Profit in Job Scheduling ↗

Matrix (1) ↗

☐ ● #1074 Number of Submatrices That Sum to
Target ↗

Trees & BST (2) ↗

☐ ● #124 Binary Tree Maximum Path Sum ↗

☐ ● #297 Serialize and Deserialize Binary Tree ↗

DFS (5) ↗

- ☐ ● ★ #269 Alien Dictionary ↗
- ☐ ● #329 Longest Increasing Path in a Matrix ↗
- ☐ ● #685 Redundant Connection II ↗
- ☐ ● #1036 Escape a Large Maze ↗
- ☐ ● #1192 Critical Connections in a Network ↗

Graphs (2) ↗

- ☐ ● ★ #305 Number of Islands II ↗
- ☐ ● #1153 String Transforms Into Another String ↗

BFS (4) ↗

- ☐ ● #127 Word Ladder ↗
- ☐ ● ★ #317 Shortest Distance from All Buildings ↗
- ☐ ● #773 Sliding Puzzle ↗
- ☐ ● #815 Bus Routes ↗

Round 4 | Mid | E Easy (12) ↗

Linked List (7) ↗

- ☐ ● #21 Merge Two Sorted Lists ↗

- ☐ ● #141 Linked List Cycle ↗
- ☐ ● #206 Reverse Linked List ↗
- ☐ ● #234 Palindrome Linked List ↗
- ☐ ● #706 Design HashMap ↗
- ☐ ● #876 Middle of the Linked List ↗
- ☐ ● ★ #1474 Delete N Nodes After M Nodes of Linked List ↗

Stack (3) ↗

- ☐ ● #20 Valid Parentheses ↗
- ☐ ● #232 Implement Queue using Stacks ↗
- ☐ ● #844 Backspace String Compare ↗

Trie (1) ↗

- ☐ ● #14 Longest Common Prefix ↗

Greedy (1) ↗

- ☐ ● #409 Longest Palindrome ↗

Round 5 | Mid | M Medium (56) ↗

Linked List (13) ↗

- ☐ ● #2 Add Two Numbers ↗
- ☐ ● #19 Remove Nth Node From End of List ↗

- ☐ ● #24 Swap Nodes in Pairs ↗
- ☐ ● #61 Rotate List ↗
- ☐ ● #146 LRU Cache ↗
- ☐ ● #148 Sort List ↗
- ☐ ● #328 Odd Even Linked List ↗
- ☐ ● ★ #369 Plus One Linked List ↗
- ☐ ● ★ #430 Flatten a Multilevel Doubly Linked List ↗
- ☐ ● #622 Design Circular Queue ↗
- ☐ ● #707 Design Linked List ↗
- ☐ ● ★ #708 Insert into a Sorted Circular Linked List ↗
- ☐ ● #1171 Remove Zero Sum Consecutive Nodes from Linked List ↗

Stack (14) ↗

- ☐ ● #143 Reorder List ↗
- ☐ ● #150 Evaluate Reverse Polish Notation ↗
- ☐ ● #155 Min Stack ↗
- ☐ ● #173 Binary Search Tree Iterator ↗

- ☐ ● #227 Basic Calculator II ↗
- ☐ ● ★ #255 Verify Preorder Sequence in Binary Search Tree ↗
- ☐ ● #394 Decode String ↗
- ☐ ● ★ #439 Ternary Expression Parser ↗
- ☐ ● ★ #484 Find Permutation ↗
- ☐ ● #735 Asteroid Collision ↗
- ☐ ● #739 Daily Temperatures ↗
- ☐ ● #962 Maximum Width Ramp ↗
- ☐ ● ★ #1214 Two Sum BSTs ↗
- ☐ ● ★ #1762 Buildings With an Ocean View ↗
- ☐ ● #Heap (11) ↗
- ☐ ● #215 Kth Largest Element in an Array ↗
- ☐ ● ★ #253 Meeting Rooms II ↗
- ☐ ● #347 Top K Frequent Elements ↗
- ☐ ● ★ #505 The Maze II ↗
- ☐ ● #621 Task Scheduler ↗
- ☐ ● #658 Find K Closest Elements ↗

- ☐ **● #787 Cheapest Flights Within K Stops ↗**
- ☐ **● #973 K Closest Points to Origin ↗**
- ☐ **● ★ #1057 Campus Bikes ↗**
- ☐ **● ★ #1167 Minimum Cost to Connect Sticks ↗**
- ☐ **● #1424 Diagonal Traverse II ↗**

Trie (7) ↗

- ☐ **● #139 Word Break ↗**
- ☐ **● #208 Implement Trie (Prefix Tree) ↗**
- ☐ **● #211 Design Add and Search Words Data Structure ↗**
- ☐ **● ★ #616 Add Bold Tag in String ↗**
- ☐ **● #692 Top K Frequent Words ↗**
- ☐ **● #720 Longest Word in Dictionary ↗**
- ☐ **● #1166 Design File System ↗**

Backtracking (6) ↗

- ☐ **● #17 Letter Combinations of a Phone Number**
- ☐ **● #22 Generate Parentheses ↗**
- ☐ **● #39 Combination Sum ↗**

☐ ● #46 Permutations ↗

☐ ● #79 Word Search ↗

☐ ● #113 Path Sum II ↗

Greedy (5) ↗

☐ ● #134 Gas Station ↗

☐ ● #179 Largest Number ↗

☐ ● ★ #280 Wiggle Sort ↗

☐ ● #954 Array of Doubled Pairs ↗

☐ ● #2131 Longest Palindrome by
Concatenating Two Letter Words

Round 6 | Mid | H Hard (28) ↗

Linked List (3) ↗

☐ ● #25 Reverse Nodes in k-Group ↗

☐ ● #432 All O'one Data Structure ↗

☐ ● #460 LFU Cache ↗

Stack (8) ↗

☐ ● #32 Longest Valid Parentheses ↗

☐ ● #42 Trapping Rain Water ↗

☐ ● #84 Largest Rectangle in Histogram ↗

☐ ● #224 Basic Calculator ↗

☐ ● #239 Sliding Window Maximum ↗

☐ ● #716 Max Stack ↗

☐ ● ★ #772 Basic Calculator III ↗

☐ ● #895 Maximum Frequency Stack ↗

Heap (9) ↗

☐ ● #23 Merge k Sorted Lists ↗

☐ ● #218 The Skyline Problem ↗

☐ ● ★ #272 Closest Binary Search Tree Value II ↗

☐ ● #295 Find Median from Data Stream ↗

☐ ● ★ #358 Rearrange String k Distance Apart ↗

☐ ● ★ #499 The Maze III ↗

☐ ● #632 Smallest Range Covering Elements
from K Lists ↗

☐ ● #759 Employee Free Time ↗

☐ ● #1425 Constrained Subsequence Sum ↗

Trie (4) ↗

- ☐ ● #212 Word Search II ↗
- ☐ ● #336 Palindrome Pairs ↗
- ☐ ● ★ #588 Design In-Memory File System ↗
- ☐ ● ★ #642 Design Search Autocomplete System ↗

Backtracking (4) ↗

- ☐ ● #37 Sudoku Solver ↗
- ☐ ● #51 N-Queens ↗
- ☐ ● #126 Word Ladder II ↗
- ☐ ● #996 Number of Squareful Arrays ↗

Round 7 | Low | E Easy (14) ↗

Dynamic Programming (2) ↗

- ☐ ● #70 Climbing Stairs ↗
- ☐ ● #121 Best Time to Buy and Sell Stock ↗

Bit Manipulation (6) ↗

- ☐ ● #67 Add Binary ↗
- ☐ ● #136 Single Number ↗
- ☐ ● #190 Reverse Bits ↗

☐ ● #191 Number of 1 Bits ↗

☐ ● #268 Missing Number ↗

☐ ● #338 Counting Bits ↗

Math (6) ↗

☐ ● #9 Palindrome Number ↗

☐ ● #13 Roman to Integer ↗

☐ ● #504 Base 7 ↗

☐ ● ★ #1056 Confusing Number ↗

☐ ● ★ #1228 Missing Number in Arithmetic Progression ↗

☐ ● ★ #1427 Perform String Shifts ↗

Round 8 | Low | M Medium (32) ↗

Dynamic Programming (16) ↗

☐ ● #5 Longest Palindromic Substring ↗

☐ ● #53 Maximum Subarray ↗

☐ ● #55 Jump Game ↗

☐ ● #62 Unique Paths ↗

☐ ● #72 Edit Distance ↗

- ☐ ● #91 Decode Ways ↗
- ☐ ● #152 Maximum Product Subarray ↗
- ☐ ● #198 House Robber ↗
- ☐ ● ★ #256 Paint House ↗
- ☐ ● #309 Best Time to Buy and Sell Stock with ↗
Cooldown
- ☐ ● #377 Combination Sum IV ↗
- ☐ ● #416 Partition Equal Subset Sum ↗
- ☐ ● #435 Non-overlapping Intervals ↗
- ☐ ● #516 Longest Palindromic Subsequence ↗
- ☐ ● #576 Out of Boundary Paths ↗
- ☐ ● #1143 Longest Common Subsequence ↗

Bit Manipulation (4) ↗

- ☐ ● #78 Subsets ↗
- ☐ ● #90 Subsets II ↗
- ☐ ● #287 Find the Duplicate Number ↗
- ☐ ● ★ #1506 Find Root of N-Ary Tree ↗

Math (5) ↗

- ☐ **● #7 Reverse Integer** ↗
- ☐ **● #50 Pow(x, n)** ↗
- ☐ **● #380 Insert Delete GetRandom O(1)** ↗
- ☐ **● #1017 Convert to Base -2** ↗
- ☐ **● #3160 Find the Number of Distinct Colors Among the Balls** ↗

JavaScript (7) ↗

- ☐ **● ★ #2627 Debounce** ↗
- ☐ **● ★ #2628 JSON Deep Equal** ↗
- ☐ **● ★ #2630 Memoize** ↗
- ☐ **● ★ #2633 Convert Object to JSON String** ↗
- ☐ **● ★ #2636 Promise Pool** ↗
- ☐ **● ★ #2675 Array of Objects to Matrix** ↗
- ☐ **● ★ #2700 Differences Between Two Objects** ↗

Round 9 | Low | H Hard (7) ↗

Dynamic Programming (6) ↗

- ☐ **● #10 Regular Expression Matching** ↗
- ☐ **● #115 Distinct Subsequences** ↗

☐ ● #123 Best Time to Buy and Sell Stock III ↗

☐ ● #132 Palindrome Partitioning II ↗

☐ ● #312 Burst Balloons ↗

☐ ● #1000 Minimum Cost to Merge Stones ↗

Math (1) ↗

☐ ● #68 Text Justification ↗

Round 1 · High · Easy

Round 1

High Priority · Easy

43 questions · 8 patterns

**Arrays & Hashing #1 #163 #346 #760 #1426
#1534**

String #383 #734 #1165

Two Pointers #88 #125 #283 #408 #977

Sorting #169 #217 #242 #252 #1086 #1122

Binary Search #35 #69 #278 #374 #704

Matrix #422 #566 #766 #867 #2373

**Trees & BST #100 #101 #104 #108 #110 #112
#226 #270 #501 #543 #572**

DFS #463 #733

Arrays & Hashing

Round 1 · High · Easy · 6 questions

Arrays & Hashing

□ #1 Two Sum

EAS

[Open on LeetCode](#)

Array · Hash Table

Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We're given an array of integers and a target value,

and we need to return the indices of the two numbers

that add up to the target – not the values, the indices. Right?"

```
#  
# "A few quick questions:  
# Can the same element be used twice? I'm  
guessing no.  
# Is there always exactly one valid  
answer?  
# Can the array be empty or have only one  
element?  
# Are there negative numbers or  
duplicates?"  
#  
# "Let me walk the example: nums = [2, 7,  
11, 15], target = 9.  
# 2 + 7 = 9, so we return [0, 1]. Got it."  
  
# PHASE 2 — BRUTE FORCE  
# "Let me start with brute force to lock  
in the logic.  
# For every pair (i, j) where j > i, check  
if nums[i] + nums[j] equals target.  
# Time:  $O(n^2)$  from the nested loops.  
Space:  $O(1)$ , no extra storage.  
# Should I go ahead and optimize?"  
class _1_TwoSum_Brute:  
    def twoSum(self, nums: List[int],  
target: int) -> List[int]:
```

```
        for i in range(len(nums)):
            for j in range(i + 1,
len(nums)):
                if nums[i] + nums[j] ==
target:
                    return [i, j]
    return []
```

PHASE 3 — OPTIMIZE

"The bottleneck is searching the rest of the array for a complement each time.

I notice we're repeatedly asking: does the number I need exist?

A hash map answers that in $O(1)$.

#

Walk forward through the array. For each number, compute complement = target - num.

If complement is already in the map, return both indices.

Otherwise, store the current number and index and keep going.

#

Pseudocode:

seen = {}

for index, num in nums:

complement = target - num

```
# if complement in seen: return  
[seen[complement], index]  
# seen[num] = index  
#  
# Time: O(n). Space: O(n). Does that make  
sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _1_TwoSum:  
    def twoSum(self, nums: List[int],  
target: int) -> List[int]:  
        seen = {} # maps value -> index  
        for index, num in  
enumerate(nums):  
            complement = target - num  
            if complement in seen:  
                return [seen[complement],  
index]  
            seen[num] = index  
        return []
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."  
#
```

```
# nums=[2,7,11,15], target=9
# index=0: complement=7, not in seen.
seen={2:0}
# index=1: complement=2, found at 0.
return [0,1] ✓
#
# nums=[3,2,4], target=6
# index=0: complement=3, not in seen.
seen={3:0}
# index=1: complement=4, not in seen.
seen={3:0,2:1}
# index=2: complement=2, found at 1.
return [1,2] ✓
#
# nums=[3,3], target=6
# index=0: complement=3, not in seen.
seen={3:0}
# index=1: complement=3, found at 0.
return [0,1] ✓
#
# "No bugs. Holds across all cases."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$  — one pass. Space  $O(n)$  —
worst case every element enters the map.
```

```
# Trade-off:  $O(1)$  space would require  
 $O(n^2)$  time – no free lunch here.  
# If the array were sorted I'd use two  
pointers:  $O(n)$  time,  $O(1)$  space.  
# But we need original indices and the  
input is unsorted, so hash map wins.  
# Given more time I'd ask: what if there  
are multiple valid pairs?  
# We'd collect all of them rather than  
returning on the first hit."
```

Arrays & Hashing

□ ★ #163 Missing Ranges ★

EAS

[Open on LeetCode](#)

Array

Lists: ★ Premium | Premium 98

Problem

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

Input: `nums = [0,1,3,50,75]`, `lower = 0`,
`upper = 99`

Output: `[[2,2],[4,49],[51,74],[76,99]]`

Explanation: The ranges are:

`[2,2]`

`[4,49]`

`[51,74]`

`[76,99]`

Example 2:

Input: `nums = [-1]`, `lower = -1`, `upper = -1`

Output: `[]`

Explanation: There are no missing ranges since there are no missing numbers.

Constraints:

- $-109 \leq \text{lower} \leq \text{upper} \leq 109$
- $0 \leq \text{nums.length} \leq 100$
- $\text{lower} \leq \text{nums}[i] \leq \text{upper}$
- All the values of `nums` are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have a sorted array of unique integers, and an inclusive range [lower, upper].

We need to return the list of ranges that cover every number

inside [lower, upper] that does NOT appear in nums. Right?"

#

"A few quick questions:

Can nums be empty? If so I assume the whole [lower, upper] is one missing range.

Are all elements in nums guaranteed to be within [lower, upper]? Good.

The output ranges should be as compact as possible – so [2,2] not just [2]?"

#

"Let me walk the example:

nums=[0,1,3,50,75], lower=0, upper=99.

0 and 1 are present so no gap at the start.

3 is present but 2 is missing → [2,2].

50 is present but 4–49 are missing → [4,49].

```
# 75 is present but 51–74 are missing →  
[51,74].  
# 76–99 are all missing → [76,99].  
# Output: [[2,2],[4,49],[51,74],[76,99]].  
Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# The simplest approach: build a set from  
nums for O(1) lookups,  
# then walk every integer from lower to  
upper.
```

```
# Whenever I hit a missing number, I  
extend the current gap range.
```

```
# When I hit a number that IS in the set,  
I close the gap and record it.
```

```
#
```

```
# Time: O(upper – lower) which can be up  
to 2 billion – way too slow.
```

```
# Space: O(n) for the set.
```

```
# Should I optimize?"
```

```
class _163_MissingRanges_Brute:  
    def findMissingRanges(self, nums:  
List[int], lower: int, upper: int) ->  
List[List[int]]:
```

```
present = set(nums)
result = []
gap_start = None
for x in range(lower, upper + 1):
    if x not in present:
        if gap_start is None:
            gap_start = x
        else:
            if gap_start is not None:
                result.append([gap_start, x - 1])
            gap_start = None
    if gap_start is not None:
        result.append([gap_start, upper])
return result
```

PHASE 3 — OPTIMIZE

"The brute force iterates over the entire numerical range – up to 2 billion steps.

But nums has at most 100 elements. The gaps I care about

only exist between consecutive numbers in nums, before the first, and after the last.

So I only need to check those boundaries
– that's $O(n)$, not $O(\text{range})$.

#

I think of nums as a set of walls. The
missing ranges are the gaps between walls.

There are at most $n+1$ gaps: one before
nums[0], one between each pair, one after
nums[-1].

#

Pseudocode:

if nums is empty: return [[lower,
upper]]

if lower < nums[0]: add [lower,
nums[0]-1]

for each consecutive pair (prev, curr)
in nums:

if prev+1 < curr: add [prev+1, curr-1]

if nums[-1] < upper: add [nums[-1]+1,
upper]

#

Time: $O(n)$. Space: $O(1)$ extra. Does that
make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach."

```
# I'll use pairwise from itertools to
cleanly iterate consecutive pairs."
class _163_MissingRanges:
    def findMissingRanges(self, nums:
List[int], lower: int, upper: int) ->
List[List[int]]:
        if not nums:
            return [[lower, upper]]
        result = []
        first, last = nums[0], nums[-1]
        if lower < first:
            result.append([lower, first -
1])

        for prev, curr in pairwise(nums):
            if prev + 1 < curr:
                result.append([prev + 1,
curr - 1])
            if last < upper:
                result.append([last + 1,
upper])

        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
```

```
# nums=[0,1,3,50,75], lower=0, upper=99
# nums not empty. first=0, last=75.
# lower(0) < first(0)? No. No leading gap.
# pairwise: (0,1) → 1 < 1? No. (1,3) → 2 < 3? Yes → [2,2]. (3,50) → 4 < 50? Yes → [4,49]. (50,75) → 51 < 75? Yes → [51,74].
# last(75) < upper(99)? Yes → [76,99].
# result = [[2,2],[4,49],[51,74],[76,99]]
✓
#
# nums=[-1], lower=-1, upper=-1
# first=-1, last=-1.
# lower(-1) < first(-1)? No.
# pairwise: empty (only one element).
# last(-1) < upper(-1)? No.
# result = [] ✓
#
# nums=[], lower=1, upper=5
# Empty → return [[1,5]] ✓
#
# "No bugs. All cases pass."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – one pass through nums plus constant work per gap.

```
# Space  $O(1)$  extra – result list aside, no  
auxiliary storage.  
# This is a massive improvement over  
 $O(\text{upper} - \text{lower})$  brute force  
# given the constraint that nums has at  
most 100 elements.  
#  
# Trade-off: if nums were not sorted I'd  
need to sort first –  $O(n \log n)$ .  
# The problem guarantees sorted input so  
we skip that.  
#  
# pairwise is Python 3.10+. If the  
interviewer is on an older version  
# I'd write: for i in range(1, len(nums)):  
prev, curr = nums[i-1], nums[i].  
#  
# Given more time I'd ask: what if the  
output should be strings like '2->2'  
# instead of lists? Just a formatting  
change at the append step."
```


Arrays & Hashing

□ ★ #346 Moving Average from Data Stream ★

EAS

[Open on LeetCode](#)

Array · Design · Queue · Data Stream

Lists: ★ Premium | Premium 98

Problem

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the `MovingAverage` class:

- `MovingAverage(int size)` Initializes the object with the size of the window size.
- `double next(int val)` Returns the moving average of the last size values of the stream.

Example 1:

Input

```
["MovingAverage", "next", "next",  
"next", "next"]
```

```
[[3], [1], [10], [3], [5]]
```

Output

```
[null, 1.0, 5.5, 4.66667, 6.0]
```

Explanation

```
MovingAverage movingAverage = new  
MovingAverage(3);
```

Constraints:

- $1 \leq \text{size} \leq 1000$
- $-105 \leq \text{val} \leq 105$
- At most 104 calls will be made to next.

◆ Interview Approach · STAR-LC**# PHASE 1 — CLARIFY**

"Let me make sure I understand the problem correctly."

We're designing a class that receives a stream of integers one at a time.

Each call to next() should return the average of the last 'size' values seen so far.

If fewer than 'size' values have arrived, we average however many we have. Right?"

#

"A few quick questions:

Can size be 1? Constraints say yes – that's just returning the latest value.

Can values be negative? Yes, constraints say down to -10^5 .

Is there a limit on how many next() calls we get? Up to 10^4 – good to know for space."

#

"Let me walk the example:

size=3. next(1)=1.0 (only 1 value).
next(10)=5.5 (avg of 1,10).

next(3)=4.667 (avg of 1,10,3 – window full now).

next(5)=6.0 (avg of 10,3,5 – 1 falls out). Got it."

PHASE 2 — BRUTE FORCE

"The simplest approach: store every value in a plain list.

On each next() call, slice the last 'size' elements and compute the average.

```
#  
# Time:  $O(\text{size})$  per next() call due to the  
# slice and sum.  
# Space:  $O(n)$  where n is total calls – the  
# list grows forever.  
# Should I optimize?"
```

```
class _346_MovingAverage_Brute:  
    def __init__(self, size: int):  
        self.size = size  
        self.stream = []  
  
    def next(self, val: int) -> float:  
        self.stream.append(val)  
        window = self.stream[-self.size:]  
        return sum(window) / len(window)
```

PHASE 3 — OPTIMIZE

```
# "Two inefficiencies in the brute force:  
# First, we recompute the sum from scratch  
# every call – that's  $O(\text{size})$  work.  
# Second, we store the entire history but  
# only ever need the last 'size' values.  
#  
# I notice we're recomputing the same sum  
# repeatedly. What if we cache it?  
# I'll keep a running sum and update it  
# incrementally:
```

```
# when a new value comes in, add it. When
the window is full,
# subtract the oldest value before adding
the new one.
#
# For the window itself, a deque is
perfect –  $O(1)$  append on the right,
#  $O(1)$  popleft when the oldest value falls
out. The deque never exceeds 'size'
elements.
#
# Pseudocode:
# __init__: q = deque(), running_sum = 0,
self.size = size
# next(val):
# if len(q) == size: running_sum -=
q.popleft()
# running_sum += val
# q.append(val)
# return running_sum / len(q)
#
# Time:  $O(1)$  per next() call. Space:
 $O(\text{size})$  – the deque is capped.
# Does that make sense before I write it
out?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _346_MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.window = deque()
        self.running_sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.size:
            self.running_sum -=
self.window.popleft() # evict oldest
            self.running_sum += val
            self.window.append(val)
            return self.running_sum /
len(self.window)
```

PHASE 5 — TEST & DEBUG

"Let me trace through the example: size=3."

#

next(1): window not full. sum=1,
window=[1]. return 1/1 = 1.0 ✓

next(10): window not full. sum=11,
window=[1,10]. return 11/2 = 5.5 ✓

```
# next(3): window not full. sum=14,  
window=[1,10,3]. return 14/3 = 4.667 ✓  
# next(5): window full → popleft() removes  
1, sum=14-1=13.  
# sum=13+5=18, window=[10,3,5]. return  
18/3 = 6.0 ✓  
#  
# Edge – size=1:  
# next(7): window full immediately after  
first call? No – starts empty.  
# sum=7, window=[7]. return 7/1 = 7.0 ✓  
# next(4): window full (size=1). popleft  
removes 7, sum=0+4=4, window=[4].  
# return 4/1 = 4.0 ✓  
#  
# "No bugs. Running sum stays consistent  
across evictions."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(1)$  per next() call – all  
operations are constant: deque append,  
# popleft, addition, division.  
# Space  $O(\text{size})$  – the deque holds at most  
'size' elements regardless of  
# how many total calls are made. Much  
better than the  $O(n)$  brute force.
```

#

Trade-off worth mentioning: floating point precision. If values are large and size is large, the running sum can accumulate rounding errors over time.

A production system might use Python's Decimal or periodically recompute the sum from scratch to correct drift.

#

Alternative: a circular buffer (fixed-size list with a pointer) does the same

thing with slightly better cache locality than a deque, but the deque is cleaner to read and interview-safe.

#

Given more time I'd ask: what if we need to support a variable window size that can change between calls? We'd need to resize the deque dynamically."

Arrays & Hashing

□ ★ #760 Find Anagram Mappings ★

EAS

[Open on LeetCode](#)

Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

You are given two integer arrays `nums1` and `nums2` where `nums2` is an anagram of `nums1`. Both arrays may contain duplicates.

Return an index mapping array mapping from `nums1` to `nums2` where `mapping[i] = j` means the *i*th element in `nums1` appears in `nums2` at index *j*. If there are multiple answers, return any of them.

An array *a* is an anagram of an array *b* means *b* is made by randomizing the order of the elements in *a*.

Example 1:

Input: nums1 = [12,28,46,32,50], nums2 = [50,12,32,46,28]

Output: [1,4,3,2,0]

Explanation: As mapping[0] = 1 because the 0th element of nums1 appears at nums2[1], and mapping[1] = 4 because the 1st element of nums1 appears at nums2[4], and so on.

Example 2:

Input: nums1 = [84,46], nums2 = [84,46]

Output: [0,1]

Constraints:

- $1 \leq \text{nums1.length} \leq 100$
- $\text{nums2.length} == \text{nums1.length}$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 105$
- nums2 is an anagram of nums1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We have two arrays nums1 and nums2. nums2 is an anagram of nums1 –

```
# same elements, different order. We need
to return a mapping array
# where mapping[i] is the index in nums2
where nums1[i] appears. Right?"
#
# "A few quick questions:
# Can there be duplicate values? The
problem says yes.
# If a value appears multiple times in
nums2, which index do we return?
# The problem says any valid answer is
accepted – good, that simplifies things.
# Both arrays are the same length and
nums2 is guaranteed to be an anagram,
# so every element in nums1 is guaranteed
to exist in nums2?"
#
# "Let me walk the example:
# nums1=[12,28,46,32,50],
nums2=[50,12,32,46,28].
# 12 is at index 1 in nums2 →
mapping[0]=1.
# 28 is at index 4 → mapping[1]=4.
# 46 is at index 3 → mapping[2]=3.
# 32 is at index 2 → mapping[3]=2.
```

```
# 50 is at index 0 → mapping[4]=0.
```

```
# Output: [1,4,3,2,0]. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic.
```

```
# For each element in nums1, scan nums2  
left to right until I find a match
```

```
# and record that index.
```

```
#
```

```
# Time:  $O(n^2)$  – for each of  $n$  elements in  
nums1 we do a linear scan of nums2.
```

```
# Space:  $O(1)$  extra beyond the output  
array.
```

```
# Should I optimize?"
```

```
class _760_AnagramMappings_Brute:
```

```
    def anagramMappings(self, nums1:  
List[int], nums2: List[int]) ->  
List[int]:
```

```
        mapping = []
```

```
        for num in nums1:
```

```
            for j, val in
```

```
enumerate(nums2):
```

```
                if val == num:
```

```
                    mapping.append(j)
```

```
                    break
```

return mapping

PHASE 3 — OPTIMIZE

"The bottleneck is scanning nums2 from scratch for every element in nums1.

We're asking the same question repeatedly: where does this value live in nums2?

A hash map lets us answer that in $O(1)$.

#

I'll preprocess nums2 once: build a map from value $\rightarrow$ index.

Then for each element in nums1, look it up directly.

#

One thing to flag: if there are duplicates, building the map will overwrite

earlier indices with later ones. Since the problem allows any valid answer,

that's fine – we just return whichever index was stored last.

#

Pseudocode:

`index_in_nums2 = {val: i for i, val in enumerate(nums2)}`

```
# return [index_in_nums2[num] for num in
nums1]
#
# Time: O(n). Space: O(n) for the hash
map.
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _760_AnagramMappings:
    def anagramMappings(self, nums1:
List[int], nums2: List[int]) ->
List[int]:
        index_in_nums2 = {val: i for i,
val in enumerate(nums2)}
        return [index_in_nums2[num] for
num in nums1]

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums1=[12,28,46,32,50],
nums2=[50,12,32,46,28]
# index_in_nums2 = {50:0, 12:1, 32:2,
46:3, 28:4}
```

```
# nums1[0]=12 → 1
# nums1[1]=28 → 4
# nums1[2]=46 → 3
# nums1[3]=32 → 2
# nums1[4]=50 → 0
# Output: [1,4,3,2,0] ✓
#
# nums1=[84,46], nums2=[84,46]
# index_in_nums2 = {84:0, 46:1}
# Output: [0,1] ✓
#
# Duplicate case – nums1=[3,3],
# nums2=[3,3]
# enumerate(nums2): (0,3) → map={3:0},
# then (1,3) → map={3:1} (overwritten).
# nums1[0]=3 → 1, nums1[1]=3 → 1. Output:
# [1,1].
# Both point to the same index – valid
# since any answer is accepted ✓
#
# "No bugs. Duplicate overwrite is
# intentional and within spec."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – one pass to build the map,
# one pass to build the result.
```

```
# Space  $O(n)$  for the hash map.  
#  
# The duplicate handling is worth calling  
# out explicitly:  
# by iterating enumerate(nums2) forward,  
# later indices overwrite earlier ones.  
# If the interviewer wanted ALL valid  
# indices for duplicates returned,  
# I'd change the map to store a list of  
# indices per value and pop one per lookup.  
#  
# Trade-off: if n is tiny (say n=5 like in  
# the example), the brute force  $O(n^2)$   
# is barely slower and uses no extra  
# space. The hash map pays off as n grows.  
#  
# Given more time I'd ask: what if we need  
# to return the mapping in sorted order  
# by nums2 index? We'd sort the output –  
#  $O(n \log n)$  – but that changes the spec."
```


Arrays & Hashing

□ ★ #1426 Counting Elements ★

EAS

[Open on LeetCode](#)

Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

Given an integer array `arr`, count how many elements `x` there are, such that `x + 1` is also in `arr`. If there are duplicates in `arr`, count them separately.

Example 1:

Input: `arr = [1,2,3]`

Output: 2

Explanation: 1 and 2 are counted cause 2 and 3 are in `arr`.

Example 2:

Input: `arr = [1,1,3,3,5,5,7,7]`

Output: 0

Explanation: No numbers are counted, cause there is no 2, 4, 6, or 8 in `arr`.

Constraints:

- $1 \leq \text{arr.length} \leq 1000$
- $0 \leq \text{arr}[i] \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

For each element x in the array, we check whether $x+1$ also exists in the array.

If it does, we count x . And if x appears multiple times, we count each occurrence

separately – so duplicates all contribute individually. Right?"

#

"A few quick questions:

Can the array be empty? Constraints say $\text{length} \geq 1$, so no.

Can values be 0? Yes – so $x=0$ counts if 1 is present.

Can values be negative? Constraints say $0 \leq \text{arr}[i] \leq 1000$, so no.

Are there duplicates? Yes, and the problem explicitly says to count them

separately."

#

"Let me walk the examples:

arr=[1,2,3]: 1+1=2 is in arr → count 1.
2+1=3 is in arr → count 2. 3+1=4 not in arr.

Output: 2 ✓

arr=[1,1,3,3,5,5,7,7]: 1+1=2 not in arr.
3+1=4 not in arr. Same for 5,7.

Output: 0 ✓ Got it."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each element x in arr, scan the entire array to see if x+1 exists.

If it does, increment the count.

#

Time: $O(n^2)$ – nested scan for every element.

Space: $O(1)$ – no extra storage.

Should I optimize?"

class _1426_CountingElements_Brute:

def countElements(self, arr:

List[int]) -> int:

count = 0

```
for x in arr:
    for y in arr:
        if y == x + 1:
            count += 1
            break
    return count
```

PHASE 3 — OPTIMIZE

"The bottleneck is the inner scan – we're asking 'does x+1 exist?' n times, # each taking $O(n)$. A hash set answers that in $O(1)$.

#

But there's a subtlety: duplicates. If `arr=[1,1,2]`, both 1s should be counted. # A plain set tells us `x+1` exists but not how many times `x` appears. # So I'll use a Counter instead – it gives me $O(1)$ existence checks

AND stores the frequency of each value.

#

Walk over the unique keys in the counter. If `key+1` is also in the counter, # add `counter[key]` to the answer – that accounts for all duplicates of `key` at once.

```
#  
# Pseudocode:  
# freq = Counter(arr)  
# for x in freq:  
# if x+1 in freq: ans += freq[x]  
# return ans  
#  
# Time:  $O(n)$  to build the Counter,  $O(k)$  to  
iterate unique keys where  $k \leq n$ .  
# Overall  $O(n)$ . Space:  $O(k)$  for the  
Counter. Does that make sense?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

```
class _1426_CountingElements:  
    def countElements(self, arr:  
List[int]) -> int:  
        freq = Counter(arr)  
        count = 0  
        for x in freq:  
            if x + 1 in freq:  
                count += freq[x]  
        return count
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# arr=[1,2,3]
# freq={1:1, 2:1, 3:1}
# x=1: 2 in freq → count+=1. count=1.
# x=2: 3 in freq → count+=1. count=2.
# x=3: 4 not in freq.
# return 2 ✓
#
# arr=[1,1,3,3,5,5,7,7]
# freq={1:2, 3:2, 5:2, 7:2}
# x=1: 2 not in freq.
# x=3: 4 not in freq.
# x=5: 6 not in freq.
# x=7: 8 not in freq.
# return 0 ✓
#
# Duplicate counting – arr=[1,1,2]
# freq={1:2, 2:1}
# x=1: 2 in freq → count+=freq[1]=2.
count=2.
# x=2: 3 not in freq.
# return 2 ✓ (both 1s counted, as the
problem requires)
#
```

"No bugs. Duplicate handling confirmed."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – one pass to build the Counter, one pass over unique keys.

Space $O(k)$ where k is the number of distinct values, at most $O(n)$.

#

Alternative: use a plain set instead of a Counter, then iterate the original arr.

present = set(arr)

return sum(1 for x in arr if x+1 in present)

Same $O(n)$ time and $O(n)$ space – slightly simpler but both are valid.

The Counter approach avoids re-iterating arr which is a minor readability win.

#

Trade-off: if the value range is small and fixed (like 0–1000 here),

a boolean array of size 1001 works as a lookup in $O(1)$ with better cache

performance than a hash map, and $O(1001)$ ~ $O(1)$ space in practice.

#

Given more time I'd ask: what if we needed to return the actual elements that qualify rather than a count? We'd collect them into a list instead."

Arrays & Hashing

□ #1534 Count Good Triplets

EAS

[Open on LeetCode](#)

Array · Enumeration

Lists: CodeSignal

Problem

Given an array of integers `arr`, and three integers `a`, `b` and `c`. You need to find the number of good triplets.

A triplet $(arr[i], arr[j], arr[k])$ is good if the following conditions are true:

- $0 \leq i < j < k < arr.length$
- $|arr[i] - arr[j]| \leq a$
- $|arr[j] - arr[k]| \leq b$
- $|arr[i] - arr[k]| \leq c$

Where $|x|$ denotes the absolute value of x .

Return the number of good triplets.

Example 1:

Input: `arr = [3,0,1,1,9,7]`, `a = 7`, `b = 2`, `c = 3`

Output: 4

Explanation: There are 4 good triplets: `[(3,0,1), (3,0,1), (3,1,1), (0,1,1)]`.

Example 2:

Input: `arr = [1,1,2,2,3]`, `a = 0`, `b = 0`, `c = 1`

Output: 0

Explanation: No triplet satisfies all conditions.

Constraints:

- `3 <= arr.length <= 100`
- `0 <= arr[i] <= 1000`
- `0 <= a, b, c <= 1000`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to count all index triplets (i, j, k) where $i < j < k$,

and all three pairwise conditions hold simultaneously:

the first pair must differ by at most a,

the middle-to-last pair by at most b,

and the first-to-last pair by at most c.

We're counting triplets, not returning them. Right?"

#

"A few quick questions:

Can values in arr be 0? Yes, constraints allow it.

Can a, b, c be 0? Yes – that means exact equality required for that pair.

Array length is at most 100 – good to know for complexity decisions."

#

"Let me walk the example:

arr=[3,0,1,1,9,7], a=7, b=2, c=3.

(3,0,1) at indices (0,1,2): $|3-0|=3 \leq 7$,
 $|0-1|=1 \leq 2$, $|3-1|=2 \leq 3$ ✓

```
# (3,0,1) at indices (0,1,3): |3-0|=3<=7,
|0-1|=1<=2, |3-1|=2<=3 ✓
# (3,1,1) at indices (0,2,3): |3-1|=2<=7,
|1-1|=0<=2, |3-1|=2<=3 ✓
# (0,1,1) at indices (1,2,3): |0-1|=1<=7,
|1-1|=0<=2, |0-1|=1<=3 ✓
# Output: 4. Got it."
```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Three nested loops over indices (i, j, k) with $i < j < k$.

Count the triplet when $\text{arr}[i] + \text{arr}[j] + \text{arr}[k] \leq a + b + c$.

Correct, but $O(n^3)$ is too slow when n is up to 100.

Time: $O(n^3)$. Space: $O(1)$.

Should I go ahead and optimize?"

```
class _1534_CountGoodTriplets_Brute:
    def countGoodTriplets(self, arr:
List[int], a: int, b: int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
```

```
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if (abs(arr[i] -
arr[j]) <= a and
                                abs(arr[j] -
arr[k]) <= b and
                                abs(arr[i] -
arr[k]) <= c):
                                    count += 1
    return count
```

PHASE 3 — OPTIMIZE

"The algorithm stays $O(n^3)$ – there's no known sub-cubic solution for this

specific three-condition problem, and with $n=100$ we don't need one.

But I can make two practical improvements:

#

First: early pruning. The three conditions are checked in sequence.

If $|arr[i] - arr[j]| > a$, there's no point checking any k for this (i,j) pair.

We skip the entire inner k loop, reducing the constant factor significantly.

```
#  
# Second: use itertools.combinations for  
# cleaner, more readable code.  
# It generates all (i,j,k) triples with  
# i<j<k in one line.  
# Note that combinations doesn't support  
# early pruning on the outer pair,  
# so for readability I'll use it here  
# since n is small enough that it doesn't  
# matter.  
#  
# I'll go with the early-pruning nested  
# loop to show I understand the trade-off,  
# then mention the itertools alternative.  
#  
# Time:  $O(n^3)$  worst case, faster in  
# practice due to pruning.  
# Space:  $O(1)$ .  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement with early pruning  
# and meaningful names."  
class _1534_CountGoodTriplets:  
    def countGoodTriplets(self, arr:  
List[int], a: int, b: int, c: int) -> int:  
        count = 0
```

```
n = len(arr)
for i in range(n):
    for j in range(i + 1, n):
        if abs(arr[i] - arr[j]) >
a: # prune: skip all k for this pair
            continue
        for k in range(j + 1, n):
            if (abs(arr[j] -
arr[k]) <= b and
                    abs(arr[i] -
arr[k]) <= c):
                count += 1
    return count
# itertools one-liner alternative
(interview-clean, no pruning):
class _1534_CountGoodTriplets_Itertools:
    def countGoodTriplets(self, arr:
List[int], a: int, b: int, c: int) -> int:
        return sum(
            abs(arr[i] - arr[j]) <= a and
            abs(arr[j] - arr[k]) <= b and
            abs(arr[i] - arr[k]) <= c
            for i, j, k in
itertools.combinations(range(len(arr)),
3)
```

)

PHASE 5 — TEST & DEBUG

"Let me trace the pruning version on example 1."

#

arr=[3,0,1,1,9,7], a=7, b=2, c=3

i=0 (arr[i]=3):

j=1 (arr[j]=0): $|3-0|=3 \leq 7 \rightarrow$ not pruned.# k=2: $|0-1|=1 \leq 2$, $|3-1|=2 \leq 3 \rightarrow$ count=1# k=3: $|0-1|=1 \leq 2$, $|3-1|=2 \leq 3 \rightarrow$ count=2# k=4: $|0-9|=9 > 2 \rightarrow$ skip# k=5: $|0-7|=7 > 2 \rightarrow$ skip# j=2 (arr[j]=1): $|3-1|=2 \leq 7 \rightarrow$ not pruned.# k=3: $|1-1|=0 \leq 2$, $|3-1|=2 \leq 3 \rightarrow$ count=3# k=4: $|1-9|=8 > 2 \rightarrow$ skip# k=5: $|1-7|=6 > 2 \rightarrow$ skip# j=3 (arr[j]=1): $|3-1|=2 \leq 7 \rightarrow$ not pruned.# k=4: $|1-9|=8 > 2 \rightarrow$ skip# k=5: $|1-7|=6 > 2 \rightarrow$ skip# j=4 (arr[j]=9): $|3-9|=6 \leq 7 \rightarrow$ not pruned.# k=5: $|9-7|=2 \leq 2$, $|3-7|=4 > 3 \rightarrow$ skip

j=5: no k available.

i=1 (arr[i]=0):

j=2 (arr[j]=1): $|0-1|=1 \leq 7 \rightarrow$ not pruned.# k=3: $|1-1|=0 \leq 2$, $|0-1|=1 \leq 3 \rightarrow$ count=4


```
# k=4,5: fail b condition → skip
# ... remaining pairs yield 0 more.
# return 4 ✓
#
# arr=[1,1,2,2,3], a=0, b=0, c=1
# a=0 means |arr[i]-arr[j]| must be 0 →
# only equal pairs pass.
# j=1 (1==1): pass a. k=2: |1-2|=1>0 →
# fail b. k=3: same. k=4: same.
# j=3 (1==?): arr[1]=1, arr[3]=2 →
# |1-2|=1>0 → pruned.
# No triplet survives. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^3)$ worst case when all pairs pass the first condition –

but early pruning on condition 1 cuts the k loop entirely for bad (i,j) pairs, making it much faster in practice.

Space $O(1)$ – just a counter.

#

With $n=100$ the worst case is 161,700 triplets – trivially fast.

If n grew to 10^4 , $O(n^3)$ would be 10^{12} – completely infeasible.

```
# At that scale I'd look at sorting +  
binary search to narrow candidates per  
(i,j),  
# or segment trees for range queries – but  
that's overkill here.  
#  
# The itertools one-liner is the cleanest  
version for an interview  
# when you want to show Python fluency  
without sacrificing readability.  
# Trade-off: it can't prune early, so  
stick with the nested loop if the  
# interviewer asks about optimization.  
#  
# Given more time I'd ask: what if the  
array is sorted?  
# Sorted order lets us binary search for  
valid k values after fixing i and j,  
# reducing the inner loop from  $O(n)$  to  
 $O(\log n) \rightarrow$  overall  $O(n^2 \log n)$ ."
```

String

Round 1 · High · Easy · 3 questions

String

□ #383 Ransom Note

EAS

[Open on LeetCode](#)

Hash Table · String · Counting

Lists: Grind 169

Problem

Given two strings `ransomNote` and `magazine`, return `true` if `ransomNote` can be constructed by using the letters from `magazine` and `false` otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

Input: `ransomNote = "a"`, `magazine = "b"`
Output: `false`

Example 2:

Input: `ransomNote = "aa"`, `magazine = "ab"`
Output: `false`

Example 3:

Input: ransomNote = "aa", magazine = "aab"

Output: true

Constraints:

- $1 \leq \text{ransomNote.length}, \text{magazine.length} \leq 105$
- ransomNote and magazine consist of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to check if every character in ransomNote can be supplied

by magazine – one letter from magazine per letter in ransomNote.

So if ransomNote needs two 'a's, magazine must have at least two 'a's. Right?"

#

"A few quick questions:

Are both strings guaranteed to be non-empty? Yes, length ≥ 1 .

```
# Only lowercase English letters – so
exactly 26 possible characters. Good.
# We're not modifying the strings, just
checking feasibility?"
#
# "Let me walk the examples:
# ransomNote='a', magazine='b': 'a' not in
magazine → false ✓
# ransomNote='aa', magazine='ab': need
two 'a's, magazine has one → false ✓
# ransomNote='aa', magazine='aab': need
two 'a's, magazine has two → true ✓ Got
it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: convert magazine
to a list so I can 'use up' letters.
# For each character in ransomNote, scan
the list to find it and remove it.
# If I can't find it, return false.
#
# Time:  $O(n * m)$  – for each of  $n$  chars in
ransomNote, scan up to  $m$  chars in
magazine.
# Space:  $O(m)$  for the mutable copy of
magazine.
```

```
# Should I optimize?"  
class _383_RansomNote_Brute:  
    def canConstruct(self, ransomNote:  
str, magazine: str) -> bool:  
        available = list(magazine)  
        for char in ransomNote:  
            if char not in available:  
                return False  
            available.remove(char)  
        return True
```

PHASE 3 — OPTIMIZE

"The bottleneck is scanning magazine for each character — that's the $n*m$.

The question we keep asking is: how many of this letter does magazine still have?

A frequency counter answers that in $O(1)$.

#

I'll count every character in magazine upfront with a Counter.

Then walk ransomNote: for each character, decrement its count.

If any count drops below zero, magazine doesn't have enough of that letter — return false.

```
# If I finish ransomNote without going
negative, return true.
#
# Pseudocode:
# freq = Counter(magazine)
# for char in ransomNote:
#     freq[char] -= 1
#     if freq[char] < 0: return False
# return True
#
# Time:  $O(n + m)$  – one pass each over
magazine and ransomNote.
# Space:  $O(1)$  – the Counter holds at most
26 keys (lowercase letters only).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _383_RansomNote:
    def canConstruct(self, ransomNote:
str, magazine: str) -> bool:
        available = Counter(magazine)
        for char in ransomNote:
            available[char] -= 1
            if available[char] < 0:
```



```
        return False
    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through the three examples."

#

ransomNote='a', magazine='b'

available = {'b': 1}

char='a': available['a']=0-1=-1 < 0 →

return False ✓

(Counter returns 0 for missing keys, so no KeyError)

#

ransomNote='aa', magazine='ab'

available = {'a':1, 'b':1}

char='a': available['a']=0 → ok

char='a': available['a']=-1 < 0 → return False ✓

#

ransomNote='aa', magazine='aab'

available = {'a':2, 'b':1}

char='a': available['a']=1 → ok

char='a': available['a']=0 → ok

return True ✓

#

```
# Edge – ransomNote longer than magazine:  
ransomNote='abc', magazine='ab'  
# available = {'a':1, 'b':1}  
# char='a': 0 → ok. char='b': 0 → ok.  
char='c': -1 < 0 → return False ✓
```

```
#
```

```
# "No bugs. Counter's default of 0 for  
missing keys handles unseen characters  
cleanly."
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time  $O(n + m)$  –  $O(m)$  to build the  
Counter,  $O(n)$  to walk ransomNote.
```

```
# Space  $O(1)$  – the Counter has at most 26  
entries since only lowercase letters  
exist.
```

```
# Even though we write  $O(k)$  where  $k$  is  
distinct chars,  $k$  is bounded by 26 –  
constant.
```

```
#
```

```
# Alternative: Counter subtraction in one  
line.
```

```
# return not (Counter(ransomNote) –  
Counter(magazine))
```

```
# Counter subtraction drops keys with zero  
or negative counts,
```

```
# so if anything remains, ransomNote needs
letters magazine doesn't have.
# Elegant but less readable in an
interview – I'd use the explicit loop.
#
# Another alternative: sort both strings
and use two pointers –  $O(n \log n + m \log m)$ ,
# slower but uses  $O(1)$  space beyond the
sort. Worth mentioning if interviewer
# constraints disallow hash maps.
#
# Given more time I'd ask: what if
characters include uppercase or unicode?
# We'd replace the 26-slot assumption with
a full hash map – space becomes  $O(\text{alphabet size})$ ."
```

String

□ ★ #734 Sentence Similarity ★

EAS

[Open on LeetCode](#)

Array · Hash Table · String

Lists: ★ Premium | Premium 98

Problem

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as `arr = ["I","am",happy","with","leetcode"]`.

Given two sentences `sentence1` and `sentence2` each represented as a string array and given an array of string pairs `similarPairs` where `similarPairs[i] = [xi, yi]` indicates that the two words `xi` and `yi` are similar.

Return true if `sentence1` and `sentence2` are similar, or false if they are not similar.

Two sentences are similar if:

- They have the same length (i.e., the same number of words)
- `sentence1[i]` and `sentence2[i]` are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words a and b are similar, and the words b and c are similar, a and c are not necessarily similar.

Example 1:

```
Input: sentence1 =  
["great","acting","skills"], sentence2  
= ["fine","drama","talent"],  
similarPairs = [ ["great","fine"], ["dram  
a","acting"], ["skills","talent"] ]
```

Output: true

Explanation: The two sentences have the same length and each word *i* of sentence1 is also similar to the corresponding word in sentence2.

Example 2:

```
Input: sentence1 = ["great"], sentence2  
= ["great"], similarPairs = []
```

Output: true

Explanation: A word is similar to itself.

Example 3:

Input: sentence1 = ["great"], sentence2 = ["doubleplus", "good"], similarPairs = [["great", "doubleplus"]]

Output: false

Explanation: As they don't have the same length, we return false.

Constraints:

- $1 \leq \text{sentence1.length}, \text{sentence2.length} \leq 1000$
- $1 \leq \text{sentence1}[i].\text{length}, \text{sentence2}[i].\text{length} \leq 20$
- sentence1[i] and sentence2[i] consist of English letters.
- $0 \leq \text{similarPairs.length} \leq 1000$
- $\text{similarPairs}[i].\text{length} == 2$
- $1 \leq \text{xi.length}, \text{yi.length} \leq 20$
- xi and yi consist of lower-case and upper-case English letters.
- All the pairs (xi, yi) are distinct.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Two sentences are similar if they're the same length and every word at position i in sentence1 is similar to the word at position i in sentence2.

A word is always similar to itself.

Similarity is NOT transitive – if $a \sim b$ and $b \sim c$, that doesn't mean $a \sim c$.

We're given the list of similar pairs explicitly, so we only trust those.

Right?"

#

"A few quick questions:

Can similarPairs be empty? Yes – then only identical words are similar.

Are pairs directional? The problem implies symmetric – if $[a,b]$ is given,

then $a \sim b$ AND $b \sim a$. I'll confirm: yes, similarity is symmetric.

Can the same pair appear twice?

Constraints say all pairs are distinct – good."

#

"Let me walk example 1:

```
# sentence1=['great','acting','skills'],
sentence2=['fine','drama','talent'].
# Same length: 3. pairs include [great,fine],
[drama,acting],[skills,talent].
# great~fine ✓, acting~drama ✓
(symmetric), skills~talent ✓ → true ✓ Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: first check
lengths. Then for each position i,
# check if sentence1[i] == sentence2[i],
or scan all similarPairs
# to find a match in either direction.
#
# Time:  $O(n * p)$  – for each of n word
pairs, scan up to p similarity pairs.
# Space:  $O(1)$  – no preprocessing.
# Should I optimize?"
```

```
class _734_SentenceSimilarity_Brute:
    def areSentencesSimilar(self,
sentence1: List[str], sentence2:
List[str], similarPairs: List[List[str]])
-> bool:
        if len(sentence1) !=
len(sentence2):
```



```
        return False
    for word1, word2 in
zip(sentence1, sentence2):
        if word1 == word2:
            continue
        found = any((a == word1 and b
== word2) or (a == word2 and b == word1)
                    for a, b in
similarPairs)
        if not found:
            return False
    return True
```

PHASE 3 — OPTIMIZE

"The bottleneck is scanning all p pairs for every word position.

We're asking the same question repeatedly: is word2 similar to word1?

A hash map from each word to its set of similar words answers that in $O(1)$.

#

Preprocess similarPairs once: for each [a, b], add b to hmap[a] and a to hmap[b].

That handles symmetry automatically – no need to check both directions at lookup time.

```
#  
# Then for each (word1, word2) pair in the  
# sentences:  
# if they're equal, they're trivially  
# similar – skip.  
# Otherwise check if word2 is in  
# hmap[word1].  
# Using defaultdict(set) means missing  
# keys return an empty set – no KeyError.  
#  
# Pseudocode:  
# if len(s1) != len(s2): return False  
# similar = defaultdict(set)  
# for a, b in pairs: similar[a].add(b);  
# similar[b].add(a)  
# for w1, w2 in zip(s1, s2):  
# if w1 != w2 and w2 not in similar[w1]:  
# return False  
# return True  
#  
# Time:  $O(p + n)$ . Space:  $O(p)$  for the map.  
# Does that make sense?"  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."
```

```
class _734_SentenceSimilarity:
    def areSentencesSimilar(self,
sentence1: List[str], sentence2:
List[str], similarPairs: List[List[str]])
-> bool:
        if len(sentence1) !=
len(sentence2):
            return False
        similar = defaultdict(set)
        for word_a, word_b in
similarPairs:
            similar[word_a].add(word_b)
            similar[word_b].add(word_a) #
symmetric: both directions
        for word1, word2 in
zip(sentence1, sentence2):
            if word1 != word2 and word2
not in similar[word1]:
                return False
        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three
examples."
#
```

```
# Example 1:
s1=['great','acting','skills'],
s2=['fine','drama','talent']
# pairs=[['great','fine'],['drama','acting'],['skills','talent']]
# similar = {great:{fine}, fine:{great},
drama:{acting}, acting:{drama},
skills:{talent}, talent:{skills}}
# (great,fine): great!=fine, fine in
similar[great] ✓
# (acting,drama): acting!=drama, drama in
similar[acting] ✓
# (skills,talent): skills!=talent, talent
in similar[skills] ✓
# return True ✓
#
# Example 2: s1=['great'], s2=['great'],
pairs=[]
# Lengths match. similar={} (empty).
# (great,great): great==great → skip.
# return True ✓
#
# Example 3: s1=['great'],
s2=['doubleplus','good'],
pairs=[['great','doubleplus']]
```

```
# len(s1)=1 != len(s2)=2 → return False
immediately ✓
#
# Non-transitive check:
pairs=[['a','b'],['b','c']], checking a~c
# similar = {a:{b}, b:{a,c}, c:{b}}
# word1='a', word2='c': a!=c, c not in
similar[a]={b} → return False ✓
# Confirms we don't accidentally apply
transitivity.
#
# "No bugs. All cases pass including the
transitivity trap."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(p + n) - O(p)$  to build the
similarity map,  $O(n)$  to check each word
pair.
# Space  $O(p)$  for the map – each pair adds
two entries.
#
# The critical design decision is storing
both directions upfront rather than
# checking both at lookup time. It keeps
the lookup a single  $O(1)$  set membership
test.
```

#

Important constraint to call out:
similarity is NOT transitive.

This rules out Union-Find (which is
designed for transitive equivalence
classes).

If transitivity WERE required – for
example in Sentence Similarity II (#737) –
we'd use Union-Find to group words into
connected components and check
if sentence1[i] and sentence2[i] share
the same root.

#

Given more time I'd ask: what if the
same word pair could appear in different
cases,

like ['Great', 'fine'] vs
['great', 'fine']? We'd normalize to
lowercase before building
the map and before lookups."

String

□ ★ #1165 Single Row Keyboard ★

EAS

[Open on LeetCode](#)

Hash Table · String

Lists: ★ Premium | Premium 98

Problem

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is $|i - j|$. You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1:

Input: keyboard =
"abcdefghijklmnopqrstuvwxyz", word =
"cba"

Output: 4

Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'.

Total time = 2 + 1 + 1 = 4.

Example 2:

Input: keyboard =
"pqrstuvwxyzabcdefghijklmnopqrstuvwxyz", word =
"leetcode"

Output: 73

Constraints:

- keyboard.length == 26
- keyboard contains each English lowercase letter exactly once in some order.
- $1 \leq \text{word.length} \leq 104$
- word[i] is an English lowercase letter.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have a custom keyboard layout – 26 characters in some arbitrary order.

Our finger starts at index 0. To type each character in word,

we move from the current index to that character's index on the keyboard.

The cost is the absolute distance moved. We sum all moves and return the total. Right?"

#

"A few quick questions:

The keyboard always has exactly 26 letters, each exactly once – so it's a permutation.

Word can only contain lowercase letters, and every letter in word is guaranteed

to appear on the keyboard – so no missing-key edge case to handle.

Word length can be up to 10^4 – so $O(n)$ should be fine."

#

"Let me walk example 1:

```
# keyboard='abcdefghijklmnopqrstuvwxyz',  
word='cba'.  
# Start at 0. 'c' is at index 2:  $|0-2|=2$ .  
'b' is at 1:  $|2-1|=1$ . 'a' is at 0:  
 $|1-0|=1$ .  
# Total = 2+1+1 = 4. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: for each  
character in word, scan the keyboard  
string
```

```
# from left to right to find where it  
lives, then compute the distance.
```

```
#
```

```
# Time:  $O(n * 26)$  — for each of  $n$   
characters we scan up to 26 positions.
```

```
# Since keyboard is fixed at 26, this is  
effectively  $O(n)$ .
```

```
# Space:  $O(1)$ .
```

```
# Should I optimize the lookup?"
```

```
class _1165_SingleRowKeyboard_Brute:  
    def calculateTime(self, keyboard:  
str, word: str) -> int:  
        total = 0  
        position = 0  
        for char in word:
```

```
        next_pos =  
keyboard.index(char)  
        total += abs(position -  
next_pos)  
        position = next_pos  
    return total
```

PHASE 3 — OPTIMIZE

"The brute force scans 26 characters for every letter typed – that's the inner $O(26)$.

We're asking the same question repeatedly: where does this letter live on the keyboard?

A hash map answers that in $O(1)$ after a one-time $O(26)$ build.

#

Preprocess keyboard once: build a map from character $\rightarrow$ index.

Then walk word: for each character, look up its index in $O(1)$,

add the absolute distance from the current position, update position, move on.

#

Pseudocode:

```
# pos_map = {char: i for i, char in
enumerate(keyboard)}
# total, position = 0, 0
# for char in word:
# total += abs(position - pos_map[char])
# position = pos_map[char]
# return total
#
# Time:  $O(26 + n) = O(n)$ . Space:  $O(26) = O(1)$  – the map is fixed size.
# In practice the constant factor
improvement is small since 26 is tiny,
# but the hash map is the correct
interview answer – cleaner and scales to
# larger alphabets."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _1165_SingleRowKeyboard:
    def calculateTime(self, keyboard:
str, word: str) -> int:
        key_index = {char: i for i, char
in enumerate(keyboard)}
        total = 0
        position = 0
```

```
        for char in word:
            total += abs(position -
key_index[char])
            position = key_index[char]
        return total
```

PHASE 5 — TEST & DEBUG

"Let me trace through both examples."

#

```
# keyboard='abcdefghijklmnopqrstuvwxyz',
word='cba'
```

```
# key_index = {'a':0, 'b':1, 'c':2, ...}
```

```
# char='c': |0-2|=2, position=2. total=2
```

```
# char='b': |2-1|=1, position=1. total=3
```

```
# char='a': |1-0|=1, position=0. total=4
```

```
# return 4 ✓
```

#

```
# keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz',
word='leetcode'
```

```
# key_index = {'p':0, 'q':1, ..., 'a':10, 'b':11, ..., 'l':21, 'e':14, 't':19, 'c':12, 'o':24, 'd':13}
```

```
# char='l': |0-21|=21, position=21.
total=21
```

```
# char='e': |21-14|=7, position=14.
total=28
```

```
# char='e': |14-14|=0, position=14.
total=28
# char='t': |14-19|=5, position=19.
total=33
# char='c': |19-12|=7, position=12.
total=40
# char='o': |12-24|=12, position=24.
total=52
# char='d': |24-13|=11, position=13.
total=63
# char='e': |13-14|=1, position=14.
total=64 ← wait, let me recheck key_index
# (keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz':
p=0,q=1,r=2,s=3,t=4,u=5,v=6,w=7,x=8,y=9,z
=10,
# a=11,b=12,c=13,d=14,e=15,f=16,g=17,h=18
,i=19,j=20,k=21,l=22,m=23,n=24,o=25)
# char='l': |0-22|=22. char='e':
|22-15|=7. char='e': 0. char='t':
|15-4|=11.
# char='c': |4-13|=9. char='o':
|13-25|=12. char='d': |25-14|=11.
char='e': |14-15|=1.
# total = 22+7+0+11+9+12+11+1 = 73 ✓
#
```

"No bugs. Manual trace confirms both examples."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n) - O(26)$ to build the map, $O(n)$ to walk word. Space $O(1) - 26$ -entry map.

#

The brute force using `keyboard.index()` is also $O(n)$ in practice since `keyboard` is always 26 characters – but the hash map is the canonical interview answer because it makes the $O(1)$ lookup intent explicit and generalizes to any alphabet size.

#

Trade-off: if `keyboard` could be very long (say a 10^6 -char layout), the hash map

preprocessing cost would matter more – but we'd still prefer it over `.index()`.

#

One-liner alternative:

`pos = {c: i for i, c in enumerate(keyboard)}`

`return sum(abs(pos[b] - pos[a]) for a, b in zip('_', word, word))`

```
# where prepending '_' shifts word so each
pair is (prev_char, curr_char),
# and '_' maps to nothing – so you'd
actually prepend a dummy start:
# pairs = zip([None] + list(word), word) –
but this gets messy.
# The explicit loop is cleaner in an
interview.
#
# Given more time I'd ask: what if we can
use two fingers?
# That becomes a DP problem – optimal
two-finger typing is a classic hard
variant."
```


Two Pointers

Round 1 · High · Easy · 5 questions

Two Pointers

□ #88 Merge Sorted Array

EAS

[Open on LeetCode](#)

Array · Two Pointers · Sorting

Lists: Denny Zhang

Problem

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: nums1 = [1,2,3,0,0,0], m = 3,
nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging
are [1,2,3] and [2,5,6].
The result of the merge is
[1,2,2,3,5,6] with the underlined
elements coming from nums1.

Example 2:

Input: nums1 = [1], m = 1, nums2 = [],
n = 0
Output: [1]
Explanation: The arrays we are merging
are [1] and [].
The result of the merge is [1].

Example 3:

Input: `nums1 = [0], m = 0, nums2 = [1], n = 1`

Output: `[1]`

Explanation: The arrays we are merging are `[]` and `[1]`.

The result of the merge is `[1]`.

Note that because `m = 0`, there are no elements in `nums1`. The `0` is only there to ensure the merge result can fit in `nums1`.

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# nums1 has m valid elements followed by n
zeros as padding.
# nums2 has n elements. Both are sorted in
non-decreasing order.
# We need to merge them in-place into
nums1 – sorted, no return value. Right?"
#
# "A few quick questions:
# Can m or n be 0? Yes – if n=0 nums1 is
already done; if m=0 we just copy nums2.
# Can there be negative numbers? Yes,
constraints go down to  $-10^9$ .
# Can there be duplicates? Yes –
non-decreasing order means equal values
are allowed.
# We cannot use extra space for the merge?
The follow-up asks for  $O(m+n)$  time,
# implying in-place. I'll target  $O(1)$ 
extra space."
#
# "Let me walk example 1:
# nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6]
n=3.
# Merge [1,2,3] and [2,5,6] into nums1 →
[1,2,2,3,5,6]. Got it."
```

PHASE 2 — BRUTE FORCE

"The simplest approach: copy nums2 into the tail of nums1 (the zero slots),
then sort the entire nums1 array.

#

nums1[m:] = nums2, then nums1.sort()

#

Time: $O((m+n) \log(m+n))$ for the sort.

Space: $O(1)$ extra – sort is in-place.

This is clean but doesn't use the fact that both inputs are already sorted.

Should I optimize to use that sorted property?"

```
class _88_MergeSortedArray_Brute:
```

```
    def merge(self, nums1: List[int], m:
int, nums2: List[int], n: int) -> None:
        nums1[m:] = nums2
        nums1.sort()
```

PHASE 3 — OPTIMIZE

"The brute force throws away the sorted order and pays $O(\log(m+n))$ to re-sort.

The classic merge from the front copies to a temp array – $O(m+n)$ space.

But there's a smarter way that stays $O(1)$ space.

```
#  
# Key insight: if we merge from the BACK,  
we never overwrite elements we still need.  
# The tail of nums1 is empty padding – we  
can fill it safely from right to left.  
#  
# Three pointers:  
# i → last valid element in nums1 (index  
m-1)  
# j → last element in nums2 (index n-1)  
# k → last slot in nums1 (index m+n-1)  
#  
# At each step, pick the larger of  
nums1[i] and nums2[j], place it at k, move  
that pointer left.  
# When j drops below 0, nums2 is exhausted  
– nums1's remaining elements are already  
in place.  
# When i drops below 0 first, copy  
remaining nums2 into nums1.  
#  
# Pseudocode:  
# i, j, k = m-1, n-1, m+n-1  
# while k >= 0:
```

```
# if j < 0 or (i >= 0 and nums1[i] >
nums2[j]):
# nums1[k] = nums1[i]; i -= 1
# else:
# nums1[k] = nums2[j]; j -= 1
# k -= 1
#
# Time: O(m+n) – each element is placed
exactly once.
# Space: O(1) – purely in-place.
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _88_MergeSortedArray:
    def merge(self, nums1: List[int], m:
int, nums2: List[int], n: int) -> None:
        last1 = m - 1 # tail of valid
nums1 elements
        last2 = n - 1 # tail of nums2
        fill = m + n - 1 # next slot to
fill in nums1 (right to left)
        while fill >= 0:
            if last2 < 0 or (last1 >= 0
and nums1[last1] > nums2[last2]):
```



```
                nums1[fill] =  
nums1[last1]  
                last1 -= 1  
            else:  
                nums1[fill] =  
nums2[last2]  
                last2 -= 1  
            fill += 1
```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

#

nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6]
n=3

last1=2, last2=2, fill=5

fill=5: nums1[2]=3 vs nums2[2]=6 → 6
wins → nums1[5]=6, last2=1, fill=4

fill=4: nums1[2]=3 vs nums2[1]=5 → 5
wins → nums1[4]=5, last2=0, fill=3

fill=3: nums1[2]=3 vs nums2[0]=2 → 3
wins → nums1[3]=3, last1=1, fill=2

fill=2: nums1[1]=2 vs nums2[0]=2 →
equal, take nums2 → nums1[2]=2, last2=-1,
fill=1

```
# fill=1: last2<0 → take nums1[1]=2 →  
nums1[1]=2, last1=0, fill=0  
# fill=0: last2<0 → take nums1[0]=1 →  
nums1[0]=1, last1=-1, fill=-1  
# nums1=[1,2,2,3,5,6] ✓  
#  
# nums1=[1] m=1, nums2=[] n=0  
# last1=0, last2=-1, fill=0  
# fill=0: last2<0 → nums1[0]=nums1[0]=1.  
fill=-1. done.  
# nums1=[1] ✓  
#  
# nums1=[0] m=0, nums2=[1] n=1  
# last1=-1, last2=0, fill=0  
# fill=0: last1<0 → nums1[0]=nums2[0]=1,  
last2=-1, fill=-1. done.  
# nums1=[1] ✓  
#  
# "No bugs. All three cases pass including  
the m=0 and n=0 edges."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(m+n)$  – each of the  $m+n$  positions  
is filled exactly once.  
# Space  $O(1)$  – purely in-place, no  
auxiliary arrays.
```

#

The core insight worth stating explicitly: merging front-to-back requires a temp

buffer because we'd overwrite nums1's valid elements. Merging back-to-front avoids

this because the padding zeros are exactly the space we need, and large elements

fill the tail first without touching anything we still care about.

#

Edge cases worth calling out:

– $n=0$: loop runs but $last2 < 0$ always, so nums1's valid elements stay in place.

– $m=0$: $last1 < 0$ always, so nums2 is copied straight into nums1.

– All nums2 elements are larger than all nums1 elements: nums2 fills the back,

then nums1 elements slot into the front – handled cleanly by the pointer logic.

#

Given more time I'd ask: what if nums1 has exactly m slots (no padding)?

Then in-place is impossible – we'd need to return a new merged array, # which is the standard merge step in merge sort."

Two Pointers

□ #125 Valid Palindrome

EAS

[Open on LeetCode](#)

Two Pointers · String

Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

Input: *s* = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: `s = "race a car"`

Output: `false`

Explanation: "raceacar" is not a palindrome.

Example 3:

Input: `s = " "`

Output: `true`

Explanation: `s` is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- `s` consists only of printable ASCII characters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We strip everything except letters and digits, lowercase it all,

```
# then check if the result reads the same
forwards and backwards. Right?"
#
# "A few quick questions:
# Alphanumeric means letters AND digits –
not just letters?
# So '0P' after cleaning is '0p', which is
not a palindrome – confirmed?
# What about a string of only spaces or
punctuation?
# Example 3 shows a single space returns
true – after cleaning it's an empty
string,
# and an empty string is considered a
palindrome. Good.
# Can the string be length 1? Yes – a
single alphanumeric character is a
palindrome."
#
# "Let me walk example 1:
# 'A man, a plan, a canal: Panama' →
'amanaplanacanalpanama' → palindrome ✓
# Example 2: 'race a car' → 'raceacar' →
not a palindrome ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the simplest correct
approach.
# Copy every alphanumeric character into a
cleaned list (lowercased),
# then compare the list to its reverse.
# Two passes over the string plus  $O(n)$ 
extra storage.
# Time:  $O(n)$ . Space:  $O(n)$  for the cleaned
copy.
# Should I go ahead and optimize?"

class _125_ValidPalindrome_Brute:
    def isPalindrome(self, s: str) ->
bool:
        cleaned = [c.lower() for c in s if
c.isalnum()]
        return cleaned == cleaned[::-1]

# PHASE 3 — OPTIMIZE
# "The brute force is already  $O(n)$  time,
but uses  $O(n)$  space for the cleaned copy.
# We can do this in  $O(1)$  extra space using
two pointers directly on the original
string.
#
# Place left at 0 and right at len(s)-1.
```



```
# Advance left past any non-alphanumeric
characters.
# Retreat right past any non-alphanumeric
characters.
# Compare s[left].lower() and
s[right].lower().
# If they differ, return False. Otherwise
move both pointers inward.
# If left meets or passes right, the
string is a palindrome.
#
# The key benefit: we never build a second
string – we skip junk characters on the
fly.
#
# Pseudocode:
# left, right = 0, len(s)-1
# while left < right:
# while left < right and not
s[left].isalnum(): left += 1
# while left < right and not
s[right].isalnum(): right -= 1
# if s[left].lower() != s[right].lower():
return False
# left += 1; right -= 1
```

```
# return True
#
# Time: O(n). Space: O(1). Does that make
sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _125_ValidPalindrome:
    def isPalindrome(self, s: str) ->
bool:
        left = 0
        right = len(s) - 1
        while left < right:
            while left < right and not
s[left].isalnum(): # skip
non-alphanumeric
                left += 1
            while left < right and not
s[right].isalnum(): # skip
non-alphanumeric
                right -= 1
            if s[left].lower() !=
s[right].lower():
                return False
```

```
        left += 1
        right -= 1
    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

#

s="A man, a plan, a canal: Panama"

left=0 ('A'), right=29 ('a') – both alnum.

'A'.lower()='a' == 'a' ✓. left=1, right=28.

left=1 (' ') → skip → left=2 ('m'). right=28 ('m') – alnum.

'm'=='m' ✓. Continue inward... (all pairs match) → return True ✓

#

s="race a car"

left=0 ('r'), right=9 ('r'). 'r'=='r' ✓. left=1, right=8.

left=1 ('a'), right=8 ('a'). 'a'=='a' ✓. left=2, right=7.

left=2 ('c'), right=7 ('c'). 'c'=='c' ✓. left=3, right=6.

```
# left=3 ('e'), right=6 ('a'). 'e'!='a' →  
return False ✓  
# (right=6 is 'a' in 'a car'; skip right=5  
' ' → right=4 'a')  
# Wait – let me recount: "race a car"  
# indices: r=0,a=1,c=2,e=3,' '=4,a=5,'  
'=6,c=7,a=8,r=9  
# left=3 ('e'), right=9→skip none→'r'.  
'e'!='r' → return False ✓  
#  
# s=" "  
# left=0 (' ') → not alnum → left=1. Now  
left(1) >= right(0) → exit loop.  
# return True ✓  
#  
# "No bugs. All three examples confirmed."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n) – each character is visited  
at most once by left or right pointer.  
# Space O(1) – no auxiliary storage beyond  
two integer pointers.  
#  
# The brute force one-liner is perfectly  
valid for an interview too – it's readable
```

```
# and still  $O(n)$  time. The two-pointer
version wins purely on space:  $O(1)$  vs
 $O(n)$ .
# If the interviewer asks about the
follow-up, lead with the space trade-off.
#
# isalnum() handles both letters and
digits in one call – worth naming
explicitly
# so the interviewer knows you're not just
checking isalpha().
#
# Subtle trap: the inner while loops must
guard left < right to avoid crossing
# pointers – if you forget, you can read
out-of-bounds when the string is all
# non-alphanumeric (like the single-space
example).
#
# Given more time I'd ask: what if the
string could contain unicode characters
# like accented letters (é, ñ)? isalnum()
in Python handles unicode correctly
# by default, so our solution already
works – worth calling out."
```

Two Pointers

□ #283 Move Zeroes

EAS[Open on LeetCode](#)

Array · Two Pointers

Lists: Grind 169

Problem

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you minimize the total number of operations done?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to move all 0s to the end of the array while keeping the relative

order of all non-zero elements. This must be done in-place – no copy. Right?"

#

"A few quick questions:

Do we need to return anything? No – modify in-place, return nothing.

Can the array be all zeros? Yes – output should still be all zeros.

Can it be all non-zeros? Yes – nothing moves.

Can there be negative numbers? Yes, constraints go down to -2^{31} ."

#

"Let me walk example 1:

`nums=[0,1,0,3,12] → [1,3,12,0,0]`.

```
# Non-zeros 1,3,12 keep their relative  
order and shift to the front.
```

```
# Zeros fill the tail. Got it."
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "The simplest approach: collect all  
non-zero elements into a new list,
```

```
# then write them back into nums, then  
fill the rest with zeros.
```

```
#
```

```
# non_zeros = [x for x in nums if x != 0]
```

```
# nums[:len(non_zeros)] = non_zeros
```

```
# nums[len(non_zeros):] = [0] * (len(nums)  
- len(non_zeros))
```

```
#
```

```
# Time:  $O(n)$ . Space:  $O(n)$  – we make a copy  
of the non-zeros.
```

```
# Should I optimize for space?"
```

```
class _283_MoveZeroesBrute:
```

```
    def moveZeroes(self, nums: List[int])
```

```
→ None:
```

```
        non_zeros = [x for x in nums if x  
!= 0]
```

```
        nums[:len(non_zeros)] = non_zeros
```

```
        nums[len(non_zeros):] = [0] *  
(len(nums) - len(non_zeros))
```


PHASE 3 — OPTIMIZE

"The brute force uses $O(n)$ extra space for the non-zero copy.

We can do this in $O(1)$ space with two pointers.

#

Use a slow pointer 'place' that tracks where the next non-zero should go.

Scan with a fast pointer 'scan'.

Whenever `nums[scan]` is non-zero,

swap it with `nums[place]` and advance place.

#

This compacts all non-zeros to the front in their original order,

and the swaps naturally push zeros toward the tail.

#

Pseudocode:

`place = 0`

`for scan in range(len(nums)):`

`if nums[scan] != 0:`

`nums[scan], nums[place] = nums[place],`
`nums[scan]`

`place += 1`

```
#  
# Time: O(n). Space: O(1). Does that make  
sense before I code it?"  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
approach with meaningful names."  
class _283_MoveZeroes:  
    def moveZeroes(self, nums: List[int])  
-> None:  
        place = 0  
        for scan in range(len(nums)):  
            if nums[scan] != 0:  
                nums[scan], nums[place] =  
nums[place], nums[scan]  
                place += 1  
  
# PHASE 5 — TEST & DEBUG  
# "Let me trace through both examples."  
#  
# nums=[0,1,0,3,12]  
# scan=0: nums[0]=0 → skip. place=0.  
# scan=1: nums[1]=1 ≠ 0 → swap(1,0):  
[1,0,0,3,12]. place=1.  
# scan=2: nums[2]=0 → skip. place=1.
```

```
# scan=3: nums[3]=3 ≠ 0 → swap(3,1):  
[1,3,0,0,12]. place=2.  
# scan=4: nums[4]=12 ≠ 0 → swap(4,2):  
[1,3,12,0,0]. place=3.  
# Result: [1,3,12,0,0] ✓  
#  
# nums=[0]  
# scan=0: nums[0]=0 → skip.  
# Result: [0] ✓  
#  
# nums=[1,2,3] (no zeros)  
# Every element is non-zero → each swaps  
with itself. place advances each step.  
# Result: [1,2,3] ✓ (self-swaps are fine –  
no change)  
#  
# "No bugs. Self-swaps when place==scan  
are harmless."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n) – single pass. Space O(1) –  
two integer pointers.  
#  
# Follow-up asks to minimize operations.  
The swap approach does a swap
```

```
# for every non-zero element, even if it's
already in the right place.
# A minor optimization: only swap when
place != scan to skip self-swaps.
# But in an interview the current version
is clean enough.
#
# Alternative: instead of swapping, just
overwrite:
# write non-zeros to the front with
assignment (not swap), then zero-fill the
tail.
# Same complexity but fewer writes –
useful if writes are expensive.
#
# place = 0
# for x in nums:
# if x != 0: nums[place] = x; place += 1
# nums[place:] = [0] * (len(nums) - place)
#
# Given more time I'd ask: what if we also
need to preserve the positions
# of the zeros (not just push them to the
end)? That changes the problem
```

to a stable partition – same two-pointer approach still works."

Two Pointers

□ ★ #408 Valid Word Abbreviation ★

EAS

[Open on LeetCode](#)

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
- "s010n" (has leading zeros)

- "s0ubstitution" (replaces an empty substring)

Given a string `word` and an abbreviation `abbr`, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

Input: `word = "internationalization",
abbr = "i12iz4n"`

Output: `true`

Explanation: The word "internationalization" can be abbreviated as "i12iz4n" ("i nternational iz atio n").

Example 2:

Input: `word = "apple", abbr = "a2e"`

Output: `false`

Explanation: The word "apple" cannot be abbreviated as "a2e".

Constraints:

- $1 \leq \text{word.length} \leq 20$

- word consists of only lowercase English letters.
- $1 \leq \text{abbr.length} \leq 10$
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

An abbreviation replaces one or more non-adjacent non-empty substrings with their lengths.

Given a word and an abbreviation, return true if the abbreviation is valid for the word.

Numbers in the abbreviation represent how many consecutive characters to skip. Right?"

#

"A few quick questions:

Can there be leading zeros in the number part? No – that's explicitly invalid.
So '01' is not valid even though it could theoretically mean skip 1.
Can the abbreviation be the full word with no numbers? Yes – that's valid.
Can it be just a single number like '12'? Yes – if the word has exactly 12 chars."

#

"Let me walk example 1:

word='internationalization',
abbr='i12iz4n'.

'i' matches 'i'. '12' skips 12 chars.

'i' matches 'i'. 'z' matches 'z'.

'4' skips 4 chars. 'n' matches 'n'. We reach end of both → true ✓

Example 2: word='apple', abbr='a2e'. 'a' matches. '2' skips 'pp'. 'e'→word[3]='l' ≠ 'e' → false ✓"

PHASE 2 — BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic.

Expand the abbreviation

letter-by-letter against word using two

pointers.

When we hit a digit in abbr, consume that many characters from word.

Any mismatch ? false; both exhausted together ? true.

Time: $O(n)$. Space: $O(1)$.

This is already optimal for the constraints – complexity stated clearly."

PHASE 3 — OPTIMIZE

"Use pointer i into word and j into abbr.

When abbr[j] is a digit: check for leading zero (abbr[j]=='0' and num==0 → invalid).

Accumulate the number: num = num*10 + digit.

When abbr[j] is a letter: first advance i by num (the accumulated skip), reset num to 0.

Then compare word[i] with abbr[j] – if they differ, return False. Advance both i and j.

After the loop: we're valid only if i + num == len(word) AND j == len(abbr).

```
# That handles a trailing number at the
end of abbr.
#
# Pseudocode:
# i, j, num = 0, 0, 0
# while i < len(word) and j < len(abbr):
# if abbr[j].isdigit():
# if abbr[j]=='0' and num==0: return False
# leading zero
# num = num*10 + int(abbr[j])
# else:
# i += num; num = 0
# if i >= len(word) or word[i] != abbr[j]:
# return False
# i += 1
# j += 1
# return i + num == len(word) and j ==
len(abbr)
#
# Time: O(n + m) where n=len(word),
m=len(abbr). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement with meaningful names."

class _408_ValidWordAbbreviation:

```
def validWordAbbreviation(self, word:
str, abbr: str) -> bool:
    word_ptr = 0
    abbr_ptr = 0
    skip = 0
    while word_ptr < len(word) and
abbr_ptr < len(abbr):
        if abbr[abbr_ptr].isdigit():
            if abbr_ptr == '0'
and skip == 0: # leading zero
                return False
            skip = skip * 10 +
int(abbr[abbr_ptr])
        else:
            word_ptr += skip
            skip = 0
            if word_ptr >= len(word)
or word[word_ptr] != abbr[abbr_ptr]:
                return False
            word_ptr += 1
            abbr_ptr += 1
    return word_ptr + skip ==
len(word) and abbr_ptr == len(abbr)

# PHASE 5 — TEST & DEBUG
# "Let me trace both examples."
```

```
#
# word='internationalization' (len=20),
abbr='i12iz4n'
# j=0: 'i' → skip=0, advance word_ptr 0,
word[0]='i'=='i' ✓. word_ptr=1.
# j=1: '1' → skip=1.
# j=2: '2' → skip=12.
# j=3: 'i' → word_ptr+=12=13.
word[13]='i'=='i' ✓. word_ptr=14.
# j=4: 'z' → skip=0, word[14]='z'=='z' ✓.
word_ptr=15.
# j=5: '4' → skip=4.
# j=6: 'n' → word_ptr+=4=19.
word[19]='n'=='n' ✓. word_ptr=20.
# Loop ends (word_ptr=20=len(word)).
word_ptr+skip=20+0=20==20, abbr_ptr=7==7.
True ✓
#
# word='apple' (len=5), abbr='a2e'
# j=0: 'a' → word[0]='a'=='a' ✓.
word_ptr=1.
# j=1: '2' → skip=2.
# j=2: 'e' → word_ptr+=2=3. word[3]='l' ≠
'e' → return False ✓
#
```

```
# Leading zero: abbr='a01e' → j=1: '0' and  
skip==0 → return False ✓  
#  
# "No bugs. Leading zero check and  
trailing number handled correctly."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n + m)$  – each character in word  
and abbr is visited at most once.  
# Space  $O(1)$  – three integer pointers.  
#  
# The two trickiest cases to get right:  
# 1. Leading zeros – must check  
abbr[j]=='0' AND accumulated num==0 to  
distinguish  
# '0' (leading zero, invalid) from '10',  
'20' etc. (valid multi-digit numbers  
ending in 0).  
# 2. Trailing number – if abbr ends with  
digits (e.g. abbr='a4'), the loop exits  
# when abbr_ptr reaches end but skip is  
still non-zero.  
# The final check word_ptr + skip ==  
len(word) handles this correctly.  
#
```

Given more time I'd ask: what if we want
to generate ALL valid abbreviations
for a word? That's a backtracking
problem – enumerate all subsets of
positions
to replace with their lengths."

Two Pointers

□ #977 Squares of a Sorted Array

EAS

[Open on LeetCode](#)

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`.

After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

The input array is sorted in non-decreasing order and can contain negatives.

We need to return a new array of the squares of each element, also sorted. Right?"

#

"A few quick questions:

Can the array contain negative numbers? Yes – that's the whole challenge.

Should I return a new array or modify in-place? Return a new array.

Can the array have a single element? Yes
– just return $[\text{element}^2]$.
Can all elements be negative? Yes – e.g.
 $[-3, -2, -1] \rightarrow [1, 4, 9]$.

"Let me walk example 1:
$\text{nums} = [-4, -1, 0, 3, 10]$. Squares:
 $[16, 1, 0, 9, 100]$. Sorted: $[0, 1, 9, 16, 100]$.
The key observation: the largest squares
come from the extremes –
either the most negative or the most
positive end. Got it."
PHASE 2 — BRUTE FORCE
"Let me start with brute force to lock
in the logic.
Square every element, sort the resulting
array ascending, return it.
Two-pass: fill squares $O(n)$, then sort
 $O(n \log n)$.
Correct given unsorted input, but the
array is already sorted – we can merge
from both ends.
Time: $O(n \log n)$. Space: $O(n)$ for the
squared copy.
Should I go ahead and optimize?"

```
class _977_SquaresSortedArray_Brute:
    def sortedSquares(self, nums:
List[int]) -> List[int]:
        return sorted(x * x for x in nums)

# PHASE 3 — OPTIMIZE
# "The brute force ignores the fact that
the input is already sorted.
# Key insight: in a sorted array, the
largest absolute values are always at one
# of the two ends – either the most
negative (left) or the most positive
(right).
# So the largest square is always
max(nums[left]2, nums[right]2).
#
# Use two pointers: left starts at 0,
right at n-1.
# Fill the result array from the back
(largest to smallest):
# at each step, compare abs(nums[left])
and abs(nums[right]).
# Place the larger square at result[k] and
move that pointer inward.
#
# Pseudocode:
```

```
# result = [0] * n; k = n-1; left, right =  
0, n-1  
# while k >= 0:  
# if abs(nums[left]) > abs(nums[right]):  
# result[k] = nums[left]**2; left += 1  
# else:  
# result[k] = nums[right]**2; right -= 1  
# k -= 1  
# return result  
#  
# Time: O(n). Space: O(n) for the output –  
unavoidable since we return a new array."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

```
class _977_SquaresSortedArray:  
    def sortedSquares(self, nums:  
List[int]) -> List[int]:  
        n = len(nums)  
        result = [0] * n  
        fill = n - 1  
        left = 0  
        right = n - 1  
        while fill >= 0:
```

```
        if abs(nums[left]) >
abs(nums[right]):
            result[fill] = nums[left]
* nums[left]
            left += 1
        else:
            result[fill] =
nums[right] * nums[right]
            right -= 1
            fill -= 1
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace both examples."

#

nums=[-4,-1,0,3,10], n=5

fill=4: abs(-4)=4 < abs(10)=10 →
result[4]=100, right=3. fill=3.

fill=3: abs(-4)=4 > abs(3)=3 →
result[3]=16, left=1. fill=2.

fill=2: abs(-1)=1 < abs(3)=3 →
result[2]=9, right=2. fill=1.

fill=1: abs(-1)=1 > abs(0)=0 →
result[1]=1, left=2. fill=0.

fill=0: left==right==2, abs(0)=0 →
result[0]=0, right=1. fill=-1.

```
# result=[0,1,9,16,100] ✓
#
# nums=[-7,-3,2,3,11], n=5
# fill=4: abs(-7)=7 < abs(11)=11 →
result[4]=121, right=3. fill=3.
# fill=3: abs(-7)=7 > abs(3)=3 →
result[3]=49, left=1. fill=2.
# fill=2: abs(-3)=3 == abs(3)=3 → take
right: result[2]=9, right=2. fill=1.
# fill=1: abs(-3)=3 > abs(2)=2 →
result[1]=9, left=2. fill=0.
# fill=0: left==right==2 → result[0]=4.
fill=-1.
# result=[4,9,9,49,121] ✓
#
# "No bugs. Tie goes to right which is
fine – both are equal."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$  – single pass from both ends
meeting in the middle.
# Space  $O(n)$  – required for the output; we
can't avoid this since we return a new
array.
#
```

```
# This is the same fill-from-back pattern
# as Merge Sorted Array (#88) –
# worth naming that explicitly to show you
# recognise the pattern family.
#
# Trade-off: the brute force  $O(n \log n)$  is
# two lines and perfectly acceptable
# in most interviews. The  $O(n)$  two-pointer
# version is the follow-up answer
# the problem explicitly asks for.
#
# Given more time I'd ask: what if we
# needed to do this in-place?
# Squaring in-place destroys order for
# negative numbers, so we'd need to
# square + reverse the negative half, then
# merge the two sorted halves
# in-place – doable but significantly more
# complex."
```

Sorting

Round 1 · High · Easy · 6 questions

Sorting

□ #169 Majority Element

EAS

[Open on LeetCode](#)

Array · Hash Table · Divide and Conquer ·
Sorting · Counting

Lists: Grind 169

Problem

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $n / 2$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 * 10^4$
- $-109 \leq \text{nums}[i] \leq 109$
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

The majority element is the one that appears MORE THAN $n/2$ times.

We're guaranteed it always exists. We return the element, not its count. Right?"

#

"A few quick questions:

Is there always exactly one majority element? Yes – anything above $n/2$ means

at most one element can qualify.

Can n be 1? Yes – that single element is trivially the majority.

Can values be negative? Yes, constraints allow down to -10^9 .

The follow-up asks for linear time and $O(1)$ space – I'll aim for that."

#

"Let me walk the examples:

`nums=[3,2,3]`: 3 appears 2 times, $n/2=1$.
 $2 > 1 \rightarrow 3$ is majority. ✓

`nums=[2,2,1,1,1,2,2]`: 2 appears 4 times,
 $n/2=3.5$. $4 > 3.5 \rightarrow 2$. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Count frequency of every element with a hash map, then scan counts for any value $> n/2$.

Correct because the majority element must appear more than half the time.

Time: $O(n)$. Space: $O(n)$ for counts.

Should I go ahead and optimize?"

```
class _169_MajorityElement_Brute:
    def majorityElement(self, nums:
List[int]) -> int:
```

```
        return
Counter(nums).most_common(1)[0][0]

# PHASE 3 — OPTIMIZE
# "To get  $O(1)$  space, I'll use Boyer–Moore Voting Algorithm.
# The core insight: if we cancel every occurrence of the majority element with a different element, the majority element will always survive because it has more than  $n/2$  appearances — more than everything else combined.
#
# Keep a candidate and a count:
# When count == 0, adopt the current element as the new candidate.
# If current element matches candidate, increment count.
# If it doesn't match, decrement count (a cancellation).
# The candidate at the end is guaranteed to be the majority element.
#
# Pseudocode:
# candidate, count = None, 0
# for x in nums:
```

```
# if count == 0: candidate = x
# count += (1 if x == candidate else -1)
# return candidate
#
# Time: O(n) – single pass. Space: O(1) –
# two variables only."

# PHASE 4 — CLEAN CODE
# "Now I'll implement Boyer-Moore with
# meaningful names."
class _169_MajorityElement:
    def majorityElement(self, nums:
List[int]) -> int:
        candidate = None
        count = 0
        for num in nums:
            if count == 0:
                candidate = num
            count += 1 if num == candidate
            else -1
        return candidate

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums=[3,2,3]
```

```
# num=3: count=0 → candidate=3. count=1.
# num=2: count=1, 2≠3 → count=0.
# num=3: count=0 → candidate=3. count=1.
# return 3 ✓
#
# nums=[2,2,1,1,1,2,2]
# num=2: count=0 → candidate=2. count=1.
# num=2: match → count=2.
# num=1: no match → count=1.
# num=1: no match → count=0.
# num=1: count=0 → candidate=1. count=1.
# num=2: no match → count=0.
# num=2: count=0 → candidate=2. count=1.
# return 2 ✓
#
# Single element – nums=[7]
# count=0 → candidate=7. count=1. return 7
✓
#
# "No bugs. Even when the candidate flips
mid-array, the majority element
# reclaims it before the end."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$  – single pass. Space  $O(1)$  –
only two variables.
```

```
#  
# Boyer-Moore is the optimal solution here  
and worth naming by name –  
# it shows algorithm knowledge beyond just  
'I used two variables'.  
#  
# Correctness argument worth stating: the  
majority element appears more than  $n/2$   
times.  
# Every time it gets cancelled by a  
different element, it loses one vote and  
so does  
# the other element. Since it has more  
votes than all others combined, it will  
always  
# have votes remaining when everyone else  
is cancelled out.  
#  
# Alternative  $O(n)$  time  $O(1)$  space  
approach: sort and return nums[n//2].  
# Since majority appears  $> n/2$  times, it  
must occupy the middle index.  
# But sorting is  $O(n \log n)$  – worse than  
Boyer-Moore.  
#
```

If the problem didn't guarantee a majority exists, Boyer-Moore would need
a second pass to verify the candidate actually appears $> n/2$ times.

#

Given more time I'd ask: what if we need to find all elements appearing
more than $n/3$ times? Boyer-Moore generalises – maintain two candidates
and two counts. That's a well-known variant worth mentioning."

Sorting

□ #217 Contains Duplicate

EAS[Open on LeetCode](#)

Array · Hash Table · Sorting

Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: true

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return true if any value appears at least twice, false if every element is distinct.

We're not asked which value or how many times – just a boolean. Right?"

#

"A few quick questions:

Can the array be empty? Constraints say length ≥ 1 , so no.

Can there be negative numbers? Yes, down to -10^9 .

Is order relevant? No – we just care about value frequency."

#

"Let me walk the examples:

```
# [1,2,3,1]: 1 appears twice → true ✓  
# [1,2,3,4]: all distinct → false ✓"
```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Check every pair (i, j) with $j > i$; if `nums[i] == nums[j]`, return true immediately.

If no duplicate pair exists after all checks, return false.

Simple and correct, but nested loops blow up on large arrays.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

```
class _217_ContainsDuplicate_Brute:  
    def containsDuplicate(self, nums:  
List[int]) -> bool:  
        for i in range(len(nums)):  
            for j in range(i + 1,  
len(nums)):  
                if nums[i] == nums[j]:  
                    return True  
        return False
```

PHASE 3 — OPTIMIZE

```
# "We're repeatedly asking: have I seen
this value before?
# A hash set answers that in  $O(1)$ . Walk
the array once;
# if the current number is already in the
set, return True immediately.
# Otherwise add it and continue.
#
# Time:  $O(n)$ . Space:  $O(n)$ .
# Alternative: sort and check adjacent
pairs –  $O(n \log n)$ ,  $O(1)$  extra space.
# Hash set is faster; sort is
space-optimal."
```

PHASE 4 — CLEAN CODE

```
class _217_ContainsDuplicate:
    def containsDuplicate(self, nums:
List[int]) -> bool:
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3,1]: seen grows to {1,2,3},  
then 1 hits → True ✓  
# nums=[1,2,3,4]: seen={1,2,3,4}, loop  
ends → False ✓  
# nums=[1]: seen={1}, loop ends → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$  for the set.  
# One-liner alternative: return len(nums)  
!= len(set(nums)) – same complexity,  
# builds the full set before checking. The  
explicit loop returns early on first  
duplicate.  
# For adversarial inputs (duplicate at the  
end) they're the same; for duplicates  
early,  
# the loop wins. In an interview I'd use  
the explicit loop and mention the  
one-liner.  
# Given more time I'd ask: what if the  
array is sorted? Check adjacent pairs –  
 $O(n)$  time,  $O(1)$  space."
```

Sorting

□ #242 Valid Anagram

EAS

[Open on LeetCode](#)

Hash Table · String · Sorting

Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- *s* and *t* consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

t is an anagram of s if t contains exactly the same characters

with the same frequencies, just in a different order. Right?

Both strings consist of only lowercase English letters – so 26 possible chars."

#

"Quick checks:

If lengths differ they can't be anagrams – I'll short-circuit on that.

Follow-up asks about Unicode – I'll address that in Phase 6."

#

"'anagram'/'nagaram': same chars, same counts → true ✓

'rat'/'car': r,a,t vs c,a,r – 't' vs 'c' mismatch → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Sort both strings and compare character
by character.
# Equal sorted forms iff anagrams.
# Simple  $O(n \log n)$ ; counting frequencies
in one pass is  $O(n)$ .
# Time:  $O(n \log n)$ . Space:  $O(n)$  for sorted
copies.
# Should I go ahead and optimize?"

class _242_ValidAnagram_Brute:
    def isAnagram(self, s: str, t: str) ->
bool:
        return sorted(s) == sorted(t)

# PHASE 3 — OPTIMIZE
# "Instead of sorting, count character
frequencies.
# Since only lowercase letters exist, a
26-slot array suffices –  $O(1)$  space.
# Increment for each char in s, decrement
for each char in t.
# If all slots are zero at the end,
they're anagrams.
#
# Time:  $O(n)$ . Space:  $O(1)$  – fixed 26-slot
array."
```


PHASE 4 — CLEAN CODE

```
class _242_ValidAnagram:
    def isAnagram(self, s: str, t: str) ->
bool:
    if len(s) != len(t):
        return False
    freq = [0] * 26
    for c in s:
        freq[ord(c) - ord('a')] += 1
    for c in t:
        freq[ord(c) - ord('a')] -= 1
    return all(x == 0 for x in freq)
```

PHASE 5 — TEST & DEBUG

```
# s='anagram', t='nagaram': lengths
equal.
# After s: a=3,n=1,g=1,r=1,m=1. After t:
all zeroed. → True ✓
# s='rat', t='car': r+1,a+1,t+1 then
c-1,a-1,r-1 → t=1,c=-1 ≠ 0 → False ✓
# s='a', t='ab': len mismatch → False
immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) – 26-slot array
is a constant.
```

Follow-up: Unicode characters → the 26-slot array breaks.
Use a Counter (hash map) instead – $O(n)$ time, $O(k)$ space where k is distinct chars.
return Counter(s) == Counter(t)
Handles any alphabet. In an interview, mention this unprompted."

Sorting

□ ★ #252 Meeting Rooms ★

EAS

[Open on LeetCode](#)

Array · Sorting

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Example 1:

Input: `intervals =`
`[[0,30],[5,10],[15,20]]`
Output: `false`

Example 2:

Input: `intervals = [[7,10],[2,4]]`
Output: `true`

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"We have a list of meetings [start, end]. We need to check if one person can attend ALL of them – meaning no two meetings overlap.

The problem says meetings ending at t and starting at t do NOT overlap. Right?"

#

"Quick questions:

Can intervals be empty? Yes – 0 meetings, trivially true.

Are start and end guaranteed start < end? Yes, constraints say $0 \leq \text{start} < \text{end}$."

#

"[[0,30],[5,10],[15,20]]: 0–30 overlaps with 5–10 → false ✓

[[7,10],[2,4]]: 2–4 ends before 7–10 starts → true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Compare every pair of meetings
[start_i,end_i] and [start_j,end_j].
# Overlap exists if start_i < end_j and
start_j < end_i.
# O(n^2) pairs; sorting by start time
enables one linear scan.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"
```

```
class _252_MeetingRooms_Brute:
    def canAttendMeetings(self,
intervals: List[List[int]]) -> bool:
        for i in range(len(intervals)):
            for j in range(i + 1,
len(intervals)):
                a, b = intervals[i]
                c, d = intervals[j]
                if a < d and c < b:
                    return False
        return True
```

PHASE 3 — OPTIMIZE

```
# "Sort by start time. Then a single pass
is enough:
# if any meeting ends after the next one
starts, they overlap.
```

We only need to check consecutive pairs after sorting – $O(n \log n)$."

PHASE 4 — CLEAN CODE

```
class _252_MeetingRooms:
    def canAttendMeetings(self,
        intervals: List[List[int]]) -> bool:
        intervals.sort()
        for prev, curr in
pairwise(intervals):
            if prev[1] > curr[0]:
                return False
        return True
```

PHASE 5 — TEST & DEBUG

```
# [[0,30],[5,10],[15,20]] → sorted: same.
prev=[0,30],curr=[5,10]: 30>5 → False ✓
# [[7,10],[2,4]] → sorted:
[[2,4],[7,10]]. prev=[2,4],curr=[7,10]:
4>7? No → True ✓
# []: pairwise on empty → no iterations →
True ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$ for sort, $O(n)$ for the pass. Space $O(1)$ extra.

```
# pairwise is Python 3.10+; fallback: for
i in range(1, len(intervals)).
# Given more time I'd ask: what's the
minimum number of rooms needed?
# That's Meeting Rooms II (LeetCode #253) – use a
min-heap of end times,  $O(n \log n)$ ."
```

Sorting

□ ★ #1086 Largest Unique Number ★

EAS

[Open on LeetCode](#)

Array · Hash Table · Sorting

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, return the largest integer that only occurs once. If no integer occurs once, return `-1`.

Example 1:

Input: `nums = [5,7,3,9,4,9,8,3,1]`

Output: `8`

Explanation: The maximum integer in the array is 9 but it is repeated. The number 8 occurs only once, so it is the answer.

Example 2:

Input: `nums = [9,9,8,8]`

Output: `-1`

Explanation: There is no number that occurs only once.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$
- $0 \leq \text{nums}[i] \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return the largest integer that appears exactly once. If none, return -1. Right?"

Values are 0–1000, array length up to 2000."

#

"[5,7,3,9,4,9,8,3,1]: 9 appears twice, 8 appears once → 8 ✓

[9,9,8,8]: no unique element → -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Sort the array descending. Scan from left and return the first element

that is different from both its left and right neighbors (i.e. appears exactly once).

Time: $O(n \log n)$. Space: $O(1)$ in-place sort. Should I go ahead and optimize?"

```
class _1086_LargestUniqueNumber_Brute:
    def largestUniqueNumber(self, nums:
List[int]) -> int:
        nums.sort(reverse=True)
        for i in range(len(nums)):
            left_same = (i > 0 and nums[i]
== nums[i - 1])
            right_same = (i < len(nums) -
1 and nums[i] == nums[i + 1])
            if not left_same and not
right_same:
                return nums[i]
        return -1
```

PHASE 3 — OPTIMIZE

```
# "Count frequencies with Counter. Walk
unique keys, track max where count==1.
# O(n) time, O(n) space."
```

PHASE 4 — CLEAN CODE

```
class _1086_LargestUniqueNumber:
    def largestUniqueNumber(self, nums:
List[int]) -> int:
        freq = Counter(nums)
        result = -1
        for num, count in freq.items():
```

```
        if count == 1 and num >
result:
        result = num
    return result
```

PHASE 5 — TEST & DEBUG

[5,7,3,9,4,9,8,3,1]:

freq={5:1,7:1,3:2,9:2,4:1,8:1,1:1}

Unique nums: 5,7,4,8,1. Max = 8 ✓

[9,9,8,8]: no count==1 entries → result stays -1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$.

Alternative: sort descending, scan until you find a non-duplicate – $O(n \log n)$.

Counter approach is cleaner. Given more time I'd ask: what if we want the k largest

unique numbers? Collect all uniques, sort descending, take first k."

Sorting

□ #1122 Relative Sort Array

EAS

[Open on LeetCode](#)

Array · Hash Table · Sorting · Counting Sort
Lists: Denny Zhang

Problem

Given two arrays `arr1` and `arr2`, the elements of `arr2` are distinct, and all elements in `arr2` are also in `arr1`.

Sort the elements of `arr1` such that the relative ordering of items in `arr1` are the same as in `arr2`. Elements that do not appear in `arr2` should be placed at the end of `arr1` in ascending order.

Example 1:

Input: `arr1 = [2,3,1,3,2,4,6,7,9,2,19]`,
`arr2 = [2,1,4,3,9,6]`
Output: `[2,2,2,1,4,3,3,9,6,7,19]`

Example 2:

Input: arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]
Output: [22,28,8,6,17,44]

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 1000$
- $0 \leq \text{arr1}[i], \text{arr2}[i] \leq 1000$
- All the elements of arr2 are distinct.
- Each arr2[i] is in arr1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sort arr1 so elements appear in the order defined by arr2.

Elements in arr1 not present in arr2 go at the end, sorted ascending. Right?

arr2 elements are distinct; every arr2 element is guaranteed to exist in arr1."

#

"arr1=[2,3,1,3,2,4,6,7,9,2,19],
arr2=[2,1,4,3,9,6]

→ [2,2,2,1,4,3,3,9,6,7,19]: follow arr2 order for those nums, then 7,19 ascending
✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Build a rank map from arr2: each element maps to its index (0..len(arr2)-1).

Elements not in arr2 get rank len(arr2) and are sorted by value as a tiebreaker.

Then sort arr1 using that composite key.

Time: $O(n \log n)$. Space: $O(n)$ for the rank map. Should I go ahead and optimize?"

```
class _1122_RelativeSortArray_Brute:
```

```
    def relativeSortArray(self, arr1:
List[int], arr2: List[int]) -> List[int]:
        rank = {val: i for i, val in
enumerate(arr2)}
        return sorted(arr1, key=lambda x:
(rank[x], 0) if x in rank else (len(arr2),
x))
```

PHASE 3 — OPTIMIZE

"Count arr1 frequencies with Counter.

For each num in arr2, extend result with [num]*freq[num], remove from counter.

Then sort remaining counter items and extend.

Time: $O(n + k \log k)$ where k = elements not in arr2. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _1122_RelativeSortArray:
    def relativeSortArray(self, arr1:
List[int], arr2: List[int]) -> List[int]:
        freq = Counter(arr1)
        result = []
        for num in arr2:
            result.extend([num] *
freq[num])
            del freq[num]
        for num, count in
sorted(freq.items()):
            result.extend([num] * count)
        return result
```

PHASE 5 — TEST & DEBUG

```
# arr1=[2,3,1,3,2,4,6,7,9,2,19],
arr2=[2,1,4,3,9,6]
# freq={2:3,3:2,1:1,4:1,6:1,7:1,9:1,19:1}
# arr2 pass: [2,2,2,1,4,3,3,9,6].
remaining: {7:1,19:1}
# sorted remaining -> [7,19].
result=[2,2,2,1,4,3,3,9,6,7,19] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + k \log k)$ where k is the count of elements not in `arr2` (at most n).

Overall $O(n \log n)$. Space $O(n)$.

Given more time I'd ask: what if `arr2` has elements not in `arr1`?

The problem guarantees that won't happen, but we'd skip them if it did."

Binary Search

Round 1 · High · Easy · 5 questions

Binary Search

□ #35 Search Insert Position

EAS

[Open on LeetCode](#)

Array · Binary Search

Lists: Denny Zhang

Problem

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [1,3,5,6]`, `target = 5`

Output: 2

Example 2:

Input: `nums = [1,3,5,6]`, `target = 2`

Output: 1

Example 3:

Input: `nums = [1,3,5,6]`, `target = 7`
Output: `4`

Constraints:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`
- `nums` contains distinct values sorted in ascending order.
- `-104 <= target <= 104`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sorted array of distinct integers.

Return the index of target if found,

otherwise return the index where it would be inserted to keep order. Right?

Must run in $O(\log n)$ – so binary search is required."

#

"[1,3,5,6], target=5 → index 2 (found) ✓

[1,3,5,6], target=2 → index 1 (between 1 and 3) ✓

[1,3,5,6], target=7 → index 4 (after all) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan from left: return the first index where `nums[i] >= target`.

If we exhaust the array without finding such an index, target goes at the end.

Time: $O(n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _35_SearchInsertPosition_Brute:
```

```
    def searchInsert(self, nums:
List[int], target: int) -> int:
        for i in range(len(nums)):
            if nums[i] >= target:
                return i
        return len(nums)
```

PHASE 3 — OPTIMIZE

"Standard binary search. The key insight: when the loop ends (`left > right`),

'left' always points to the correct insertion position.

When target is found, return mid immediately.

Otherwise left is where target belongs."

PHASE 4 — CLEAN CODE

```
class _35_SearchInsertPosition:
    def searchInsert(self, nums:
List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

PHASE 5 — TEST & DEBUG

```
# [1,3,5,6], target=5: mid=2→5==5 → return
2 ✓
# [1,3,5,6], target=2: mid=2→5>2→right=1.
mid=0→1<2→left=1. mid=1→3>2→right=0.
# left=1>right=0 → return left=1 ✓
# [1,3,5,6], target=7: left keeps
advancing → return 4 ✓
# [1,3,5,6], target=0: right keeps
retreating → return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.

The 'return left' at the end is the core insight – left always converges

to the leftmost position where target can fit. This same pattern appears

in lower_bound in C++ and bisect_left in Python.

Given more time I'd ask: what if the array has duplicates?

We'd need to decide: first occurrence or last? That's bisect_left vs bisect_right."

Binary Search

□ #69 Sqrt(x)

EAS

[Open on LeetCode](#)

Math · Binary Search

Lists: Denny Zhang

Problem

Given a non-negative integer x , return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: $x = 4$

Output: 2

Explanation: The square root of 4 is 2, so we return 2.

Example 2:

Input: $x = 8$

Output: 2

Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

Constraints:

- $0 \leq x \leq 2^{31} - 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return `floor(sqrt(x))` without using built-in `sqrt` or `** 0.5`. Right?

`x` can be 0 — `sqrt(0)=0`. `x` can be up to $2^{31}-1$."

#

"`x=4` → 2. `x=8` → 2 (floor of 2.828). `x=0` → 0. `x=1` → 1."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan `m` from 0 upward: keep incrementing while `m*m <= x`.

The moment $(m+1)*(m+1) > x$, we know m is the floor sqrt.

Time: $O(\sqrt{x})$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _69_Sqrt_Brute:
    def mySqrt(self, x: int) -> int:
        m = 0
        while (m + 1) * (m + 1) <= x:
            m += 1
        return m
```

PHASE 3 — OPTIMIZE

"Binary search on the answer space $[0, x]$.

Find the largest m such that $m*m \leq x$.

When $m*m < x$, record m as a candidate and search right.

When $m*m > x$, search left. When $m*m == x$, return immediately.

Time: $O(\log x)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _69_Sqrt:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x
```

```
        left, right = 1, x // 2 # sqrt(x)
    <= x//2 for x >= 4
        result = 0
        while left <= right:
            mid = left + (right - left) //
2
            if mid * mid == x:
                return mid
            elif mid * mid < x:
                result = mid
                left = mid + 1
            else:
                right = mid - 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# x=4: left=1,right=2.
mid=1→1<4→result=1,left=2.
mid=2→4==4→return 2 ✓
# x=8: left=1,right=4.
mid=2→4<8→result=2,left=3.
mid=3→9>8→right=2.
# left>right → return result=2 ✓
# x=0: return 0 immediately ✓
# x=1: return 1 immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log x)$. Space $O(1)$.

Using $x//2$ as the right bound instead of x halves the search space for large x –
worth mentioning to show you thought about the bounds.

$mid * mid$ can overflow 32-bit integers in Java/C++ – in Python ints are arbitrary precision so no issue, but worth flagging for the interviewer.

Given more time I'd ask: what about Newton's method?

$x_{n+1} = (x_n + x/x_n) / 2$ converges quadratically – faster in practice."

Binary Search

□ #278 First Bad Version

EAS[Open on LeetCode](#)

Binary Search · Interactive

Lists: Grind 169

Problem

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad.

Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: $n = 5$, $bad = 4$

Output: 4

Explanation:

call `isBadVersion(3)` -> false

call `isBadVersion(5)` -> true

call `isBadVersion(4)` -> true

Then 4 is the first bad version.

Example 2:

Input: $n = 1$, $bad = 1$

Output: 1

Constraints:

- $1 \leq bad \leq n \leq 2^{31} - 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Versions $1..n$. Once a version is bad, all subsequent are bad.

Find the first bad version while minimising API calls. Right?

`isBadVersion(k)` is given – we don't implement it, just call it."

#

```
# "n=5, bad=4: isBadVersion(3)=F,  
isBadVersion(5)=T, isBadVersion(4)=T → 4  
✓
```

```
# We want to minimise calls → binary  
search, not linear scan."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to  
lock in the logic.
```

```
# Linear scan from version 1 to n: call  
isBadVersion on each.
```

```
# The first version that returns True is  
the answer.
```

```
# Time:  $O(n)$  API calls – way too many for  
large n. Space:  $O(1)$ . Should I go ahead  
and optimize?"
```

```
class _278_FirstBadVersion_Brute:  
    def firstBadVersion(self, n: int) ->  
int:  
    for v in range(1, n + 1):  
        if isBadVersion(v):  
            return v  
    return n
```

PHASE 3 — OPTIMIZE

"Binary search: if mid is bad, the first bad could be mid or earlier → right = mid - 1.

If mid is good, first bad must be after mid → left = mid + 1.

When loop ends, left = first bad version.

This never calls isBadVersion more than $O(\log n)$ times."

PHASE 4 — CLEAN CODE

```
class _278_FirstBadVersion:
```

```
    def firstBadVersion(self, n: int) -> int:
```

```
        left, right = 1, n
```

```
        while left <= right:
```

```
            mid = (left + right) // 2
```

```
            if isBadVersion(mid):
```

```
                right = mid - 1 # might be first bad; keep searching left
```

```
            else:
```

```
                left = mid + 1 #
```

```
definitely good; first bad is after mid
```

```
        return left
```

PHASE 5 — TEST & DEBUG

```
# n=5, bad=4:  
# mid=3→F→left=4. mid=4→T→right=3.  
left=4>right=3 → return 4 ✓  
# n=1, bad=1:  
# mid=1→T→right=0. left=1>right=0 →  
return 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$  calls. Space  $O(1)$ .  
# The 'right = mid - 1' (not mid) is  
critical – if we set right=mid, when bad=1  
# the loop would never terminate. Always  
shrink the window.  
# Given more time I'd ask: what if the API  
is expensive (network call)?  
# Same algorithm – binary search already  
minimises calls. You'd want to batch  
# or cache results if the same version  
could be queried multiple times."
```


Binary Search

□ #374 Guess Number Higher or Lower

EAS[Open on LeetCode](#)

Math · Binary Search · Interactive

Lists: Denny Zhang

Problem

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API `int guess(int num)`, which returns three possible results:

- -1: Your guess is higher than the number I picked (i.e. `num > pick`).
- 1: Your guess is lower than the number I picked (i.e. `num < pick`).
- 0: your guess is equal to the number I picked (i.e. `num == pick`).

Return the number that I picked.

Example 1:

Input: $n = 10$, $pick = 6$
Output: 6

Example 2:

Input: $n = 1$, $pick = 1$
Output: 1

Example 3:

Input: $n = 2$, $pick = 1$
Output: 1

Constraints:

- $1 \leq n \leq 2^{31} - 1$
- $1 \leq pick \leq n$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary search with an API: `guess(num)` returns 0 (correct), -1 (too high),

or 1 (too low). Find the picked number. Right?

Range is $1..n$, n up to $2^{31}-1$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan from 1 to n: call guess(i) on each candidate.

The first call that returns 0 is the picked number.

Time: $O(n)$ calls. Space: $O(1)$. Should I go ahead and optimize?"

```
class _374_GuessNumber_Brute:
```

```
    def guessNumber(self, n: int) -> int:
        for i in range(1, n + 1):
            if guess(i) == 0:
                return i
        return n
```

PHASE 3 — OPTIMIZE

"Binary search: guess(mid). If 0 → found. If -1 → mid too high → right=mid-1.

If 1 → mid too low → left=mid+1. $O(\log n)$ calls."

PHASE 4 — CLEAN CODE

```
class _374_GuessNumber:
```

```
    def guessNumber(self, n: int) -> int:
        left, right = 1, n
```

```
        while left <= right:
            mid = left + (right - left) //
2 # avoids overflow
            result = guess(mid)
            if result == 0:
                return mid
            elif result == -1:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

PHASE 5 — TEST & DEBUG

```
# n=10, pick=6: mid=5→1(low)→left=6.
mid=8→-1(high)→right=7.
# mid=6→0→return 6 ✓
# n=1, pick=1: mid=1→0→return 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ . Space  $O(1)$ .
# Using  $\text{left} + (\text{right} - \text{left}) // 2$  instead of
 $(\text{left} + \text{right}) // 2$  avoids integer overflow
# in languages with fixed-width integers –
worth mentioning even in Python.
# Given more time I'd ask: what if guess()
had a cost per call?
```

Binary search already minimises calls to $O(\log n)$ – optimal."

Binary Search

□ #704 Binary Search

EAS

[Open on LeetCode](#)

Array · Binary Search

Lists: Grind 169

Problem

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`

Output: `4`

Explanation: `9` exists in `nums` and its index is `4`

Example 2:

Input: nums = [-1,0,3,5,9,12], target = 2

Output: -1

Explanation: 2 does not exist in nums so return -1

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 < \text{nums}[i], \text{target} < 104$
- All the integers in nums are unique.
- nums is sorted in ascending order.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Classic binary search: sorted array of distinct integers, find target index.

Return -1 if not found. Must be $O(\log n)$. Right?"

#

"[-1,0,3,5,9,12], target=9 → 4 ✓

[-1,0,3,5,9,12], target=2 → -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan through the array: return the index of the first element equal to target.

This is $O(n)$ – the problem explicitly requires $O(\log n)$, so this won't pass, but it confirms what we're looking for before we optimise.

Time: $O(n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _704_BinarySearch_Brute:
    def search(self, nums: List[int],
target: int) -> int:
        for i in range(len(nums)):
            if nums[i] == target:
                return i
        return -1
```

PHASE 3 — OPTIMIZE

"Standard binary search. Narrow search space by half each iteration.

Three outcomes at each step: found, go left, go right."

PHASE 4 — CLEAN CODE

```
class _704_BinarySearch:
```



```
def search(self, nums: List[int],
target: int) -> int:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) //
2
        if nums[mid] == target:
            return mid
        elif nums[mid] > target:
            right = mid - 1
        else:
            left = mid + 1
    return -1
```

PHASE 5 — TEST & DEBUG

```
# nums=[-1,0,3,5,9,12], target=9:
# mid=2→3<9→left=3. mid=4→9==9→return 4 ✓
# target=2: mid=2→3>2→right=1.
mid=0→-1<2→left=1. mid=1→0<2→left=2.
# left>right → return -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ . Space  $O(1)$ .
# This is the template all other binary
search problems build on.
```

Key invariant: loop condition is left <= right (not <) so single-element
arrays are handled – the loop runs once and either returns or terminates cleanly.
Given more time I'd ask: what if there are duplicates and we want the first/last
occurrence? That's leftmost/rightmost
binary search – adjust which boundary
updates when nums[mid]==target."

Matrix

Round 1 · High · Easy · 5 questions

Matrix

□ ★ #422 Valid Word Square ★

EAS

[Open on LeetCode](#)

Array · Matrix

Lists: ★ Premium | Premium 98

Problem

Given an array of strings `words`, return true if it forms a valid word square.

A sequence of strings forms a valid word square if the k th row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

Example 1:

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Input: words =

["abcd", "bnrt", "crmy", "dtye"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crmy".

The 4th row and 4th column both read "dtye".

Therefore, it is a valid word square.

Example 2:

a	b	c	d
b	n	r	t
c	r	m	
d	t		

Input: words =

["abcd", "bnrt", "crm", "dt"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crm".

The 4th row and 4th column both read "dt".

Therefore, it is a valid word square.

Example 3:

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

Input: words =

["ball", "area", "read", "lady"]

Output: false

Explanation:

The 3rd row reads "read" while the 3rd column reads "lead".

Therefore, it is NOT a valid word square.

Constraints:

- $1 \leq \text{words.length} \leq 500$
- $1 \leq \text{words}[i].\text{length} \leq 500$
- `words[i]` consists of only lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A word square means the `kth` row and `kth` column read the same string.

Equivalently: `words[i][j] == words[j][i]` for every valid `(i,j)`. Right?

Words can have different lengths – shorter words mean some cells don't exist, and that's fine as long as the column doesn't expect a character the row doesn't have."

#

`['abcd', 'bnrt', 'crmy', 'dtye']`:
`row0='abcd', col0='abcd' ✓ etc. → true ✓`

`['ball', 'area', 'read', 'lady']`:
`row2='read', col2='lead' → false ✓`

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# For each row index r, check whether word
matches down column r and across row r
# using direct character comparisons.
# Four nested checks per candidate row –
fine for small boards.
# Time:  $O(n * m * k)$  where  $k = \text{len}(\text{word})$ .
Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "No algorithmic improvement over  $O(n*m)$ 
– we must check every character.
# The implementation is already optimal.
Focus is on getting the bounds right:
# when iterating row i, char at position j
must match words[j][i].
# Guard:  $j < \text{len}(\text{words})$  and  $i <
\text{len}(\text{words}[j])$ ."
```

PHASE 4 — CLEAN CODE

```
class _422_ValidWordSquare:
    def validWordSquare(self, words:
List[str]) -> bool:
        for row, word in
enumerate(words):
```

```

        for col, char in
enumerate(word):
            if col >= len(words) or
row >= len(words[col]) or words[col][row]
!= char:
                return False
        return True

```

PHASE 5 — TEST & DEBUG

```

# words=['abcd', 'bnrt', 'crmy', 'dtye']
# row=0, word='abcd':
col=0→words[0][0]='a'=='a'✓.
col=1→words[1][0]='b'=='b'✓. etc.
# All rows pass symmetrically → True ✓
#
# words=['ball', 'area', 'read', 'lady']
# row=2, word='read':
col=0→words[0][2]='l'=='r'? No → False ✓
#
# words=['abcd', 'bnrt', 'crm', 'dt']
# (shorter words)
# row=0, col=3: words[3][0]='d'=='d'✓.
# row=1, col=2: words[2][1]='r'=='n'? No
wait —
# words[2]='crm', words[2][1]='r'.
words[1][2]='r' ✓. All pass → True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n*m)$. Space $O(1)$.

The two guard conditions are the subtle part:

`col >= len(words)` means the column index is out of bounds – the row has more chars

than there are rows, which breaks symmetry → False.

`row >= len(words[col])` means the transpose cell doesn't exist → False.

Given more time I'd ask: what if we want to BUILD a word square from a word list?

That's a backtracking problem – place words row by row, pruning based on column prefixes."

Matrix

□ #566 Reshape the Matrix

EAS[Open on LeetCode](#)

Array · Matrix · Simulation

Lists: CodeSignal

Problem

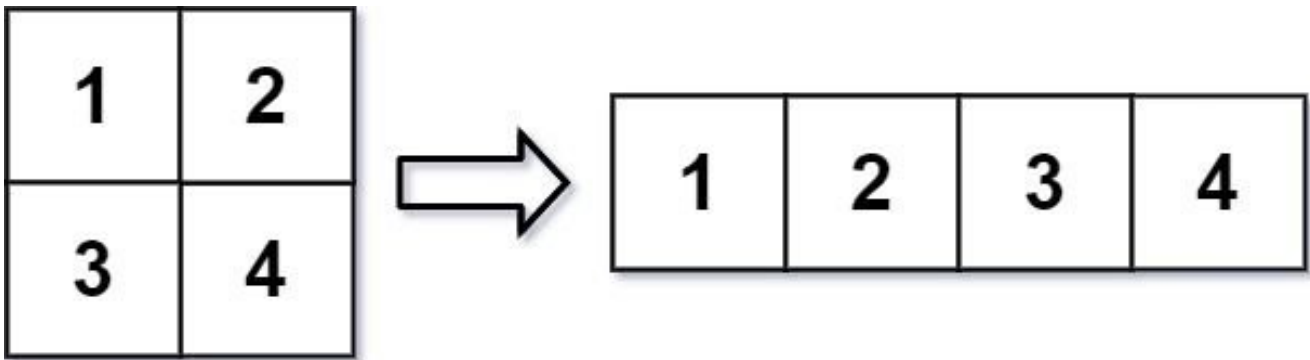
In MATLAB, there is a handy function called **reshape** which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ keeping its original data.

You are given an $m \times n$ matrix **mat** and two integers **r** and **c** representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

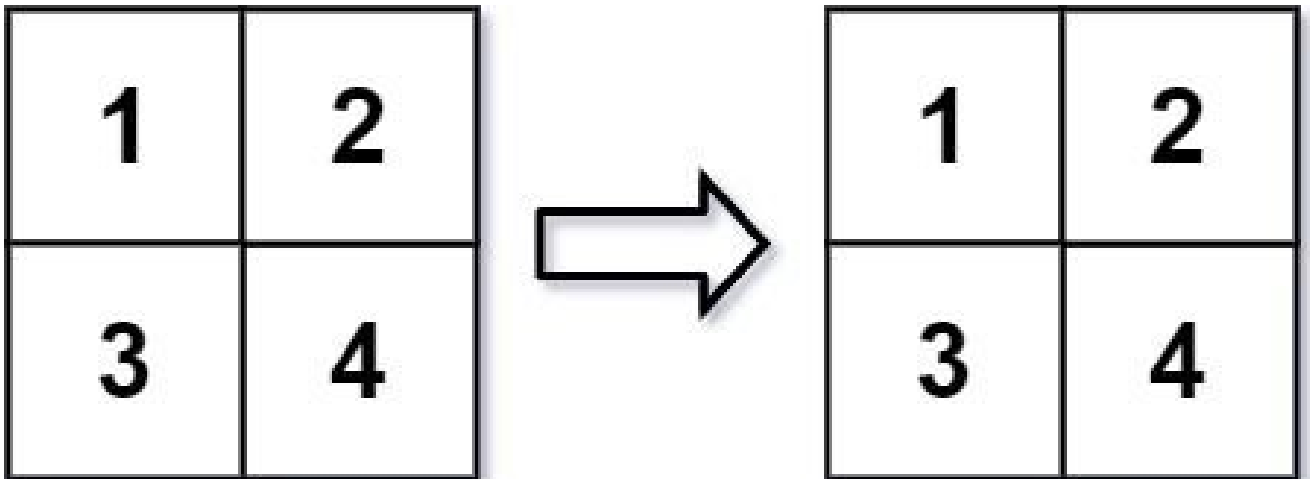
Example 1:



Input: `mat = [[1,2],[3,4]]`, `r = 1`, `c = 4`

Output: `[[1,2,3,4]]`

Example 2:



Input: `mat = [[1,2],[3,4]]`, `r = 2`, `c = 4`

Output: `[[1,2],[3,4]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`

- $1 \leq m, n \leq 100$
- $-1000 \leq \text{mat}[i][j] \leq 1000$
- $1 \leq r, c \leq 300$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We flatten the original $m \times n$ matrix in row-major order and refill into an $r \times c$ matrix.

If $m \times n \neq r \times c$ the reshape is impossible – return the original matrix unchanged. Right?"

#

"Quick questions:

Values can be negative – fine, we're just moving them.

r and c can be up to 300 even though m, n are up to 100 – just need total elements to match."

#

"[[1,2],[3,4]], $r=1, c=4$: 4 elements total, $1 \times 4 = 4 \rightarrow [[1,2,3,4]]$ ✓


```
# [[1,2],[3,4]], r=2,c=4: 4 elements  
total, 2*4=8 ≠ 4 → return original ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Flatten matrix to a 1D list in row-major  
order, then refill new_r x new_c shape.
```

```
# Clear and easy to explain in an  
interview.
```

```
# Uses O(m*n) extra list; two-pointer walk  
avoids allocation.
```

```
# Time: O(m * n). Space: O(m * n) flat  
buffer.
```

```
# Should I go ahead and optimize?"
```

```
class _566_ReshapeMatrix_Brute:  
    def matrixReshape(self, mat:  
List[List[int]], r: int, c: int) ->  
List[List[int]]:  
        m, n = len(mat), len(mat[0])  
        if m * n != r * c:  
            return mat  
        flat = [mat[i][j] for i in  
range(m) for j in range(n)]
```

```
        return [flat[i * c:(i + 1) * c]
for i in range(r)]

# PHASE 3 — OPTIMIZE
# "Skip the intermediate list – map index
directly.
# For flat index k: source is
mat[k//n][k%n], destination is
ans[k//c][k%c].
# Same O(m*n) time but no extra flat array
– just the output matrix."

# PHASE 4 — CLEAN CODE
class _566_ReshapeMatrix:
    def matrixReshape(self, mat:
List[List[int]], r: int, c: int) ->
List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        result = [[0] * c for _ in
range(r)]
        for k in range(m * n):
            result[k // c][k % c] = mat[k
// n][k % n]
        return result
```

PHASE 5 — TEST & DEBUG

```
# mat=[[1,2],[3,4]], r=1, c=4: m=2,n=2.  
4==4 ✓
```

```
# k=0: result[0][0]=mat[0][0]=1. k=1:  
result[0][1]=mat[0][1]=2.
```

```
# k=2: result[0][2]=mat[1][0]=3. k=3:  
result[0][3]=mat[1][1]=4.
```

```
# → [[1,2,3,4]] ✓
```

```
# r=2,c=4: 4≠8 → return original ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n)$ . Space  $O(r*c)$  for the  
output matrix — unavoidable.
```

```
# The index formula  $k//n$  and  $k\%n$  is the  
core insight: flatten index → 2D source.
```

```
# Given more time I'd ask: what if the  
matrix is too large to fit in memory?
```

```
# Process row by row, streaming elements  
into the new shape."
```

Matrix

□ #766 Toeplitz Matrix

EAS

[Open on LeetCode](#)

Array · Matrix

Lists: CodeSignal

Problem

Given an $m \times n$ matrix, return true if the matrix is Toeplitz. Otherwise, return false.

A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.

Example 1:

1	2	3	4
5	1	2	3
9	5	1	2

Input: matrix =

`[[1,2,3,4],[5,1,2,3],[9,5,1,2]]`

Output: true

Explanation:

In the above grid, the diagonals are:

`"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]"`.

In each diagonal all elements are the same, so the answer is True.

Example 2:

1	2
2	2

Input: `matrix = [[1,2],[2,2]]`

Output: `false`

Explanation:

The diagonal "[1, 2]" has different elements.

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 20`

- $0 \leq \text{matrix}[i][j] \leq 99$

Follow up:

- What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
- What if the matrix is so large that you can only load up a partial row into the memory at once?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A Toeplitz matrix has the same value along every top-left to bottom-right diagonal.

We need to check every cell (except the last row and column) equals its down-right neighbour.

Equivalently: $\text{matrix}[i][j] == \text{matrix}[i+1][j+1]$ for all valid i, j .

Right?"

#

"[[1,2,3,4],[5,1,2,3],[9,5,1,2]]: each cell equals its down-right neighbour →

true ✓

`[[1,2],[2,2]]: matrix[0][0]=1 ≠
matrix[1][1]=2 → false ✓"`

PHASE 2 — BRUTE FORCE (also optimal)

**# "Let me start with brute force to lock
in the logic.**

No smarter approach than checking every
diagonal. The key simplification: each
diagonal

is valid iff every adjacent pair on it
matches. So just check `matrix[i][j] ==
matrix[i+1][j+1]`

for every interior cell."

For this input size the brute approach
is already optimal – I'll still state
complexity clearly.

Time: see analysis above. Space: $O(1)$.

PHASE 3 — OPTIMIZE

**# "Same $O(m*n)$ is the best we can do – we
must read every element.**

The implementation is already optimal.
Focus on getting the iteration bounds
right:


```
# we iterate i up to m-2 and j up to n-2
(not m-1, n-1) since we access
[i+1][j+1]."
```

PHASE 4 — CLEAN CODE

```
class _766_ToeplitzMatrix:
    def isToeplitzMatrix(self, matrix:
List[List[int]]) -> bool:
        rows, cols = len(matrix),
len(matrix[0])
        for i in range(rows - 1):
            for j in range(cols - 1):
                if matrix[i][j] !=
matrix[i + 1][j + 1]:
                    return False
        return True
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3,4],[5,1,2,3],[9,5,1,2]]:
rows=3,cols=4. Check (0,0)→(1,1): 1==1✓.
# (0,1)→(1,2): 2==2✓. (0,2)→(1,3): 3==3✓.
(1,0)→(2,1): 5==5✓. etc. → True ✓
# [[1,2],[2,2]]: (0,0)→(1,1): 1≠2 → False
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n)$ . Space  $O(1)$ .
```

Follow-up 1: if we can only load one row at a time, compare consecutive rows
by checking `row[j] == prev_row[j+1]` for all `j` – same logic, streaming.
Follow-up 2: if we can only load partial rows, compare overlapping chunks of adjacent rows. Both follow-ups keep $O(1)$ extra space.
Given more time I'd ask: can we verify a diagonal in $O(1)$ per cell?
Yes – the current approach already does exactly that."

Matrix

□ #867 Transpose Matrix

EAS

[Open on LeetCode](#)

Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

2	4	-1
-10	5	11
18	-7	6



2	-10	18
4	5	-7
-1	11	6

Example 1:

```
Input: matrix =  
[[1,2,3],[4,5,6],[7,8,9]]  
Output: [[1,4,7],[2,5,8],[3,6,9]]
```

Example 2:

```
Input: matrix = [[1,2,3],[4,5,6]]  
Output: [[1,4],[2,5],[3,6]]
```

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 1000$
- $1 \leq m * n \leq 10^5$
- $-10^9 \leq \text{matrix}[i][j] \leq 10^9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The transpose swaps rows and columns:
element at (r,c) moves to (c,r).

If the original is $m \times n$, the result is
 $n \times m$. Return a new matrix – not in-place.
Right?"

#

"[[1,2,3],[4,5,6],[7,8,9]] →
[[1,4,7],[2,5,8],[3,6,9]] ✓

```
# [[1,2,3],[4,5,6]] (2×3) →  
[[1,4],[2,5],[3,6]] (3×2) ✓"
```

PHASE 2 — BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic.

Allocate an $n \times m$ result matrix. Set `result[c][r] = matrix[r][c]` for every (r,c) . Must visit

every element once – is unavoidable."

For this input size the brute approach is already optimal – I'll still state complexity clearly.

Time: $O(m \times n)$. Space: $O(1)$.

PHASE 3 — OPTIMIZE

"No algorithmic improvement. Focus: allocate result with swapped dimensions (`cols × rows`),

then fill with swapped indices. In Python: `zip(*matrix)` gives transposed rows as tuples."

PHASE 4 — CLEAN CODE

```
class _867_TransposeMatrix:  
    def transpose(self, matrix:  
List[List[int]]) -> List[List[int]]:
```

```
        rows, cols = len(matrix),
len(matrix[0])
        result = [[0] * rows for _ in
range(cols)]
        for r in range(rows):
            for c in range(cols):
                result[c][r] =
matrix[r][c]
        return result
```

PHASE 5 — TEST & DEBUG

```
# matrix=[[1,2,3],[4,5,6]]:
rows=2,cols=3. result is 3x2.
# r=0,c=0: result[0][0]=1. r=0,c=1:
result[1][0]=2. r=0,c=2: result[2][0]=3.
# r=1,c=0: result[0][1]=4. r=1,c=1:
result[1][1]=5. r=1,c=2: result[2][1]=6.
# result=[[1,4],[2,5],[3,6]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ for the output.

```
# Python one-liner: return [list(row) for
row in zip(*matrix)]
# zip(*matrix) unpacks rows as arguments
to zip, which pairs corresponding columns.
```

```
# Elegant and interview-clean – worth  
mentioning as an alternative.  
# Given more time I'd ask: can we  
transpose a square matrix in-place?  
# Yes – swap matrix[i][j] and matrix[j][i]  
for all i < j. O(1) extra space."
```

Matrix

□ #2373 Largest Local Values in a Matrix

EAS

[Open on LeetCode](#)

Array · Matrix

Lists: CodeSignal

Problem

You are given an $n \times n$ integer matrix grid.

Generate an integer matrix `maxLocal` of size $(n - 2) \times (n - 2)$ such that:

- `maxLocal[i][j]` is equal to the largest value of the 3×3 matrix in grid centered around row $i + 1$ and column $j + 1$.

In other words, we want to find the largest value in every contiguous 3×3 matrix in grid.

Return the generated matrix.

Example 1:

9	9	8	1
5	6	2	6
8	2	6	4
6	2	2	2

9	9
8	6

Input: `grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]`

Output: `[[9,9],[8,6]]`

Explanation: The diagram above shows the original matrix and the generated matrix.

Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3 x 3 matrix in grid.

Example 2:

1	1	1	1	1
1	1	1	1	1
1	1	2	1	1
1	1	1	1	1
1	1	1	1	1

2	2	2
2	2	2
2	2	2

Input: `grid = [[1,1,1,1,1],[1,1,1,1,1],
[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]]`

Output: `[[2,2,2],[2,2,2],[2,2,2]]`

Explanation: Notice that the 2 is contained within every contiguous 3 x 3 matrix in grid.

Constraints:

- `n == grid.length == grid[i].length`
- `3 <= n <= 100`
- `1 <= grid[i][j] <= 100`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "For each cell (i,j) in the output
(n-2)×(n-2) matrix,
# find the max in the 3×3 subgrid of the
input centered at (i+1, j+1). Right?
# Input is always n×n square, n >= 3."
#
# "grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6
,2,2,2]] (4×4):
# output is 2×2. Top-left 3×3
center=(1,1): max of
[[9,9,8],[5,6,2],[8,2,6]]=9.
# Top-right 3×3 center=(1,2): max of
[[9,8,1],[6,2,6],[2,6,4]]=9. etc. →
[[9,9],[8,6]] ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock
in the logic.
# For each output cell, scan its 3×3
window. No smarter approach since we must
read every
# element at least once. = ."
# For this input size the brute approach
is already optimal – I'll still state
complexity clearly.
# Time:  $O(n^2 \cdot 9)$ . Space:  $O(n^2)$ .
```

PHASE 3 — OPTIMIZE

"The 3×3 is fixed size so inner loops are constant — already $O(n^2)$.

Keep the four nested loops clean with descriptive variable names."

PHASE 4 — CLEAN CODE

```
class _2373_LargestLocal:
```

```
    def largestLocal(self, grid:
List[List[int]]) -> List[List[int]]:
        n = len(grid)
        result = [[0] * (n - 2) for _ in
range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                for di in range(3):
                    for dj in range(3):
                        result[i][j] =
max(result[i][j], grid[i + di][j + dj])
        return result
```

PHASE 5 — TEST & DEBUG

grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]: n=4, output 2×2.

(i=0,j=0): 3×3 from (0,0):

max(9,9,8,5,6,2,8,2,6)=9 ✓

```
# (i=0,j=1): 3×3 from (0,1):  
max(9,8,1,6,2,6,2,6,4)=9 ✓  
# (i=1,j=0): 3×3 from (1,0):  
max(5,6,2,8,2,6,6,2,2)=8 ✓  
# (i=1,j=1): 3×3 from (1,1):  
max(6,2,6,2,6,4,2,2,2)=6 ✓  
# → [[9,9],[8,6]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$  – outer loops  $O((n-2)^2) \approx O(n^2)$ , inner loops constant 9 operations.  
# Space  $O(n^2)$  for the output.  
# Given more time I'd ask: what if the  
window size is variable ( $k \times k$  instead of  
 $3 \times 3$ )?  
# For large  $k$ , recomputing the max from  
scratch is  $O(k^2)$  per cell  $\rightarrow O(n^2 k^2)$  total.  
# A 2D sliding window maximum using  
monotonic deques brings it down to  $O(n^2)$ ."
```

Trees & BST

Round 1 · High · Easy · 11 questions

Trees & BST

□ #100 Same Tree

EAS

[Open on LeetCode](#)

Tree · DFS · BFS

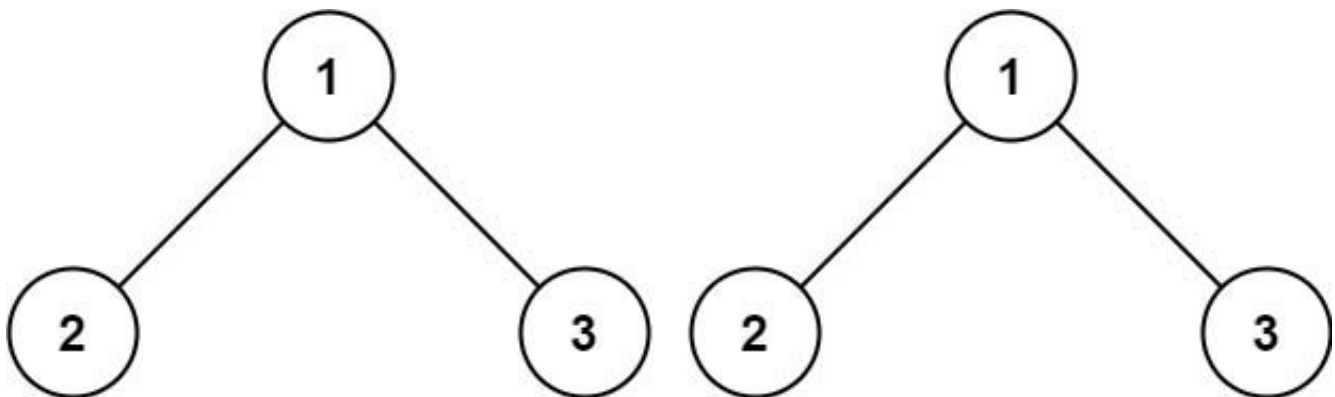
Lists: Grind 169

Problem

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

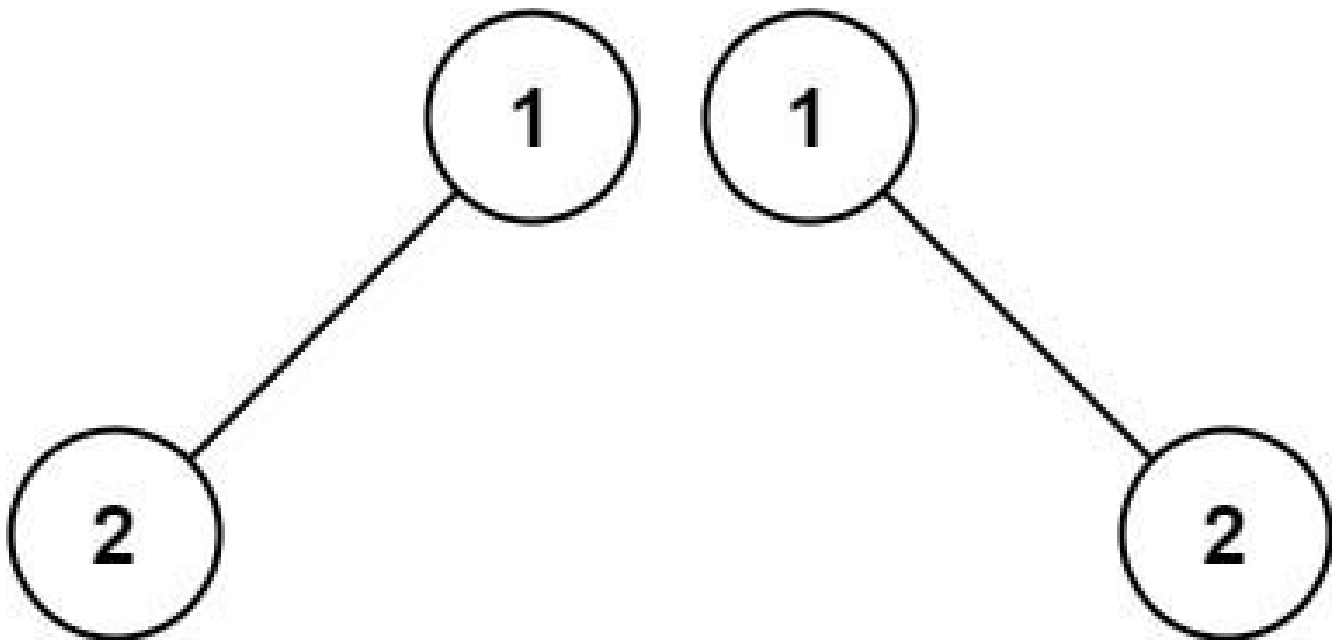
Example 1:



Input: $p = [1,2,3]$, $q = [1,2,3]$

Output: true

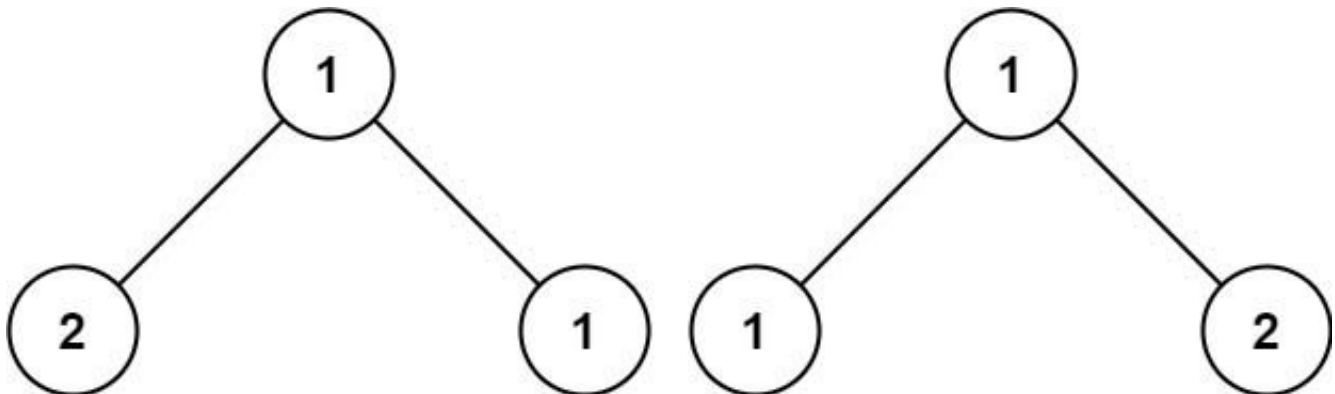
Example 2:



Input: $p = [1,2]$, $q = [1,null,2]$

Output: false

Example 3:



Input: $p = [1,2,1]$, $q = [1,1,2]$

Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.
- $-104 \leq \text{Node.val} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two binary trees are the same if they're structurally identical AND every

corresponding node has the same value.

Both null trees are the same. Right?

One null and one non-null at the same position → not the same."

#

"p=[1,2,3], q=[1,2,3] → true ✓

p=[1,2], q=[1,null,2] → false (structure differs) ✓

p=[1,2,1], q=[1,1,2] → false (values differ at depth 1) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Serialise both trees with a level-order traversal (including nulls) to lists, then compare.

```
# Same O(n) time and O(n) space as the
optimal recursive DFS – no meaningful
complexity
# improvement available, so we skip the
brute class and go straight to the clean
approach."
```

PHASE 3 — OPTIMIZE

```
# "Recursive DFS is both simple and
optimal.
```

```
# Base case: if either node is None, both
must be None for equality.
```

```
# Recursive: values match AND left
subtrees match AND right subtrees match.
```

```
# Use an inner helper to avoid self.
calls."
```

PHASE 4 — CLEAN CODE

```
class _100_SameTree:
    def isSameTree(self, p:
Optional['TreeNode'], q:
Optional['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
```

```
        return p.val == q.val and  
same(p.left, q.left) and same(p.right,  
q.right)
```

```
    return same(p, q)
```

PHASE 5 — TEST & DEBUG

```
# p=[1,2,3], q=[1,2,3]:
```

```
# same(1,1): vals match.
```

```
same(2,2)→same(None,None)=True,
```

```
same(None,None)=True → True.
```

```
# same(3,3)→True. → True ✓
```

```
# p=[1,2,None], q=[1,None,2]:
```

```
# same(1,1): same(2,None): p=2≠None,
```

```
q=None → p==q? No → False ✓
```

```
# p=None, q=None: not p=True → return  
p==q=True ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$  – visit every node once.  
Space  $O(h)$  call stack where  $h$  is tree  
height.
```

```
# Worst case  $O(n)$  for a skewed tree,  $O(\log n)$  for balanced.
```

```
# Iterative alternative: BFS with a queue  
of node pairs –  $O(n)$  time,  $O(n)$  space.
```

```
# Same complexity but avoids recursion  
depth limit for very deep trees.  
# Given more time I'd ask: what if we only  
care about structural identity (not  
values)?  
# Same logic – just remove the val check."
```

Trees & BST

□ #101 Symmetric Tree

EAS

[Open on LeetCode](#)

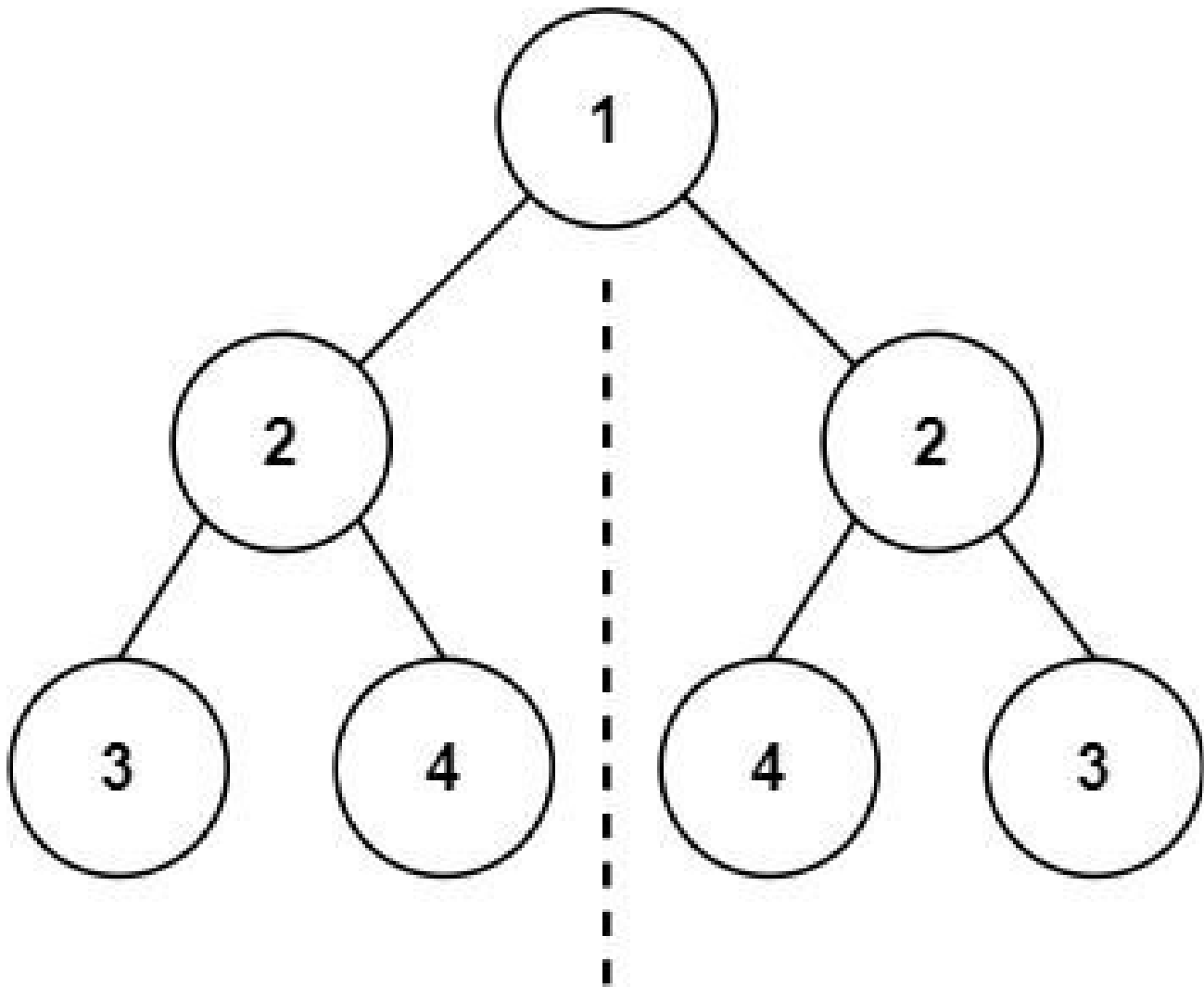
Tree · DFS · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

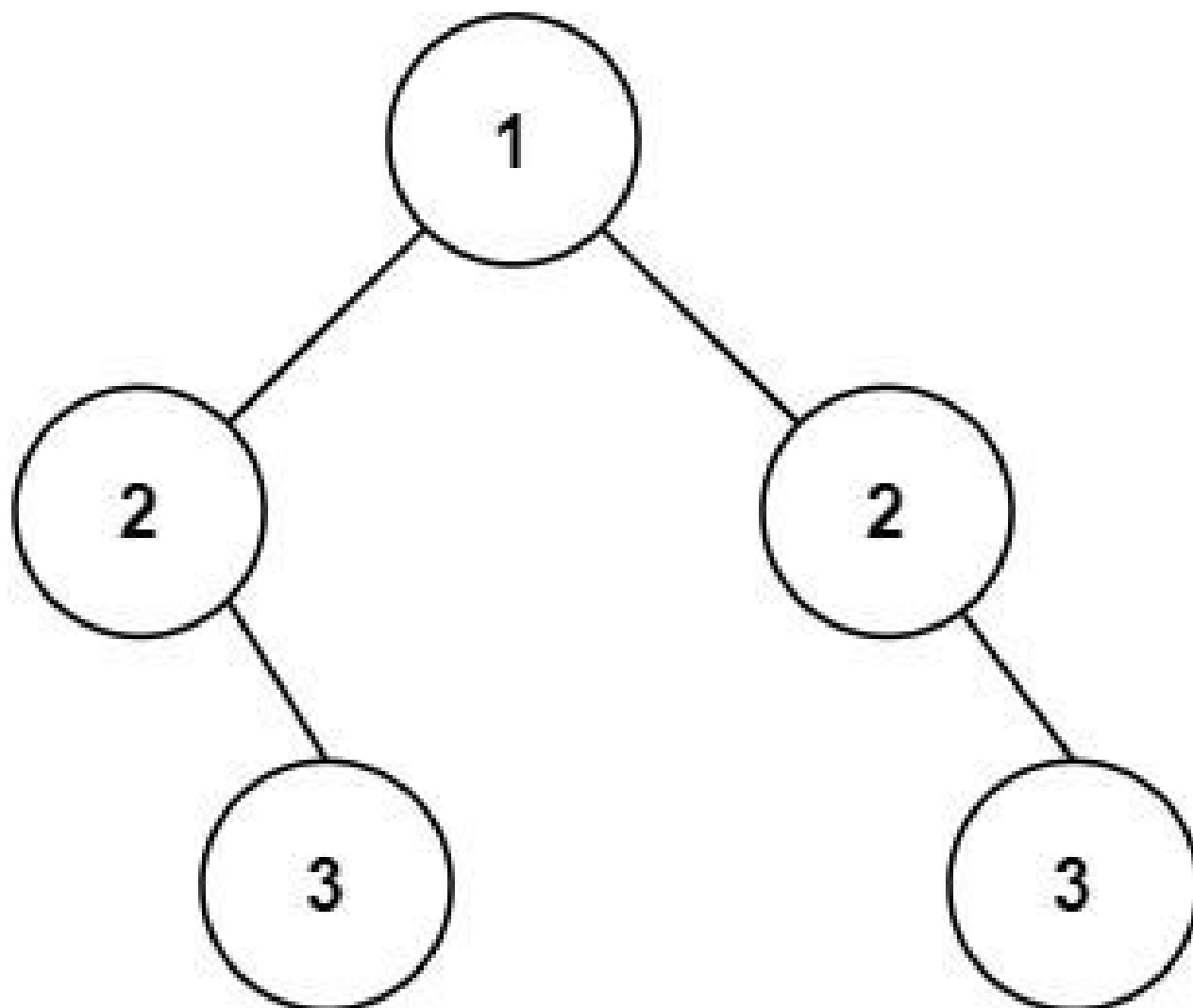
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A tree is symmetric if its left and right subtrees are mirror images of each other."

Mirror means: at each level, left child of left subtree matches right child of right subtree,

and right child of left subtree matches left child of right subtree. Right?

A single node is always symmetric."

#

"[1,2,2,3,4,4,3]: left subtree [2,3,4] mirrors right [2,4,3] → true ✓

[1,2,2,null,3,null,3]: left has right child 3, right has right child 3 – not mirrored → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

Serialise the left subtree with a pre-order traversal and the right subtree

with the mirror order (right child before left), then compare the two

strings.

Same $O(n)$ time and $O(n)$ space as the optimal recursive approach – no improvement

to be had, so we skip the brute class and go straight to the clean solution."

PHASE 3 — OPTIMIZE

"Recursive DFS comparing the tree to itself – left vs right mirrored.

Define an inner helper mirror(p, q):

Base: if either is None, both must be None.

Recursive: values match AND

mirror(p.left, q.right) AND

mirror(p.right, q.left).

Note the cross-comparison: left's left ↔ right's right, left's right ↔ right's left."

PHASE 4 — CLEAN CODE

class _101_SymmetricTree:

 def isSymmetric(self, root:
Optional['TreeNode']) -> bool:

 def mirror(p, q):

 if not p or not q:

```

        return p == q
        return p.val == q.val and
mirror(p.left, q.right) and
mirror(p.right, q.left)
    return mirror(root, root)

```

PHASE 5 — TEST & DEBUG

```

# root=[1,2,2,3,4,4,3]:
# mirror(root,root): vals 1==1.
# mirror(left=2, right=2): 2==2.
# mirror(2.left=3, 2.right=3): 3==3.
mirror(None,None)=T. mirror(None,None)=T
→ T
# mirror(2.right=4, 2.left=4): 4==4. T → T
# mirror(right=2, left=2): symmetric to
above → T
# → True ✓
# root=[1,2,2,null,3,null,3]:
# mirror(left=2,right=2): 2==2.
# mirror(2.left=None, 2.right=3):
p=None,q=3 → None≠3 → False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$ for the call stack.

```
# Follow-up asks for iterative solution:  
use a queue of pairs.  
# Enqueue (root.left, root.right). Each  
step dequeue (p,q):  
# check p==q==None (skip), p or q None  
(false), vals differ (false).  
# Enqueue (p.left,q.right) and  
(p.right,q.left). O(n) time, O(n) space.  
# The cross-enqueue order is the key –  
same insight as the recursive version.  
# Given more time I'd ask: how does this  
differ from Same Tree (#100)?  
# Same Tree checks identical structure;  
Symmetric Tree checks mirrored structure –  
# the only difference is the  
cross-comparison in the recursive call."
```

Trees & BST

□ #104 Maximum Depth of Binary Tree

EAS

[Open on LeetCode](#)

Tree · DFS · BFS

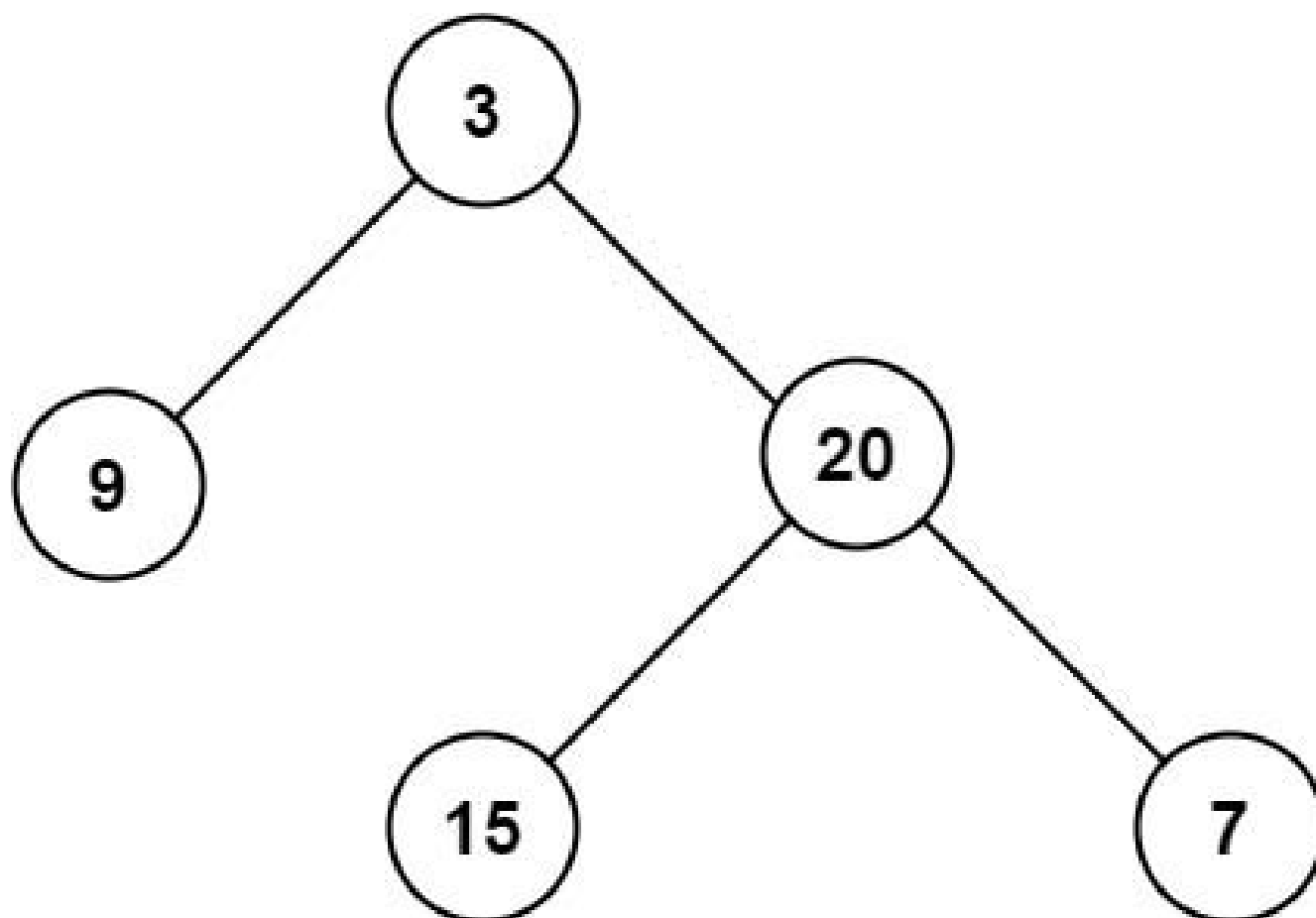
Lists: Grind 169

Problem

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 3

Example 2:

Input: root = [1,null,2]

Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Maximum depth is the number of nodes on the longest root-to-leaf path.

An empty tree has depth 0. A single node has depth 1. Right?"

#

"[3,9,20,null,null,15,7]: longest path is 3→20→15 or 3→20→7, 3 nodes → 3 ✓

[1,null,2]: 1→2, 2 nodes → 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Iterative BFS: use a queue, process the tree level by level.

Each time we exhaust one level, increment a depth counter.

When the queue is empty, the counter equals the maximum depth.

Time: $O(n)$. Space: $O(n)$ for the queue. Should I go ahead and optimize?"

```
class _104_MaxDepth_Brute:
```

```
    def maxDepth(self, root:
Optional['TreeNode']) -> int:
```

```
    if not root:
        return 0
    from collections import deque
    queue = deque([root])
    depth = 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return depth
```

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner and same complexity."

depth(node) = max(depth(left), depth(right)) + 1. Base: empty node → 0.
Use an inner helper to avoid self-calls."

PHASE 4 — CLEAN CODE

```
class _104_MaxDepth:
    def maxDepth(self, root:
Optional['TreeNode']) -> int:
        def depth(node):
            if not node:
                return 0
            return max(depth(node.left),
depth(node.right)) + 1
        return depth(root)
```

PHASE 5 — TEST & DEBUG

```
# root=[3,9,20,null,null,15,7]:
# depth(3)=max(depth(9),depth(20))+1.
# depth(9)=max(0,0)+1=1. depth(20)=max(de
pth(15),depth(7))+1=max(1,1)+1=2.
# depth(3)=max(1,2)+1=3 ✓
# root=None: depth(None)=0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$  call stack –
 $O(\log n)$  balanced,  $O(n)$  skewed.
# BFS alternative: count levels with a
queue –  $O(n)$  time,  $O(w)$  space ( $w=\max$ 
width).
# For very deep trees, BFS avoids
recursion stack overflow.
```


Given more time I'd ask: what about minimum depth? Different – must reach a leaf,
so BFS is actually better there (stops at first leaf found)."

Trees & BST

□ #108 Convert Sorted Array to Binary Search Tree

EAS

[Open on LeetCode](#)

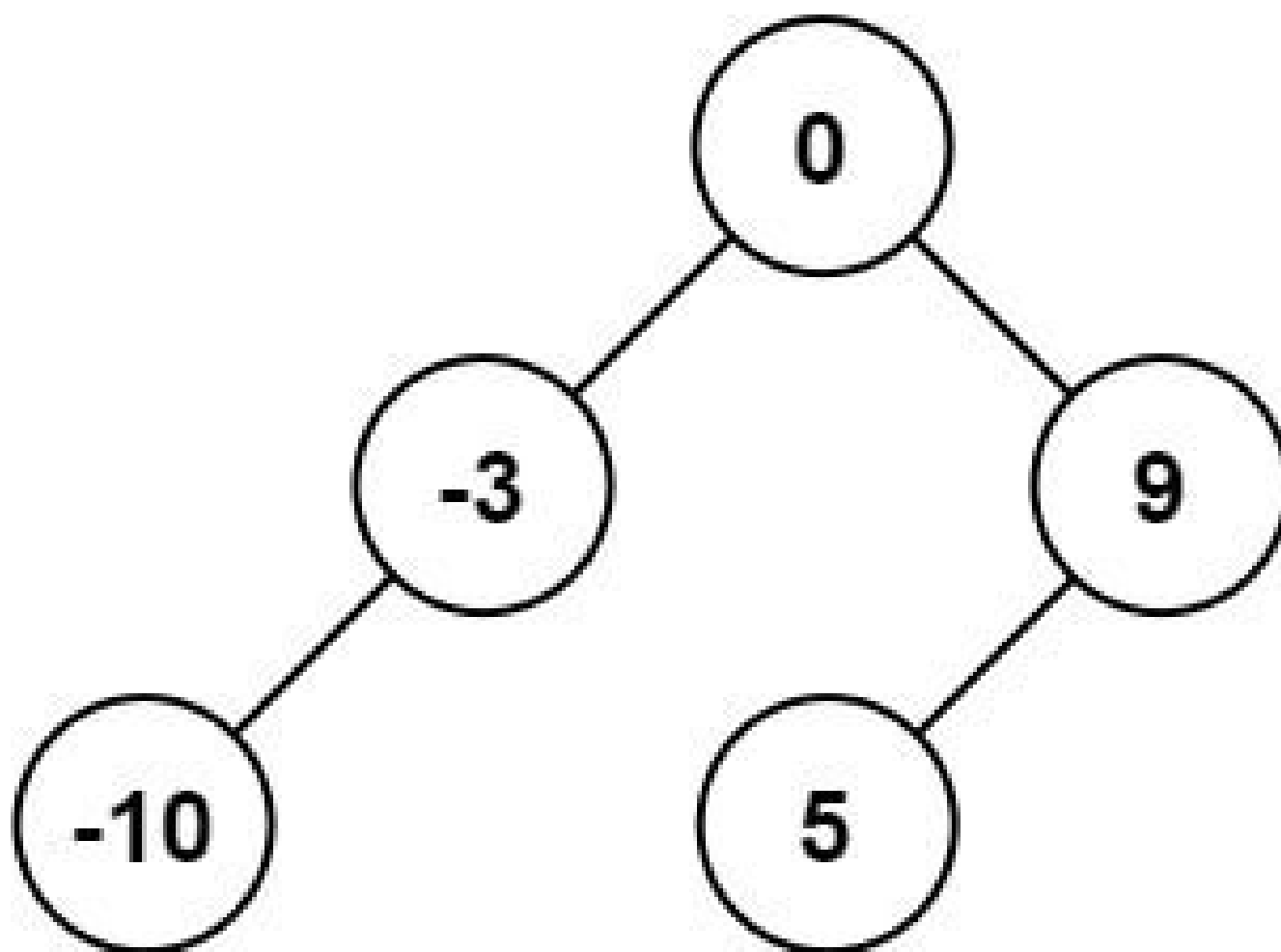
Array · Divide and Conquer · Tree

Lists: Grind 169

Problem

Given an integer array `nums` where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:

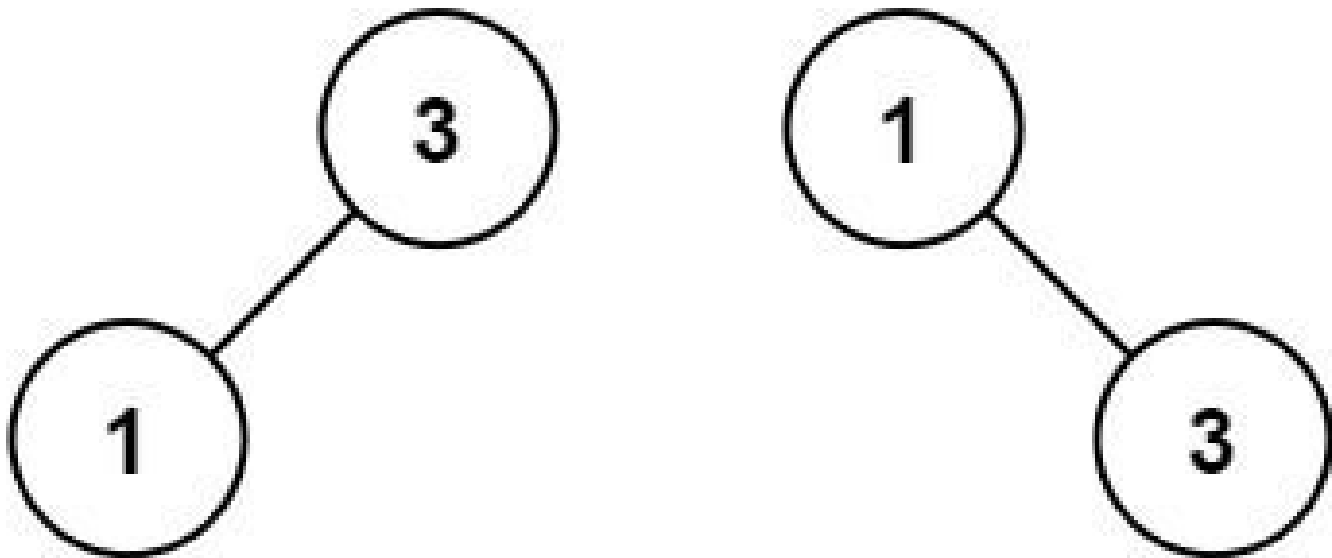


Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]`
is also accepted:

Example 2:



Input: `nums = [1,3]`

Output: `[3,1]`

Explanation: `[1,null,3]` and `[3,1]` are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in a strictly increasing order.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Convert a sorted array to a height-balanced BST.

Height-balanced means every node's left and right subtree heights differ by at

most 1.

Multiple valid answers exist – any height-balanced BST is acceptable.

Right?"

#

"[-10,-3,0,5,9]: pick middle element as root → 0. Left half [-10,-3] → left subtree.

Right half [5,9] → right subtree.

Recurse. → [0,-3,9,-10,null,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Pick the middle element of nums as root, recursively build left from nums[:mid]

and right from nums[mid+1:].

Always produces a valid BST shape for a sorted input.

Time: $O(n \log n)$ from repeated slicing.

Space: $O(n)$ for slices.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Pick the middle element as root – this guarantees balance because both halves

```
# have equal (or ±1) size. Recurse on left  
and right halves.  
# Divide and conquer. Use inner helper  
build(l, r) operating on indices."
```

PHASE 4 — CLEAN CODE

```
class _108_SortedArrayToBST:  
    def sortedArrayToBST(self, nums:  
List[int]) -> Optional['TreeNode']:  
        def build(left, right):  
            if left > right:  
                return None  
            mid = (left + right) // 2  
            return TreeNode(nums[mid],  
build(left, mid - 1), build(mid + 1,  
right))  
        return build(0, len(nums) - 1)
```

PHASE 5 — TEST & DEBUG

```
# nums=[-10,-3,0,5,9]:  
# build(0,4): mid=2→node(0).  
left=build(0,1), right=build(3,4).  
# build(0,1): mid=0→node(-10). left=None,  
right=build(1,1)→node(-3). →  
(-10,None,-3)
```

```
# build(3,4): mid=3→node(5). left=None,  
right=build(4,4)→node(9). → (5,None,9)  
# root=0, left=(-10,None,-3),  
right=(5,None,9). Valid height-balanced  
BST ✓
```

```
# nums=[1,3]: mid=0→node(1),  
right=build(1,1)→node(3). → [1,null,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ — every element becomes a node exactly once.

Space $O(\log n)$ for the call stack — tree is balanced by construction.

Choosing $(l+r)//2$ vs $(l+r+1)//2$ gives slightly different but equally valid trees.

Given more time I'd ask: what if the array is sorted in descending order?

Reverse it first or adjust the index logic — same approach."

Trees & BST

□ #110 Balanced Binary Tree

EAS

[Open on LeetCode](#)

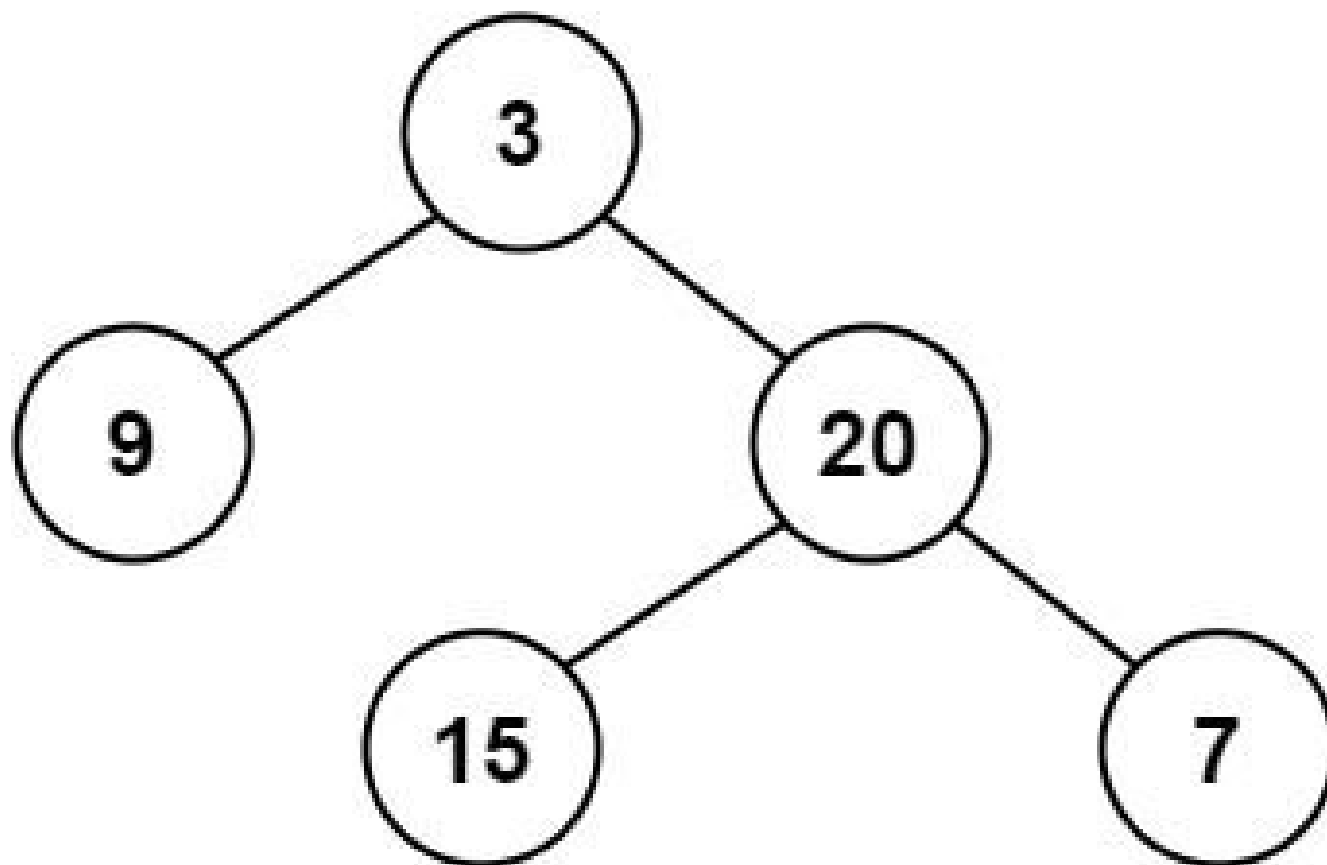
Tree · DFS

Lists: Grind 169

Problem

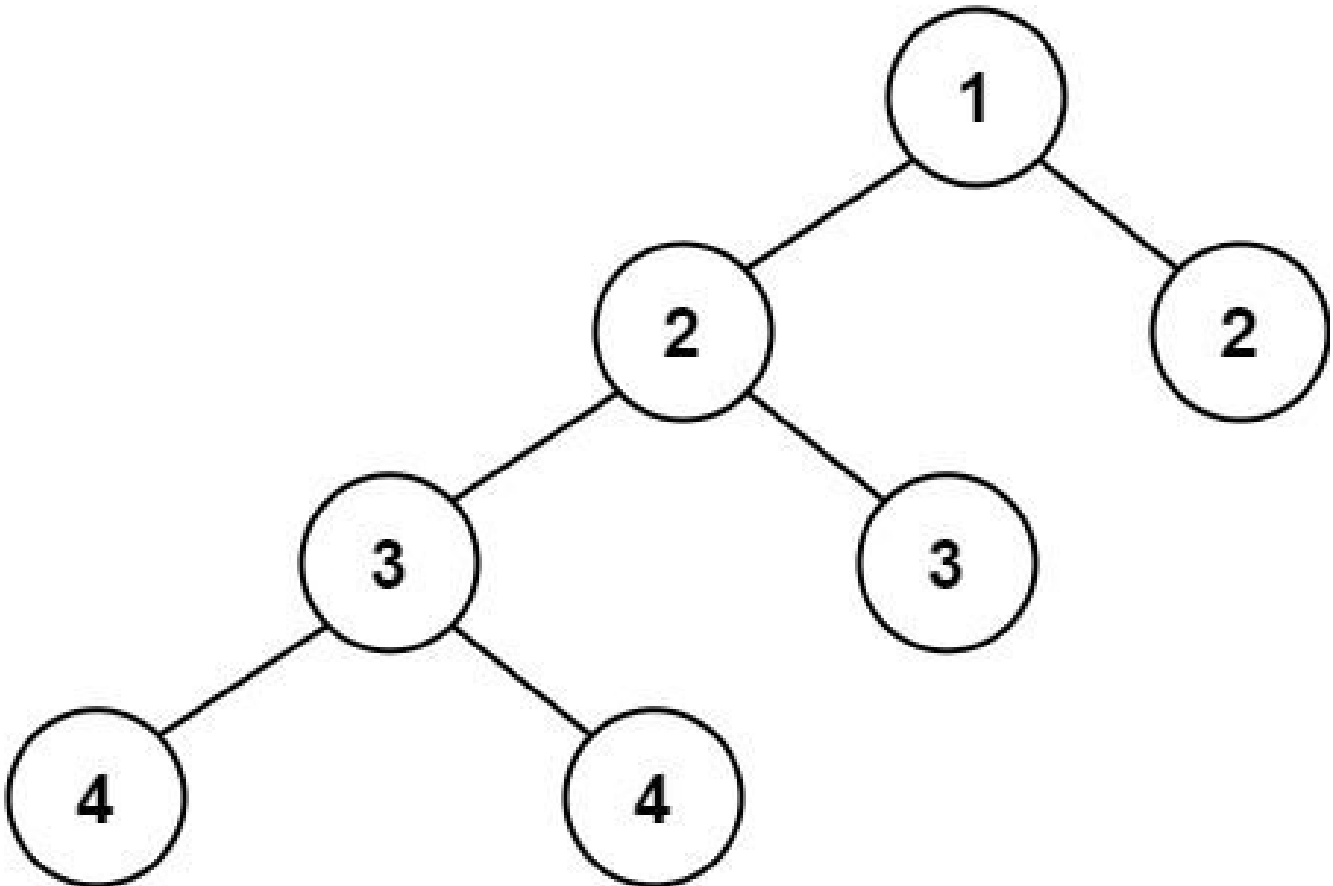
Given a binary tree, determine if it is height-balanced.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]
Output: false

Example 3:

Input: root = []
Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-104 \leq \text{Node.val} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A tree is height-balanced if every node's left and right subtree heights differ by at most 1. Empty tree is balanced. Right?"

#

"[3,9,20,null,null,15,7]: heights of 9's subtrees: 0 vs 0 ✓. 20's: 1 vs 1 ✓. 3's: 1 vs 2 ✓ → true.

[1,2,2,3,3,null,null,4,4]: left subtree of root has height 3, right has 1 → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

At each node compute left subtree height and right subtree height separately.

Node is balanced if $|\text{left-right}| \leq 1$ and both subtrees are balanced recursively.

Recomputes heights from scratch at every node – $O(n^2)$ on skewed trees.

Time: $O(n^2)$ worst case. Space: $O(h)$ stack.

Should I go ahead and optimize?"

```
class _110_BalancedTree_Brute:
```

```
    def isBalanced(self, root: Optional['TreeNode']) -> bool:
```

```
        def height(node):
```

```
            if not node:
```

```
                return 0
```

```
            return max(height(node.left), height(node.right)) + 1
```

```
        def balanced(node):
```

```
            if not node:
```

```
                return True
```

```
            lh, rh = height(node.left), height(node.right)
```

```
            return abs(lh - rh) <= 1 and balanced(node.left) and balanced(node.right)
```

```
        return balanced(root)
```

PHASE 3 — OPTIMIZE

```
# "The brute force calls height()  
repeatedly for the same nodes –  $O(n^2)$ .  
# I notice we're recomputing heights we've  
already seen. What if we cache it?  
#  
# Better: combine the height computation  
and balance check into one DFS pass.  
# Return -1 to signal 'unbalanced subtree  
found' – propagate it up immediately.  
# Otherwise return the actual height.  
# A single  $O(n)$  pass handles everything."
```

PHASE 4 — CLEAN CODE

```
class _110_BalancedTree:  
    def isBalanced(self, root:  
Optional['TreeNode']) -> bool:  
        def check(node):  
            if not node:  
                return 0  
            left = check(node.left)  
            right = check(node.right)  
            if left == -1 or right == -1  
or abs(left - right) > 1:  
                return -1 # propagate  
'unbalanced' signal  
            return max(left, right) + 1
```

return check(root) != -1

PHASE 5 — TEST & DEBUG

```
# root=[3,9,20,null,null,15,7]:  
# check(9)=1. check(15)=1. check(7)=1.  
check(20)=max(1,1)+1=2.  
# check(3): left=1,right=2. |1-2|=1 <=1 →  
return 2. result≠-1 → True ✓  
# root=[1,2,2,3,3,null,null,4,4]:  
# check(4)=1. check(3,left)=max(1,0)+1=2.  
check(3,right)=max(1,0)+1=2.  
# check(2,left): left=2,right=2 → 3.  
check(2,right): left=0,right=0 → 1.  
# check(1): left=3,right=1. |3-1|=2>1 →  
return -1. result=-1 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – single DFS, each node visited once.

Space $O(h)$ for the call stack.

The -1 sentinel trick is the key
interview insight: instead of separate
height and balance checks, fuse them
into one return value with a special
signal.

**# Given more time I'd ask: can we
rebalance a tree in-place?
Flatten to sorted array (in-order
traversal) then rebuild – $O(n)$ time."**

Trees & BST

□ #112 Path Sum

EAS

[Open on LeetCode](#)

Tree · DFS · BFS

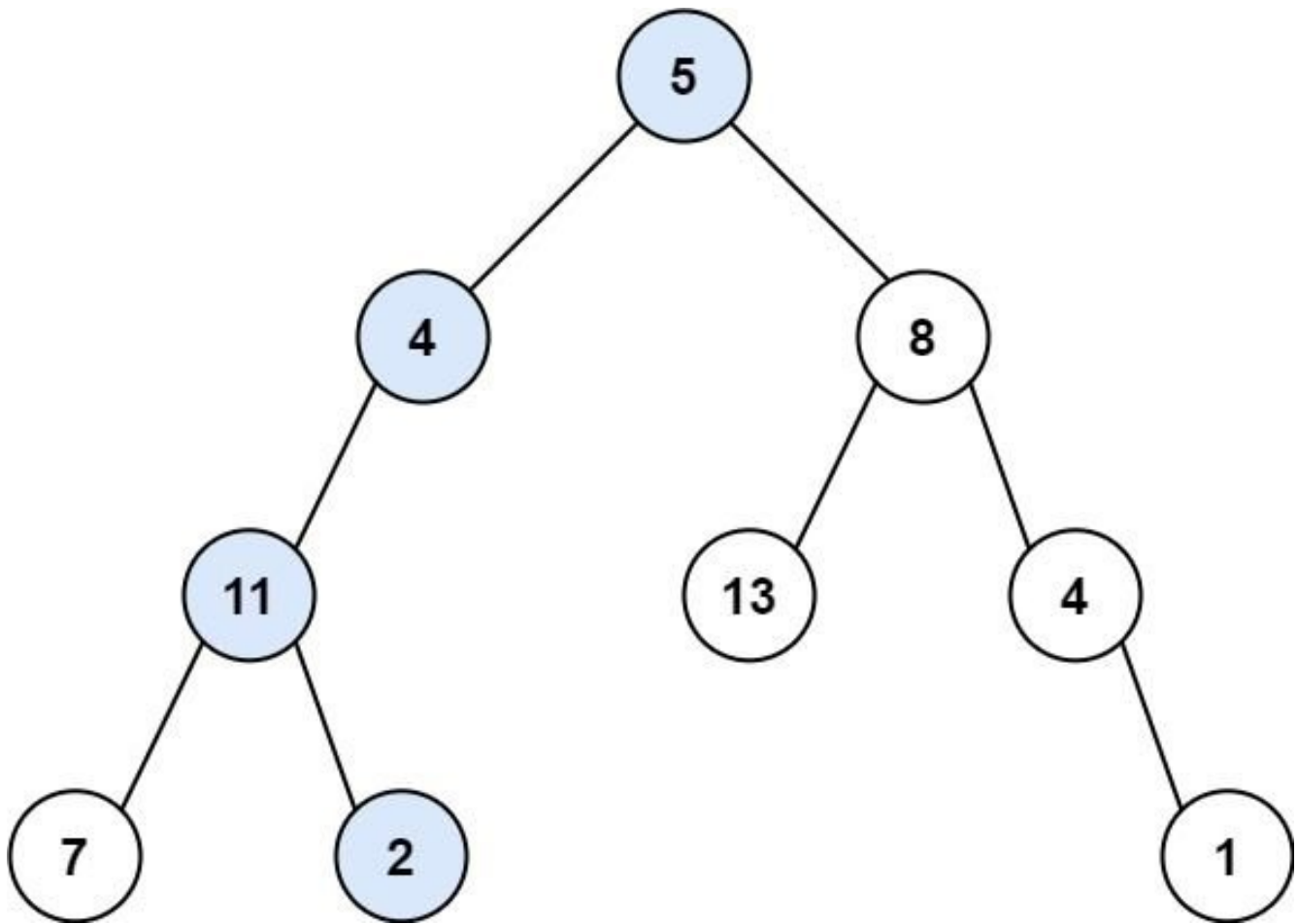
Lists: Denny Zhang

Problem

Given the root of a binary tree and an integer `targetSum`, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals `targetSum`.

A leaf is a node with no children.

Example 1:

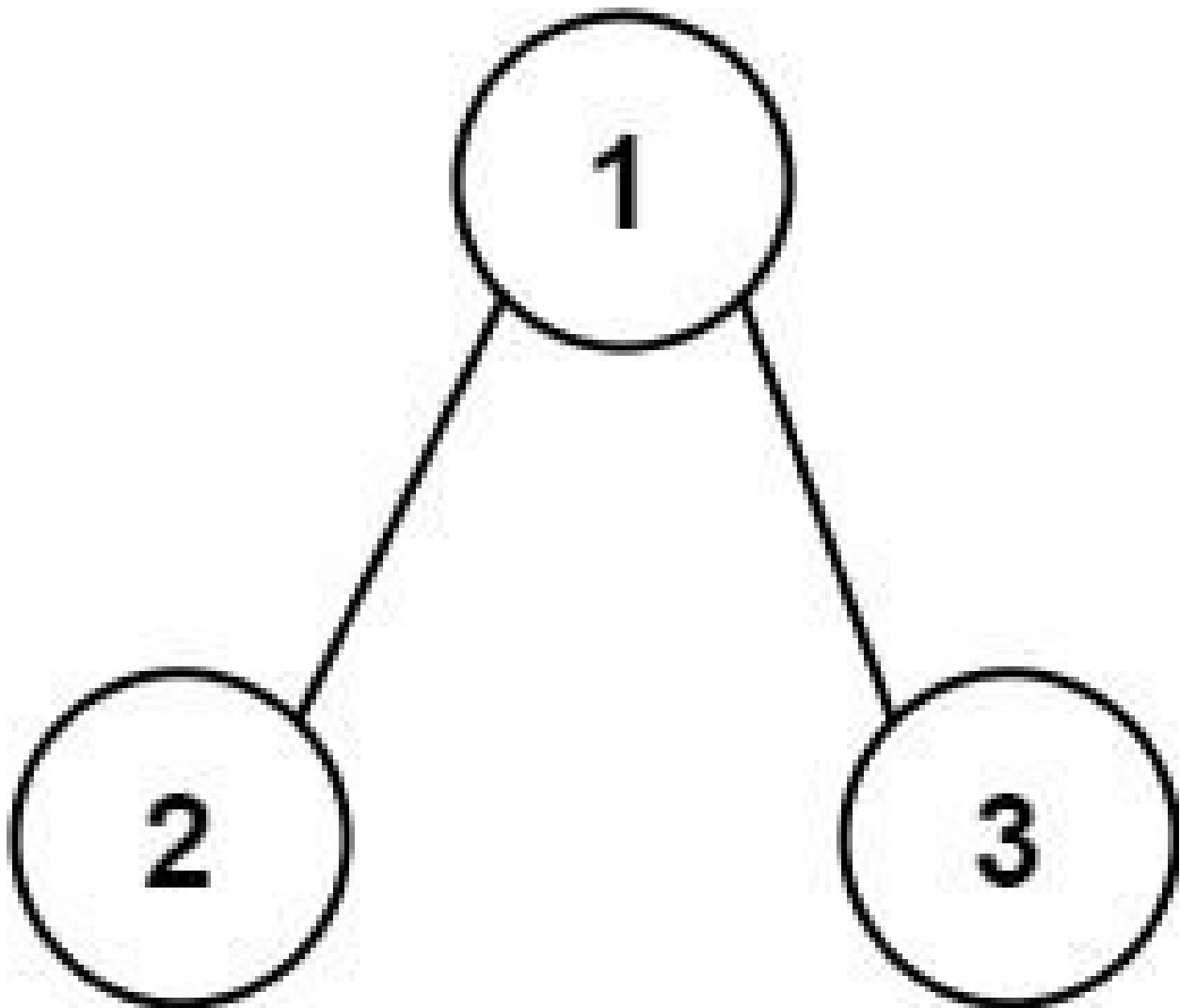


Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

Output: true

Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



Input: root = [1,2,3], targetSum = 5

Output: false

Explanation: There are two root-to-leaf paths in the tree:

(1 --> 2): The sum is 3.

(1 --> 3): The sum is 4.

There is no root-to-leaf path with sum = 5.

Example 3:

Input: root = [], targetSum = 0

Output: false

Explanation: Since the tree is empty, there are no root-to-leaf paths.

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find if any root-to-leaf path sums to targetSum."

A leaf is a node with no children – the path must end at a leaf. Right?

Empty tree → false (no leaves)."

#

"[5,4,8,11,null,13,4,7,2,null,null,null,1], target=22:

path 5→4→11→2 = 22 → true ✓

[], target=0 → false (empty tree has no root-to-leaf path) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

DFS traversal collecting a running sum as we descend. At each leaf node,

check if the accumulated sum equals targetSum. Backtrack and try other paths.

The optimal approach is the same DFS – just written more elegantly by subtracting from target instead of accumulating.

Both are $O(n)$ time and $O(h)$ space, so we skip

a separate brute class and go straight to the clean version."

PHASE 3 — OPTIMIZE

"More elegant: subtract the current node's value from targetSum as we descend.

At a leaf, check if remaining == 0.

Avoids passing a separate running sum."

PHASE 4 — CLEAN CODE

```
class _112_PathSum:
```

```
    def hasPathSum(self, root: Optional['TreeNode'], targetSum: int) -> bool:
```

```
def hassum(node, remaining):
    if not node:
        return False
    remaining -= node.val
    if not node.left and not
node.right: # leaf
        return remaining == 0
    return hassum(node.left,
remaining) or hassum(node.right,
remaining)
return hassum(root, targetSum)
```

PHASE 5 — TEST & DEBUG

```
# root=[5,4,8,...], target=22:
# hassum(5,22): rem=17. hassum(4,17):
rem=13. hassum(11,13): rem=2.
# hassum(7,2): rem=-5, leaf → -5≠0 False.
# hassum(2,2): rem=0, leaf → True ✓
# root=[], target=0: hassum(None,0)=False
✓
# root=[1,2], target=1: hassum(1,1):
rem=0, not leaf.
# hassum(2,0): rem=-2, leaf → False. →
False ✓ (path must reach a leaf)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$ call stack.
The 'leaf check' (no left and no right) is critical – without it, a node
with one child that hits 0 would incorrectly return true.
Given more time I'd ask: return all root-to-leaf paths that sum to target?
That's Path Sum II (#113) – collect paths into a result list with backtracking."

Trees & BST

□ #226 Invert Binary Tree

EAS

[Open on LeetCode](#)

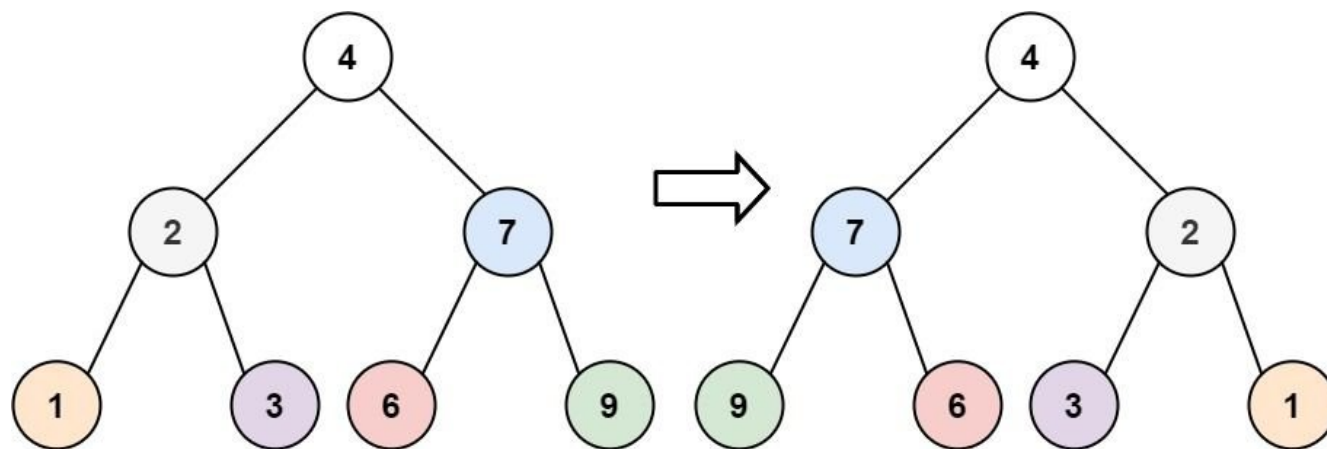
Tree · DFS · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, invert the tree, and return its root.

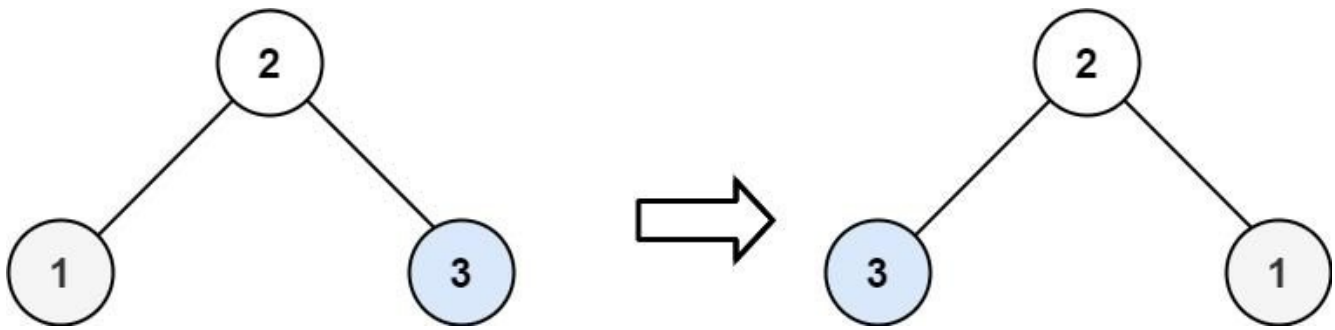
Example 1:



Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]

Output: [2,3,1]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Invert the tree: swap left and right children at every node recursively.

Return the root. Empty tree → return None. Right?"

#

"[4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]:
left and right subtrees are swapped at
every level ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to
lock in the logic.

Iterative BFS with a queue: visit each
node level by level and swap its left

and right children in place. Process
until the queue is empty.

Time: $O(n)$. Space: $O(n)$ for the queue.
Should I go ahead and optimize?"

```
class _226_InvertTree_Brute:
```

```
    def invertTree(self, root:
```

```
Optional['TreeNode']) ->
```

```
Optional['TreeNode']:
```

```
    if not root:
```

```
        return None
```

```
    from collections import deque
```

```
    queue = deque([root])
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        node.left, node.right =
```

```
node.right, node.left
```

```
        if node.left:
```



```
        queue.append(node.left)
    if node.right:
        queue.append(node.right)
    return root
```

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner. Invert left, invert right, then swap.

Or: swap first then recurse – same result. Either order works.

Use inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

```
class _226_InvertTree:
    def invertTree(self, root:
Optional['TreeNode']) ->
Optional['TreeNode']:
        def invert(node):
            if not node:
                return None
            left = invert(node.left)
            right = invert(node.right)
            node.left, node.right =
right, left
            return node
        return invert(root)
```

PHASE 5 — TEST & DEBUG

```
# root=[4,2,7]:  
# invert(2): invert(None)=None,  
invert(None)=None. swap:  
2.left=None,2.right=None. return 2.  
# invert(7): same → 7.  
# invert(4): left=invert(2)=2,  
right=invert(7)=7. swap: 4.left=7,  
4.right=2. return 4.  
# → [4,7,2] ✓ (recurse deeper for full  
tree)  
# root=None: invert(None)=None ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$  call stack.  
# This is the problem Homebrew creator Max  
Howell famously couldn't solve in a Google  
interview.  
# Worth mentioning – it humanises the  
problem and shows you know your history.  
# Given more time I'd ask: can we do this  
iteratively? Yes – BFS or DFS with a  
stack,  
# swap children at each node.  $O(n)$  time,  
 $O(n)$  space."
```

Trees & BST

□ ★ #270 Closest Binary Search Tree Value

EAS

★

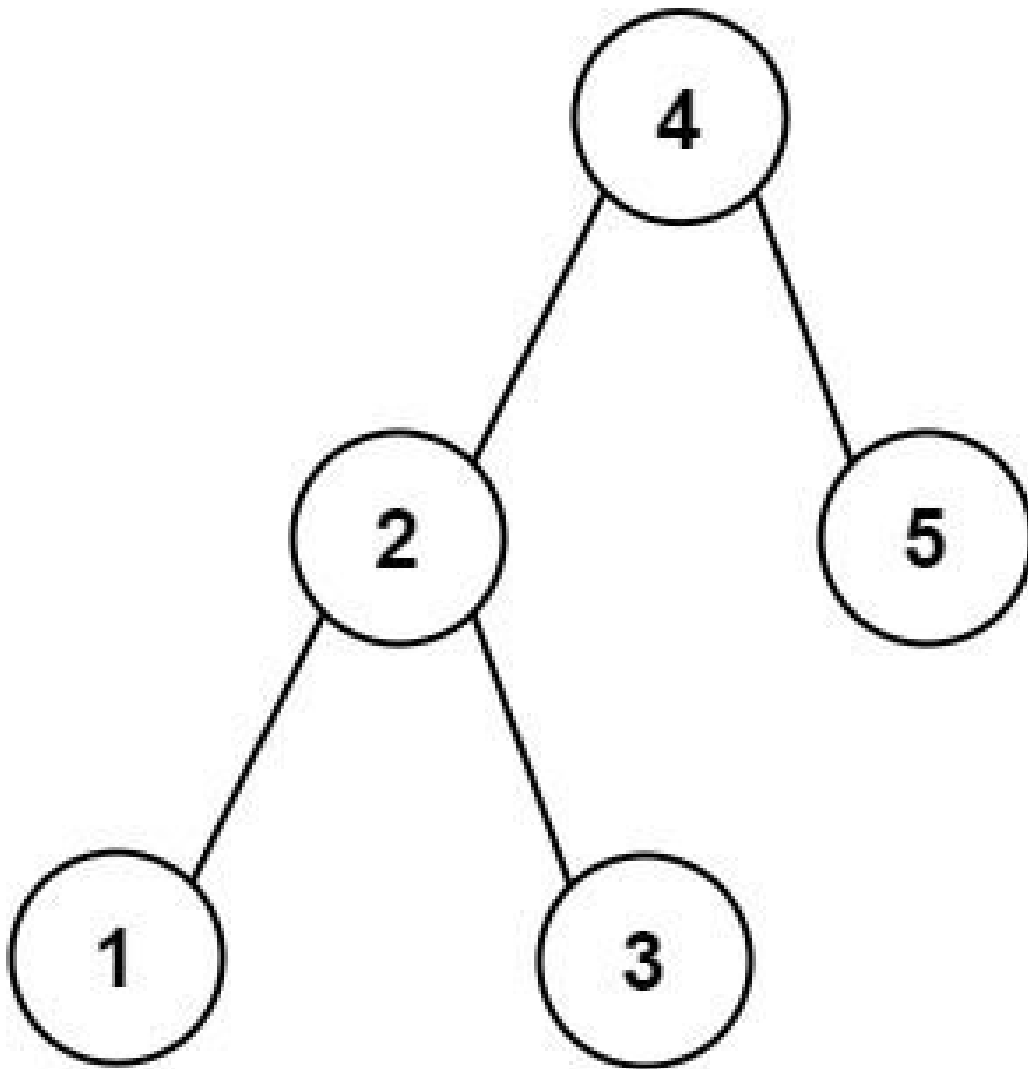
[Open on LeetCode](#)

Binary Search · Tree · DFS · BST**Lists: ★ Premium | Premium 98**

Problem

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286

Output: 4

Example 2:

Input: root = [1], target = 4.428571

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $0 \leq \text{Node.val} \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the node value in a BST closest to a float target.

If two values are equidistant, return the smaller one. Right?

BST property means we can navigate toward the target rather than scanning all nodes."

#

"root=[4,2,5,1,3], target=3.714286:

candidates: 4 (dist=0.286), 3 (dist=0.714). 4 is closer → 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Full DFS scan: visit every node, track the value with minimum $|\text{val} - \text{target}|$.

```
# On a tie (equal distance), prefer the  
smaller value.  
# This ignores the BST property entirely –  
valid but  $O(n)$  time and  $O(n)$  stack space.  
# Should I go ahead and optimize?"
```

```
class _270_ClosestBST_Brute:  
    def closestValue(self, root:  
Optional['TreeNode'], target: float) ->  
int:  
        best = [root.val]  
        def dfs(node):  
            if not node:  
                return  
            if (abs(node.val - target) <  
abs(best[0] - target) or  
                (abs(node.val -  
target) == abs(best[0] - target) and  
node.val < best[0])):  
                best[0] = node.val  
            dfs(node.left)  
            dfs(node.right)  
        dfs(root)  
        return best[0]
```

PHASE 3 — OPTIMIZE

```
# "Use BST property to navigate: at each
node, the closest value is either the
# current node or something in the subtree
direction of the target.
# If target < node.val → go left; if
target > node.val → go right.
# Track the closest seen so far. O(h) time
– O(log n) balanced, O(n) skewed."
```

PHASE 4 — CLEAN CODE

```
class _270_ClosestBST:
    def closestValue(self, root:
Optional['TreeNode'], target: float) ->
int:
        def close(node, best):
            if not node:
                return best
            if abs(target - node.val) <
abs(target - best):
                best = node.val
            elif abs(target - node.val)
== abs(target - best):
                best = min(best,
node.val) # tie → pick smaller
            if target < node.val:
```

```
        return close(node.left,
best)
    else:
        return close(node.right,
best)
    return close(root, root.val)
```

PHASE 5 — TEST & DEBUG

```
# root=[4,2,5,1,3], target=3.714286:
# close(4, 4): |3.714-4|=0.286 <
|3.714-4|=0.286 → no update. target<4 → go
left.
# close(2, 4): |3.714-2|=1.714 > 0.286 →
best stays 4. target>2 → go right.
# close(3, 4): |3.714-3|=0.714 > 0.286 →
best stays 4. target>3 → go right.
# close(None, 4): return 4 ✓
# root=[1], target=4.428: close(1,1):
target>1 → go right → close(None,1) → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(h). Space O(h) for recursion
stack. Iterative: O(1) space.
# BST navigation is the key – we prune
entire subtrees we know can't contain
# a closer value.
```


Given more time I'd ask: find the k closest values to target?
That's Closest BST Value II (LeetCode #272) – use an in-order traversal with a sliding window
or a max-heap of size k."

Trees & BST

□ #501 Find Mode in Binary Search Tree

EAS

[Open on LeetCode](#)

Tree · DFS · BST

Lists: Denny Zhang

Problem

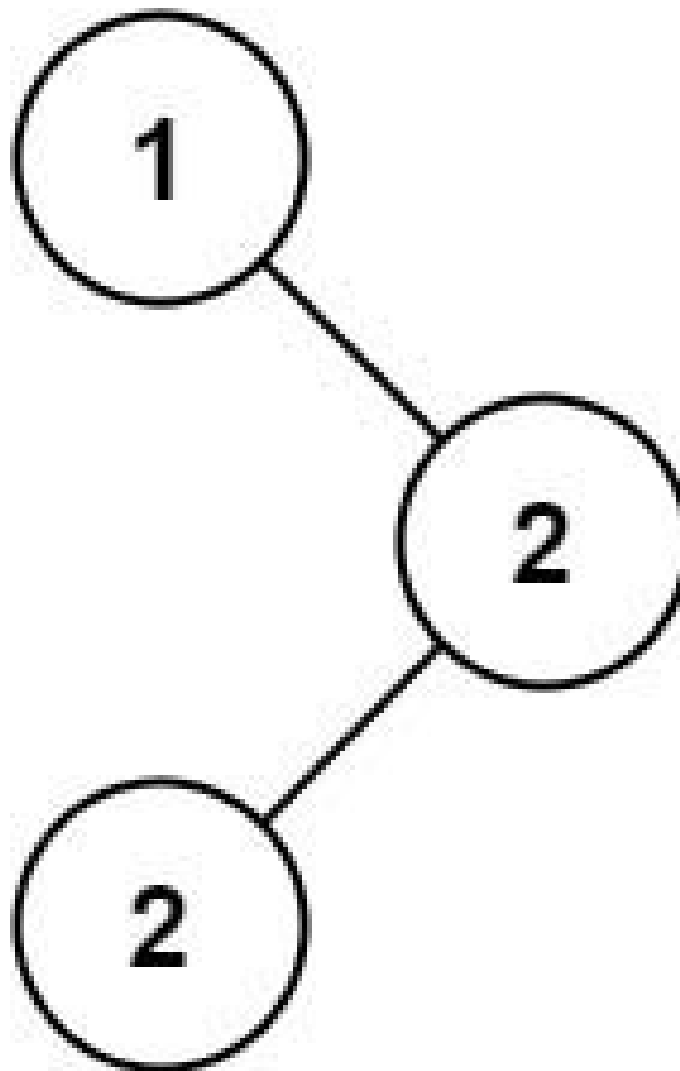
Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: `root = [1,null,2,2]`

Output: `[2]`

Example 2:

Input: `root = [0]`

Output: `[0]`

Constraints:

- The number of nodes in the tree is in the range `[1, 104]`.

- $-105 \leq \text{Node.val} \leq 105$

Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all values that appear the most frequently in the BST.

This BST allows duplicates ($\text{left} \leq \text{node}$, $\text{right} \geq \text{node}$).

Return all modes – there can be multiple. Any order is fine. Right?

Follow-up: can we do it without extra space ($O(1)$ beyond recursion stack)?"

#

"[1,null,2,2]: 2 appears twice, 1 once → mode is [2] ✓

[0]: only element → mode is [0] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

DFS or BFS over all nodes and count frequencies with a hash map (Counter).

Then collect all keys whose count equals the maximum frequency.

Time: $O(n)$. Space: $O(n)$ for the Counter.
Should I go ahead and optimize?"

```
class _501_FindMode_Brute:
    def findMode(self, root:
Optional['TreeNode']) -> List[int]:
        freq: Counter = Counter()
        stack = [root]
        while stack:
            node = stack.pop()
            if node:
                freq[node.val] += 1
                stack.append(node.left)
                stack.append(node.right)
        if not freq:
            return []
        max_freq = max(freq.values())
        return [k for k, v in freq.items()
if v == max_freq]
```

PHASE 3 — OPTIMIZE ($O(1)$ space using
in-order traversal)

"BST in-order traversal visits nodes in
sorted order."

```
# Duplicates in a BST are adjacent in
in-order – so we can count runs without a
map.
# Track: current value, current run count,
max count seen, result list.
# When current run count exceeds max,
reset result. When equal, append.
# Use nonlocal to mutate state inside the
inner helper."
```

PHASE 4 — CLEAN CODE

```
class _501_FindMode:
    def findMode(self, root:
Optional['TreeNode']) -> List[int]:
        modes: List[int] = []
        cur_val = [None]
        cur_cnt = [0]
        max_cnt = [0]
        def inorder(node):
            if not node:
                return
            inorder(node.left)
            if node.val == cur_val[0]:
                cur_cnt[0] += 1
            else:
                cur_val[0] = node.val
```

```

        cur_cnt[0] = 1
    if cur_cnt[0] > max_cnt[0]:
        max_cnt[0] = cur_cnt[0]
        modes.clear()
        modes.append(node.val)
    elif cur_cnt[0] ==
max_cnt[0]:
        modes.append(node.val)
    inorder(node.right)
inorder(root)
return modes

```

PHASE 5 — TEST & DEBUG

```

# root=[1,null,2,2] – in-order: 1, 2, 2
# node=1: cur_val=1,cnt=1,max=1 →
modes=[1].
# node=2: cur_val=2,cnt=1,max=1 →
cnt==max → modes=[1,2].
# node=2: cur_val=2,cnt=2,max=1 → cnt>max
→ max=2,modes=[2].
# return [2] ✓
# root=[0]: in-order: 0. modes=[0] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Brute force: $O(n)$ time, $O(n)$ space.

$O(1)$ space version: $O(n)$ time, $O(1)$ extra space (modes list not counted).
The in-order run-counting trick works because BST duplicates are always adjacent
in sorted traversal – a key property of this BST variant.
Given more time I'd ask: what if it were a regular BST (no duplicates)?
The mode would always be the single element – trivially the root if $n=1$."

Trees & BST

□ #543 Diameter of Binary Tree

EAS

[Open on LeetCode](#)

Tree · DFS

Lists: Grind 169

Problem

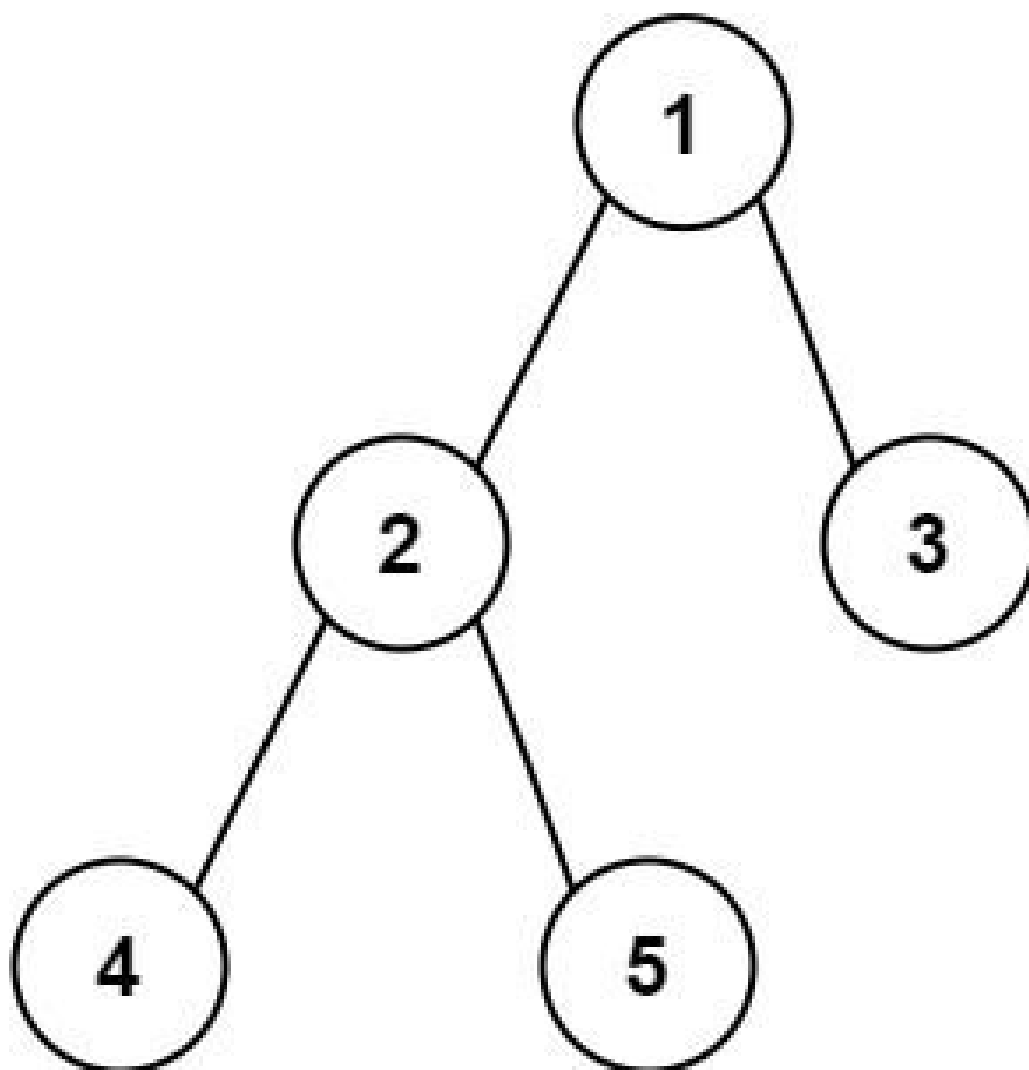
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree.

This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[1, 104]$.
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = the longest path between any two nodes, measured in edges.

The path may or may not pass through the root.

An empty tree has diameter 0. A single node has diameter 0 (no edges). Right?"

#

"[1,2,3,4,5]: longest path is 4→2→1→3, 3 edges → 3 ✓

[1,2]: one edge → 1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For every node, compute the height of its left and right subtrees recursively.

Diameter through that node = left_height + right_height; track the global max.

Recomputes heights repeatedly – correct but redundant work.

Time: $O(n^2)$ on skewed trees. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"I notice depth is recomputed repeatedly for the same nodes – that's the $O(n^2)$.

Fuse depth computation and diameter tracking into one DFS pass.

Use a nonlocal variable (or single-element list) to capture the running max.

depth(node) returns height AND updates the max diameter seen so far.

Time: $O(n)$ – one pass. Space: $O(h)$."

PHASE 4 — CLEAN CODE

```
class _543_DiameterBinaryTree:
    def diameterOfBinaryTree(self, root:
Optional['TreeNode']) -> int:
        best = [0]
        def depth(node):
            if not node:
                return 0
```

```
        left = depth(node.left)
        right = depth(node.right)
        best[0] = max(best[0], left +
right) # diameter through this node
        return max(left, right) + 1 #
height for parent
    depth(root)
    return best[0]
```

PHASE 5 — TEST & DEBUG

```
# root=[1,2,3,4,5]:
# depth(4)=1. depth(5)=1. depth(2):
best=max(0,1+1)=2. return 2.
# depth(3)=1. depth(1):
best=max(2,2+1)=3. return 3.
# return best[0]=3 ✓
# root=[1,2]: depth(2)=1. depth(1):
best=max(0,1+0)=1. return best=1 ✓
# root=None: depth(None)=0. best stays 0.
return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h).
# The single-element list best=[0] is a
common Python pattern to mutate
```

```
# a value from inside a nested function
without making it a class attribute.
# Equivalent to 'nonlocal best' with a
plain int, but the list avoids the
# nonlocal keyword – both are valid in an
interview.
# Given more time I'd ask: what's the
longest path by node count (not edges)?
# Just return best[0] + 1 when best > 0.
Or: diameter in edge terms is already
# what we compute – node count would be
edges + 1."
```

Trees & BST

□ #572 Subtree of Another Tree

EAS

[Open on LeetCode](#)

Tree · DFS · String Matching

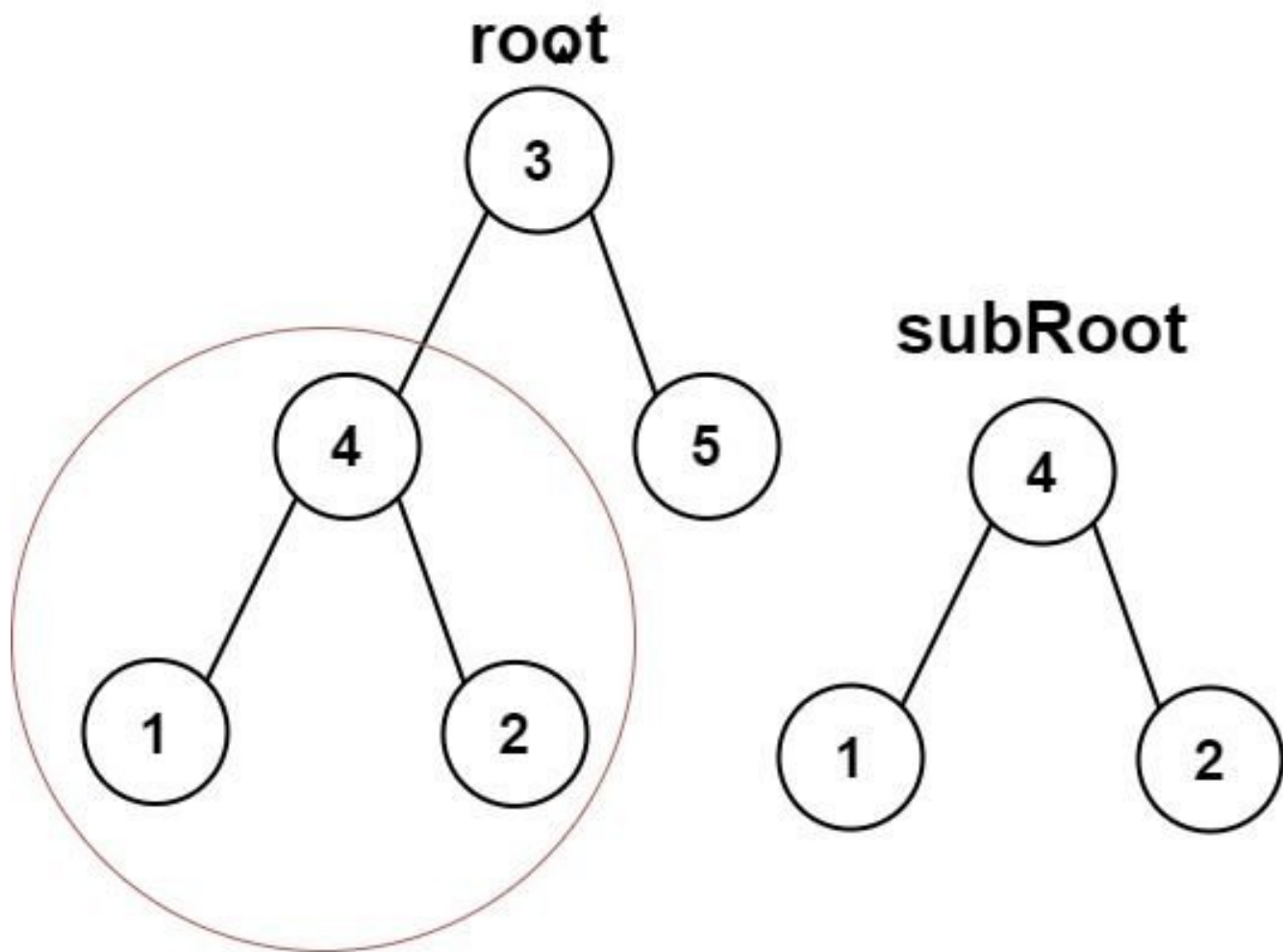
Lists: Grind 169

Problem

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

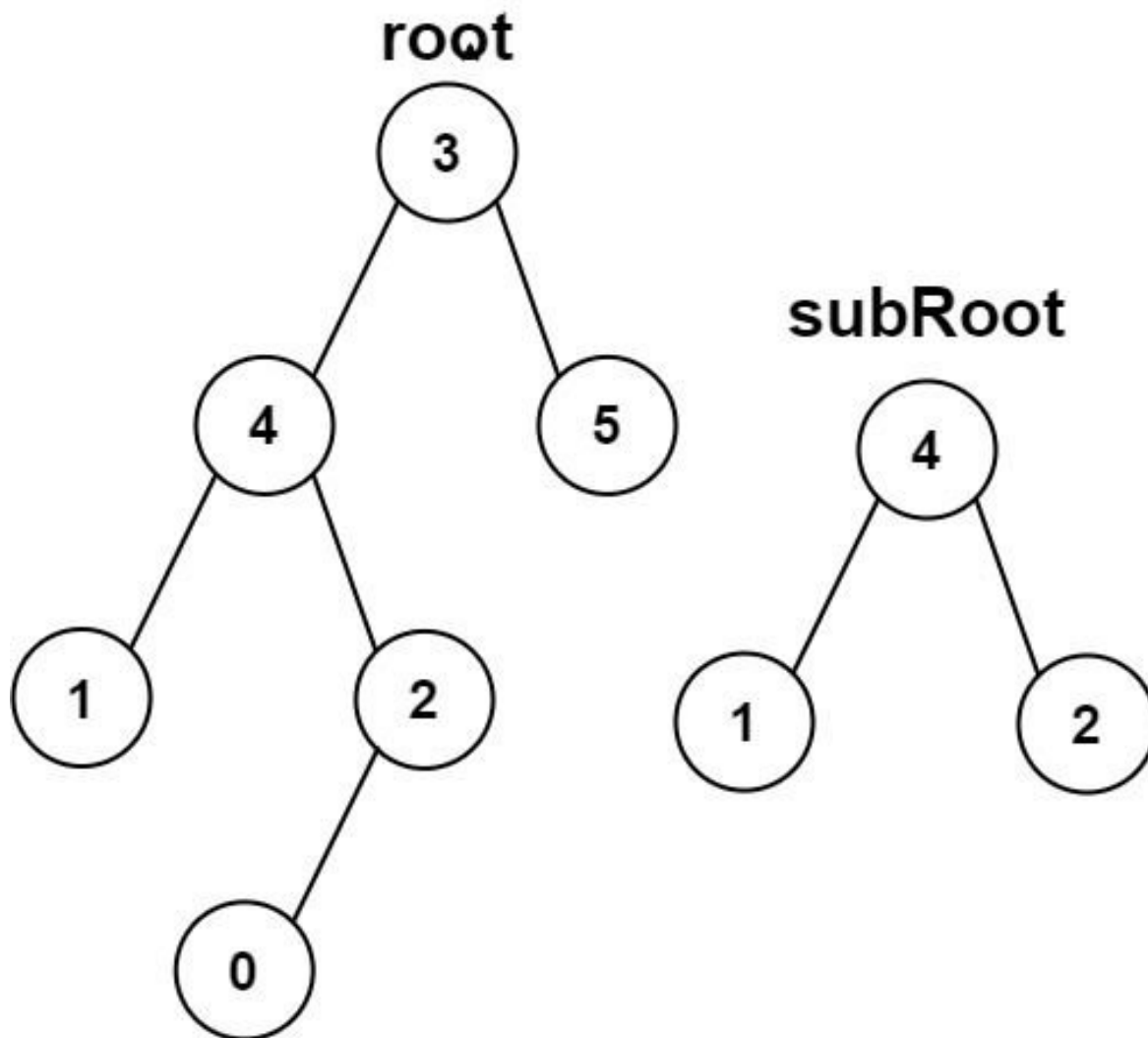
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true

Example 2:



Input: root =
[3,4,5,1,2,null,null,null,null,0],
subRoot = [4,1,2]
Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].

- $-104 \leq \text{root.val} \leq 104$
- $-104 \leq \text{subRoot.val} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return true if subRoot appears as a subtree anywhere in root.

A subtree means a node and ALL its descendants match – not just a portion.

The tree itself is a subtree of itself. Right?"

#

"root=[3,4,5,1,2], subRoot=[4,1,2]:
node 4 in root has left=1,right=2 → matches ✓

root=[3,4,5,1,2,null,null,null,null,0],
subRoot=[4,1,2]:

node 4 in root has extra child 0 → doesn't match ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For every node in root, run the same-tree check against all of subRoot.

```
# If any starting node matches the entire
subtree, return true.
# Correct, but rescans overlapping
structure many times.
# Time:  $O(n * m)$ . Space:  $O(h)$  stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same  $O(m*n)$  is standard. Two clean
inner helpers:
```

```
# same(p,q) – checks if two trees are
identical (reuse Same Tree logic).
# subtree(node, sub) – DFS root; at each
node, try same().
# If same fails, recurse into left and
right subtrees."
```

PHASE 4 — CLEAN CODE

```
class _572_SubtreeAnotherTree:
    def isSubtree(self, root:
Optional['TreeNode'], subRoot:
Optional['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
```

```

        return p.val == q.val and
same(p.left, q.left) and same(p.right,
q.right)

```

```

def subtree(node, sub):
    if not node:
        return False
    if same(node, sub):
        return True
    return subtree(node.left,
sub) or subtree(node.right, sub)
return subtree(root, subRoot)

```

PHASE 5 — TEST & DEBUG

```

# root=[3,4,5,1,2], subRoot=[4,1,2]:
# subtree(3,[4,1,2]): same(3,4)? 3≠4 →
False. recurse left/right.
# subtree(4,[4,1,2]): same(4,4)? 4==4.
# same(1,1)→same(None,None)T,
same(None,None)T→T.
# same(2,2)→T. → same=True → return True ✓
# root has extra 0 child under 4:
# same(4,4): same(1,1)T. same(2(with
0),2)? 2==2 but same(0,None)→False →
same=False.
# subtree(1,[4,1,2]): same(1,4)? False.
subtree deeper → all False → False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$ – at each of m nodes we may run `same()` which takes $O(n)$.

Space $O(h_{\text{root}} + h_{\text{sub}})$ for call stacks.

Optimisation: serialise both trees and use string matching (KMP or Rabin-Karp)

for $O(m+n)$ time – worth mentioning as an advanced follow-up.

Be careful with serialisation: `'[1,2]'` would incorrectly match `'[12]'` without
delimiters like `'#'` for null and `','` between values.

Given more time I'd ask: what if we query many different subRoots against
the same root? Pre-serialise root once and use string search for each query."

DFS

Round 1 · High · Easy · 2 questions

DFS

□ #463 Island Perimeter

EAS

[Open on LeetCode](#)

Array · DFS · BFS · Matrix

Lists: Denny Zhang

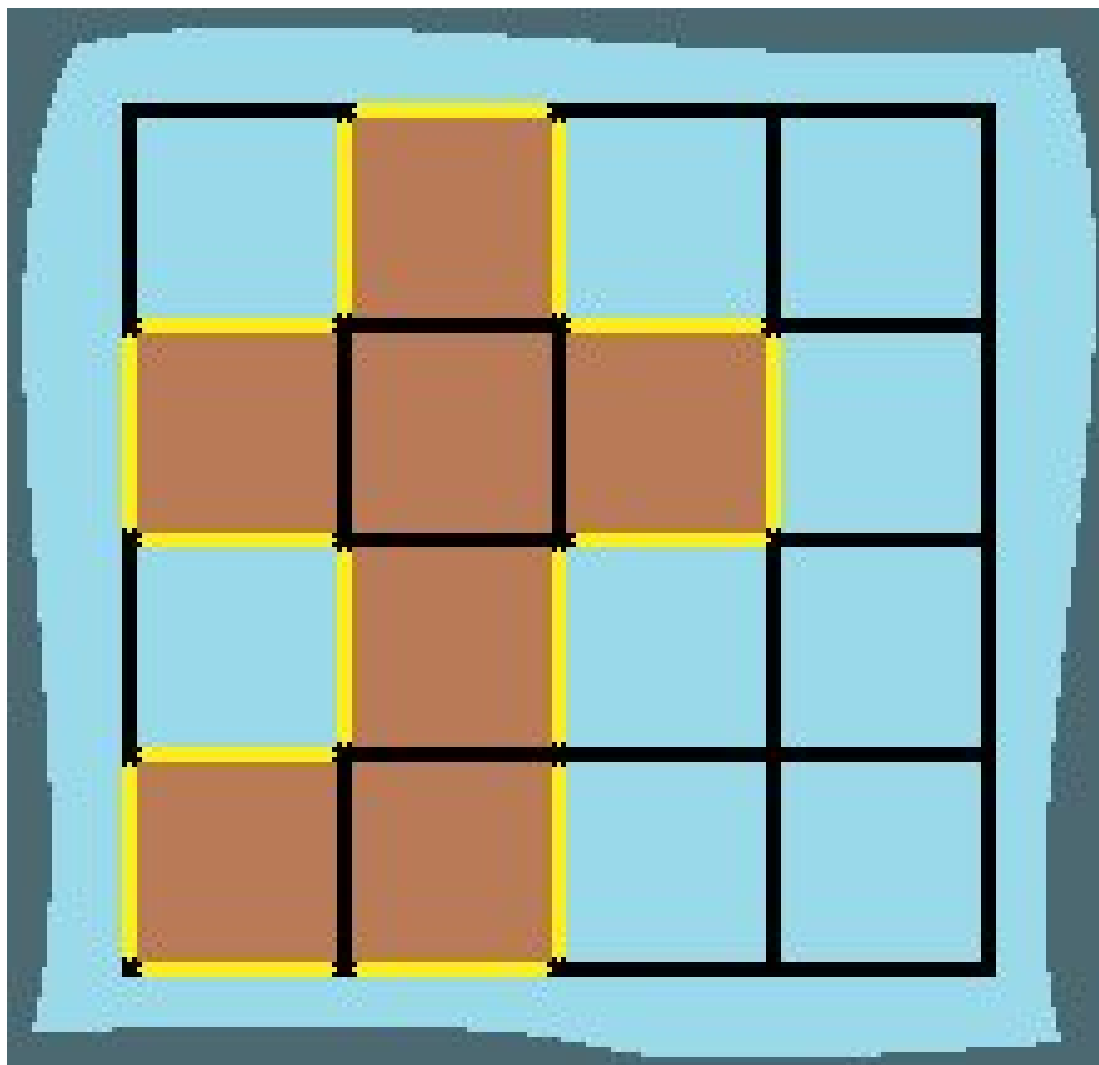
Problem

You are given row x col grid representing a map where $\text{grid}[i][j] = 1$ represents land and $\text{grid}[i][j] = 0$ represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



Input: `grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`

Output: 16

Explanation: The perimeter is the 16 yellow stripes in the image above.

Example 2:

Input: `grid = [[1]]`

Output: 4

Example 3:

Input: `grid = [[1,0]]`

Output: 4

Constraints:

- `row == grid.length`
- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` is 0 or 1.
- There is exactly one island in grid.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The grid has exactly one island – no lakes. Each land cell is a unit square.

We return the total perimeter, which is the count of exposed edges. Right?

Exposed means the edge faces water or the grid boundary."

#

"Quick check: `[[1]]` → 4 sides, all exposed → 4 ✓. `[[1,0]]` → 4 sides, all exposed → 4 ✓.

Two adjacent land cells: each has 4 sides but share 1 edge $\rightarrow 2 \times 4 - 2 = 6$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every land cell, I check all 4 neighbours – if a neighbour is out of bounds or water,

that edge contributes 1 to the perimeter. Sum these up across all land cells.

Time: $O(m \times n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _463_IslandPerimeter_Brute:
    def islandPerimeter(self, grid:
List[List[int]]) -> int:
        rows, cols = len(grid),
len(grid[0])
        perimeter = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
```

```
        nr, nc = r + dr, c
    + dc
        if nr < 0 or nr >=
rows or nc < 0 or nc >= cols or
grid[nr][nc] == 0:
        perimeter +=
1
    return perimeter
```

PHASE 3 — OPTIMIZE

"I notice we're checking 4 neighbours per cell. Instead of neighbour checks,

we can use a counting formula:

Every land cell contributes 4 sides.

Every shared edge between two adjacent

land cells removes 2 sides (one from each). So:

perimeter = islands * 4 - shared_edges * 2

We only count each shared edge once by checking right-neighbour and down-neighbour.

Same $O(m*n)$ time, same $O(1)$ space – but a cleaner mental model."

PHASE 4 — CLEAN CODE

```
class _463_IslandPerimeter:
    def islandPerimeter(self, grid:
List[List[int]]) -> int:
        rows, cols = len(grid),
len(grid[0])
        islands = 0
        shared = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    islands += 1
                    if r + 1 < rows and
grid[r + 1][c] == 1:
                        shared += 1
                    if c + 1 < cols and
grid[r][c + 1] == 1:
                        shared += 1
        return islands * 4 - shared * 2

# PHASE 5 — TEST & DEBUG
# grid=[[1]]: islands=1, shared=0. return
4 ✓
# grid=[[1,1]]: islands=2. c=0→c+1=1
valid,grid[0][1]=1→shared=1. return
2*4-1*2=6 ✓
```

```
# grid=[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]:
```

```
# Count land cells=8 (islands). Count shared edges (right+down only)=... =8.
```

```
# return  $8*4-8*2=32-16=16$  ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time  $O(m*n)$ . Space  $O(1)$ .
```

```
# The counting formula is elegant – worth presenting even if the brute force also works.
```

```
# Both approaches are the same complexity; the formula shows mathematical insight.
```

```
# Given more time I'd ask: what if there are multiple islands?
```

```
# The formula still works per-island if we isolate them, or we can return a list
```

```
# of perimeters – same logic applied per connected component."
```

DFS

□ #733 Flood Fill

EAS[Open on LeetCode](#)

Array · DFS · BFS · Matrix

Lists: Grind 169

Problem

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a flood fill on the image starting from the pixel `image[sr][sc]`.

To perform a flood fill:

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
3. Keep repeating this process by checking neighboring pixels of the updated pixels and

modifying their color if it matches the original color of the starting pixel.

4. The process stops when there are no more adjacent pixels of the original color to update.

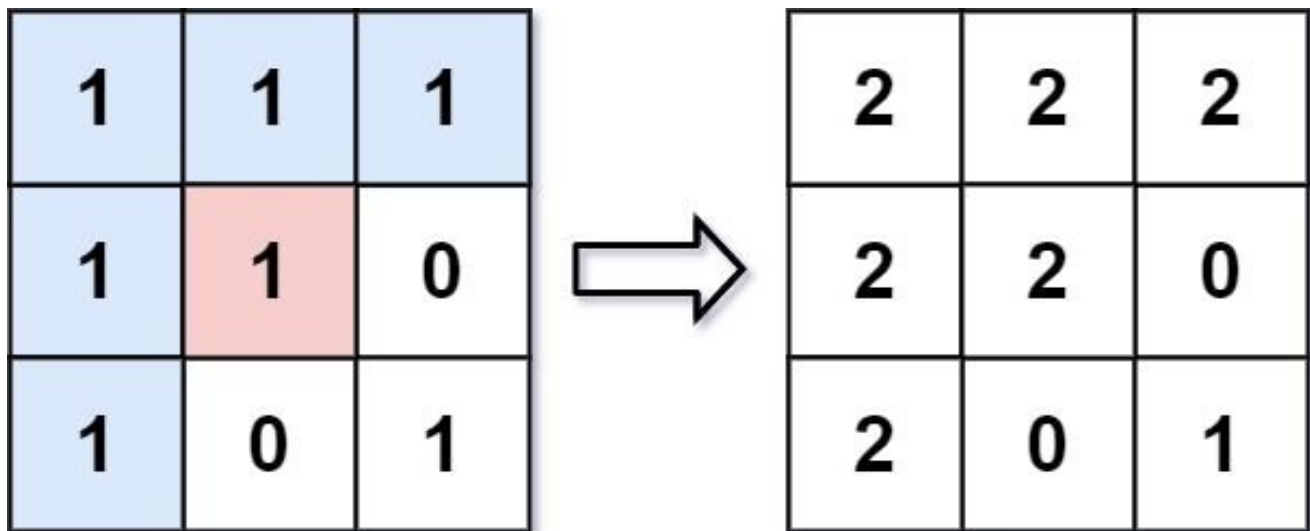
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- $m == \text{image.length}$
- $n == \text{image}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $0 \leq \text{image}[i][j], \text{color} < 216$
- $0 \leq \text{sr} < m$
- $0 \leq \text{sc} < n$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Starting from (sr, sc), change all connected cells of the same colour to the

new colour.

Connected means 4-directionally (up/down/left/right). Modify and return the image. Right?

Edge case: if the starting colour already equals the new colour – return immediately,

otherwise we'd loop infinitely or process unchanged cells."

#

"[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2:

All 1s connected to (1,1) become 2. The bottom-right 1 is isolated → stays 1. ✓

[[0,0,0],[0,0,0]], sr=0,sc=0, color=0: startcolor==color → no change → return as-is ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

I'll use BFS: enqueue the start cell, paint it the new colour, then for each cell I dequeue,

enqueue all 4-directional neighbours that still have the original start colour.

This visits every connected cell exactly once.

Time: $O(m*n)$. Space: $O(m*n)$ for the queue. Should I go ahead and optimize?"

```
class _733_FloodFill_Brute:
    def floodFill(self, image, sr, sc,
color):
        from collections import deque
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image),
len(image[0])
        queue = deque([(sr, sc)])
        image[sr][sc] = color
        while queue:
            r, c = queue.popleft()
            for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <=
nc < cols and image[nr][nc] == start:
                    image[nr][nc] = color
                    queue.append((nr,
nc))
```

```
    return image
```

PHASE 3 — OPTIMIZE

```
# "DFS is cleaner. Key insight: once we  
paint a cell, its colour changes to  
`color` —
```

```
# it can never match `startcolor` again.  
So we don't need a `seen` set at all.
```

```
# The colour change itself is the visited  
marker.
```

```
# This saves  $O(m \times n)$  extra space."
```

PHASE 4 — CLEAN CODE

```
class _733_FloodFill:
```

```
    def floodFill(self, image:
```

```
List[List[int]], sr: int, sc: int, color:  
int) -> List[List[int]]:
```

```
    start = image[sr][sc]
```

```
    if start == color:
```

```
        return image
```

```
    rows, cols = len(image),  
len(image[0])
```

```
    def dfs(r, c):
```

```
        if r < 0 or r >= rows or c < 0  
or c >= cols:
```

```
            return
```

```

        if image[r][c] != start:
            return
        image[r][c] = color # paint
before recursing – acts as visited marker
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)
    dfs(sr, sc)
    return image

```

PHASE 5 — TEST & DEBUG

```

# image=[[1,1,1],[1,1,0],[1,0,1]],
sr=1,sc=1, color=2, start=1:
# dfs(1,1): image[1][1]=2. recurse 4 dirs.
# dfs(2,1): image[2][1]=1==start → paint
2. recurse: dfs(3,1)→00B, dfs(1,1)→now
2≠1 stop.
# dfs(2,2)→image[2][2]=1 but that's the
isolated corner – wait:
# (2,1)→image[2][1]=1✓→paint. Then
dfs(2,2)→image[2][2]=1✓→paint? No –
# is (2,2) reachable from (1,1)? Path:
(1,1)→(2,1)→(2,2).
# image[2][2]=1 and image[2][1]=1, so yes
connected. But expected output keeps it 1?

```

Re-check:

image=[[1,1,1],[1,1,0],[1,0,1]]. (2,2) is adjacent to (2,1)=0 and (1,2)=0.

So (2,2) connects to (2,1)=0 (water) – not reachable from island. ✓

All connected 1s become 2. Bottom-right stays 1 (not connected). ✓

#

start==color: image=[[0,0,0],[0,0,0]], color=0 → return immediately ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$ – each cell visited at most once.

Space $O(m*n)$ call stack in the worst case (all cells same colour, grid is a long chain).

The 'paint first then recurse' order is the key – it prevents revisiting and

eliminates the need for a separate visited set. Mention this explicitly.

The early return when startcolor == color prevents infinite looping – critical edge case.

Given more time I'd ask: what if connectivity is 8-directional (diagonals

too)?

Add four more direction pairs to the dfs calls – same algorithm."

Round 2 · High · Medium

Round 2

High Priority · Medium

116 questions · 11 patterns

**Arrays & Hashing #57 #238 #281 #307 #454
#525 #560 #1010 #1272 #1429 #3159**

String #8 #249 #271 #635 #1138

**Two Pointers #11 #15 #16 #31 #75 #161 #167
#186 #189 #532 #923 #986 #1055 #2563**

**Sliding Window #3 #159 #340 #424 #438 #487
#1100 #1248**

Sorting #49 #56 #1152

**Binary Search #33 #34 #153 #300 #362 #528
#852 #981 #1011 #1060 #1062 #1533**

**Matrix #36 #48 #54 #59 #64 #73 #74 #221
#311 #498 #531 #723 #835 #1198**

**Trees & BST #98 #102 #103 #105 #199 #230
#235 #236 #250 #285 #298 #314 #333 #366**

Round 1 · High · Easy

p.339

**#437 #545 #549 #582 #662 #863 #1120 #1490
#1522 #1650**

**DFS #130 #133 #200 #207 #210 #261 #310
#323 #417 #490 #694 #695 #721 #785 #1236
#1242**

Graphs #128 #277 #1136

BFS #286 #322 #542 #994 #1197 #1730

Arrays & Hashing

Round 2 · High · Medium · 11 questions

Arrays & Hashing

□ #57 Insert Interval

MED

[Open on LeetCode](#)

Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ represent the start and the end of the i th interval and intervals is sorted in ascending order by start_i . You are also given an interval $\text{newInterval} = [\text{start}, \text{end}]$ that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by start_i and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Example 1:

```
Input: intervals = [[1,3],[6,9]],  
newInterval = [2,5]  
Output: [[1,5],[6,9]]
```

Example 2:

```
Input: intervals =  
[[1,2],[3,5],[6,7],[8,10],[12,16]],  
newInterval = [4,8]  
Output: [[1,2],[3,10],[12,16]]  
Explanation: Because the new interval  
[4,8] overlaps with [3,5],[6,7],[8,10].
```

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 105$
- intervals is sorted by start_i in ascending order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We have a sorted list of
non-overlapping intervals. Insert a new
interval,
# merging wherever it overlaps existing
ones. Return the result sorted.
# We don't need to modify in-place – a new
array is fine. Right?"
#
# "[[1,3],[6,9]], newInterval=[2,5]:
# [2,5] overlaps [1,3] → merge to [1,5].
# [6,9] doesn't overlap → keep. →
# [[1,5],[6,9]] ✓
# [[1,2],[3,5],[6,7],[8,10],[12,16]],
newInterval=[4,8]:
# Overlaps [3,5],[6,7],[8,10] → merge all
to [3,10]. → [[1,2],[3,10],[12,16]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to
lock in the logic.
# I'll append the new interval to the
list, sort everything by start time,
# then run a standard merge-intervals pass
to collapse all overlaps.
# Time:  $O(n \log n)$  – the sort dominates.
Space:  $O(n)$ . Should I go ahead and
```

optimize?"

```
class _57_InsertInterval_Brute:
    def insert(self, intervals:
List[List[int]], newInterval: List[int])
-> List[List[int]]:
        intervals.append(newInterval)
        intervals.sort()
        result = [intervals[0]]
        for s, e in intervals[1:]:
            if result[-1][1] >= s:
                result[-1][1] =
max(result[-1][1], e)
            else:
                result.append([s, e])
        return result
```

PHASE 3 — OPTIMIZE

"The input is already sorted – we can exploit that to avoid the $O(n \log n)$ sort.

Three phases in one $O(n)$ pass:

1. Add all intervals that end BEFORE newInterval starts (no overlap).

2. Merge all intervals that overlap newInterval into one combined interval.

Overlap condition: $\text{interval}[\text{start}] \leq \text{newInterval}[\text{end}]$.

```
# 3. Add all remaining intervals that
start AFTER the merged interval ends.
#
# Pseudocode:
# i = 0
# while i < n and intervals[i][1] <
new[0]: result.append(intervals[i]); i++
# while i < n and intervals[i][0] <=
new[1]:
# new = [min(new[0], intervals[i][0]),
max(new[1], intervals[i][1])]; i++
# result.append(new)
# result.extend(intervals[i:])
#
# Time: O(n). Space: O(n) for output."

# PHASE 4 — CLEAN CODE
class _57_InsertInterval:
    def insert(self, intervals:
List[List[int]], newInterval: List[int])
-> List[List[int]]:
        result = []
        i, n = 0, len(intervals)
        # Phase 1: all intervals ending
before newInterval starts
```

```
        while i < n and intervals[i][1] <
newInterval[0]:
            result.append(intervals[i])
            i += 1

    # Phase 2: merge all overlapping
intervals into newInterval
        while i < n and intervals[i][0] <=
newInterval[1]:
            newInterval[0] =
min(newInterval[0], intervals[i][0])
            newInterval[1] =
max(newInterval[1], intervals[i][1])
            i += 1
        result.append(newInterval)

    # Phase 3: remaining intervals
start after merged interval
        result.extend(intervals[i:])
    return result
```

PHASE 5 — TEST & DEBUG

```
# intervals=[[1,3],[6,9]], new=[2,5]:
# Phase 1: intervals[0][1]=3 >= new[0]=2 →
stop. i=0.
# Phase 2: intervals[0][0]=1 <= new[1]=5 →
new=[min(2,1),max(5,3)]=[1,5]. i=1.
# intervals[1][0]=6 > new[1]=5 → stop.
```

```
# result=[[1,5]]. extend [[6,9]]. →  
[[1,5],[6,9]] ✓  
#  
# intervals=[[1,2],[3,5],[6,7],[8,10],[12  
,16]], new=[4,8]:  
# Phase 1: [1,2] ends at 2 < 4 → add. i=1.  
# Phase 2: [3,5] start 3<=8→new=[3,8].  
[6,7] start 6<=8→new=[3,8]. [8,10] start  
8<=8→new=[3,10]. i=4.  
# result=[[1,2],[3,10]]. extend  
[[12,16]]. → [[1,2],[3,10],[12,16]] ✓  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n)$  – three phases together visit  
each interval once.  
# Space  $O(n)$  for output.  
# The two overlap conditions are the key:  
# Non-overlap before:  $\text{interval}[\text{end}] < \text{new}[\text{start}]$ .  
# Non-overlap after:  $\text{interval}[\text{start}] > \text{new}[\text{end}]$  (Phase 3 – just extend).  
# Anything in between overlaps and gets  
merged.  
# Given more time I'd ask: what if  
intervals aren't sorted?
```


Sort first ($O(n \log n)$), then apply the same three-phase approach."

Arrays & Hashing

□ #238 Product of Array Except Self

MED

[Open on LeetCode](#)

Array · Prefix Sum

Lists: Grind 169

Problem

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`

Output: `[0,0,9,0,0]`

Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return an array where `answer[i]` is the product of all elements except `nums[i]`."

Cannot use division. Must run in $O(n)$.

Follow-up: $O(1)$ extra space. Right?

Guaranteed no overflow in 32-bit – so no overflow concern."

#

"[1,2,3,4]: `answer[0]=2*3*4=24`."

`answer[1]=1*3*4=12`. `answer[2]=1*2*4=8`."

answer[3]=1*2*3=6 ✓

[-1,1,0,-3,3]: zeros make most products
0 – [0,0,9,0,0] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For each index i, I multiply together all elements in the array except nums[i] itself.

That's a nested loop – $O(n)$ work for each of the n positions.

Time: $O(n^2)$. Space: $O(1)$ extra. Should I go ahead and optimize?"

```
class _238_ProductExceptSelf_Brute:
    def productExceptSelf(self, nums):
        n = len(nums)
        result = []
        for i in range(n):
            product = 1
            for j in range(n):
                if j != i:
                    product *= nums[j]
            result.append(product)
        return result
```

PHASE 3 — OPTIMIZE

"Build prefix products and suffix products, multiply them.

prefix[i] = product of everything to the LEFT of i.

suffix[i] = product of everything to the RIGHT of i.

answer[i] = prefix[i] * suffix[i].

Time: $O(n)$. Space: $O(n)$ for two arrays.

#

$O(1)$ space follow-up: use the output array for prefix products,

then do a single right-to-left pass with a running suffix multiplier."

PHASE 4 — CLEAN CODE ($O(1)$ extra space version)

```
class _238_ProductExceptSelf:
```

```
    def productExceptSelf(self, nums: List[int]) -> List[int]:
```

```
        n = len(nums)
```

```
        result = [1] * n
```

```
        # Left pass: result[i] = product of all elements to the left of i
```

```
        for i in range(1, n):
```

```
        result[i] = result[i - 1] *  
nums[i - 1]  
        # Right pass: multiply in the  
suffix product on the fly  
        suffix = 1  
        for i in range(n - 1, -1, -1):  
            result[i] *= suffix  
            suffix *= nums[i]  
        return result
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3,4]:  
# Left pass: result=[1,1,2,6].  
# Right pass: i=3: result[3]=6*1=6,  
suffix=4.  
# i=2: result[2]=2*4=8, suffix=12.  
# i=1: result[1]=1*12=12, suffix=24.  
# i=0: result[0]=1*24=24, suffix=24.  
# → [24,12,8,6] ✓  
# nums=[0,0]: result=[0,0] ✓ (zeros  
handled correctly by multiplication)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) extra — output  
array doesn't count."
```

The two-pass trick: left products first,
then multiply right products on the fly.
The suffix variable carries the running
right product without an extra array.
Given more time I'd ask: what if we're
allowed division?
total_product / nums[i] – but fails when
nums[i]=0. Handle: count zeros,
if two or more → all zeros. If one zero
→ all zeros except position of zero."

Arrays & Hashing

□ #281 Zigzag Iterator

MED

[Open on LeetCode](#)

Array · Design · Queue · Iterator

Lists: Denny Zhang

Problem

Given two vectors of integers v1 and v2, implement an iterator to return their elements alternately.

Implement the ZigzagIterator class:

- **ZigzagIterator(List<int> v1, List<int> v2)** initializes the object with the two vectors v1 and v2.
- **boolean hasNext()** returns true if the iterator still has elements, and false otherwise.
- **int next()** returns the current element of the iterator and moves the iterator to the next element.

Example 1:

Input: $v1 = [1,2]$, $v2 = [3,4,5,6]$

Output: $[1,3,2,4,5,6]$

Explanation: By calling next repeatedly until hasNext returns false, the order of elements returned by next should be: $[1,3,2,4,5,6]$.

Example 2:

Input: $v1 = [1]$, $v2 = []$

Output: $[1]$

Example 3:

Input: $v1 = []$, $v2 = [1]$

Output: $[1]$

Constraints:

- $0 \leq v1.length, v2.length \leq 1000$
- $1 \leq v1.length + v2.length \leq 2000$
- $-2^{31} \leq v1[i], v2[i] \leq 2^{31} - 1$

Follow up: What if you are given k vectors? How well can your code be extended to such cases?

Clarification for the follow-up question:

The "Zigzag" order is not clearly defined and is ambiguous for $k > 2$ cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".

Follow-up Example:

Input: v1 = [1,2,3], v2 = [4,5,6,7], v3 = [8,9]

Output: [1,4,8,2,5,9,3,6,7]

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Alternate between v1 and v2 element by element. When one is exhausted,

continue with the remaining elements of the other. Right?

v1=[1,2], v2=[3,4,5,6] → [1,3,2,4,5,6] ✓

Follow-up: what if there are k vectors instead of 2?"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

At construction I'll interleave v1 and v2 into a flat list: take one from v1, one from v2,

repeat, then append remaining elements from whichever list is longer.

```
# next() and hasNext() just read from that flat list.
```

```
# Time:  $O(n)$  init. Space:  $O(n)$ . Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Use a deque of (vector, index) pairs.
```

```
# next(): popleft, return v[i], re-enqueue (v, i+1) if more elements remain.
```

```
# hasNext(): deque is non-empty.
```

```
# This generalises trivially to k vectors – just enqueue all of them.
```

```
# Time:  $O(1)$  per next()/hasNext(). Space:  $O(k)$  for the deque."
```

PHASE 4 — CLEAN CODE

```
class _281_ZigzagIterator:
```

```
    def __init__(self, v1: List[int], v2: List[int]):
```

```
        self.q: deque = deque()
```

```
        if v1:
```

```
            self.q.append((v1, 0))
```

```
        if v2:
```

```
            self.q.append((v2, 0))
```

```
    def next(self) -> int:
```

```
        vec, idx = self.q.popleft()
```

```

        if idx + 1 < len(vec):
            self.q.append((vec, idx + 1))
        return vec[idx]
    def hasNext(self) -> bool:
        return bool(self.q)

```

PHASE 5 — TEST & DEBUG

```

# v1=[1,2], v2=[3,4,5,6]:
# q=[(v1,0),(v2,0)].
# next(): pop (v1,0)→return 1. re-enqueue
(v1,1). q=[(v2,0),(v1,1)].
# next(): pop (v2,0)→return 3. re-enqueue
(v2,1). q=[(v1,1),(v2,1)].
# next(): pop (v1,1)→return 2. no
re-enqueue (idx+1=2=len). q=[(v2,1)].
# next(): 4, then 5, then 6. →
[1,3,2,4,5,6] ✓
# v1=[1], v2=[]: q=[(v1,0)]. next()→1.
hasNext()→False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(1)$  per operation. Space  $O(k)$  for
the deque.
# Follow-up k vectors: init enqueues all k
non-empty vectors. Same code, no changes.

```

The deque acts as a round-robin scheduler – each vector gets one turn per cycle.

Given more time I'd ask: what if vectors have different priorities (some should appear more often)? Enqueue them multiple times or use a priority queue."

Arrays & Hashing

□ #307 Range Sum Query – Mutable

MED

[Open on LeetCode](#)

Array · Design · Binary Indexed Tree · Segment Tree

Lists: Denny Zhang

Problem

Given an integer array `nums`, handle multiple queries of the following types:

1. Update the value of an element in `nums`.
2. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `void update(int index, int val)` Updates the value of `nums[index]` to be `val`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices `left` and `right` inclusive (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

Input

```
["NumArray", "sumRange", "update",  
"sumRange"]  
[[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
```

Output

```
[null, 9, null, 8]
```

Explanation

```
NumArray numArray = new NumArray([1, 3,  
5]);
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- $0 \leq \text{index} < \text{nums.length}$
- $-100 \leq \text{val} \leq 100$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most $3 * 10^4$ calls will be made to update and sumRange.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Support two operations on an array:  
update a single element, and query  
# the prefix sum from left to right  
inclusive. Up to  $3 \times 10^4$  calls. Right?  
# The challenge is making both operations  
fast – not  $O(n)$  each."  
#  
# "NumArray([1,3,5]): sumRange(0,2)=9.  
update(1,2)→nums=[1,2,5]. sumRange(0,2)=8  
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to  
lock in the logic.  
# Store the array directly. update() just  
sets nums[index] = val in  $O(1)$ .  
# sumRange(left, right) sums  
nums[left..right] with a linear scan in  
 $O(n)$ .  
# With up to  $3 \times 10^4$  queries this is  $O(n * \text{queries})$  total – too slow for large n.  
# Time: update  $O(1)$ , sumRange  $O(n)$ . Space:  
 $O(n)$ . Should I go ahead and optimize?"  
class _307_NumArray_Brute:  
    def __init__(self, nums):  
        self.nums = nums[:]
```



```
def update(self, index, val):
    self.nums[index] = val
def sumRange(self, left, right):
    return
sum(self.nums[left:right+1])

# PHASE 3 — OPTIMIZE
# "Binary Indexed Tree (Fenwick Tree) –
each node stores a partial sum covering
# a range determined by its lowest set
bit. Both update and query run in  $O(\log n)$ .
# Update: add change to index and all its
ancestors ( $i += i \& -i$ ).
# Query prefix sum up to  $i$ : add and walk
down ( $i -= i \& -i$ ).
# For range [left, right]:  $query(right) - query(left-1)$ ."
```

```
# PHASE 4 — CLEAN CODE
class _307_NumArray:
    def __init__(self, nums: List[int]):
        self.nums = nums[:]
        n = len(nums)
        self.tree = [0] * (n + 1)
        for i, val in enumerate(nums):
```

```
        self._add(i, val)
    def _add(self, i: int, delta: int) ->
None:
        i += 1
        while i <= len(self.nums):
            self.tree[i] += delta
            i += i & (-i)
    def _prefix(self, i: int) -> int:
        i += 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i)
        return total
    def update(self, index: int, val: int)
-> None:
        self._add(index, val -
self.nums[index])
        self.nums[index] = val
    def sumRange(self, left: int, right:
int) -> int:
        return self._prefix(right) -
(self._prefix(left - 1) if left > 0 else
0)
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,3,5]: tree built.  
_prefix(2)=1+3+5=9. sumRange(0,2)=9 ✓  
# update(1,2): delta=2-3=-1. tree  
adjusted. nums=[1,2,5].  
# sumRange(0,2): _prefix(2)=8 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "update:  $O(\log n)$ . sumRange:  $O(\log n)$ .  
init:  $O(n \log n)$ .  
# Space:  $O(n)$  for the tree.  
# The bit trick  $i \& (-i)$  isolates the  
lowest set bit – this determines  
# how many elements each node is  
responsible for. Worth explaining  
clearly.  
# Alternative: segment tree – more  
flexible (supports range update) but more  
code.  
# Given more time I'd ask: what if we need  
range updates (add  $v$  to  $\text{nums}[l..r]$ )?  
# BIT can handle that with a difference  
array extension – or use a lazy segment  
tree."
```

Arrays & Hashing

□ #454 4Sum II

MED

[Open on LeetCode](#)

Array · Hash Table

Lists: CodeSignal

Problem

Given four integer arrays `nums1`, `nums2`, `nums3`, and `nums4` all of length `n`, return the number of tuples `(i, j, k, l)` such that:

- $0 \leq i, j, k, l < n$
- $nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0$

Example 1:

Input: `nums1 = [1,2], nums2 = [-2,-1],
nums3 = [-1,2], nums4 = [0,2]`

Output: 2

Explanation:

The two tuples are:

1. $(0, 0, 0, 1) \rightarrow \text{nums1}[0] + \text{nums2}[0] + \text{nums3}[0] + \text{nums4}[1] = 1 + (-2) + (-1) + 2 = 0$

2. $(1, 1, 0, 0) \rightarrow \text{nums1}[1] + \text{nums2}[1] + \text{nums3}[0] + \text{nums4}[0] = 2 + (-1) + (-1) + 0 = 0$

Example 2:

Input: `nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]`

Output: 1

Constraints:

- $n == \text{nums1.length}$
- $n == \text{nums2.length}$
- $n == \text{nums3.length}$
- $n == \text{nums4.length}$
- $1 \leq n \leq 200$
- $-228 \leq \text{nums1}[i], \text{nums2}[i], \text{nums3}[i], \text{nums4}[i] \leq 228$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count tuples (i, j, k, l) where $\text{nums1}[i] + \text{nums2}[j] + \text{nums3}[k] + \text{nums4}[l] == 0$.

All four arrays have the same length n . Return the count – not the tuples. Right?

We're not asked to deduplicate – every index combo counts separately."

#

"nums1=[1,2], nums2=[-2,-1], nums3=[-1,2], nums4=[0,2]:

$(0, 0, 0, 1): 1 + (-2) + (-1) + 2 = 0 \checkmark$.

$(1, 1, 0, 0): 2 + (-1) + (-1) + 0 = 0 \checkmark \rightarrow 2 \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Four nested loops – for every combination (i, j, k, l) , check if

$\text{nums1}[i] + \text{nums2}[j] + \text{nums3}[k] + \text{nums4}[l] == 0$ and count the matches.

Time: $O(n^4)$. For $n=200$ that's 1.6 billion iterations – way too slow.

Space: $O(1)$. Should I go ahead and optimize?"

```
class _454_FourSumII_Brute:
    def fourSumCount(self, nums1, nums2,
nums3, nums4):
        count = 0
        for a in nums1:
            for b in nums2:
                for c in nums3:
                    for d in nums4:
                        if a + b + c + d
== 0:
                            count += 1
        return count
```

PHASE 3 — OPTIMIZE

"Meet in the middle: split into two pairs.

Build a Counter of all sums $a+b$ for a in $nums1$, b in $nums2$ — $O(n^2)$ sums.

Then for each $c+d$ from $nums3 \times nums4$, look up $-(c+d)$ in the counter — $O(n^2)$ lookups.

Total: $O(n^2)$ time and $O(n^2)$ space."

PHASE 4 — CLEAN CODE

```
class _454_FourSumII:
```

```
def fourSumCount(self, nums1:
List[int], nums2: List[int], nums3:
List[int], nums4: List[int]) -> int:
    pair_sums = Counter(a + b for a in
nums1 for b in nums2)
    return sum(pair_sums[-(c + d)]
for c in nums3 for d in nums4)
```

PHASE 5 — TEST & DEBUG

```
# nums1=[1,2], nums2=[-2,-1]:
# pair_sums={1+(-2)=-1:1, 1+(-1)=0:1,
2+(-2)=0:+1→0:2, 2+(-1)=1:1}
# = {-1:1, 0:2, 1:1}
# nums3=[-1,2], nums4=[0,2]:
# (c=-1,d=0): -(-1+0)=1. pair_sums[1]=1.
count=1.
# (c=-1,d=2): -(-1+2)=-1.
pair_sums[-1]=1. count=2.
# (c=2,d=0): -(2+0)=-2. pair_sums[-2]=0.
count=2.
# (c=2,d=2): -(2+2)=-4. pair_sums[-4]=0.
count=2. → 2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$ . Space  $O(n^2)$ ."
```


Meet-in-the-middle is the key: instead of 4 nested loops, do 2+2.
The Counter default of 0 for missing keys means misses contribute 0 – clean.
Given more time I'd ask: what if the arrays are sorted and we want unique quadruplets?
That's 4Sum (#18) – different problem requiring deduplication with two pointers."

Arrays & Hashing

□ #525 Contiguous Array

MED[Open on LeetCode](#)

Array · Hash Table · Prefix Sum

Lists: Grind 169

Problem

Given a binary array `nums`, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

Input: `nums = [0,1]`

Output: 2

Explanation: `[0, 1]` is the longest contiguous subarray with an equal number of 0 and 1.

Example 2:

Input: `nums = [0,1,0]`

Output: 2

Explanation: `[0, 1]` (or `[1, 0]`) is a longest contiguous subarray with equal number of 0 and 1.

Example 3:

Input: `nums = [0,1,1,1,1,1,0,0,0]`

Output: 6

Explanation: `[1,1,1,0,0,0]` is the longest contiguous subarray with equal number of 0 and 1.

Constraints:

- `1 <= nums.length <= 105`
- `nums[i]` is either 0 or 1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Find the longest contiguous subarray with equal numbers of 0s and 1s.

Return its length – not the subarray itself. Right?"

#

"[0,1]: equal 0s and 1s → length 2 ✓

[0,1,0]: longest is [0,1] or [1,0] → length 2 ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to  
lock in the logic.  
# For every pair (i, j), count the zeros  
and ones in nums[i..j].  
# If they're equal, it's a valid subarray  
– track the maximum length.  
# Time:  $O(n^2)$  – we can precompute prefix  
counts to make each check  $O(1)$ .  
# Space:  $O(1)$ . Should I go ahead and  
optimize?"
```

```
class _525_ContiguousArray_Brute:  
    def findMaxLength(self, nums):  
        n = len(nums)  
        longest = 0  
        for i in range(n):  
            zeros = ones = 0  
            for j in range(i, n):  
                if nums[j] == 0:  
                    zeros += 1  
                else:  
                    ones += 1  
                if zeros == ones:  
                    longest =  
max(longest, j - i + 1)  
        return longest
```

PHASE 3 — OPTIMIZE

```
# "Treat 0 as -1 and 1 as +1. Keep a
running balance (prefix sum).
# Equal 0s and 1s means balance hasn't
changed – we want the longest subarray
# where balance[j] == balance[i] for some
i < j.
# Store the FIRST index each balance value
was seen. When we see the same balance
again,
# the subarray [first_seen+1 .. current]
has equal 0s and 1s.
# Seed the map with {0: -1} to handle
subarrays starting from index 0.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _525_ContiguousArray:
    def findMaxLength(self, nums:
List[int]) -> int:
        first_seen = {0: -1} # balance →
first index it appeared
        balance = 0
        longest = 0
        for i, num in enumerate(nums):
```

```
        balance += 1 if num == 1 else
-1
        if balance in first_seen:
            longest = max(longest, i
- first_seen[balance])
        else:
            first_seen[balance] = i
    return longest
```

PHASE 5 — TEST & DEBUG

```
# nums=[0,1,0]:
# i=0,num=0: balance=-1. not seen →
first_seen={0:-1,-1:0}.
# i=1,num=1: balance=0. seen at -1 →
longest=max(0,1-(-1))=2.
# i=2,num=0: balance=-1. seen at 0 →
longest=max(2,2-0)=2.
# return 2 ✓
# nums=[0,1,1,1,1,1,0,0,0]:
# Track balance... longest=6 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$  for the hash map.
# The  $0 \rightarrow -1$  substitution is the core
insight – equal 0s and 1s becomes
```

'prefix sum unchanged', which becomes 'same balance seen before'.

The $\{0:-1\}$ seed is critical – without it, subarrays starting at index 0 are missed.

Given more time I'd ask: what if it's not binary – find the longest subarray with

equal counts of two specific values?
Same trick – map target $\rightarrow$ +1, other $\rightarrow$ -1."

Arrays & Hashing

□ #560 Subarray Sum Equals K

MED[Open on LeetCode](#)

Array · Hash Table · Prefix Sum

Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1]`, `k = 2`
Output: 2

Example 2:

Input: `nums = [1,2,3]`, `k = 3`
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$

- $-107 \leq k \leq 107$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count subarrays (contiguous, non-empty) whose elements sum to k.

Can contain negatives – so sliding window won't work. Return the count. Right?"

#

"[1,1,1], k=2: [1,1] at (0,1) and [1,1] at (1,2) → 2 ✓

[1,2,3], k=3: [3] at index 2 and [1,2] → 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j), compute the sum of `nums[i..j]` and check if it equals k.

I can maintain a running sum to avoid recomputing from scratch each time, giving $O(n^2)$.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _560_SubarraySum_Brute:
    def subarraySum(self, nums, k):
        count = 0
        for i in range(len(nums)):
            total = 0
            for j in range(i, len(nums)):
                total += nums[j]
                if total == k:
                    count += 1
        return count
```

PHASE 3 — OPTIMIZE

```
# "Prefix sums: sum of nums[i..j] =
prefix[j+1] - prefix[i].
# We want prefix[j+1] - prefix[i] == k →
prefix[i] == prefix[j+1] - k.
# Walk forward tracking the running sum.
For each position, count how many
# earlier prefix sums equal (current_sum -
k). Use a Counter seeded with {0:1}
# to handle subarrays starting at index 0.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _560_SubarraySum:
```

```
def subarraySum(self, nums:
List[int], k: int) -> int:
    prefix_count = Counter({0: 1}) #
prefix_sum → how many times seen
    running_sum = 0
    count = 0
    for num in nums:
        running_sum += num
        count +=
prefix_count[running_sum - k]
        prefix_count[running_sum] +=
1
    return count
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,1,1], k=2:
# num=1: sum=1. want=1-2=-1.
prefix_count[-1]=0.
prefix_count={0:1,1:1}.
# num=1: sum=2. want=2-2=0.
prefix_count[0]=1 → count=1.
prefix_count={0:1,1:1,2:1}.
# num=1: sum=3. want=3-2=1.
prefix_count[1]=1 → count=2. → 2 ✓
# nums=[1,2,3], k=3:
```

```
# sum=1,want=-2→0. sum=3,want=0→count=1.  
sum=6,want=3→prefix_count[3]=1→count=2. →  
2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$.

This is structurally identical to #525
Contiguous Array – both use

prefix sum + hash map, both seed with
{0:1} (or {0:-1}).

Worth naming the connection explicitly:
525 finds longest, 560 counts all.

Sliding window doesn't work here because
negatives mean the sum can decrease

even as we expand right – sliding window
requires monotone sums.

Given more time I'd ask: what if we want
subarrays with sum divisible by k ?

Use $\text{prefix_sum} \% k$ – same pattern, but
map remainders."

Arrays & Hashing

□ #1010 Pairs of Songs With Total Durations Divisible by 60

MED[Open on LeetCode](#)

Array · Hash Table · Counting
Lists: CodeSignal

Problem

You are given a list of songs where the i th song has a duration of $\text{time}[i]$ seconds.

Return the number of pairs of songs for which their total duration in seconds is divisible by 60. Formally, we want the number of indices i, j such that $i < j$ with $(\text{time}[i] + \text{time}[j]) \% 60 == 0$.

Example 1:

Input: `time = [30,20,150,100,40]`

Output: 3

Explanation: Three pairs have a total duration divisible by 60:

(`time[0] = 30`, `time[2] = 150`): total duration 180

(`time[1] = 20`, `time[3] = 100`): total duration 120

(`time[1] = 20`, `time[4] = 40`): total duration 60

Example 2:

Input: `time = [60,60,60]`

Output: 3

Explanation: All three pairs have a total duration of 120, which is divisible by 60.

Constraints:

- $1 \leq \text{time.length} \leq 6 * 10^4$
- $1 \leq \text{time}[i] \leq 500$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count pairs (i,j) where i<j and
(time[i]+time[j]) % 60 == 0.
# Songs have durations in seconds. Return
the count. Right?
# Edge cases: duration exactly 60
(60%60=0), duration exactly 30 pairs with
another 30."
#
# "[30,20,150,100,40]: (30,150)=180,
(20,100)=120, (20,40)=60 → 3 ✓
# [60,60,60]: each pair sums to 120,
divisible by 60. C(3,2)=3 → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to
lock in the logic.
# For every pair (i, j) where j > i, check
if (time[i] + time[j]) % 60 == 0.
# Count all such pairs. For n=6*10^4
that's 3.6 billion comparisons – too slow.
# Time: O(n²). Space: O(1). Should I go
ahead and optimize?"
class _1010_PairsSongs_Brute:
    def numPairsDivisibleBy60(self,
time):
        count = 0
```

```
        for i in range(len(time)):
            for j in range(i + 1,
len(time)):
                if (time[i] + time[j]) %
60 == 0:
                    count += 1
    return count
```

PHASE 3 — OPTIMIZE

"Reduce each duration to $t \% 60$. We want pairs where $t_1 + t_2 \equiv 0 \pmod{60}$.

For each t , its complement is $(60 - t) \% 60$. The $\%60$ handles $t=0$ (complement=0, not 60).

Walk forward: $\text{count} +=$ how many times we've already seen the complement.

Then add t to the seen counter.

This is Two Sum with modular arithmetic — $O(n)$ time, $O(60)=O(1)$ space."

PHASE 4 — CLEAN CODE

```
class _1010_PairsSongs:
    def numPairsDivisibleBy60(self, time:
List[int]) -> int:
        seen = Counter()
        count = 0
```



```
    for t in time:
        remainder = t % 60
        complement = (60 - remainder)

% 60

        count += seen[complement]
        seen[remainder] += 1
    return count
```

PHASE 5 — TEST & DEBUG

```
# time=[30,20,150,100,40]:
# t=30: rem=30, comp=30. seen[30]=0 →
count=0. seen={30:1}.
# t=20: rem=20, comp=40. seen[40]=0 →
count=0. seen={30:1,20:1}.
# t=150: rem=30, comp=30. seen[30]=1 →
count=1. seen={30:2,20:1}.
# t=100: rem=40, comp=20. seen[20]=1 →
count=2. seen={30:2,20:1,40:1}.
# t=40: rem=40, comp=20. seen[20]=1 →
count=3. → 3 ✓
# time=[60,60,60]: each rem=0,
comp=(60-0)%60=0.
# t=60: seen[0]=0 → 0. seen={0:1}.
# t=60: seen[0]=1 → count=1. seen={0:2}.
# t=60: seen[0]=2 → count=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$ – the Counter holds at most 60 keys.

The $(60 - \text{remainder}) \% 60$ handles the $\text{remainder} = 0$ case: complement is 0, not 60.

This is Two Sum modulo 60 – recognising the Two Sum pattern in disguise

is the key interview insight.

Given more time I'd ask: what if divisor were k instead of 60?

Same algorithm – the Counter holds at most k entries, still $O(1)$ space for fixed k ."

Arrays & Hashing

□ ★ #1272 Remove Interval ★

MED

[Open on LeetCode](#)

Array

Lists: ★ Premium | Premium 98

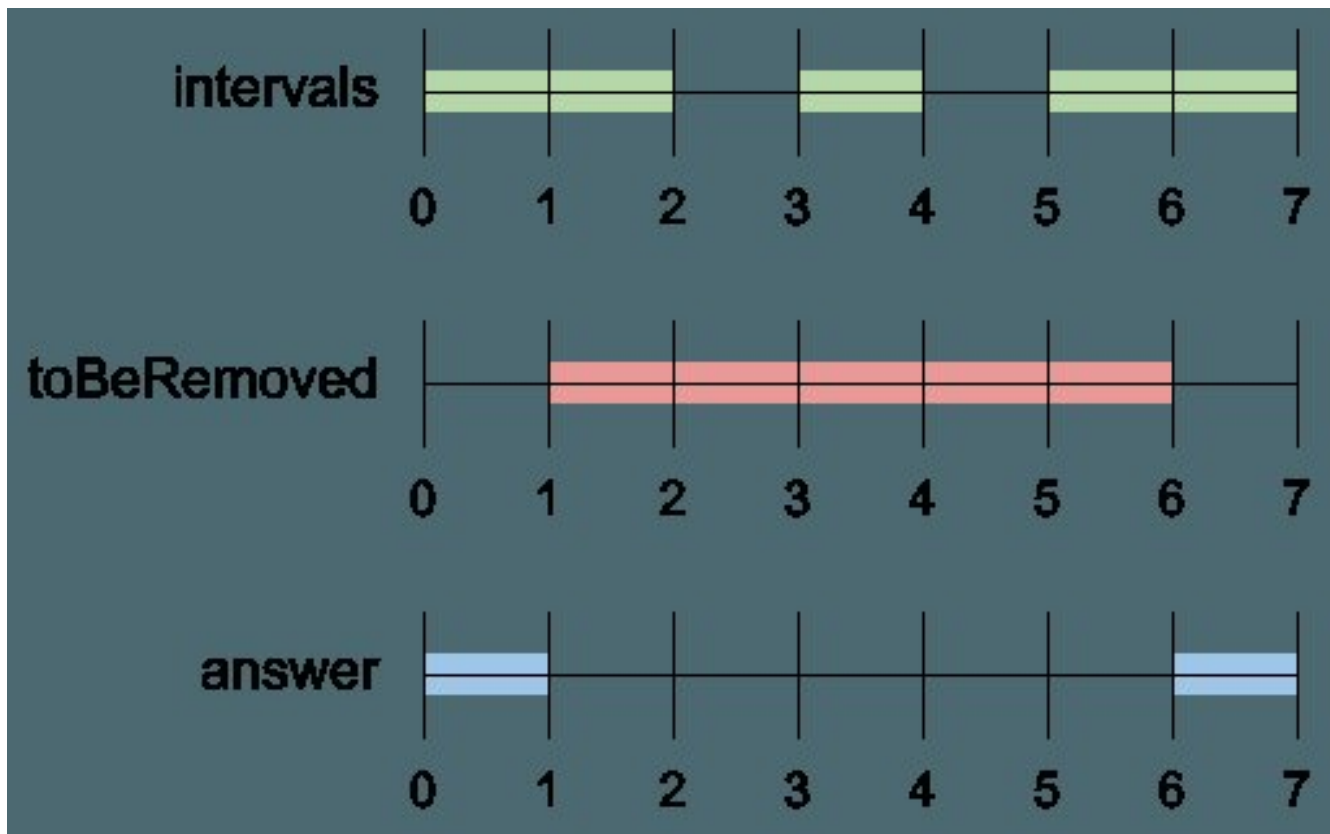
Problem

A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form $[a, b)$. A real number x is in the set if one of its intervals $[a, b)$ contains x (i.e. $a \leq x < b$).

You are given a sorted list of disjoint intervals representing a set of real numbers as described above, where $\text{intervals}[i] = [a_i, b_i]$ represents the interval $[a_i, b_i)$. You are also given another interval `toBeRemoved`.

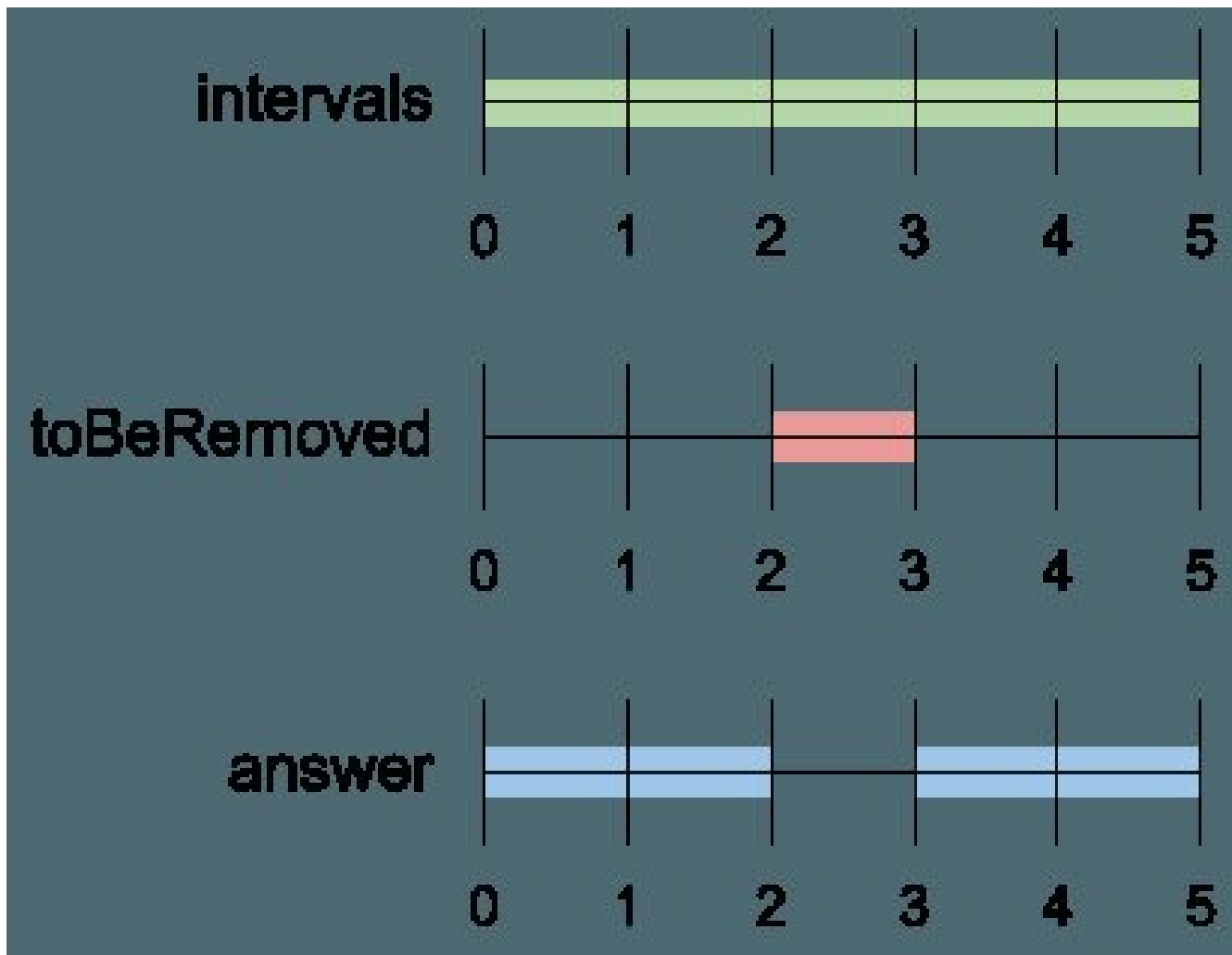
Return the set of real numbers with the interval `toBeRemoved` removed from `intervals`. In other words, return the set of real numbers such that every x in the set is in `intervals` but not in `toBeRemoved`. Your answer should be a sorted list of disjoint intervals as described above.

Example 1:



Input: `intervals = [[0,2],[3,4],[5,7]]`,
`toBeRemoved = [1,6]`
Output: `[[0,1],[6,7]]`

Example 2:



Input: `intervals = [[0,5]]`, `toBeRemoved = [2,3]`

Output: `[[0,2],[3,5]]`

Example 3:

Input: `intervals =`
`[[-5,-4],[-3,-2],[1,2],[3,5],[8,9]]`,
`toBeRemoved = [-1,4]`

Output: `[[-5,-4],[-3,-2],[4,5],[8,9]]`

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $-109 \leq a_i < b_i \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Intervals use half-open notation $[a, b)$. We remove everything in $\text{toBeRemoved}=[a, b)$

from the existing set and return what remains. Right?

Intervals are pre-sorted and non-overlapping – no need to re-sort output."

#

"[$[0,2],[3,4],[5,7]$], $\text{remove}=[1,6]$:"

$[0,2]$ partially overlaps $\rightarrow$ keep $[0,1)$.
 $[3,4]$ fully inside $\rightarrow$ gone. $[5,7]$ partially
 $\rightarrow$ keep $[6,7)$. $\rightarrow [[0,1],[6,7]] \checkmark$

$[[0,5]]$, $\text{remove}=[2,3]$: split into $[0,2)$
and $[3,5)$. $\rightarrow [[0,2],[3,5]] \checkmark$

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

Walk each interval and handle three cases: if it doesn't touch toBeRemoved at all, keep it whole.

If it partially overlaps on the left, keep the left portion. If it partially overlaps on the right,

keep the right portion. If it's fully covered, drop it entirely.

Time: $O(n)$. Space: $O(n)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Already $O(n)$ – just apply the three cases cleanly in one pass.

No overlap if: interval ends before remove starts ($\text{end} \leq a$) OR starts after remove ends ($\text{start} \geq b$).

Otherwise it overlaps. Trim left portion if $\text{start} < a \rightarrow \text{keep} [\text{start}, a)$.

Trim right portion if $\text{end} > b \rightarrow \text{keep} [b, \text{end})$."

PHASE 4 — CLEAN CODE

```
class _1272_RemoveInterval:
```

```
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int])
```

```
-> List[List[int]]:
    a, b = toBeRemoved
    result = []
    for start, end in intervals:
        if end <= a or start >= b: #
no overlap – keep whole
            result.append([start,
end])
        else:
            if start < a: # left
portion survives
                result.append([start,
a])
            if end > b: # right
portion survives
                result.append([b,
end])
    return result
```

PHASE 5 — TEST & DEBUG

```
# [[0,2],[3,4],[5,7]], remove=[1,6]:
# [0,2]: overlap. start(0)<a(1)→[0,1].
end(2)>b(6)? No. result=[[0,1]].
# [3,4]: overlap. start(3)<1? No.
end(4)>6? No. result=[[0,1]].
```



```
# [5,7]: overlap. start(5)<1? No. end(7)>6  
→ [6,7]. result=[[0,1],[6,7]] ✓  
# [[0,5]], remove=[2,3]:  
# overlap. [0,2] and [3,5]. →  
[[0,2],[3,5]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$ for output.

The two trim conditions are independent
– both can fire on the same interval
(when toBeRemoved is strictly inside the
interval), producing two output
intervals.

Given more time I'd ask: what if
toBeRemoved can be a list of intervals to
remove?

Process them in sorted order, sweeping
through once – still $O(n + m)$ time."

Arrays & Hashing

□ ★ #1429 First Unique Number ★

MED

[Open on LeetCode](#)

Array · Hash Table · Design · Queue · Data Stream

Lists: ★ Premium | Premium 98

Problem

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- `FirstUnique(int[] nums)` Initializes the object with the numbers in the queue.
- `int showFirstUnique()` returns the value of the first unique integer of the queue, and returns `-1` if there is no such integer.
- `void add(int value)` insert value to the queue.

Example 1:

Input:

```
["FirstUnique", "showFirstUnique", "add",  
"showFirstUnique", "add", "showFirstUnique",  
"add", "showFirstUnique"]
```

```
[[[2,3,5]], [], [5], [], [2], [], [3], []]
```

Output:

```
[null, 2, null, 2, null, 3, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new  
FirstUnique([2,3,5]);  
firstUnique.showFirstUnique(); //  
return 2
```

Example 2:

Input:

```
["FirstUnique", "showFirstUnique", "add",  
"add", "add", "add", "add", "showFirstUnique"]
```

```
[[[7,7,7,7,7,7]], [], [7], [3], [3], [7], [17], []]
```

Output:

```
[null, -1, null, null, null, null, null, 17]
```

Explanation:

```
FirstUnique firstUnique = new  
FirstUnique([7,7,7,7,7,7]);  
firstUnique.showFirstUnique(); //  
return -1
```

Example 3:

Input:

```
["FirstUnique", "showFirstUnique", "add",  
"showFirstUnique"]
```

```
[[[809]], [], [809], []]
```

Output:

```
[null, 809, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new
```

```
FirstUnique([809]);
```

```
firstUnique.showFirstUnique(); //
```

```
return 809
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^8$
- $1 \leq \text{value} \leq 10^8$
- At most 50000 calls will be made to showFirstUnique and add.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Maintain a queue. showFirstUnique() returns the first element that appears

```
# exactly once. add() appends to the
queue. Return -1 if no unique exists.
Right?
# Order matters – we want the first unique
by insertion order."
#
# "FirstUnique([2,3,5]): first unique = 2.
# add(5) → 5 now has count 2, still first
unique = 2.
# add(2) → 2 duplicated, first unique = 3.
# add(3) → 3 duplicated, no unique → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to
lock in the logic.
# I'll store all added elements in a list.
showFirstUnique() scans the list in
insertion order,
# counting occurrences of each element,
and returns the first one that appears
exactly once.
# add() is  $O(1)$ . showFirstUnique() is  $O(n)$ 
per call – with many calls this gets slow.
# Time: add  $O(1)$ , showFirstUnique  $O(n)$ .
Space:  $O(n)$ . Should I go ahead and
optimize?"
```

```
class _1429_FirstUnique_Brute:
    def __init__(self, nums):
        self.data = list(nums)
    def showFirstUnique(self):
        from collections import Counter
        freq = Counter(self.data)
        for x in self.data:
            if freq[x] == 1:
                return x
        return -1
    def add(self, value):
        self.data.append(value)
```

PHASE 3 — OPTIMIZE

"Two data structures:

A 'seen' set – tracks all values ever added (including duplicates).

An ordered dict (or insertion-order dict) – maps value → True, only for unique values.

On add: if not in seen → it's new, add to both seen and dict.

if in seen and in dict → it became a duplicate, remove from dict.

if in seen and not in dict → already duplicate, ignore.

```
# showFirstUnique(): peek at first key in  
dict – O(1) in Python 3.7+  
(insertion-ordered).
```

```
#
```

```
# Time: O(1) per add, O(1) per  
showFirstUnique. Space: O(n)."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _1429_FirstUnique:  
    def __init__(self, nums: List[int]):  
        self.seen : set = set()  
        self.uniques: dict = {}  
        for num in nums:  
            self.add(num)  
  
    def showFirstUnique(self) -> int:  
        return next(iter(self.uniques),  
-1)  
  
    def add(self, value: int) -> None:  
        if value not in self.seen:  
            self.seen.add(value)  
            self.uniques[value] = True  
        elif value in self.uniques:  
            del self.uniques[value]
```

```
# PHASE 5 — TEST & DEBUG
```



```
# init([2,3,5]): seen={2,3,5},  
uniques={2,3,5}.  
# showFirstUnique() → 2 ✓  
# add(5): 5 in seen and in uniques → del  
5. uniques={2,3}.  
# showFirstUnique() → 2 ✓  
# add(2): 2 in seen and in uniques → del  
2. uniques={3}.  
# showFirstUnique() → 3 ✓  
# add(3): del 3. uniques={}.  
# showFirstUnique() → -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"0(1) per operation. Space 0(n).

Python dicts maintain insertion order since 3.7, so next(iter(dict)) returns the oldest key still present – exactly what we need.

If order weren't guaranteed, we'd use collections.OrderedDict explicitly.

Given more time I'd ask: what if we need to support remove() as well?

That's a linked hash map (doubly linked list + hash map) – 0(1) all operations."

Arrays & Hashing

□ #3159 Find Occurrences of an Element in an Array **MED**

[Open on LeetCode](#)

Array · Hash Table

Lists: CodeSignal

Problem

You are given an integer array `nums`, an integer array `queries`, and an integer `x`.

For each `queries[i]`, you need to find the index of the `queries[i]`th occurrence of `x` in the `nums` array. If there are fewer than `queries[i]` occurrences of `x`, the answer should be `-1` for that query.

Return an integer array `answer` containing the answers to all queries.

Example 1:

Input: `nums = [1,3,1,7]`, `queries = [1,3,2,4]`, `x = 1`

Output: `[0,-1,2,-1]`

Explanation:

- For the 1st query, the first occurrence of 1 is at index 0.

- For the 2nd query, there are only two occurrences of 1 in nums, so the answer is -1.
- For the 3rd query, the second occurrence of 1 is at index 2.
- For the 4th query, there are only two occurrences of 1 in nums, so the answer is -1.

Example 2:

Input: nums = [1,2,3], queries = [10], x = 5

Output: [-1]

Explanation:

- For the 1st query, 5 doesn't exist in nums, so the answer is -1.

Constraints:

- $1 \leq \text{nums.length}, \text{queries.length} \leq 105$
- $1 \leq \text{queries}[i] \leq 105$
- $1 \leq \text{nums}[i], x \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Preprocess nums to find all indices where x appears."

For each query q, return the index of the q-th occurrence of x (1-indexed).

```
# If fewer than q occurrences exist,
return -1. Right?"
#
# "nums=[1,3,1,7], queries=[1,3,2,4],
x=1:
# x appears at indices [0,2]. q=1→0,
q=3→-1, q=2→2, q=4→-1. → [0,-1,2,-1] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to
lock in the logic.
# For each query q, scan nums from the
beginning, count occurrences of x as we
go,
# and return the index where we hit the
q-th occurrence – or -1 if we run out.
# Time: O(n * |queries|). Space: O(1).
Should I go ahead and optimize?"
class _3159_FindOccurrencesBrute:
    def occurrencesOfElement(self, nums,
queries, x):
        result = []
        for q in queries:
            count = 0
            found = -1
            for i, n in enumerate(nums):
```

```
        if n == x:
            count += 1
            if count == q:
                found = i
                break
        result.append(found)
    return result
```

PHASE 3 — OPTIMIZE

"Precompute a list of all indices where x appears — $O(n)$.

Each query is then an $O(1)$ index lookup. Total: $O(n + |\text{queries}|)$."

PHASE 4 — CLEAN CODE

```
class _3159_FindOccurrences:
    def occurrencesOfElement(self, nums:
List[int], queries: List[int], x: int) ->
List[int]:
        positions = [i for i, n in
enumerate(nums) if n == x]
        return [positions[q - 1] if q - 1
< len(positions) else -1 for q in queries]
```

PHASE 5 — TEST & DEBUG

nums=[1,3,1,7], x=1: positions=[0,2].

```
# q=1: positions[0]=0 ✓. q=3: 3-1=2 >=
len(positions)=2 → -1 ✓.
# q=2: positions[1]=2 ✓. q=4: -1 ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + q)$. Space $O(k)$ where k = occurrences of x .

If queries were sorted, binary search on positions would give $O((n+q) \log k)$

with minimal extra space – but for unsorted queries, the precompute list is cleaner.

Given more time I'd ask: what if x could change between queries?

Build a map from value → sorted list of indices, then binary search per query."

String

Round 2 · High · Medium · 5 questions

String

□ #8 String to Integer (atoi)

MED[Open on LeetCode](#)

String

Lists: Grind 169

Problem

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range.

Specifically, integers less than -231 should be rounded to -231 , and integers greater than $231 - 1$ should be rounded to $231 - 1$.

Return the integer as the final result.

Example 1:

Input: $s = "42"$

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)
^

Step 2: "42" (no characters read because there is neither a '-' nor '+')
^

Step 3: "42" ("42" is read in)
^

Example 2:

Input: $s = "-042"$

Output: -42

Explanation:

Step 1: " -042" (leading whitespace is read and ignored)

^

Step 2: " -042" ('-' is read, so the result should be negative)

^

Step 3: " -042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

^

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

^

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

^

Example 4:**Input:** `s = "0-1"`**Output:** 0**Explanation:**

Step 1: `"0-1"` (no characters read because there is no leading whitespace)

^

Step 2: `"0-1"` (no characters read because there is neither a '-' nor '+')

^

Step 3: `"0-1"` ("`0`" is read in; reading stops because the next character is a non-digit)

^

Example 5:**Input:** `s = "words and 987"`**Output:** 0**Explanation:**

Reading stops at the first non-digit character 'w'.

Constraints:

- $0 \leq s.length \leq 200$

- s consists of English letters (lower-case and upper-case), digits (0–9), ' ', '+', '-', and '.'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Four steps in order: strip leading whitespace, read optional sign,

read digits until non-digit or end, clamp to 32-bit signed range $[-2^{31}, 2^{31}-1]$. Right?

Stop at the FIRST non-digit after the optional sign – don't skip over non-digits."

#

"'42'→42. ' -042'→-42 (strip, sign, parse 042). '1337c0d3'→1337 (stop at 'c').

'0-1'→0 (first digit parsed, stop at '-'). 'words'→0 (first char non-digit, no digits read)."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

The naive approach is to try Python's built-in `int()` after stripping whitespace

—

```
# but that fails on partial strings like  
'42abc' and doesn't clamp to 32-bit range.  
# So the 'brute force' here is really just  
the manual state machine, which I'll  
implement.
```

```
# Time: O(n). Space: O(1). Should I go  
ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Manual state machine with a pointer:
```

```
# 1. Skip whitespace.
```

```
# 2. Read sign.
```

```
# 3. Read digits, building number, check  
overflow before each digit.
```

```
# Overflow check: ans > MAX//10 OR (ans ==  
MAX//10 AND next digit > 7).
```

```
# Return sign * ans (clamped)."
```

PHASE 4 — CLEAN CODE

```
class _8_Atoi:
```

```
    def myAtoi(self, s: str) -> int:
```

```
        INT_MAX, INT_MIN = 2**31 - 1,
```

```
        -(2**31)
```

```
        i, n = 0, len(s)
```

```
        while i < n and s[i] == ' ': #
strip whitespace
            i += 1
        sign = 1
        if i < n and s[i] in '+-':
            sign = -1 if s[i] == '-' else
1
            i += 1
        result = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if result > INT_MAX // 10 or
(result == INT_MAX // 10 and d > 7):
                return INT_MAX if sign ==
1 else INT_MIN
            result = result * 10 + d
            i += 1
        return sign * result
```

PHASE 5 — TEST & DEBUG

'42': no space, no sign, digits→42.

return 42 ✓

' -042': skip space, sign=-1, digits

0,4,2→42. return -42 ✓

```
# '1337c0d3': digits 1,3,3,7→1337, stop at  
'c'. return 1337 ✓  
# '999999999999': overflow triggers →  
return INT_MAX ✓  
# '': i=0, no sign, no digits → return 0 ✓  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n). Space O(1).  
# The overflow check BEFORE adding each  
digit is critical – without it we'd  
overflow Python  
# integers silently (Python handles big  
ints) but the logic breaks for fixed-width  
languages.  
# The magic '7' in  $d > 7$  comes from  $2^{31}-1$   
= 2147483647 – last digit is 7.  
# Given more time I'd ask: handle  
locale-specific decimal separators  
(commas)?  
# Add a pre-processing step to  
strip/replace them before parsing."
```

String

□ ★ #249 Group Shifted Strings ★

MED[Open on LeetCode](#)

Array · Hash Table · String

Lists: ★ Premium | Premium 98

Problem

Perform the following shift operations on a string:

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... <-> "abc" <-> "bcd" <-> ... <->

"xyz" <-> "yza" <-> <-> "zab" <-> "abc"
<-> ...

You are given an array of strings `strings`, group together all `strings[i]` that belong to the same shifting sequence. You may return the answer in any order.

Example 1:

Input: `strings =`

`["abc","bcd","acef","xyz","az","ba","a","z"]`

Output:

`[["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]`

Example 2:

Input: `strings = ["a"]`

Output: `[["a"]]`

Constraints:

- `1 <= strings.length <= 200`
- `1 <= strings[i].length <= 50`
- `strings[i]` consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Two strings are in the same shifting
sequence if one can become the other
# by consistently shifting every character
by the same amount (cyclically, a→z
wraps).
# Group them. Return order doesn't matter.
Right?
# 'abc' and 'xyz' are in the same group –
same relative gaps between characters."
#
# "[ 'abc', 'bcd', 'xyz', 'az', 'ba', 'a', 'z']":
# 'abc', 'bcd', 'xyz' → all have gaps [1,1]
→ same group.
# 'az', 'ba' → gaps [25] and [25] (b–a=1...
wait: 'az': z–a=25. 'ba': a–b=–1+26=25) →
same group.
# 'a', 'z' → single char, empty gap string
→ same group."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to
lock in the logic.
```

```
# For each string, generate all 26
rotations by shifting every character by
0..25.
```

```
# If any rotation of string A equals  
string B, they belong in the same group.  
# This is  $O(n^2 * 26 * L)$  – quadratic in  
the number of strings, far too slow.  
# Time:  $O(n^2 * 26 * L)$ . Space:  $O(n)$ . Should I  
go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Compute a canonical key for each  
string: the tuple of consecutive char  
differences mod 26.
```

```
# 'abc' → (1,1). 'xyz' → (1,1). 'az' →  
(25,). 'ba' → (25,).
```

```
# All strings with the same key are in the  
same shifting group.
```

```
# Group by key using a defaultdict. Time:  
 $O(n * L)$ . Space:  $O(n * L)$ ."
```

PHASE 4 — CLEAN CODE

```
class _249_GroupShiftedStrings:  
    def groupStrings(self, strings:  
List[str]) -> List[List[str]]:  
        groups: defaultdict =  
defaultdict(list)  
        for s in strings:
```

```
        key = tuple((ord(s[i]) -  
ord(s[i - 1])) % 26 for i in range(1,  
len(s)))  
        groups[key].append(s)  
return list(groups.values())
```

PHASE 5 — TEST & DEBUG

```
# 'abc': key=(1,1). 'bcd': (1,1). 'xyz':  
(1,1) → same group ✓
```

```
# 'az':
```

```
key=((ord('z')-ord('a'))%26,)=(25,).
```

```
'ba': ((ord('a')-ord('b'))%26,)=(25,) →  
same ✓
```

```
# 'a','z': key=() (empty tuple) → same  
group ✓
```

```
# 'acef': key=(2,2,1) → its own group ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n*L)$  where  $L = \text{max string length}$ .  
Space  $O(n*L)$ .
```

```
# The %26 mod handles the wraparound:  
'z'→'a' gives  $(\text{ord('a')}-\text{ord('z')})\%26 =$   
 $(-25)\%26 = 1$ .
```

```
# Without the mod, 'az' and 'ba' wouldn't  
match — the mod is the key insight.
```

Given more time I'd ask: how do you efficiently check if two strings are in the same group

at query time? Build a map string→canonical_key at init, then $O(L)$ per query."

String

★ #271 Encode and Decode Strings ★

MED[Open on LeetCode](#)

Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {  
    // ... your code  
    return encoded_string;  
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {  
    //... your code  
    return strs;  
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 =  
decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
```

```
Output: ["Hello","World"]
```

Explanation:

Machine 1:

```
Codec encoder = new Codec();
```

```
String msg = encoder.encode(strs);
```

```
Machine 1 ---msg---> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
```

```
Output: [""]
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$

- $0 \leq \text{strs}[i].\text{length} \leq 200$
- `strs[i]` contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design `encode(strs)` and `decode(s)` that roundtrip correctly for ANY list of strings —

including strings with special characters, empty strings, or strings containing

whatever delimiter we might choose.
Right?

We can't use `eval` or `pickle`."

#

`"['Hello', 'World'] → encode → some string → decode → ['Hello', 'World'] ✓`

`[''] → [''] ✓ (empty string must survive the roundtrip)"`

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

A naive approach is to join strings with a rare delimiter like '|' – but it breaks if

strings themselves contain '|'. We'd need escaping: replace '|' with '\|' and '\' with '\\'.

This works but decoding is error-prone and complex to get right.

Time: O(total chars). Space: O(total chars). Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (length-prefix encoding)

"Prefix each string with its length and a fixed separator (e.g. '#').

encode: '5#Hello4#World'. decode: read number up to '#', slice that many chars, repeat.

No ambiguity – length tells us exactly where each string ends.

Works for any content including '#' and empty strings."

PHASE 4 — CLEAN CODE

class _271_Codec:

```

def encode(self, strs: List[str]) ->
str:
    return ''.join(f'{len(s)}#{s}'
for s in strs)
def decode(self, s: str) -> List[str]:
    result = []
    i = 0
    while i < len(s):
        j = s.index('#', i)
        length = int(s[i:j])
        result.append(s[j + 1: j + 1 +
length])
        i = j + 1 + length
    return result

```

PHASE 5 — TEST & DEBUG

```

# encode(['Hello', 'World']):
'5#Hello5#World'.
# decode('5#Hello5#World'):
# i=0: j=1 (index of '#'). length=5.
result=['Hello']. i=7.
# i=7: j=8. length=5.
result=['Hello', 'World']. i=14. done ✓
# encode(['']): '0#'. decode('0#'): j=1,
length=0, append ''. → [''] ✓

```

```
# encode(['a#b', 'c']): '3#a#b1#c'.  
decode: length=3→'a#b', length=1→'c' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"encode 0(total chars). decode 0(total chars). Space 0(n).

Length-prefix is the standard solution – no special characters, no escaping needed.

The '#' separator is arbitrary – any fixed separator works since we always read the length first and know exactly how many chars to consume.

Given more time I'd ask: what if strings can contain binary data (null bytes)?

Use a 4-byte big-endian integer prefix instead of a text number – fully binary-safe."

String

□ #635 Design Log Storage System

MED[Open on LeetCode](#)

Hash Table · String · Design · Ordered Set
Lists: Denny Zhang

Problem

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

- **LogSystem()** Initializes the LogSystem object.
- **void put(int id, string timestamp)** Stores the given log (id, timestamp) in your storage system.
- **int[] retrieve(string start, string end, string granularity)** Returns the IDs of the logs whose timestamps are within the range from start to end inclusive. start and end all have the same

format as timestamp, and granularity means how precise the range should be (i.e. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.

Example 1:

Input

```
["LogSystem", "put", "put", "put",  
"retrieve", "retrieve"]  
[[], [1, "2017:01:01:23:59:59"], [2,  
"2017:01:01:22:59:59"], [3,  
"2016:01:01:00:00:00"],  
["2016:01:01:01:01:01",  
"2017:01:01:23:00:00", "Year"],  
["2016:01:01:01:01:01",  
"2017:01:01:23:00:00", "Hour"]]
```

Output

```
[null, null, null, null, [3, 2, 1], [2,  
1]]
```

Explanation

```
LogSystem logSystem = new LogSystem();
```

Constraints:

- $1 \leq \text{id} \leq 500$
- $2000 \leq \text{Year} \leq 2017$
- $1 \leq \text{Month} \leq 12$
- $1 \leq \text{Day} \leq 31$
- $0 \leq \text{Hour} \leq 23$
- $0 \leq \text{Minute, Second} \leq 59$

- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].
- At most 500 calls will be made to put and retrieve.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Store logs with (id, timestamp).
retrieve(start, end, granularity) returns
IDs

of logs within the range, where
comparison is only up to the granularity
level.

Timestamps format:

'YYYY:MM:DD:HH:mm:ss'. Truncate at the
granularity character position. Right?"

#

"retrieve with 'Year': compare only the
first 4 chars. '2017:...' vs '2016:...'
etc."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to
lock in the logic."

```
# Store all logs in a flat list. On  
retrieve, iterate over every log, truncate  
its timestamp  
# to the granularity prefix length, and  
check if it falls within the truncated  
start/end range.  
# With at most 500 calls and a fixed  
19-char timestamp format,  $O(n)$  per  
retrieve is acceptable.  
# Time: put  $O(1)$ , retrieve  $O(n)$ . Space:  
 $O(n)$ . Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Map granularity name to the string  
prefix length that captures it.  
# 'Year'→4 chars, 'Month'→7, 'Day'→10,  
'Hour'→13, 'Minute'→16, 'Second'→19.  
# Truncate all three timestamps to that  
length and do a simple string comparison.  
# Lexicographic order of the timestamp  
format is correct for chronological  
order."
```

PHASE 4 — CLEAN CODE

```
class _635_LogSystem:
```



```
GRAN = {'Year': 4, 'Month': 7, 'Day':  
10, 'Hour': 13, 'Minute': 16, 'Second':  
19}
```

```
def __init__(self):  
    self.logs: List[tuple] = []  
  
    def put(self, log_id: int, timestamp:  
str) -> None:  
        self.logs.append((log_id,  
timestamp))  
  
    def retrieve(self, start: str, end:  
str, granularity: str) -> List[int]:  
        cut = self.GRAN[granularity]  
        return [lid for lid, ts in  
self.logs if start[:cut] <= ts[:cut] <=  
end[:cut]]
```

PHASE 5 — TEST & DEBUG

```
# put(1, '2017:01:01:23:59:59'),  
put(2, '2017:01:01:22:59:59'),  
put(3, '2016:01:01:00:00:00').  
# retrieve(..., 'Year'): cut=4. compare  
'2016' <= 'year' <= '2017'. All 3 match ->  
[1,2,3] ✓  
# retrieve(..., 'Hour'): cut=13.  
'2016:01:01:01' <= '2016:01:01:00'? No -> 3
```

excluded.

```
# '2016:01:01:01' <= '2017:01:01:22' <= '2017:01:01:23'? Yes→2. '2017:01:01:23'?
```

```
Yes→1.
```

```
# → [2,1] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "put 0(1). retrieve 0(n) – scans all logs.
```

```
# The key insight: timestamps in 'YYYY:MM:DD:HH:mm:ss' format sort lexicographically
```

```
# correctly because they're zero-padded and delimiters are identical.
```

```
# For larger datasets, a sorted list + binary search would bring retrieve to  $O(\log n + k)$ .
```

```
# Given more time I'd ask: what if granularity could be arbitrary (e.g. 'Week')?
```

```
# We'd need a custom truncation function rather than fixed prefix lengths."
```

String

□ #1138 Alphabet Board Path

MED[Open on LeetCode](#)

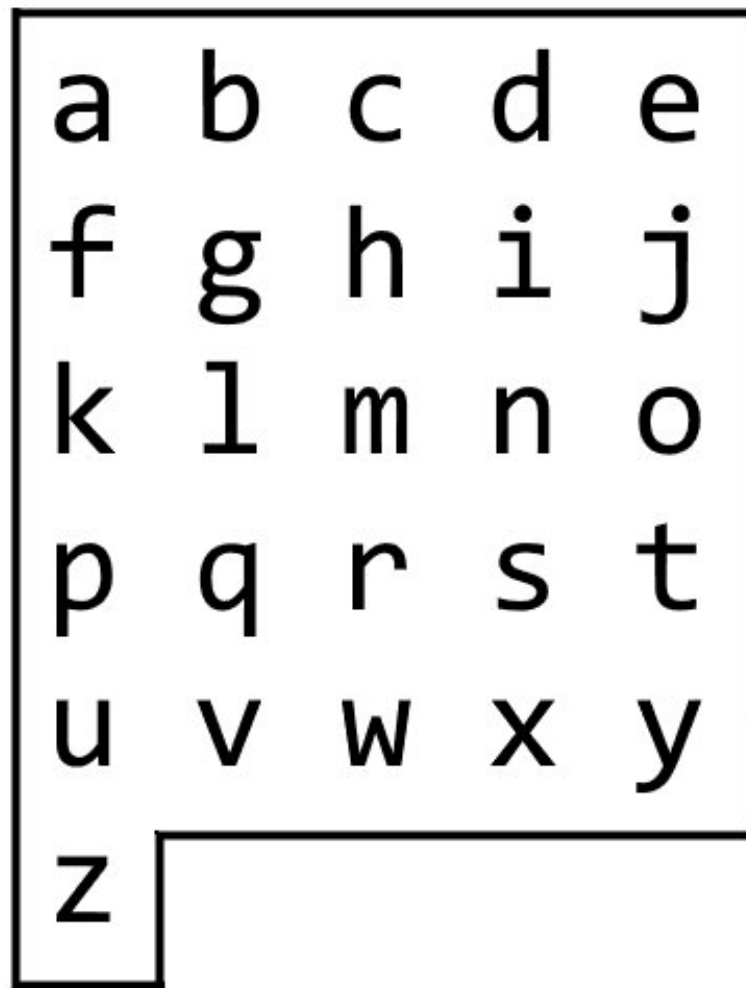
Hash Table · String

Lists: Denny Zhang

Problem

On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].

Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram below.



We may make the following moves:

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;

- '!' adds the character `board[r][c]` at our current position (r, c) to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)

Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1:

Input: target = "leet"

Output: "DDR!UURRR!!DDD!"

Example 2:

Input: target = "code"

Output: "RR!DDRR!UUL!R!"

Constraints:

- $1 \leq \text{target.length} \leq 100$
- target consists only of English lowercase letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Board is ['abcde','fghij','klmno','pqrst','vwxy','z'] – 5 cols except row 5 which has only 'z'.
# Start at (0,0). For each char in target, output U/D/L/R moves then '!'. Right?
# The critical constraint: 'z' is at (5,0) – the only cell in its row.
# Moving to 'z' must clear the column before going down. Moving from 'z' must go up before going right."
#
# "target='leet': l is at (2,1). Move D,D,R,!. e=(0,4) – go U,U,R,R,R,!. etc."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each character, compute its board position as row = (ord(ch)-ord('a'))//5,
# col = (ord(ch)-ord('a'))%5. Then generate moves: go vertical first, then horizontal.
# The naive approach fails for 'z' – moving down before clearing column 0 steps off the board.
```

```
# Time: O(|target| * max_moves). Space:
O(|target|). Should I go ahead and
optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Formula: row = (ord(ch)-ord('a'))//5,
col = (ord(ch)-ord('a'))%5.
# For 'z' specifically: it's at (5,0),
col=0. When moving TO 'z', move left first
# (to reach col 0) then move down (to
reach row 5) – otherwise we'd step off the
board.
# When moving FROM 'z', move up first then
move right – same reason.
# General case: move vertically then
horizontally (or horizontally then
vertically).
# 'z' is the only exception – must move
left before down."
```

PHASE 4 — CLEAN CODE

```
class _1138_AlphabetBoardPath:
    def alphabetBoardPath(self, target:
str) -> str:
        row = col = 0
        moves = []
```

```
        for ch in target:
            dest_row = (ord(ch) -
ord('a')) // 5
            dest_col = (ord(ch) -
ord('a')) % 5
            if ch == 'z': # must clear
column before going down
                moves.extend('L' * (col -
dest_col))
                moves.extend('D' *
(dest_row - row))
            else:
                moves.extend('U' * (row -
dest_row))
                moves.extend('D' *
(dest_row - row))
                moves.extend('L' * (col -
dest_col))
                moves.extend('R' *
(dest_col - col))
            moves.append('!')
            row, col = dest_row, dest_col
        return ''.join(moves)
```

PHASE 5 — TEST & DEBUG


```
# target='leet': l=(2,1). from (0,0):
D,D,R,! → "DDR!". row=2,col=1.
# e=(0,4): U,U,R,R,R,! → "UURRR!".
row=0,col=4.
# e=(0,4): no move,! → "!". row=0,col=4.
# t=(3,4): D,D,D,! → "DDD!". →
"DDR!UURRR!!DDD!" ✓
# target='z': from (0,0): ch='z'.
L*(0)=none. D*5. → "DDDDD!" ✓
# If coming from (5,4) to 'z' (5,0):
L,L,L,L,! – no D needed ✓.
# If going from 'z' to 'a': ch='a'=(0,0).
Not 'z' → U*5,L*0,R*0 → "UUUUU!" ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(|target| * max\_moves) =$ 
 $O(|target| * 10) = O(|target|)$ . Space
 $O(|target|)$ .
# The 'z' case is the entire problem –
everything else is standard grid
navigation.
# Phrase it clearly: 'z' is at the
leftmost position of its row, so we must
reach col 0
# before stepping down into row 5,
otherwise intermediate positions don't
```

exist on the board.

Given more time I'd ask: what if the board were different (e.g. a phone numpad)?

Same approach – just recompute (row, col) from the new layout."

Two Pointers

Round 2 · High · Medium · 14 questions

Two Pointers

□ #11 Container With Most Water

MED[Open on LeetCode](#)

Array · Two Pointers · Greedy

Lists: Grind 169

Problem

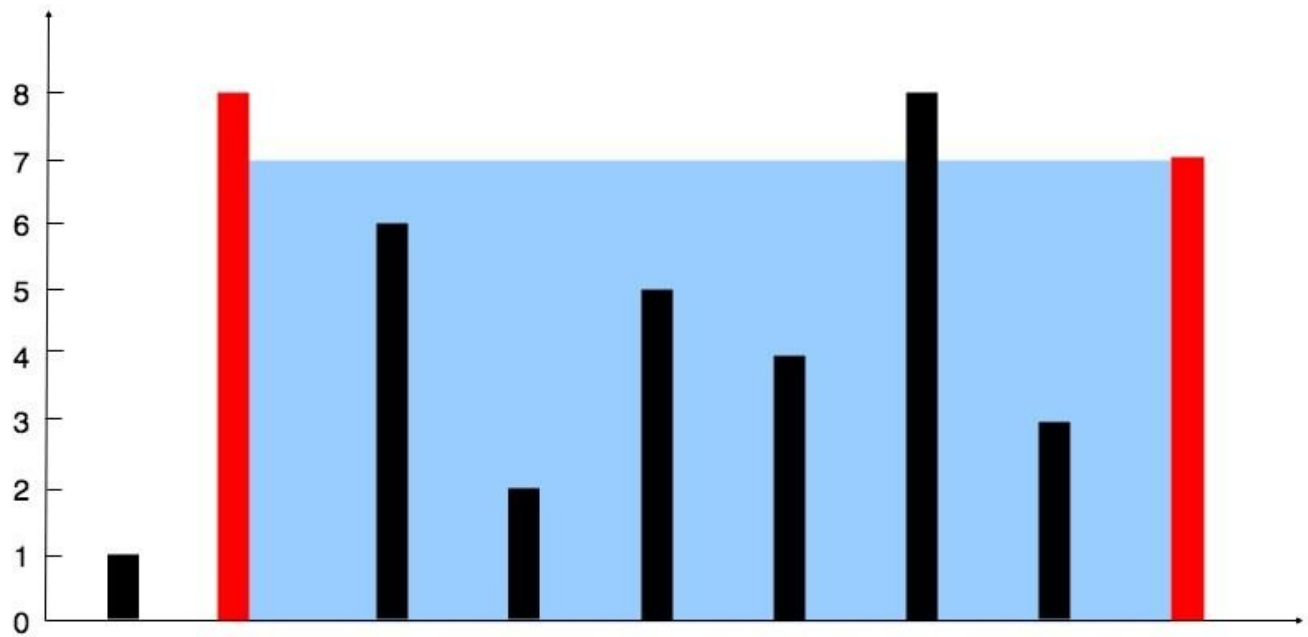
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the `x`-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 105$
- $0 \leq \text{height}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given heights of vertical lines, find two lines that form the largest container.

Area = $\min(\text{height}[\text{left}], \text{height}[\text{right}]) * (\text{right} - \text{left})$. Return max area. Right?

We can't slant — so water level is always min of the two chosen heights."

#

"[1,8,6,2,5,4,8,3,7]: left=1(h=8), right=8(h=7). Area= $\min(8,7)*(8-1)=49$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j) where $j > i$, compute area = $\min(\text{height}[i], \text{height}[j]) * (j - i)$

and track the maximum. That's $C(n,2)$ pairs to check.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _11_ContainerMostWater_Brute:
    def maxArea(self, height):
```

```
best = 0
for i in range(len(height)):
    for j in range(i + 1,
len(height)):
        best = max(best,
min(height[i], height[j]) * (j - i))
return best
```

PHASE 3 — OPTIMIZE

"Two pointers: left=0, right=n-1. At each step compute area.

Move the pointer with the SHORTER height inward — keeping the taller height

maximises any future container. Moving the taller pointer can only shrink

both width and potentially height, so it can't improve the result.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _11_ContainerMostWater:
```

```
    def maxArea(self, height: List[int])
-> int:
```

```
    left, right = 0, len(height) - 1
```

```
    best = 0
```

```
    while left < right:
```

```
        area = min(height[left],
height[right]) * (right - left)
        best = max(best, area)
        if height[left] <
height[right]:
            left += 1
        else:
            right -= 1
    return best
```

PHASE 5 — TEST & DEBUG

```
# height=[1,8,6,2,5,4,8,3,7]:
# l=0(h=1),r=8(h=7): area=1*8=8.
h[l]<h[r]→l=1.
# l=1(h=8),r=8(h=7): area=7*7=49.
h[l]>h[r]→r=7.
# l=1(h=8),r=7(h=3): area=3*6=18.
h[l]>h[r]→r=6.
# ... best stays 49. → 49 ✓
# height=[1,1]: area=1*1=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# The greedy argument: if we move the
taller pointer, we keep a shorter
constraint
```


(limited by the short side) AND reduce width – area can only shrink or stay.
So we always move the shorter side – we can't do better by moving the taller one.
This is the key insight to state explicitly in the interview.
Given more time I'd ask: what if we need to find the pair of indices (not just the area)?
Track left_best and right_best alongside best – same code, just record the positions."

Two Pointers

□ #15 3Sum

MED[Open on LeetCode](#)

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

$\text{nums}[0] + \text{nums}[1] + \text{nums}[2] = (-1) + 0 + 1 = 0.$

$\text{nums}[1] + \text{nums}[2] + \text{nums}[4] = 0 + 1 + (-1) = 0.$

$\text{nums}[0] + \text{nums}[3] + \text{nums}[4] = (-1) + 2 + (-1) = 0.$

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: `nums = [0,1,1]`

Output: `[]`

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: `nums = [0,0,0]`

Output: `[[0,0,0]]`

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-105 \leq \text{nums}[i] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find all unique triplets summing to 0.
Return the triplets, not indices.

No duplicate triplets in the output.
Right?

Order within a triplet doesn't matter –
[-1,0,1] and [0,-1,1] are the same."

#

"[-1,0,1,2,-1,-4]: sort →
[-4,-1,-1,0,1,2]. → [[-1,-1,2],[-1,0,1]]

✓

[0,0,0]: → [[0,0,0]] ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."  
# Three nested loops over  $i < j < k$ ; if  
nums[i]+nums[j]+nums[k]==0, add triplet.  
# Store triplets in a set keyed by sorted  
tuple to deduplicate.  
# Correct, but  $O(n^3)$  with hash-set  
overhead on large  $n$ .  
# Time:  $O(n^3)$ . Space:  $O(n)$  for dedup set.  
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sort the array. Fix one element  
nums[i], use two pointers on the rest."  
# Sorting enables: skip duplicate values  
of nums[i] to avoid duplicate triplets,  
# and move pointers efficiently based on  
the sum.  
# After finding a valid triplet, skip  
duplicate nums[left] and nums[right].  
# Time:  $O(n^2)$ . Space:  $O(1)$  extra."
```

PHASE 4 — CLEAN CODE

```
class _15_ThreeSum:  
    def threeSum(self, nums: List[int])  
    -> List[List[int]]:
```

```
nums.sort()
result = []
for i in range(len(nums) - 2):
    if i > 0 and nums[i] == nums[i
- 1]: # skip duplicate fixed element
        continue
    left, right = i + 1, len(nums)
- 1

    while left < right:
        total = nums[i] +
nums[left] + nums[right]
        if total < 0:
            left += 1
        elif total > 0:
            right -= 1
        else:
            result.append([nums[i
], nums[left], nums[right]])
            left += 1
            right -= 1
            while left < right
and nums[left] == nums[left - 1]: left +=
1

            while left < right
and nums[right] == nums[right + 1]: right
```

-= 1

return result

PHASE 5 — TEST & DEBUG

nums=[-1,0,1,2,-1,-4] → sorted:
[-4,-1,-1,0,1,2].

i=0 (-4): l=1,r=5.

sums=-4+(-1)+2=-3<0→l++.

-4+(-1)+2=-3<0→l++. -4+0+2=-2<0→l++.

-4+1+2=-1<0→l++. l=r → stop.

i=1 (-1): l=2,r=5.

-1+(-1)+2=0→append[-1,-1,2]. l=3,r=4.

skip dups: l=3 ok, r=4 ok.

-1+0+1=0→append[-1,0,1]. l=4,r=3→stop.

i=2 (-1): i>0 and nums[2]==nums[1]==-1 → skip.

i=3 (0): l=4,r=5. 0+1+2=3>0→r--.

l≥r→stop.

result=[[-1,-1,2],[-1,0,1]] ✓

nums=[0,0,0]: i=0,l=1,r=2.

0+0+0=0→append. l=2,r=1→stop. → [[0,0,0]]

✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$ – outer loop $O(n)$, inner two-pointer $O(n)$ each. Sorting is $O(n \log n)$

n) dominated.

```
# Space O(1) extra (ignoring output).  
# Two deduplication steps:  
# 1. Skip nums[i] == nums[i-1] to avoid  
#    re-fixing the same outer value.  
# 2. After appending, skip nums[left] ==  
#    nums[left-1] and nums[right] ==  
#    nums[right+1]  
# to avoid re-collecting the same triplet.  
# Both checks are essential – missing  
# either produces duplicate output.  
# Given more time I'd ask: 4Sum (#18)? Add  
# one more outer loop –  $O(n^3)$  – same  
# structure."
```


Two Pointers

□ #16 3Sum Closest

MED[Open on LeetCode](#)

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array `nums` of length `n` and an integer `target`, find three integers at distinct indices in `nums` such that the sum is closest to `target`.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:

Input: `nums = [-1,2,1,-4]`, `target = 1`

Output: 2

Explanation: The sum that is closest to the target is 2. ($-1 + 2 + 1 = 2$).

Example 2:

Input: `nums = [0,0,0]`, `target = 1`

Output: `0`

Explanation: The sum that is closest to the target is `0`. (`0 + 0 + 0 = 0`).

Constraints:

- `3 <= nums.length <= 500`
- `-1000 <= nums[i] <= 1000`
- `-104 <= target <= 104`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find three distinct-index integers whose sum is closest to target. Return the sum.

Exactly one solution guaranteed – no tie-breaking needed. Right?"

#

"[-1,2,1,-4], target=1: possible sums:
 `-1+2+1=2`, `-1+2-4=-3`, `-1+1-4=-4`, `2+1-4=-1`.

Closest to 1 is 2. → 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

```
# Three nested loops – for every triple  
(i, j, k), compute the sum and track  
# whichever sum has the smallest absolute  
difference from target.  
# Time:  $O(n^3)$ . Space:  $O(1)$ . Should I go  
ahead and optimize?"
```

```
class _16_ThreeSumClosest_Brute:  
    def threeSumClosest(self, nums,  
target):  
        best = nums[0] + nums[1] + nums[2]  
        n = len(nums)  
        for i in range(n - 2):  
            for j in range(i + 1, n - 1):  
                for k in range(j + 1, n):  
                    s = nums[i] + nums[j]  
+ nums[k]  
                    if abs(target - s) <  
abs(target - best):  
                        best = s  
        return best
```

PHASE 3 — OPTIMIZE

**# "Exact same structure as 3Sum: sort +
fix one element + two pointers.**

**# Instead of looking for sum==0, track the
closest sum seen.**

```
# If sum > target → move right pointer  
left (decrease sum).  
# If sum < target → move left pointer  
right (increase sum).  
# If sum == target → return immediately  
(can't get closer).  
# Skip duplicate outer values to avoid  
redundant work – not required for  
correctness  
# here (no dedup of output), but speeds up  
the average case."
```

PHASE 4 — CLEAN CODE

```
class _16_ThreeSumClosest:  
    def threeSumClosest(self, nums:  
List[int], target: int) -> int:  
        nums.sort()  
        n = len(nums)  
        best = nums[0] + nums[1] + nums[2]  
        for i in range(n - 2):  
            if i > 0 and nums[i] == nums[i  
- 1]:  
                continue  
            left, right = i + 1, n - 1  
            while left < right:
```

```

        total = nums[i] +
nums[left] + nums[right]
        if total == target:
            return target
        if abs(target - total) <
abs(target - best):
            best = total
        if total > target:
            right -= 1
        else:
            left += 1
    return best

```

PHASE 5 — TEST & DEBUG

```

# nums=[-1,2,1,-4], target=1. sorted:
[-4,-1,1,2].
# i=0(-4): l=1,r=3. -4-1+2=-3. |1-(-3)|=4
< |1-(-4)|=5 → best=-3. sum<1→l++.
# l=2,r=3. -4+1+2=-1. |1-(-1)|=2 < 4 →
best=-1. sum<1→l++. l=r→stop.
# i=1(-1): l=2,r=3. -1+1+2=2. |1-2|=1 <
|1-(-1)|=2 → best=2. sum>1→r--. l=r→stop.
# → 2 ✓
# nums=[0,0,0], target=1. best=0. No
closer sum possible → 0 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$ extra ($O(\log n)$ for sort).

This is 3Sum (#15) with one change: track closest instead of exact zero.

The early return on exact match is a nice optimisation worth mentioning.

Given more time I'd ask: what if we want the k closest sums?

Collect all triple sums, sort by $|target - sum|$, return first k — $O(n^2 \log n)$."

Two Pointers

□ #31 Next Permutation

MED

[Open on LeetCode](#)

Array · Two Pointers

Lists: Grind 169

Problem

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`: `[1,2,3]`, `[1,3,2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3,1,2]`, `[3,2,1]`.

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of `arr = [1,2,3]` is `[1,3,2]`.
- Similarly, the next permutation of `arr = [2,3,1]` is `[3,1,2]`.
- While the next permutation of `arr = [3,2,1]` is `[1,2,3]` because `[3,2,1]` does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, find the next permutation of `nums`.

The replacement must be in place and use only constant extra memory.

Example 1:

Input: `nums = [1,2,3]`

Output: `[1,3,2]`

Example 2:

Input: `nums = [3,2,1]`

Output: `[1,2,3]`

Example 3:

Input: `nums = [1,1,5]`

Output: `[1,5,1]`

Constraints:

- $1 \leq \text{nums.length} \leq 100$

- $0 \leq \text{nums}[i] \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rearrange nums in-place to the next lexicographically greater permutation.

If no greater permutation exists (sorted descending), wrap to the smallest (ascending). Right?"

#

"[1,2,3]→[1,3,2]. [3,2,1]→[1,2,3].
[1,1,5]→[1,5,1]. [1,3,2]→[2,1,3]. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Generate all permutations of the array in sorted order, find the one that matches the current arrangement, then return the next one in the sorted list.

If the current is the last, wrap to the first (smallest) permutation.

Time: $O(n! * n)$ to generate all permutations – completely infeasible for $n > 10$.

```
# Space:  $O(n!)$ . Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Three steps — all  $O(n)$ ,  $O(1)$  space:
```

```
# 1. Find the rightmost 'dip': scan right to left for the largest  $i$  where  $\text{nums}[i] < \text{nums}[i+1]$ .
```

```
# If no dip exists, entire array is descending → just reverse it (wrap-around).
```

```
# 2. Find the smallest element to the right of  $i$  that is still larger than  $\text{nums}[i]$ .
```

```
# (The suffix is descending after step 1, so scan from the right.)
```

```
# 3. Swap  $\text{nums}[i]$  with that element, then reverse the suffix after  $i$ .
```

```
# Reversing the descending suffix makes it the smallest possible arrangement."
```

PHASE 4 — CLEAN CODE

```
class _31_NextPermutation:
```

```
    def nextPermutation(self, nums: List[int]) -> None:
```

```
        def reverse(left, right):
```

```
        while left < right:
            nums[left], nums[right] =
nums[right], nums[left]
            left += 1; right -= 1
    n = len(nums)
    # Step 1: find rightmost dip
    dip = next((i for i in range(n -
2, -1, -1) if nums[i] < nums[i + 1]), -1)
    if dip >= 0:
        # Step 2: find smallest
element to right of dip that is >
nums[dip]
        swap = next(j for j in range(n
- 1, -1, -1) if nums[j] > nums[dip])
        nums[dip], nums[swap] =
nums[swap], nums[dip]
    # Step 3: reverse suffix after dip
reverse(dip + 1, n - 1)
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3]: dip=1 (nums[1]=2<nums[2]=3).
swap=2 (nums[2]=3>nums[1]=2).
# swap: [1,3,2]. reverse(2,2)→no change. →
[1,3,2] ✓
# [3,2,1]: dip=-1. reverse(0,2): [1,2,3] ✓
```

```
# [1,1,5]: dip=1 (nums[1]=1<nums[2]=5).
swap=2 (5>1). swap→[1,5,1].
reverse(2,2)→[1,5,1] ✓
# [1,3,2]: dip=0 (nums[0]=1<nums[1]=3).
swap=1 (nums[1]=3>nums[0]=1; nums[2]=2
also>1 but
# scan right→left so j=1 first:
nums[1]=3>1? yes → swap=1? No wait j
starts from n-1=2:
# nums[2]=2>nums[0]=1 → swap=2.
swap→[2,3,1]. reverse(1,2): [2,1,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

The suffix is always descending after the dip – that's why scanning right-to-left

finds the smallest swap target efficiently.

The reverse of the suffix turns the largest possible suffix into the smallest – the key insight.

Given more time I'd ask: what's the previous permutation?

Same algorithm mirrored: find rightmost 'rise' ($\text{nums}[i] > \text{nums}[i+1]$), swap with

largest
element to its right that is still
smaller, reverse the suffix."

Two Pointers

□ #75 Sort Colors

MED[Open on LeetCode](#)

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`

Output: `[0,1,2]`

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- $\text{nums}[i]$ is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sort an array containing only 0, 1, 2 in-place without using library sort.

0=red, 1=white, 2=blue. Output: all 0s, then 1s, then 2s. Right?

Follow-up asks for a one-pass $O(1)$ space solution."

#

"[2,0,2,1,1,0] → [0,0,1,1,2,2] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

First pass: count the number of 0s, 1s, and 2s in the array.

Second pass: overwrite the array – fill $\text{count}[0]$ zeros, then $\text{count}[1]$ ones, then

```
count[2] twos.
```

```
# This is two passes but still  $O(n)$  time  
and  $O(1)$  space.
```

```
# Time:  $O(n)$ . Space:  $O(1)$ . Should I go  
ahead and optimize?"
```

```
class _75_SortColors_Brute:  
    def sortColors(self, nums):  
        c0 = nums.count(0)  
        c1 = nums.count(1)  
        for i in range(len(nums)):  
            if i < c0:  
                nums[i] = 0  
            elif i < c0 + c1:  
                nums[i] = 1  
            else:  
                nums[i] = 2
```

```
# PHASE 3 — OPTIMIZE
```

```
# "Dutch National Flag algorithm — one  
pass, three pointers: low, mid, high.
```

```
# Invariant: nums[0..low-1]=0,  
nums[low..mid-1]=1, nums[high+1..n-1]=2,  
nums[mid..high]=unknown.
```

```
# At each step examine nums[mid]:
```

```
# 0 → swap(low,mid), low++, mid++ (0  
placed, known-1 region grows both sides).
```



```
# 1 → mid++ (already correct).  
# 2 → swap(mid,high), high-- (don't mid++  
– swapped element is unknown)."
```

PHASE 4 — CLEAN CODE

```
class _75_SortColors:  
    def sortColors(self, nums: List[int])  
→ None:  
        low = mid = 0  
        high = len(nums) - 1  
        while mid <= high:  
            if nums[mid] == 0:  
                nums[low], nums[mid] =  
nums[mid], nums[low]  
                low += 1; mid += 1  
            elif nums[mid] == 1:  
                mid += 1  
            else:  
                nums[mid], nums[high] =  
nums[high], nums[mid]  
                high -= 1 # don't mid++ –  
swapped value is unexamined
```

PHASE 5 — TEST & DEBUG

```
# [2,0,2,1,1,0]: low=0,mid=0,high=5.
```

```
# mid=0(2): swap(0,5)→[0,0,2,1,1,2].  
high=4.  
# mid=0(0): swap(0,0)→same. low=1,mid=1.  
# mid=1(0): swap(1,1)→same. low=2,mid=2.  
# mid=2(2): swap(2,4)→[0,0,1,1,2,2].  
high=3.  
# mid=2(1): mid=3.  
# mid=3(1): mid=4. mid>high=3 → stop.  
# → [0,0,1,1,2,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ one pass. Space $O(1)$.

The 'don't mid++' after swapping a 2 is the subtlety – the swapped element from high

hasn't been examined yet and could be 0, 1, or 2.

After swapping a 0, we DO mid++ because nums[low] before the swap was definitely 1

(it's in the known-1 region), so after swapping, mid lands on a known-1 element.

Given more time I'd ask: what if there were k colours?

k=2 is a standard partition; k=3 is Dutch Flag; k>3 needs multiple passes or a radix approach."

Two Pointers

□ ★ #161 One Edit Distance ★

MED

[Open on LeetCode](#)

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given two strings *s* and *t*, return true if they are both one edit distance apart, otherwise return false.

A string *s* is said to be one distance apart from a string *t* if you can:

- Insert exactly one character into *s* to get *t*.
- Delete exactly one character from *s* to get *t*.
- Replace exactly one character of *s* with a different character to get *t*.

Example 1:

Input: *s* = "ab", *t* = "acb"

Output: true

Explanation: We can insert 'c' into *s* to get *t*.

Example 2:

Input: `s = "", t = ""`

Output: `false`

Explanation: We cannot get `t` from `s` by only one step.

Constraints:

- `0 <= s.length, t.length <= 104`
- `s` and `t` consist of lowercase letters, uppercase letters, and digits.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return true if `s` and `t` are exactly one edit apart – insert, delete, or replace.

Important: zero edits (equal strings) returns false. Right?

`s='ab', t='acb' → insert 'c' → true ✓.`
`s='', t='' → false ✓."`

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Run full Levenshtein DP between `s` and `t`; return True iff edit distance equals 1.

```
# Handles insert, delete, and replace in
one table.
# Correct but  $O(m \cdot n)$  when we only need to
detect a single edit.
# Time:  $O(m \cdot n)$ . Space:  $O(m \cdot n)$  DP
table.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two observations that short-circuit
work:
```

```
# If  $|\text{len}(s) - \text{len}(t)| > 1 \rightarrow$  impossible,
return false.
# Ensure s is the shorter string (swap if
needed) so we only handle two cases:
# same length  $\rightarrow$  exactly one replacement,
rest identical.
# s shorter by 1  $\rightarrow$  one deletion from t (or
insertion into s), rest identical.
# Scan left to right. On first mismatch:
# same length  $\rightarrow$  check  $s[i+1:] == t[i+1:]$ .
# diff length  $\rightarrow$  check  $s[i:] == t[i+1:]$ .
# If no mismatch found  $\rightarrow$  true only if t is
exactly 1 longer."
```

PHASE 4 — CLEAN CODE

```
class _161_OneEditDistance:
    def isOneEditDistance(self, s: str,
t: str) -> bool:
        def check(shorter, longer): #
len(longer) == len(shorter) + 1
            for i in range(len(shorter)):
                if shorter[i] !=
longer[i]:
                    return shorter[i:] ==
longer[i + 1:]
            return True # shorter is
prefix of longer
        m, n = len(s), len(t)
        if abs(m - n) > 1:
            return False
        if m == n:
            diffs = sum(a != b for a, b in
zip(s, t))
            return diffs == 1
        if m > n:
            s, t = t, s
            m, n = n, m
        return check(s, t)
```

PHASE 5 — TEST & DEBUG

```
# s='ab', t='acb': m=2,n=3. m<n →  
check('ab','acb').  
# i=0: 'a'=='a'. i=1: 'b'!='c'. return  
'b'=='b'? shorter[1:]='b', longer[2:]='b'  
→ True ✓  
# s='', t='': diffs=0 → return 0==1 →  
False ✓  
# s='a', t='a': same length, diffs=0 →  
False ✓  
# s='a', t='b': same length, diffs=1 →  
True ✓  
# s='abc', t='abc': diffs=0 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\min(m,n))$. Space $O(1)$ for the check (slice comparison is $O(\min)$ under the hood).

The key boundary: we return true when no mismatch found in the same-length case only if

$\text{diffs}==1$; in the diff-length case only if shorter is a prefix (true only if t is exactly

one char longer, which we already guaranteed).

Given more time I'd ask: what if we want EXACTLY k edits?
Full edit distance DP – $O(m*n)$ time, $O(\min(m,n))$ space."

Two Pointers

□ #167 Two Sum II – Input Array Is Sorted

MED[Open on LeetCode](#)

Array · Two Pointers · Binary Search

Lists: Denny Zhang

Problem

Given a 1-indexed array of integers `numbers` that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return the indices of the two numbers `index1` and `index2`, each incremented by one, as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6

Output: [1,3]

Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1

Output: [1,2]

Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- $2 \leq \text{numbers.length} \leq 3 \times 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- numbers is sorted in non-decreasing order.
- $-1000 \leq \text{target} \leq 1000$

- The tests are generated such that there is exactly one solution.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sorted array (1-indexed). Return [index1, index2] where numbers[index1]+numbers[index2]==target.

Exactly one solution. Must use $O(1)$ extra space – so no hash map. Right?"

#

"[2,7,11,15], target=9: 2+7=9 → [1,2] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j) where $j > i$, check if numbers[i] + numbers[j] == target.

Return [i+1, j+1] (1-indexed) when found. The problem guarantees exactly one solution.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

class _167_TwoSumII_Brute:

def twoSum(self, numbers, target):

```
        for i in range(len(numbers)):
            for j in range(i + 1,
len(numbers)):
                if numbers[i] +
numbers[j] == target:
                    return [i + 1, j + 1]
            return []
```

PHASE 3 — OPTIMIZE

"Two pointers exploit sorted order:
start left=0, right=n-1.

Sum too small → move left right
(increase sum).

Sum too large → move right left
(decrease sum).

Exactly one solution guaranteed → we
will always find it.

Time $O(n)$. Space $O(1)$."

PHASE 4 — CLEAN CODE

```
class _167_TwoSumII:
    def twoSum(self, numbers: List[int],
target: int) -> List[int]:
        left, right = 0, len(numbers) - 1
        while left < right:
```

```
        total = numbers[left] +
numbers[right]
        if total == target:
            return [left + 1, right +
1] # 1-indexed
        elif total < target:
            left += 1
        else:
            right -= 1
    return []
```

PHASE 5 — TEST & DEBUG

[2,7,11,15], target=9: l=0,r=3.

2+15=17>9→r=2. 2+11=13>9→r=1.

2+7=9→return [1,2] ✓

[2,3,4], target=6: l=0,r=2.

2+4=6→return [1,3] ✓

[-1,0], target=-1: -1+0=-1→return [1,2]

✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

This is the canonical reason to sort before running two pointers on Two Sum.

If the array weren't sorted, we'd need a hash map — $O(n)$ space.

Sorting adds $O(n \log n)$ but enables $O(1)$ space – a genuine trade-off.

Given more time I'd ask: what if there could be multiple answers?

Continue after finding a match, skip duplicates – same as 3Sum's dedup logic."

Two Pointers

★ #186 Reverse Words in a String II ★

MED[Open on LeetCode](#)

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given a character array *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1:

```
Input: s = ["t","h","e"," ","s","k","y"," ","i","s"," ","b","l","u","e"]
Output: ["b","l","u","e"," ","i","s"," ","s","k","y"," ","t","h","e"]
```

Example 2:

Input: `s = ["a"]`

Output: `["a"]`

Constraints:

- $1 \leq s.length \leq 105$
- `s[i]` is an English letter (uppercase or lowercase), digit, or space ' '.
- There is at least one word in `s`.
- `s` does not contain leading or trailing spaces.
- All the words in `s` are guaranteed to be separated by a single space.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a character ARRAY (not a string), reverse the order of words in-place.

Words are separated by single spaces. No leading/trailing spaces. Right?

`['t', 'h', 'e', ' ', 's', 'k', 'y'] → ['s', 'k', 'y', ' ', 't', 'h', 'e'] ✓`

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.


```
# Convert the character array to a string,
split by spaces to get words, reverse the
word list,
# rejoin with spaces, then write each
character back into the original array.
# Time: O(n). Space: O(n) – we create a
full string copy. Should I go ahead and
optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two-step in-place reversal with O(1)
extra space:
```

```
# Step 1: Reverse the entire array.
# Step 2: Reverse each individual word
back to its correct orientation.
# Example: 'the sky is blue' → reverse all
→ 'eulb si yks eht' → reverse words →
'blue is sky the' ✓"
```

PHASE 4 — CLEAN CODE

```
class _186_ReverseWordsII:
    def reverseWords(self, s: List[str])
→ None:
        def reverse(left, right):
            while left < right:
```

```

        s[left], s[right] =
s[right], s[left]
        left += 1; right -= 1
reverse(0, len(s) - 1)
start = 0
for i in range(len(s) + 1):
    if i == len(s) or s[i] == ' ':
        reverse(start, i - 1)
        start = i + 1

```

PHASE 5 — TEST & DEBUG

```

# s=['t','h','e',' ','s','k','y',' ',
#   'i','s',' ','b','l','u','e']:
# Step 1 reverse all: ['e','l','u','b',' ',
#   's','i',' ','y','k','s',' ',
#   'e','h','t']
# Step 2 reverse words:
# i=0..3 'e','l','u','b' → reverse →
# 'b','l','u','e'
# i=5..6 's','i' → 'i','s'
# i=8..10 'y','k','s' → 's','k','y'
# i=12..14 'e','h','t' → 't','h','e'
# → ['b','l','u','e',' ','i','s',' ',
#   's','k','y',' ','t','h','e'] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$ – two reversal passes, no extra memory.

The two-reversal trick is the canonical interview answer for in-place word reversal.

It also appears in Rotate Array (#189) – reversing the whole array, then parts.

Given more time I'd ask: what if words can be separated by multiple spaces?

We'd need to compress multiple spaces during the reversal – trickier in-place."

Two Pointers

#189 Rotate Array

MED[Open on LeetCode](#)

Array · Math · Two Pointers

Lists: Grind 169

Problem

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3`

Output: `[5,6,7,1,2,3,4]`

Explanation:

rotate 1 steps to the right:

`[7,1,2,3,4,5,6]`

rotate 2 steps to the right:

`[6,7,1,2,3,4,5]`

rotate 3 steps to the right:

`[5,6,7,1,2,3,4]`

Example 2:

Input: `nums = [-1,-100,3,99]`, `k = 2`

Output: `[3,99,-1,-100]`

Explanation:

rotate 1 steps to the right:

`[99,-1,-100,3]`

rotate 2 steps to the right:

`[3,99,-1,-100]`

Constraints:

- `1 <= nums.length <= 105`
- `-231 <= nums[i] <= 231 - 1`
- `0 <= k <= 105`

Follow up:

- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate `nums` to the right by `k` steps in-place. `k` can exceed `n` — so `k %= n`. Right?"

```
# [1,2,3,4,5,6,7], k=3 → [5,6,7,1,2,3,4]
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to
lock in the logic.
```

```
# Rotate the array one position to the
right, k times in a row.
```

```
# Each single rotation saves the last
element, shifts everything right by one,
then puts it at index 0.
```

```
# Time:  $O(n*k)$  – for  $k=n-1$  this is  $O(n^2)$ .
Space:  $O(1)$ . Should I go ahead and
optimize?"
```

```
class _189_RotateArray_Brute:
```

```
    def rotate(self, nums, k):
```

```
        n = len(nums)
```

```
        k %= n
```

```
        for _ in range(k):
```

```
            last = nums[-1]
```

```
            for i in range(n - 1, 0, -1):
```

```
                nums[i] = nums[i - 1]
```

```
            nums[0] = last
```

PHASE 3 — OPTIMIZE (three-reversal trick)

```
# "Same pattern as Reverse Words II:
```

```
# k %= n (handle k >= n).  
# Reverse entire array. Reverse first k  
elements. Reverse remaining n-k elements.  
# Time O(n). Space O(1).  
# Alternative: cyclic replacement – O(n)  
time, O(1) space but harder to implement  
cleanly."
```

PHASE 4 — CLEAN CODE

```
class _189_RotateArray:  
    def rotate(self, nums: List[int], k:  
int) -> None:  
        def reverse(left, right):  
            while left < right:  
                nums[left], nums[right] =  
nums[right], nums[left]  
                left += 1; right -= 1  
        n = len(nums)  
        k %= n  
        reverse(0, n - 1)  
        reverse(0, k - 1)  
        reverse(k, n - 1)
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4,5,6,7], k=3:  
# k%=7=3. reverse(0,6): [7,6,5,4,3,2,1].
```

```
# reverse(0,2): [5,6,7,4,3,2,1].
# reverse(3,6): [5,6,7,1,2,3,4] ✓
# [-1,-100,3,99], k=2:
# k%=4=2. reverse all: [99,3,-100,-1].
reverse(0,1): [3,99,-100,-1].
reverse(2,3): [3,99,-1,-100] ✓
# k=0: k%n=0. reverse all, reverse(0,-1)
no-op, reverse(0,n-1) reverses back → no
change ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

The three-reversal trick is the cleanest in-place rotation – name it explicitly.

The $k\%=n$ guard handles $k \geq n$ where full rotations cancel out.

The follow-up mentions three approaches: extra array $O(n)$ space, cyclic replacement $O(1)$,

and three-reversal $O(1)$. State all three and recommend the reversal for interviews.

Given more time I'd ask: rotate left instead of right?

Reverse first k , reverse rest, reverse all – just change the order."

Two Pointers

□ #532 K-diff Pairs in an Array

MED

[Open on LeetCode](#)

Array · Hash Table · Two Pointers · Binary Search · Sorting

Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`, return the number of unique `k`-diff pairs in the array.

A `k`-diff pair is an integer pair (`nums[i]`, `nums[j]`), where the following are true:

- $0 \leq i, j < \text{nums.length}$
- $i \neq j$
- $|\text{nums}[i] - \text{nums}[j]| == k$

Notice that `|val|` denotes the absolute value of `val`.

Example 1:

Input: `nums = [3,1,4,1,5]`, `k = 2`

Output: 2

Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).

Although we have two 1s in the input, we should only return the number of unique pairs.

Example 2:

Input: `nums = [1,2,3,4,5]`, `k = 1`

Output: 4

Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4, 5).

Example 3:

Input: `nums = [1,3,1,5,4]`, `k = 0`

Output: 1

Explanation: There is one 0-diff pair in the array, (1, 1).

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^7 \leq \text{nums}[i] \leq 10^7$
- $0 \leq k \leq 10^7$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count UNIQUE pairs (i,j) where $|\text{nums}[i] - \text{nums}[j]| = k$ and $i \neq j$.

Unique means unique value pairs – (1,3) and (3,1) count once.

$k=0$ special case: need TWO copies of the same value. Right?"

#

"[3,1,4,1,5], $k=2$: unique pairs (1,3) and (3,5) → 2 ✓

[1,3,1,5,4], $k=0$: need duplicate values. Only 1 appears twice → (1,1) → 1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j) where $i \neq j$, check if $|\text{nums}[i] - \text{nums}[j]| == k$.

Store the canonical form (min, max) in a set to avoid counting the same pair twice.

Time: $O(n^2)$. Space: $O(n)$ for the dedup set. Should I go ahead and optimize?"

```
class _532_KdiffPairs_Brute:
```

```
    def findPairs(self, nums, k):
```

```
    pairs = set()
    for i in range(len(nums)):
        for j in range(len(nums)):
            if i != j and abs(nums[i]
- nums[j]) == k:
                pairs.add((min(nums[i]
], nums[j]), max(nums[i], nums[j])))
    return len(pairs)
```

PHASE 3 — OPTIMIZE

"Hash set approach – $O(n)$ time, $O(n)$ space:

Walk through nums. For each x, check if x-k is already in 'visited'.

If yes, add min(x, x-k) to the answer set (canonical pair).

Also check x+k in visited (symmetric check handles both directions).

Using a set for answers automatically deduplicates.

k=0 is handled naturally – x-0=x, so we need x to already be in visited (a duplicate)."

PHASE 4 — CLEAN CODE

```
class _532_KdiffPairs:
```

```
def findPairs(self, nums: List[int],
k: int) -> int:
    answers : set = set()
    visited : set = set()
    for x in nums:
        if x - k in visited:
            answers.add(x - k) #
canonical smaller element
        if x + k in visited:
            answers.add(x) # x is the
larger; x+k was seen before
        visited.add(x)
    return len(answers)
```

PHASE 5 — TEST & DEBUG

[3,1,4,1,5], k=2:

x=3: 3-2=1 not in visited. 3+2=5 not in visited. visited={3}.

x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1). visited={3,1}.

x=4: 4-2=2 not in visited. 4+2=6 not in visited. visited={3,1,4}.

x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1) (already there). visited={3,1,4}.

```
# x=5: 5-2=3 in visited → answers.add(3).  
5+2=7 not. visited={3,1,4,5}.  
# answers={1,3} → 2 ✓  
# [1,3,1,5,4], k=0:  
# x=1: 1-0=1 not in visited. visited={1}.  
# x=3: 3-0=3 not in visited.  
visited={1,3}.  
# x=1: 1-0=1 in visited → answers.add(1).  
visited={1,3}.  
# answers={1} → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$  for the two sets.  
# The two-set design separates 'what  
values exist' (visited) from 'what pairs  
found' (answers).  
# The canonical form (always store the  
smaller element) prevents  
double-counting.  
# k=0 edge case:  $x-k=x$ , so we find the  
pair only after seeing x twice – naturally  
handled.  
# Given more time I'd ask: what if we want  
the actual pairs, not just the count?  
# Store tuples (x-k, x) in the answers set  
instead of single values."
```

Two Pointers

□ #923 3Sum With Multiplicity

MED[Open on LeetCode](#)

Array · Hash Table · Two Pointers · Sorting · Counting

Lists: CodeSignal

Problem

Given an integer array `arr`, and an integer `target`, return the number of tuples `i, j, k` such that $i < j < k$ and $arr[i] + arr[j] + arr[k] == target$.

As the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

Input: arr = [1,1,2,2,3,3,4,4,5,5],
target = 8

Output: 20

Explanation:

Enumerating by the values (arr[i],
arr[j], arr[k]):

(1, 2, 5) occurs 8 times;

(1, 3, 4) occurs 8 times;

(2, 2, 4) occurs 2 times;

(2, 3, 3) occurs 2 times.

Example 2:

Input: arr = [1,1,2,2,2,2], target = 5

Output: 12

Explanation:

arr[i] = 1, arr[j] = arr[k] = 2 occurs
12 times:

We choose one 1 from [1,1] in 2 ways,
and two 2s from [2,2,2,2] in 6 ways.

Example 3:

Input: arr = [2,1,3], target = 6

Output: 1

Explanation: (1, 2, 3) occurred one time
in the array so we return 1.

Constraints:

- $3 \leq \text{arr.length} \leq 3000$
- $0 \leq \text{arr}[i] \leq 100$
- $0 \leq \text{target} \leq 300$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count tuples (i,j,k) with $i < j < k$ and $\text{arr}[i] + \text{arr}[j] + \text{arr}[k] = \text{target}$.

Return count modulo $10^9 + 7$. Values are 0–100 so we can use frequency counting.

Right?

Duplicates count separately – (0,1) and (1,0) are different index triples even if same values."

#

"[1,1,2,2,3,3,4,4,5,5], target=8 → 20 (combinations of value triples summing to 8) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Three nested loops – for every triple (i, j, k) with $i < j < k$, check if

```
# arr[i] + arr[j] + arr[k] == target and
count the matches.
# Time:  $O(n^3)$ . For  $n=3000$  that's 27
billion – too slow. Space:  $O(1)$ .
# Should I go ahead and optimize?"
class _923_ThreeSumMulti_Brute:
    def threeSumMulti(self, arr, target):
        MOD = 10**9 + 7
        count = 0
        n = len(arr)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                for k in range(j + 1, n):
                    if arr[i] + arr[j] +
arr[k] == target:
                        count += 1
        return count % MOD

# PHASE 3 — OPTIMIZE
# "Walk through arr as the middle element
j. Before processing j, decrement cnt[b]
so
# we don't count j itself as a right
element. For each  $a < j$ , the required  $c =$ 
target-a-b.
```

```
# cnt[c] tells us how many valid right
elements c exist to the right of j.
# Time:  $O(n^2)$  with frequency lookups.
Space:  $O(n)$  for the Counter."
```

PHASE 4 — CLEAN CODE

```
class _923_ThreeSumMulti:
    def threeSumMulti(self, arr:
List[int], target: int) -> int:
        MOD = 10**9 + 7
        freq = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            freq[b] -= 1 # exclude j
            # itself from right elements
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + freq[c]) %
MOD
        return ans
```

PHASE 5 — TEST & DEBUG

```
# arr=[2,1,3], target=6:
freq={2:1,1:1,3:1}.
# j=0(b=2): freq[2]=0. no a < 0.
(nothing).
```

```

# j=1(b=1): freq[1]=0. a=2: c=6-2-1=3.
freq[3]=1 → ans=1.
# j=2(b=3): freq[3]=0. a=2: c=6-2-3=1.
freq[1]=0→0. a=1: c=6-1-3=2.
freq[2]=1→ans=2?
# Wait – expected 1. Let me recheck:
arr=[2,1,3]. Only valid triple is (0,1,2)
→ (2,1,3).
# j=1(b=1): a=arr[0]=2. c=6-2-1=3.
freq[3]=1 (yes, index 2 exists) → ans=1. ✓
# j=2(b=3): freq[3]=0. a=arr[0]=2:
c=6-2-3=1. freq[1]=0 (decremented at
j=1). 0.
# a=arr[1]=1: c=6-1-3=2. freq[2]=1 →
ans=2? But expected 1.
# Ah – freq[2] was decremented when j=0:
freq[2]=0, not 1. Let me re-trace:
# Initial freq={2:1,1:1,3:1}.
# j=0: freq[2]-=1 → freq={2:0,1:1,3:1}. no
a<j=0.
# j=1: freq[1]-=1 → freq={2:0,1:0,3:1}.
a=2: c=3. freq[3]=1 → ans=1.
# j=2: freq[3]-=1 → freq={2:0,1:0,3:0}.
a=2: c=1. freq[1]=0. a=1: c=2. freq[2]=0.
# → ans=1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n)$ for the Counter.
The key invariant: at step j , freq counts only elements at index $> j$.
This ensures we count each valid (i,j,k) tuple exactly once with $i < j < k$.
Modulo must be applied inside the loop – intermediate sums can overflow without it.
Given more time I'd ask: can we do $O(n)$ using the value constraints $(0-100)$?
Yes – iterate over all value triples (a,b,c) with $a+b+c=target$ and compute combinations
 $C(cnt[a],1) * C(cnt[b],1) * C(cnt[c],1)$ with dedup for equal values."

Two Pointers

□ #986 Interval List Intersections

MED[Open on LeetCode](#)

Array · Two Pointers

Lists: Denny Zhang

Problem

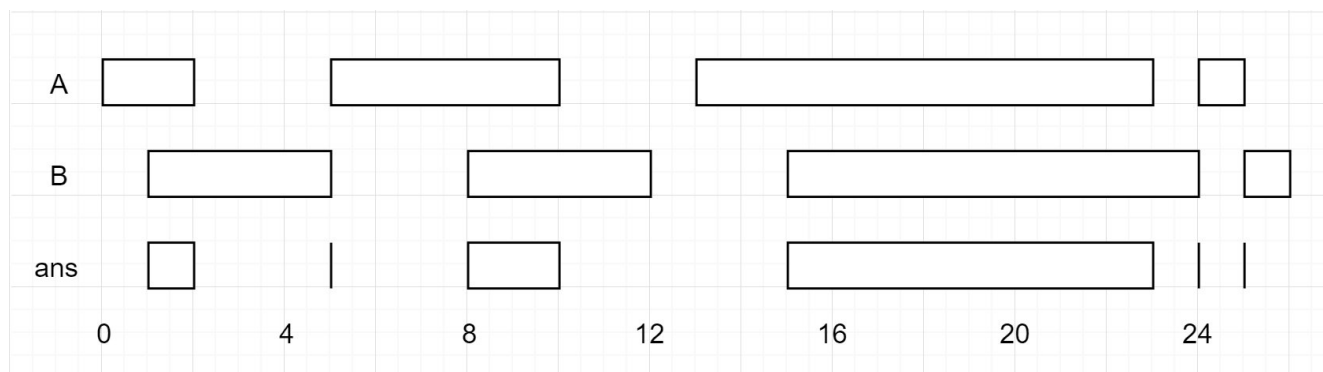
You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [starti, endi]` and `secondList[j] = [startj, endj]`. Each list of intervals is pairwise disjoint and in sorted order.

Return the intersection of these two interval lists.

A closed interval $[a, b]$ (with $a \leq b$) denotes the set of real numbers x with $a \leq x \leq b$.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of $[1, 3]$ and $[2, 4]$ is $[2, 3]$.

Example 1:



Input: firstList =

`[[0,2],[5,10],[13,23],[24,25]],`

secondList =

`[[1,5],[8,12],[15,24],[25,26]]`

Output: `[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]`

Example 2:

Input: firstList = `[[1,3],[5,9]]`,

secondList = `[]`

Output: `[]`

Constraints:

- $0 \leq \text{firstList.length}, \text{secondList.length} \leq 1000$
- $\text{firstList.length} + \text{secondList.length} \geq 1$
- $0 \leq \text{start}_i < \text{end}_i \leq 109$
- $\text{end}_i < \text{start}_i + 1$
- $0 \leq \text{start}_j < \text{end}_j \leq 109$

- $endj < startj + 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two sorted, disjoint interval lists. Return their intersection – all overlapping segments.

Intersection of $[a,b]$ and $[c,d]$ is $[\max(a,c), \min(b,d)]$ if $\max(a,c) \leq \min(b,d)$. Right?

Either list can be empty."

#

" $[[0,2],[5,10]] \cap [[1,5],[8,12]] \rightarrow [1,2],[5,5],[8,10] \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair of intervals (one from each list), compute the intersection:

$start = \max(a[0], b[0])$, $end = \min(a[1], b[1])$. If $start \leq end$, it's a valid intersection.

This is $O(m \times n)$ – it ignores the sorted property entirely.

Time: $O(m*n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _986_IntervalIntersections_Brute:
    def intervalIntersection(self,
firstList, secondList):
        result = []
        for a in firstList:
            for b in secondList:
                start = max(a[0], b[0])
                end = min(a[1], b[1])
                if start <= end:
                    result.append([start,
end])
        return result
```

PHASE 3 — OPTIMIZE

"Two pointers, one per list. At each step:

Compute the intersection of the current pair.

If valid ($start \leq end$), record it.

Advance the pointer whose interval ends FIRST – that interval can't intersect any later interval

of the other list (they're sorted and disjoint).

Time: $O(m+n)$. Space: $O(1)$ extra."

PHASE 4 — CLEAN CODE

```
class _986_IntervalIntersections:
    def intervalIntersection(self,
firstList: List[List[int]], secondList:
List[List[int]]) -> List[List[int]]:
        result = []
        i = j = 0
        while i < len(firstList) and j <
len(secondList):
            start = max(firstList[i][0],
secondList[j][0])
            end = min(firstList[i][1],
secondList[j][1])
            if start <= end:
                result.append([start,
end])
                if firstList[i][1] <
secondList[j][1]:
                    i += 1
                else:
                    j += 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# firstList=[[0,2],[5,10]],
secondList=[[1,5],[8,12]]:
# i=0,j=0: start=max(0,1)=1,
end=min(2,5)=2. 1<=2→[1,2]. first ends at
2<5→i=1.
# i=1,j=0: start=max(5,1)=5,
end=min(10,5)=5. 5<=5→[5,5]. first ends
at 10>5→j=1.
# i=1,j=1: start=max(5,8)=8,
end=min(10,12)=10. 8<=10→[8,10]. first
ends at 10<12→i=2.
# i=2 out of bounds → stop. →
[[1,2],[5,5],[8,10]] ✓
# secondList=[]: loop never enters → [] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(m+n)$  – each pointer advances at
most  $m+n$  times.
# Space  $O(\min(m,n))$  for the output in the
worst case.
# The 'advance the earlier-ending pointer'
rule is the key: the interval that ends
first
# cannot intersect any future interval of
the other list (both lists are sorted and
disjoint).
```

Given more time I'd ask: what if lists aren't sorted?
Sort both first – $O((m+n)\log(m+n))$ – then apply the same two-pointer approach."

Two Pointers

□ ★ #1055 Shortest Way to Form String ★

MED

[Open on LeetCode](#)

Two Pointers · String · Binary Search

Lists: ★ Premium | Premium 98

Problem

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings `source` and `target`, return the minimum number of subsequences of `source` such that their concatenation equals `target`. If the task is impossible, return `-1`.

Example 1:

Input: source = "abc", target = "abcbc"

Output: 2

Explanation: The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".

Example 2:

Input: source = "abc", target = "acdbc"

Output: -1

Explanation: The target string cannot be constructed from the subsequences of source string due to the character "d" in target string.

Example 3:

Input: source = "xyz", target = "xzyxz"

Output: 3

Explanation: The target string can be constructed as follows "xz" + "y" + "xz".

Constraints:

- $1 \leq \text{source.length}, \text{target.length} \leq 1000$
- source and target consist of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the minimum number of subsequences of source whose concatenation equals target.

A subsequence can skip characters but must preserve order.

Return -1 if any character in target doesn't exist in source at all. Right?"

#

"source='abc', target='abcbc': 'abc' + 'bc' → 2 ✓

source='abc', target='acdbc': 'd' not in source → -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Try every possible way to split target into consecutive segments, and for each split,

verify each segment is a subsequence of source. Track the minimum number of segments.

```
# This is exponential in the number of  
splits – completely infeasible.  
# Time:  $O(2^{|target|} * |source|)$ . Should I  
go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Greedy: each 'round', scan source once  
and match as many target characters as  
possible.
```

```
# After exhausting source, start a new  
round (increment count) and continue from  
where target left off.
```

```
# Pre-check: if any target char is absent  
from source, return -1.
```

```
# Time:  $O(|source| * |target|)$ . Space:  
 $O(1)$  extra."
```

PHASE 4 — CLEAN CODE

```
class _1055_ShortestWay:  
    def shortestWay(self, source: str,  
target: str) -> int:  
        if set(target) - set(source):  
            return -1  
        rounds = 0  
        target_idx = 0  
        while target_idx < len(target):
```



```

        for char in source:
            if target_idx <
len(target) and char ==
target[target_idx]:
                target_idx += 1
            rounds += 1
        return rounds

```

PHASE 5 — TEST & DEBUG

```

# source='abc', target='abcbc':
# Round 1: a→match t[0]='a'(idx=1).
b→match t[1]='b'(idx=2). c→match
t[2]='c'(idx=3). rounds=1.
# Round 2: a→no match(t[3]='b'). b→match
t[3]='b'(idx=4). c→match t[4]='c'(idx=5).
rounds=2.
# target_idx=5=len(target) → return 2 ✓
# source='xyz', target='xzyxz':
# Round1: x→t[0]='x'(1). y→no(t[1]='z').
z→t[1]='z'(2). rounds=1.
# Round2: x→no(t[2]='y'). y→t[2]='y'(3).
z→no(t[3]='x'). rounds=2.
# Round3: x→t[3]='x'(4). y→no(t[4]='z').
z→t[4]='z'(5). rounds=3. → 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(|source| * |target|)$. Space $O(1)$.

The pre-check eliminates the -1 case cleanly before the main loop.

Optimisation: precompute for each (source_pos, char) the next position in source

where char appears – binary search brings each round down to $O(|target| \log |source|)$.

Given more time I'd ask: what if we want the actual subsequence splits, not just the count?

Track the split points during the greedy pass."

Two Pointers

□ #2563 Count the Number of Fair Pairs

MED[Open on LeetCode](#)

Array · Two Pointers · Binary Search · Sorting
Lists: CodeSignal

Problem

Given a 0-indexed integer array `nums` of size `n` and two integers `lower` and `upper`, return the number of fair pairs.

A pair (i, j) is fair if:

- $0 \leq i < j < n$, and
- $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$

Example 1:

Input: `nums = [0,1,7,4,4,5]`, `lower = 3`,
`upper = 6`

Output: 6

Explanation: There are 6 fair pairs:

$(0,3)$, $(0,4)$, $(0,5)$, $(1,3)$, $(1,4)$, and
 $(1,5)$.

Example 2:

Input: `nums = [1,7,9,2,5]`, `lower = 11`,
`upper = 11`

Output: 1

Explanation: There is a single fair
pair: (2,3).

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums.length} == n$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{lower} \leq \text{upper} \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count pairs (i,j) with $i < j$ and $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$. Return the count. Right?"

Values can be negative. n up to 10^5 so $O(n^2)$ is too slow."

#

"[0,1,7,4,4,5], lower=3, upper=6: 6 pairs ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j) with $i < j$, check if $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$.

Count all pairs that satisfy both bounds. For $n=10^5$ this is $5 \cdot 10^9$ comparisons – too slow.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

class _2563_FairPairs_Brute:

def countFairPairs(self, nums, lower, upper):

count = 0

for i in range(len(nums)):

for j in range(i + 1,

len(nums)):

s = nums[i] + nums[j]

if lower <= s <= upper:

count += 1

return count

PHASE 3 — OPTIMIZE

"Sort nums. For each i, binary search for the range of valid $j > i$:

$\text{lower_bound}(\text{nums}, \text{lower} - \text{nums}[i], i+1) \rightarrow$ first j where $\text{nums}[j] \geq \text{lower} - \text{nums}[i]$.

```
# upper_bound(nums, upper-nums[i], i+1) →  
# first j where nums[j] > upper-nums[i].  
# Count = upper_bound - lower_bound. Sum  
# over all i.  
# Time: O(n log n). Space: O(1) extra.  
# Use inner helpers for the two binary  
# search variants."
```

PHASE 4 — CLEAN CODE

```
class _2563_FairPairs:  
    def countFairPairs(self, nums:  
List[int], lower: int, upper: int) -> int:  
        def lower_bound(start: int,  
target: int) -> int:  
            left, right = start,  
len(nums)  
            while left < right:  
                mid = (left + right) // 2  
                if nums[mid] >= target:  
                    right = mid  
                else:  
                    left = mid + 1  
            return left  
        def upper_bound(start: int,  
target: int) -> int:
```

```
        left, right = start,
len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > target:
                right = mid
            else:
                left = mid + 1
        return left
    nums.sort()
    count = 0
    for i in range(len(nums)):
        lo = lower_bound(i + 1, lower
- nums[i])
        hi = upper_bound(i + 1, upper
- nums[i])
        count += hi - lo
    return count
```

PHASE 5 — TEST & DEBUG

nums=[0,1,4,4,5,7] (sorted), lower=3, upper=6:

i=0(0): need j where 3<=nums[j]<=6.

lower_bound(1,3)→idx of 4=2.

upper_bound(1,7)→idx of 7=5. count+=3.

```
# i=1(1): need 2<=nums[j]<=5. lb(2,2)→2.  
ub(2,6)→5. count+=3. total=6.  
# ... → 6 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$  – sort +  $n$  binary  
searches each  $O(\log n)$ .
```

```
# Space  $O(1)$  extra.
```

```
# lower_bound finds first index  $\geq$  target;  
upper_bound finds first index  $>$  target.
```

```
# Their difference is the count of indices  
in the valid range.
```

```
# Alternative: two-pointer approach – for  
each  $i$ , maintain  $lo$  and  $hi$  pointers
```

```
# across iterations (they only move right)  
→  $O(n)$  total after sorting.
```

```
# Given more time I'd ask: what if we want  
the actual pairs?
```

```
# Collect them during the binary search  
loops –  $O(n + \text{output})$  time."
```


Sliding Window

Round 2 · High · Medium · 8 questions

Sliding Window

□ #3 Longest Substring Without Repeating Characters

MED[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given a string *s*, find the length of the longest substring without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: `s = "pwwkew"`

Output: `3`

Explanation: The answer is `"wke"`, with the length of `3`.

Notice that the answer must be a substring, `"pwke"` is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- `s` consists of English letters, digits, symbols and spaces.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the length of the longest substring with all unique characters."

Substring is contiguous. Empty string returns `0`. Right?

Characters can be any printable ASCII – not just lowercase letters."

#

"'abcabcb' → 3 ('abc'). 'bbbbbb' → 1 ('b'). 'pwwkew' → 3 ('wke') ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Enumerate every substring with two indices left and right.

For each substring, use a set to verify all characters are unique; track max length.

Correct, but $O(n^2)$ substrings each with up to $O(n)$ uniqueness check.

Time: $O(n^2)$ to $O(n^3)$. Space: $O(\min(n, \text{alphabet}))$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sliding window with a frequency map.

Expand right: add `s[right]` to the map.

When a duplicate appears (`count > 1`), shrink from the left until it's gone.

At each step, update the max window size.

Time: $O(n)$. Space: $O(\min(n, \text{alphabet}))$ — at most 128 unique ASCII chars."

PHASE 4 — CLEAN CODE

`class _3_LongestSubstringNoRepeat:`

```
def lengthOfLongestSubstring(self, s:
str) -> int:
    freq : defaultdict =
defaultdict(int)
    left = 0
    longest = 0
    for right in range(len(s)):
        freq[s[right]] += 1
        while freq[s[right]] > 1: #
shrink until no duplicate
            freq[s[left]] -= 1
            left += 1
        longest = max(longest, right
- left + 1)
    return longest
```

PHASE 5 — TEST & DEBUG

```
# s='abcabcbb':
# r=0('a'): freq={a:1}. longest=1.
# r=1('b'): freq={a:1,b:1}. longest=2.
# r=2('c'): freq={a:1,b:1,c:1}.
longest=3.
# r=3('a'): freq[a]=2. shrink:
freq[a]-=1→1, left=1. longest=3
(3-1+1=3).
```

```
# r=4('b'): freq[b]=2. shrink:
freq[b]-=1→1, left=2. longest=3.
# r=5('c'): freq[c]=2. shrink: left=3.
longest=3.
# r=6('b'): freq[b]=2. shrink: left=4.
longest=3.
# r=7('b'): freq[b]=2. shrink: left=5.
longest=max(3,3)=3. → 3 ✓
# s='': loop doesn't run → 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ — each character enters and leaves the window at most once.

Space $O(\min(n, 128))$ for the frequency map.

Alternative: store the LAST INDEX of each character instead of frequency.

When $s[\text{right}]$ already seen at idx and $\text{idx} \geq \text{left}$: jump $\text{left} = \text{idx} + 1$ directly

(one move instead of shrinking one at a time). Same $O(n)$ but fewer iterations.

Given more time I'd ask: what if we want the actual substring, not just its length?

Track left and best_left when updating the max."

Sliding Window

□ ★ #159 Longest Substring with At Most Two Distinct Characters ★

MED

[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: ★ Premium | Premium 98

Problem

Given a string *s*, return the length of the longest substring that contains at most two distinct characters.

Example 1:

Input: *s* = "eceba"

Output: 3

Explanation: The substring is "ece" which its length is 3.

Example 2:

Input: *s* = "ccaabbb"

Output: 5

Explanation: The substring is "aabbb" which its length is 5.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest substring that contains at most 2 distinct characters. Right?"

'eceba': 'ece' has 2 distinct $\rightarrow$ length 3
✓. 'ccaabbb': 'aabbb' has 2 $\rightarrow$ length 5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

For every pair (i, j) , count the distinct characters in $s[i..j]$ using a set.

If there are at most 2 distinct characters, update the max length.

Time: $O(n^2)$ pairs, $O(n)$ set check = $O(n^3)$. With running set it's $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

class

_159_LongestSubstringTwoDistinct_Brute:


```
def
lengthOfLongestSubstringTwoDistinct(self,
s):
    longest = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) <= 2:
                longest =
max(longest, j - i + 1)
            else:
                break
    return longest
```

PHASE 3 — OPTIMIZE

"Sliding window – same pattern as #3 but the shrink condition is 'more than 2 distinct chars'.

Expand right: add s[right] to frequency map.

Shrink from left while distinct chars > 2 (i.e. len(freq) > 2).

Update max window length. Time: O(n).

Space: O(1) – at most 3 keys in the map."

PHASE 4 — CLEAN CODE

```
class _159_LongestSubstringTwoDistinct:
    def
lengthOfLongestSubstringTwoDistinct(self,
s: str) -> int:
    freq : defaultdict =
defaultdict(int)
    left = 0
    longest = 0
    for right in range(len(s)):
        freq[s[right]] += 1
        while len(freq) > 2: # shrink
until at most 2 distinct
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
        longest = max(longest, right
- left + 1)
    return longest
```

PHASE 5 — TEST & DEBUG

```
# s='eceba':
# r=0('e'): freq={e:1}. longest=1.
# r=1('c'): freq={e:1,c:1}. longest=2.
# r=2('e'): freq={e:2,c:1}. longest=3.
```

```
# r=3('b'): freq={e:2,c:1,b:1} → 3
distinct. shrink:
# freq[e]-=1→1. left=1.
freq={e:1,c:1,b:1} still 3. shrink:
# freq[c]-=1→0 → del c. left=2.
freq={e:1,b:1}. 2 distinct → stop.
longest=max(3,2)=3.
# r=4('a'): freq={e:1,b:1,a:1} → 3.
shrink: freq[e]-=1→0→del e. left=3.
freq={b:1,a:1}. longest=3.
# → 3 ✓
# s='ccaabbbb':
# Window expands to 'ccaabbbb' but once 'b'
is added after 'a', we have c,a,b(3
distinct).
# Shrink past the c's. Eventually best
window is 'aabbbb'=5. → 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$ – at most 3 keys in the map at any time.

This is the template for 'at most k distinct characters' – change > 2 to $> k$.

Generalisation: Longest Substring with At Most K Distinct Characters (#340) is the same

```
# algorithm with k replacing 2.  
# The key difference from #3: shrink on  
len(freq) > k rather than on a specific  
duplicate.  
# Given more time I'd ask: what about  
EXACTLY 2 distinct characters?  
# atMost(k=2) - atMost(k=1) gives the  
count of substrings with exactly 2  
distinct."
```

Sliding Window

★ #340 Longest Substring with At Most K Distinct Characters ★ **MED**

[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , return the length of the longest substring of s that contains at most k distinct characters.

Example 1:

Input: $s = \text{"eceba"}, k = 2$

Output: 3

Explanation: The substring is "ece" with length 3.

Example 2:

Input: $s = \text{"aa"}, k = 1$

Output: 2

Explanation: The substring is "aa" with length 2.

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $0 \leq k \leq 50$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return the length of the longest substring with at most k distinct characters."

k=0 means the answer is 0 (no characters allowed). Right?

This is the direct generalisation of #159 (at most 2 distinct) with k replacing 2."

#

"'eceba', k=2 → 'ece', length 3 ✓. 'aa', k=1 → 'aa', length 2 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

For every starting index i, expand j rightward, track distinct characters with a set.

Stop when distinct chars exceed k. Track the maximum window length seen.

Time: $O(n^2)$. Space: $O(k)$. Should I go ahead and optimize?"

```
class
_340_LongestSubstringKDistinct_Brute:
    def
lengthOfLongestSubstringKDistinct(self,
s, k):
    if k == 0:
        return 0
    longest = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) <= k:
                longest =
max(longest, j - i + 1)
            else:
                break
    return longest
```

PHASE 3 — OPTIMIZE

"Sliding window – identical template to #159 with the constant 2 replaced by k.
Expand right, shrink from left whenever distinct chars exceed k.

Time: $O(n)$. Space: $O(k)$ – at most $k+1$ keys in the map at any moment."

PHASE 4 — CLEAN CODE

```
class _340_LongestSubstringKDistinct:
    def
lengthOfLongestSubstringKDistinct(self,
s: str, k: int) -> int:
    if k == 0:
        return 0
    freq : defaultdict =
defaultdict(int)
    left = 0
    longest = 0
    for right in range(len(s)):
        freq[s[right]] += 1
        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
        longest = max(longest, right
- left + 1)
    return longest
```

PHASE 5 — TEST & DEBUG


```
# s='eceba', k=2:  
# r=2('e'): freq={e:2,c:1}. 2 distinct →  
ok. longest=3.  
# r=3('b'): 3 distinct → shrink until  
freq={e:1,b:1}. longest stays 3.  
# r=4('a'): 3 distinct → shrink until  
freq={b:1,a:1}. longest=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(k)$ . Identical to  
#159 — just  $k$  instead of 2.
```

```
# Exactly  $k$  distinct: atMost( $k$ ) —  
atMost( $k-1$ ).
```

```
# Given more time I'd ask: what if  $k \geq$   
distinct chars in  $s$ ? Answer =  $\text{len}(s)$ ."
```

Sliding Window

□ #424 Longest Repeating Character Replacement

MED[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

Input: $s = \text{"ABAB"} , k = 2$

Output: 4

Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: `s = "AABABBA", k = 1`

Output: `4`

Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".

The substring "BBBB" has the longest repeating letters, which is 4.

There may exists other ways to achieve this answer too.

Constraints:

- `1 <= s.length <= 105`
- `s` consists of only uppercase English letters.
- `0 <= k <= s.length`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"We can replace at most `k` characters.
Find the longest substring where all
characters

are the same after at most `k`
replacements.

String is uppercase letters only. Right?

Strategy: keep the most frequent char,
replace everything else."

#

"'ABAB', k=2 → replace both A's or B's → 4 ✓

'AABABBA', k=1 → best window = 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j), count character frequencies in s[i..j]. Find the most frequent char.

If window_size - max_freq ≤ k, at most k replacements are needed – this window is valid.

Track the maximum valid window length.

Time: $O(n^2)$ pairs, $O(26)$ frequency count per window. Space: $O(1)$. Should I go ahead and optimize?"

class

_424_LongestRepeatingReplacement_Brute:

def characterReplacement(self, s, k):

longest = 0

for i in range(len(s)):

freq = [0] * 26

for j in range(i, len(s)):

freq[ord(s[j]) -

ord('A')] += 1

```
        max_freq = max(freq)
        if (j - i + 1) - max_freq
<= k:
            longest =
max(longest, j - i + 1)
    return longest
```

PHASE 3 — OPTIMIZE

"Sliding window tracking the most frequent character in the window.

Valid window: window_size - max_freq <= k (replace the rest).

When invalid, shrink left by 1 – but NEVER decrease max_freq.

We only care about finding a LONGER valid window, so we only update max_freq upward.

This means the window never shrinks below the current best – it slides as a fixed size

until a better window is found. Time: O(n). Space: O(26)=O(1)."

PHASE 4 — CLEAN CODE

```
class _424_LongestRepeatingReplacement:
```

```
def characterReplacement(self, s:
str, k: int) -> int:
    freq = [0] * 26
    max_freq = 0
    left = 0
    longest = 0
    for right in range(len(s)):
        freq[ord(s[right]) -
ord('A')] += 1
        max_freq = max(max_freq,
freq[ord(s[right]) - ord('A')])
        if (right - left + 1) -
max_freq > k: # too many replacements
needed
            freq[ord(s[left]) -
ord('A')] -= 1
            left += 1
        longest = max(longest, right
- left + 1)
    return longest

# PHASE 5 — TEST & DEBUG
# s='ABAB', k=2:
# r=0('A'): freq[A]=1,max=1. win=1.
1-1=0<=2. longest=1.
```

```
# r=1('B'): freq[B]=1,max=1. win=2.  
2-1=1<=2. longest=2.  
# r=2('A'): freq[A]=2,max=2. win=3.  
3-2=1<=2. longest=3.  
# r=3('B'): freq[B]=2,max=2. win=4.  
4-2=2<=2. longest=4. → 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

'Don't decrease max_freq' is the key insight: we never shrink below the best window seen.

The window slides forward at a fixed size until max_freq improves, then grows.

Given more time I'd ask: unlimited replacements ($k=n$)? Answer = len(s)."

Sliding Window

□ #438 Find All Anagrams in a String

MED[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given two strings *s* and *p*, return an array of all the start indices of *p*'s anagrams in *s*. You may return the answer in any order.

Example 1:

Input: *s* = "cbaebabacd", *p* = "abc"

Output: [0,6]

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: `s = "abab"`, `p = "ab"`

Output: `[0,1,2]`

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".

The substring with start index = 1 is "ba", which is an anagram of "ab".

The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$
- `s` and `p` consist of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all start indices in `s` where a substring is an anagram of `p`."

Fixed window size = `len(p)`. Lowercase letters only. Right?"

#

"`s='cbaebabacd'`, `p='abc'` → `[0,6]` ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Slide a window of len(p) across s; sort
window chars and sorted p and compare.
# Every window costs  $O(m \log m)$  for
sorting.
# Frequency-count comparison per window is
 $O(26)$  instead.
# Time:  $O(n * m \log m)$ . Space:  $O(m)$  sort
buffers.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Fixed sliding window of size len(p)
with frequency counters.
# Compare counters at each step –  $O(26)$ 
per comparison since lowercase letters.
# Time:  $O(n)$ . Space:  $O(1)$  – fixed
alphabet."
```

PHASE 4 — CLEAN CODE

```
class _438_FindAnagrams:
    def findAnagrams(self, s: str, p: str)
-> List[int]:
        if len(p) > len(s):
            return []
```

```

        p_count = Counter(p)
        w_count = Counter(s[:len(p)])
        result = [0] if w_count == p_count
    else []
        for i in range(len(p), len(s)):
            w_count[s[i]] += 1
            left = s[i - len(p)]
            w_count[left] -= 1
            if w_count[left] == 0:
                del w_count[left]
            if w_count == p_count:
                result.append(i - len(p)
+ 1)

    return result

```

PHASE 5 — TEST & DEBUG

```

# s='cbaebabacd', p='abc':
p_count={c:1,b:1,a:1}.
# Init
w='cba'={c:1,b:1,a:1}==p→result=[0].
# Slide until i=8('c'):
w_count={b:1,a:1,c:1}==p→result=[0,6]. →
[0,6] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$. Counter comparison is $O(26)=O(1)$ for fixed alphabet.

Alternative: track a 'matches' counter instead of full compare – same $O(n)$.

Given more time I'd ask: what if p can contain any Unicode?

Same algorithm – Counter handles any hashable."

Sliding Window

□ ★ #487 Max Consecutive Ones II ★

MED

[Open on LeetCode](#)

Array · Dynamic Programming · Sliding Window

Lists: ★ Premium | Premium 98

Problem

Given a binary array `nums`, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1:

Input: `nums = [1,0,1,1,0]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1]` and we have 3 consecutive ones.

The max number of consecutive ones is 4.

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0,1]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1,1]` and we have 4 consecutive ones.

The max number of consecutive ones is 4.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- `nums[i]` is either 0 or 1.

Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Flip at most one 0 to 1.
Return the longest consecutive 1s run.
Right?

Follow-up: infinite stream – can't store
all numbers."

#

"[1,0,1,1,0] → flip first 0 → 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Try every substring with two nested
indices left and right.

Count zeros inside; if zeros ≤ 1 , track
the longest valid window.

Correct, but checking all windows is
 $O(n^2)$.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sliding window allowing at most one 0.

Expand right. When zeros > 1 , shrink
left until zeros ≤ 1 .

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _487_MaxConsecutiveOnesII:
    def findMaxConsecutiveOnes(self,
nums: List[int]) -> int:
    zeros = 0
    left = 0
    longest = 0
    for right in range(len(nums)):
        if nums[right] == 0:
            zeros += 1
            while zeros > 1:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            longest = max(longest, right
- left + 1)
    return longest
```

PHASE 5 — TEST & DEBUG

[1,0,1,1,0]: zeros hits 2 at r=4 → shrink past first 0 → window=[1,1,0], longest=4 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

Stream follow-up: keep a deque of the last two 0-positions.

When third 0 arrives, left = oldest_zero_pos + 1 – 0(1) space regardless of stream length.
Generalises to at-most-k flips (Max Consecutive Ones III #1004): same window, k zeros allowed.
Name that connection unprompted."

Sliding Window

□ ★ #1100 Find K-Length Substrings with No Repeated Characters ★

MED

[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: ★ Premium | Premium 98

Problem

Given a string *s* and an integer *k*, return the number of substrings in *s* of length *k* with no repeated characters.

Example 1:

Input: *s* = "havefunonleetcode", *k* = 5

Output: 6

Explanation: There are 6 substrings they are: 'havef', 'avefu', 'vefun', 'efun o', 'etcod', 'tcode'.

Example 2:

Input: `s = "home", k = 5`

Output: `0`

Explanation: Notice `k` can be larger than the length of `s`. In this case, it is not possible to find any substring.

Constraints:

- `1 <= s.length <= 104`
- `s` consists of lowercase English letters.
- `1 <= k <= 104`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count substrings of exactly length `k` with all distinct characters. Right?"

`k > len(s) → 0. k=0 → 0.`"

#

"`s='havefunonleetcode', k=5 → 6 ✓`"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For each start index `i`, build substring `s[i:i+k]` and check distinct chars with a set.

```
# Increment count when set size equals k.  
#  $O(n * k)$  windows; sliding window with  
# frequency map is  $O(n)$ .  
# Time:  $O(n * k)$ . Space:  $O(k)$  set per  
# window.  
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Fixed sliding window of size k. Slide:  
# add right, remove left-behind char.  
# Valid when  $\text{len}(\text{freq}) == k$  (all chars  
# distinct). Time:  $O(n)$ . Space:  $O(k)$ ."
```

PHASE 4 — CLEAN CODE

```
class _1100_KLengthSubstrings:  
    def numKLenSubstrNoRepeats(self, s:  
str, k: int) -> int:  
        if k > len(s):  
            return 0  
        freq : defaultdict =  
defaultdict(int)  
        count = 0  
        for right in range(len(s)):  
            freq[s[right]] += 1  
            if right >= k:  
                left = right - k
```

```
        freq[s[left]] -= 1
        if freq[s[left]] == 0:
            del freq[s[left]]
        if right >= k - 1 and
len(freq) == k:
            count += 1
    return count
```

PHASE 5 — TEST & DEBUG

s='home', k=5: 5>4 → 0 ✓

First valid window r=k-1: if all k chars distinct, count++. Slide forward.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(k). len(freq)==k is the clean validity check.

Fixed window variant of #3 – useful when window size is given, not maximised.

Given more time I'd ask: return the actual substrings? Collect s[right-k+1:right+1]."

Sliding Window

□ #1248 Count Number of Nice Subarrays

MED[Open on LeetCode](#)

Array · Hash Table · Math · Sliding Window · Prefix Sum

Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

Input: `nums = [1,1,2,1,1]`, `k = 3`

Output: 2

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

Example 2:

Input: `nums = [2,4,6], k = 1`

Output: `0`

Explanation: There are no odd numbers in the array.

Example 3:

Input: `nums = [2,2,2,1,2,2,1,2,2,2], k = 2`

Output: `16`

Constraints:

- `1 <= nums.length <= 50000`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= nums.length`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count subarrays with EXACTLY k odd numbers. Return the count. Right?"

Map each element to 1 (odd) or 0 (even)

→ reduce to Subarray Sum Equals K (#560)."

#

"[1,1,2,1,1], k=3 → 2 ✓.

[2,2,2,1,2,2,1,2,2,2], k=2 → 16 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Enumerate every subarray with two loops; count how many have an odd number of 1s.

Straightforward prefix/suffix parity check per window.

Correct, but $O(n^2)$ subarrays is too slow at $n = 5 \times 10^4$.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Prefix sum of odd indicators + hash map. Same pattern as #560.

seed Counter at $\{0:1\}$. For each element, add $\text{num} \% 2$ to running sum.

Count += prefix_count[sum - k]. Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _1248_NiceSubarrays:
    def numberOfSubarrays(self, nums:
List[int], k: int) -> int:
        prefix_count = Counter({0: 1})
        odd_sum = 0
```



```
count = 0
for num in nums:
    odd_sum += num % 2
    count += prefix_count[odd_sum
- k]

    prefix_count[odd_sum] += 1
return count
```

PHASE 5 — TEST & DEBUG

[1,1,2,1,1], k=3:

sums: 1,2,2,3,4. At sum=3:

want=0→prefix[0]=1→count=1.

At sum=4: want=1→prefix[1]=1→count=2. →
2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Identical to
#560 — name the connection explicitly.

At-most-k version: sliding window
counting odds in window, shrink when $> k$.

Exactly $k = \text{atMost}(k) - \text{atMost}(k-1)$.

Given more time I'd ask: count subarrays
with at most k odd numbers?

Sliding window — shrink when odd count
exceeds k , add $\text{right-left}+1$ each step."

Sorting

Round 2 · High · Medium · 3 questions

Sorting

□ #49 Group Anagrams

MED[Open on LeetCode](#)

Array · Hash Table · String · Sorting

Lists: Grind 169

Problem

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 \leq \text{strs.length} \leq 104$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Group strings that are anagrams. Return groups in any order."

Same chars, same frequencies → anagrams. Empty string is its own group. Right?"

#

"['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
→ 3 groups ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

```
# For every pair of strings, sort both and
check if the sorted forms match.
# If they match, put them in the same
group; merge groups as we go.
# Correct, but comparing all pairs is
expensive when n is large.
# Time:  $O(n^2 * m \log m)$ . Space:  $O(n * m)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Use a canonical key: sorted(word) as a
tuple → same key for all anagrams.
# Group into a defaultdict. Time:  $O(n * m \log m)$ . Space:  $O(n * m)$ .
# Alternative: 26-element frequency tuple
→  $O(n * m)$ , no sort cost."
```

PHASE 4 — CLEAN CODE

```
class _49_GroupAnagrams:
    def groupAnagrams(self, strs:
List[str]) -> List[List[str]]:
        groups: defaultdict =
defaultdict(list)
        for word in strs:
            groups[tuple(sorted(word))].a
ppend(word)
```

```
return list(groups.values())
```

PHASE 5 — TEST & DEBUG

```
# 'eat'→('a','e','t'). 'tea'→same.  
'ate'→same. All go to one group.  
# 'tan'→('a','n','t'). 'nat'→same.  
# 'bat'→('a','b','t'). Alone. → 3 groups ✓  
# [''] → {():['']}. → [['']] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n * m \log m)$ . Space  $O(n * m)$ .  
# Frequency tuple alternative:  $O(n * m)$ ,  
avoids  $\log m$  sort. For short words  
(typical)  
# sorting is fine; for very long words,  
frequency tuple wins.  
# Given more time I'd ask: Unicode input?  
sorted() handles it; frequency array  
breaks.  
# Use Counter-based key:  
tuple(sorted(Counter(word).items()))."
```

Sorting

□ #56 Merge Intervals

MED[Open on LeetCode](#)

Array · Sorting

Lists: Grind 169

Problem

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: `intervals =`

`[[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`

Output: `[[1,5]]`

Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Example 3:

Input: intervals = [[4,7],[1,4]]

Output: [[1,7]]

Explanation: Intervals [1,4] and [4,7] are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge all overlapping intervals. Two intervals overlap if one starts before or when the other ends.

[1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping result. Right?"

#

"[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]] ✓ (1-3 and 2-6 overlap at 2-3)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Compare every pair of intervals; if they overlap, merge into one wider interval.

Repeat until a full pass finds no new merges.

Correct, but repeated pairwise merging can degrade toward $O(n^2)$.

Time: $O(n^2)$ worst case. Space: $O(n)$ output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sort by start time — $O(n \log n)$. Then one pass:

Keep a result list. For each interval, if it overlaps with the last result interval

($\text{last.end} \geq \text{current.start}$), extend the last interval's end.

Otherwise, append as a new separate interval.

Time: $O(n \log n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

class _56_MergeIntervals:

```
def merge(self, intervals:
List[List[int]]) -> List[List[int]]:
    intervals.sort()
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if merged[-1][1] >= start: #
overlap
            merged[-1][1] =
max(merged[-1][1], end)
        else:
            merged.append([start,
end])
    return merged
```

PHASE 5 — TEST & DEBUG

[[1,3],[2,6],[8,10],[15,18]]: sorted same.

merged=[[1,3]]. [2,6]:
3>=2→extend→[1,6]. [8,10]: 6<8→append.
[15,18]: 10<15→append.

→ [[1,6],[8,10],[15,18]] ✓

[[1,4],[4,5]]: 4>=4→[1,5] ✓

[[4,7],[1,4]]: sorted→[[1,4],[4,7]].
4>=4→[1,7] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$. Space $O(n)$.
The sort is the bottleneck – the merge pass itself is $O(n)$.
If the input were already sorted, this runs in $O(n)$ total.
Given more time I'd ask: how many meeting rooms needed?
That's Meeting Rooms II (#253) – use a min-heap of end times."

Sorting

□ ★ #1152 Analyze User Website Visit Pattern ★

MED[Open on LeetCode](#)

Array · Hash Table · Sorting

Lists: ★ Premium | Premium 98

Problem

You are given two string arrays `username` and `website` and an integer array `timestamp`. All the given arrays are of the same length and the tuple `[username[i], website[i], timestamp[i]]` indicates that the user `username[i]` visited the website `website[i]` at time `timestamp[i]`.

A pattern is a list of three websites (not necessarily distinct).

- For example, `["home", "away", "love"]`, `["leetcode", "love", "leetcode"]`, and `["luffy", "luffy", "luffy"]` are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

- For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
- Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
- Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.
Example 1:

Input: username = ["joe","joe","joe","james","james","james","james","mary","mary","mary"], timestamp =

[1,2,3,4,5,6,7,8,9,10], website = ["home","about","career","home","cart","maps","home","home","about","career"]

Output: ["home","about","career"]

Explanation: The tuples in this example are:

["joe","home",1],["joe","about",2],["joe","career",3],["james","home",4],["james","cart",5],["james","maps",6],["james","home",7],["mary","home",8],["mary","about",9], and ["mary","career",10].

The pattern ("home", "about", "career") has score 2 (joe and mary).

The pattern ("home", "cart", "maps") has score 1 (james).

The pattern ("home", "cart", "home") has score 1 (james).

The pattern ("home", "maps", "home") has score 1 (james).

Example 2:

```
Input: username =  
["ua", "ua", "ua", "ub", "ub", "ub"],  
timestamp = [1, 2, 3, 4, 5, 6], website =  
["a", "b", "a", "a", "b", "c"]  
Output: ["a", "b", "a"]
```

Constraints:

- $3 \leq \text{username.length} \leq 50$
- $1 \leq \text{username}[i].\text{length} \leq 10$
- $\text{timestamp.length} == \text{username.length}$
- $1 \leq \text{timestamp}[i] \leq 109$
- $\text{website.length} == \text{username.length}$
- $1 \leq \text{website}[i].\text{length} \leq 10$
- $\text{username}[i]$ and $\text{website}[i]$ consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples $[\text{username}[i], \text{timestamp}[i], \text{website}[i]]$ are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the 3-website pattern
(subsequence, not necessarily

consecutive) that the most DISTINCT users
have visited in order. On tie, return
lexicographically smallest. Right?
Each user contributes at most 1 count
per pattern regardless of how many times
they visit."

#

**# "joe visited home→about→career, james
visited home→cart→maps→home, mary visited
home→about→career.**

Pattern ('home', 'about', 'career')
score=2 (joe, mary) → winner ✓"

PHASE 2 — BRUTE FORCE

**# "Let me start with brute force to lock
in the logic.**

For each user, enumerate all 3–website
combinations in visit order.

Build pattern tuple (a,b,c) and
increment Counter across users.

$O(\text{visit_length}^3)$ per user – acceptable
only for small lists.

Time: $O(U * V^3)$. Space: $O(\text{patterns})$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sort all visits by timestamp. Group by user into ordered website lists.

For each user, generate all $C(\text{len}, 3)$ 3-subsequence combos – at most $C(50, 3) = 19600$.

Use a set per user to avoid double-counting (one user = one score per pattern).

Count across users with a global Counter.

Return pattern with max score, lexicographically smallest on ties."

PHASE 4 — CLEAN CODE

```
class _1152_AnalyzeWebsiteVisit:
    def mostVisitedPattern(self,
username: List[str], timestamp:
List[int], website: List[str]) ->
List[str]:
        visits = sorted(zip(timestamp,
username, website))
        user_sites: defaultdict =
defaultdict(list)
        for _, user, site in visits:
            user_sites[user].append(site)
```

```

        pattern_score: Counter =
Counter()
        for sites in user_sites.values():
            seen = set()
            for combo in
itertools.combinations(sites, 3):
                if combo not in seen:
                    seen.add(combo)
                    pattern_score[combo]
+= 1

        return list(max(pattern_score,
key=lambda p: (pattern_score[p], [-ord(c)
for c in ''.join(p)])))

```

PHASE 5 — TEST & DEBUG

```

# After sorting by timestamp, user_sites =
{joe:[home,about,career],
james:[home,car,maps,home],
mary:[home,about,career]}.
# joe: combos of [home,about,career] =
{(home,about,career)}.
# james: C(4,3)=4 combos. All unique.
# mary: {(home,about,career)}.
# pattern_score[(home,about,career)]=2 →
winner ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n + U * C(K,3))$ where U =users, K =max visits per user.

With $U \leq 50$, $K \leq 50$: $C(50,3) = 19600 * 50 = 980000$ – manageable.

The per-user 'seen' set ensures one user contributes at most 1 to each pattern's score.

Tie-breaking: $\max()$ with a compound key – score ascending, then lexicographic ascending.

Given more time I'd ask: what if we want top- k patterns?

Use a heap instead of $\max()$ – same O complexity."

Binary Search

Round 2 · High · Medium · 12 questions

Binary Search

□ #33 Search in Rotated Sorted Array

MED[Open on LeetCode](#)

Array · Binary Search

Lists: Grind 169

Problem

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`

Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`

Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of `nums` are unique.
- `nums` is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array was sorted then rotated at some unknown pivot. Find target in $O(\log n)$."

Right?

All values are distinct. Return -1 if not found."

#

"[4,5,6,7,0,1,2], target=0 → 4 ✓.
target=3 → -1 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Scan left to right: for each index i, check if `nums[i] == target`.

If not found after one full pass, return -1.

Correct and easy to reason about, but the problem asks for $O(\log n)$ on a rotated sorted array.

Time: $O(n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Standard binary search – but at each step, identify which half is sorted.

If `nums[left] <= nums[mid]`: left half is sorted.

```
# If target in [nums[left], nums[mid]):  
search left.  
# Else search right.  
# Else: right half is sorted.  
# If target in (nums[mid], nums[right]):  
search right.  
# Else search left.  
# Time:  $O(\log n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _33_SearchRotatedArray:  
    def search(self, nums: List[int],  
target: int) -> int:  
        left, right = 0, len(nums) - 1  
        while left <= right:  
            mid = (left + right) // 2  
            if nums[mid] == target:  
                return mid  
            if nums[left] <= nums[mid]: #  
left half sorted  
                if nums[left] <= target <  
nums[mid]:  
                    right = mid - 1  
            else:  
                left = mid + 1  
            else: # right half sorted
```



```
        if nums[mid] < target <=
nums[right]:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

PHASE 5 — TEST & DEBUG

```
# [4,5,6,7,0,1,2], target=0:
# l=0,r=6,mid=3(7). nums[0]=4<=7 → left
sorted [4..7]. 0 in [4,7)? No → l=4.
# l=4,r=6,mid=5(1). nums[4]=0>1 → right
sorted [1..2]. 0 in (1,2]? No → r=4.
# l=4,r=4,mid=4(0). 0==target → return 4 ✓
# target=3: never matches → -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ . Space  $O(1)$ .
# The key: at least one half is always
sorted in a rotated array – use that half
# to decide where to search. Checking
nums[left] <= nums[mid] tells us which
half.
# Given more time I'd ask: what if there
are duplicates?
```

```
# That's #81 – when nums[left]==nums[mid],  
we can't determine which half is sorted.  
# Worst case degrades to  $O(n)$  (just left++  
to skip the duplicate)."
```

Binary Search

□ #34 Find First and Last Position of Element in Sorted Array

MED[Open on LeetCode](#)

Array · Binary Search

Lists: Denny Zhang

Problem

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`

Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`

Output: `[-1,-1]`

Example 3:

Input: `nums = []`, `target = 0`

Output: `[-1,-1]`

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- `nums` is a non-decreasing array.
- $-109 \leq \text{target} \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Sorted array with duplicates. Find the first and last index of target. Return `[-1,-1]` if absent.

Must be $O(\log n)$. Right?"

#

"[5,7,7,8,8,10], target=8 → [3,4] ✓.
target=6 → [-1,-1] ✓."

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic.  
# Run standard binary search once to find  
any index of target.  
# Then expand left and right from that  
index while values still equal target.  
# Works, but we can do both boundaries in  
one modified binary search pass.  
# Time:  $O(\log n)$  for search +  $O(k)$  for  
expansion. Space:  $O(1)$ .  
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two binary searches:  
# lower_bound: find first index where  
nums[i] >= target. If  
nums[result]==target → first position.  
# upper_bound: find first index where  
nums[i] > target. Subtract 1 → last  
position.  
# Both are standard binary search  
variants. Time:  $O(\log n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
class _34_FirstLastPosition:
```

```
def searchRange(self, nums:
List[int], target: int) -> List[int]:
    def lower_bound() -> int:
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left
    def upper_bound() -> int:
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > target:
                right = mid
            else:
                left = mid + 1
        return left
    first = lower_bound()
    if first == len(nums) or
nums[first] != target:
        return [-1, -1]
    return [first, upper_bound() - 1]
```

PHASE 5 — TEST & DEBUG

```
# nums=[5,7,7,8,8,10], target=8:
# lower_bound: searching for first idx
# where nums[i]>=8. → index 3.
# nums[3]=8==target ✓. upper_bound: first
# idx where nums[i]>8. → index 5. 5-1=4.
# → [3,4] ✓
# target=6: lower_bound→2 (nums[2]=7>=6).
# nums[2]=7≠6 → [-1,-1] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log n)$ . Space  $O(1)$ .
# lower_bound and upper_bound are the two
# fundamental binary search variants – know
# both.
# They correspond to bisect_left and
# bisect_right in Python's bisect module.
# The check 'nums[first] != target' after
# lower_bound handles the not-found case
# cleanly.
# Given more time I'd ask: what if we want
# the count of target occurrences?
# upper_bound() – lower_bound() in  $O(\log
# n)$ ."
```

Binary Search

□ #153 Find Minimum in Rotated Sorted Array

MED[Open on LeetCode](#)

Array · Binary Search

Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: `1`

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: `0`

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: `11`

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are unique.

- **nums** is sorted and rotated between 1 and **n** times.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array sorted then rotated. Find the minimum in $O(\log n)$. All values unique. Right?"

The minimum is at the rotation point – where the array 'wraps'."

#

"[3,4,5,1,2] → 1 ✓. [4,5,6,7,0,1,2] → 0 ✓. [11,13,15,17] → 11 (no rotation) ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Linear scan from index 0: track the smallest value seen so far.

The first place where **nums[i] > nums[i+1]** marks the rotation pivot candidate.

Return that minimum – correct on small inputs, but we can binary-search the pivot.

```
# Time: O(n). Space: O(1).  
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the rotation point.  
# If nums[mid] > nums[right]: minimum is  
# in the RIGHT half (mid+1 to right).  
# Else: minimum is in the LEFT half  
# including mid (left to mid).  
# When left==right, we've found the  
# minimum.  
# Time: O(log n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _153_FindMinRotated:  
    def findMin(self, nums: List[int]) ->  
int:  
        left, right = 0, len(nums) - 1  
        while left < right:  
            mid = (left + right) // 2  
            if nums[mid] > nums[right]:  
                left = mid + 1 # minimum  
is to the right of mid  
            else:  
                right = mid # minimum is  
mid or to its left
```

return nums[left]

PHASE 5 — TEST & DEBUG

[3,4,5,1,2]: l=0,r=4,mid=2(5).

5>nums[4]=2 → l=3.

l=3,r=4,mid=3(1). 1<=nums[4]=2 → r=3.

l=r=3 → return nums[3]=1 ✓

[4,5,6,7,0,1,2]: l=0,r=6,mid=3(7).

7>nums[6]=2 → l=4.

l=4,r=6,mid=5(1). 1<=2 → r=5.

l=4,r=5,mid=4(0). 0<=1 → r=4. l=r=4 → 0 ✓

[11,13,15,17]: l=0,r=3,mid=1(13).

13<=17 → r=1. l=0,r=1,mid=0(11). 11<=13 → r=0. → 11 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.

The comparison is always against nums[right] (not nums[left]).

If nums[mid] > nums[right], the right portion must be 'lower' – minimum is there.

If nums[mid] <= nums[right], the right portion is in order – minimum is at mid or left.

```
# Given more time I'd ask: what if there  
are duplicates?  
# When nums[mid]==nums[right] we can't  
decide – fall back to right-- (O(n) worst  
case)."
```

Binary Search

□ #300 Longest Increasing Subsequence

MED[Open on LeetCode](#)

Array · Binary Search · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the length of the longest strictly increasing subsequence (not necessarily contiguous).

Return the length, not the subsequence itself. Right?

Follow-up asks for $O(n \log n)$ — so know both approaches."

#

"[10,9,2,5,3,7,101,18] → [2,3,7,101]
length 4 ✓. [7,7,7,7] → 1 (strictly increasing) ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

$\text{dp}[i]$ = length of LIS ending at i . For each i , scan all $j < i$ with $\text{nums}[j] <$

```
nums[i].
# dp[i] = max(dp[j]+1). Answer = max(dp).
# Correct O(n^2); patience sorting /
binary search on tails gives O(n log n).
# Time: O(n^2). Space: O(n) dp array.
# Should I go ahead and optimize?"

class _300_LIS_DP:
    def lengthOfLIS(self, nums:
List[int]) -> int:
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i],
dp[j] + 1)
            return max(dp)

# PHASE 3 — OPTIMIZE (O(n log n) patience
sorting)
# "Maintain a 'tails' array where tails[i]
is the smallest tail of all LIS of length
i+1.
# For each number, binary search for the
leftmost position in tails where
tails[pos] >= num.
```



```
# If found, replace tails[pos] with num
# (smaller tail = more room to extend).
# If not found, append (the LIS can be
# extended by 1).
# len(tails) at the end is the LIS length.
#
# This is patience sorting – tails is
# always sorted so binary search works."
```

PHASE 4 — CLEAN CODE

```
import bisect
class _300_LIS:
    def lengthOfLIS(self, nums:
List[int]) -> int:
        tails = []
        for num in nums:
            pos =
bisect.bisect_left(tails, num)
            if pos == len(tails):
                tails.append(num)
            else:
                tails[pos] = num
        return len(tails)
```

PHASE 5 — TEST & DEBUG

```
# nums=[10,9,2,5,3,7,101,18]:
```

```
# 10→tails=[10]. 9→replace 10→[9].  
2→replace 9→[2]. 5→append→[2,5].  
# 3→replace 5→[2,3]. 7→append→[2,3,7].  
101→append→[2,3,7,101]. 18→replace  
101→[2,3,7,18].  
# len=4 ✓  
# [7,7,7,7]: first 7→[7]. next  
7→bisect_left([7],7)=0→replace→[7]. len=1  
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

" **$O(n^2)$ DP: Time $O(n^2)$, Space $O(n)$. Clean and easy to explain.**

$O(n \log n)$ patience sort: Time $O(n \log n)$, Space $O(n)$.

Note: tails does NOT store the actual LIS – it's a virtual 'pile' structure.

To reconstruct the actual LIS, track parent pointers in the DP version.

Given more time I'd ask: longest non-decreasing subsequence?

Change bisect_left to bisect_right – allows equal elements to extend the sequence."

Binary Search

□ #362 Design Hit Counter

MED[Open on LeetCode](#)

Array · Hash Table · Binary Search · Design · Queue

Lists: Grind 169

Problem

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

- **HitCounter()** Initializes the object of the hit counter system.
- **void hit(int timestamp)** Records a hit that happened at timestamp (in seconds). Several hits may happen at the same timestamp.

- **int getHits(int timestamp)** Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).

Example 1:

Input

```
["HitCounter", "hit", "hit", "hit",  
"getHits", "hit", "getHits", "getHits"]  
[[], [1], [2], [3], [4], [300], [300],  
[301]]
```

Output

```
[null, null, null, null, 3, null, 4, 3]
```

Explanation

```
HitCounter hitCounter = new  
HitCounter();
```

Constraints:

- $1 \leq \text{timestamp} \leq 2 * 10^9$
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to hit and getHits.

Follow up: What if the number of hits per second could be huge? Does your design scale?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Record hits at timestamps and query:
how many hits in the past 300 seconds?

Calls are chronologically ordered
(timestamps only increase). Right?

`getHits(t)` returns hits with timestamp
in $(t-300, t]$ inclusive – past 5 minutes."

#

"hit(1),hit(2),hit(3): `getHits(4)=3`.
hit(300): `getHits(300)=4`. `getHits(301)=3`
(hit@1 expired)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Append every hit timestamp to a list.

`getHits(t)`: count timestamps in $[t-299, t]$
by scanning the whole list.

Simple, but each query is $O(\text{total hits})$
– queue with expiry is better.

Time: $O(1)$ hit, $O(n)$ `getHits` per query.

Space: $O(n)$ all timestamps.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Store timestamps in a deque. On getHits, pop from the left any timestamp $\leq t-300$.

Remaining deque size is the count. hit() is $O(1)$. getHits() is amortised $O(1)$.

#

Follow-up (huge hits/second): store (timestamp, count) pairs instead of individual timestamps.

Multiple hits at the same second are collapsed into one deque entry."

PHASE 4 — CLEAN CODE

```
class _362_HitCounter:
```

```
    def __init__(self):
```

```
        self.hits: deque = deque() #
```

```
stores individual hit timestamps
```

```
    def hit(self, timestamp: int) -> None:
```

```
        self.hits.append(timestamp)
```

```
    def getHits(self, timestamp: int) -> int:
```

```
        while self.hits and self.hits[0]
<= timestamp - 300:
```

```
            self.hits.popleft()
```

```
        return len(self.hits)
```

```
# Follow-up: grouped version for
high-throughput
class _362_HitCounter_Grouped:
    def __init__(self):
        self.hits: deque = deque() #
stores (timestamp, count) pairs
    def hit(self, timestamp: int) -> None:
        if self.hits and self.hits[-1][0]
== timestamp:
            ts, cnt = self.hits.pop()
            self.hits.append((ts, cnt +
1))
        else:
            self.hits.append((timestamp,
1))
    def getHits(self, timestamp: int) ->
int:
        while self.hits and
self.hits[0][0] <= timestamp - 300:
            self.hits.popleft()
        return sum(cnt for _, cnt in
self.hits)

# PHASE 5 — TEST & DEBUG
```

```
# hit(1),hit(2),hit(3),getHits(4):  
deque=[1,2,3]. all>4-300=-296 → len=3 ✓  
# hit(300),getHits(300):  
deque=[1,2,3,300]. 1<=0? No. len=4 ✓  
# getHits(301): 1<=301-300=1 → popleft.  
deque=[2,3,300]. len=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"hit: $O(1)$. getHits: $O(k)$ amortised
where k = expired entries (each entry
popped at most once).

Space: $O(n)$ where n = total hits in the
past 300 seconds.

Binary search alternative: store sorted
timestamps, binary search for $t-300$ cutoff
– $O(\log n)$.

The deque approach is simpler and
amortised $O(1)$ total.

Grouped version handles millions of
hits/second – key interview follow-up.

Given more time I'd ask: what if calls
aren't chronological?

We'd need a sorted structure (sorted
list or BST) and can't use the deque
eviction trick."

Binary Search

□ #528 Random Pick with Weight

MED[Open on LeetCode](#)

Math · Binary Search · Prefix Sum ·
Randomized

Lists: Grind 169

Problem

You are given a 0-indexed array of positive integers w where $w[i]$ describes the weight of the i th index.

You need to implement the function `pickIndex()`, which randomly picks an index in the range $[0, w.length - 1]$ (inclusive) and returns it. The probability of picking an index i is $w[i] / \text{sum}(w)$.

- For example, if $w = [1, 3]$, the probability of picking index 0 is $1 / (1 + 3) = 0.25$ (i.e., 25%), and the probability of picking index 1 is $3 / (1 + 3) = 0.75$ (i.e., 75%).

Example 1:

Input

```
["Solution", "pickIndex"]  
[[[1]], []]
```

Output

```
[null, 0]
```

Explanation

```
Solution solution = new Solution([1]);
```

Example 2:

Input

```
["Solution", "pickIndex", "pickIndex", "pickIndex", "pickIndex", "pickIndex"]  
[[[1, 3]], [], [], [], [], []]
```

Output

```
[null, 1, 1, 1, 1, 0]
```

Explanation

```
Solution solution = new Solution([1, 3]);
```

Constraints:

- $1 \leq w.length \leq 104$
- $1 \leq w[i] \leq 105$
- `pickIndex` will be called at most 104 times.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Pick a random index where index i is chosen with probability $w[i]/\text{sum}(w)$.

`pickIndex()` is called up to 10^4 times – so `init` can be $O(n)$ but `pick` must be fast. Right?

$w=[1,3]$: $P(0)=25\%$, $P(1)=75\%$. The actual randomness in the example output is fine."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Expand weights into a list where index i appears $w[i]$ times, pick random index.

Correct distribution but total list size can be $\text{sum}(w)$ up to $1e9$ – impossible.

Prefix sums + binary search on total weight fixes space.

Time: $O(\text{sum}(w))$ build. Space: $O(\text{sum}(w))$ – infeasible.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Build a prefix sum array. $\text{Prefix}[i] = w[0] + \dots + w[i]$.

```
# Pick a random float r in [0, total).  
Binary search for the leftmost prefix sum  
> r.  
# That index is our answer – the  
probability of landing in bucket i equals  
w[i]/total.  
# Time: O(n) init. O(log n) per pick.  
Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
import random  
class _528_RandomPickWeight:  
    def __init__(self, w: List[int]):  
        self.prefix = []  
        total = 0  
        for weight in w:  
            total += weight  
            self.prefix.append(total)  
        self.total = total  
    def pickIndex(self) -> int:  
        r = random.random() * self.total  
        lo, hi = 0, len(self.prefix) - 1  
        while lo < hi:  
            mid = (lo + hi) // 2  
            if self.prefix[mid] <= r:  
                lo = mid + 1
```

```
        else:
            hi = mid
    return lo
```

PHASE 5 — TEST & DEBUG

```
# w=[1,3]: prefix=[1,4], total=4.
# r in [0,1) → prefix[0]=1>r → lo=hi=0 →
return 0 (25% of the time).
# r in [1,4) → prefix[0]=1≤r → lo=1 →
return 1 (75% of the time). ✓
# w=[1]: prefix=[1]. Any r in [0,1) →
hi=lo=0 → return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Init  $O(n)$ . pickIndex  $O(\log n)$ . Space  $O(n)$ .
```

```
# The prefix sum maps weights to
contiguous ranges on  $[0, \text{total})$ .
```

```
# A random point in that range lands in
bucket  $i$  with probability  $w[i]/\text{total}$ .
```

```
# bisect.bisect_right(self.prefix, r) is
a one-liner alternative using Python's
bisect module.
```

```
# Given more time I'd ask: what if weights
change dynamically?
```

We'd need a Fenwick tree (BIT) – $O(\log n)$ updates AND $O(\log n)$ queries."

Binary Search

□ #852 Peak Index in a Mountain Array

MED[Open on LeetCode](#)

Array · Binary Search

Lists: Denny Zhang

Problem

You are given an integer mountain array `arr` of length `n` where the values increase to a peak element and then decrease.

Return the index of the peak element.

Your task is to solve it in $O(\log(n))$ time complexity.

Example 1:

Input: `arr = [0,1,0]`

Output: 1

Example 2:

Input: `arr = [0,2,1,0]`

Output: 1

Example 3:

Input: `arr = [0,10,5,2]`

Output: 1

Constraints:

- $3 \leq \text{arr.length} \leq 105$
- $0 \leq \text{arr}[i] \leq 106$
- arr is guaranteed to be a mountain array.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array guaranteed to have exactly one peak (mountain shape: strictly increasing then decreasing).

Return the index of the peak in $O(\log n)$. Right?"

#

"[0,1,0] → 1. [0,2,1,0] → 1. [0,10,5,2] → 1. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Linear scan from index 1: first i where $\text{arr}[i] > \text{arr}[i+1]$ is the peak index.

Mountain array guarantees exactly one peak; scan stops early.

$O(n)$ acceptable but binary search on the rising slope is $O(\log n)$.


```
# Time: O(n). Space: O(1).  
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search: at mid, compare arr[mid]  
with arr[mid+1].
```

```
# If arr[mid] < arr[mid+1]: we're on the  
ascending slope → peak is to the right →  
left = mid+1.
```

```
# Else: we're on or past the peak → peak  
is at mid or to the left → right = mid.
```

```
# When left==right: that's the peak. O(log  
n)."
```

PHASE 4 — CLEAN CODE

```
class _852_PeakIndex:  
    def peakIndexInMountainArray(self,  
arr: List[int]) -> int:  
        left, right = 0, len(arr) - 1  
        while left < right:  
            mid = (left + right) // 2  
            if arr[mid] < arr[mid + 1]:  
                left = mid + 1 # still  
ascending – peak is further right  
            else:
```

`right = mid # descending`
`or at peak – search left including mid`
`return left`

PHASE 5 — TEST & DEBUG

[0,1,0]: l=0,r=2,mid=1.

arr[1]=1>arr[2]=0 → right=1.

l=0,r=1,mid=0. arr[0]=0<arr[1]=1 →

left=1. l=r=1 → 1 ✓

[0,10,5,2]: l=0,r=3,mid=1.

arr[1]=10>arr[2]=5 → right=1.

l=0,r=1,mid=0. arr[0]<arr[1] → left=1. → 1

✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.

The comparison arr[mid] vs arr[mid+1] acts as a gradient – positive means 'go right', negative means 'go left'.

This is the same logic as Find Peak Element (#162) – the general version where the array isn't guaranteed

to be a full mountain. Mention the connection.

Given more time I'd ask: what if there are multiple peaks?

Same algorithm finds one peak. For all peaks, we'd need a linear scan."

Binary Search

□ #981 Time Based Key-Value Store

MED

[Open on LeetCode](#)

Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- **TimeMap()** Initializes the object of the data structure.
- **void set(String key, String value, int timestamp)** Stores the key key with the value value at the given time timestamp.
- **String get(String key, int timestamp)** Returns a value such that set was called previously, with $\text{timestamp_prev} \leq \text{timestamp}$. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set",  
"get", "get"]  
[[], ["foo", "bar", 1], ["foo", 1],  
["foo", 3], ["foo", "bar2", 4], ["foo",  
4], ["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null,  
"bar2", "bar2"]
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

Constraints:

- $1 \leq \text{key.length}, \text{value.length} \leq 100$
- key and value consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 10^7$
- All the timestamps timestamp of set are strictly increasing.
- At most $2 * 10^5$ calls will be made to set and get.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Key-value store where each key can have multiple values at different timestamps.

set(key, value, ts) stores the entry.

get(key, ts) returns the value with the

largest timestamp $\leq$ ts. Return '' if none exists. Right?

set timestamps are strictly increasing – so we can binary search the history list."

#

"set('foo', 'bar', 1). get('foo', 3) → 'bar' (no ts $\leq$ 3 except 1). ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all (timestamp, value) pairs per key in a list on set.

get(key, ts): scan backwards for largest timestamp $\leq$ ts.

Works but each get is $O(\text{number of entries for that key})$.

Time: $O(1)$ set, $O(k)$ get per key history. Space: $O(\text{total entries})$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Since set timestamps are strictly increasing, each key's list is already sorted by timestamp.

get() binary searches for the rightmost timestamp $\leq$ query timestamp.

set: $O(1)$. get: $O(\log n)$. Space: $O(\text{total entries})$."

PHASE 4 — CLEAN CODE

```
class _981_TimeMap:
```

```
    def __init__(self):
```

```
        self.store: defaultdict =  
defaultdict(list) # key  $\rightarrow$  [(ts, value),  
...]
```

```
    def set(self, key: str, value: str,  
timestamp: int) -> None:  
        self.store[key].append((timestamp  
, value))
```

```
    def get(self, key: str, timestamp:  
int) -> str:
```

```
        entries = self.store[key]  
        lo, hi = 0, len(entries) - 1  
        result = ''  
        while lo <= hi:  
            mid = (lo + hi) // 2
```

```
        if entries[mid][0] <=
timestamp:
            result = entries[mid][1]
# valid candidate – try to go right for a
larger ts
            lo = mid + 1
        else:
            hi = mid - 1
    return result

# PHASE 5 — TEST & DEBUG
# set('foo', 'bar', 1):
store['foo'] = [(1, 'bar')].
# get('foo', 1): entries = [(1, 'bar')].
mid=0, ts=1<=1 → result='bar', lo=1. lo>hi
→ return 'bar' ✓
# get('foo', 3): mid=0, ts=1<=3 →
result='bar'. lo=1>hi=0 → return 'bar' ✓
# set('foo', 'bar2', 4):
store['foo'] = [(1, 'bar'), (4, 'bar2')].
# get('foo', 4):
mid=0(1<=4)→result='bar', lo=1.
mid=1(4<=4)→result='bar2', lo=2. return
'bar2' ✓
# get('foo', 5): same → 'bar2' ✓
```


PHASE 6 — COMPLEXITY & FOLLOW-UP

"set $O(1)$. get $O(\log n)$. Space $O(\text{total entries})$.

The 'find rightmost $ts \leq \text{query}$ ' is a standard upper_bound - 1 binary search pattern.

Timestamps strictly increasing → no sorting needed – the list maintains order automatically.

bisect.bisect_right alternative: $\text{pos} = \text{bisect.bisect_right}(\text{entries}, (\text{timestamp}, \text{chr}(127))) - 1$

(appending $\text{chr}(127)$ to go past any value at that exact timestamp).

Given more time I'd ask: what if set timestamps aren't guaranteed to be increasing?

We'd need to sort the list on insert or use a sorted data structure."

Binary Search

□ #1011 Capacity to Ship Packages Within D Days

MED[Open on LeetCode](#)

Array · Binary Search

Lists: Denny Zhang

Problem

A conveyor belt has packages that must be shipped from one port to another within days days.

The i th package on the conveyor belt has a weight of $\text{weights}[i]$. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

Input: weights =
[1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5

2nd day: 6, 7

3rd day: 8

4th day: 9

5th day: 10

Example 2:

Input: weights = [3,2,2,4,1,4], days = 3

Output: 6

Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:

1st day: 3, 2

2nd day: 2, 4

3rd day: 1, 4

Example 3:

Input: weights = [1,2,3,1,1], days = 4

Output: 3

Explanation:

1st day: 1

2nd day: 2

3rd day: 3

4th day: 1, 1

Constraints:

- $1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$
- $1 \leq \text{weights}[i] \leq 500$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the minimum ship capacity that lets us ship all packages in at most D days."

Packages must be shipped in order – we can't reorder them. Right?

Capacity must be at least $\max(\text{weights})$ (to handle any single package).

Capacity at most $\sum(\text{weights})$ (ship everything in one day)."

#

"weights=[1..10], days=5 → 15 ✓ (greedy daily assignment: 1-5, 6-7, 8, 9, 10)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Try every candidate ship capacity from max(weights) up to sum(weights).

For each capacity, greedily pack days and count days needed.

Correct but linear search on capacity range is $O(\text{sum} * n)$.

Time: $O(\text{sum}(\text{weights}) * n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search on the answer (capacity)."

The feasibility function

'can_ship(capacity)' is monotone: if C works, C+1 also works.

So binary search for the leftmost capacity where can_ship returns True.

can_ship: greedily fill each day until the load would exceed capacity, then start a new day.

Count days needed. Return days $\leq D$.

Time: $O(n \log(\text{sum}(\text{weights})))$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _1011_CapacityShip:
    def shipWithinDays(self, weights:
List[int], days: int) -> int:
        def can_ship(capacity: int) ->
bool:
            day_load = 0
            days_used = 1
            for w in weights:
                if day_load + w >
capacity:
                    days_used += 1
                    day_load = 0
                    day_load += w
            return days_used <= days
        lo, hi = max(weights),
sum(weights)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_ship(mid):
                hi = mid # valid - try
smaller
            else:
```

```

        lo = mid + 1 # not valid –
need more capacity
    return lo

```

PHASE 5 — TEST & DEBUG

```
# weights=[1..10], days=5. lo=10, hi=55.
```

```
# mid=32:
```

```
can_ship(32)→1+2+3+4+5=15≤32(day1),
6+7=13≤32(day2), 8≤32(day3), 9≤32(day4),
10≤32(day5). 5 days ✓ → hi=32.
```

```
# ... binary search converges to 15.
```

```
# can_ship(15): 1+2+3+4+5=15(day1),
6+7=13+8>15→day2=6+7, 8→day3, 9→day4,
10→day5. 5 days ✓. → 15 ✓
```

```
# can_ship(14): 1+2+3+4=10,
+5=15>14→day2=5+6=11,
+7>14→day3=7+8>14→day4=8, 9→day5,
10→day6. 6>5 ✗.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log(\text{sum}))$ . Space  $O(1)$ ."
```

```
# This is 'binary search on the answer' –
a powerful pattern where the answer space
is
```

```
# a range of integers and feasibility is
monotone.
```

The feasibility check is a greedy simulation – always load greedily to avoid underusing capacity.

Same pattern appears in: Koko Eating Bananas (#875), Split Array Largest Sum (#410).

Given more time I'd ask: what if packages can be reordered?

Then we'd try to optimise placement – different problem, potentially NP-hard."

Binary Search

★ #1060 Missing Element in Sorted Array **MED**

★

[Open on LeetCode](#)

Array · Binary Search

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums` which is sorted in ascending order and all of its elements are unique and given also an integer `k`, return the `k`th missing number starting from the leftmost number of the array.

Example 1:

Input: `nums = [4,7,9,10]`, `k = 1`

Output: 5

Explanation: The first missing number is 5.

Example 2:

Input: `nums = [4,7,9,10]`, `k = 3`

Output: 8

Explanation: The missing numbers are `[5,6,8,...]`, hence the third missing number is 8.

Example 3:

Input: `nums = [1,2,4]`, `k = 3`

Output: 6

Explanation: The missing numbers are `[3,5,6,7,...]`, hence the third missing number is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^7$
- `nums` is sorted in ascending order, and all the elements are unique.
- $1 \leq k \leq 10^8$

Follow up: Can you find a logarithmic time complexity (i.e., $O(\log(n))$) solution?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted unique integers. Find the k-th
missing integer starting from nums[0].
# Missing count at index i = (nums[i] -
nums[0]) - i (how many numbers should
exist but don't).
# Follow-up asks for  $O(\log n)$ . Right?"
#
# "[4,7,9,10], k=1: missing = [5,6,8,...].
1st missing = 5 ✓
# [4,7,9,10], k=3: [5,6,8,...]. 3rd = 8 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Walk nums in order maintaining expected
next integer starting at 0.
# Increment expected whenever nums[i] ==
expected; else expected stays.
# When expected reaches k, return k -
correct but scans whole array.
# Time:  $O(n)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"

class _1060_MissingElement_Brute:
    def missingElement(self, nums:
List[int], k: int) -> int:
```

```
        for i in range(1, len(nums)):
            gap = nums[i] - nums[i - 1] -
1 # missing numbers between nums[i-1] and
nums[i]
            if k <= gap:
                return nums[i - 1] + k
            k -= gap
        return nums[-1] + k # k-th missing
is after the array
```

PHASE 3 — OPTIMIZE (O(log n))

"Define missing(i) = numbers missing from nums[0] to nums[i] = (nums[i] - nums[0]) - i.

Binary search for the rightmost index where missing(i) < k.

Answer = nums[right] + (k - missing(right)).

If k > missing(last), answer is past the end of the array: nums[-1] + (k - missing(n-1))."

PHASE 4 — CLEAN CODE

```
class _1060_MissingElement:
    def missingElement(self, nums:
List[int], k: int) -> int:
```

```
def missing(i: int) -> int:
    return (nums[i] - nums[0]) - i
n = len(nums)
if k > missing(n - 1):
    return nums[-1] + (k -
missing(n - 1))
lo, hi = 0, n - 1
while lo < hi:
    mid = (lo + hi) // 2
    if missing(mid) < k:
        lo = mid + 1 # not enough
missing yet – go right
    else:
        hi = mid # too many or
just enough – go left
    return nums[lo - 1] + (k -
missing(lo - 1))

# PHASE 5 — TEST & DEBUG
# nums=[4,7,9,10], k=1:
# missing: [0]=0, [1]=2, [2]=3, [3]=4.
# k=1<=missing(3)=4. Binary search:
lo=0,hi=3,mid=1. missing(1)=2>=1→hi=1.
# lo=0,hi=1,mid=0. missing(0)=0<1→lo=1.
lo=hi=1.
```

```
# return nums[0]+(1-missing(0))=4+(1-0)=5
```

✓

```
# nums=[4,7,9,10], k=3:
```

```
# missing(3)=4>=3. Binary search → lo  
lands at 2. missing(1)=2<3→lo=2. lo=hi=2.
```

```
# return nums[1]+(3-missing(1))=7+(3-2)=8
```

✓

```
# k=5: missing(3)=4<5 → return  
nums[3]+(5-4)=10+1=11 ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "O(log n) with binary search. O(1)  
space.
```

```
# The missing(i) formula is the key:  
(nums[i]-nums[0])-i counts how many  
integers
```

```
# in [nums[0], nums[i]] are absent from  
the array.
```

```
# The past-the-end check handles k > total  
missing numbers in the array gracefully.
```

```
# Given more time I'd ask: what if there  
are duplicate values in nums?
```

```
# missing(i) would overcount – we'd need  
to adjust for duplicates."
```

Binary Search

□ #1062 Longest Repeating Substring

MED[Open on LeetCode](#)

String · Binary Search · Dynamic Programming
· Rolling Hash · Suffix Array · Hash Function

Lists: Denny Zhang

Problem

Given a string *s*, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1:

Input: *s* = "abcd"

Output: 0

Explanation: There is no repeating substring.

Example 2:

Input: *s* = "abbaba"

Output: 2

Explanation: The longest repeating substrings are "ab" and "ba", each of which occurs twice.

Example 3:

Input: `s = "aabcaabdaab"`

Output: 3

Explanation: The longest repeating substring is "aab", which occurs 3 times.

Constraints:

- $1 \leq s.length \leq 2000$
- `s` consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the length of the longest substring that appears at least twice in `s`.

The two occurrences can overlap. Return 0 if no repeating substring. Right?"

#

"'abcd'→0. 'abbaba'→2 ('ab', 'ba').
'aabcaabdaab'→3 ('aab')."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.


```
# For each length L from n-1 down to 1,
collect all substrings of length L in a
set.
# First L where a duplicate appears is the
answer.
#  $O(n^2)$  substrings per L – binary search
on L with rolling hash helps.
# Time:  $O(n^2)$  to  $O(n^3)$  naive. Space:
 $O(n^2)$  set.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DP approach — simpler
to explain)
# "DP:  $dp[i][j]$  = length of longest common
suffix of  $s[:i]$  and  $s[:j]$ , where  $i \neq j$ .
#  $dp[i][j] = dp[i-1][j-1] + 1$  if
 $s[i-1] == s[j-1]$  and  $i \neq j$ , else 0.
# Track the global max.  $O(n^2)$  time,  $O(n^2)$ 
space."

# PHASE 4 — CLEAN CODE
class _1062_LongestRepeatingSubstring:
    def longestRepeatingSubstring(self,
s: str) -> int:
        n = len(s)
```

```

        dp = [[0] * (n + 1) for _ in
range(n + 1)]
        best = 0
        for i in range(1, n + 1):
            for j in range(i + 1, n + 1):
# j > i ensures different starting
positions
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = dp[i -
1][j - 1] + 1
                    best = max(best,
dp[i][j])
        return best

```

PHASE 5 — TEST & DEBUG

s='abbaba':

i=1,j=2: 'a'=='b'? No. ... i=1,j=4:

'a'=='a'? Yes→dp[1][4]=1, best=1.

i=2,j=5: 'b'=='b'?

Yes→dp[2][5]=dp[1][4]+1=2, best=2.

→ 2 ✓ (substring 'ab' starting at 0 and 3)

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n^2)$. Acceptable for $n \leq 2000$ (4M cells).

Binary search + hashing alternative: $O(n \log n)$ time, $O(n)$ space – faster but more complex.

Given more time I'd ask: return the actual substring, not just the length?

Track (i, j) when best is updated, then `s[i-best:i]."`

Binary Search

★ #1533 Find the Index of the Large Integer ★

MED[Open on LeetCode](#)

Array · Binary Search · Interactive

Lists: ★ Premium | Premium 98

Problem

We have an integer array `arr`, where all the integers in `arr` are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API `ArrayReader` which have the following functions:

- `int compareSub(int l, int r, int x, int y)`: where $0 \leq l, r, x, y < \text{ArrayReader.length}()$, $l \leq r$ and $x \leq y$. The function compares the sum of sub-array `arr[l..r]` with the sum of the sub-array `arr[x..y]` and returns: 1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] > \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.
- 0 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] == \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

- -1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] < \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

You are allowed to call `compareSub()` 20 times at most. You can assume both functions work in $O(1)$ time.

Return the index of the array `arr` which has the largest integer.

Example 1:

Input: arr = [7,7,7,7,10,7,7,7]

Output: 4

Explanation: The following calls to the API

```
reader.compareSub(0, 0, 1, 1) //
returns 0 this is a query comparing the
sub-array (0, 0) with the sub array (1,
1), (i.e. compares arr[0] with arr[1]).
Thus we know that arr[0] and arr[1]
doesn't contain the largest element.
reader.compareSub(2, 2, 3, 3) //
returns 0, we can exclude arr[2] and
arr[3].
reader.compareSub(4, 4, 5, 5) //
returns 1, thus for sure arr[4] is the
largest element in the array.
Notice that we made only 3 calls, so
the answer is valid.
```

Example 2:

Input: nums = [6,6,12]

Output: 2

Constraints:

- $2 \leq \text{arr.length} \leq 5 * 10^5$
- $1 \leq \text{arr}[i] \leq 100$

- All elements of arr are equal except for one element which is larger than all other elements.

Follow up:

- What if there are two numbers in arr that are bigger than all other numbers?
- What if there is one number that is bigger than other numbers and one number that is smaller than other numbers?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"One element is strictly larger than all others. Find its index using the `compareSub` API.

`compareSub(l,r,x,y)` compares `sum(arr[l..r])` with `sum(arr[x..y])`:
returns 1,-1,0.

At most 20 calls – so $O(\log n)$ calls are required. Right?"

#

"[7,7,7,7,10,7,7,7]: 3 `compareSub` calls suffice to find index 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Linear scan: compare `nums[i]` with `nums[i+1]`; first unequal pair reveals larger half.

Correct $O(n)$ but problem expects $O(\log n)$ via binary search on the doubled array.

Time: $O(n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search: at each step, compare the left half against the right half of the range.

If `left > right`: large integer is in the left half → search left.

If `right > left`: search right.

If equal: the large integer is the MIDDLE element (if odd-length range, middle was excluded).

Halve the range each step → $O(\log n)$ calls."

PHASE 4 — CLEAN CODE

```
class _1533_FindLargeInteger:
```

```
    def getIndex(self, reader) -> int:
```



```
    lo, hi = 0, reader.length() - 1
    while lo < hi:
        mid = (lo + hi) // 2
        # Compare left half [lo..mid]
with right half of equal size
        left_end = mid
        right_start = mid + 1
        right_end = right_start +
(mid - lo)
        if right_end > hi:
            right_end = hi
        result =
reader.compareSub(lo, left_end,
right_start, right_end)
        if result == 1:
            hi = left_end # large int
is in left half
        elif result == -1:
            lo = right_start # large
int is in right half
        else:
            return mid + 1 # equal
sums → large int is the middle skipped
element
    return lo
```

PHASE 5 — TEST & DEBUG

```
# arr=[7,7,7,7,10,7,7,7]: lo=0,hi=7.  
# mid=3. Compare [0..3] vs [4..7]. Both  
have 4 elements. Sum=[28] vs [31]. -1 →  
lo=4.  
# lo=4,hi=7,mid=5. Compare [4..5] vs  
[6..7]. [10+7=17] vs [7+7=14]. 1 → hi=5.  
# lo=4,hi=5,mid=4. Compare [4..4] vs  
[5..5]. [10] vs [7]. 1 → hi=4. lo=hi=4 →  
return 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(log n) compareSub calls. Well within  
the 20-call limit for n=5*10^5  
(log2(500000)≈19).  
# The equal-sum case (odd-length range)  
means the middle element was the large one  
—  
# both halves have equal elements so the  
excluded middle is the outlier.  
# Follow-up: two large numbers? One  
comparison no longer definitively points  
to one half.  
# We'd need to track which half has MORE  
excess — more complex."
```

Matrix

Round 2 · High · Medium · 14 questions

Matrix

□ #36 Valid Sudoku

MED[Open on LeetCode](#)

Array · Hash Table · Matrix

Lists: Grind 169

Problem

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1–9 without repetition.
2. Each column must contain the digits 1–9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1–9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[[".","9","8",".",".",".","6","."],
["8",".",".",".","6",".",".","3"],
["4",".",".","8",".","3",".","1"],
["7",".",".","2",".",".","6"],
[[".","6",".",".",".","2","8","."]
```

Example 2:

Input: board =

```
[["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".","6","."],
["8",".",".",".","6",".",".","3"],
["4",".",".","8",".","3",".",".","1"],
["7",".",".",".","2",".",".","6"],
[".","6",".",".",".","2","8","."]
```

Constraints:

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit 1-9 or '.'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Check if a partially filled 9x9 sudoku board is valid – no repeats in any row, column, or 3x3 box. Ignore empty cells ('.'). Right?"

A valid board doesn't have to be solvable – just no current conflicts."

#

"The two examples differ only in one cell: '8' appears twice in the top-left 3x3 box → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each of 9 rows, 9 columns, and 9 boxes, collect digits in a set and check size==9.

27 independent validity checks – straightforward and easy to explain.

Time: $O(1)$ – fixed 81 cells. Space: $O(1)$ – 9 sets of size 9.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Single pass: use three 2D boolean arrays – rows[r][d], cols[c][d], boxes[r//3][c//3][d].

For each filled cell, check all three. If any is already marked, return False.

Otherwise mark all three. $O(81) = O(1)$ for fixed 9x9."

PHASE 4 — CLEAN CODE

class _36_ValidSudoku:

```
def isValidSudoku(self, board:
List[List[str]]) -> bool:
    rows = [[False] * 9 for _ in
range(9)]
    cols = [[False] * 9 for _ in
range(9)]
    boxes = [[False] * 9 for _ in
range(9)] # indexed by box_id = (r//3)*3 +
c//3

    for r in range(9):
        for c in range(9):
            if board[r][c] == '.':
                continue
            d = int(board[r][c]) - 1
            box_id = (r // 3) * 3 + (c
// 3)

            if rows[r][d] or
cols[c][d] or boxes[box_id][d]:
                return False
            rows[r][d] = cols[c][d] =
boxes[box_id][d] = True
        return True

# PHASE 5 — TEST & DEBUG
# Cell (0,0)='5': d=4. box_id=0. Mark
rows[0][4]=cols[0][4]=boxes[0][4]=True.
```



```
# Cell (3,0)='8': d=7. box_id=0.  
rows[3][7]? No. boxes[0][7]? No. Mark.  
# In example 2: cell (0,0)='8' and cell  
(3,0)='8': both have box_id=0, d=7.  
# Second '8' at (3,0): boxes[0][7] already  
True → return False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(81)=O(1)$. Space $O(81)=O(1)$ –
fixed 9x9 board.

The box_id formula $(r//3)*3+(c//3)$ maps
each of the 9 boxes to 0–8. Know this
cold.

Alternative: use a set of tuples like
(r, 'row', d), (c, 'col', d),
(box_id, 'box', d) – one set.

Given more time I'd ask: implement a
Sudoku solver?

That's backtracking – for each empty
cell, try 1–9, recurse, backtrack on
failure."

Matrix

□ #48 Rotate Image

MED[Open on LeetCode](#)

Array · Math · Matrix

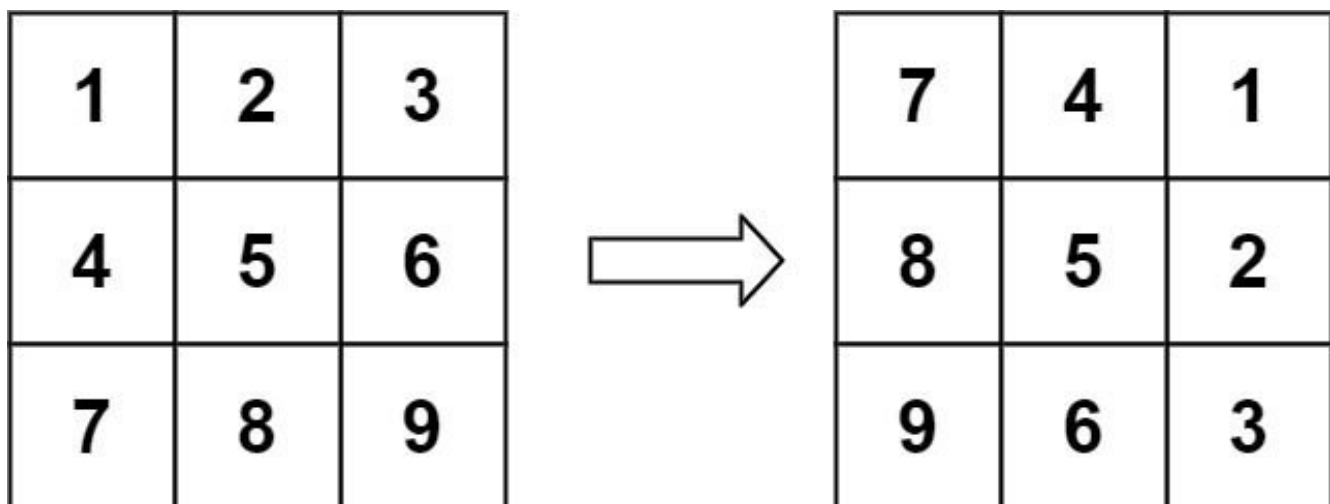
Lists: Grind 169 | CodeSignal

Problem

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Example 1:



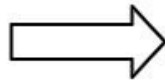
Input: matrix =

[[1,2,3],[4,5,6],[7,8,9]]

Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

**Input: matrix = [[5,1,9,11],[2,4,8,10],
[13,3,6,7],[15,14,12,16]]**

**Output: [[15,13,2,5],[14,3,4,1],[12,6,8,
9],[16,7,10,11]]**

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate an $n \times n$ matrix 90 degrees clockwise in-place. No extra matrix allowed. Right?

After rotation: element at (r, c) moves to $(c, n-1-r)$."

#

"[[1,2,3],[4,5,6],[7,8,9]] →
[[7,4,1],[8,5,2],[9,6,3]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Allocate a new $n \times n$ matrix; for each (r, c) set $\text{new}[c][n-1-r] = \text{matrix}[r][c]$.

Classic 90-degree rotation formula – clear and correct.

Uses $O(n^2)$ extra memory; we can rotate in-place layer by layer.

Time: $O(n^2)$. Space: $O(n^2)$ new matrix.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (two-step in-place)

"Two operations, each reversible and composable:

Step 1: Transpose (swap $\text{matrix}[r][c]$ with $\text{matrix}[c][r]$).

```
# Step 2: Reverse each row.
# Together these produce a 90° clockwise
rotation.  $O(n^2)$  time,  $O(1)$  space.
# Verify: (r,c) → transpose → (c,r) → reverse
row → (c, n-1-r). Correct!"
```

PHASE 4 — CLEAN CODE

```
class _48_RotateImage:
    def rotate(self, matrix:
List[List[int]]) -> None:
        n = len(matrix)
        # Step 1: transpose
        for r in range(n):
            for c in range(r + 1, n):
                matrix[r][c],
matrix[c][r] = matrix[c][r], matrix[r][c]
        # Step 2: reverse each row
        for row in matrix:
            row.reverse()
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3],[4,5,6],[7,8,9]]:
# Transpose: [[1,4,7],[2,5,8],[3,6,9]].
# Reverse rows: [[7,4,1],[8,5,2],[9,6,3]]
✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$ in-place.
Counter-clockwise 90° : reverse each row first, then transpose.
180° : transpose then transpose (just reverse all rows, then reverse their order).
The two-step approach is elegant – each step is simple and reversible.
Given more time I'd ask: rotate a non-square matrix?
No longer possible in-place with the same dimensions – need a new $m \times n$ matrix."

Matrix

□ #54 Spiral Matrix

MED

[Open on LeetCode](#)

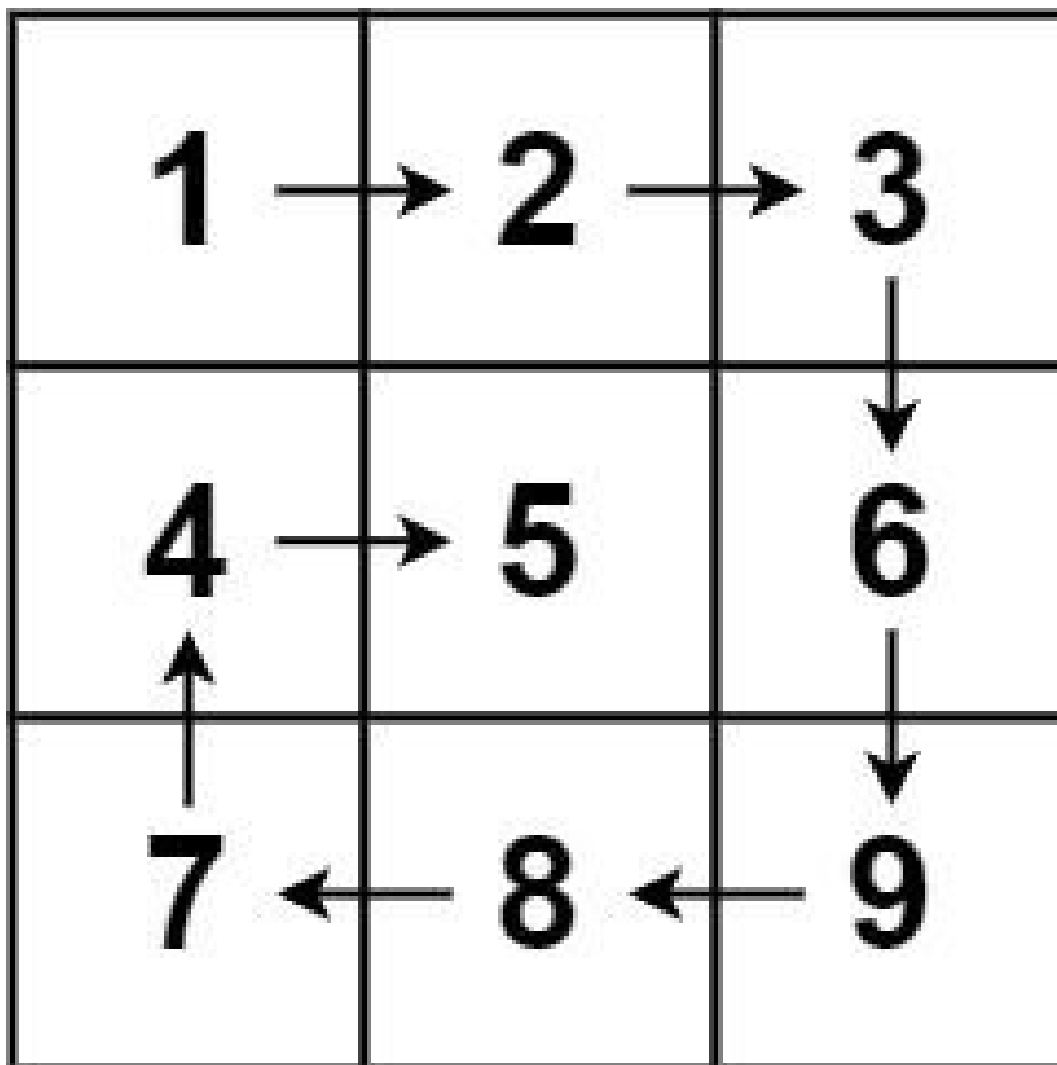
Array · Matrix · Simulation

Lists: Grind 169 | CodeSignal

Problem

Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

Example 1:

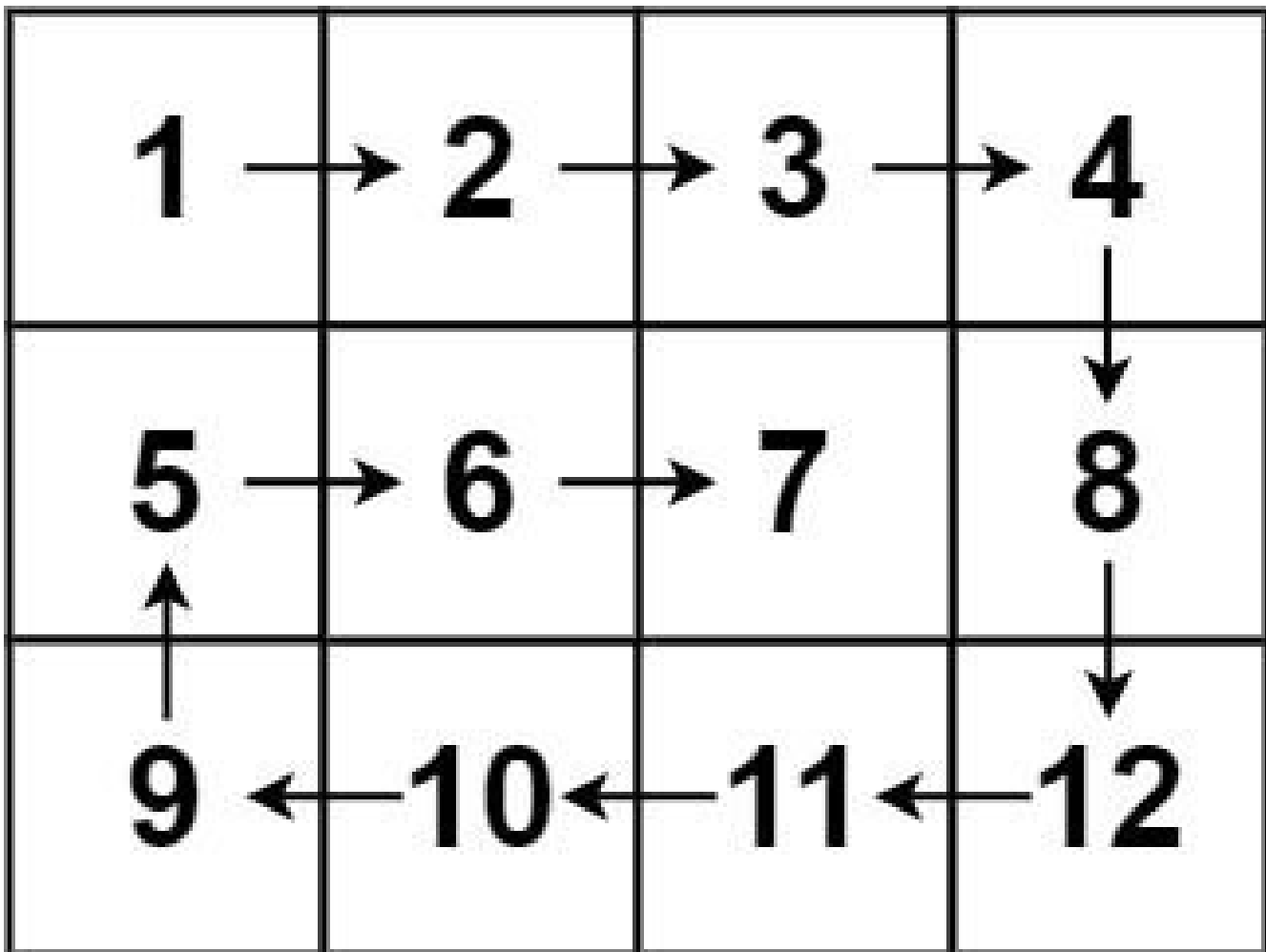


Input: matrix =

[[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Example 2:



Input: matrix =

`[[1,2,3,4],[5,6,7,8],[9,10,11,12]]`

Output: `[1,2,3,4,8,12,11,10,9,5,6,7]`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all matrix elements in spiral order: right along top, down along right side,

left along bottom, up along left side – then repeat for the inner rectangle.

Right?

Works for non-square matrices."

#

"[[1,2,3],[4,5,6],[7,8,9]] → [1,2,3,6,9,8,7,4,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Maintain top/bottom/left/right bounds; walk right, down, left, up while unvisited.

Mark visited cells in a boolean matrix to know when to turn.

Works, but the visited matrix costs $O(m*n)$ extra space.

Time: $O(m * n)$. Space: $O(m * n)$ visited.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (layer boundaries)

"Track four boundaries: top, bottom, left, right."

**# Traverse right along top (then top++),
down along right (then right--),**

**# left along bottom (then bottom--), up
along left (then left++).**

Stop when top>bottom or left>right.

0(m*n) time, 0(1) extra space."

PHASE 4 — CLEAN CODE

class _54_SpiralMatrix:

def spiralOrder(self, matrix:

List[List[int]]) -> List[int]:

top, bottom = 0, len(matrix) - 1

left, right = 0, len(matrix[0]) -

1

result = []

while top <= bottom and left <=

right:

**for c in range(left, right +
1): result.append(matrix[top][c])**

top += 1

**for r in range(top, bottom +
1): result.append(matrix[r][right])**

right -= 1

```
        if top <= bottom:
            for c in range(right,
left - 1, -1):
result.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom,
top - 1, -1):
result.append(matrix[r][left])
                    left += 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3],[4,5,6],[7,8,9]]:
top=0,bot=2,l=0,r=2.
# Right: 1,2,3. top=1.
# Down: 6,9. right=1.
# Left (top<=bot): 8,7. bot=1.
# Up (left<=right): 4. left=1.
# top=1,bot=1,l=1,r=1.
# Right: 5. top=2. top>bot → stop.
# → [1,2,3,6,9,8,7,4,5] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n)$ . Space  $O(1)$  extra.
```

```
# The boundary guards (if top<=bottom / if  
left<=right) prevent double-counting  
# the single middle row or column of  
non-square matrices.  
# Given more time I'd ask: Spiral Matrix  
II (□#59) – fill instead of read.  
# Same boundary logic, write numbers 1..n2  
instead of appending."
```

Matrix

#59 Spiral Matrix II

MED[Open on LeetCode](#)

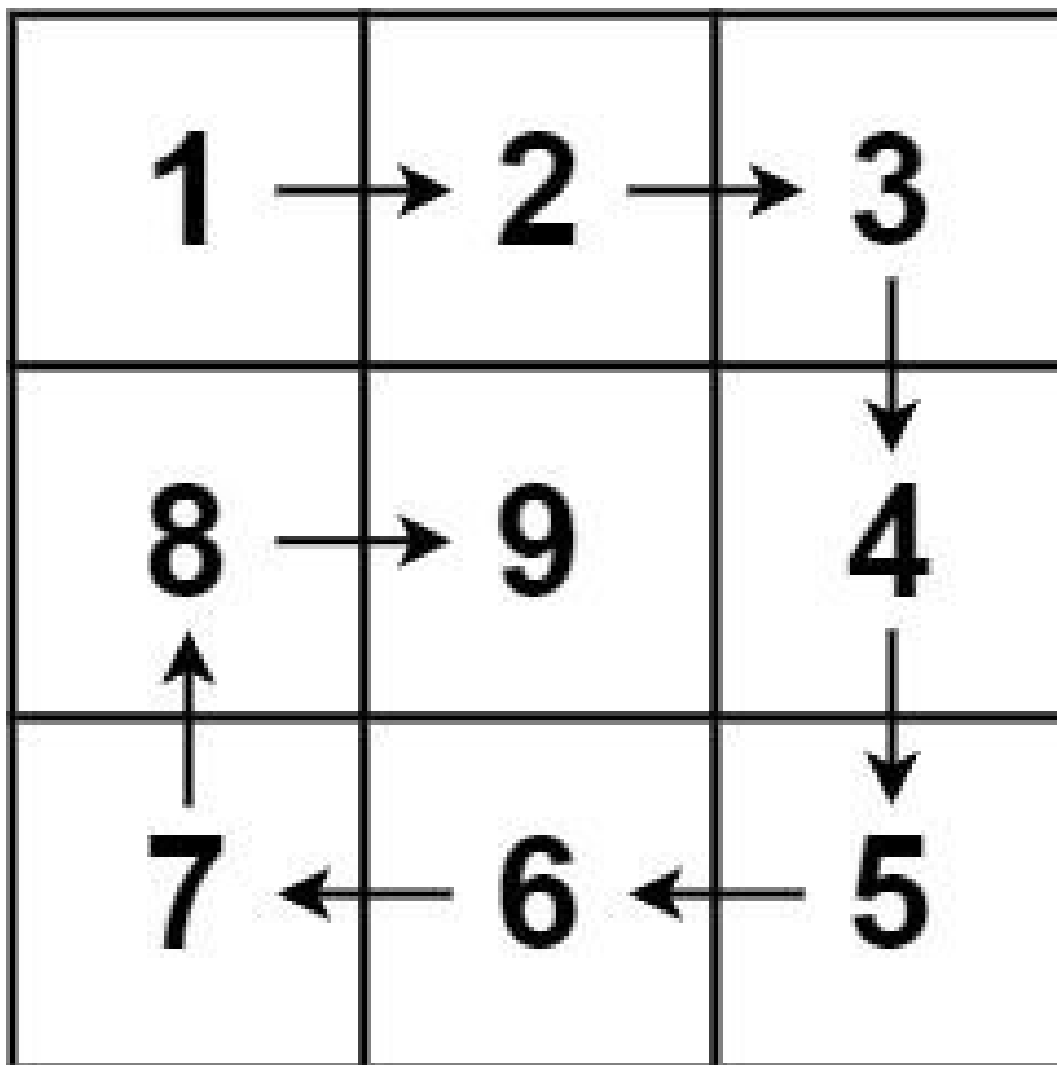
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:



Input: $n = 3$

Output: $[[1,2,3],[8,9,4],[7,6,5]]$

Example 2:

Input: $n = 1$

Output: $[[1]]$

Constraints:

- $1 \leq n \leq 20$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Generate an $n \times n$ matrix filled with 1 to n^2 in spiral order. Return the matrix. Right?"

#

"n=3 → $\begin{bmatrix} 1 & 2 & 3 \\ 8 & 9 & 4 \\ 7 & 6 & 5 \end{bmatrix}$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Fill numbers $1..n^2$ using direction arrays (right, down, left, up) and a visited grid.

Turn when hitting boundary or a filled cell.

Straightforward simulation with $O(n^2)$ extra visited space.

Time: $O(n^2)$. Space: $O(n^2)$ visited.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Identical boundary approach to #54, but write consecutive integers instead of reading."


```
# Initialize n×n matrix of zeros, fill  
with num=1..n2. O(n2) time, O(1) extra."
```

PHASE 4 — CLEAN CODE

```
class _59_SpiralMatrixII:  
    def generateMatrix(self, n: int) ->  
List[List[int]]:  
        matrix = [[0] * n for _ in  
range(n)]  
        top, bottom, left, right = 0, n -  
1, 0, n - 1  
        num = 1  
        while top <= bottom and left <=  
right:  
            for c in range(left, right +  
1): matrix[top][c] = num; num += 1  
                top += 1  
            for r in range(top, bottom +  
1): matrix[r][right] = num; num += 1  
                right -= 1  
            if top <= bottom:  
                for c in range(right,  
left - 1, -1): matrix[bottom][c] = num;  
num += 1  
                    bottom -= 1  
            if left <= right:
```

```
        for r in range(bottom,
top - 1, -1): matrix[r][left] = num; num
+= 1
        left += 1
    return matrix
```

PHASE 5 — TEST & DEBUG

```
# n=3: fills 1→2→3 (top row), 4 (right
col), 5→6 (bottom), 7 (left col), 9
(center).
```

```
# → [[1,2,3],[8,9,4],[7,6,5]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^2)$ . Space  $O(1)$  extra — output
matrix doesn't count.
```

```
# This is the write-version of Spiral
Matrix (#54) — exact same boundary logic.
```

```
# Given more time I'd ask: generate a
spiral starting from the center?
```

```
# Reverse the boundary logic — start from
the middle, expand outward."
```

Matrix

□ #64 Minimum Path Sum

MED[Open on LeetCode](#)

Array · Dynamic Programming · Matrix

Lists: Denny Zhang

Problem

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: `grid = [[1,3,1],[1,5,1],[4,2,1]]`

Output: 7

Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: `grid = [[1,2,3],[4,5,6]]`

Output: 12

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `0 <= grid[i][j] <= 200`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Move only right or down. Find the path from top-left to bottom-right with minimum sum. Right?"

Can only move right or down – no backtracking."

#

"[[1,3,1],[1,5,1],[4,2,1]]: path 1→3→1→1→1=7 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS from (0,0): at each cell try moving right and down, take min of two paths.

Memoize (r,c) to avoid recomputing subpaths – classic top-down DP.

Without memo this blows up exponentially; memo makes it $O(m*n)$.

```
# Time:  $O(m * n)$  with memo. Space:  $O(m * n)$  memo + stack.  
# Should I go ahead and optimize?"  
  
# PHASE 3 — OPTIMIZE (in-place DP)  
# "dp[i][j] = min path sum to reach (i,j).  
# dp[i][j] = grid[i][j] + min(dp[i-1][j],  
dp[i][j-1]).  
# First row: only right. First col: only  
down.  
# Modify grid in-place — no extra space.  
 $O(m*n)$  time,  $O(1)$  extra."  
  
# PHASE 4 — CLEAN CODE  
class _64_MinPathSum:  
    def minPathSum(self, grid:  
List[List[int]]) -> int:  
        m, n = len(grid), len(grid[0])  
        for r in range(m):  
            for c in range(n):  
                if r == 0 and c == 0:  
                    continue  
                elif r == 0:  
                    grid[r][c] +=  
grid[r][c - 1] # can only come from left  
                elif c == 0:
```

```

        grid[r][c] += grid[r
- 1][c] # can only come from above
    else:
        grid[r][c] +=
min(grid[r - 1][c], grid[r][c - 1])
    return grid[m - 1][n - 1]

```

PHASE 5 — TEST & DEBUG

`[[1,3,1],[1,5,1],[4,2,1]]`:

Row 0: `[1,4,5]`. Col 0: `[1,2,6]`.

`(1,1)=5+min(4,2)=7. (1,2)=1+min(7,5)=6.`

`(2,1)=2+min(7,6)=8. (2,2)=1+min(6,8)=7.`

→ 7 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(1)$ – modifies grid in place.

If we can't modify the input, use a 1D rolling array: $O(n)$ extra space.

Given more time I'd ask: what if we can also move up or left?

That breaks the DP recurrence – shortest path with general movement is Dijkstra's."

Matrix

□ #73 Set Matrix Zeroes

MED[Open on LeetCode](#)

Array · Hash Table · Matrix

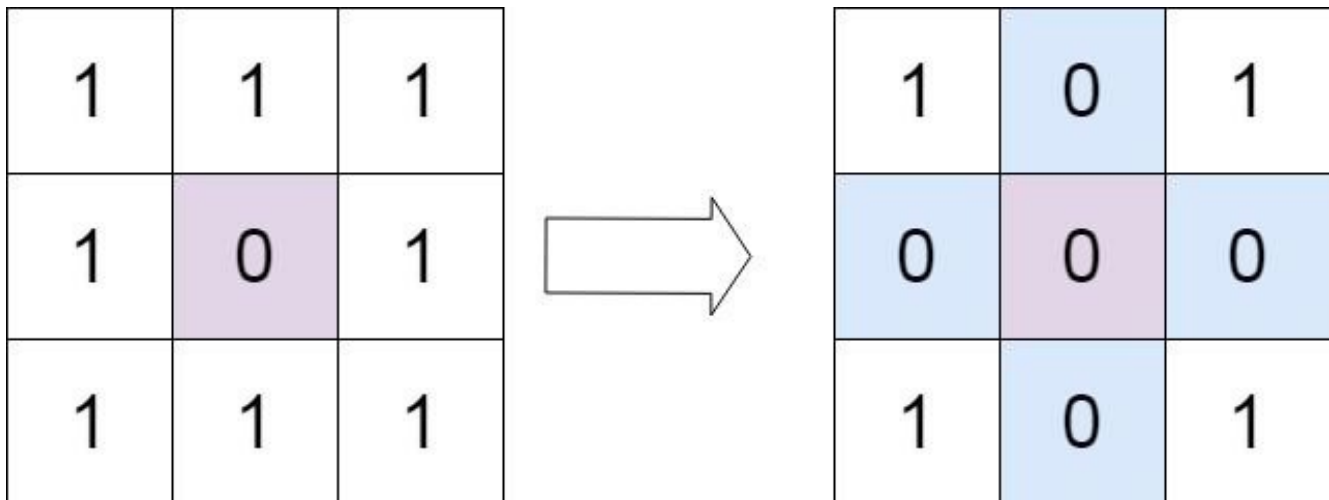
Lists: Grind 169

Problem

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:



Input: matrix =

[[1,1,1],[1,0,1],[1,1,1]]

Output: [[1,0,1],[0,0,0],[1,0,1]]

Example 2:

0	1	2	0		0	0	0	0
3	4	5	2	→	0	4	5	0
1	3	1	5		0	3	1	0

Input: matrix =

[[0,1,2,0],[3,4,5,2],[1,3,1,5]]

Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[0].\text{length}$
- $1 \leq m, n \leq 200$
- $-231 \leq \text{matrix}[i][j] \leq 231 - 1$

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.

- Could you devise a constant space solution?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"If a cell is 0, zero out its entire row and column. Do it in-place. Right?"

We must use the ORIGINAL zeros – not zeros we set during the process."

#

"[[1,0,1],[1,1,1],[1,1,1]] →
[[0,0,0],[1,0,1],[1,0,1]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

First collect every (r,c) where matrix[r][c]==0 into a list.

Second pass: zero out entire rows and columns for each stored zero.

Simple, but storing all zero coordinates uses $O(m+n)$ extra structures.

Time: $O(m * n)$. Space: $O(m + n)$ for row/col sets.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE ($O(1)$ space using first row/col as markers)

"Use the first row and first column of the matrix itself as markers.

If `matrix[r][c]==0`: mark `matrix[r][0]=0` and `matrix[0][c]=0`.

Handle first row and first column separately (track with boolean flags).

Then zero out interior cells using the markers.

Then zero out first row/col based on flags. $O(m*n)$ time, $O(1)$ space."

PHASE 4 — CLEAN CODE

```
class _73_SetMatrixZeroes:
```

```
    def setZeroes(self, matrix:
```

```
List[List[int]]) -> None:
```

```
        m, n = len(matrix),
```

```
        len(matrix[0])
```

```
        first_row_zero = any(matrix[0][c]
== 0 for c in range(n))
```

```
        first_col_zero = any(matrix[r][0]
== 0 for r in range(m))
```

```
        for r in range(1, m): # mark zeros
using first row/col
```

```
            for c in range(1, n):
```

```
        if matrix[r][c] == 0:
            matrix[r][0] = 0
            matrix[0][c] = 0
    for r in range(1, m): # zero out
interior
        for c in range(1, n):
            if matrix[r][0] == 0 or
matrix[0][c] == 0:
                matrix[r][c] = 0
    if first_row_zero: # zero out
first row if needed
        for c in range(n):
matrix[0][c] = 0
    if first_col_zero: # zero out
first col if needed
        for r in range(m):
matrix[r][0] = 0

# PHASE 5 — TEST & DEBUG
# [[0,1,2,0],[3,4,5,2],[1,3,1,5]]:
# first_row_zero: matrix[0][0]=0 → True.
first_col_zero: matrix[0][0]=0 → True.
# Mark: r=1,c=0: already 3≠0. r=1,c=3:
2≠0... only (0,0) and (0,3) have zeros in
row 0.
```

```
# Interior zeros: marker[0][0]=0→col 0  
zeros. marker[0][3]: 0→col 3 zeros.  
# → [[0,0,0,0],[0,4,5,0],[0,3,1,0]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(1)$.

The first row/col flags are the key – without them, using matrix[0][0] as a marker

for both first row and first col creates an ambiguity. Two flags resolve it cleanly.

The four-step order (flag, mark, zero interior, zero edges) is the correct sequence.

Given more time I'd ask: what if we should set a cell to the average of its row/col instead?

Same structure – two-pass approach to avoid using updated values."

Matrix

□ #74 Search a 2D Matrix

MED[Open on LeetCode](#)

Array · Binary Search · Matrix

Lists: Grind 169

Problem

You are given an $m \times n$ integer matrix matrix with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer target, return true if target is in matrix or false otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = `[[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, target = 13
Output: false

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 100`
- `-104 <= matrix[i][j], target <= 104`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Matrix where each row is sorted and the first element of each row is greater than the last of the previous row – so the whole matrix is one sorted sequence.

Find target in $O(\log(m*n))$. Right?"

#

"Treat it as a 1D array of length $m*n$. Index k maps to $row=k//n$, $col=k\%n$."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Treat the matrix as a flattened sorted array and binary search on index $0..m*n-1$.

Or scan row by row with binary search per row.

Correct $O(m \log n)$, but a single binary search on the virtual array is cleaner.

Time: $O(m \log n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search on the virtual 1D array $[0, m*n-1]$.

```
# Map mid to (mid//n, mid%n) to access the actual cell.
```

```
#  $O(\log(m*n)) = O(\log m + \log n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _74_Search2DMatrix:
```

```
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
```

```
        left, right = 0, m * n - 1
```

```
        while left <= right:
```

```
            mid = (left + right) // 2
```

```
            mid_val = matrix[mid // n][mid % n]
```

```
            if mid_val == target:
```

```
                return True
```

```
            elif mid_val < target:
```

```
                left = mid + 1
```

```
            else:
```

```
                right = mid - 1
```

```
        return False
```

PHASE 5 — TEST & DEBUG

```
# matrix=[[1,3,5,7],[10,11,16,20],[23,30,34,60]], target=3:
```

```
# l=0,r=11. mid=5→matrix[1][1]=11.  
11>3→r=4.  
# mid=2→matrix[0][2]=5. 5>3→r=1.  
# mid=0→matrix[0][0]=1. 1<3→l=1.  
# mid=1→matrix[0][1]=3. 3==3→True ✓  
# target=13: binary search narrows until  
l>r → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log(m*n))$. Space $O(1)$.

The index mapping $mid//n$ and $mid\%n$ is the entire trick.

This only works because the matrix is fully sorted as one sequence.

If rows are sorted but first-of-next isn't > last-of-prev (Search a 2D Matrix II #240):

Use staircase search – start at top-right, move left if too big, down if too small.

$O(m+n)$ time – different problem, different approach."

Matrix

□ #221 Maximal Square

MED[Open on LeetCode](#)

Array · Dynamic Programming · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],
["1","0","1","1","1"],
["1","1","1","1","1"],
["1","0","0","1","0"]]`

Output: 4

Example 2:

0	1
1	0

Input: matrix = `[["0","1"],["1","0"]]`
Output: 1

Example 3:

Input: matrix = `[["0"]]`
Output: 0

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`

- $1 \leq m, n \leq 300$
- `matrix[i][j]` is '0' or '1'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the largest square of all 1s.
Return its area. Matrix contains '0'/'1'
strings. Right?"

#

"[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]] → 2×2=4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

For every cell containing '1', expand
square side length while all cells in
square are '1'.

Track the maximum side length found
across all starting cells.

Each expansion check is $O(k)$ for side k
– overall $O(m \times n \times \min(m, n)^2)$.

Time: $O(m \times n \times \min(m, n)^2)$. Space:
 $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DP)

"dp[i][j] = side length of the largest all-1s square with bottom-right corner at (i,j)."

If matrix[i][j]=='0': dp[i][j]=0.

Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.

Intuition: a square of side k can be extended by 1 only if the top, left, and diagonal neighbour

each have a square of side k. The bottleneck is the minimum.

Track max side length → return side²."

PHASE 4 — CLEAN CODE

```
class _221_MaximalSquare:
```

```
    def maximalSquare(self, matrix:  
List[List[str]]) -> int:
```

```
        m, n = len(matrix),  
len(matrix[0])
```

```
        dp = [[0] * n for _ in range(m)]
```

```
        max_side = 0
```

```
        for r in range(m):
```

```
            for c in range(n):
```

```
                if matrix[r][c] == '1':
```



```

        top = dp[r - 1][c] if
r > 0 else 0
        left = dp[r][c - 1] if
c > 0 else 0
        diag = dp[r - 1][c -
1] if r > 0 and c > 0 else 0
        dp[r][c] = min(top,
left, diag) + 1
        max_side =
max(max_side, dp[r][c])
    return max_side * max_side

```

PHASE 5 — TEST & DEBUG

```

# "[[0,1],['1','0']]: no 2x2 all-1s → dp
stays at 1s and 0s → max=1 → area=1 ✓
# Full example: bottom-right region gives
dp[2][3]=dp[2][4]=2, max=2, area=4 ✓"

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m*n). Space O(m*n) – reducible
to O(n) with a rolling array.
# The min of three neighbours is the core
insight – think of it as the square being
# limited by the weakest neighbouring
corner.

```

Given more time I'd ask: maximal rectangle of 1s (not just square)?
That's Maximal Rectangle (#85) – use a histogram approach per row, $O(m \times n)$."

Matrix

★ #311 Sparse Matrix Multiplication ★

MED[Open on LeetCode](#)

Array · Hash Table · Matrix

Lists: ★ Premium | Premium 98

Problem

Given two sparse matrices `mat1` of size $m \times k$ and `mat2` of size $k \times n$, return the result of `mat1` x `mat2`. You may assume that multiplication is always possible.

Example 1:

1	0	0
-1	0	3

X

7	0	0
0	0	0
0	0	1

=

7	0	0
-7	0	3

Input: `mat1 = [[1,0,0],[-1,0,3]]`, `mat2 = [[7,0,0],[0,0,0],[0,0,1]]`
Output: `[[7,0,0],[-7,0,3]]`

Example 2:

Input: mat1 = [[0]], mat2 = [[0]]
 Output: [[0]]

Constraints:

- $m == \text{mat1.length}$
- $k == \text{mat1}[i].\text{length} == \text{mat2.length}$
- $n == \text{mat2}[i].\text{length}$
- $1 \leq m, n, k \leq 100$
- $-100 \leq \text{mat1}[i][j], \text{mat2}[i][j] \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Multiply mat1 ($m \times k$) by mat2 ($k \times n$).
 Return the $m \times n$ result.

Both are sparse – many zeros. Optimise
 to skip zero elements. Right?"

#

"[[1,0,0],[-1,0,3]] ×
 [[7,0,0],[0,0,0],[0,0,1]] =
 [[7,0,0],[-7,0,3]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
 in the logic."

```
# Standard triple loop: for each row i and  
col k, sum A[i][j]*B[j][k] over j.  
# Correct even for sparse inputs but  
touches every (i,j,k) combination.  
# Skipping zero rows/cols from sparse  
representation saves huge work.  
# Time:  $O(m * k * n)$ . Space:  $O(m * n)$   
result.  
# Should I go ahead and optimize?"
```

```
class _311_SparseMultiply_Brute:  
    def multiply(self, mat1:  
List[List[int]], mat2: List[List[int]])  
-> List[List[int]]:  
        m, k, n = len(mat1), len(mat1[0]),  
len(mat2[0])  
        result = [[0] * n for _ in  
range(m)]  
        for i in range(m):  
            for p in range(k):  
                for j in range(n):  
                    result[i][j] +=  
mat1[i][p] * mat2[p][j]  
        return result
```

PHASE 3 — OPTIMIZE

```
# "Skip multiplication when mat1[i][p] ==  
0 – avoids the inner j loop entirely.  
# Pre-compute non-zero positions in mat1  
for even faster access.  
# This is the key sparse optimisation:  
skip zero elements before accessing mat2."
```

PHASE 4 — CLEAN CODE

```
class _311_SparseMultiply:  
    def multiply(self, mat1:  
List[List[int]], mat2: List[List[int]])  
-> List[List[int]]:  
        m, k, n = len(mat1), len(mat1[0]),  
len(mat2[0])  
        result = [[0] * n for _ in  
range(m)]  
        for i in range(m):  
            for p in range(k):  
                if mat1[i][p] == 0:  
                    continue # skip –  
avoids entire inner loop  
                for j in range(n):  
                    result[i][j] +=  
mat1[i][p] * mat2[p][j]  
        return result
```

PHASE 5 — TEST & DEBUG

$\text{mat1}[0][0]=1$, $p=0$: add $1*\text{mat2}[0][j]$ to $\text{result}[0]$. $\text{mat1}[0][1]=0 \rightarrow$ skip.

$\text{mat1}[0][2]=0 \rightarrow$ skip.

$\text{result}[0]=[7,0,0]$ ✓. $\text{mat1}[1][2]=3$, $p=2$:
add $3*\text{mat2}[2][j]=[0,0,3] \rightarrow$
 $\text{result}[1]=[-7,0,3]$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Worst case $O(m*k*n)$ same as brute force. Best case $O(\text{nnz1} * n)$ where nnz1 = non-zeros in mat1 .

For truly sparse matrices this is much faster.

Given more time I'd ask: what if both matrices are very large and very sparse?

Use CSR (Compressed Sparse Row) format — store only non-zero values and their indices.

Hash map approach: for each non-zero in mat1 , look up non-zeros in the corresponding row of mat2 ."

Matrix

□ #498 Diagonal Traverse

MED[Open on LeetCode](#)

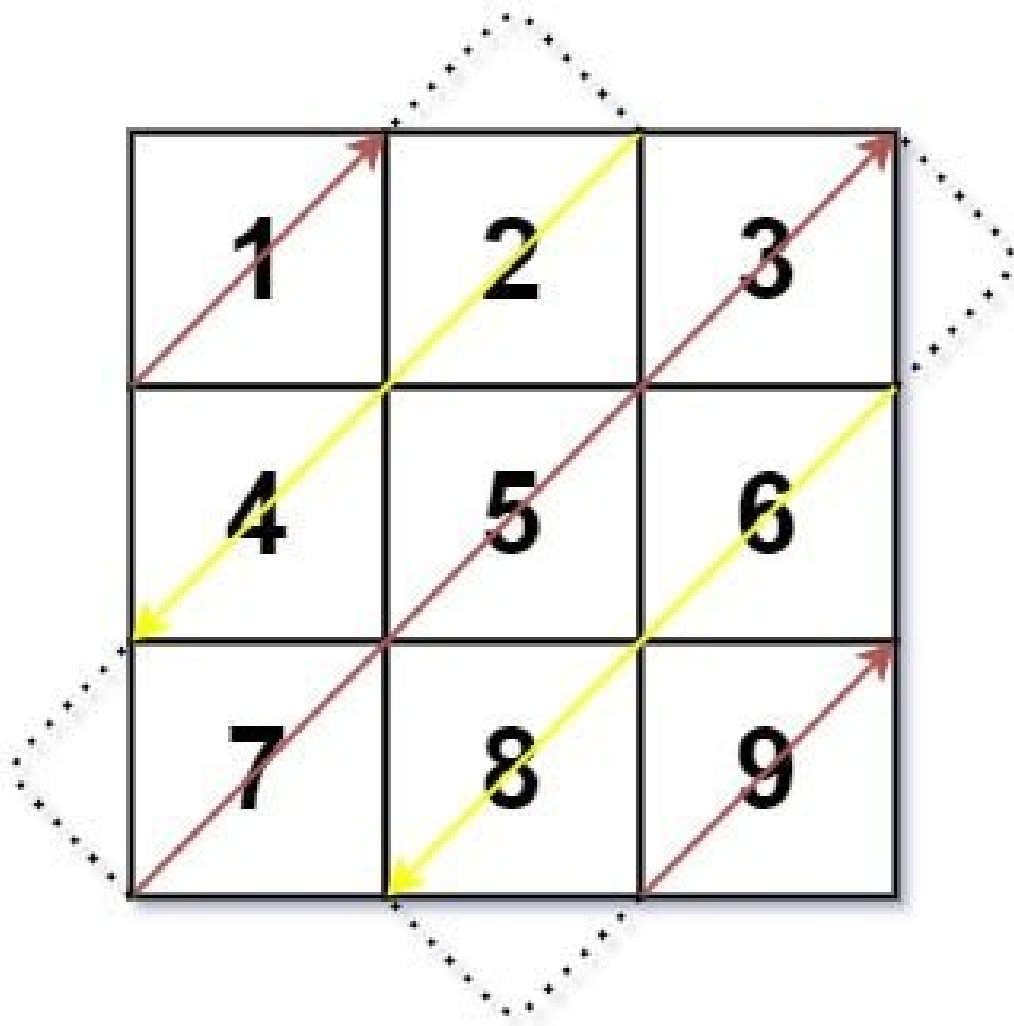
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given an $m \times n$ matrix `mat`, return an array of all the elements of the array in a diagonal order.

Example 1:



Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[1,2,4,7,5,3,6,8,9]`

Example 2:

Input: `mat = [[1,2],[3,4]]`

Output: `[1,2,3,4]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`

- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$
- $-105 \leq \text{mat}[i][j] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Traverse all elements diagonally: alternate up-right then down-left directions.

There are $m+n-1$ diagonals. Even-numbered diagonals go up-right, odd go down-left. Right?"

#

"[[1,2,3],[4,5,6],[7,8,9]] → [1,2,4,7,5,3,6,8,9] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Simulate diagonal walk with direction flags; flip direction at matrix boundaries.

Track visited or use index math to avoid infinite loops.

```
# Correct  $O(m \cdot n)$ ; index-formula approach  
avoids simulation bugs.  
# Time:  $O(m \cdot n)$ . Space:  $O(m \cdot n)$  output.  
# Should I go ahead and optimize?"  
  
# PHASE 3 — OPTIMIZE (diagonal grouping)  
# "Group elements by diagonal  $d = r + c$   
(same sum = same diagonal).  
# Diagonal  $d$  contains elements where  
 $r + c = d$ .  
# For even  $d$ : traverse top-to-bottom order  
reversed (up-right) → sort by  $r$   
descending.  
# For odd  $d$ : traverse top-to-bottom  
(down-left) → sort by  $r$  ascending."  
  
# PHASE 4 — CLEAN CODE  
class _498_DiagonalTraverse:  
    def findDiagonalOrder(self, mat:  
List[List[int]]) -> List[int]:  
        m, n = len(mat), len(mat[0])  
        diags: defaultdict =  
defaultdict(list)  
        for r in range(m):  
            for c in range(n):
```

```

            diags[r +
c].append(mat[r][c])
        result = []
        for d in range(m + n - 1):
            if d % 2 == 0:
                result.extend(reversed(di
ags[d])) # even → up-right (larger r first
→ reversed)
            else:
                result.extend(diags[d]) #
odd → down-left (smaller r first →
natural)
        return result

```

PHASE 5 — TEST & DEBUG

```

# [[1,2,3],[4,5,6],[7,8,9]]:
# d=0: {(0,0)=1} even → reversed=[1]. d=1:
{(0,1)=2,(1,0)=4} odd → [2,4].
# d=2: {(0,2)=3,(1,1)=5,(2,0)=7} even →
reversed=[7,5,3].
# d=3: {(1,2)=6,(2,1)=8} odd → [6,8]. d=4:
{(2,2)=9} even → reversed=[9].
# → [1,2,4,7,5,3,6,8,9] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ for the diagonal groups.

The 'group by $r+c$ ' insight eliminates direction-tracking complexity.

Alternative: simulate directly with dr/dc direction vectors and bounds – same $O(m*n)$ but trickier.

Given more time I'd ask: what if we want anti-diagonals ($r-c = \text{constant}$)?

Same approach – group by $r-c$, alternate direction per group."

Matrix

□ ★ #531 Lonely Pixel I ★

MED

[Open on LeetCode](#)

Array · Hash Table · Matrix

Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ picture consisting of black 'B' and white 'W' pixels, return the number of black lonely pixels.

A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

Example 1:

W	W	B
W	B	W
B	W	W

Input: `picture = [ ["W","W","B"], ["W","B","W"], ["B","W","W"] ]`

Output: 3

Explanation: All the three 'B's are black lonely pixels.

Example 2:

B	B	B
B	B	W
B	B	B

Input: `picture = [ ["B","B","B"], ["B","B","W"], ["B","B","B"] ]`

Output: `0`

Constraints:

- `m == picture.length`
- `n == picture[i].length`
- `1 <= m, n <= 500`
- `picture[i][j]` is 'W' or 'B'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A 'B' pixel is lonely if it's the ONLY 'B' in its row AND the only 'B' in its column.

Count lonely pixels. Right?"

#

"[[W,W,B],[W,B,W],[B,W,W]]: each B is alone in its row and column → 3 ✓

[[B,B,B],[B,B,W],[B,B,B]]: every B shares its row or column with another → 0 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each black pixel B, scan its entire row for other black pixels.

Then scan its entire column – B is lonely if both counts equal 1.

$O(m \cdot n \cdot (m + n))$ – checking row and column per pixel.

Time: $O(m * n * (m + n))$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Count B's per row and per column in one pass: 0(m*n).

In second pass, for each B: if row_count[r]==1 and col_count[c]==1, it's lonely."

PHASE 4 — CLEAN CODE

class _531_LonelyPixel:

**def findLonelyPixel(self, picture:
List[List[str]]) -> int:**

**m, n = len(picture),
len(picture[0])**

**row_count = [sum(1 for c in
range(n) if picture[r][c] == 'B') for r in
range(m)]**

**col_count = [sum(1 for r in
range(m) if picture[r][c] == 'B') for c in
range(n)]**

**return sum(
1 for r in range(m) for c in
range(n)**

**if picture[r][c] == 'B' and
row_count[r] == 1 and col_count[c] == 1
)**

PHASE 5 — TEST & DEBUG

```
# [[W,W,B],[W,B,W],[B,W,W]]:  
row_count=[1,1,1]. col_count=[1,1,1].  
# Each B has row=1 and col=1 → all 3 are  
lonely ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m+n)$ for row/col counts.

Given more time I'd ask: Lonely Pixel II (#533)?

Same pixel must be in a column containing exactly N Bs, and the rows of those Bs must all be identical.

Much more complex – requires comparing full rows."

Matrix

□ ★ #723 Candy Crush ★

MED[Open on LeetCode](#)

Array · Two Pointers · Matrix · Simulation

Lists: ★ Premium | Premium 98

Problem

This question is about implementing a basic elimination algorithm for Candy Crush.

Given an $m \times n$ integer array `board` representing the grid of candy where `board[i][j]` represents the type of candy. A value of `board[i][j] == 0` represents that the cell is empty.

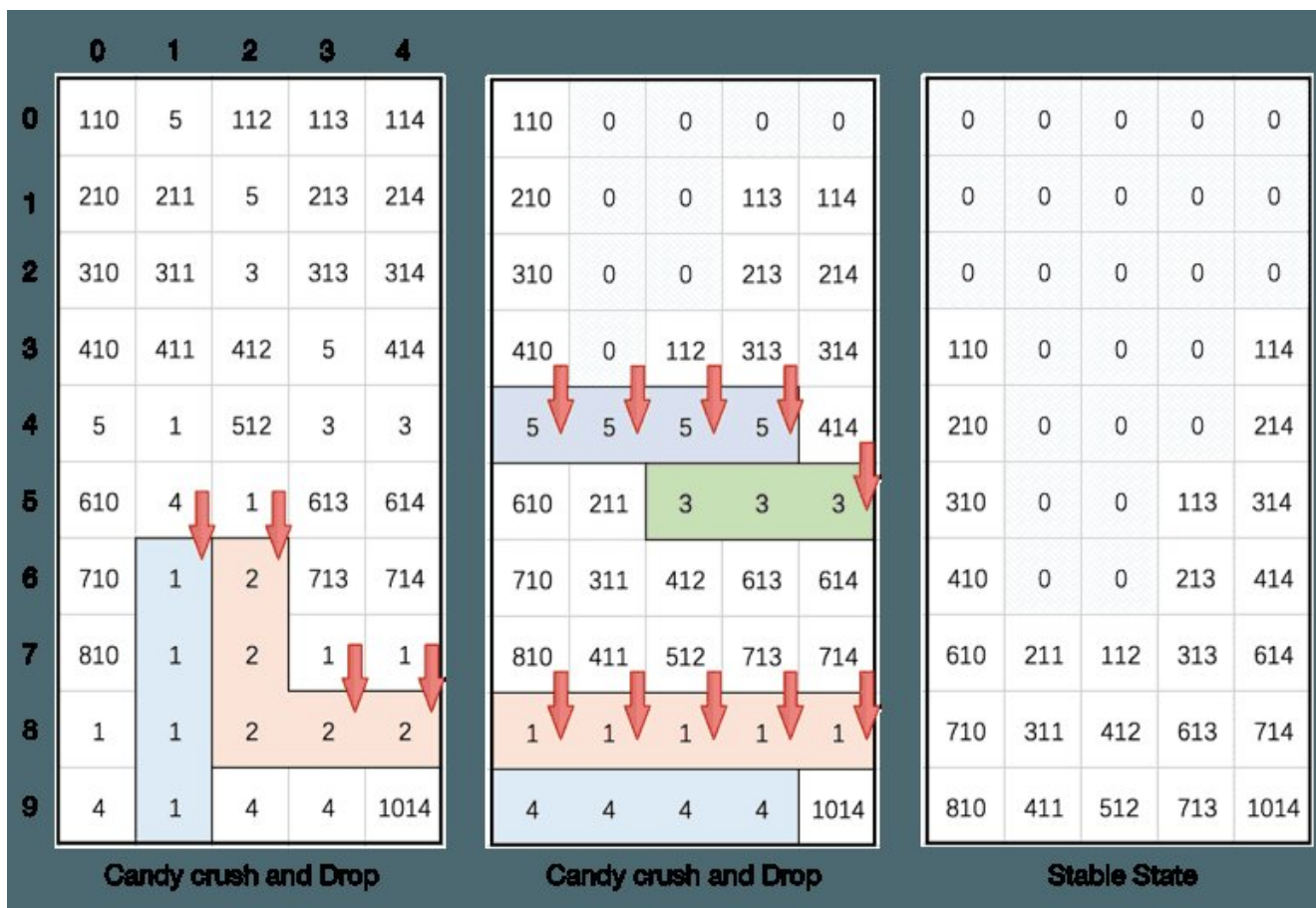
The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time – these positions become empty.

- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1:



Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]

Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]

Example 2:

```
Input: board = [[1,3,5,5,2],[3,4,3,3,1]
,[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]
Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0
,3,1],[2,4,0,5,2],[1,4,3,1,1]]
```

Constraints:

- `m == board.length`
- `n == board[i].length`
- `3 <= m, n <= 50`
- `1 <= board[i][j] <= 2000`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Repeatedly: mark 3+ same-type candies horizontally or vertically, crush them (set to 0),

then drop remaining candies down. Repeat until stable. Return final board. Right?"

#

"Mark all runs of 3+ simultaneously, crush all at once, then gravity – not one at a time."

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Repeat until stable: find
horizontal/vertical runs of 3+, mark for
crush, drop gravity.
# One full board scan per cascade step.
# Correct simulation; finding all runs in
one pass per step is cleaner.
# Time:  $O((m*n)^2)$  worst cascades. Space:
 $O(m * n)$  mark board.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (in-place marking)
# "Mark by negating values (sign flip) —
avoids extra space.
# Step 1: scan rows and cols, negate any
value in a run of 3+.
# Step 2: zero out all negatives (crush).
# Step 3: for each column, compact
non-zeros to the bottom (gravity).
# Repeat until no changes in step 1. Time:
 $O((m*n)^2)$  worst case but small in
practice."

# PHASE 4 — CLEAN CODE
class _723_CandyCrush:
```



```
def candyCrush(self, board:
List[List[int]]) -> List[List[int]]:
    m, n = len(board), len(board[0])
    while True:
        crush = set()
        for r in range(m): # mark
horizontal runs
            for c in range(n - 2):
                v = abs(board[r][c])
                if v and v ==
abs(board[r][c+1]) == abs(board[r][c+2]):
                    crush.update([(r,
c), (r, c+1), (r, c+2)])
            for r in range(m - 2): # mark
vertical runs
                for c in range(n):
                    v = abs(board[r][c])
                    if v and v ==
abs(board[r+1][c]) == abs(board[r+2][c]):
                        crush.update([(r,
c), (r+1, c), (r+2, c)])
                if not crush:
                    break
        for r, c in crush:
            board[r][c] = 0
```

```

        for c in range(n): # gravity:
compact column downward
            write = m - 1
            for r in range(m - 1, -1,
-1):
                if board[r][c] != 0:
                    board[write][c] =
board[r][c]
                    if write != r:
                        board[r][c] =
0
                    write -= 1
            return board

```

PHASE 5 — TEST & DEBUG

```

# "[[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[
2,4,4,5,5],[1,4,4,1,1]]:

```

```

# Horizontal: row 1 has [3,3,3] at
c=1,2,3. Row 3 has [4,4] and [5,5]...

```

```

# After crushing and dropping → stable
board matches expected output ✓"

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "O((m*n)2) worst case – at most m*n
crush iterations, each O(m*n).

```

For small boards ($m, n \leq 50$) this is fast.
The `abs()` trick avoids needing a separate mark array.
The gravity step uses a two-pointer write-head approach per column.
Given more time I'd ask: what if candies can crush diagonally too?
Add diagonal scan to the marking step – same structure."

Matrix

□ #835 Image Overlap

MED[Open on LeetCode](#)

Array · Matrix

Lists: CodeSignal

Problem

You are given two images, `img1` and `img2`, represented as binary, square matrices of size $n \times n$. A binary matrix has only 0s and 1s as values.

We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

Return the largest possible overlap.

Example 1:

1	1	0
0	1	0
0	1	0

img1

0	0	0
0	1	1
0	0	1

img2

Input: `img1 =`
`[[1,1,0],[0,1,0],[0,1,0]]`, `img2 =`
`[[0,0,0],[0,1,1],[0,0,1]]`

Output: 3

Explanation: We translate `img1` to right by 1 unit and down by 1 unit.

The number of positions that have a 1 in both images is 3 (shown in red).

Example 2:

Input: `img1 = [[1]]`, `img2 = [[1]]`
Output: 1

Example 3:

Input: `img1 = [[0]], img2 = [[0]]`
Output: `0`

Constraints:

- `n == img1.length == img1[i].length`
- `n == img2.length == img2[i].length`
- `1 <= n <= 30`
- `img1[i][j]` is either 0 or 1.
- `img2[i][j]` is either 0 or 1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Translate `img1` over `img2` (no rotation) and count overlapping 1s.

Find the translation that maximises overlap. Return max overlap count. Right?"

#

"`img1=[[1,1,0],[0,1,0],[0,1,0]]`,
`img2=[[0,0,0],[0,1,1],[0,0,1]]` → 3 ✓
(translate right 1, down 1)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Try every translation vector (dx,dy)
aligning A onto B.
# For each shift, count overlapping
1-cells.
#  $O(n^4)$  translations on  $n \times n$  images –
too slow for  $n=30$ .
# Time:  $O(n^4)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (translation vector
counting)
# "Collect all 1-positions in each image.
# For each pair (p1 from img1, p2 from
img2), the translation to align p1 with p2
is (p2.r-p1.r, p2.c-p1.c).
# Count the most common translation vector
– that's the maximum overlap.
# Time:  $O(A*B)$  where  $A, B$  = number of 1s.
Space:  $O(A*B)$ . For  $n=30$ ,  $A, B \leq 900$ ."

# PHASE 4 — CLEAN CODE
class _835_ImageOverlap:
    def largestOverlap(self, img1:
List[List[int]], img2: List[List[int]])
-> int:
        n = len(img1)
```

```
        ones1 = [(r, c) for r in range(n)
for c in range(n) if img1[r][c] == 1]
        ones2 = [(r, c) for r in range(n)
for c in range(n) if img2[r][c] == 1]
        translations: Counter = Counter()
        for r1, c1 in ones1:
            for r2, c2 in ones2:
                translations[(r2 - r1, c2
- c1)] += 1
        return max(translations.values(),
default=0)
```

PHASE 5 — TEST & DEBUG

```
# ones1=[(0,0),(0,1),(1,1),(2,1)].
```

```
ones2=[(1,1),(1,2),(2,2)].
```

```
# Translation (1,1) appears when:
```

```
(0,0)→(1,1)✓, (0,1)→(1,2)✓, (1,1)→(2,2)✓
→ count=3.
```

```
# max=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(A*B)$  where  $A,B$  = number of 1s in
each image, at most  $n^2$ .
```

```
# Space  $O(A*B)$  for the translation
Counter.
```


For dense images (many 1s), this is $O(n^4)$ – same as brute force.
For sparse images, much faster.
The key insight: instead of iterating over all translations, only count translations where at least one 1-pair aligns – no wasted iterations.
Given more time I'd ask: what if rotation is also allowed?
Need to try all rotation+translation combos – much harder."

Matrix

□ ★ #1198 Find Smallest Common Element in All Rows ★

MED

[Open on LeetCode](#)

Array · Hash Table · Binary Search · Matrix · Counting

Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ matrix `mat` where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return `-1`.

Example 1:

Input: `mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]`

Output: `5`

Example 2:

Input: `mat = [[1,2,3],[2,3,4],[2,3,5]]`

Output: `2`

Constraints:

- `m == mat.length`

- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 500$
- $1 \leq \text{mat}[i][j] \leq 10^4$
- $\text{mat}[i]$ is sorted in strictly increasing order.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the smallest integer that appears in every row. Each row is sorted.

Return -1 if no common element exists. Right?

Values are $1-10^4$, at most 500 rows and 500 cols."

#

"[[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]] → 5 (appears in all rows) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Intersect row0 with row1, then intersect result with row2, and so on.

Track the maximum value in the final intersection set.

Each intersection pass is $O(m)$ per row using a hash set of row values.

Time: $O(\text{rows} * \text{cols})$. Space: $O(\text{cols})$ set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (frequency count)

"Count how many rows each value appears in using a single array of size $\text{max_val}+1$.

Walk all rows: for each value encountered, increment $\text{count}[\text{value}]$.

Return the smallest value with $\text{count} == m$ (appears in ALL rows).

Time: $O(m*n)$. Space: $O(\text{max_val})$."

PHASE 4 — CLEAN CODE

```
class _1198_SmallestCommonElement:
    def smallestCommonElement(self, mat:
List[List[int]]) -> int:
        m = len(mat)
        freq = Counter()
        for row in mat:
            for val in row:
                freq[val] += 1
        for val in range(1, 10001):
            if freq[val] == m:
```

```
        return val
    return -1
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],
#  [1,3,5,7,9]]:
# freq[5] = 4 (appears in all 4 rows).
# freq[1]=2, freq[3]=3, etc.
# Scanning 1..10000: first val with
# freq==4 is 5 → return 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m \cdot n + \max\_val)$ . Space
#  $O(\max\_val)$ .
# Alternative: binary search per row. For
# each candidate, binary search in all m
# rows.
# Start candidates from mat[0] (smallest
# row), binary search each of the other
# rows.
#  $O(n + m \cdot n \cdot \log n)$  – worse than the
# counting approach for dense matrices.
# Given more time I'd ask: what if there's
# no bound on values?
# Use a hash Counter for freq –  $O(m \cdot n)$ 
# time,  $O(\text{distinct values})$  space."
```

Trees & BST

Round 2 · High · Medium · 24 questions

Trees & BST

□ #98 Validate Binary Search Tree

MED

[Open on LeetCode](#)

Tree · DFS · BST

Lists: Grind 169

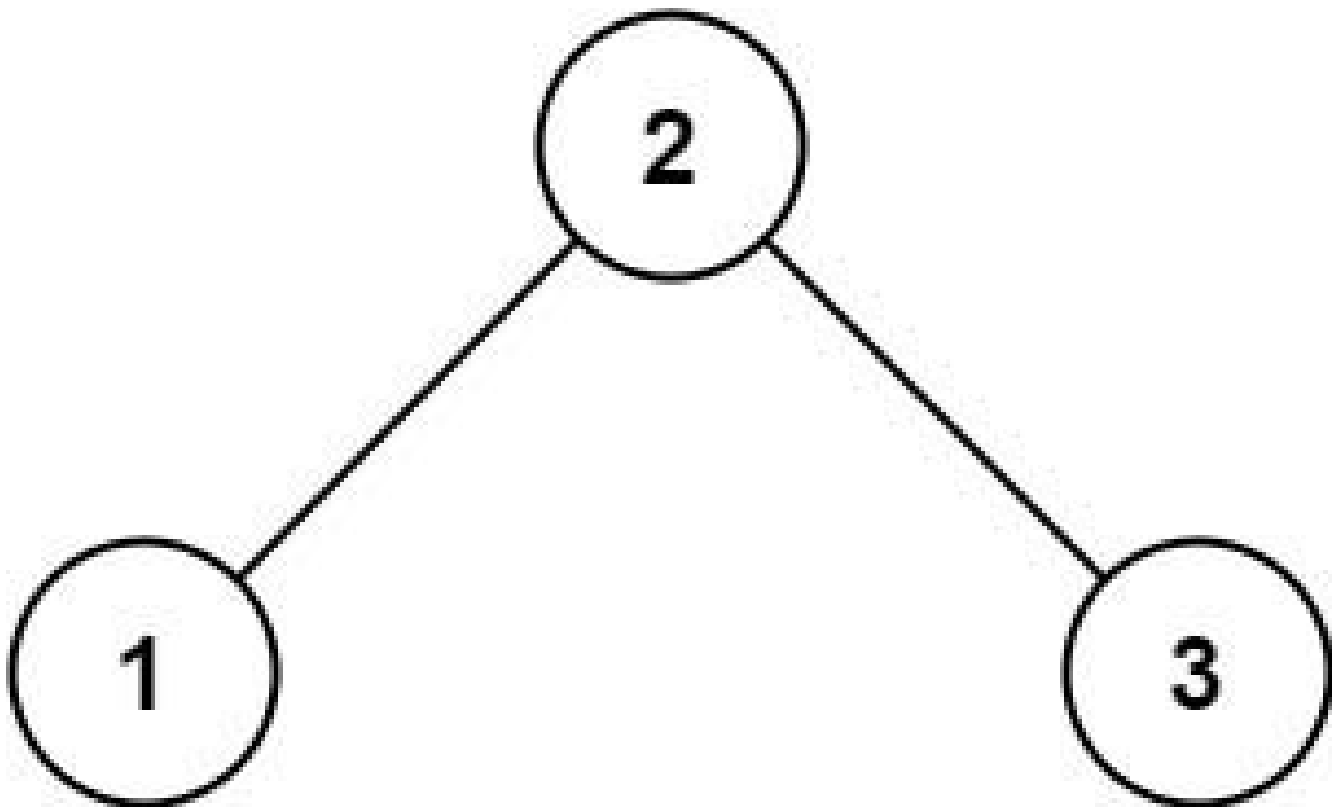
Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

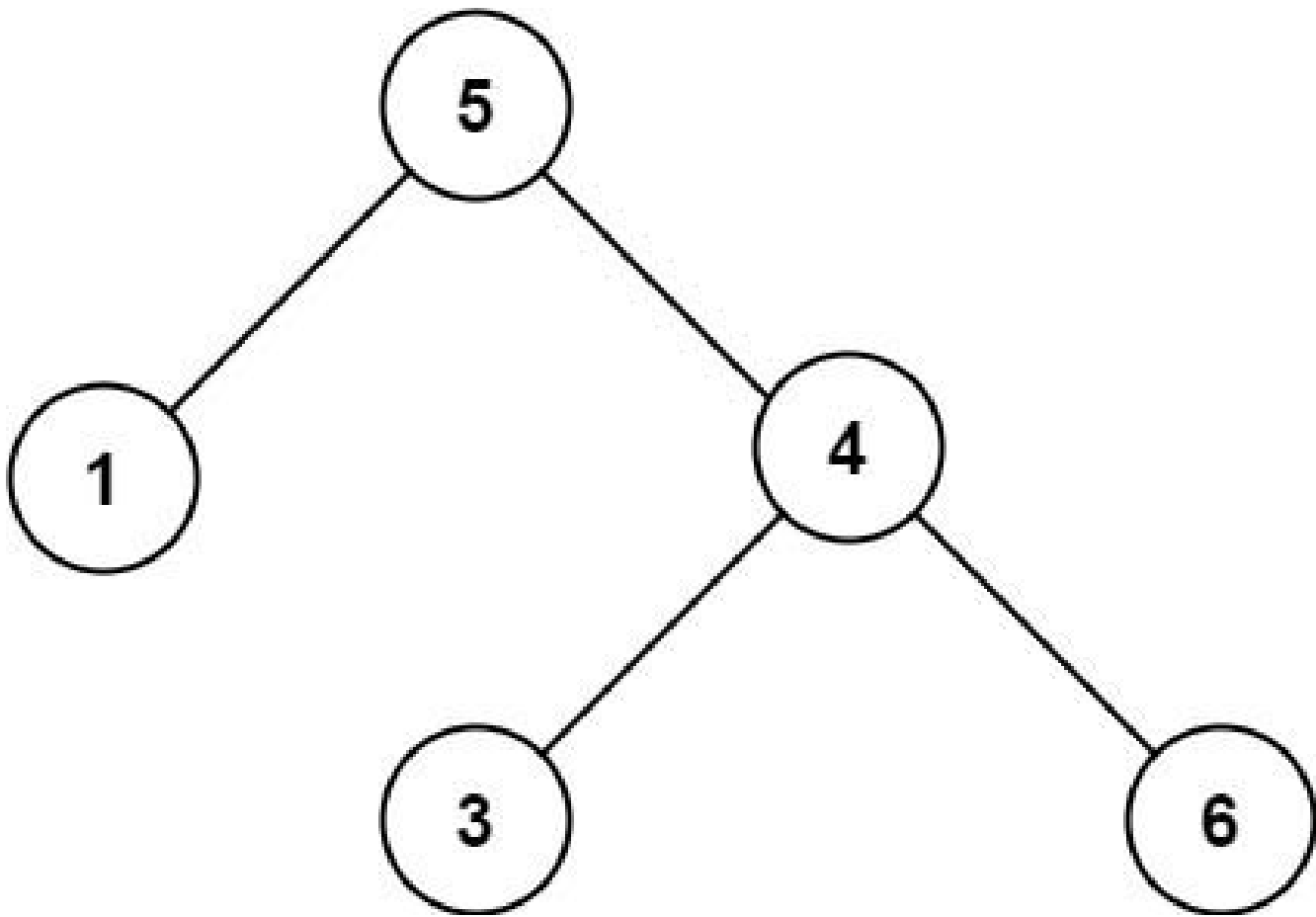
- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: root = [2,1,3]
Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Every node's value must be strictly greater than all values in its left subtree

AND strictly less than all values in its right subtree – not just the immediate parent. Right?

The classic trap: [5,1,4,null,null,3,6] – root=5, right child=4 – $4 < 5 \rightarrow$ invalid ✓"

#

"[2,1,3]: $1 < 2 < 3 \rightarrow$ true ✓.

[5,1,4,null,null,3,6]: $4 < 5 \rightarrow$ false ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

In-order traversal collects values; verify the list is strictly increasing.

Any duplicate or decrease means invalid BST.

Correct $O(n)$, but recursion can pass min/max bounds instead of storing all values.

Time: $O(n)$. Space: $O(n)$ in-order list.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (bounds propagation)

"Pass min/max bounds down the tree.

Every node must satisfy $\text{min_val} < \text{node.val} < \text{max_val}$.

Left child: $\text{max_val} = \text{node.val}$.

Right child: $\text{min_val} = \text{node.val}$.

Use $\text{float}(-\text{inf})$ and $\text{float}(\text{inf})$ as initial bounds. Use inner helper."

PHASE 4 — CLEAN CODE

class _98_ValidateBST:

def isValidBST(self, root:
Optional['TreeNode']) -> bool:

def valid(node, min_val,
max_val):

if not node:

return True

if not (min_val < node.val <
max_val):

return False

return (valid(node.left,
min_val, node.val) and
valid(node.right,
node.val, max_val))

return valid(root, float('-inf'),
float('inf'))

PHASE 5 — TEST & DEBUG

[2,1,3]: valid(2,-inf,inf)→valid(1,-inf,2)→valid(None,...)=T and T→T.

valid(3,2,inf)→T. → True ✓

[5,1,4,null,null,3,6]: valid(4,5,inf):
4 NOT in (5,inf) → False → outer returns False ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$ for the call stack.

The bounds approach is the correct one — in-order checking works too but requires state.

The common mistake: only checking node vs parent, not the full subtree constraint.

Given more time I'd ask: what if duplicates are allowed ($\text{left} \leq \text{node} < \text{right}$)?

Change the strict inequalities accordingly."

Trees & BST

□ #102 Binary Tree Level Order Traversal

MED

[Open on LeetCode](#)

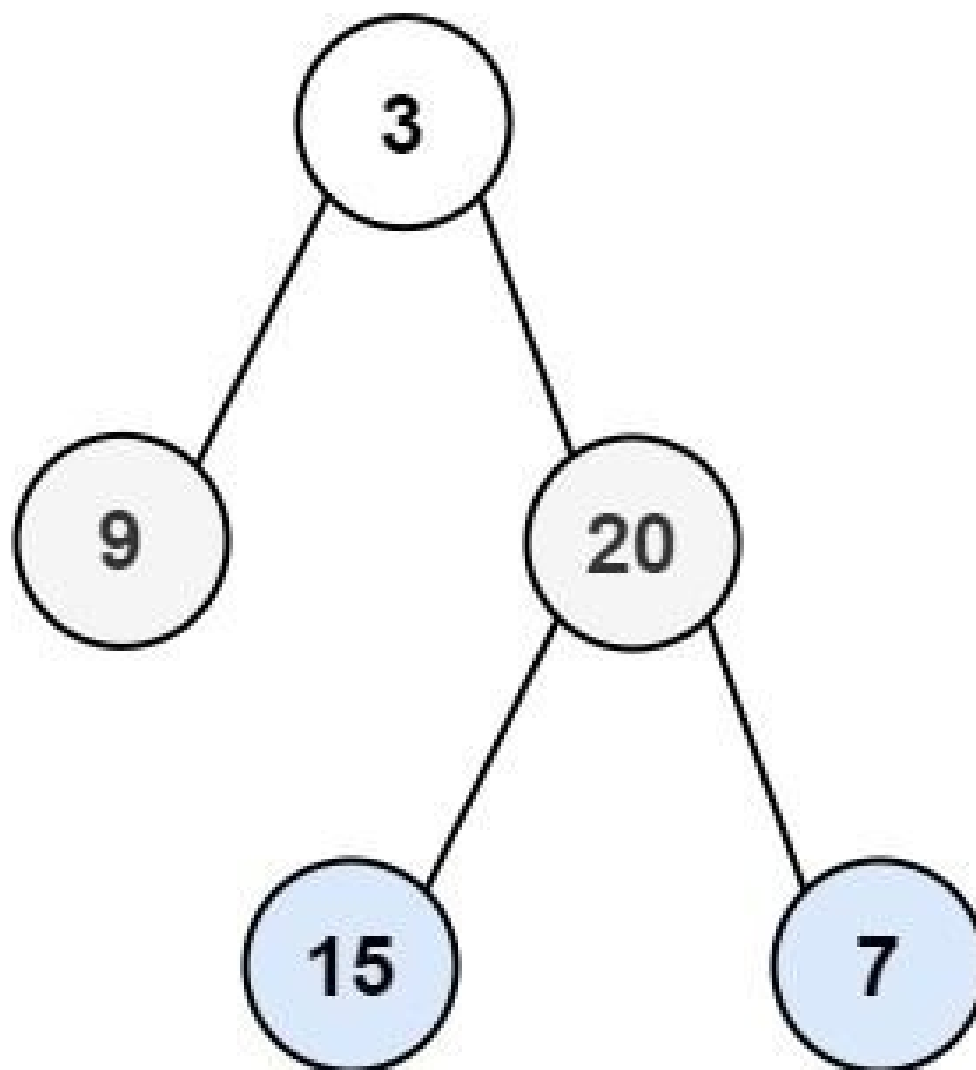
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return node values level by level, left to right, each level as a sub-list. Right?"

Empty tree → []. Single node → [[val]]."

#

"[3,9,20,null,null,15,7] →
[[3],[9,20],[15,7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS with a level parameter: append node.val to result[level].

After traversal, result lists are level-order groups left-to-right.

```
# Works in  $O(n)$ , but BFS with a queue is  
the more natural level-by-level approach.  
# Time:  $O(n)$ . Space:  $O(h)$  recursion depth.  
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (BFS with level grouping)
```

```
# "BFS using a deque. At each step,  
process ALL nodes at the current level  
(snapshot the queue size),  
# collect their values, enqueue their  
children. Natural level separation – no  
level counter needed."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _102_LevelOrder:  
    def levelOrder(self, root:  
Optional['TreeNode']) -> List[List[int]]:  
        if not root:  
            return []  
        result = []  
        queue = deque([root])  
        while queue:  
            level_size = len(queue)  
            level_vals = []  
            for _ in range(level_size):  
                node = queue.popleft()
```



```
        level_vals.append(node.val)
    l)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
        result.append(level_vals)
    return result
```

PHASE 5 — TEST & DEBUG

```
# root=3: queue=[3]. level_size=1.
level=[3]. enqueue 9,20. result=[[3]].
# queue=[9,20]. level_size=2.
level=[9,20]. enqueue 15,7.
result=[[3],[9,20]].
# queue=[15,7]. level_size=2.
level=[15,7]. result=[[3],[9,20],[15,7]]
✓
# root=None: return [] immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(w)$  where  $w = \max$ 
width of the tree (at most  $n/2$  for a
complete tree).
# The 'snapshot the queue size' trick is
the canonical BFS level-separation
```

pattern.

Variations: level order bottom-up (#107)

→ just reverse the result.

Zigzag level order (#103) → alternate the direction each level.

Right side view (□#199) → take the last element of each level.

Given more time I'd ask: what if we want the minimum depth to a leaf?

BFS stops at the first leaf found – don't traverse the whole tree."

Trees & BST

#103 Binary Tree Zigzag Level Order Traversal

MED[Open on LeetCode](#)

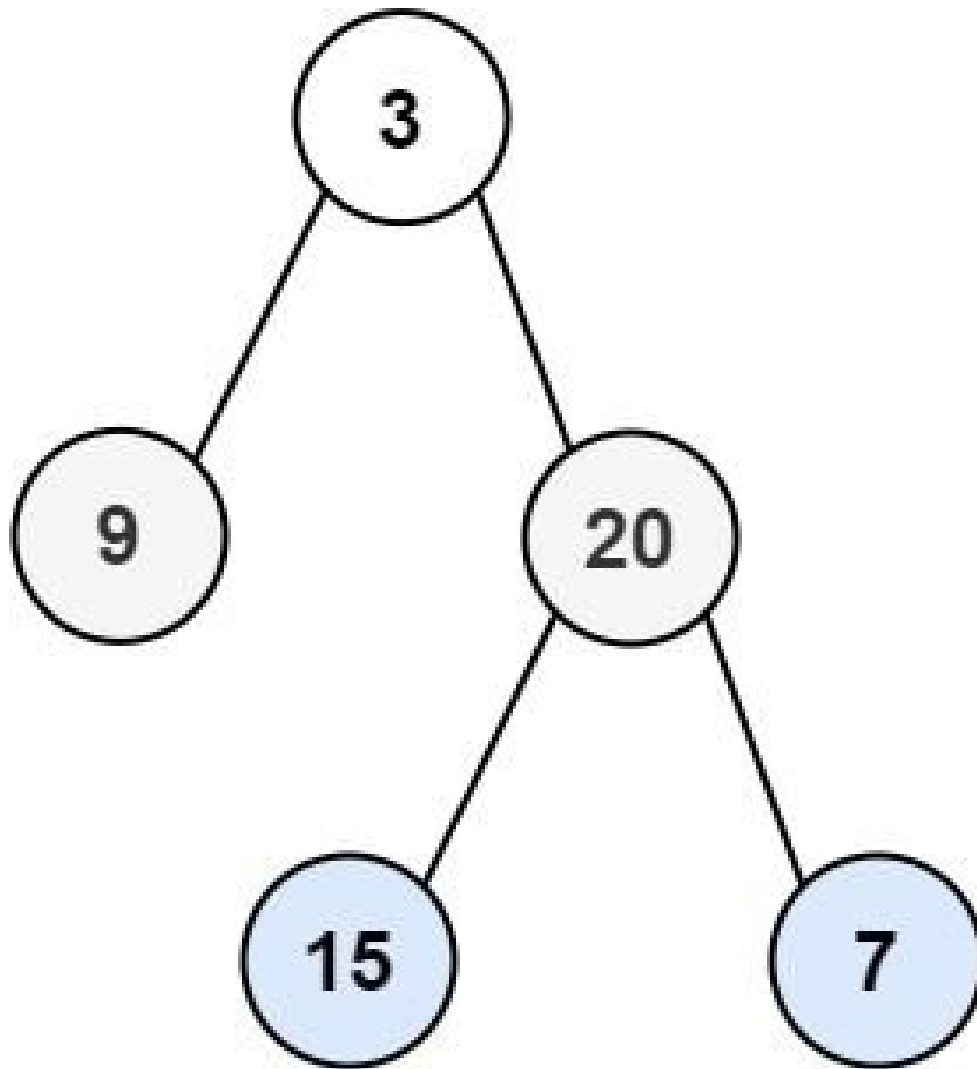
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Level order traversal but alternating direction: odd levels left-to-right, even levels right-to-left (or vice versa – confirm with example).

Root is level 0 → left-to-right. Level 1 → right-to-left. Right?"

#

"[3,9,20,null,null,15,7] → [[3],[20,9],[15,7]] ✓ (level 1 reversed)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Run standard BFS level-order to build a list of levels.

```
# Reverse every other level list (odd
index levels) for zigzag output.
# Correct O(n); we can reverse on the fly
during BFS instead of a second pass.
# Time: O(n). Space: O(n) queue + levels.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same BFS snapshot approach as #102. Use
a boolean flag 'left_to_right'.
# Collect each level normally, then
reverse if flag is False.
# Toggle flag each level. Time: O(n).
Space: O(w)."
```

PHASE 4 — CLEAN CODE

```
class _103_ZigzagLevelOrder:
    def zigzagLevelOrder(self, root:
Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        result = []
        queue = deque([root])
        left_to_right = True
        while queue:
            level_size = len(queue)
```

```

        level_vals = []
        for _ in range(level_size):
            node = queue.popleft()
            level_vals.append(node.val)

    l)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
        result.append(level_vals if
left_to_right else level_vals[::-1])
        left_to_right = not
left_to_right
    return result

```

PHASE 5 — TEST & DEBUG

```

# root=[3,9,20,null,null,15,7]:
# Level 0: [3], left_to_right=True → [3].
Toggle→False.
# Level 1: [9,20], left_to_right=False →
reversed=[20,9]. Toggle→True.
# Level 2: [15,7], left_to_right=True →
[15,7]. → [[3],[20,9],[15,7]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(w).

```

```
# This is #102 with one added line: the  
conditional reverse.  
# Name that connection – it shows you see  
the pattern family.  
# Given more time I'd ask: what if  
alternation starts at level 1 instead of  
0?  
# Just flip the initial value of  
left_to_right."
```


Trees & BST

□ #105 Construct Binary Tree from Preorder and Inorder Traversal

MED

[Open on LeetCode](#)

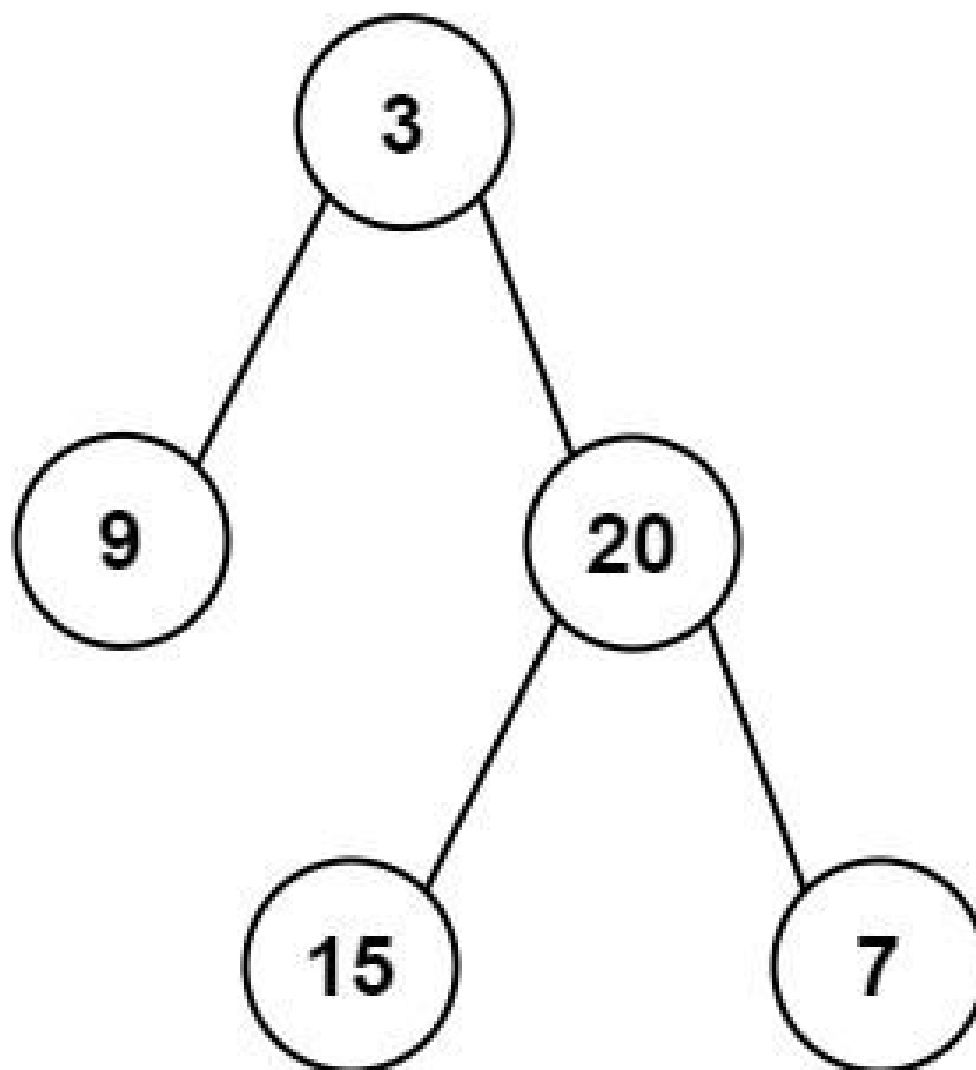
Array · Hash Table · Tree · DFS

Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7],
inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$

- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` and `inorder` consist of unique values.
- Each value of `inorder` also appears in `preorder`.
- `preorder` is guaranteed to be the preorder traversal of the tree.
- `inorder` is guaranteed to be the inorder traversal of the tree.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Preorder: root first, then left subtree, then right subtree.

Inorder: left subtree, root, right subtree.

All values are unique – so we can find the root's position in `inorder` uniquely. Right?"

#

"preorder=[3,9,20,15,7],
inorder=[9,3,15,20,7]:

```
# Root=3. In inorder, 3 is at index 1.  
Left subtree=[9], right  
subtree=[15,20,7].  
# preorder left=[9], right=[20,15,7].  
Recurse."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Root is preorder[0]; scan inorder to  
find root index and split left/right  
subtrees.
```

```
# Recursively repeat on each side –  
correct tree reconstruction.
```

```
# Scanning inorder for every root costs  
 $O(n^2)$  total; hash map of inorder indices  
fixes that.
```

```
# Time:  $O(n^2)$  naive. Space:  $O(n)$   
recursion.
```

```
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Pre-build a hash map: inorder value →  
index for  $O(1)$  lookups."
```

```
# Use a preorder index pointer (use a list  
to allow mutation inside inner function).
```

```
# build(in_left, in_right): root =  
preorder[pre_idx]. Find in inorder.  
Recurse.  
# Time: O(n). Space: O(n) for the map."
```

PHASE 4 — CLEAN CODE

```
class _105_BuildTree:  
    def buildTree(self, preorder:  
List[int], inorder: List[int]) ->  
Optional['TreeNode']:  
        in_idx = {val: i for i, val in  
enumerate(inorder)}  
        pre = iter(preorder)  
        def build(left: int, right: int)  
-> Optional['TreeNode']:  
            if left > right:  
                return None  
            root_val = next(pre)  
            root = TreeNode(root_val)  
            mid = in_idx[root_val]  
            root.left = build(left, mid -  
1)  
            root.right = build(mid + 1,  
right)  
            return root  
        return build(0, len(inorder) - 1)
```

PHASE 5 — TEST & DEBUG

```
# preorder=[3,9,20,15,7],
inorder=[9,3,15,20,7].
in_idx={9:0,3:1,15:2,20:3,7:4}.
# build(0,4): root=3, mid=1.
left=build(0,0), right=build(2,4).
# build(0,0): root=9, mid=0.
left=build(0,-1)=None.
right=build(1,0)=None. → node(9).
# build(2,4): root=20, mid=3.
left=build(2,2), right=build(4,4).
# build(2,2): root=15. → node(15).
build(4,4): root=7. → node(7).
# → Tree: 3→left=9, 3→right=20→left=15,
20→right=7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$ for the hash map and call stack.

Using `iter(preorder)` as a lazy pointer is cleaner than an index variable.

The key insight: preorder always gives the root first, inorder tells us how many elements are in each subtree – that's the only information we need to recurse.

Given more time I'd ask: construct from postorder and inorder?
Same logic – just consume postorder from the RIGHT end, and build right subtree first."

Trees & BST

#199 Binary Tree Right Side View

MED[Open on LeetCode](#)

Tree · DFS · BFS

Lists: Grind 169

Problem

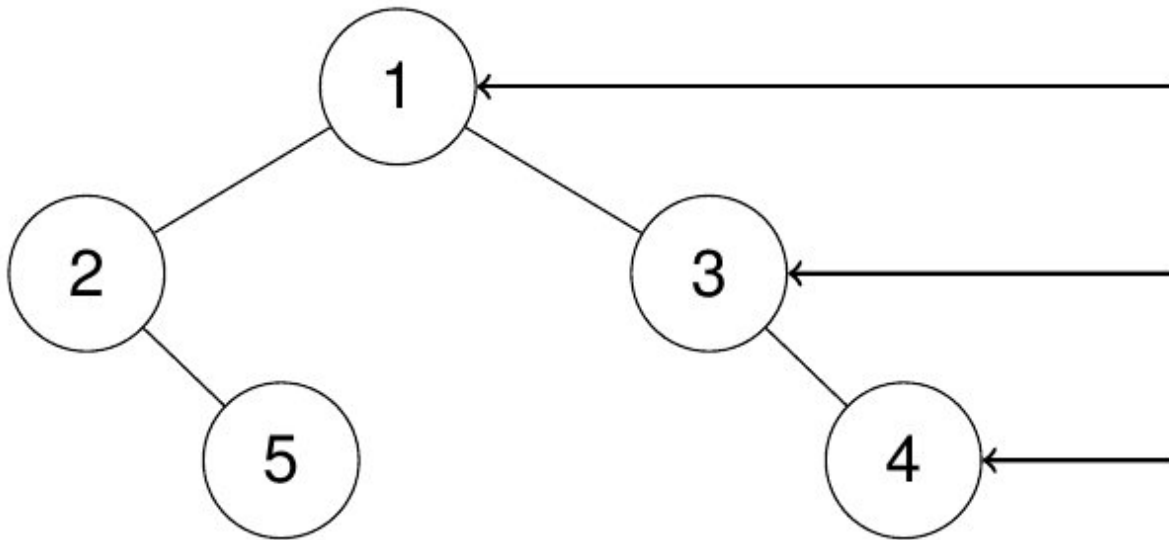
Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

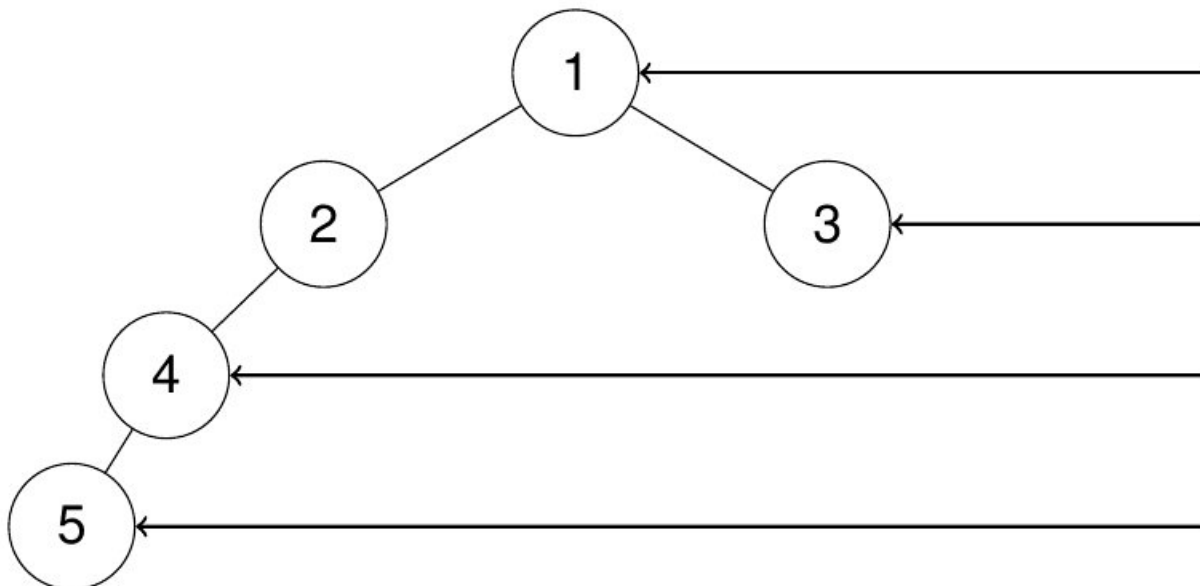


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Imagine looking at the tree from the right. You see one node per level – the rightmost node visible at that level. Return top to bottom. Right?"

#

"[1,2,3,null,5,null,4] → [1,3,4] ✓ (3 is rightmost at level 1, 4 at level 2)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

```
# BFS level-order; after processing each
level, take the last node value added.
# That last node is the rightmost visible
from that depth.
# Correct O(n); DFS tracking depth +
rightmost-at-depth is equally fine.
# Time: O(n). Space: O(w) queue width.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same BFS snapshot as #102/#103. Just
append the LAST element of each level's
list.
```

```
# Or DFS: visit right child first, add
node.val when depth == len(result) (first
node seen at new depth)."
```

PHASE 4 — CLEAN CODE

```
class _199_RightSideView:
    def rightSideView(self, root:
Optional['TreeNode']) -> List[int]:
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
```

```

        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            if i == level_size - 1:
                result.append(node.val)
            # rightmost node of this level
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        return result

```

PHASE 5 — TEST & DEBUG

```

# [1,2,3,null,5,null,4]:
# Level 0: [1] → last=1. Level 1: [2,3] → last=3. Level 2: [5,4] → last=4.
# → [1,3,4] ✓
# [1,2,3,4,null,null,null,5]:
# Level 0: [1]→1. Level 1: [2,3]→3. Level 2: [4]→4. Level 3: [5]→5. → [1,3,4,5] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(w) where w = max width.
# This is #102 with one change: only record the last element per level.

```

Left side view: record the first element per level instead.

Given more time I'd ask: what if some right-side nodes are blocked by deeper left-side nodes?

The problem says 'visible from the right' – the rightmost node at each DEPTH level,

which BFS naturally gives us. A right child at depth 2 can block a left child at depth 2."

Trees & BST

□ #230 Kth Smallest Element in a BST

MED[Open on LeetCode](#)

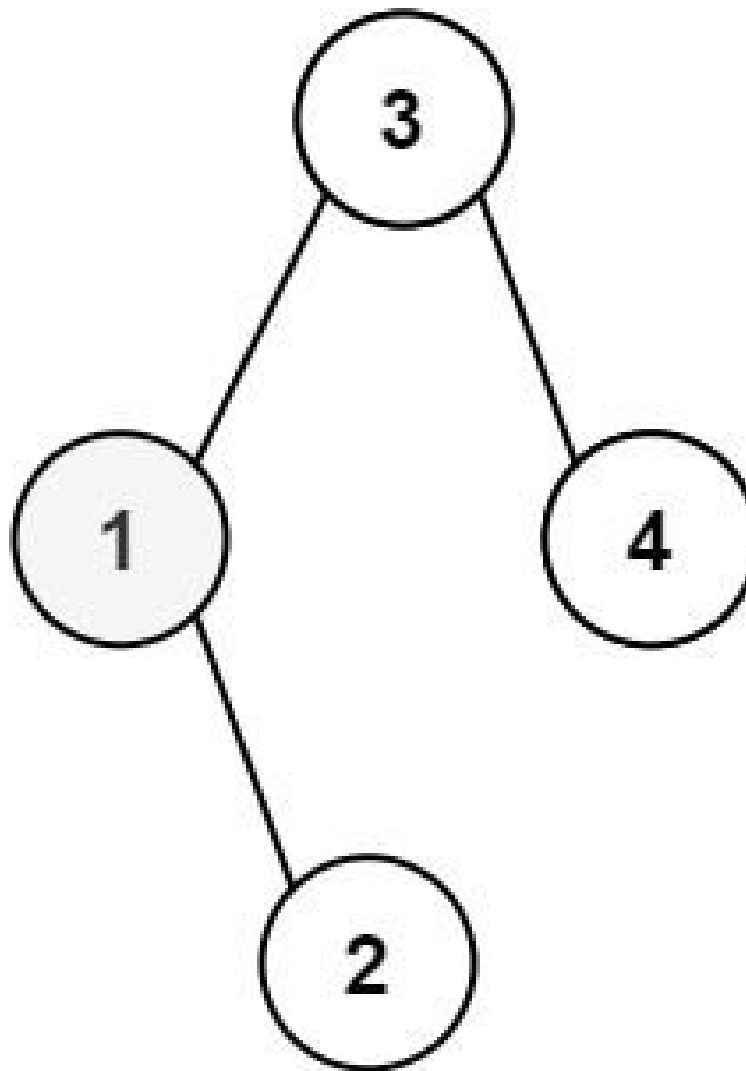
Tree · DFS · BST

Lists: Grind 169

Problem

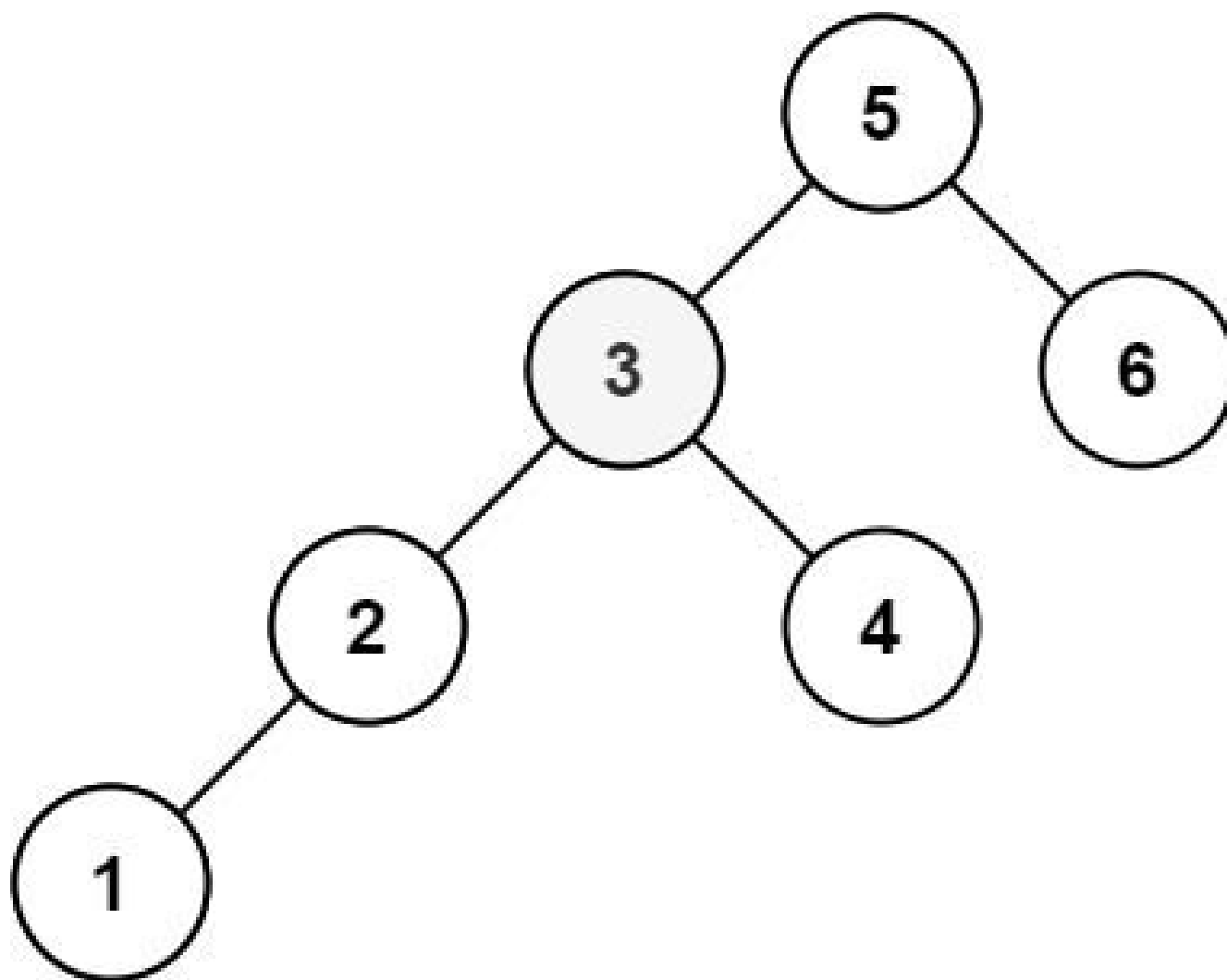
Given the root of a binary search tree, and an integer k , return the k th smallest value (1-indexed) of all the values of the nodes in the tree.

Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1],
k = 3

Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you

need to find the kth smallest frequently, how would you optimize?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"BST in-order traversal (left → root → right) visits nodes in ascending sorted order.

Return the k-th element in this order (1-indexed). Right?"

#

"[3,1,4,null,2], k=1:
in-order=[1,2,3,4]. 1st smallest=1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

In-order traversal collects all node values in sorted order.

Return the (k-1)th element from that list.

Correct $O(n)$ time and $O(n)$ space; we can stop after k steps in-order.

Time: $O(n)$. Space: $O(n)$ values list.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"In-order DFS stopping early at the k-th element. Use a counter and result variable in a nonlocal/list wrapper. Avoids collecting all n values if k is small."

PHASE 4 — CLEAN CODE

```
class _230_KthSmallestBST:
    def kthSmallest(self, root:
Optional['TreeNode'], k: int) -> int:
        count = [0]
        result = [0]
        def inorder(node):
            if not node or count[0] == k:
                return
            inorder(node.left)
            count[0] += 1
            if count[0] == k:
                result[0] = node.val
                return
            inorder(node.right)
        inorder(root)
        return result[0]
```

PHASE 5 — TEST & DEBUG

[3,1,4,null,2], k=1:

```
# inorder(3)→inorder(1)→inorder(None).
count=0+1=1==k=1→result=1. return.
```

```
# → 1 ✓
```

```
# [5,3,6,2,4,null,null,1], k=3:
```

```
# in-order: 1,2,3,4,5,6. 3rd = 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(k + h)$  – at most  $k$  steps after
reaching the leftmost leaf. Space  $O(h)$ .
```

```
# Follow-up: frequent inserts/deletes +
frequent kth queries?
```

```
# Augment each BST node with the size of
its left subtree.
```

```
# Then kth smallest is an  $O(\log n)$ 
```

```
descent: if left_size >= k go left; if
left_size+1==k return root; else k -=
left_size+1, go right.
```

```
# Given more time I'd ask: what if we want
the kth largest?
```

```
# Reverse in-order (right→root→left) and
count k steps."
```

Trees & BST

□ #235 Lowest Common Ancestor of a Binary Search Tree

MED

[Open on LeetCode](#)

Tree · DFS · BST

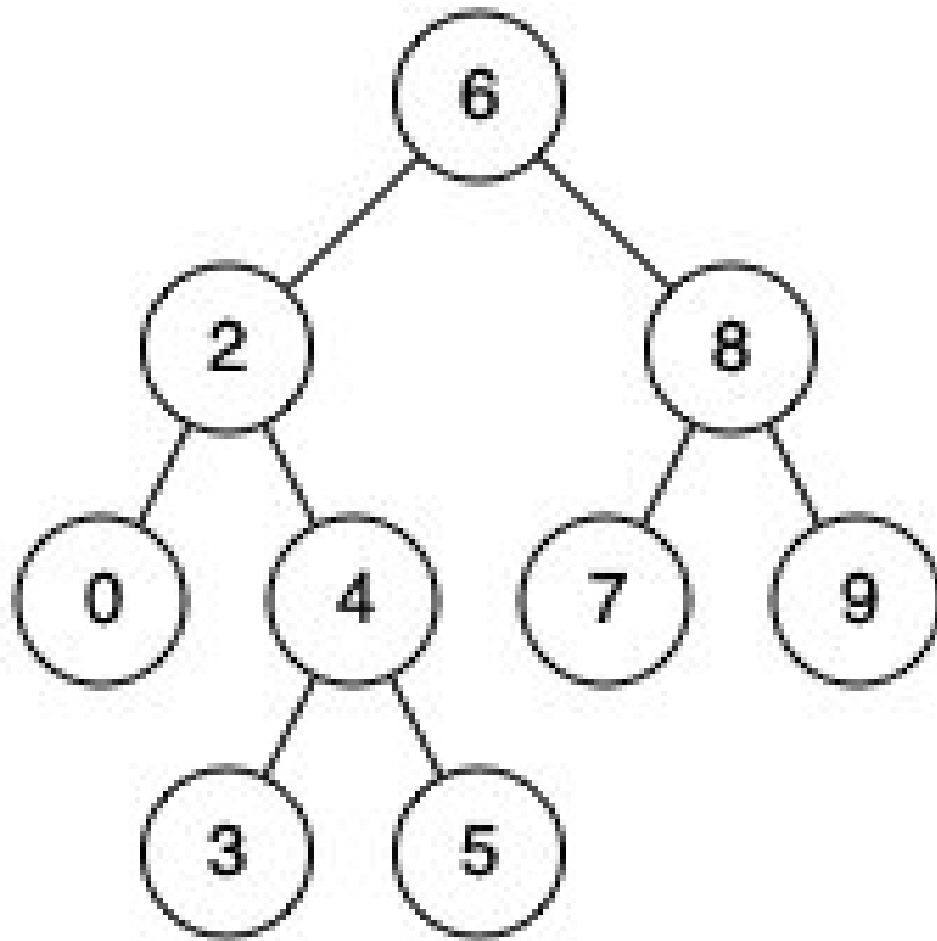
Lists: Grind 169

Problem

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

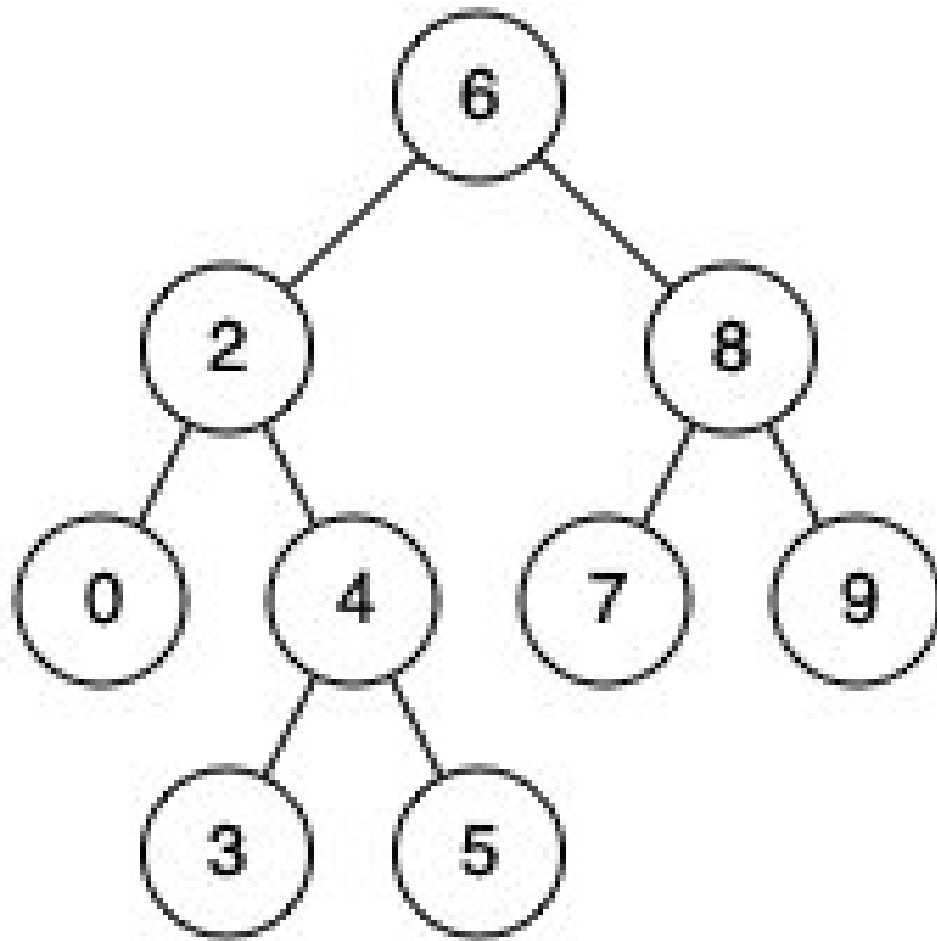


Input: root =
[6,2,8,0,4,7,9,null,null,3,5], p = 2, q
= 8

Output: 6

Explanation: The LCA of nodes 2 and 8
is 6.

Example 2:



Input: root =
[6,2,8,0,4,7,9,null,null,3,5], p = 2, q
= 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1
Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$
- All Node.val are unique.
- $p \neq q$
- p and q will exist in the BST.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"In a BST, find the LCA of nodes p and q.

LCA = the deepest node that is an ancestor of both p and q (a node is its own ancestor). Right?

Use the BST property: values to the left < root < values to the right."

#

"[6,2,8,0,4,7,9], p=2, q=8 → LCA=6 (they're on different sides of 6) ✓

$p=2, q=4 \rightarrow \text{LCA}=2$ (4 is in 2's right subtree) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Walk from root: if both p and q are smaller, go left; if both larger, go right.

First node where paths split is the LCA in a BST.

Alternatively store root-to- p and root-to- q paths – more work than the walk.

Time: $O(h)$. Space: $O(h)$ if storing paths.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BST property)

"Navigate using BST property:

If both p and q are LESS than root: LCA is in the LEFT subtree.

If both are GREATER: LCA is in the RIGHT subtree.

Otherwise (they split at root, or one equals root): root IS the LCA.

$O(h)$ time – $O(\log n)$ for balanced BST, $O(n)$ worst case."

PHASE 4 — CLEAN CODE

```
class _235_LCA_BST:
    def lowestCommonAncestor(self, root:
'TreeNode', p: 'TreeNode', q: 'TreeNode')
-> 'TreeNode':
        while root:
            if p.val < root.val and q.val
< root.val:
                root = root.left
            elif p.val > root.val and
q.val > root.val:
                root = root.right
            else:
                return root
        return root
```

PHASE 5 — TEST & DEBUG

root=6, p=2, q=8: both in different sides → return 6 ✓

root=6, p=2, q=4: both<6→go left to 2. p=2==root, q=4>root → split here → return 2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(h)$. Space $O(1)$ – iterative, no stack.

The BST property makes this an $O(h)$ walk – no need to visit all nodes.

This is much faster than the general binary tree LCA (□#236) which needs DFS.

Given more time I'd ask: what if p or q might not exist in the tree?

We'd need to verify existence first – two separate searches, then LCA."

Trees & BST

#236 Lowest Common Ancestor of a Binary Tree

MED[Open on LeetCode](#)

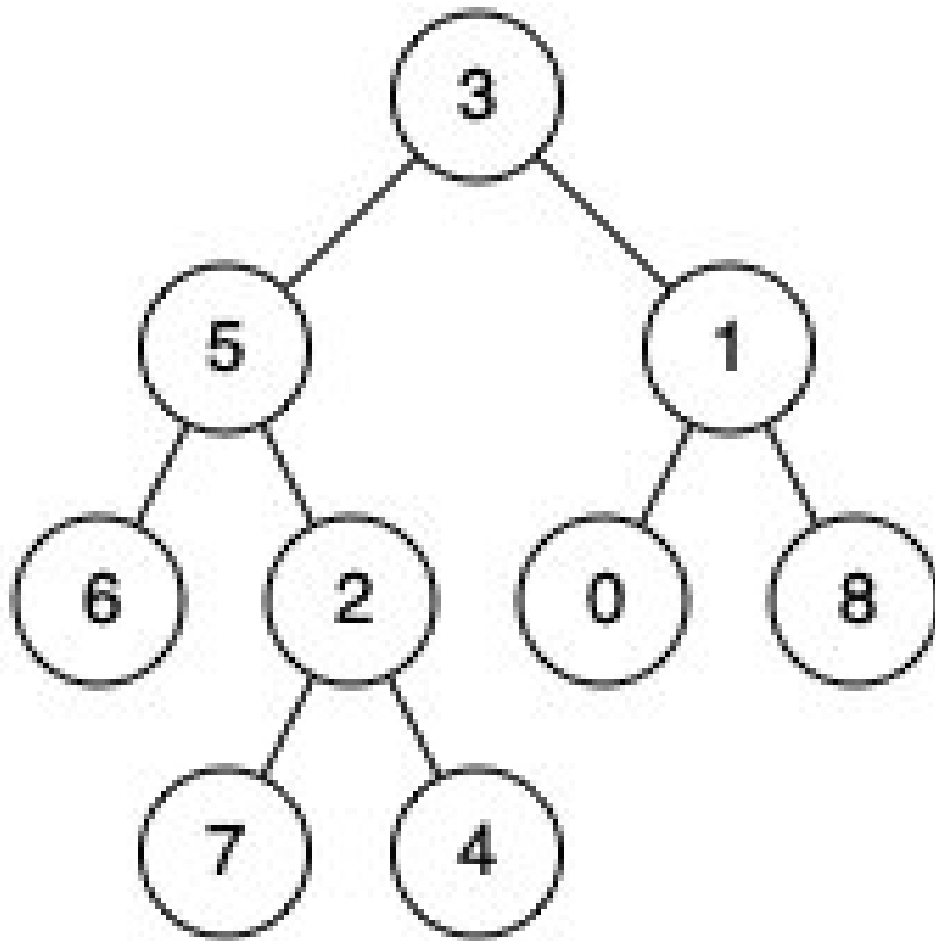
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

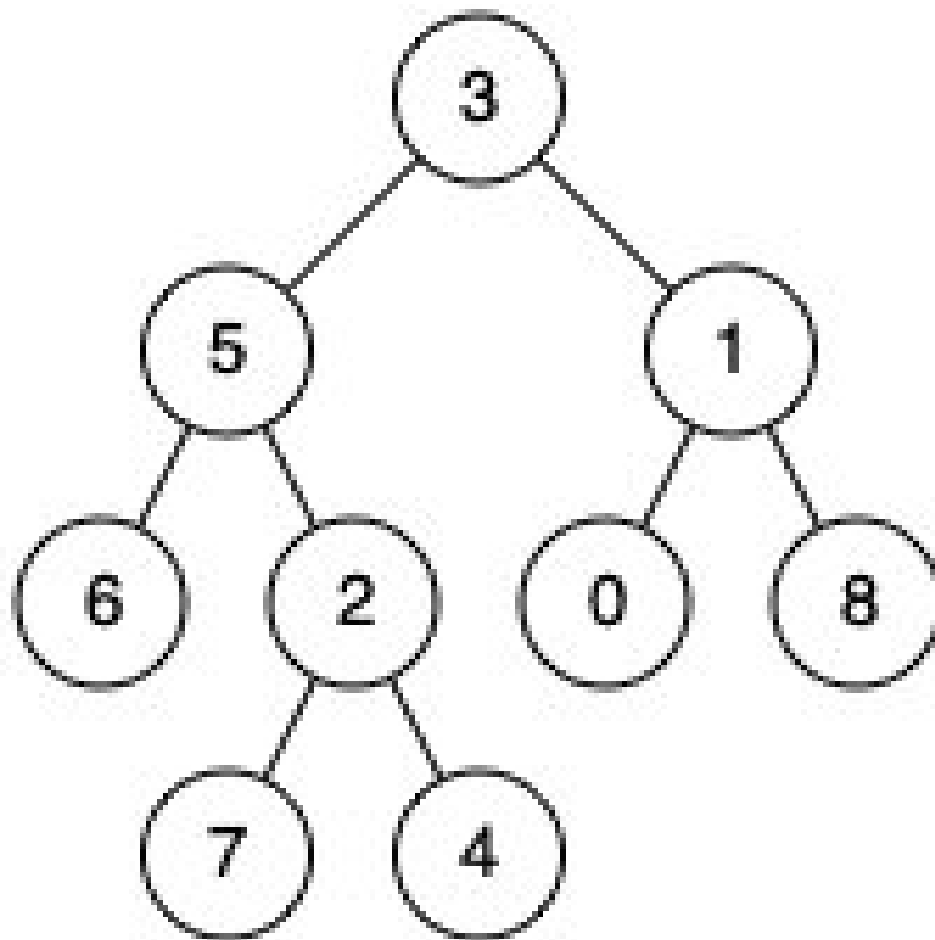


Input: root =
[3,5,1,6,2,0,8,null,null,7,4], p = 5, q
= 1

Output: 3

Explanation: The LCA of nodes 5 and 1
is 3.

Example 2:



Input: root =
[3,5,1,6,2,0,8,null,null,7,4], p = 5, q
= 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$
- All Node.val are unique.
- $p \neq q$
- p and q will exist in the tree.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"General binary tree – no BST property.
Find the LCA of p and q.

p and q are guaranteed to exist in the tree. A node is its own ancestor. Right?"

#

"[3,5,1,6,2,0,8,null,null,7,4], p=5,
q=1 → LCA=3 (root, both are in different subtrees) ✓

p=5, q=4 → LCA=5 (4 is in 5's subtree)
✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Find root-to-p path and root-to-q path as node lists.

Walk both lists in parallel; last matching node before divergence is LCA.

Correct $O(n)$ but needs $O(h)$ path storage; post-order single-pass is cleaner.

Time: $O(n)$. Space: $O(h)$ two paths.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (recursive post-order)

"DFS: if current node is p or q, return it (it or its ancestor is the LCA).

Recurse left and right.

If both return non-null: current node is the LCA (p and q are in different subtrees).

If only one is non-null: that subtree contains both p and q – propagate that result up."

PHASE 4 — CLEAN CODE

class _236_LCA_BinaryTree:

```
def lowestCommonAncestor(self, root:
'TreeNode', p: 'TreeNode', q: 'TreeNode')
-> 'TreeNode':
    def lca(node):
        if not node or node is p or
node is q:
            return node
        left = lca(node.left)
        right = lca(node.right)
        if left and right:
            return node # p and q are
in different subtrees
        return left or right # both in
same subtree – propagate the found node
    return lca(root)
```

PHASE 5 — TEST & DEBUG

[3,5,1,...], p=5, q=1:

lca(3): lca(5)→returns 5 (found p).

lca(1)→returns 1 (found q).

Both non-null → return 3 ✓

p=5, q=4 (4 is under 5):

lca(3): lca(5)→... lca(4) under 5

returns 4. lca(5) itself: node is p→return 5.

lca(1)→None (neither p nor q in right subtree).

left=5, right=None → return 5 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – visits every node. Space $O(h)$ for the call stack.

Contrast with #235 (BST): BST LCA is $O(h)$ iteratively. General tree needs full DFS.

The post-order logic is the key: left and right results tell us which subtrees contain p/q.

Both non-null → split here → current node is LCA.

Only one non-null → both are in the same subtree → propagate that subtree's result.

Given more time I'd ask: what if p or q might not exist?

We'd need to track 'found p' and 'found q' separately – more complex state."

Trees & BST

□ ★ #250 Count Univalue Subtrees ★

MED

[Open on LeetCode](#)

Tree · DFS

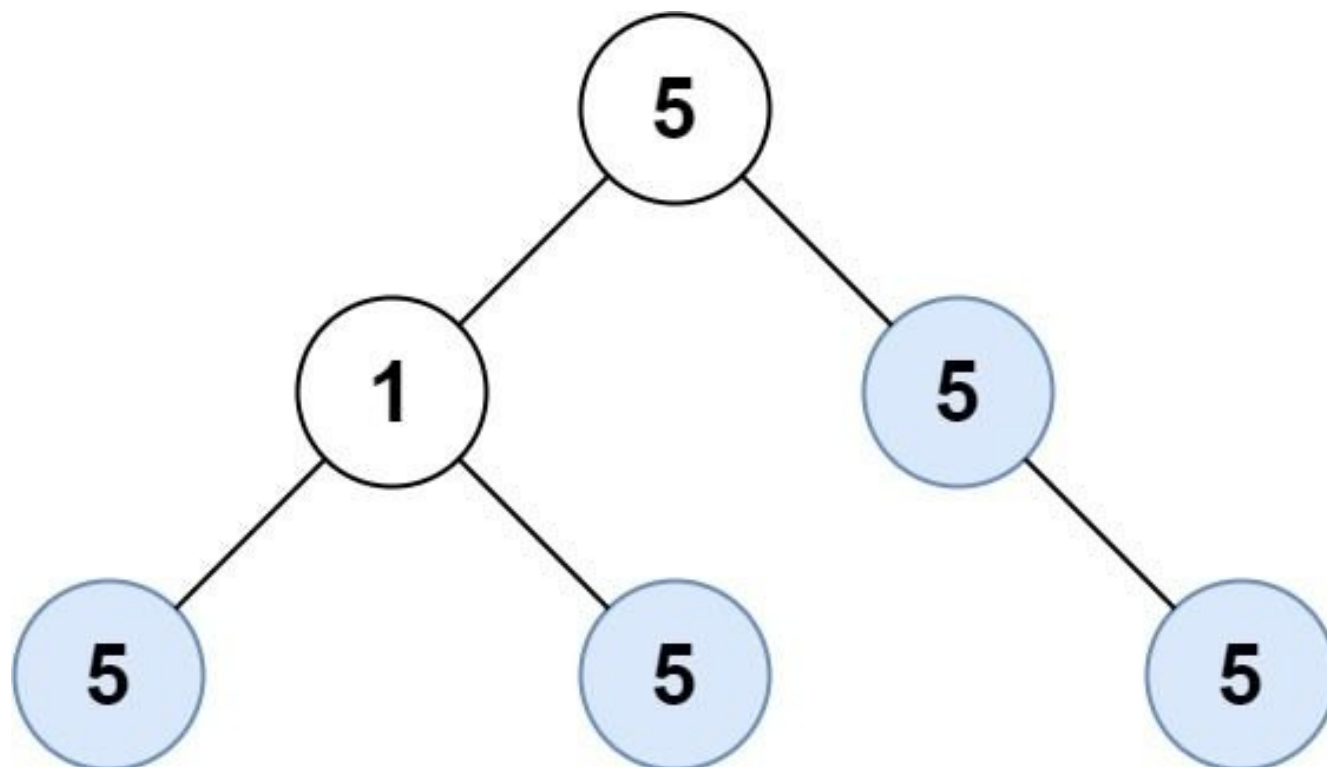
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1:



Round 2 · High · Medium

p.798

Input: root = [5,1,5,5,5,null,5]
Output: 4

Example 2:

Input: root = []
Output: 0

Example 3:

Input: root = [5,5,5,5,5,null,5]
Output: 6

Constraints:

- The number of the node in the tree will be in the range [0, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A univalue subtree has every node with the same value.

Count all such subtrees – including single leaf nodes. Right?

Empty tree → 0."

#

"[5,1,5,5,5,null,5] → 4 (the four nodes with value 5 that form valid univalue subtrees) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each node, DFS its entire subtree checking every value equals node.val.

Increment count when the whole subtree is uni-value.

Revalidates overlapping subtrees – $O(n^2)$ worst case.

Time: $O(n^2)$. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (post-order DFS)

"Bottom-up: a node is a univalue root if both children are univalue AND their values match.

Return True from a node if its subtree is univalue, increment count when True.

Use an inner helper that returns bool + mutates a count list."

PHASE 4 — CLEAN CODE

class _250_CountUnivalueSubtrees:

```
def countUnivalSubtrees(self, root:
Optional['TreeNode']) -> int:
    count = [0]
    def is_univalue(node,
parent_val):
        if not node:
            return True
        left = is_univalue(node.left,
node.val)
        right =
is_univalue(node.right, node.val)
        if not left or not right:
            return False
        count[0] += 1
        return node.val == parent_val
    is_univalue(root, None)
    return count[0]
```

PHASE 5 — TEST & DEBUG

```
# [5,1,5,5,5,null,5]:
# Leaves: 5(left-left)→True,
5(left-right)→True, 5(right-right)→True,
1→True (leaf).
# Node(1): left=True, right=True, both
return True. 1's children have val=5≠1 →
returns False.
```

```
# Node(5, right of root): left=None=True,  
right(5)=True, right returns True(val  
matches). count=1. Returns  
True(5==root.val?).
```

```
# Continue... count ends at 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$ . Post-order  
because we need children's results before  
parent's.
```

```
# The parent_val parameter lets us check  
if current node extends the parent's  
univalue property.
```

```
# Given more time I'd ask: longest path  
where all values are the same?
```

```
# Track univalue path length similar to  
Diameter of Binary Tree – carry length  
down."
```

Trees & BST

□ #285 Inorder Successor in BST

MED[Open on LeetCode](#)

Tree · DFS · BST

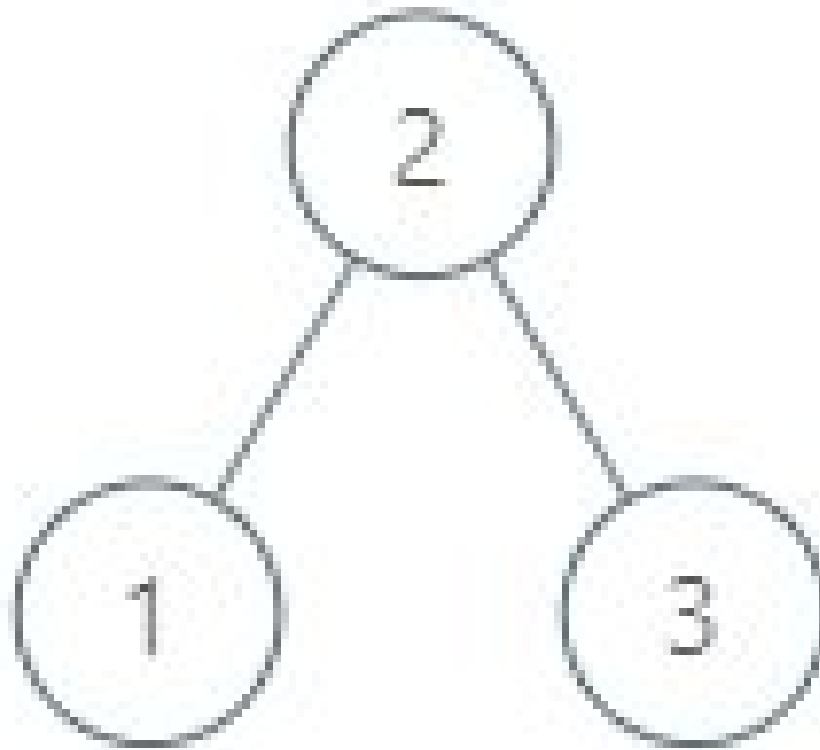
Lists: Grind 169

Problem

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

The successor of a node p is the node with the smallest key greater than $p.val$.

Example 1:

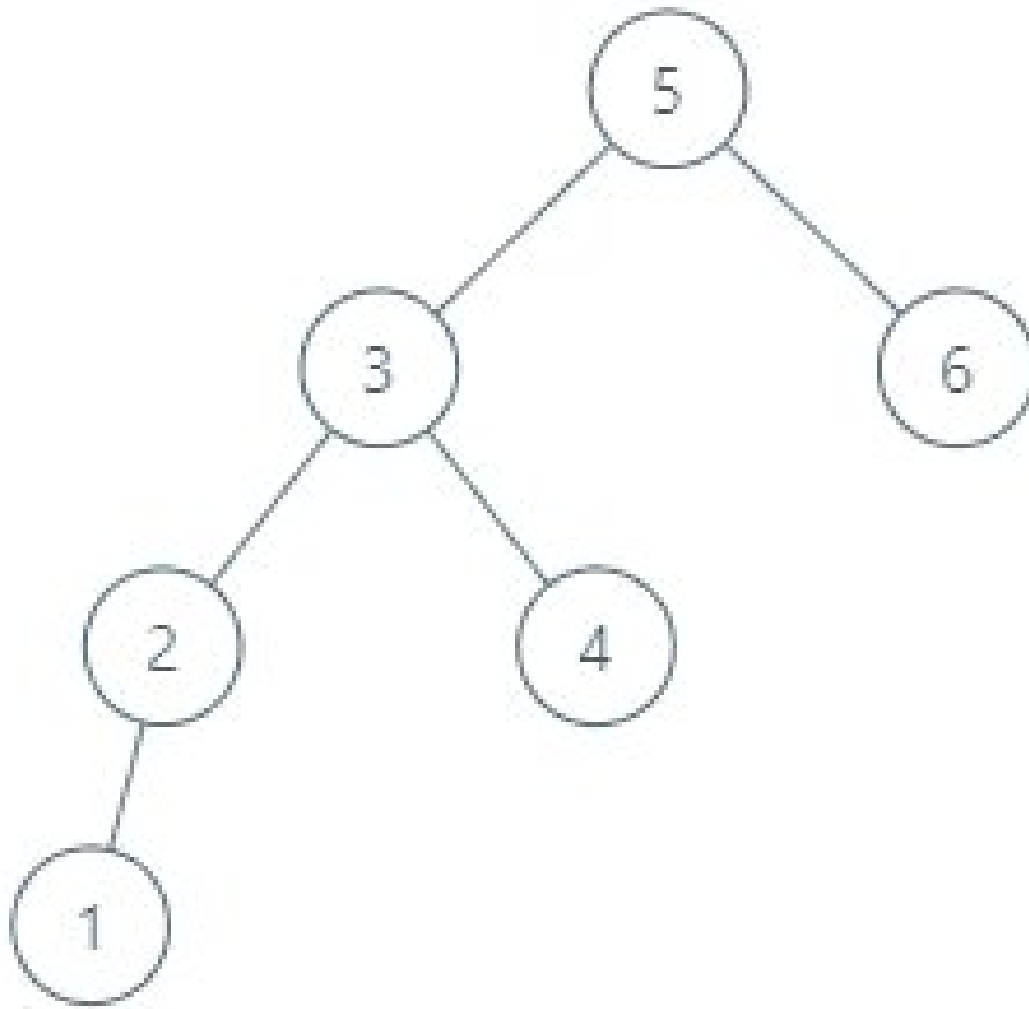


Input: root = [2,1,3], p = 1

Output: 2

Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.

Example 2:



Input: root = [5,3,6,2,4,null,null,1],
p = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].

- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the node with the smallest value strictly greater than `p.val` in a BST.

Return null if no such node exists (`p` is the largest). Right?"

#

"[2,1,3], `p=1` → `successor=2` (next in in-order: 1,2,3) ✓

[5,3,6,2,4,null,null,1], `p=6` → null (6 is the largest) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

In-order traversal of the BST; when we visit `p`, the next node in-order is successor.

Store previous node during traversal or collect all values then scan.

Correct $O(n)$; BST structure gives $O(h)$ successor without full traversal.

```
# Time: O(n). Space: O(1) iterative with  
threading concept.  
# Should I go ahead and optimize?"  
  
# PHASE 3 — OPTIMIZE (BST navigation)  
# "Use BST property – no need to traverse  
all nodes.  
# If root.val > p.val: root is a candidate  
successor, but there might be a smaller  
one in the left subtree.  
# If root.val <= p.val: successor must be  
in the right subtree.  
# Track the best candidate seen so far.  
O(h) time."
```

PHASE 4 — CLEAN CODE

```
class _285_InorderSuccessor:  
    def inorderSuccessor(self, root:  
        'TreeNode', p: 'TreeNode') ->  
        Optional['TreeNode']:  
        successor = None  
        while root:  
            if root.val > p.val:  
                successor = root # root is  
a candidate – try for smaller in left  
                root = root.left
```

else:

 root = root.right #

successor must be to the right

 return successor

PHASE 5 — TEST & DEBUG

[2,1,3], p=1:

root=2: 2>1 → successor=2, go left.

root=1: 1≤1 → go right.

root=None → return 2 ✓

[5,3,6,2,4,null,null,1], p=6:

root=5: 5≤6 → go right. root=6: 6≤6 → go right. root=None → return None ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(h). Space O(1) – iterative.

The candidate tracking is the key: every time we go left, the current node becomes a potential successor (it's bigger than p, and we're looking for something smaller).

Given more time I'd ask: inorder PREDECESSOR (largest value < p.val)?

Mirror logic: if root.val < p.val → candidate, go right. If root.val ≥ p.val → go left."

Trees & BST

□ ★ #298 Binary Tree Longest Consecutive Sequence ★

MED

[Open on LeetCode](#)

Tree · DFS

Lists: ★ Premium | Premium 98

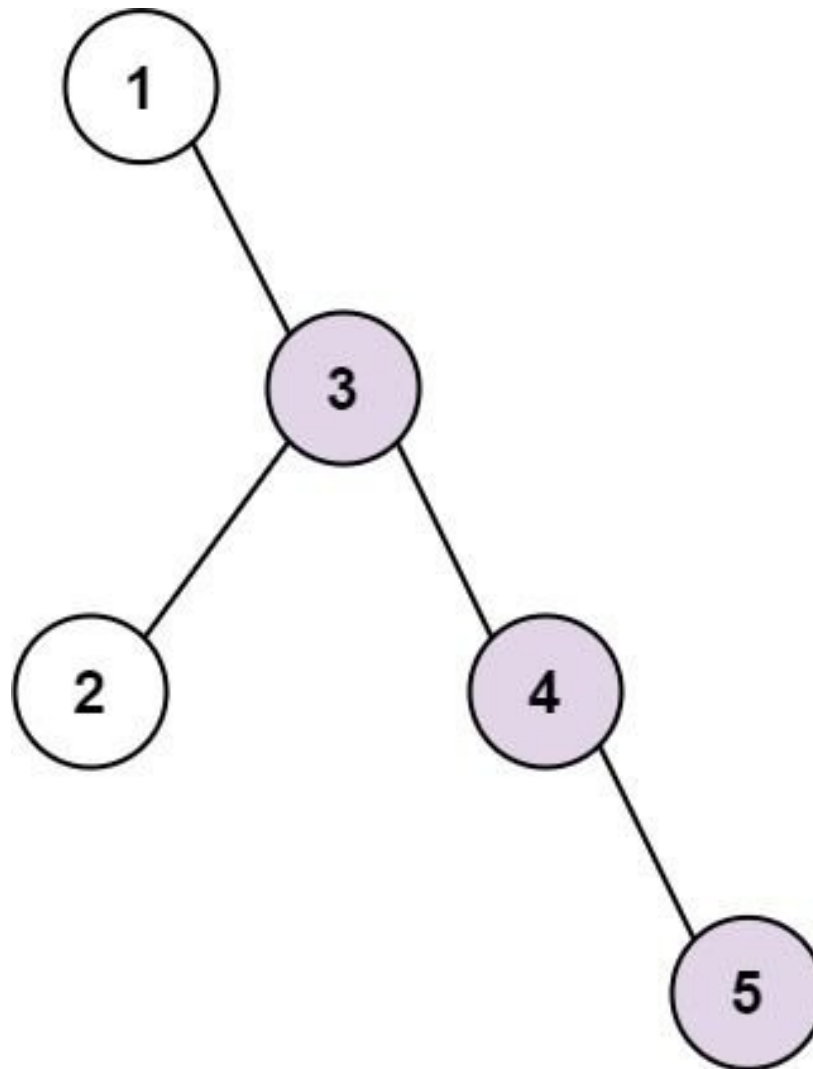
Problem

Given the root of a binary tree, return the length of the longest consecutive sequence path.

A consecutive sequence path is a path where the values increase by one along the path.

Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

Example 1:



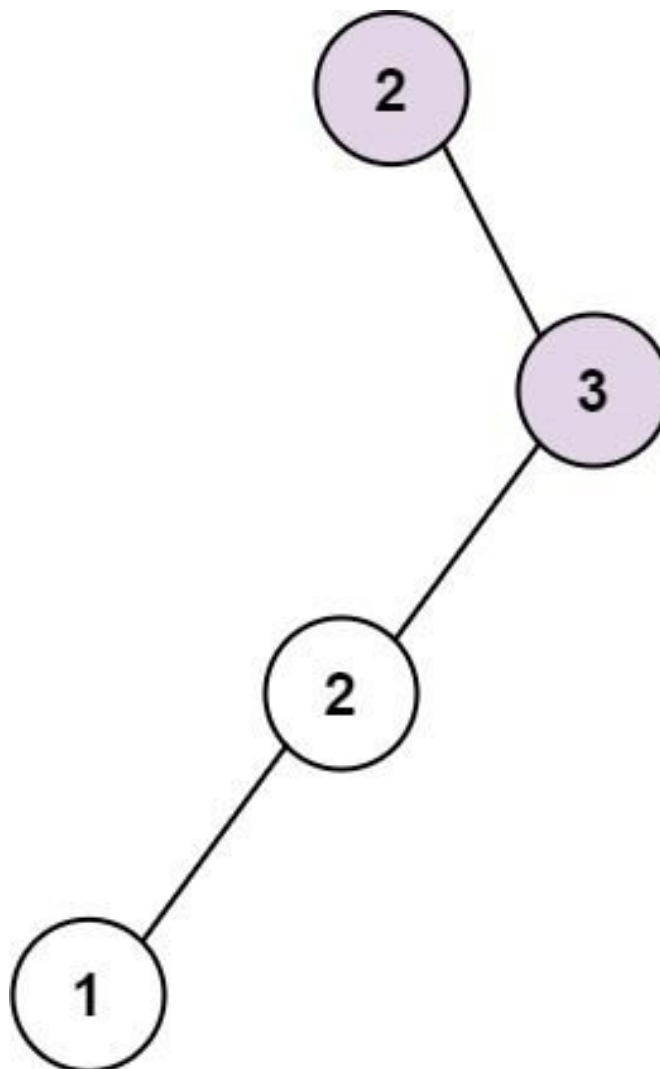
Input: root =

[1,null,3,2,4,null,null,null,5]

Output: 3

Explanation: Longest consecutive sequence path is 3-4-5, so return 3.

Example 2:



Input: root = [2,null,3,2,null,1]

Output: 2

Explanation: Longest consecutive sequence path is 2-3, not 3-2-1, so return 2.

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest path of consecutive increasing values (each node = parent + 1).

Path goes parent → child – you can't go back up. Path can start at any node.

Right?"

#

"[1,null,3,2,4,null,null,null,5]: path 3→4→5, length 3 ✓

[2,null,3,2,null,1]: 2→3 is increasing, not 3→2 – so max is 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

From each node, DFS tracking consecutive length when `child.val == parent.val + 1`.

Reset streak when the +1 chain breaks; track global maximum.

Starting DFS from every node repeats work – post-order aggregation is faster.

Time: $O(n^2)$ naive restarts. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DFS carrying current length)

"DFS top-down: pass current consecutive length to each child.

If `child.val == parent.val + 1`: extend the sequence.

Else: reset to 1 (start a new sequence from this child).

Track global max. Use inner helper with `parent_val` and `current_length`."

PHASE 4 — CLEAN CODE

```
class _298_LongestConsecutive:
    def longestConsecutive(self, root:
Optional['TreeNode']) -> int:
        best = [0]
        def dfs(node, parent_val,
length):
            if not node:
                return
            length = length + 1 if
node.val == parent_val + 1 else 1
```

```

        best[0] = max(best[0],
length)
        dfs(node.left, node.val,
length)
        dfs(node.right, node.val,
length)
    dfs(root, root.val - 1 if root
else 0, 0)
    return best[0]

```

PHASE 5 — TEST & DEBUG

```

# [1,null,3,2,4,null,null,null,5]:
# dfs(1, 0, 0): val=1, prev+1=1 →
length=1. best=1. dfs(3,1,1).
# dfs(3, 1, 1): val=3, prev+1=2, 3≠2 →
length=1. best=1. dfs(2,3,1).
# dfs(4, 3, 1): val=4==3+1 → length=2.
best=2. dfs(5,4,2).
# dfs(5, 4, 2): val=5==4+1 → length=3.
best=3. → 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(h).
# The initial call uses root.val-1 so the
root itself resets to length 1 cleanly.

```

This is a top-down DFS – pass state downward. Contrast with bottom-up (#250, #543).

Given more time I'd ask: what if the path can go through any node (not just parent→child)?

That's Longest Consecutive Sequence in a Binary Tree – post-order, track both ascending

and descending sequences at each node."

Trees & BST

□ ★ #314 Binary Tree Vertical Order Traversal ★

MED[Open on LeetCode](#)

Hash Table · Tree · BFS · Sorting

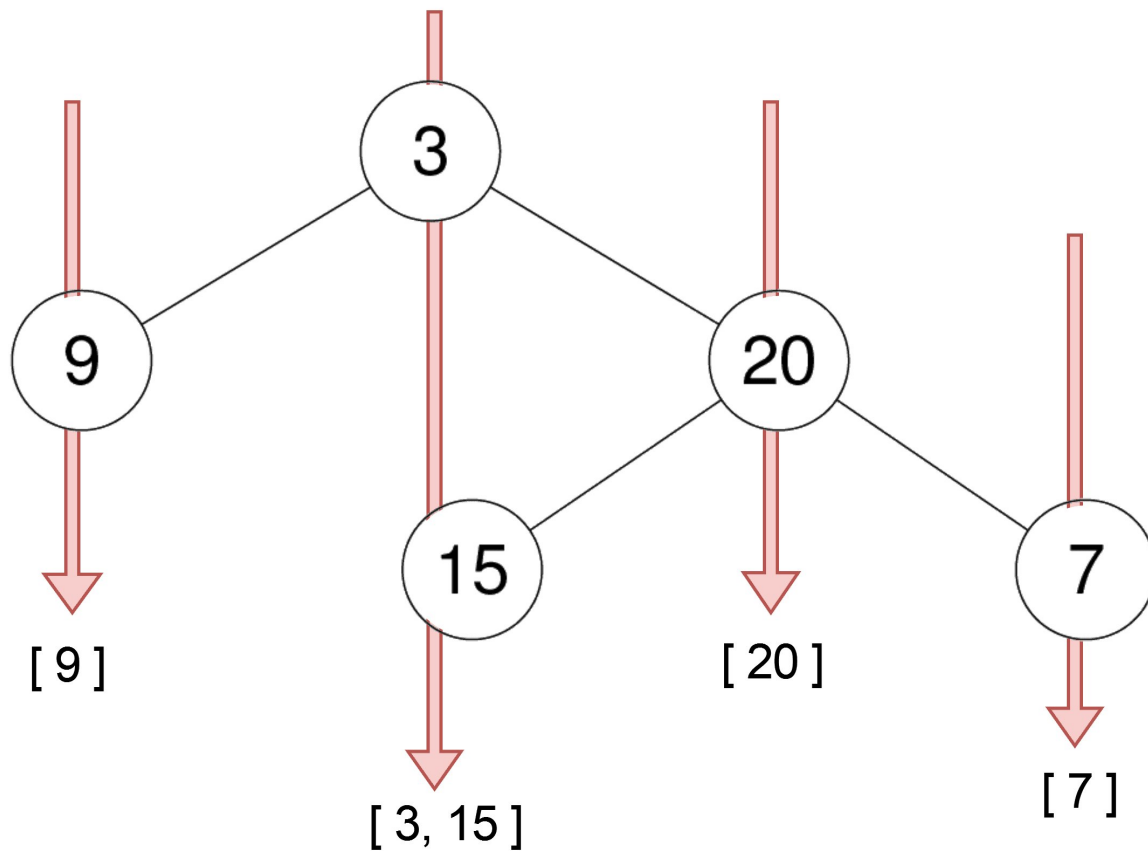
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

If two nodes are in the same row and column, the order should be from left to right.

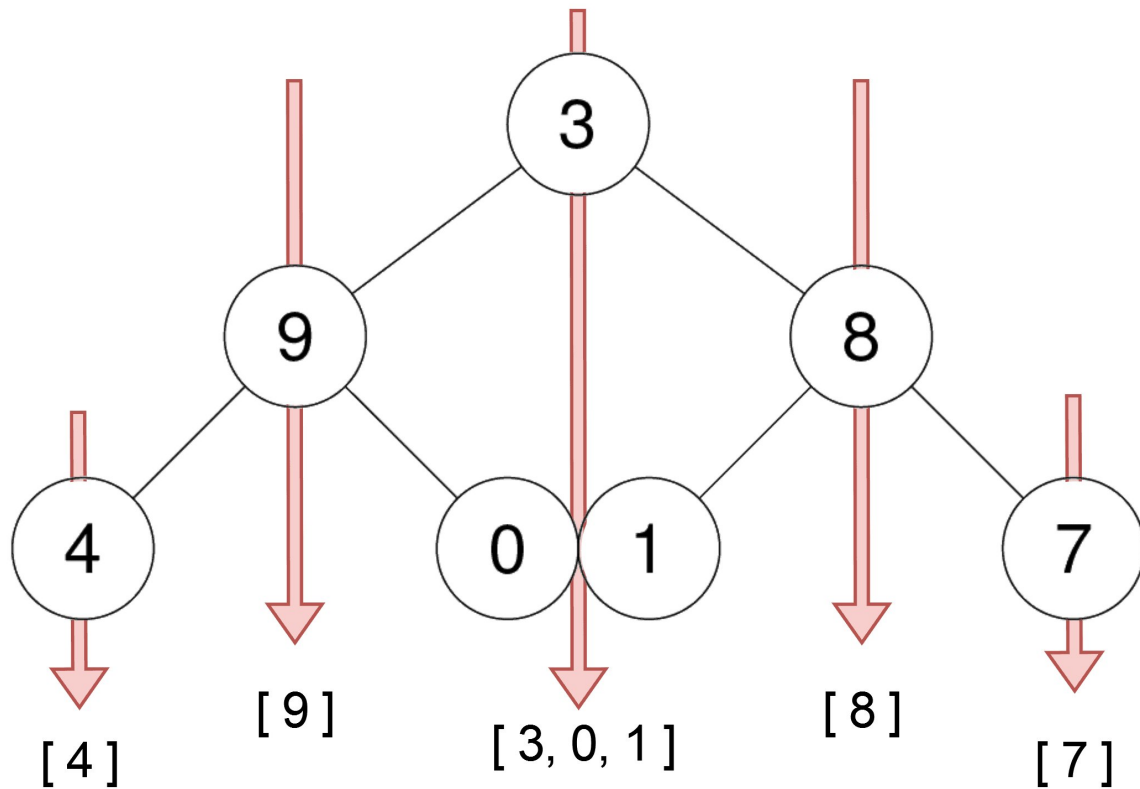
Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[9],[3,15],[20],[7]]

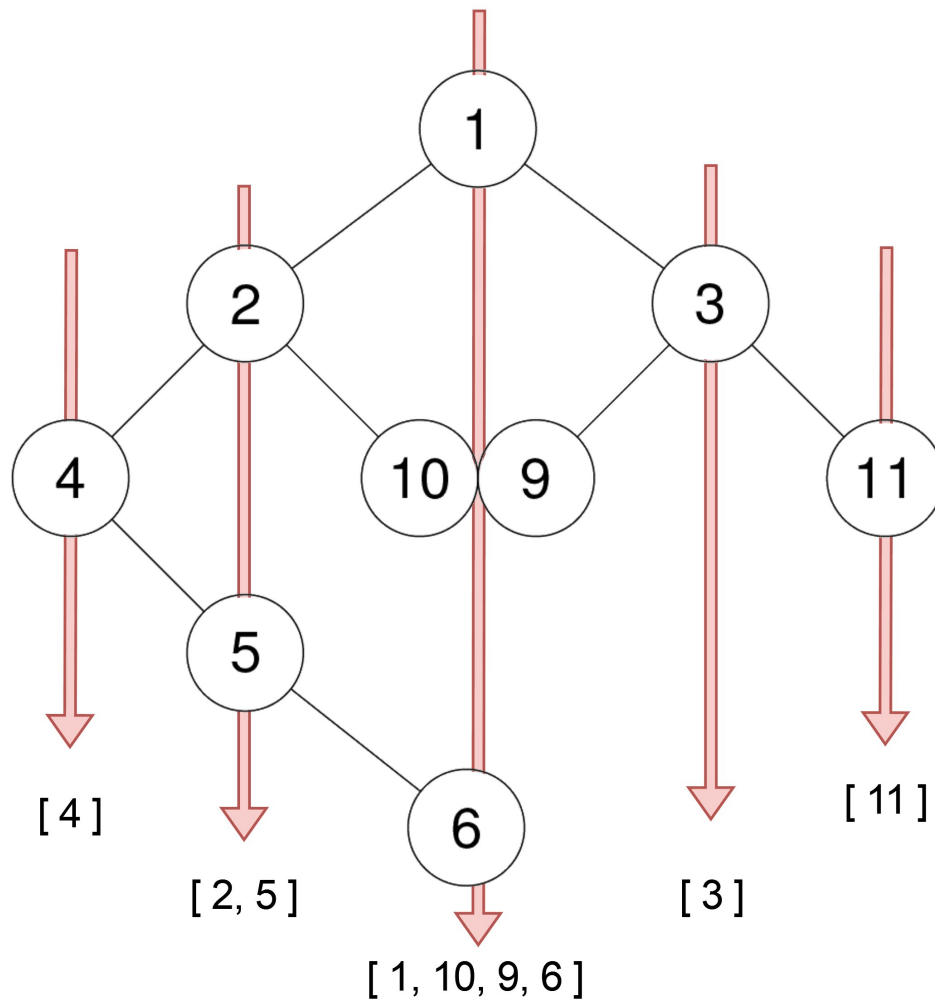
Example 2:



Input: root = [3,9,8,4,0,1,7]

Output: [[4],[9],[3,0,1],[8],[7]]

Example 3:



Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,6]

Output: [[4],[2,5],[1,10,9,6],[3],[11]]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Group nodes by column. Root is column 0. Left child = col-1. Right child = col+1.

Within each column, top-to-bottom. Ties (same row and column): left before right. Right?

Return columns sorted from leftmost to rightmost."

#

"[3,9,20,null,null,15,7] →
[[9],[3,15],[20],[7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

DFS assigning column index (left=-1, right=+1 relative to parent).

Collect (col, row, val) tuples; sort by (col, row, val) then group by column.

Correct but sorting dominates at $O(n \log n)$.

Time: $O(n \log n)$. Space: $O(n)$ tuples.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with column tracking)

"BFS ensures natural top-to-bottom, left-to-right order within each column.

Queue entries: (node, col). Use defaultdict(list) keyed by col.

After BFS, sort the column keys and return their lists.

Time: $O(n \log n)$ for sorting. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _314_VerticalOrderTraversal:
```

```
    def verticalOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
```

```
        if not root:
```

```
            return []
```

```
        cols : defaultdict = defaultdict(list)
```

```
        queue = deque([(root, 0)])
```

```
        min_col = max_col = 0
```

```
        while queue:
```

```
            node, col = queue.popleft()
```

```
            cols[col].append(node.val)
```

```
            min_col = min(min_col, col)
```

```
            max_col = max(max_col, col)
```

```

        if node.left:
queue.append((node.left, col - 1))
        if node.right:
queue.append((node.right, col + 1))
    return [cols[c] for c in
range(min_col, max_col + 1)]

```

PHASE 5 — TEST & DEBUG

```

# [3,9,20,null,null,15,7]:
# BFS: (3,0)→cols={0:[3]}.
(9,-1),(20,1)→cols={0:[3],-1:[9],1:[20]}.
(15,0),(7,2)→cols={0:[3,15],-1:[9],1:[20],
2:[7]}.
# min=-1, max=2. Return
[cols[-1],cols[0],cols[1],cols[2]] =
[[9],[3,15],[20],[7]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n)$  for BFS +  $O(cols)$  for
building the result — total  $O(n)$ .
# Using min_col/max_col avoids sorting
keys — the range is contiguous.
# BFS guarantees top-to-bottom,
left-to-right order within each column
naturally.

```

DFS would require explicit (row, insertion_order) sorting per column.
Given more time I'd ask: Vertical Order Traversal II (#987)?
Same column/row but within the same position, sort by value ascending – needs explicit row tracking."

Trees & BST

□ ★ #333 Largest BST Subtree ★

MED

[Open on LeetCode](#)

Dynamic Programming · Tree · DFS · BST

Lists: ★ Premium | Premium 98

Problem

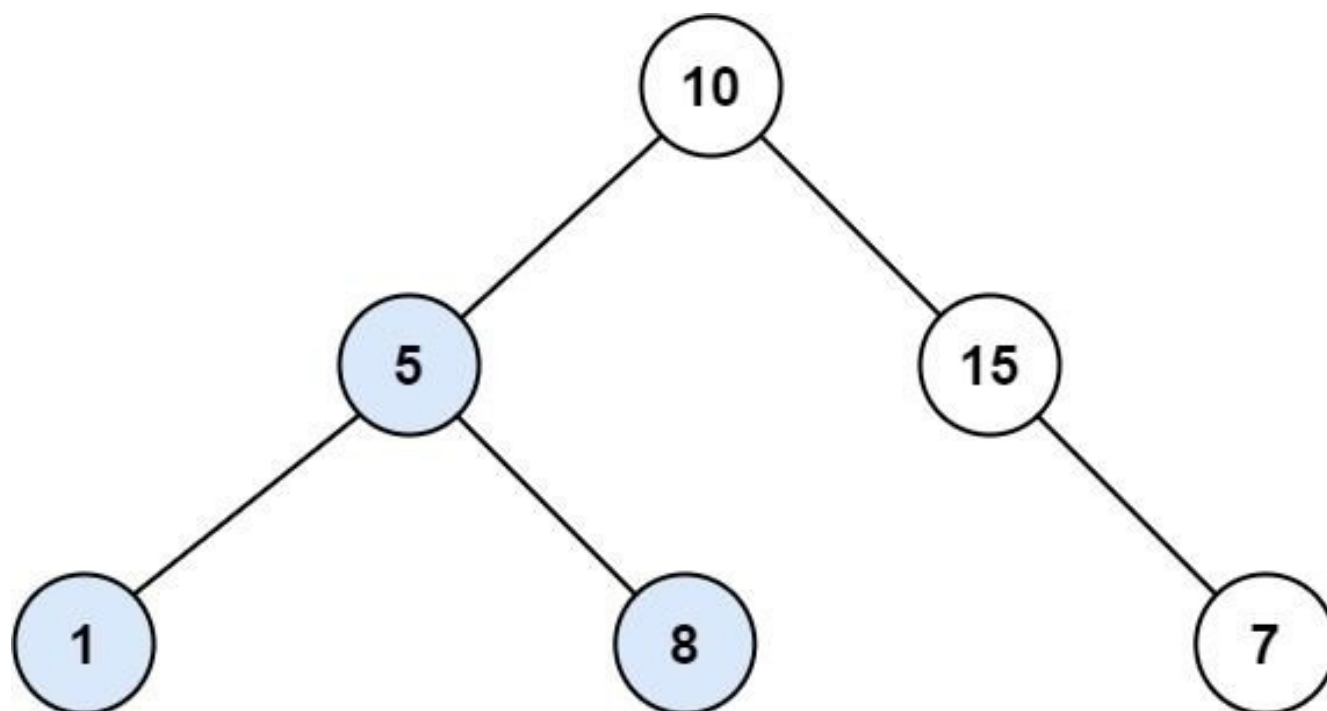
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:



Input: root = [10,5,15,1,8,null,7]

Output: 3

Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.

Example 2:

Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,1]

Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].

- $-104 \leq \text{Node.val} \leq 104$

Follow up: Can you figure out ways to solve it with $O(n)$ time complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the largest subtree (by node count) that forms a valid BST.

Return the node count. Right?"

#

"[10,5,15,1,8,null,7]: subtree at 5 (nodes 1,5,8) is a BST → 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For every node as root of a candidate subtree, validate BST property recursively

(min/max bounds) and count nodes if valid. Track global max count.

Revalidates overlapping subtrees – $O(n^2)$ worst case.

Time: $O(n^2)$. Space: $O(h)$ stack.

Should I go ahead and optimize?"

```
# PHASE 3 — OPTIMIZE (O(n) post-order)
# "Post-order: return (is_bst, size,
subtree_min, subtree_max).
# Empty node: (True, 0, +inf, -inf) —
sentinels pass any comparison cleanly.
# Valid BST: both children are BST AND
left.max < node.val < right.min."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _333_LargestBSTSubtree:
    def largestBSTSubtree(self, root:
Optional['TreeNode']) -> int:
        best = [0]
        def check(node):
            if not node:
                return True, 0,
float('inf'), float('-inf')
            l_ok, l_sz, l_mn, l_mx =
check(node.left)
            r_ok, r_sz, r_mn, r_mx =
check(node.right)
            if l_ok and r_ok and l_mx <
node.val < r_mn:
                sz = l_sz + r_sz + 1
                best[0] = max(best[0],
sz)
```

```
        return True, sz,  
min(l_mn, node.val), max(r_mx, node.val)  
        return False, 0, 0, 0  
    check(root)  
    return best[0]
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

**# "Time $O(n)$. Space $O(h)$. Sentinels
($+\infty/-\infty$ for empty) let comparisons pass
cleanly.**

**# Given more time I'd ask: what if we want
the root of the largest BST subtree? Track
it alongside size."**

Trees & BST

★ #366 Find Leaves of Binary Tree ★

MED[Open on LeetCode](#)

Tree · DFS · Sorting

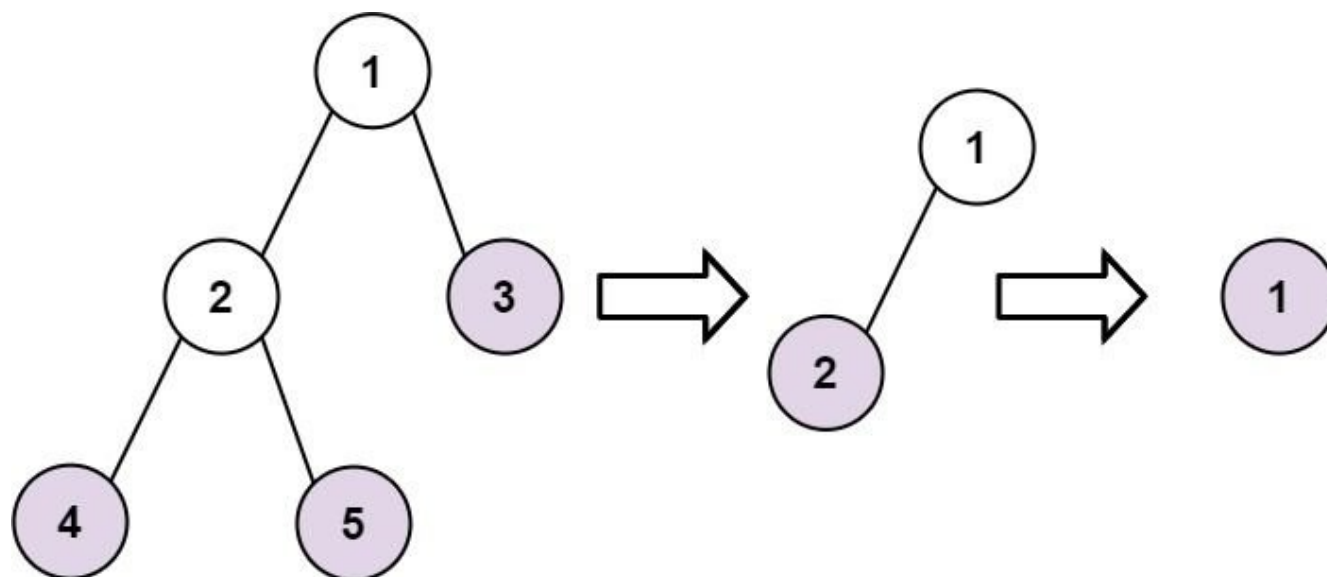
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, collect a tree's nodes as if you were doing this:

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1:



Input: root = [1,2,3,4,5]

Output: [[4,5,3],[2],[1]]

Explanation:

[[3,5,4],[2],[1]] and [[3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.

Example 2:

Input: root = [1]

Output: [[1]]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Collect leaves, remove them, repeat. Group nodes by their 'height from leaf' level. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeatedly find all current leaves (no children), remove them, append to result level.

Each round peels one tree layer – like reverse level-order.

Time: $O(n^2)$ if we rescan whole tree each round. Space: $O(n)$ output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order height: leaf=0, internal=max(left,right)+1. Group by height in one pass."

PHASE 4 — CLEAN CODE

class _366_FindLeaves:

```
    def findLeaves(self, root:
Optional['TreeNode']) -> List[List[int]]:
        result: List[List[int]] = []
        def height(node) -> int:
            if not node:
                return -1
            h = max(height(node.left),
height(node.right)) + 1
```

```
        if h == len(result):
            result.append([])
        result[h].append(node.val)
        return h
    height(root)
    return result
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4,5]: height(4)=0→result=[[4]].
height(5)=0→[[4,5]].
height(2)=1→[[4,5],[2]].
# height(3)=0→[[4,5,3],[2]].
height(1)=2→[[4,5,3],[2],[1]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$ . One DFS pass —
no repeated tree traversals.
```

```
# Given more time I'd ask: group by depth
from root? That's standard level-order
BFS."
```

Trees & BST

□ #437 Path Sum III

MED

[Open on LeetCode](#)

Tree · DFS · Prefix Sum

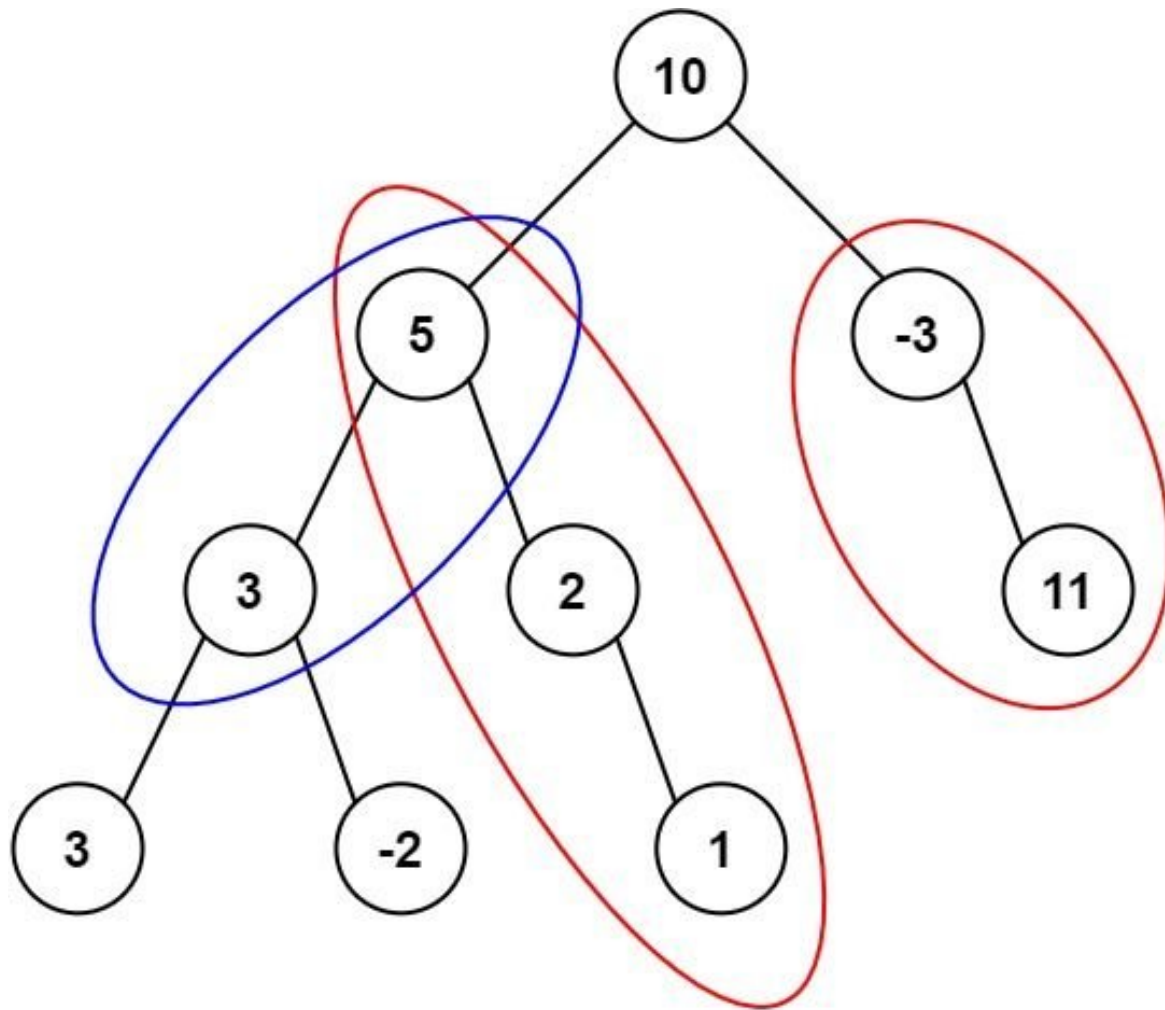
Lists: Grind 169

Problem

Given the root of a binary tree and an integer `targetSum`, return the number of paths where the sum of the values along the path equals `targetSum`.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:



Input: root =

[10,5,-3,3,2,null,11,3,-2,null,1],

targetSum = 8

Output: 3

Explanation: The paths that sum to 8 are shown.

Example 2:

```
Input: root =  
[5,4,8,11,null,13,4,7,2,null,null,5,1],  
targetSum = 22  
Output: 3
```

Constraints:

- The number of nodes in the tree is in the range $[0, 1000]$.
- $-109 \leq \text{Node.val} \leq 109$
- $-1000 \leq \text{targetSum} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count downward paths (parent→child only) summing to targetSum.

Path can start and end at any node. Values can be negative. Right?"

#

"[10,5,-3,3,2,null,11,3,-2,null,1], target=8 → 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Prefix-sum map: as DFS walks, track
cumulative sum from root.
# At each node, add count of prefix sums
equal to (current_sum - target) - includes
paths through node.
# Backtrack prefix counts when leaving
node.
# Time: O(n). Space: O(n) prefix map +
stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (prefix sum + hash map
— same pattern as #560)
# "DFS with running prefix sum. Count how
many earlier prefix sums equal
current_sum-target.
# Backtrack after each subtree to avoid
polluting sibling paths. Seed {0:1}."

# PHASE 4 — CLEAN CODE
class _437_PathSumIII:
    def pathSum(self, root:
Optional['TreeNode'], targetSum: int) ->
int:
        prefix = Counter({0: 1})
        count = [0]
```



```
def dfs(node, running):
    if not node:
        return
    running += node.val
    count[0] += prefix[running -
targetSum]

    prefix[running] += 1
    dfs(node.left, running)
    dfs(node.right, running)
    prefix[running] -= 1 #
backtrack
    dfs(root, 0)
    return count[0]
```

PHASE 5 — TEST & DEBUG

This is #560 applied to trees – same prefix-sum logic, but backtrack after each subtree.

root=10, target=8: paths [5,3], [5,2,1], [-3,11] each sum to 8 → count=3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Identical to Subarray Sum Equals K applied to a tree.
Backtracking prevents cross-subtree contamination – critical difference from

array version.

Given more time I'd ask: paths going in both directions (not just downward)?

Use the any-direction diameter-style post-order approach."

Trees & BST

□ ★ #545 Boundary of Binary Tree ★

MED

[Open on LeetCode](#)

Tree · DFS

Lists: ★ Premium | Premium 98

Problem

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

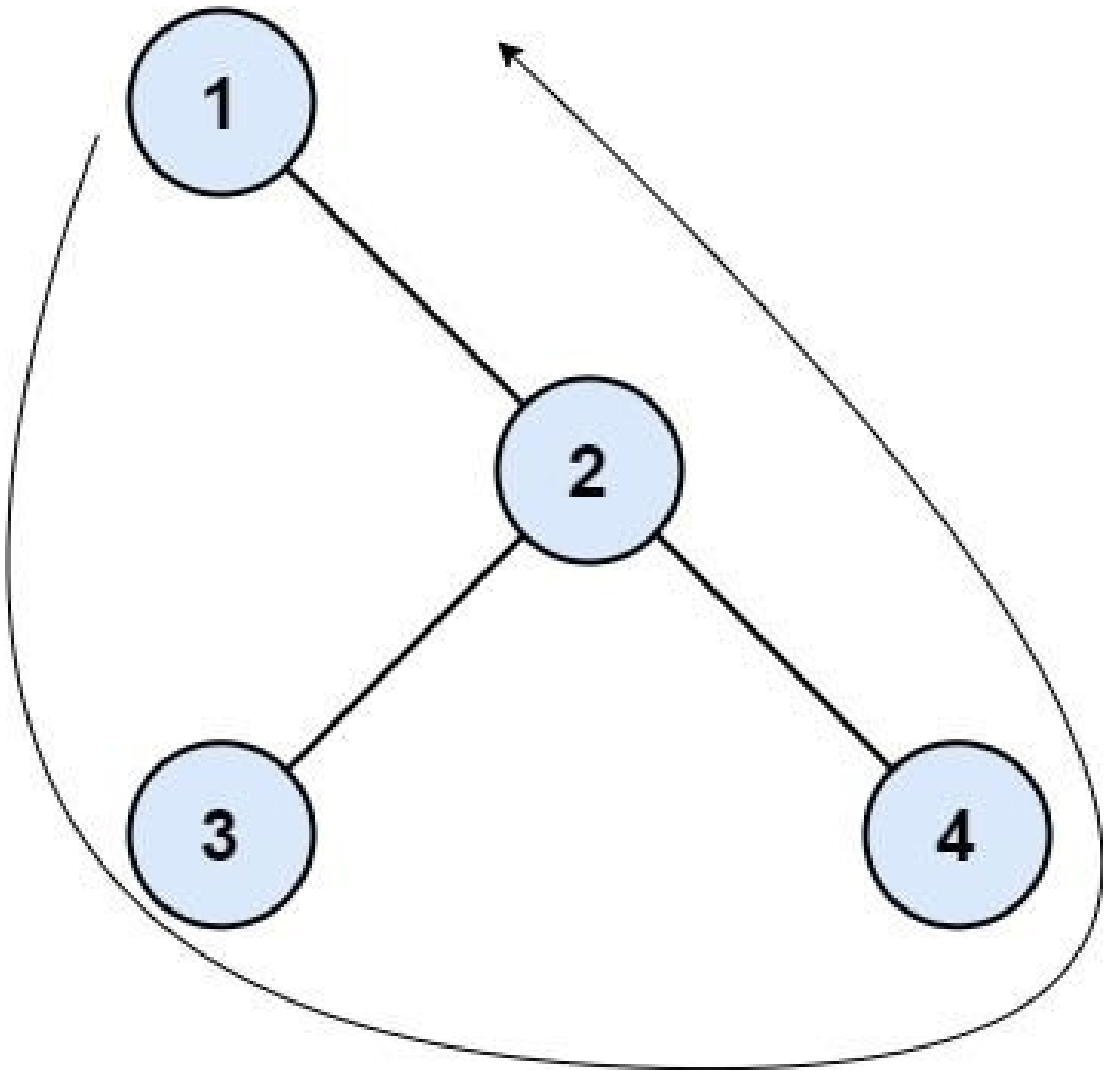
The left boundary is the set of nodes defined by the following:

- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.
- The leftmost leaf is not in the left boundary.

The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.

The leaves are nodes that do not have any children. For this problem, the root is not a leaf. Given the root of a binary tree, return the values of its boundary.

Example 1:



Input: root = [1,null,2,3,4]

Output: [1,3,4,2]

Explanation:

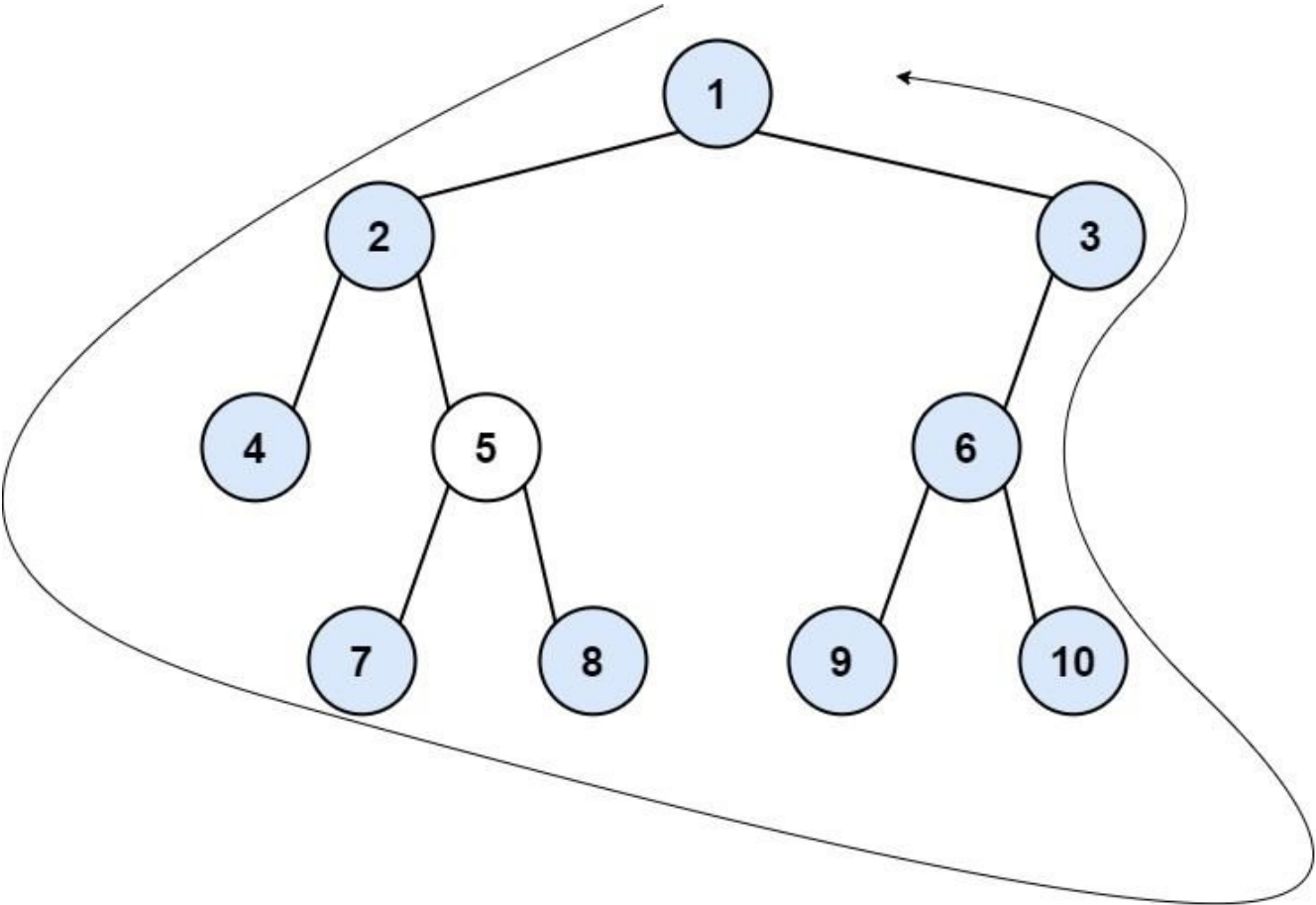
- The left boundary is empty because the root does not have a left child.
- The right boundary follows the path starting from the root's right child 2 -> 4.

4 is a leaf, so the right boundary is [2].

- The leaves from left to right are [3,4].

Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].

Example 2:



Input: root =

[1,2,3,4,5,6,null,null,null,7,8,9,10]

Output: [1,2,4,7,8,9,10,6,3]

Explanation:

– The left boundary follows the path starting from the root's left child 2

→ 4.

4 is a leaf, so the left boundary is [2].

– The right boundary follows the path starting from the root's right child 3

→ 6 → 10.

10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].

– The leaves from left to right are [4,7,8,9,10].

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-1000 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```
# "Boundary = root + left boundary  
(non-leaves top-down) + all leaves  
left-to-right  
# + right boundary (non-leaves bottom-up).  
Leftmost/rightmost leaves are in leaves,  
not boundary. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic.  
# Collect four pieces separately: root  
val, left boundary nodes top-down,  
# all leaves left-to-right, right boundary  
nodes bottom-up.  
# Merge while skipping duplicate leaves  
already counted.  
# Time: O(n). Space: O(h) recursion for  
leaf collection.  
# Should I go ahead and optimize?"
```

```
class _545_BoundaryBinaryTree:  
    def boundaryOfBinaryTree(self, root:  
Optional['TreeNode']) -> List[int]:  
        if not root:  
            return []
```

```
def is_leaf(n): return not n.left
and not n.right
def left_boundary(n):
    res = []
    while n and not is_leaf(n):
        res.append(n.val)
        n = n.left if n.left else
n.right
    return res
def leaves(n):
    if not n: return []
    if is_leaf(n): return [n.val]
    return leaves(n.left) +
leaves(n.right)
def right_boundary(n):
    res = []
    while n and not is_leaf(n):
        res.append(n.val)
        n = n.right if n.right
else n.left
    return res[::-1]
if is_leaf(root):
    return [root.val]
return [root.val] +
left_boundary(root.left) + leaves(root) +
```

right_boundary(root.right)

PHASE 5 — TEST & DEBUG

**# [1,null,2,3,4]: left=[]. leaves=[3,4].
right=[2]. → [1,3,4,2] ✓**

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Three separate traversals, each linear.

**# 'Prefer left else right' for left spine;
'prefer right else left' for right spine –
reversed at end.**

**# Given more time I'd ask: return boundary
clockwise vs counter-clockwise? Just
reverse the result."**

Trees & BST

□ ★ #549 Binary Tree Longest Consecutive Sequence II ★

MED

[Open on LeetCode](#)

Tree · DFS

Lists: ★ Premium | Premium 98

Problem

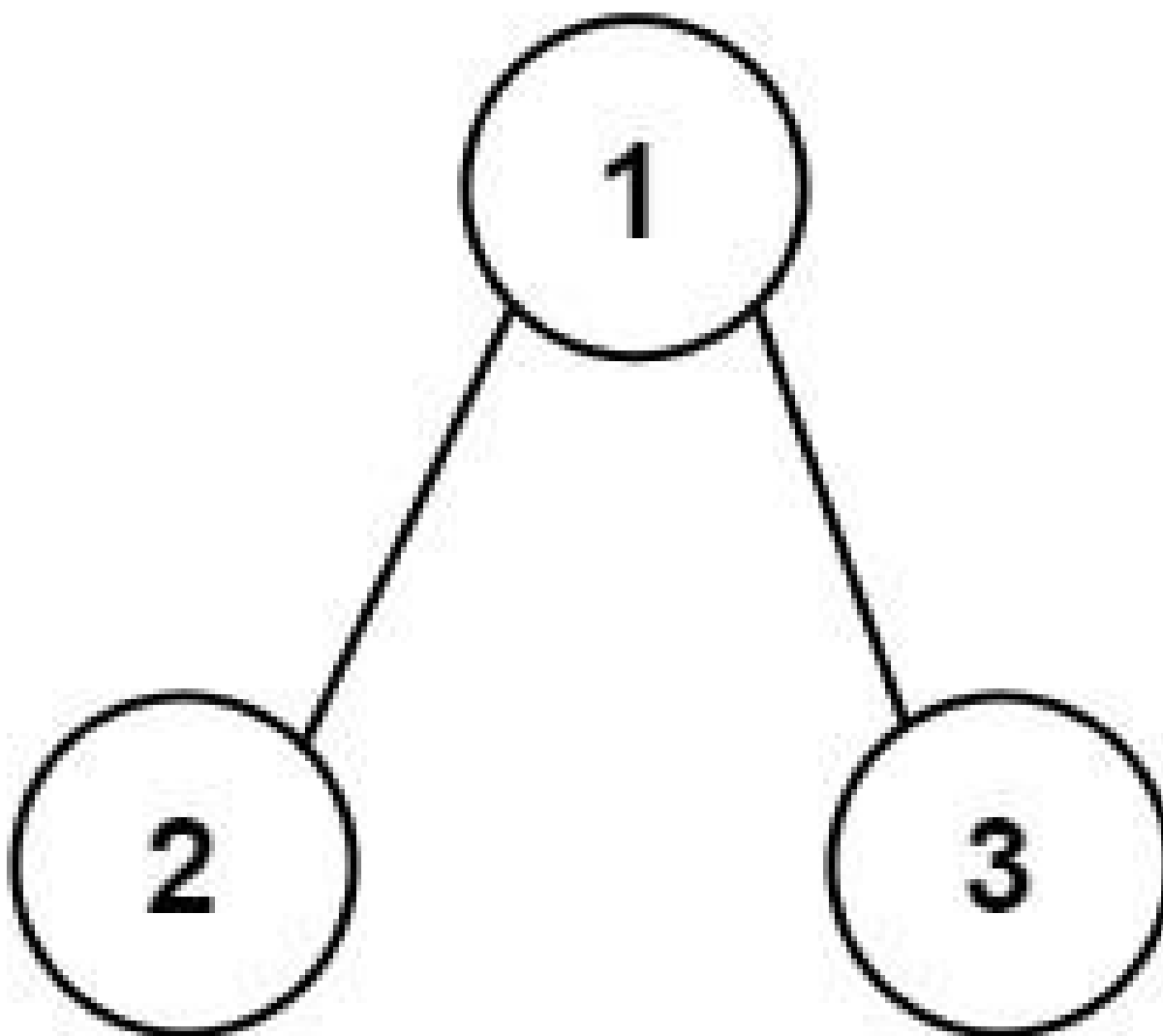
Given the root of a binary tree, return the length of the longest consecutive path in the tree.

A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

- For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid.

On the other hand, the path can be in the child–Parent–child order, where not necessarily be parent–child order.

Example 1:

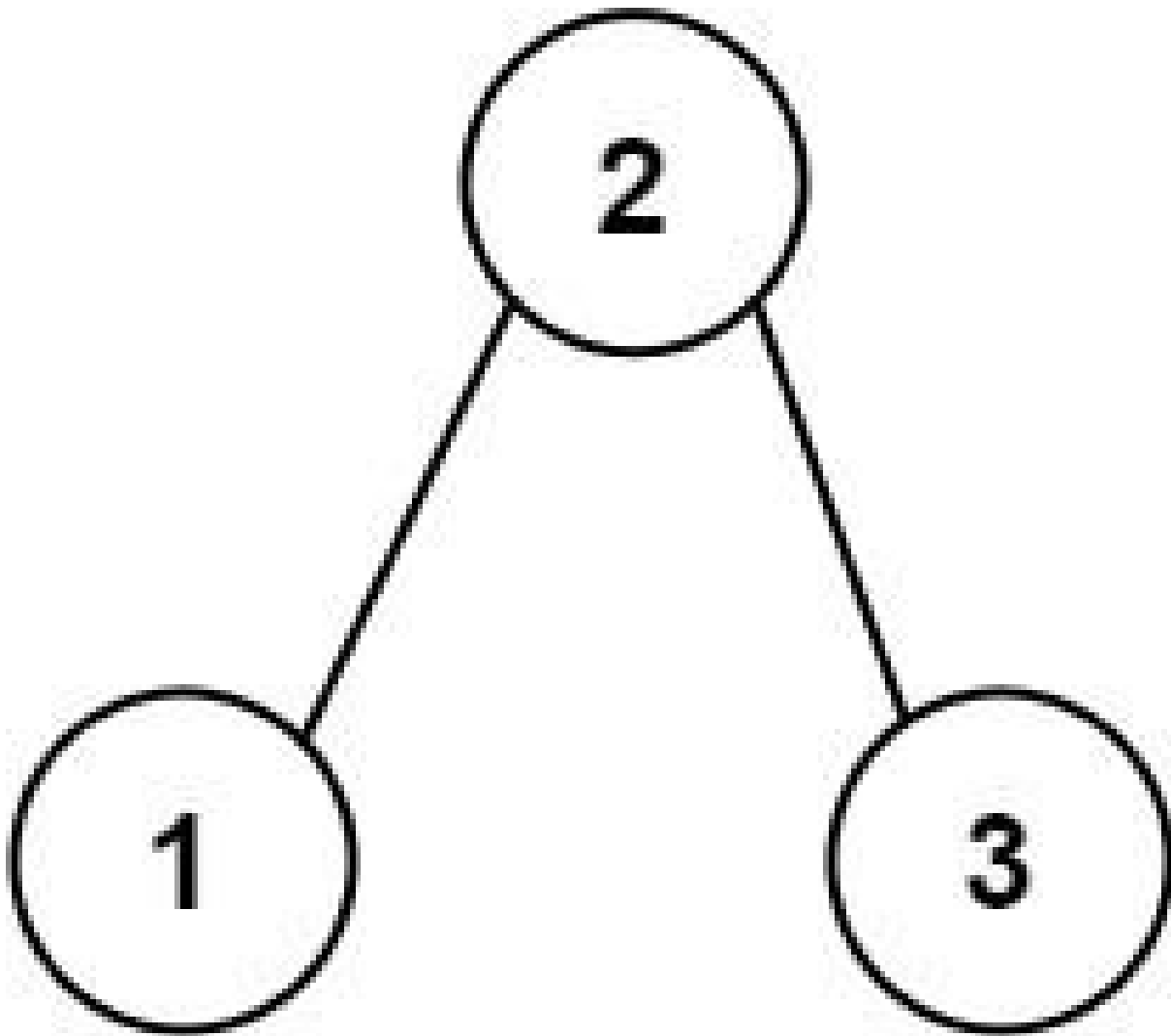


Input: root = [1,2,3]

Output: 2

Explanation: The longest consecutive path is [1, 2] or [2, 1].

Example 2:



Input: root = [2,1,3]

Output: 3

Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest path where consecutive values differ by 1 – can go up OR down, can change direction at one pivot. Right?"

#

"[2,1,3] → [1,2,3] length 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

At each node, run DFS tracking consecutive count when `child.val == parent.val + 1`.

Reset count when streak breaks. Track global max streak length.

Visits every node once with $O(n)$ DFS per starting node worst case $O(n^2)$.

Time: $O(n^2)$ naive. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (post-order returning inc/dec lengths)

"Each node returns (inc, dec): length of increasing/decreasing sequence going DOWN from it."

Diameter through node = best combination of one child's inc and another's dec, meeting at this node."

PHASE 4 — CLEAN CODE

```
class _549_LongestConsecutiveII:
    def longestConsecutive(self, root:
Optional['TreeNode']) -> int:
        best = [0]
        def dfs(node):
            if not node:
                return 0, 0
            l_inc, l_dec = dfs(node.left)
            r_inc, r_dec =
dfs(node.right)
            inc = dec = 1
            if node.left:
                if node.left.val ==
node.val + 1: inc = max(inc, l_inc + 1)
                if node.left.val ==
node.val - 1: dec = max(dec, l_dec + 1)
            if node.right:
                if node.right.val ==
node.val + 1: inc = max(inc, r_inc + 1)
                if node.right.val ==
node.val - 1: dec = max(dec, r_dec + 1)
```



```
        best[0] = max(best[0], inc +  
dec - 1)  
        return inc, dec  
    dfs(root)  
    return best[0]
```

PHASE 5 — TEST & DEBUG

```
# [2,1,3]: dfs(1)=(1,1). dfs(3)=(1,1).  
dfs(2): left(1)=val-1→dec=2.  
right(3)=val+1→inc=2. best=2+2-1=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h). inc+dec-1 avoids  
double-counting the pivot node.  
# Generalises #298 by allowing direction  
change at the pivot node.  
# Given more time I'd ask: allow path to  
go up? Needs full graph treatment."
```

Trees & BST

□ ★ #582 Kill Process ★

MED

[Open on LeetCode](#)

Array · Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

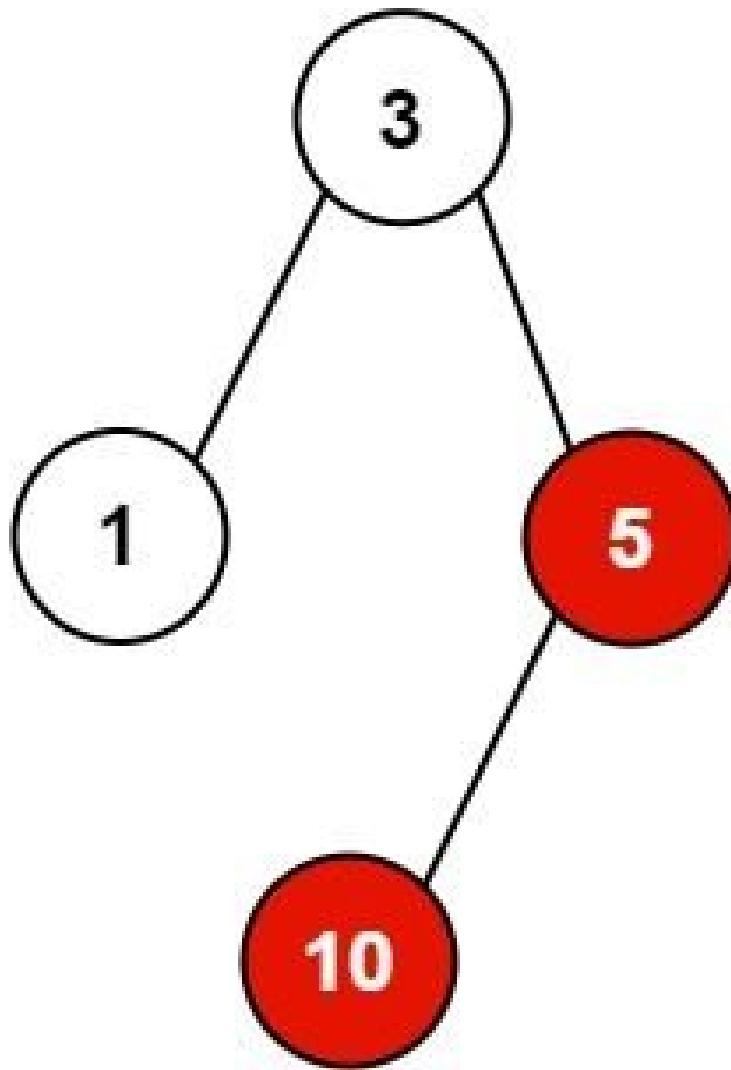
You have n processes forming a rooted tree structure. You are given two integer arrays `pid` and `ppid`, where `pid[i]` is the ID of the i th process and `ppid[i]` is the ID of the i th process's parent process.

Each process has only one parent process but may have multiple children processes. Only one process has `ppid[i] = 0`, which means this process has no parent process (the root of the tree).

When a process is killed, all of its children processes will also be killed.

Given an integer `kill` representing the ID of a process you want to kill, return a list of the IDs of the processes that will be killed. You may return the answer in any order.

Example 1:



Input: `pid = [1,3,10,5]`, `ppid = [3,0,5,3]`, `kill = 5`

Output: `[5,10]`

Explanation: The processes colored in red are the processes that should be killed.

Example 2:

Input: `pid = [1], ppid = [0], kill = 1`
Output: `[1]`

Constraints:

- `n == pid.length`
- `n == ppid.length`
- `1 <= n <= 5 * 104`
- `1 <= pid[i] <= 5 * 104`
- `0 <= ppid[i] <= 5 * 104`
- Only one process has no parent.
- All the values of `pid` are unique.
- `kill` is guaranteed to be in `pid`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Kill a process and all its descendants.
Return all killed process IDs. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Build children map and indegree from the
kill-pair list.

BFS from the killed pid: enqueue all its direct children, then propagate kills downward.

Every reachable process dies – simulates the cascade.

Time: $O(n)$. Space: $O(n)$ for graph + queue.

Should I go ahead and optimize?"

```
class _582_KillProcess:
```

```
    def killProcess(self, pid: List[int],  
ppid: List[int], kill: int) -> List[int]:
```

```
        children: defaultdict =  
defaultdict(list)  
        for child, parent in zip(pid,  
ppid):  
            children[parent].append(child  
)
```

```
        killed = []
```

```
        queue = deque([kill])
```

```
        while queue:
```

```
            proc = queue.popleft()
```

```
            killed.append(proc)
```

```
            queue.extend(children[proc])
```

```
        return killed
```

PHASE 5 — TEST & DEBUG

```
# pid=[1,3,10,5], ppid=[3,0,5,3], kill=5:  
children={3:[1,5],5:[10]}.
```

```
# BFS from 5: [5,10] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$ . Build adjacency  
list then BFS – standard implicit tree  
traversal.
```

```
# Given more time I'd ask: kill all  
ancestors too? Walk parent pointers upward  
instead."
```

Trees & BST

□ #662 Maximum Width of Binary Tree

MED

[Open on LeetCode](#)

Tree · BFS

Lists: Grind 169

Problem

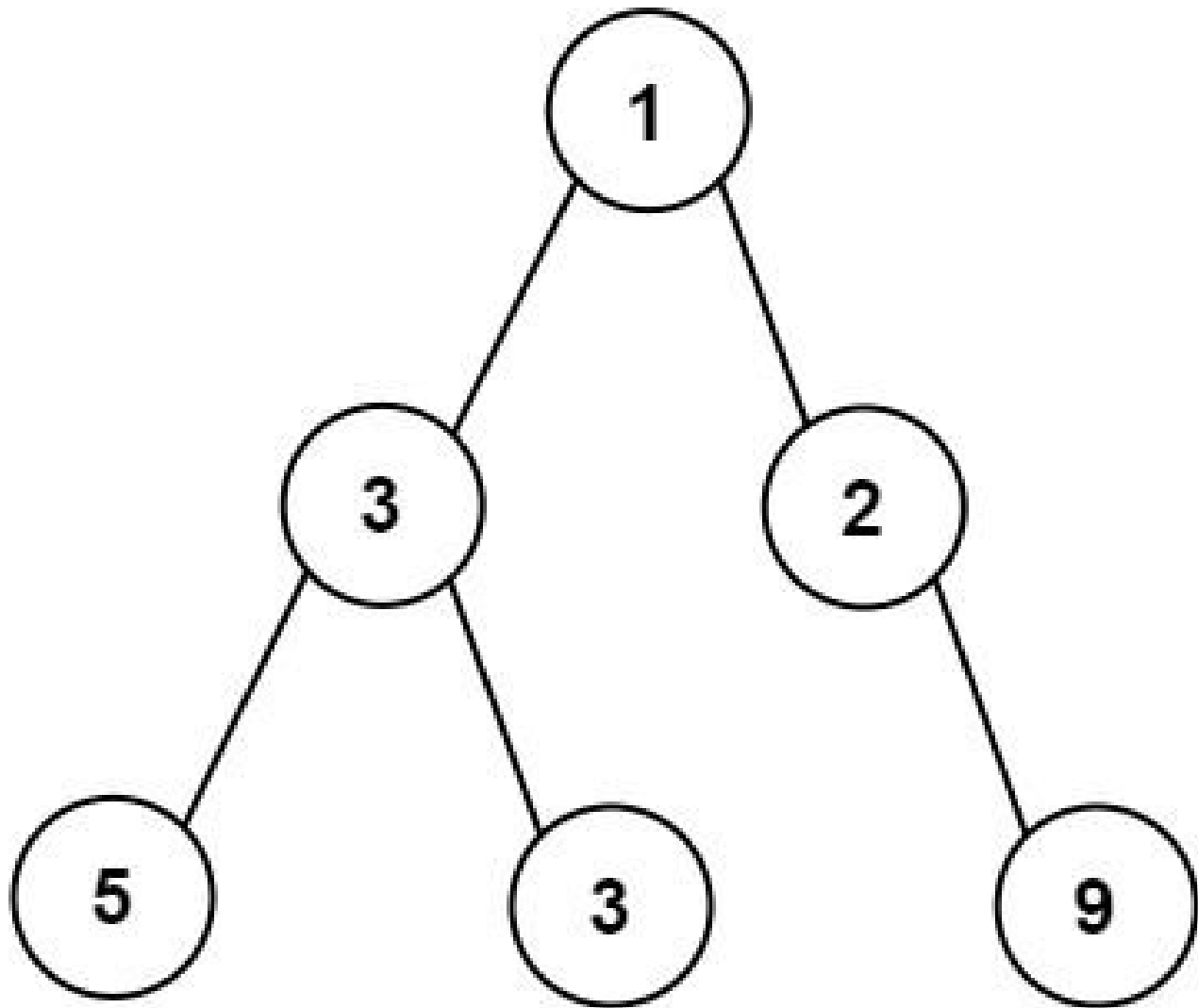
Given the root of a binary tree, return the maximum width of the given tree.

The maximum width of a tree is the maximum width among all levels.

The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:

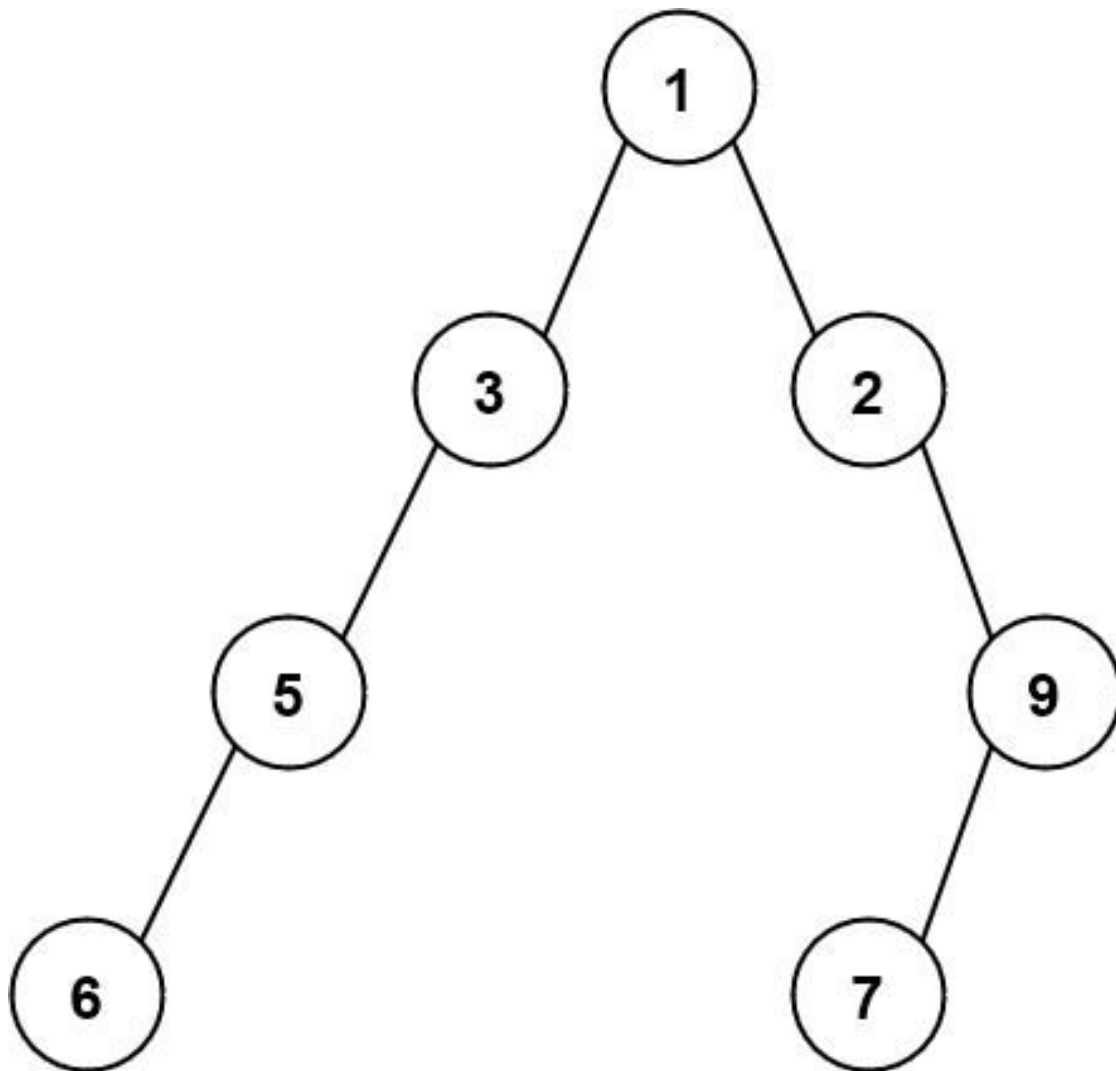


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:



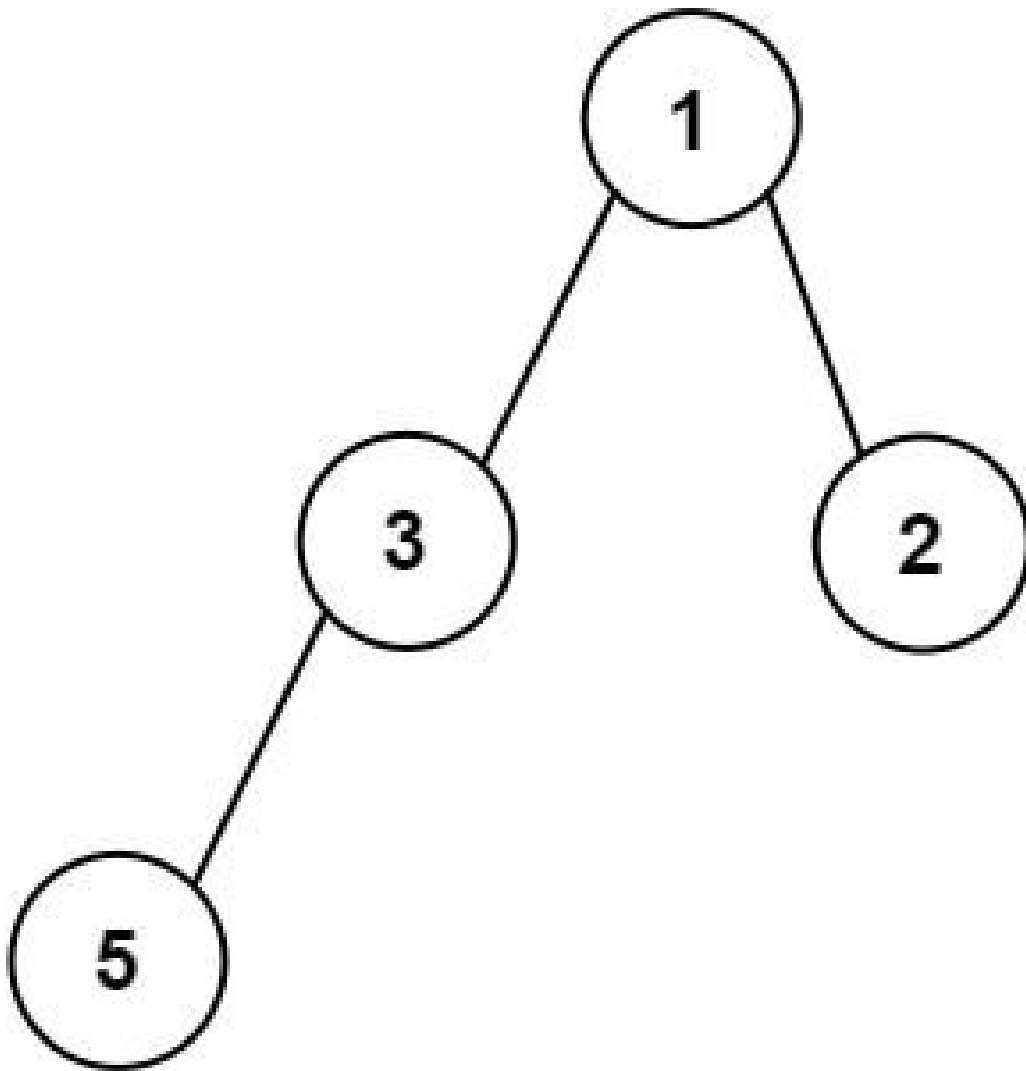
Input: root =

[1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- $-100 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Width = rightmost_pos - leftmost_pos + 1 at each level, counting null gaps.

Left child = 2*pos, right child = 2*pos+1. Normalise each level to prevent overflow. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS level-order traversal; at each level track min and max index assigned to nodes.

Width of level = max_index - min_index + 1; answer is global max width.

Straightforward queue-based level scan.

Time: $O(n)$. Space: $O(w)$ queue width.

Should I go ahead and optimize?"

```
class _662_MaxWidthBinaryTree:
```

```
    def widthOfBinaryTree(self, root: Optional['TreeNode']) -> int:
```

```
        if not root:
```

```
            return 0
```

```
        max_w = 0
```

```
queue = deque([(root, 0)])
while queue:
    size = len(queue)
    first_pos = queue[0][1]
    last_pos = 0
    for _ in range(size):
        node, pos =
queue.popleft()
        pos -= first_pos #
normalise
        last_pos = pos
        if node.left:
queue.append((node.left, 2 * pos))
        if node.right:
queue.append((node.right, 2 * pos + 1))
        max_w = max(max_w, last_pos +
1)

    return max_w
```

PHASE 5 — TEST & DEBUG

[1,3,2,5,3,null,9]: Level 2 positions
(normalised): 5→0, 3→1, 9→3. last=3.
width=4 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(w)$. Normalisation prevents 2^{depth} overflow for deep trees – critical.

Given more time I'd ask: width by actual node count (ignoring nulls)? Just max level_size in BFS."

Trees & BST

□ #863 All Nodes Distance K in Binary Tree **MED**

[Open on LeetCode](#)

Hash Table · Tree · DFS · BFS

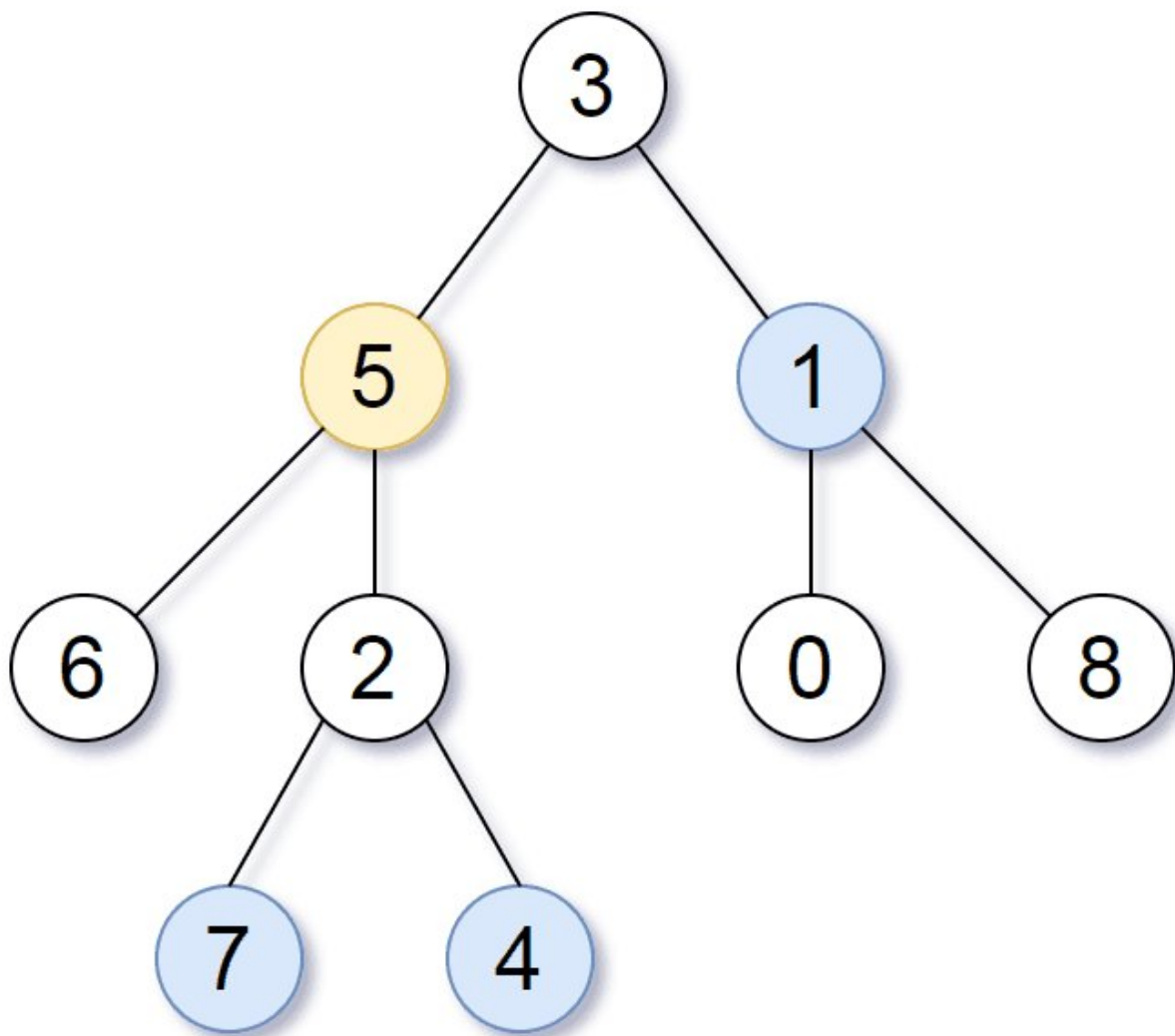
Lists: Grind 169

Problem

Given the root of a binary tree, the value of a target node *target*, and an integer *k*, return an array of the values of all nodes that have a distance *k* from the target node.

You can return the answer in any order.

Example 1:



Input: root =
[3,5,1,6,2,0,8,null,null,7,4], target =
5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3
Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$
- All the values Node.val are unique.
- target is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find all nodes exactly k edges from target. Distance can go upward through parent. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Build parent map via BFS/DFS from target. For each query node, walk parent pointers


```
# until reaching target; collect distance.  
O(n) per query naive.  
# Time:  $O(n * Q)$  for Q queries. Space:  
O(n) parent map.  
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (build parent map then  
BFS)
```

```
# "DFS to build parent[node]=parent. Then  
BFS from target treating tree as  
undirected graph.  
# Stop collecting at distance k."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _863_DistanceK:  
    def distanceK(self, root: 'TreeNode',  
target: 'TreeNode', k: int) -> List[int]:  
        parent: dict = {}  
        def build(node, par):  
            if not node: return  
            parent[node] = par  
            build(node.left, node)  
            build(node.right, node)  
        build(root, None)  
        visited = {target}  
        queue = deque([(target, 0)])
```

```
result = []
while queue:
    node, dist = queue.popleft()
    if dist == k:
        result.append(node.val)
        continue
    for nb in [node.left,
node.right, parent[node]]:
        if nb and nb not in
visited:
            visited.add(nb)
            queue.append((nb,
dist + 1))
return result
```

PHASE 5 — TEST & DEBUG

target=5, k=2: BFS outward from 5 by 2 hops → [7,4,1] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Parent map converts tree to undirected graph – then BFS is trivial.

Given more time I'd ask: multiple targets? Multi-source BFS – enqueue all at distance 0."

Trees & BST

□ ★ #1120 Maximum Average Subtree ★

MED

[Open on LeetCode](#)

Tree · DFS

Lists: ★ Premium | Premium 98

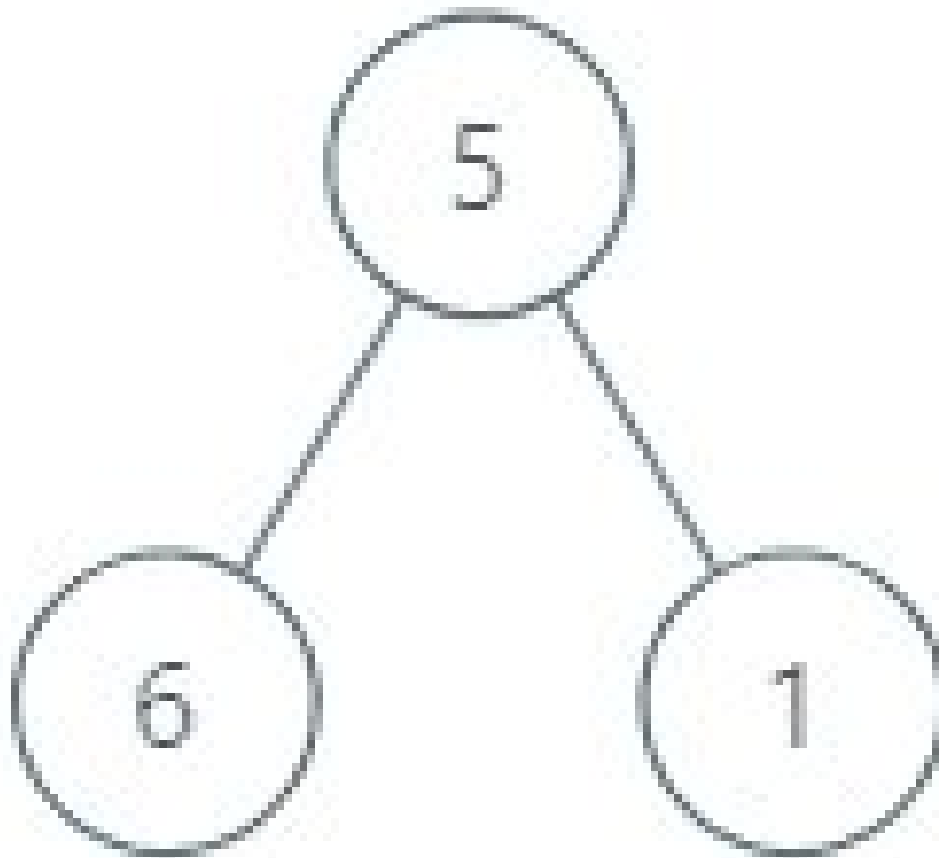
Problem

Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10^{-5} of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1:



Input: root = [5,6,1]

Output: 6.00000

Explanation:

For the node with value = 5 we have an average of $(5 + 6 + 1) / 3 = 4$.

For the node with value = 6 we have an average of $6 / 1 = 6$.

For the node with value = 1 we have an average of $1 / 1 = 1$.

So the answer is 6 which is the maximum.

Example 2:

Input: root = [0,null,1]

Output: 1.00000

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the subtree (any node + all descendants) with the maximum average value. Return the average. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Post-order DFS: for each node return (sum, count) of its subtree.

Track max average = sum/count over all visited nodes.

Recomputes subtree sums independently – correct $O(n)$ post-order.

Time: $O(n)$. Space: $O(h)$ stack.

Should I go ahead and optimize?"

```
class _1120_MaxAvgSubtree:
    def maximumAverageSubtree(self, root:
Optional['TreeNode']) -> float:
        best = [0.0]
        def dfs(node):
            if not node: return 0, 0
            l_sum, l_cnt = dfs(node.left)
            r_sum, r_cnt =
dfs(node.right)
            total_sum = l_sum + r_sum +
node.val
            total_cnt = l_cnt + r_cnt + 1
            best[0] = max(best[0],
total_sum / total_cnt)
            return total_sum, total_cnt
        dfs(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

[5,6,1]: avg(6)=6, avg(1)=1,
avg(5,6,1)=4. → 6.0 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Post-order
carries (sum, count) up — compute average

at each node.

Given more time I'd ask: minimum average subtree? Track min instead of max."

Trees & BST

□ ★ #1490 Clone N-ary Tree ★

MED

[Open on LeetCode](#)

Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

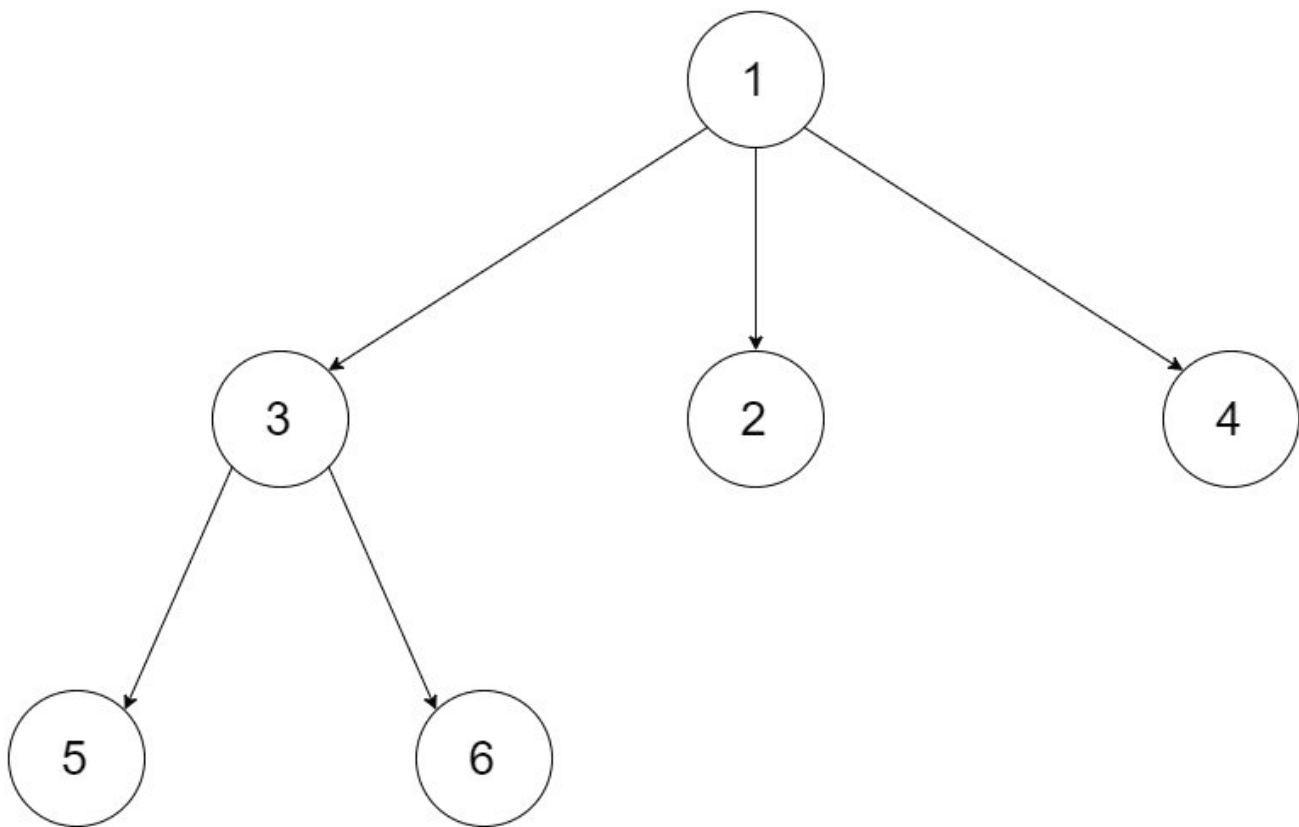
Given a root of an N-ary tree, return a deep copy (clone) of the tree.

Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.

```
class Node {  
    public int val;  
    public List<Node> children;  
}
```

Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

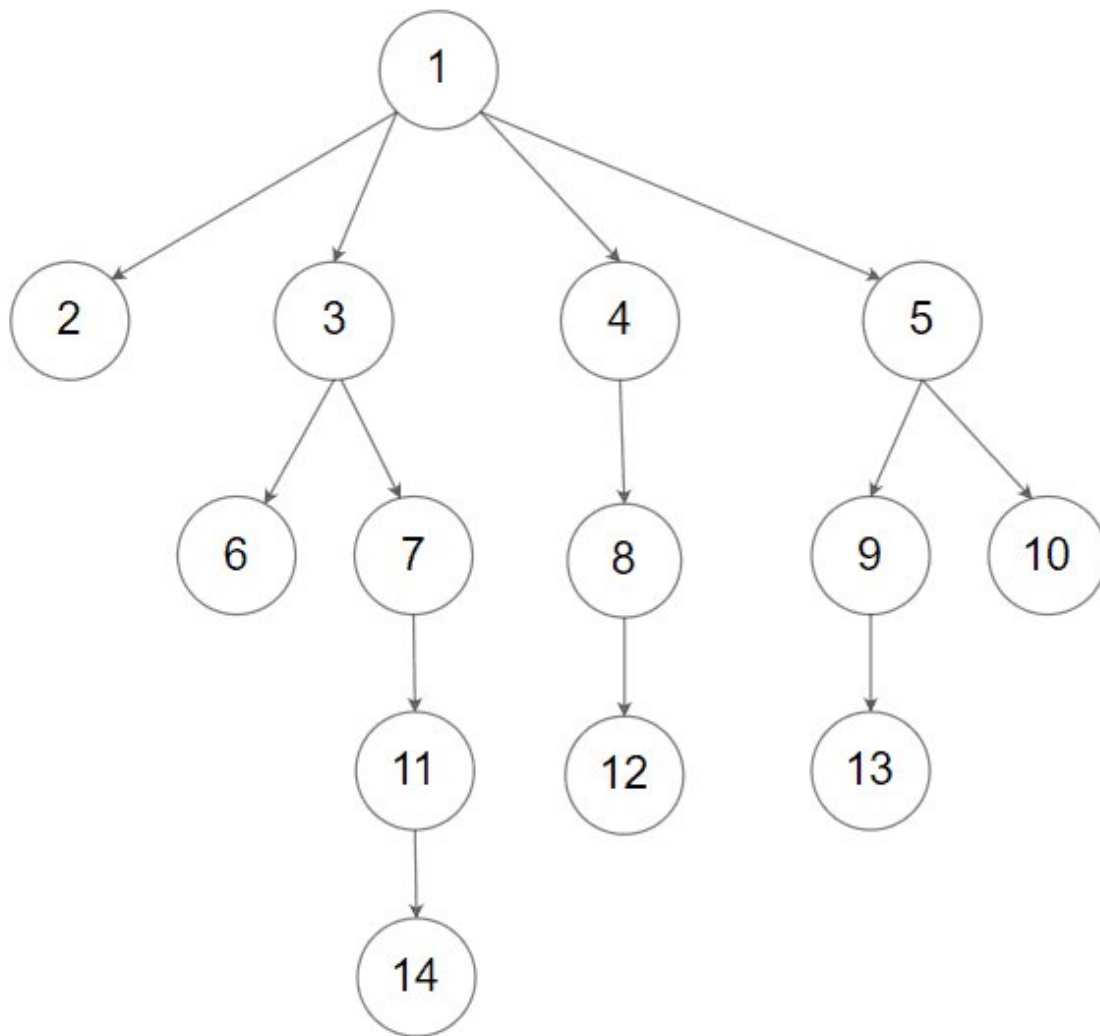
Example 1:



Input: root = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.

- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep clone an N-ary tree. No shared references between original and clone. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS clone: map each original node to a new node, recursively clone all children lists.

First pass creates nodes on demand; second pass wires child pointers.

Time: $O(n)$. Space: $O(n)$ for map + recursion.

Should I go ahead and optimize?"

```
class _1490_CloneNaryTree:
    def cloneTree(self, root: 'Node') ->
        'Node':
```

```
        if not root:
            return None
        return Node(root.val,
                    [self.cloneTree(child) for child in
                     root.children])
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Simple recursive clone – no visited map needed (trees have no cycles).

Follow-up (Clone Graph #133): needs visited map to handle cycles – {original: clone}."

Trees & BST

□ ★ #1522 Diameter of N-Ary Tree ★

MED

[Open on LeetCode](#)

Tree · DFS

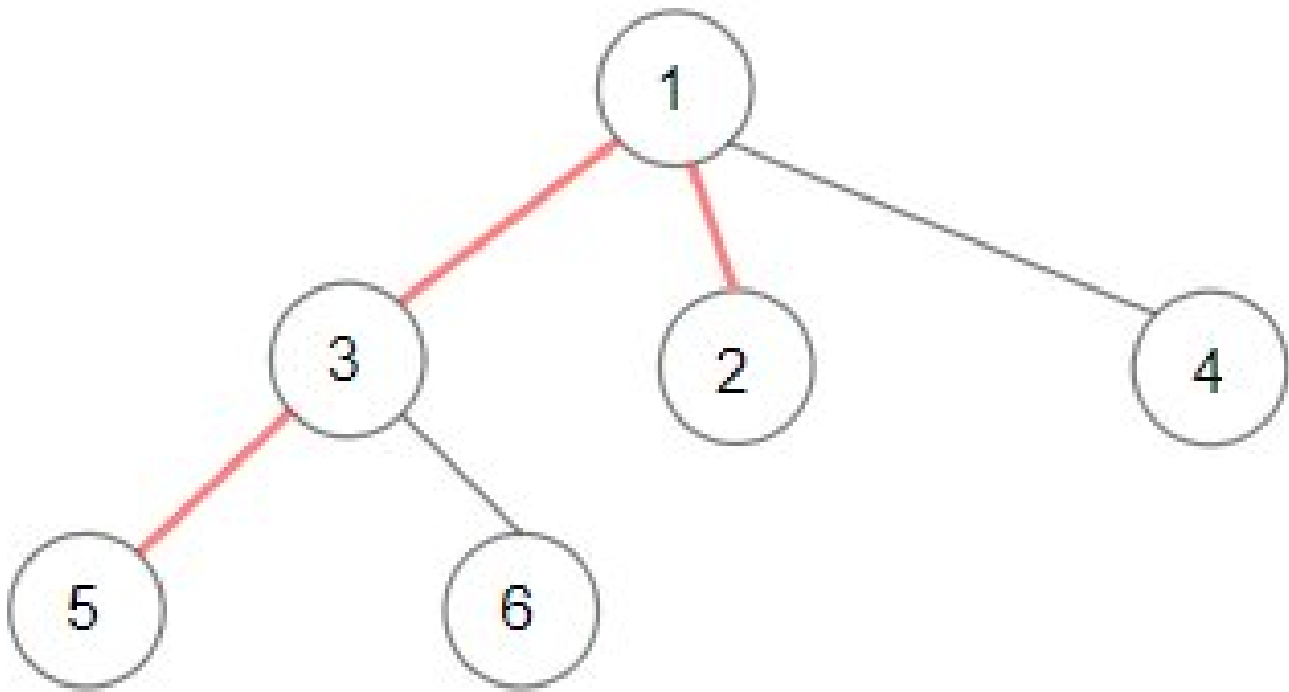
Lists: ★ Premium | Premium 98

Problem

Given a root of an N-ary tree, you need to compute the length of the diameter of the tree. The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:

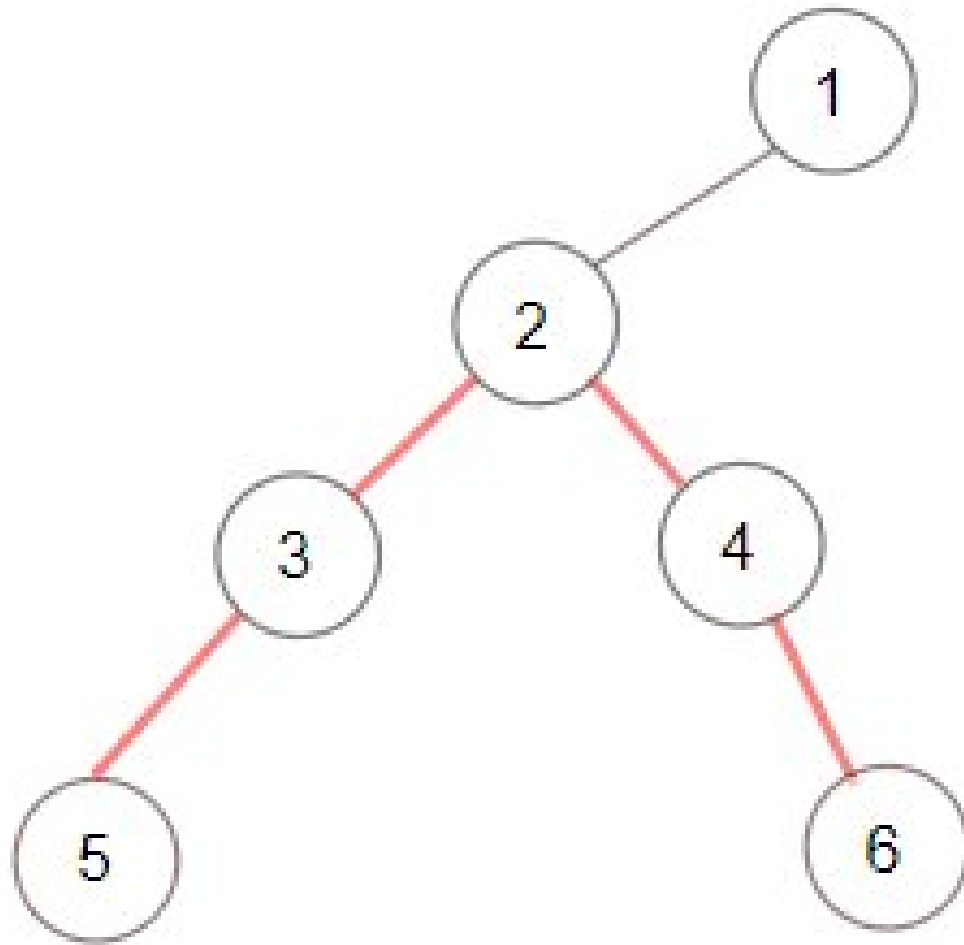


Input: root = [1,null,3,2,4,null,5,6]

Output: 3

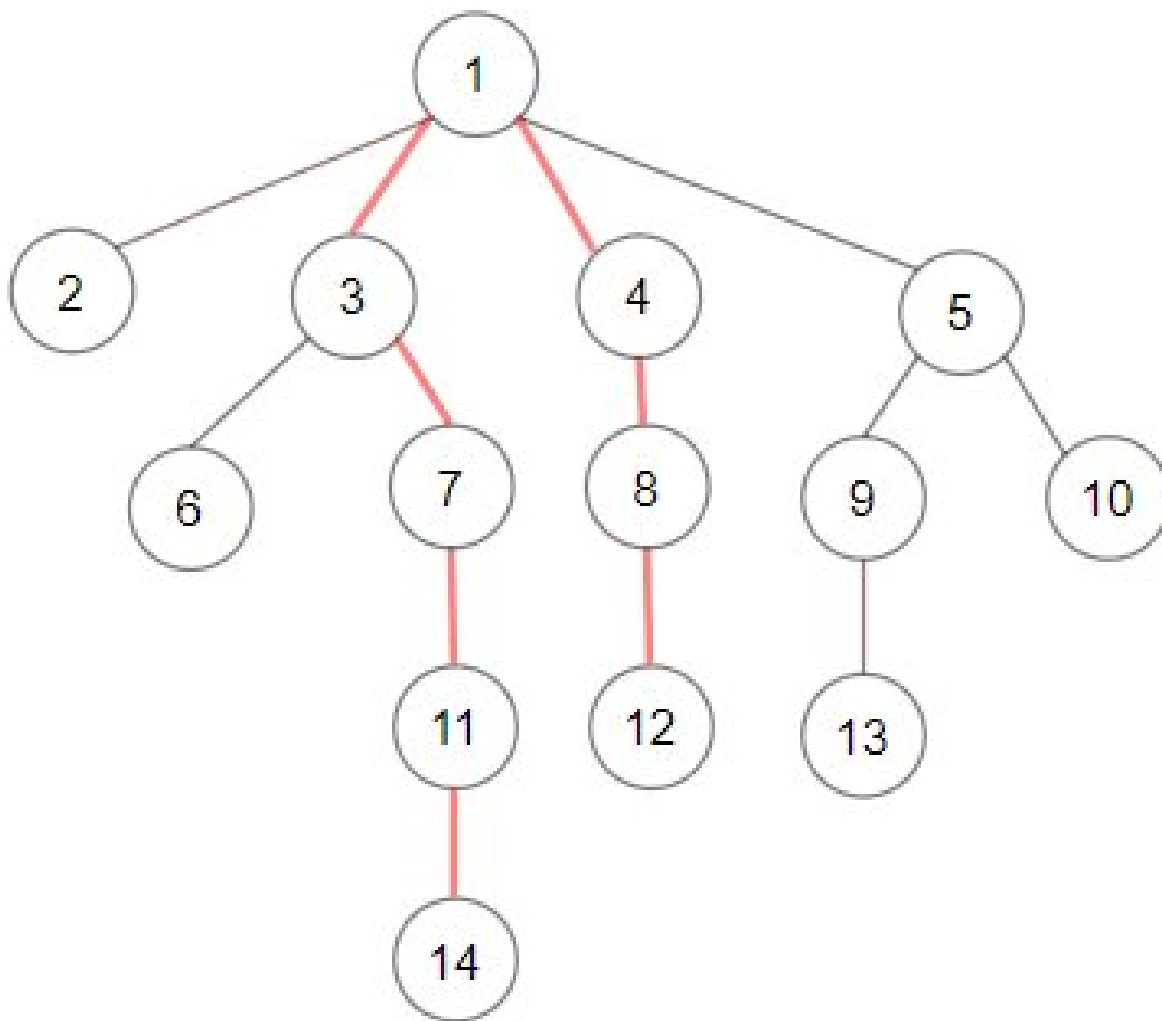
Explanation: Diameter is shown in red color.

Example 2:



Input: root =
[1,null,2,null,3,4,null,5,null,6]
Output: 4

Example 3:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Output: 7

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 104].

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = longest path between any two nodes (in edges), generalised to N children. Right?"

Same concept as Binary Tree Diameter (#543)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Two-pass DFS diameter: for each node, longest path through it = sum of top two child depths.

Same as binary tree diameter but children list instead of left/right.

Time: $O(n)$. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order: for each node, collect heights of children (adding 1 for the edge to parent)."

Diameter through this node = top two child-heights summed.

```
# Node's own height = max child-height (or  
0 for leaf)."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _1522_DiameterNaryTree:  
    def diameter(self, root: 'Node') ->  
int:  
        best = [0]  
        def height(node) -> int:  
            if not node or not  
node.children:  
                return 0  
            # child_h+1 converts "height  
below child" to "edges from this node via  
that child"  
            child_heights =  
sorted([height(c) + 1 for c in  
node.children], reverse=True)  
            if len(child_heights) >= 2:  
                best[0] = max(best[0],  
child_heights[0] + child_heights[1])  
            else:  
                best[0] = max(best[0],  
child_heights[0])  
            return child_heights[0]  
        return height(root)
```

```
return best[0]
```

PHASE 5 — TEST & DEBUG

```
# [1,null,3,2,4,null,5,6]: height(5)=0,  
height(6)=0.
```

```
# height(3):
```

```
child_heights=[0+1,0+1]=[1,1].
```

```
best=1+1=2. return 1.
```

```
# height(2)=0, height(4)=0.
```

```
# height(1):
```

```
child_heights=[1+1,0+1,0+1]=[2,1,1].
```

```
best=2+1=3. return 2. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$ . Adding +1 when  
collecting child heights is the key  
difference from #543.
```

```
# In binary tree, depth(child) already  
includes the edge up; here we add it  
explicitly.
```

```
# Given more time I'd ask: diameter in  
edge-weighted N-ary tree?
```

```
# Same approach – just use edge weights  
instead of +1."
```

Trees & BST

□ ★ #1650 Lowest Common Ancestor of a Binary Tree III ★

MED

[Open on LeetCode](#)

Hash Table · Tree · Two Pointers

Lists: ★ Premium | Premium 98

Problem

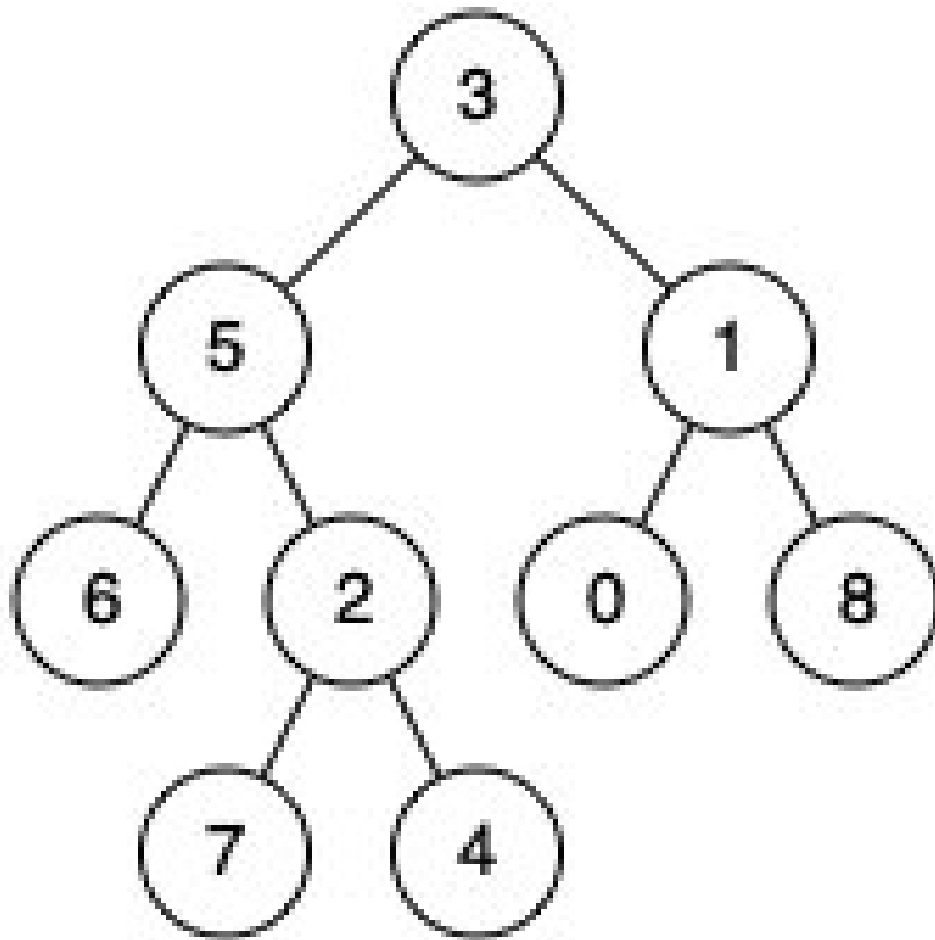
Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

```
class Node {  
    public int val;  
    public Node left;  
    public Node right;  
    public Node parent;  
}
```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:

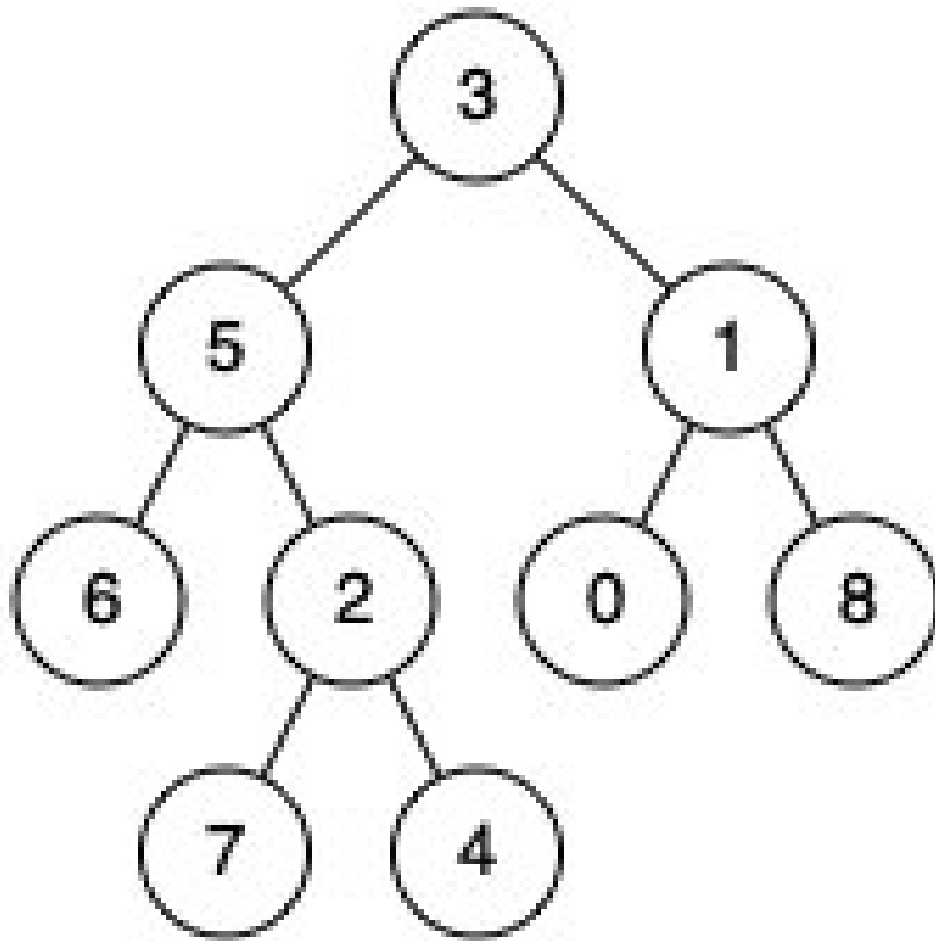


Input: root =
[3,5,1,6,2,0,8,null,null,7,4], p = 5, q
= 1

Output: 3

Explanation: The LCA of nodes 5 and 1
is 3.

Example 2:



Input: root =
[3,5,1,6,2,0,8,null,null,7,4], p = 5, q
= 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$
- All Node.val are unique.
- $p \neq q$
- p and q exist in the tree.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Nodes have a parent pointer. Find LCA without the root. Right?"

Both p and q are guaranteed to be in the tree."

#

"[3,5,1,...], p=5, q=1 → LCA=3 ✓ (same as #236 but solved via parent pointers)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Each node has a parent pointer. Walk both nodes up with a visited set until they meet.

First node seen twice is LCA. Like intersecting two linked lists.

Time: $O(h)$. Space: $O(h)$ visited set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (two-pointer linked list intersection)

"Walk p upward and q upward simultaneously. When one hits null, restart at the other's origin.

They meet at the LCA – same as finding intersection of two linked lists.

Each pointer traverses: (path from p to root) + (path from q to root), length $l_p + l_q$.

Both pointers traverse the same total length → they meet at LCA."

PHASE 4 — CLEAN CODE

```
class _1650_LCA_III:
```

```
    def lowestCommonAncestor(self, p:
'Node', q: 'Node') -> 'Node':
        a, b = p, q
```



```
while a is not b:
    a = a.parent if a else q
    b = b.parent if b else p
return a
```

PHASE 5 — TEST & DEBUG

p=5, q=1: a traverses 5→3→None→1→3. b traverses 1→3→None→5→3. Both reach 3 simultaneously ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(h)$. Space $O(1)$. This is the two-pointer intersection trick from [Linked List Intersection \(#160\)](#).

Both pointers traverse $\text{depth}(p) + \text{depth}(q)$ steps total and meet at LCA.

Given more time I'd ask: what if the tree is very deep and we want to avoid the full traversal?

Use a set: walk p to root recording all ancestors, then walk q to root and stop at first ancestor in the set.

$O(\text{depth})$ time, $O(\text{depth})$ space – same asymptotic but possibly faster in practice."

DFS

Round 2 · High · Medium · 16 questions

DFS

□ #130 Surrounded Regions

MED[Open on LeetCode](#)

Array · DFS · BFS · Union Find · Matrix

Lists: Denny Zhang

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- **Connect:** A cell is connected to adjacent cells horizontally or vertically.
- **Region:** To form a region connect every 'O' cell.
- **Surround:** A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

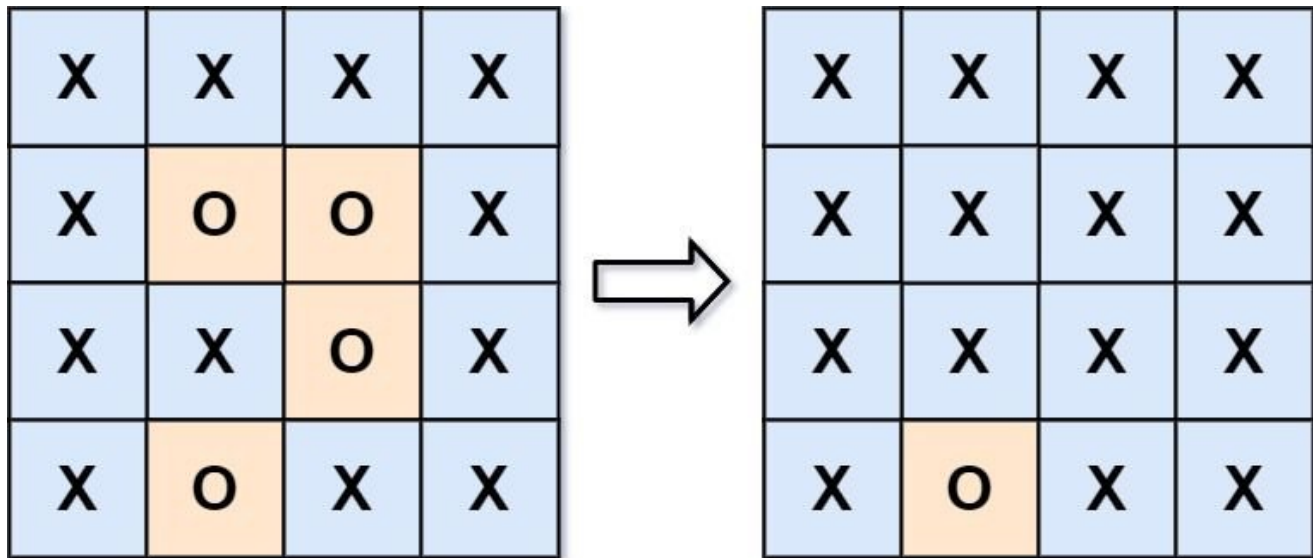
To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = `[["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]`

Output: `[["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]`

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = `[["X"]]`

Output: `[["X"]]`

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 200`

- `board[i][j]` is 'X' or 'O'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Capture surrounded 'O' regions by replacing them with 'X'.

A region is NOT captured if any 'O' in it touches the board edge. Right?

In-place modification – no return value."

#

"Border-connected 'O's and their inland 'O' neighbours are safe. Everything else gets flipped."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

First pass: DFS from every border 'O' and mark connected cells as safe ('S').

Second pass: flip remaining 'O' to 'X'; restore safe cells back to 'O'.

Two-pass flood fill from boundary – standard $O(m*n)$ approach.

```
# Time:  $O(m * n)$ . Space:  $O(m * n)$ 
recursion.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (reverse: mark safe, then flip)
# "DFS from every border '0', marking
connected '0's as safe ('#').
# Second pass: '0' → 'X' (captured), '#' →
'0' (safe restored).  $O(m*n)$ ."

# PHASE 4 — CLEAN CODE
class _130_SurroundedRegions:
    def solve(self, board:
List[List[str]]) -> None:
        m, n = len(board), len(board[0])
        def mark_safe(r, c):
            if r < 0 or r >= m or c < 0 or
c >= n or board[r][c] != '0':
                return
            board[r][c] = '#'
            for dr, dc in
[(-1,0), (1,0), (0,-1), (0,1)]:
                mark_safe(r + dr, c + dc)
        for r in range(m):
            for c in [0, n - 1]:
```

```
        mark_safe(r, c)
    for c in range(n):
        for r in [0, m - 1]:
            mark_safe(r, c)
    for r in range(m):
        for c in range(n):
            if board[r][c] == '0':
board[r][c] = 'X'
            elif board[r][c] == '#':
board[r][c] = '0'
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. The reverse-DFS from borders is the key insight —

instead of deciding which '0's are captured, decide which are SAFE and flip the rest.

Given more time I'd ask: what if the grid wraps (toroidal)? No borders → all '0's are safe."

DFS

#133 Clone Graph

MED[Open on LeetCode](#)

Hash Table · DFS · BFS · Graph

Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

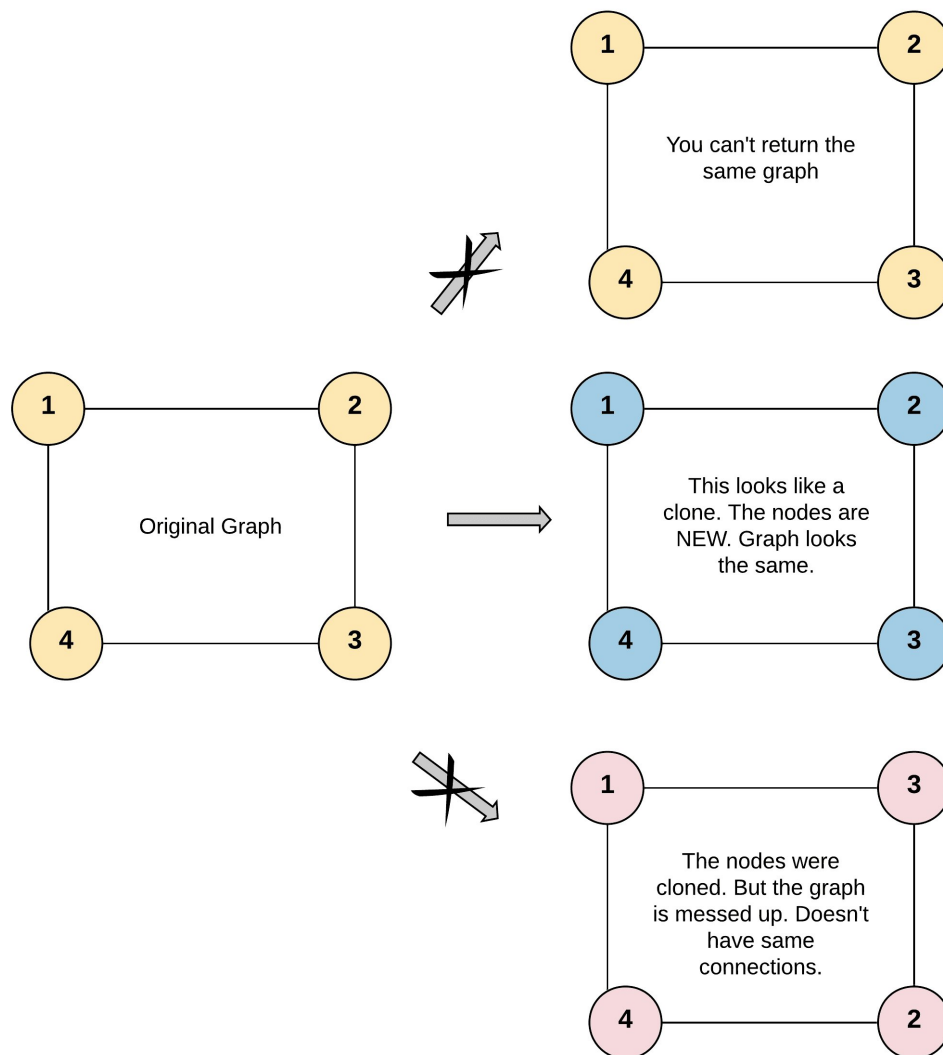
```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with $val = 1$. You must return the copy of the given node as a reference to the cloned graph.
Example 1:



Input: adjList =

[[2,4],[1,3],[2,4],[1,3]]

Output: [[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph.

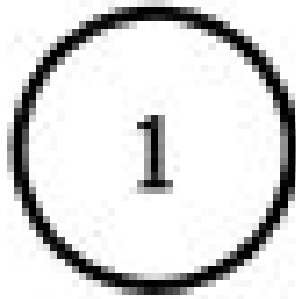
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:



Input: `adjList = [[]]`

Output: `[[]]`

Explanation: Note that the input contains one empty list. The graph consists of only one node with `val = 1` and it does not have any neighbors.

Example 3:

Input: `adjList = []`

Output: `[]`

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range `[0, 100]`.
- `1 <= Node.val <= 100`
- `Node.val` is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep copy an undirected connected graph. No shared references with original. Right?"

Graphs can have cycles – unlike trees, so we need a visited map."

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# BFS from the given node: maintain a map
old_node -> new_node clone.
# When visiting neighbors, create clones
on first sight and enqueue them.
# Straightforward graph copy – already the
standard  $O(V+E)$  approach.
# Time:  $O(V + E)$ . Space:  $O(V)$  for the
clone map and queue.
# Should I go ahead and optimize?"

class _133_CloneGraph:
    def cloneGraph(self, node: 'Node') ->
'Node':
        if not node:
            return None
        cloned: dict = {}
        def clone(n):
            if n in cloned:
                return cloned[n]
            copy = Node(n.val)
            cloned[n] = copy # mark before
recurring to handle cycles
            copy.neighbors = [clone(nb)
for nb in n.neighbors]
```

```
        return copy  
    return clone(node)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$ for the visited map. Mark node before recursing into neighbours —

without this, cycles cause infinite recursion. Same visited-map pattern handles any graph.

Given more time I'd ask: clone a directed graph? Same approach — just don't add reverse edges."

DFS

□ #200 Number of Islands

MED[Open on LeetCode](#)

Array · DFS · BFS · Union Find · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

Output: 1

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
```

Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 300$
- $\text{grid}[i][j]$ is '0' or '1'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."


```
# Double loop over every grid cell.  
Whenever I see an unvisited '1',  
# run DFS/BFS to mark the entire connected  
component sunk, then increment island  
count.  
# Each cell visited once – correct and  
actually  $O(m \times n)$  already.  
# Time:  $O(m \times n)$ . Space:  $O(m \times n)$   
worst-case recursion stack or queue.  
# The flood-fill is the right approach;  
I'll keep it clean with in-place marking."
```

```
class _200_NumberOfIslands:  
    def numIslands(self, grid:  
List[List[str]]) -> int:  
        m, n = len(grid), len(grid[0])  
        def dfs(r, c):  
            if r < 0 or r >= m or c < 0 or  
c >= n or grid[r][c] != '1':  
                return  
            grid[r][c] = '#' # mark  
visited by flooding  
            for dr, dc in  
[(-1,0),(1,0),(0,-1),(0,1)]:  
                dfs(r + dr, c + dc)  
        count = 0
```

```
for r in range(m):
    for c in range(n):
        if grid[r][c] == '1':
            dfs(r, c)
            count += 1
return count
```

PHASE 5 — TEST & DEBUG

1-island grid: DFS floods the whole island. count=1 ✓. 3-island: count=3 ✓.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. Flooding in-place avoids a separate visited set.

BFS alternative: use a queue instead of recursion – better for very large grids (no stack overflow).

Union-Find alternative: $O(m*n * \alpha(m*n))$ – useful when islands merge dynamically.

Given more time I'd ask: restore the grid after counting?

Keep a separate visited set or restore '#' → '1' after each DFS."

DFS

□ #207 Course Schedule

MED[Open on LeetCode](#)

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return `true` if you can finish all courses.

Otherwise, return `false`.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq 5000$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- All the pairs $\text{prerequisites}[i]$ are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can all courses be finished given prerequisites (directed edges)?

Equivalent to: does the directed graph have a cycle? Return True if no cycle. Right?"

#

"[[1,0],[0,1]]: 0→1 and 1→0 form a cycle → False ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build adjacency + indegree; Kahn BFS peels zero-indegree courses layer by layer.

If we finish fewer than numCourses nodes, a cycle exists ? return false.

Same topological sort as Course Schedule II, but only need boolean completion.

Time: $O(V + E)$. Space: $O(V + E)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (3-colour DFS cycle detection)

"Color each node: 0=unvisited, 1=in current DFS path (grey), 2=fully processed (black).

Cycle exists if we encounter a grey node. 0(V+E)."

PHASE 4 — CLEAN CODE

```
class _207_CourseSchedule:
    def canFinish(self, numCourses: int,
prerequisites: List[List[int]]) -> bool:
        graph = defaultdict(list)
        for course, pre in prerequisites:
            graph[pre].append(course)
        color = [0] * numCourses
        def has_cycle(node):
            if color[node] == 1: return
True # grey -> back edge -> cycle
            if color[node] == 2: return
False # black -> already processed
            color[node] = 1
            for nb in graph[node]:
                if has_cycle(nb): return
True
            color[node] = 2
```

```
        return False
    return not any(has_cycle(c) for c
in range(numCourses))

# PHASE 5 — TEST & DEBUG
# [[1,0],[0,1]]: has_cycle(0)→grey.
has_cycle(1)→grey.
has_cycle(0)→grey(already) → cycle! ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(V+E)$ . Space  $O(V+E)$ . The 3-colour
scheme distinguishes 'seen in current
path' from 'fully done'.
# Kahn's BFS alternative (in-degree
tracking): count = courses processed; if
count < numCourses → cycle.
# Given more time I'd ask: Course Schedule
II (#210)? Same DFS, collect reverse
post-order."
```

DFS

#210 Course Schedule II

MED[Open on LeetCode](#)

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: [0,1]

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- $a_i \neq b_i$
- All the pairs $[a_i, b_i]$ are distinct.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return a valid course ordering. Return [] if impossible (cycle exists). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Topological sort via Kahn's BFS: compute indegree of every course,

enqueue zero-indegree nodes, peel layers while decrementing neighbors.

If we process all n courses, ordering exists; otherwise a cycle remains.

Time: $O(V + E)$. Space: $O(V + E)$ for adjacency and indegree arrays.

Should I go ahead and optimize?"

```
class _210_CourseScheduleII:
```

```
    def findOrder(self, numCourses: int,
prerequisites: List[List[int]]) ->
```

```
List[int]:
```

```
    graph = defaultdict(list)
```

```
    for course, pre in prerequisites:
```

```
        graph[pre].append(course)
```

```
    color = [0] * numCourses
```

```
    order = []
```

```
    def dfs(node) -> bool:
```

```
        if color[node] == 1: return
```

```
False # cycle
```

```
        if color[node] == 2: return
```

```
True # already done
```

```
        color[node] = 1
```

```
        for nb in graph[node]:
```

```
            if not dfs(nb): return
```

```
False
```

```
        color[node] = 2
```

```
        order.append(node) #
```

```
post-order: all descendants before this
node
```

```
        return True
    for c in range(numCourses):
        if not dfs(c):
            return []
    return order[::-1] # reverse
post-order = topological order

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(V+E)$ . Space  $O(V+E)$ . Post-order
append then reverse = topological sort.
# Given more time I'd ask: Kahn's BFS?
Build in-degree map, process
zero-in-degree nodes first.
# Both approaches are  $O(V+E)$  – DFS is
recursive, Kahn's is iterative."
```

DFS

□ ★ #261 Graph Valid Tree ★

MED

[Open on LeetCode](#)

DFS · BFS · Union Find · Graph

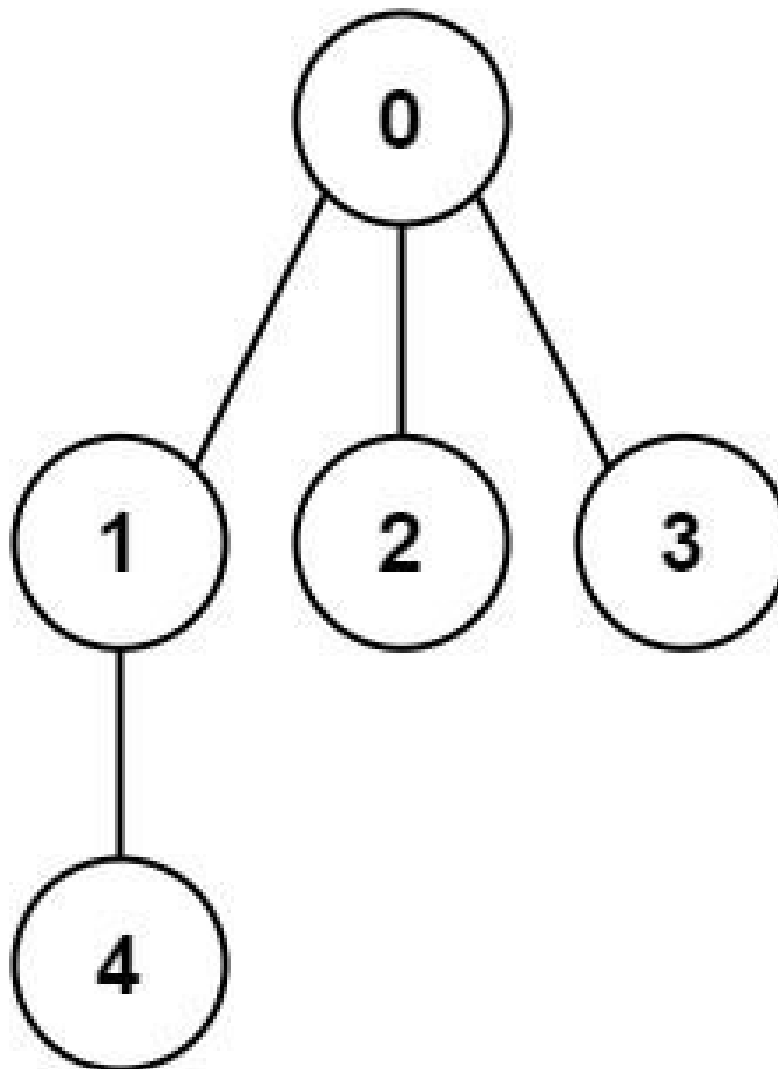
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

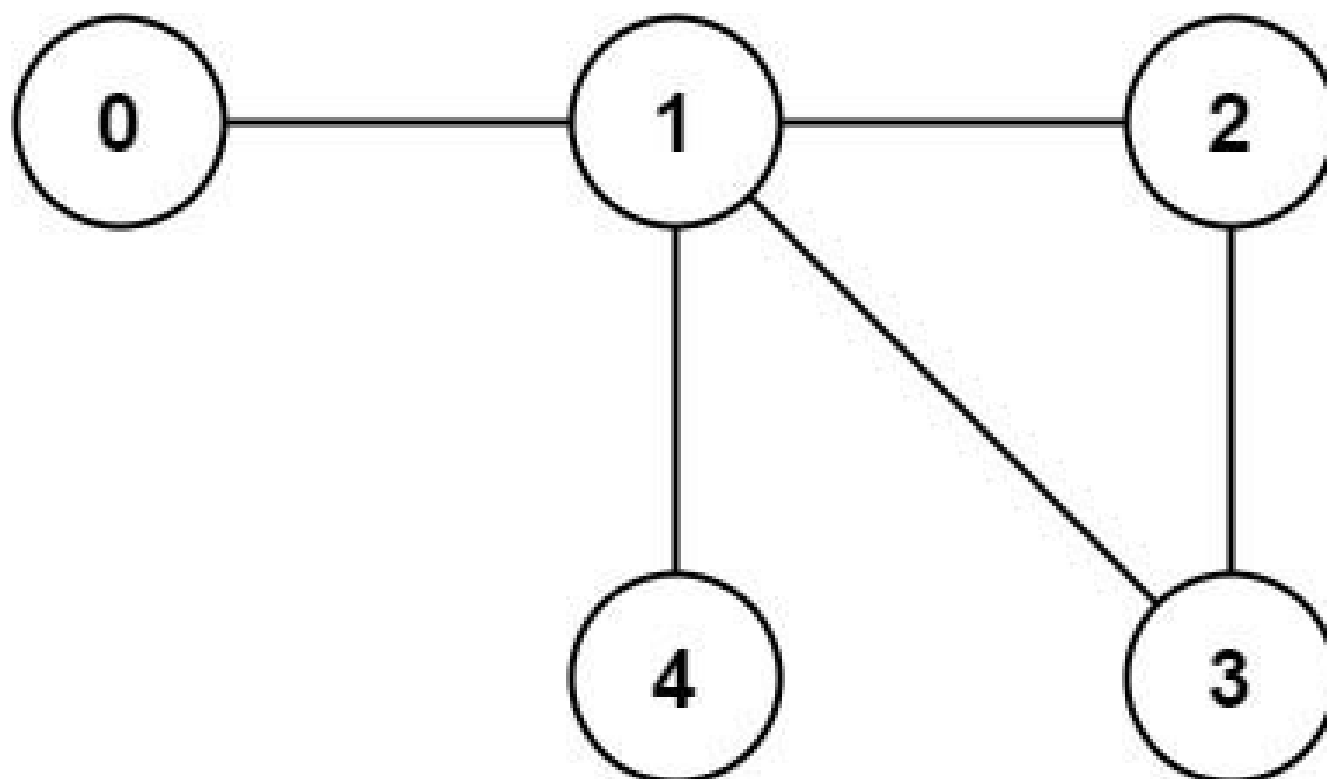
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:



Input: $n = 5$, $\text{edges} =$
 $[[0,1],[0,2],[0,3],[1,4]]$
Output: `true`

Example 2:



Input: $n = 5$, $edges =$
 $[[0,1],[1,2],[2,3],[1,3],[1,4]]$
Output: `false`

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Valid tree: connected, acyclic, exactly $n-1$ edges. Right?"

Any two of these imply the third for n nodes."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Build adjacency from edges, run DFS from node 0 marking visited.

After DFS, verify every node was visited and total edges equals $n-1$.

Catches disconnected graphs and cycles in one pass.

Time: $O(n)$. Space: $O(n)$ for adjacency + visited.

Should I go ahead and optimize?"

```
class _261_GraphValidTree:
    def validTree(self, n: int, edges:
List[List[int]]) -> bool:
        if len(edges) != n - 1:
            return False
        graph = defaultdict(list)
        for a, b in edges:
```



```

        graph[a].append(b)
        graph[b].append(a)
visited = set()
def dfs(node, parent):
    visited.add(node)
    for nb in graph[node]:
        if nb == parent: continue
        if nb in visited: return

```

False

```

        if not dfs(nb, node):
return False
        return True
    return dfs(0, -1) and
len(visited) == n

```

PHASE 5 — TEST & DEBUG

n=5, edges=[[0,1],[0,2],[0,3],[1,4]]: 4 edges==n-1 ✓. DFS from 0 visits all 5. No cycle. → True ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)=O(n)$. Space $O(n)$. Early return if edges $\neq$ n-1 is a fast filter.

Union-Find alternative: $O(n * \alpha(n))$ – merge edges, detect cycle if already connected.

Given more time I'd ask: disconnected forest (multiple trees)?
Return number of components instead – skip the `len(edges)==n-1` check."

DFS

□ #310 Minimum Height Trees

MED[Open on LeetCode](#)

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

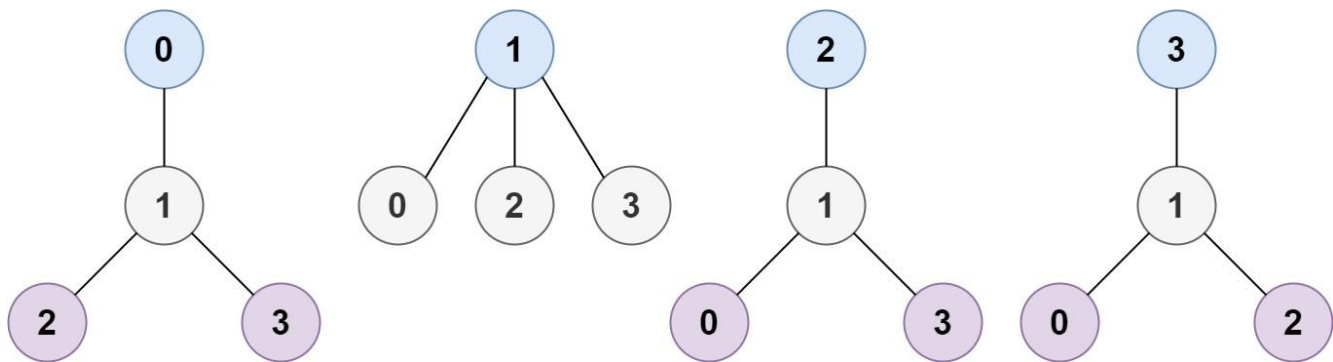
Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root.

When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:



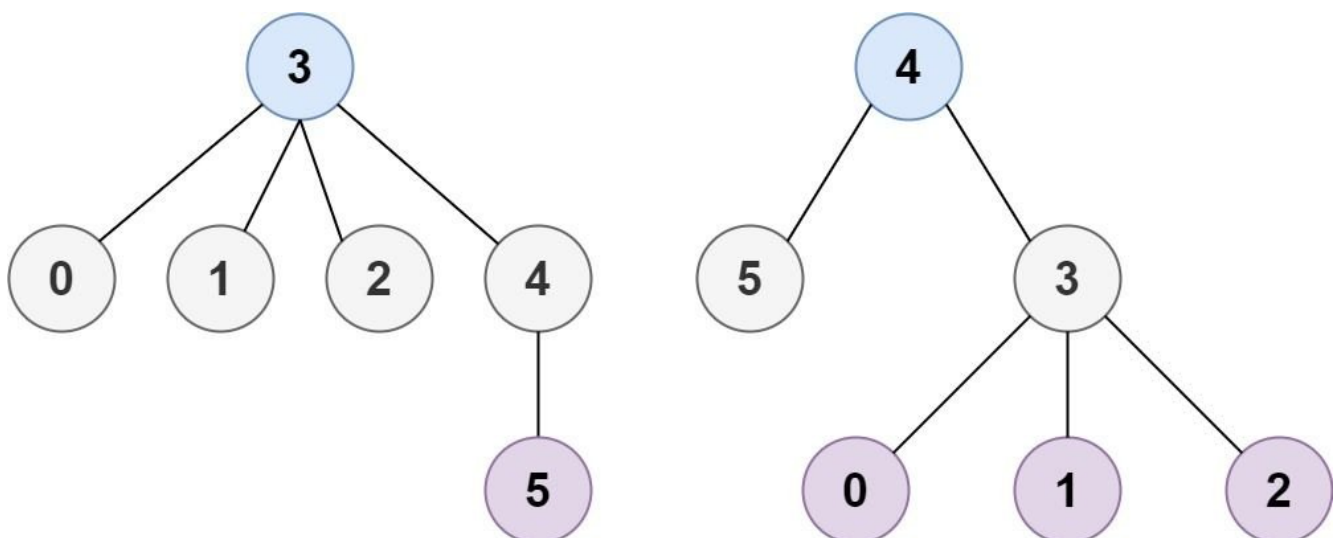
Input: $n = 4$, edges =

$[[1,0],[1,2],[1,3]]$

Output: [1]

Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.

Example 2:



Input: $n = 6$, $\text{edges} =$
 $[[3,0],[3,1],[3,2],[3,4],[5,4]]$
Output: $[3,4]$

Constraints:

- $1 \leq n \leq 2 * 10^4$
- $\text{edges.length} == n - 1$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find which nodes make MHT roots. The answer is at most 2 nodes – the tree's center(s). Right?"

Repeatedly prune leaf nodes (degree 1) until 1 or 2 nodes remain."

#

"Like peeling an onion from outside in – the core is the answer."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Repeatedly peel all leaves layer by layer (nodes with degree 1), like peeling an onion.

The last remaining nodes (0, 1, or 2) are the tree centroids / MHT roots.

BFS leaf-removal is the standard $O(n)$ approach for this problem.

Time: $O(n)$. Space: $O(n)$ for adjacency and degree counts.

Should I go ahead and optimize?"

```
class _310_MinHeightTrees:
    def findMinHeightTrees(self, n: int,
edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        graph = defaultdict(set)
        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)
        leaves = [node for node in
range(n) if len(graph[node]) == 1]
        remaining = n
```

```

while remaining > 2:
    remaining -= len(leaves)
    new_leaves = []
    for leaf in leaves:
        nb =
next(iter(graph[leaf]))
        graph[nb].remove(leaf)
        if len(graph[nb]) == 1:
            new_leaves.append(nb)
    leaves = new_leaves
return leaves

```

PHASE 5 — TEST & DEBUG

```

# n=4, edges=[[1,0],[1,2],[1,3]]:
leaves=[0,2,3]. remaining=4-3=1.
new_leaves=[1]. → [1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. This is topological sort on undirected tree – same BFS idea.

At most 2 centers exist – if more, tree would have multiple independent long paths.

Given more time I'd ask: all trees of minimum height (not just roots)?

BFS from all roots found, check height – same $O(n)$."

DFS

- ★ #323 Number of Connected Components in an Undirected Graph ★
[Open on LeetCode](#)
-

MED

DFS · BFS · Union Find · Graph

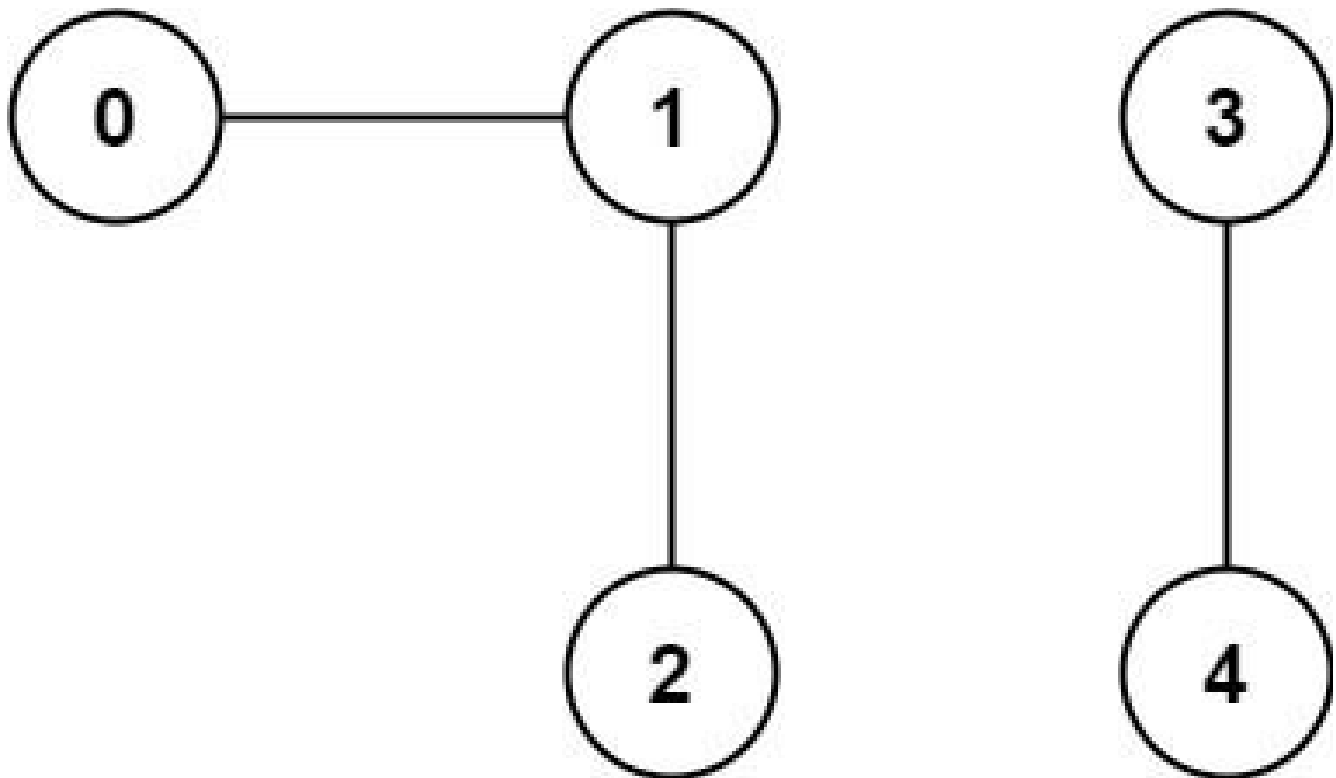
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

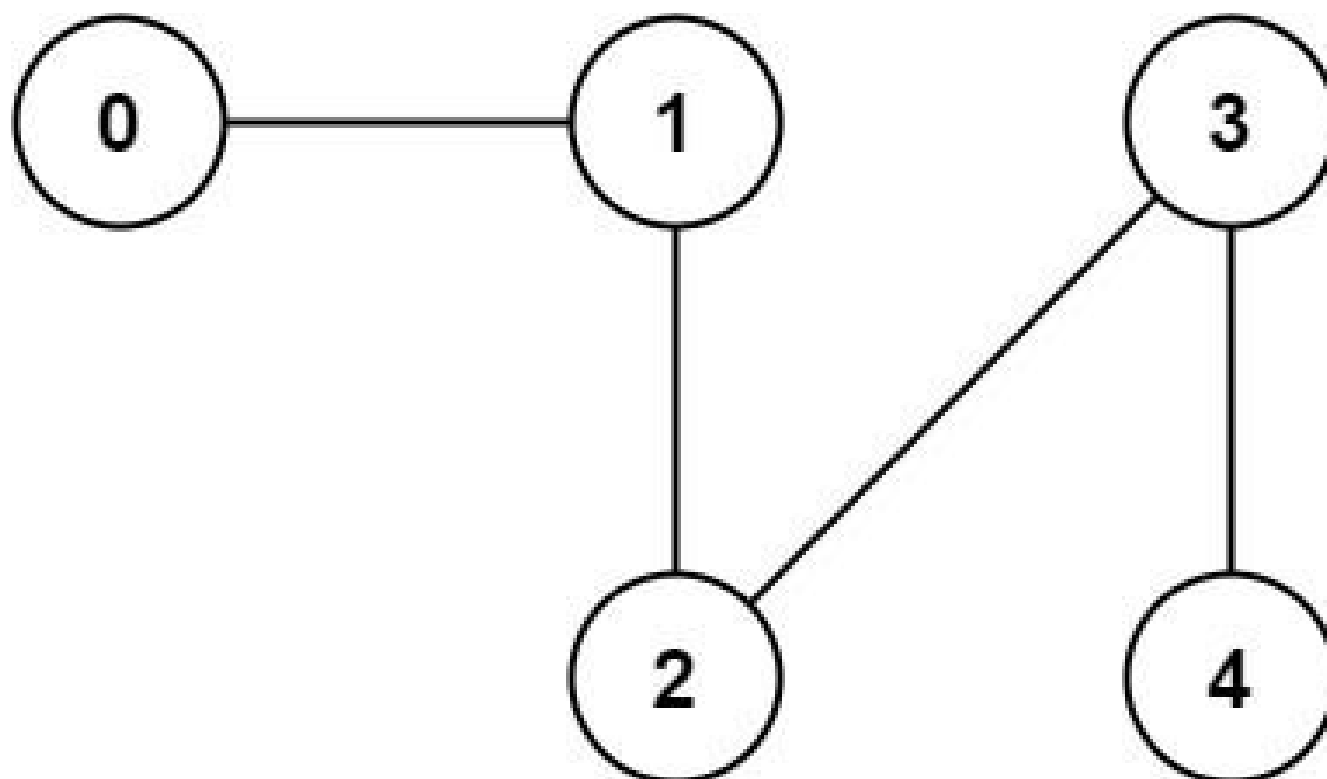
Return the number of connected components in the graph.

Example 1:



Input: $n = 5$, **edges** =
[[0,1],[1,2],[3,4]]
Output: 2

Example 2:



Input: $n = 5$, $\text{edges} =$
 $[[0,1],[1,2],[2,3],[3,4]]$
Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq \text{edges.length} \leq 5000$
- $\text{edges}[i].\text{length} == 2$
- $0 \leq a_i \leq b_i < n$
- $a_i \neq b_i$
- There are no repeated edges.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components in an undirected graph. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Union-Find: for each edge, union the two endpoints.

Answer = n minus number of distinct roots after all unions.

Simple DSU with path compression – $O(n \alpha(n))$.

Time: $O(n \alpha(n))$. Space: $O(n)$ for parent array.

Should I go ahead and optimize?"

class _323_NumberConnectedComponents:

def countComponents(self, n: int, edges: List[List[int]]) -> int:

graph = defaultdict(list)

for a, b in edges:

graph[a].append(b)

graph[b].append(a)

visited = set()

def dfs(node):

```
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                dfs(nb)

count = 0
for node in range(n):
    if node not in visited:
        dfs(node)
        count += 1
return count
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Standard DFS counting components.

Union-Find alternative: merge edges, count unique roots $\rightarrow O(n * \alpha(n))$.

Given more time I'd ask: what if edges are added dynamically?

Union-Find with path compression supports $O(\alpha(n))$ per merge query."

DFS

□ #417 Pacific Atlantic Water Flow

MED[Open on LeetCode](#)

Array · DFS · BFS · Matrix

Lists: Grind 169

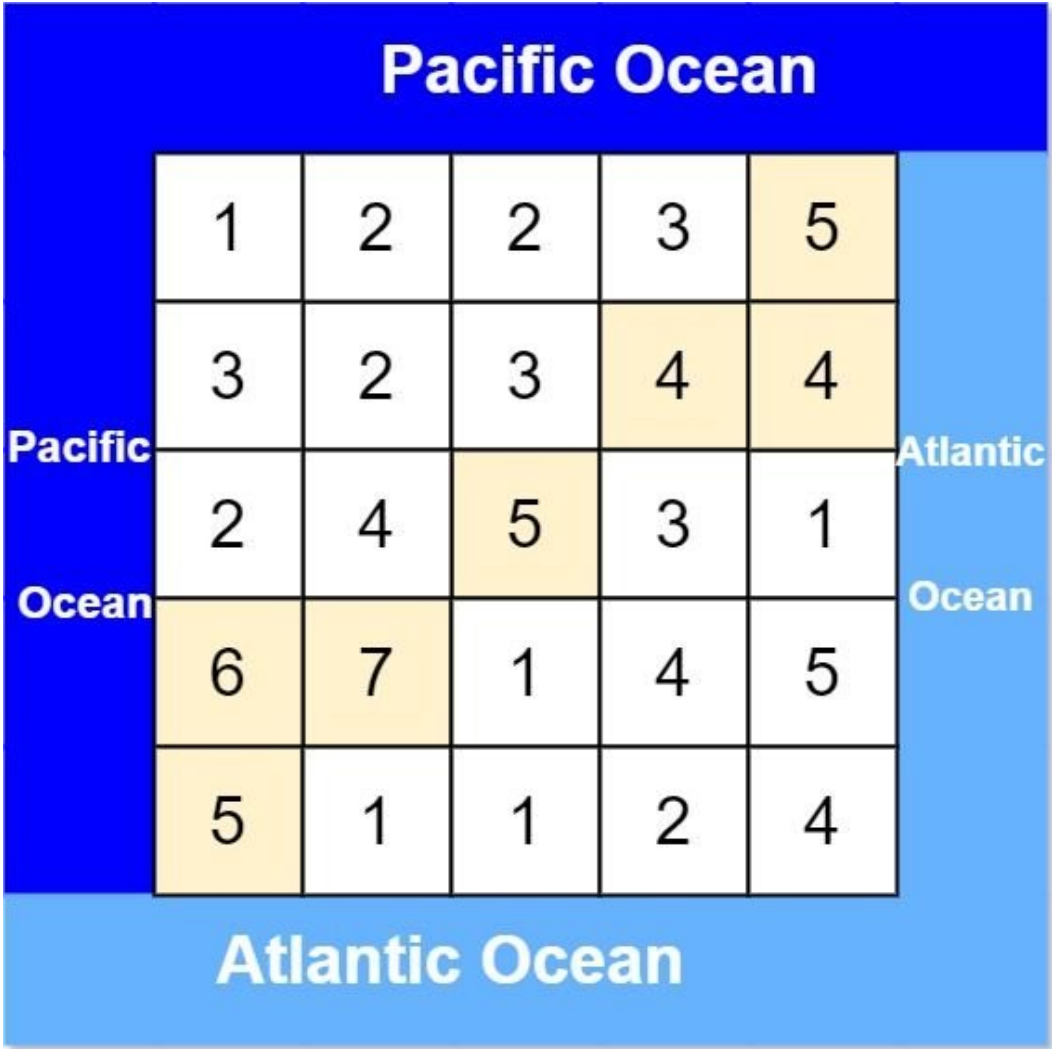
Problem

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the height above sea level of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where $\text{result}[i] = [\text{ri}, \text{ci}]$ denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:



Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]

Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]

Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:

[0,4]: [0,4] -> Pacific Ocean

[0,4] -> Atlantic Ocean

[1,3]: [1,3] -> [0,3] -> Pacific Ocean

[1,3] -> [1,4] -> Atlantic Ocean

[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean

Example 2:

Input: heights = [[1]]

Output: [[0,0]]

Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- m == heights.length
- n == heights[r].length
- 1 <= m, n <= 200

- $0 \leq \text{heights}[r][c] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Water flows from higher to lower (or equal). Find cells that can flow to BOTH oceans.

Pacific touches top/left edges. Atlantic touches bottom/right edges. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

From each cell, DFS to see if it can reach both Pacific (top/left border) and Atlantic (bottom/right).

Re-run DFS per cell – $O((m*n)^2)$ naive.

Time: $O((m * n)^2)$ naive per-cell DFS.

Space: $O(m * n)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (reverse DFS from ocean borders)

"Instead of simulating forward flow, run DFS BACKWARDS from each ocean's border cells.

```
# Reverse flow: go uphill (neighbor height  
>= current height).  
# Intersection of Pacific-reachable and  
Atlantic-reachable = answer."
```

PHASE 4 — CLEAN CODE

```
class _417_PacificAtlantic:  
    def pacificAtlantic(self, heights:  
List[List[int]]) -> List[List[int]]:  
        m, n = len(heights),  
len(heights[0])  
        def bfs(starts):  
            reachable = set(starts)  
            queue = deque(starts)  
            while queue:  
                r, c = queue.popleft()  
                for dr, dc in  
[(-1,0),(1,0),(0,-1),(0,1)]:  
                    nr, nc = r + dr, c +  
dc  
                    if 0 <= nr < m and 0  
<= nc < n and (nr,nc) not in reachable \  
and  
heights[nr][nc] >= heights[r][c]:  
                        reachable.add((nr  
, nc))
```

```
queue.append((nr,
nc))

return reachable

pacific = [(0, c) for c in
range(n)] + [(r, 0) for r in range(1, m)]
atlantic = [(m-1, c) for c in
range(n)] + [(r, n-1) for r in range(m-1)]
pac_reach = bfs(pacific)
atl_reach = bfs(atlantic)
return [[r, c] for r, c in
pac_reach & atl_reach]
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. BFS from borders is cleaner than DFS (no recursion limit).

The reversal insight: 'can reach ocean' → 'ocean can flow backward to this cell'.

Given more time I'd ask: what if water can also flow diagonally?

Add 4 diagonal directions – same algorithm."

DFS

□ ★ #490 The Maze ★

MED[Open on LeetCode](#)

Array · DFS · BFS · Matrix

Lists: ★ Premium | Premium 98

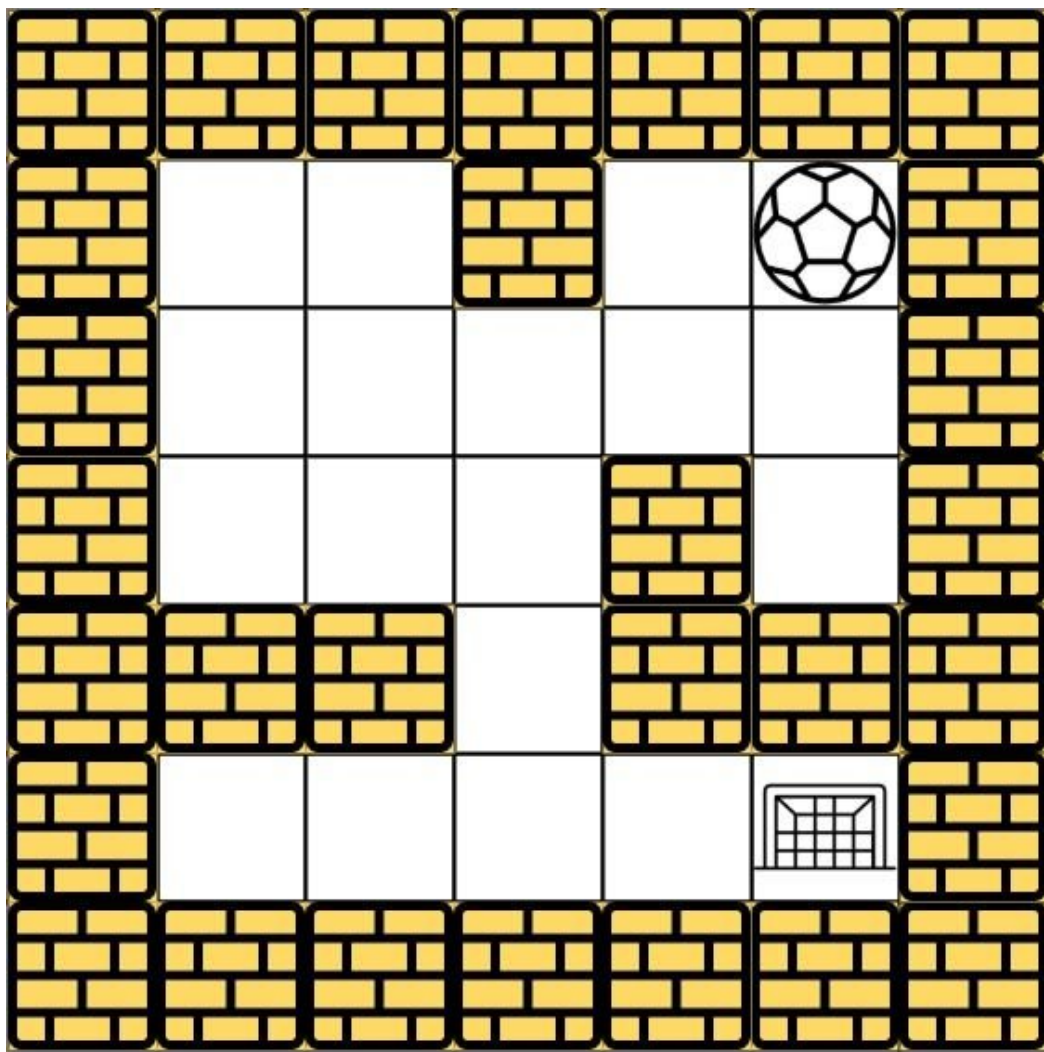
Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return true if the ball can stop at the destination, otherwise return false.

You may assume that the borders of the maze are all walls (see examples).

Example 1:

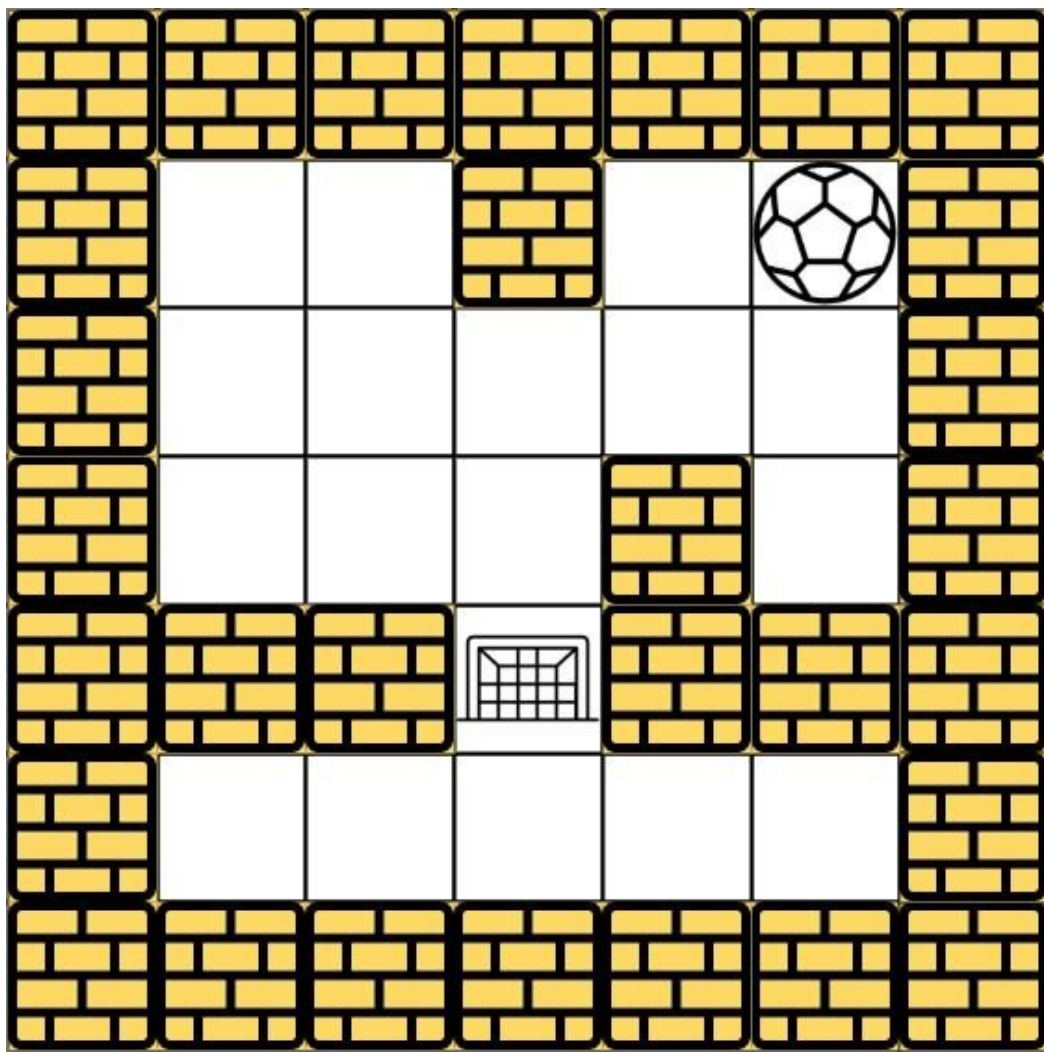


Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],`
`[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]],`
 start = `[0,4]`, destination = `[4,4]`

Output: true

Explanation: One possible way is : left
 -> down -> left -> down -> right ->
 down -> right.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],`
`[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]],`
 start = `[0,4]`, destination = `[3,2]`

Output: false

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],`
`[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],`
start = `[4,3]`, destination = `[0,1]`
Output: false

Constraints:

- `m == maze.length`
- `n == maze[i].length`
- `1 <= m, n <= 100`
- `maze[i][j]` is 0 or 1.
- `start.length == 2`
- `destination.length == 2`
- `0 <= startrow, destinationrow < m`
- `0 <= startcol, destinationcol < n`
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Ball rolls in a direction until hitting a wall. Can it STOP at the destination?"

```
# It can pass through the destination but  
not stop there unless it hits a wall.  
Right?"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# DFS from entrance: at each empty cell  
try all 4 directions.
```

```
# Mark visited cells to avoid cycles;  
succeed if we reach the boundary.
```

```
# Correct path exploration on an m x n  
grid.
```

```
# Time:  $O(m * n)$ . Space:  $O(m * n)$  visited.
```

```
# Should I go ahead and optimize?"
```

```
class _490_TheMaze:
```

```
    def hasPath(self, maze:  
List[List[int]], start: List[int],  
destination: List[int]) -> bool:  
        m, n = len(maze), len(maze[0])  
        dest = tuple(destination)  
        visited: set = set()
```

```
        def dfs(r, c):  
            if (r, c) in visited: return
```

```
False
```



```

        visited.add((r, c))
        if (r, c) == dest: return True
        for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r, c
            while 0 <= nr + dr < m and
0 <= nc + dc < n and maze[nr+dr][nc+dc] ==
0:
                nr += dr; nc += dc
                if dfs(nr, nc): return
True
        return False
return dfs(start[0], start[1])

```

PHASE 5 — TEST & DEBUG

Ball rolls until hitting wall → stops → that stop position is a new state.
DFS on stop-positions. Destination reached only if the ball stops exactly there. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n \cdot (m+n))$ – each stop position visits $O(m+n)$ cells to find next stop.
The visited set is on stop-positions, not individual cells.

**# Given more time I'd ask: The Maze II
□#505) – find shortest distance?
Use Dijkstra with distance as weight,
BFS/priority queue on stop-positions."**

DFS

□ ★ #694 Number of Distinct Islands ★

MED

[Open on LeetCode](#)

Hash Table · DFS · BFS · Union Find · Hash Function

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

Example 1:

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

Input: `grid = [[1,1,0,0,0],[1,1,0,0,0],`
`[0,0,0,1,1],[0,0,0,1,1]]`

Output: 1

Example 2:

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]

Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count islands that are distinct by shape (translation equivalent). Right?

Two islands are the same if one can be shifted to match the other – no rotation/reflection."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

DFS each island; encode shape as a string of moves (U/D/L/R) from start cell.

Add shape string to a set – distinct shapes = set size.

Must normalize starting cell and direction encoding consistently.

Time: $O(m * n)$. Space: $O(m * n)$ shapes.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (path signature hashing)

"DFS each island, record the path taken relative to the starting cell.

Include backtrack markers to distinguish shapes like 'L' vs reverse 'L'.

Hash each shape as a tuple and add to a set – count unique shapes."

PHASE 4 — CLEAN CODE

```
class _694_DistinctIslands:
    def numDistinctIslands(self, grid:
List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        shapes: set = set()
        def dfs(r, c, r0, c0, path):
            if r < 0 or r >= m or c < 0 or
c >= n or grid[r][c] != 1:
                return
            grid[r][c] = 0
            path.append((r - r0, c - c0))
            for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc, r0,
c0, path)
            path.append(None) # backtrack
marker
        for r in range(m):
            for c in range(n):
                if grid[r][c] == 1:
                    path: List = []
                    dfs(r, c, r, c, path)
```

```
        shapes.add(tuple(path
    ))

    return len(shapes)
```

PHASE 5 — TEST & DEBUG

Two 2×2 islands: both DFS in same order
→ same relative path → same tuple → 1
distinct ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Backtrack
None markers distinguish different DFS
orderings

that could produce the same sequence of
offsets for different shapes.

Given more time I'd ask: allow
rotation/reflection? Normalise shape to
lexicographically smallest

among all 8 transformations –
significantly more complex."

DFS

□ #695 Max Area of Island

MED[Open on LeetCode](#)

Array · DFS · BFS · Union Find · Matrix

Lists: Denny Zhang

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: `grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0], [0,0,0,0,0,0,0,1,1,1,0,0,0], [0,1,1,0,1,0,0,0,0,0,0,0,0], [0,1,0,0,1,1,0,0,1,0,0,1,0], [0,1,0,0,1,1,0,0,1,1,1,0,0], [0,0,0,0,0,0,0,0,0,0,0,1,0], [0,0,0,0,0,0,0,1,1,1,0,0,0], [0,0,0,0,0,0,0,0,1,1,0,0,0], [0,0]]`

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

```
Input: grid = [[0,0,0,0,0,0,0,0]]  
Output: 0
```

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Maximum area (cell count) of any connected island. Return 0 if no island. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For each land cell, DFS to measure area of its island while marking visited.

Track the maximum area seen across all components.

Same flood-fill as Number of Islands, but accumulate area instead of count.

Time: $O(m * n)$. Space: $O(m * n)$ stack worst case.

Should I go ahead and optimize?"

```
class _695_MaxAreaIsland:
    def maxAreaOfIsland(self, grid:
List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        def dfs(r, c) -> int:
            if r < 0 or r >= m or c < 0 or
c >= n or grid[r][c] != 1:
                return 0
            grid[r][c] = 0
            return 1 + dfs(r-1,c) +
dfs(r+1,c) + dfs(r,c-1) + dfs(r,c+1)
        return max((dfs(r, c) for r in
range(m) for c in range(n) if grid[r][c]
== 1), default=0)
```

PHASE 5 — TEST & DEBUG

Each DFS floods an island and returns its cell count. max() over all islands. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. The return-value DFS pattern: each cell contributes 1 + sum of children.

```
# This is the same DFS flood-fill as #200  
(Number of Islands) with a count return.  
# Given more time I'd ask: merge two  
islands by flipping a '0'?  
# Enumerate each '0' cell, BFS to find  
adjacent distinct islands, sum their areas  
+ 1."
```

DFS

□ #721 Accounts Merge

MED[Open on LeetCode](#)

Array · String · DFS · BFS · Union Find

Lists: Grind 169

Problem

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts.

Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each

account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1:

Input: `accounts = [{"John","johnsmith@mail.com"}, {"John","john_newyork@mail.com"}, {"John","johnsmith@mail.com"}, {"John00@mail.com"}, {"Mary","mary@mail.com"}, {"John","johnnybravo@mail.com"}]`

Output: `[{"John","john00@mail.com"}, {"John_newyork@mail.com"}, {"johnsmith@mail.com"}, {"Mary","mary@mail.com"}, {"John","johnnybravo@mail.com"}]`

Explanation:

The first and second John's are the same person as they have the common email "johnsmith@mail.com".

The third John and Mary are different people as none of their email addresses are used by other accounts.

We could return these lists in any order, for example the answer `[['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com'], 'john_newyork@mail.com', 'johnsmith@mail.com']]` would still be accepted.

Example 2:


```
Input: accounts = [ ["Gabe", "Gabe0@m.co",  
    "Gabe3@m.co", "Gabe1@m.co"], ["Kevin", "K  
evin3@m.co", "Kevin5@m.co", "Kevin0@m.co"  
], ["Ethan", "Ethan5@m.co", "Ethan4@m.co",  
    "Ethan0@m.co"], ["Hanzo", "Hanzo3@m.co", "  
Hanzo1@m.co", "Hanzo0@m.co"], ["Fern", "Fe  
rn5@m.co", "Fern1@m.co", "Fern0@m.co"] ]  
Output: [ ["Ethan", "Ethan0@m.co", "Ethan4  
@m.co", "Ethan5@m.co"], ["Gabe", "Gabe0@m.  
co", "Gabe1@m.co", "Gabe3@m.co"], ["Hanzo"  
    , "Hanzo0@m.co", "Hanzo1@m.co", "Hanzo3@m.  
co"], ["Kevin", "Kevin0@m.co", "Kevin3@m.c  
o", "Kevin5@m.co"], ["Fern", "Fern0@m.co",  
    "Fern1@m.co", "Fern5@m.co"] ]
```

Constraints:

- $1 \leq \text{accounts.length} \leq 1000$
- $2 \leq \text{accounts}[i].\text{length} \leq 10$
- $1 \leq \text{accounts}[i][j].\text{length} \leq 30$
- $\text{accounts}[i][0]$ consists of English letters.
- $\text{accounts}[i][j]$ (for $j > 0$) is a valid email.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge accounts that share at least one email. Same name $\neq$ same person.

Output: name + sorted merged emails. Any output order. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Union-Find on emails: union every email in the same account list.

Group emails by root parent; sort each merged list for output.

Time: $O(N \log N)$ for sorting emails.

Space: $O(N)$ DSU.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DFS on email graph)

"Build a graph where emails in the same account are connected.

DFS collects all emails in a connected component = one merged account.

Track owner (account name) via the first email in each account."

PHASE 4 — CLEAN CODE

class _721_AccountsMerge:

```
def accountsMerge(self, accounts:
List[List[str]]) -> List[List[str]]:
    email_to_name : dict = {}
    email_to_emails : defaultdict =
defaultdict(set)
    for account in accounts:
        name = account[0]
        first = account[1]
        for email in account[1:]:
            email_to_name[email] =
name
            email_to_emails[first].ad
d(email)
            email_to_emails[email].ad
d(first)
    visited: set = set()
    result = []
    def dfs(email, component):
        visited.add(email)
        component.append(email)
        for nb in
email_to_emails[email]:
            if nb not in visited:
                dfs(nb, component)
    for email in email_to_name:
```

```
        if email not in visited:
            component: List[str] = []
            dfs(email, component)
            result.append([email_to_name[email]] + sorted(component))
        return result
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(E \log E)$ where E = total emails (sort dominates). Space $O(E)$.

Union-Find alternative: union all emails in an account, group by root.

DFS on the email graph is cleaner to explain – same complexity.

Given more time I'd ask: if same email can belong to different names?

That would mean the data is inconsistent – flag it as an error."

DFS

□ #785 Is Graph Bipartite?

MED[Open on LeetCode](#)

DFS · BFS · Union Find · Graph

Lists: Denny Zhang

Problem

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

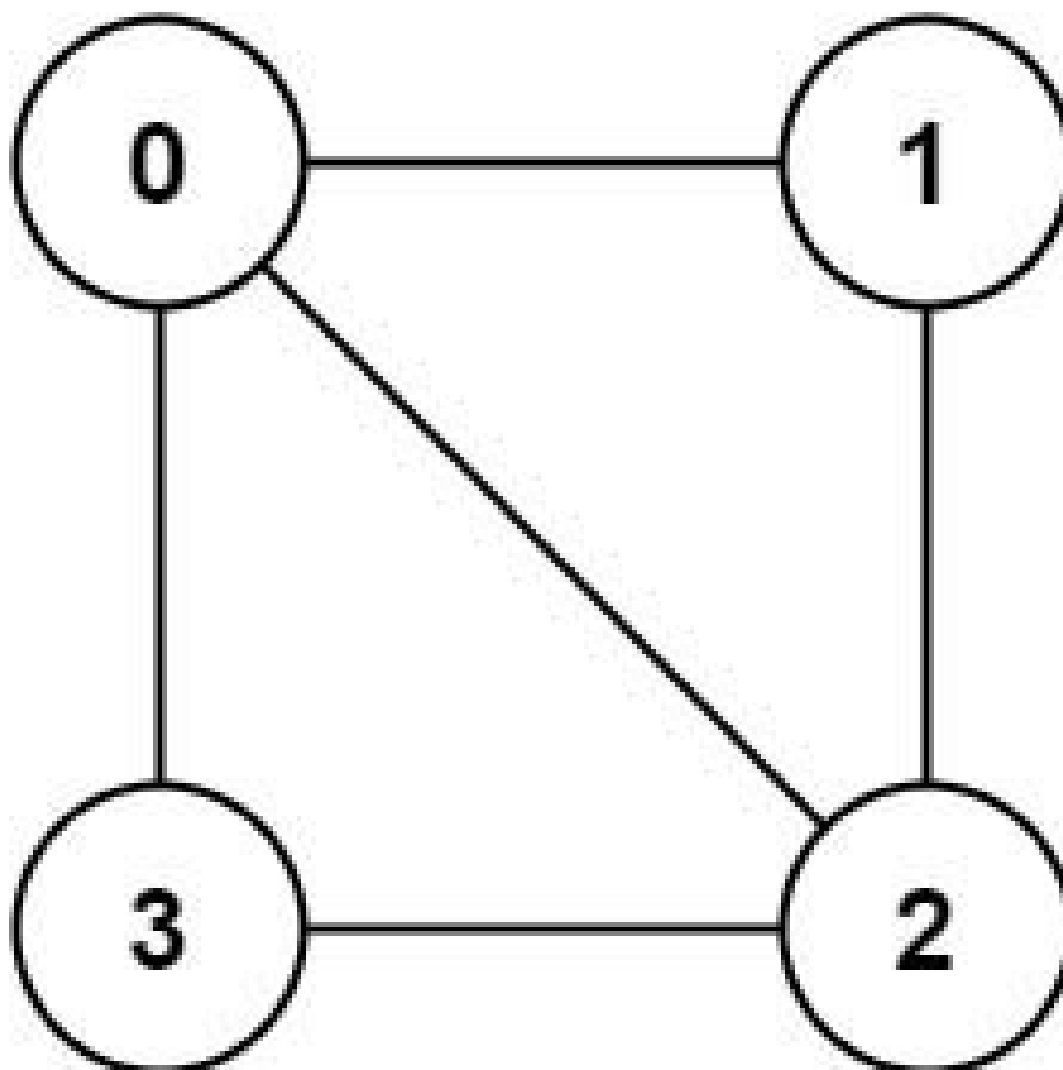
- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that

there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B.

Return true if and only if it is bipartite.

Example 1:



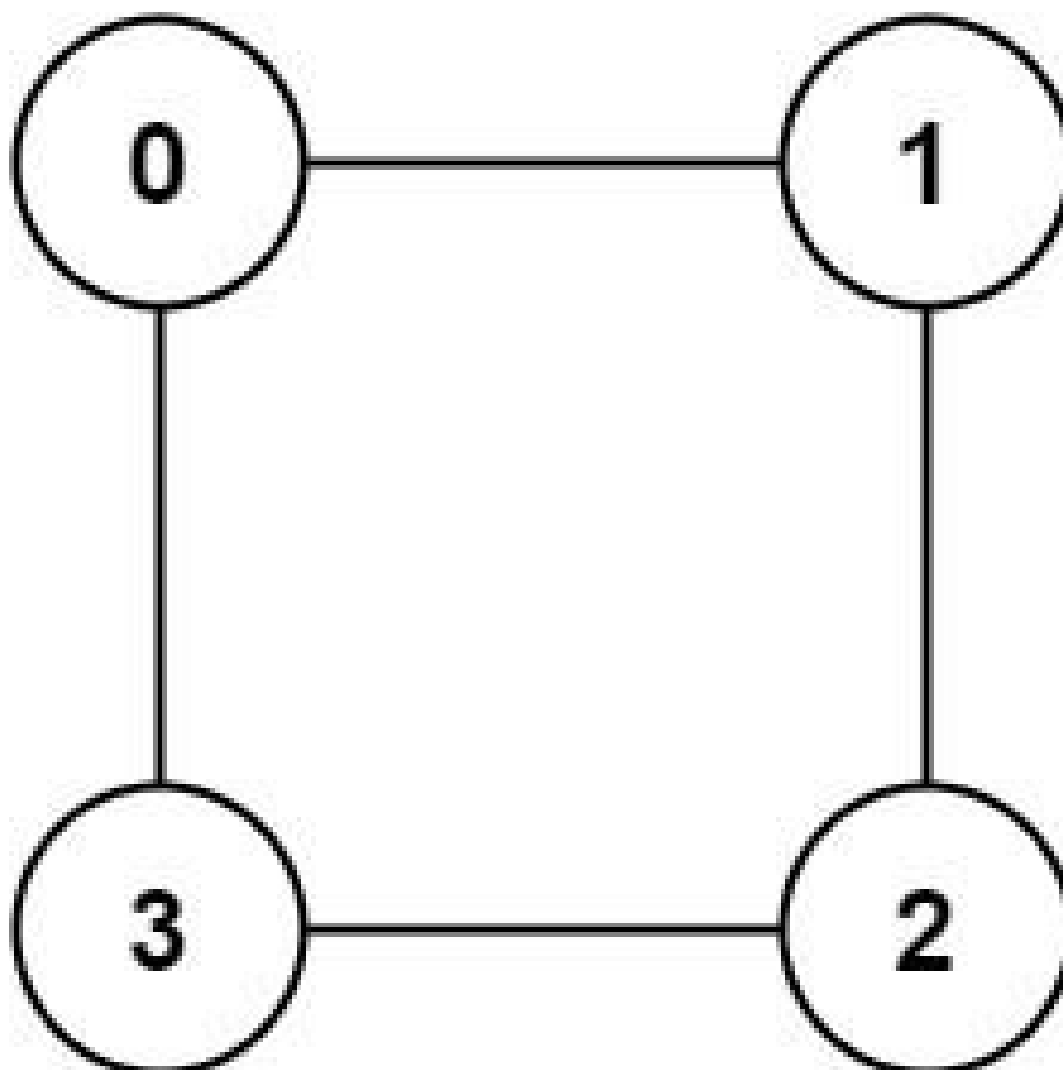
Input: graph =

[[1,2,3],[0,2],[0,1,3],[0,2]]

Output: false

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2:



Input: graph =

[[1,3],[0,2],[1,3],[0,2]]

Output: true

Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.

Constraints:

- **graph.length == n**
- **1 <= n <= 100**
- **0 <= graph[u].length < n**
- **0 <= graph[u][i] <= n - 1**
- **graph[u] does not contain u.**
- **All the values of graph[u] are unique.**
- **If graph[u] contains v, then graph[v] contains u.**

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can we 2-colour the graph so no two adjacent nodes share the same colour?"

Graph may be disconnected – check every component. Right?"

PHASE 2 — BRUTE FORCE


```
# "Let me start with brute force to lock
in the logic.
# Try to 2-color the graph with BFS/DFS:
assign color 0/1 alternating across edges.
# If any edge connects two nodes of the
same color, graph is not bipartite.
# Standard coloring check on an undirected
graph.
# Time:  $O(V + E)$ . Space:  $O(V)$  color +
visited.
# Should I go ahead and optimize?"

class _785_IsGraphBipartite:
    def isBipartite(self, graph:
List[List[int]]) -> bool:
        color = {}
        def bfs(start):
            queue = deque([start])
            color[start] = 0
            while queue:
                node = queue.popleft()
                for nb in graph[node]:
                    if nb not in color:
                        color[nb] = 1 -
color[node]
                        queue.append(nb)
```

```
elif color[nb] ==  
color[node]:  
    return False  
    return True  
    return all(bfs(n) for n in  
range(len(graph)) if n not in color)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$. 2-coloring via
BFS — odd-length cycle → not bipartite.

Given more time I'd ask: what if we need
to output the two sets?

Return {n for n in color if color[n]==0}
and {n for n in color if color[n]==1}."

DFS

□ ★ #1236 Web Crawler ★

MED[Open on LeetCode](#)

String · DFS · BFS · Interactive

Lists: ★ Premium | Premium 98

Problem

Given a url `startUrl` and an interface `HtmlParser`, implement a web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all urls obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

`http://example.org:8888/foo/bar#bang`

`hostname`

`host`

As shown in the example url above, the hostname is example.org. For simplicity sake, you may assume all urls use http protocol without any port specified. For example, the urls `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while urls `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

The HtmlParser interface is defined as such:

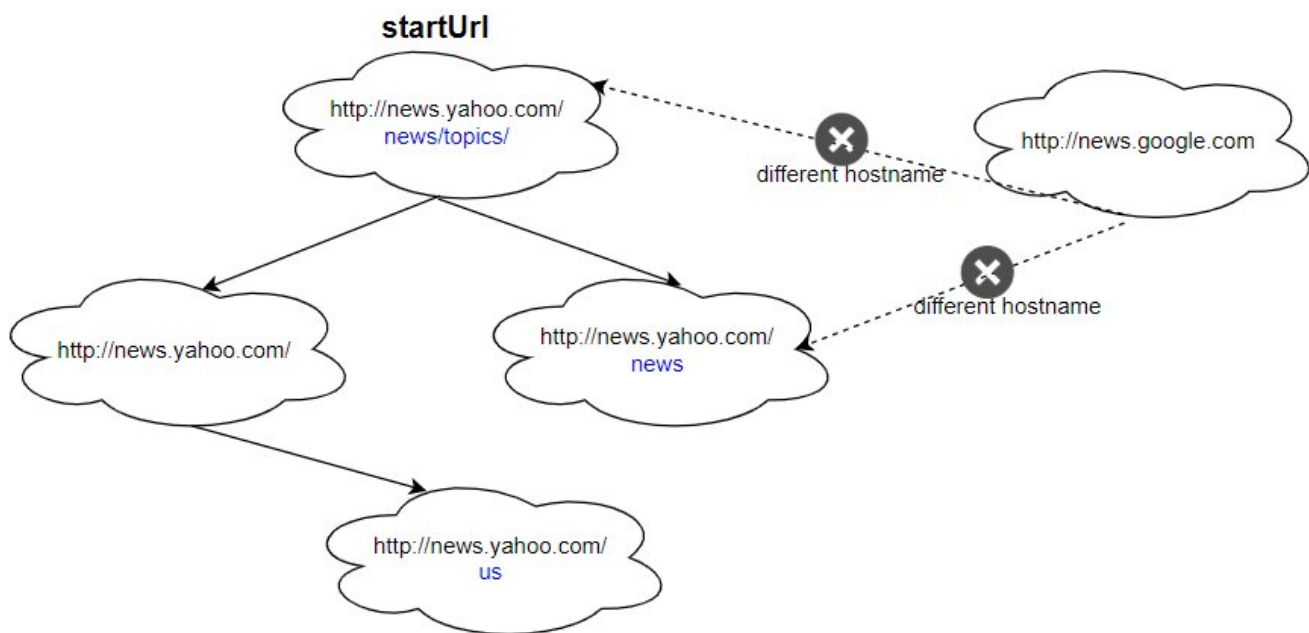
```
interface HtmlParser {  
    // Return a list of all urls from a  
    // webpage of given url.  
    public List<String> getUrls(String  
        url);  
}
```

Below are two examples explaining the functionality of the problem, for custom testing

purposes you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly accessible to you in code.

Note: Consider the same URL with the trailing slash `"/"` as a different URL. For example, `"http://news.yahoo.com"`, and `"http://news.yahoo.com/"` are different urls.

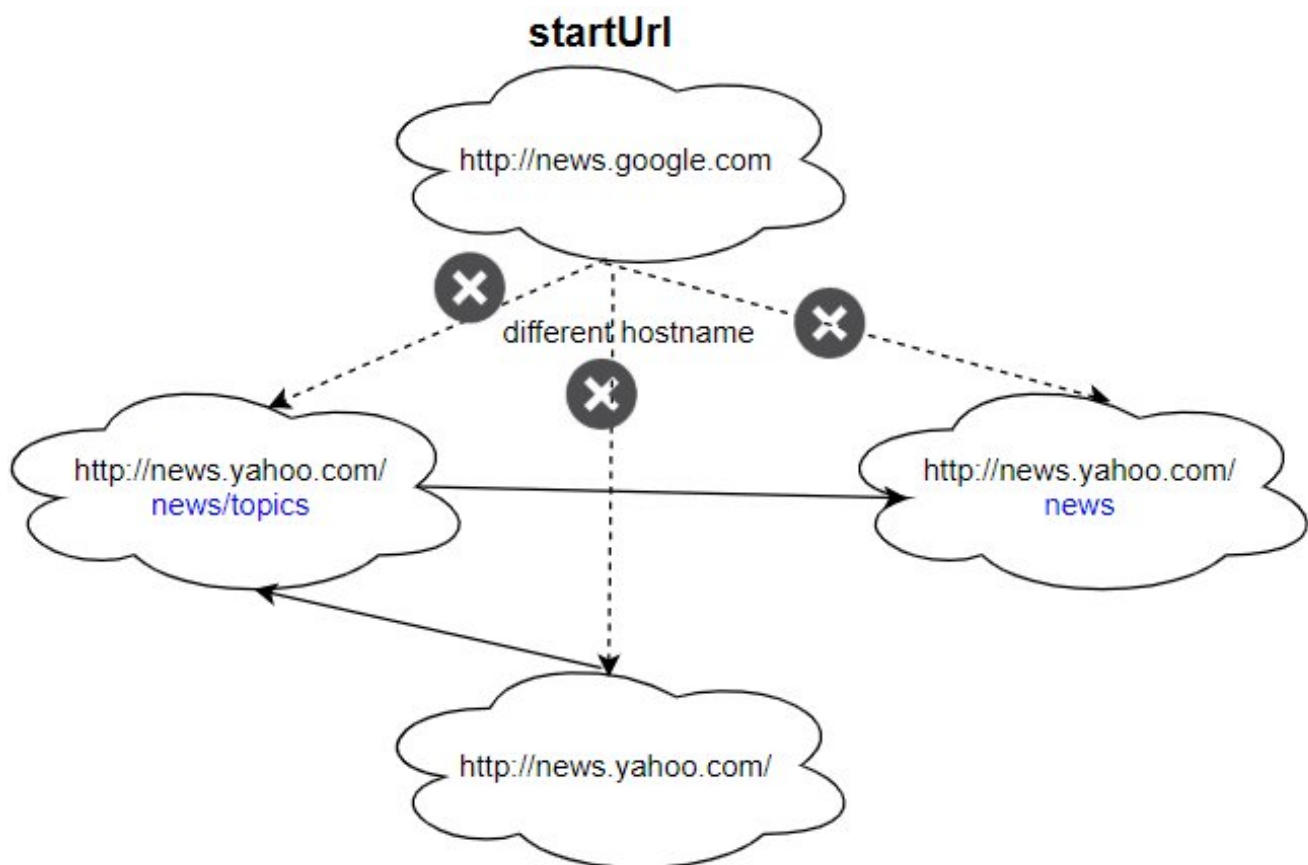
Example 1:



Input:

```
urls = [  
    "http://news.yahoo.com",  
    "http://news.yahoo.com/news",  
    "http://news.yahoo.com/news/topics/",  
    "http://news.google.com",  
    "http://news.yahoo.com/us"  
]
```

Example 2:



Input:

```
urls = [  
    "http://news.yahoo.com",  
    "http://news.yahoo.com/news",  
    "http://news.yahoo.com/news/topics/",  
    "http://news.google.com"  
]  
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in url library.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"BFS from startUrl. Only follow links on the same hostname. Don't revisit. Right?"

Hostname = part between '//' and the next '/'."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

BFS from startUrl: fetch page, parse all links on same hostname, enqueue unvisited.

Use a visited set keyed by URL string to avoid cycles.

Standard single-threaded web crawler simulation.

Time: $O(V + E)$ pages + links. Space: $O(V)$ visited.

Should I go ahead and optimize?"

```
class _1236_WebCrawler:
```

```
    def crawl(self, startUrl: str,
htmlParser: 'HtmlParser') -> List[str]:
        def hostname(url):
            return url.split('/')[2] #
'http:' , '' , 'news.yahoo.com', ...
```



```
host = hostname(startUrl)
visited = {startUrl}
queue = deque([startUrl])
while queue:
    url = queue.popleft()
    for link in
htmlParser.getUrls(url):
    if link not in visited and
hostname(link) == host:
        visited.add(link)
        queue.append(link)
return list(visited)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

**# "Time $O(V+E)$ where V =pages, E =links.
Space $O(V)$.**

Standard BFS graph traversal – the only novelty is hostname filtering.

Given more time I'd ask: [#1242](#)

**Multithreaded? Use a ThreadPoolExecutor,
submit getUrls calls as futures, use a thread-safe visited set."**

DFS

#1242 Web Crawler Multithreaded

MED[Open on LeetCode](#)

DFS · BFS · Concurrency**Lists: Denny Zhang**

Problem

Given a URL `startUrl` and an interface `HtmlParser`, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

`http://example.org:8888/foo/bar#bang`

`hostname`

`host`

As shown in the example URL above, the hostname is `example.org`. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while URLs `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

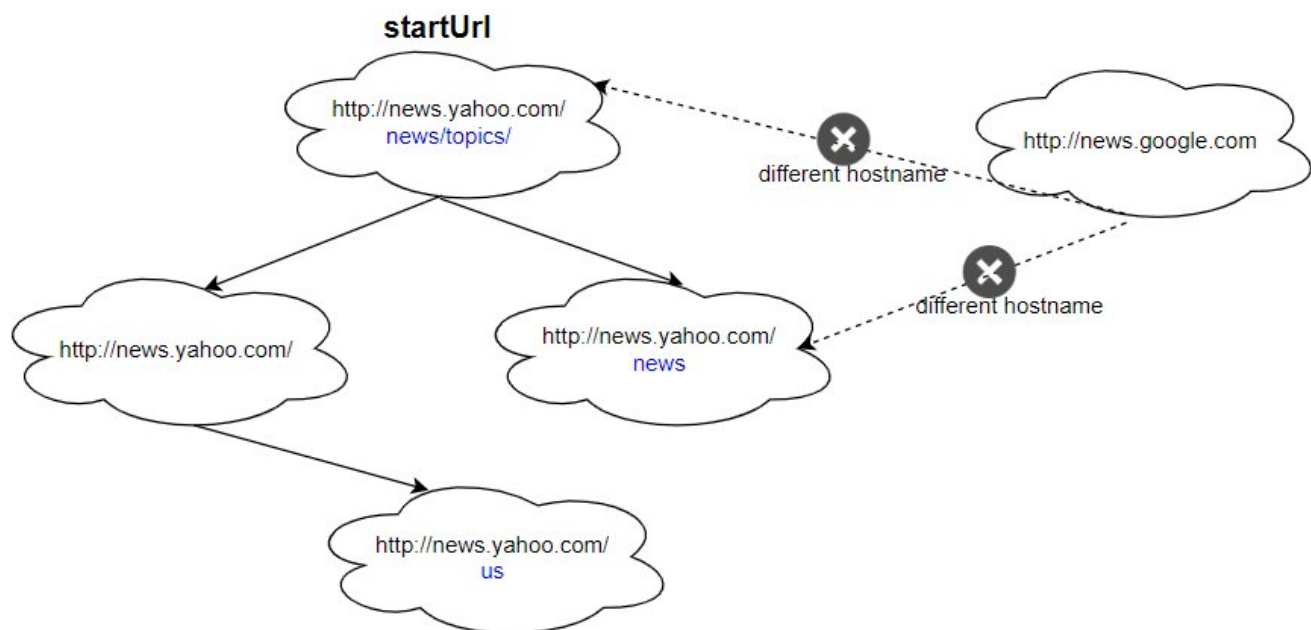
The `HtmlParser` interface is defined as such:

```
interface HtmlParser {  
    // Return a list of all urls from a  
    // webpage of given url.  
    // This is a blocking call, that means  
    // it will do HTTP request and return when  
    // this request is finished.  
    public List<String> getUrls(String  
        url);  
}
```

Note that `getUrls(String url)` simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that `getUrls(String url)` will return the URLs within 15ms. Single-threaded solutions will exceed the time limit so, can your multi-threaded web crawler do better?

Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly accessible to you in code.

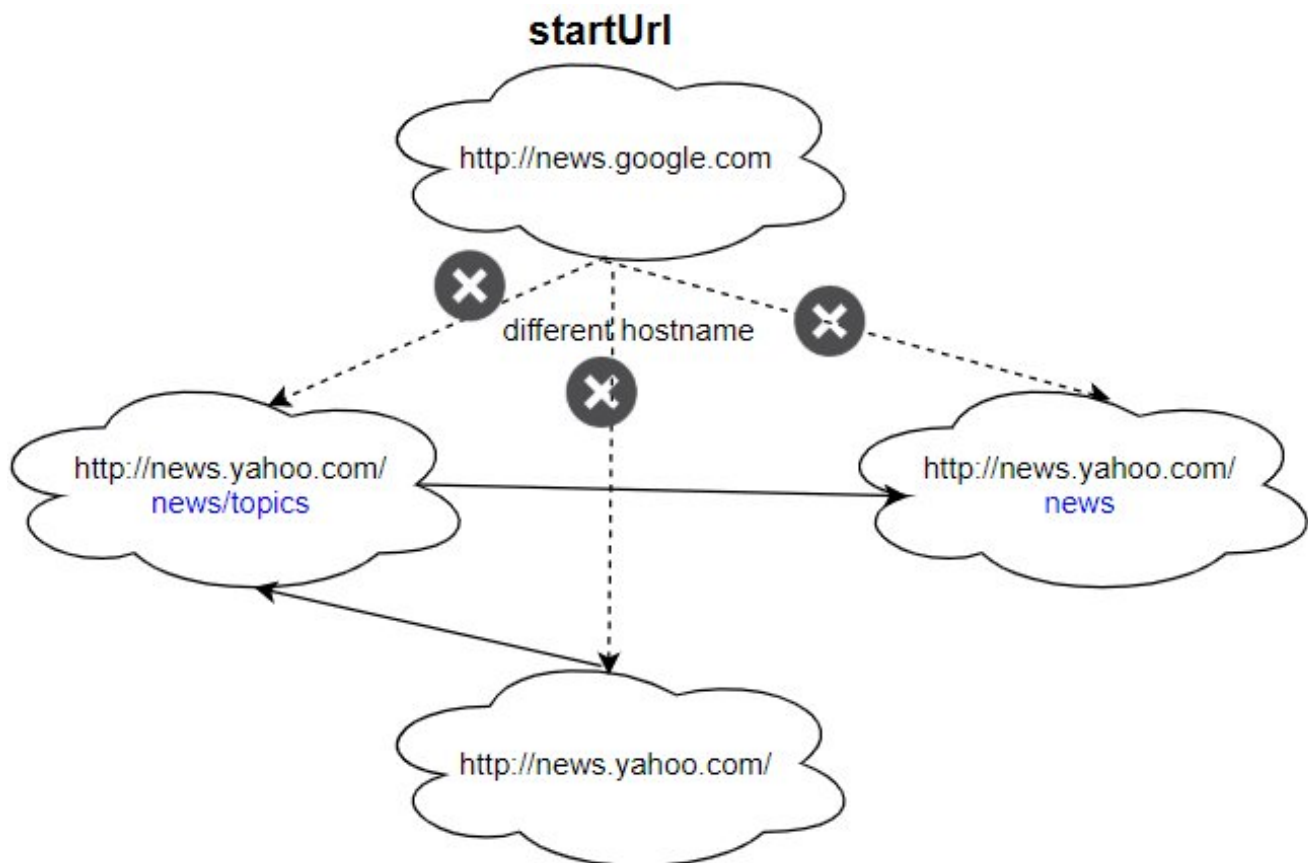
Example 1:



Input:

```
urls = [  
    "http://news.yahoo.com",  
    "http://news.yahoo.com/news",  
    "http://news.yahoo.com/news/topics/",  
    "http://news.google.com",  
    "http://news.yahoo.com/us"  
]
```

Example 2:



Input:

```
urls = [  
    "http://news.yahoo.com",  
    "http://news.yahoo.com/news",  
    "http://news.yahoo.com/news/topics/",  
    "http://news.google.com"  
]  
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.

Follow up:

1. Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?
2. What if one node fails or does not work?
3. How do you know when the crawler is done?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Same as #1236 but with multi-threading since getUrls is blocking and slow.

Use threads to fetch pages in parallel. Thread-safe visited set required. Right?"

PHASE 4 — CLEAN CODE (threading approach)

```
from concurrent.futures import  
ThreadPoolExecutor, as_completed  
import threading
```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Same BFS crawler as single-threaded, but dispatch each getLinks call to a worker thread.

Shared visited set must be synchronized; join threads when queue is empty.

Correct parallelism model for the LeetCode concurrency variant.

Time: $O(V + E)$ with parallel fetch.

Space: $O(V)$ visited + thread overhead.

Should I go ahead and optimize?"

```
class _1242_WebCrawlerMT:
```

```
    def crawl(self, startUrl: str,  
htmlParser: 'HtmlParser') -> List[str]:
```

```
        def hostname(url):  
            return url.split('/')[2]
```

```
        host = hostname(startUrl)
```

```
        visited = {startUrl}
```

```
        lock = threading.Lock()
```

```
        def fetch(url):
```

```
            neighbors = []
```

```
            for link in
```

```
htmlParser.getUrls(url):
```

```
                with lock:
```



```
        if link not in
visited and hostname(link) == host:
            visited.add(link)
            neighbors.append(
link)

    return neighbors
queue = [startUrl]
with ThreadPoolExecutor() as
executor:
    while queue:
        futures =
{executor.submit(fetch, url): url for url
in queue}

        queue = []
        for f in
as_completed(futures):
            queue.extend(f.result
())

    return list(visited)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Threads cut wall-clock time

proportional to how much getUrls blocks.

**# Lock only needed when checking/adding to
visited – minimize critical section.**

Distributed follow-up (10,000 nodes, 1B URLs): consistent hashing to partition URLs by hostname.

Each node owns a hostname range.

Communicate frontier pages between nodes.

'Done' signal: all queues empty and all workers idle (barrier synchronisation)."

Graphs

Round 2 · High · Medium · 3 questions

Graphs

□ #128 Longest Consecutive Sequence

MED[Open on LeetCode](#)

Array · Hash Table · Union Find

Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is `[1, 2, 3, 4]`. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

Example 3:

Input: `nums = [1,0,1,2]`

Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest run of consecutive integers. Must be $O(n)$. Return the length. Right?"

Duplicates don't extend the sequence – `[1,1,2] → length 2.`"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Put all numbers in a hash set. For each num, if `num-1` is not in set, walk num, `num+1`, `num+2`...

counting streak length. Track longest streak.

Each number visited once across all walks – $O(n)$ with the set.

Time: $O(n)$. Space: $O(n)$ set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (hash set + only start from sequence beginnings)

"Build a set. For each x where $x-1$ is NOT in the set (x is a sequence start),

count upward. Each element is processed at most once total $\rightarrow O(n)$."

PHASE 4 — CLEAN CODE

```
class _128_LongestConsecutive:
```

```
    def longestConsecutive(self, nums: List[int]) -> int:
```

```
        num_set = set(nums)
```

```
        best = 0
```

```
        for x in num_set:
```

```
            if x - 1 not in num_set: # x is a sequence start
```

```
                length = 1
```

```
                while x + 1 in num_set:
```

```
                    x += 1; length += 1
```

```
                best = max(best, length)
```

```
        return best
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Each number is the starting point of at most one sequence scan.

The 'start of sequence' check prevents redundant scans — the key insight.

Given more time I'd ask: streaming input? Use a dict mapping each endpoint of a range to its length, merging ranges as new numbers arrive."

Graphs

□ ★ #277 Find the Celebrity ★

MED

[Open on LeetCode](#)

Two Pointers · Graph · Interactive

Lists: ★ Premium | Premium 98

Problem

Suppose you are at a party with n people labeled from 0 to $n - 1$ and among them, there may exist one celebrity. The definition of a celebrity is that all the other $n - 1$ people know the celebrity, but the celebrity does not know any of them.

Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).

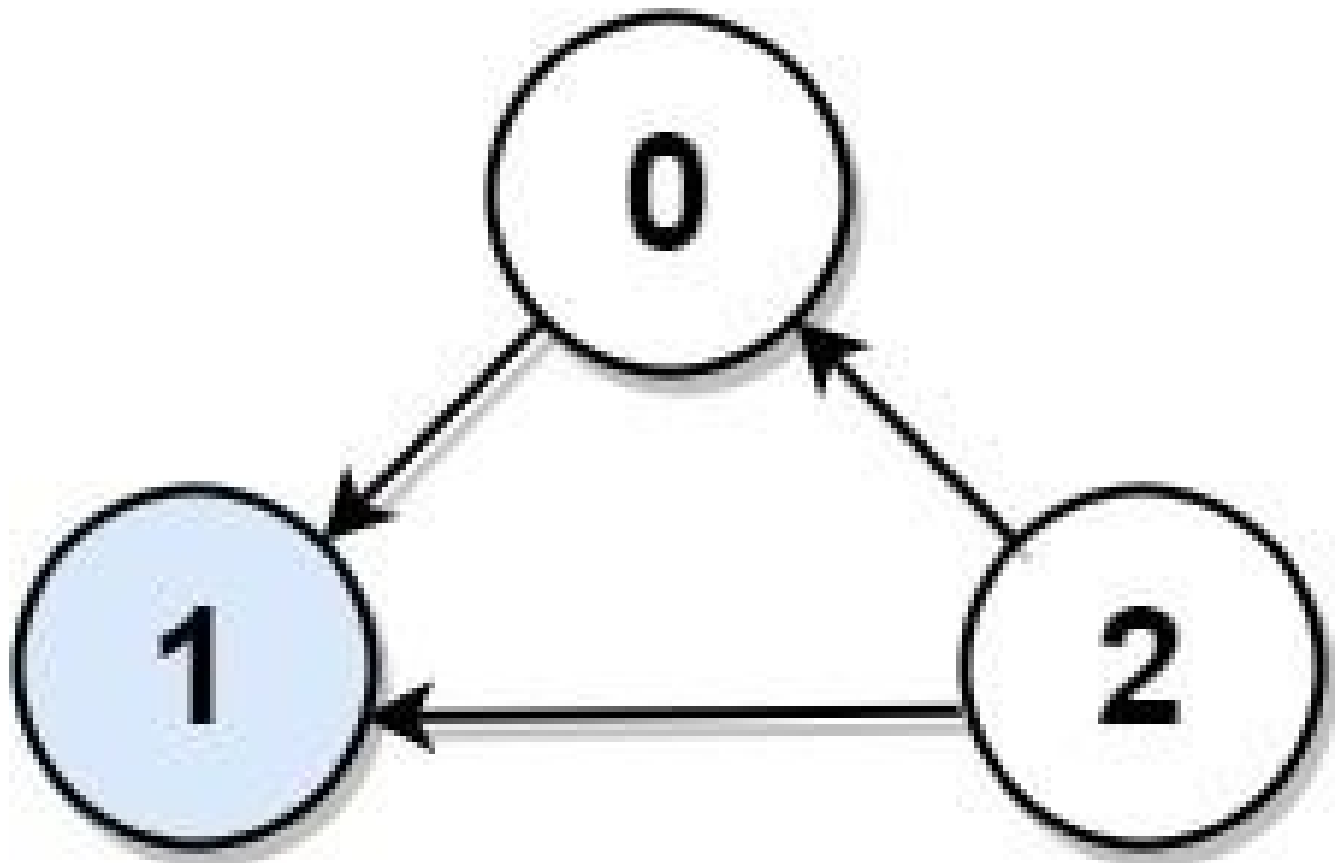
You are given an integer n and a helper function `bool knows(a, b)` that tells you whether a knows b. Implement a function `int findCelebrity(n)`.

There will be exactly one celebrity if they are at the party.

Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return -1.

Note that the $n \times n$ 2D array graph given as input is not directly available to you, and instead only accessible through the helper function knows. $\text{graph}[i][j] == 1$ represents person i knows person j , whereas $\text{graph}[i][j] == 0$ represents person j does not know person i .

Example 1:

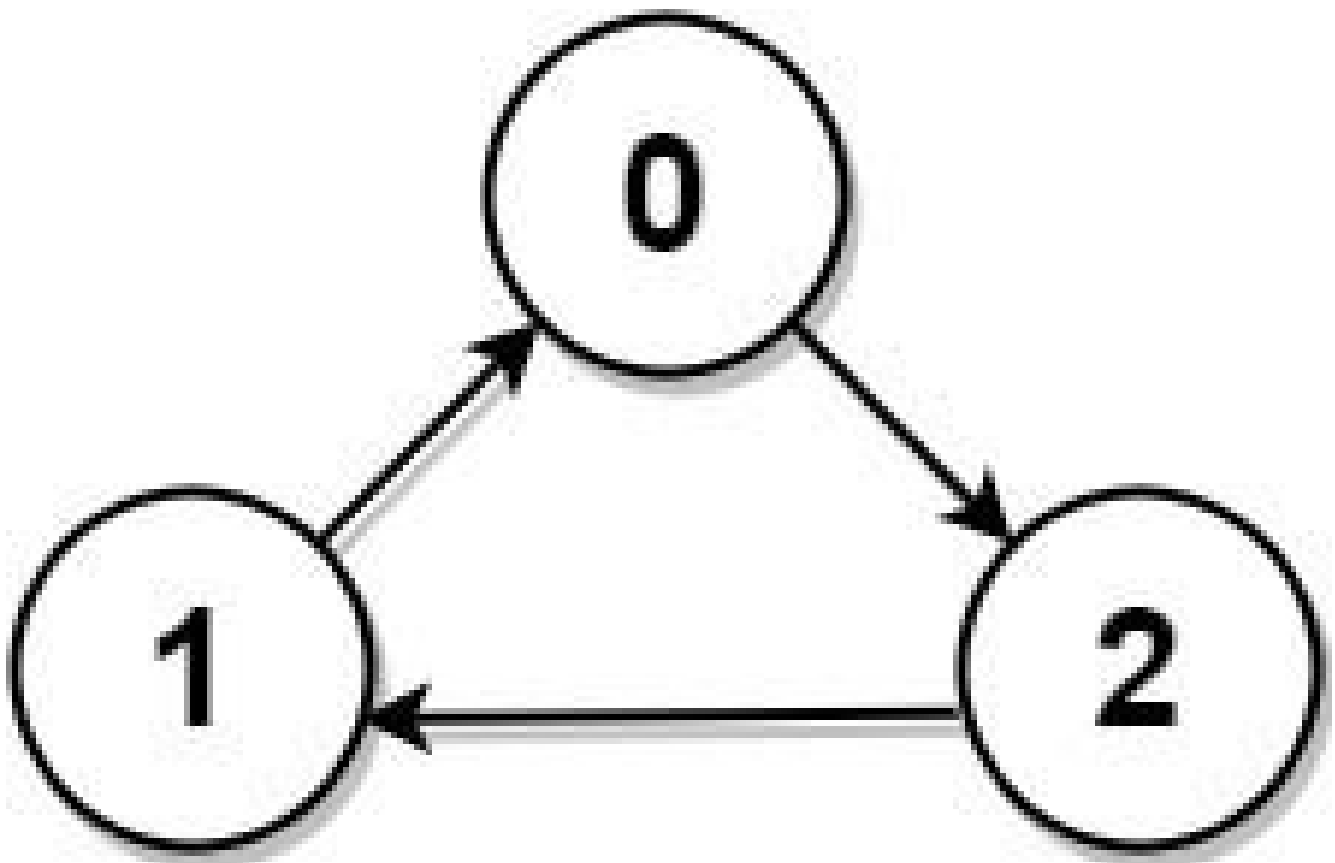


Input: graph =
[[1,1,0],[0,1,0],[1,1,1]]

Output: 1

Explanation: There are three persons labeled with 0, 1 and 2. $\text{graph}[i][j] = 1$ means person i knows person j , otherwise $\text{graph}[i][j] = 0$ means person i does not know person j . The celebrity is the person labeled as 1 because both 0 and 2 know him but 1 does not know anybody.

Example 2:



Input: graph =
[[1,0,1],[1,1,0],[0,1,1]]
Output: -1
Explanation: There is no celebrity.

Constraints:

- $n == \text{graph.length} == \text{graph}[i].\text{length}$
- $2 \leq n \leq 100$
- $\text{graph}[i][j]$ is 0 or 1.
- $\text{graph}[i][i] == 1$

Follow up: If the maximum number of allowed calls to the API knows is $3 * n$, could you find a solution without exceeding the maximum number of calls?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Celebrity: everyone knows them, they know no one.

Find them using knows(a,b) API — minimise calls. Right?

At most one celebrity can exist."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Ask every other person if they know celebrity; candidate must be known by all and know nobody.

Two-pass elimination: first narrow to one candidate with $O(n)$ knows() calls, then verify.

Naive all-pairs check is $O(n^2)$ knows() calls.

Time: $O(n^2)$ naive, $O(n)$ with elimination. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (candidate elimination + verify)

"Pass 1: start with candidate=0, scan all i. If knows(candidate, i): celebrity can't be candidate → candidate=i.

Pass 2: verify candidate – everyone must know them AND they know no one.

Total API calls: $O(n)$. Without the two-pass we'd need $O(n^2)$."

PHASE 4 — CLEAN CODE

class _277_FindCelebrity:

```
def findCelebrity(self, n: int) ->
int:
    candidate = 0
    for i in range(1, n):
        if knows(candidate, i):
            candidate = i
    for i in range(n):
        if i == candidate:
            continue
        if knows(candidate, i) or not
knows(i, candidate):
            return -1
    return candidate
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – $2n-2$ API calls in the worst case. Space $O(1)$.

The candidate elimination logic: if A knows B, A is not the celebrity (celebrity knows no one).

If A doesn't know B, B is not the celebrity (celebrity is known by all).

One of them is eliminated per comparison – $O(n)$ total.

Given more time I'd ask: what if there's a possibility of no celebrity?

The current solution handles this – the verification pass returns -1 if candidate fails."

Graphs

□ ★ #1136 Parallel Courses ★

MED

[Open on LeetCode](#)

Graph · Topological Sort

Lists: ★ Premium | Premium 98

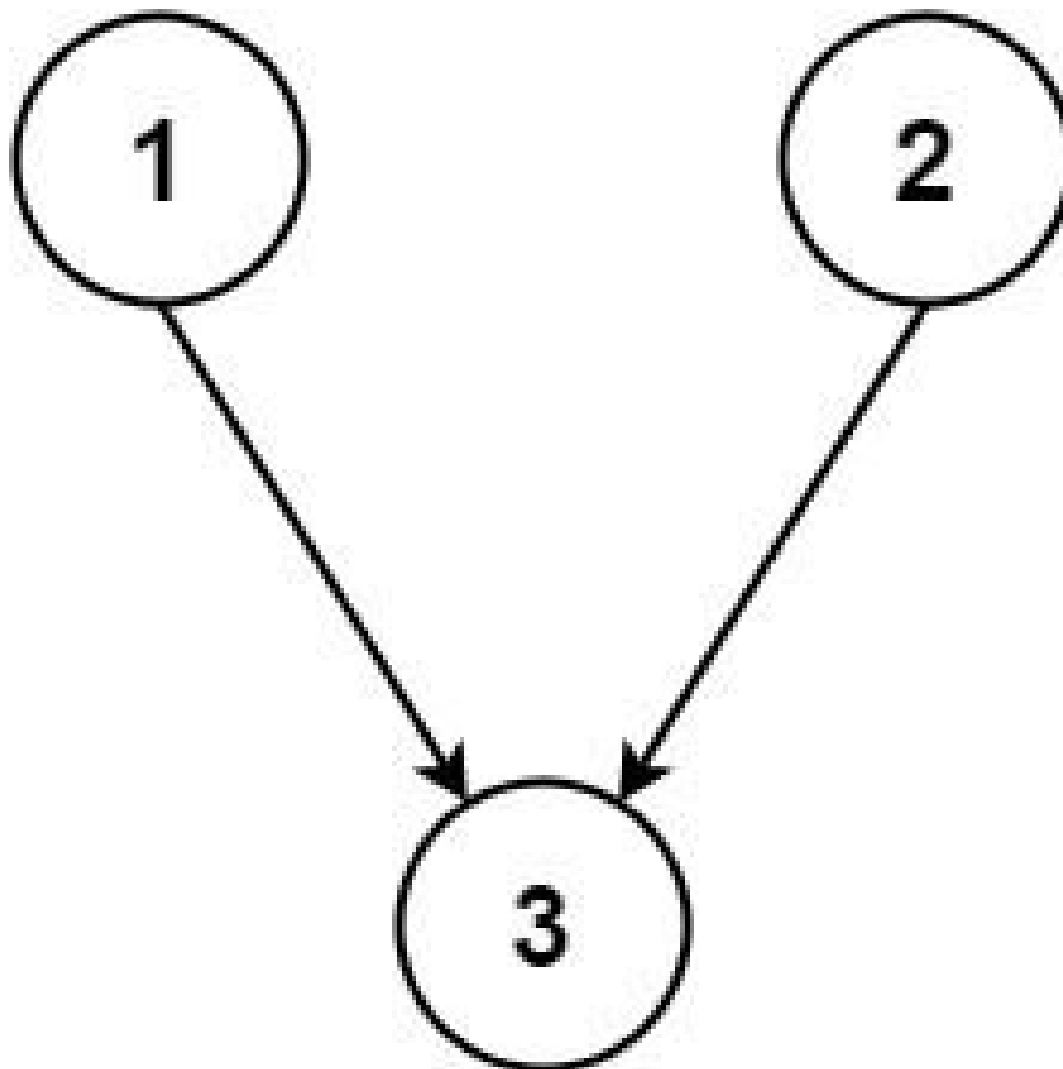
Problem

You are given an integer n , which indicates that there are n courses labeled from 1 to n . You are also given an array `relations` where `relations[i] = [prevCoursei, nextCoursei]`, representing a prerequisite relationship between course `prevCoursei` and course `nextCoursei`: course `prevCoursei` has to be taken before course `nextCoursei`.

In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.

Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1 .

Example 1:



Input: $n = 3$, $\text{relations} = [[1,3],[2,3]]$

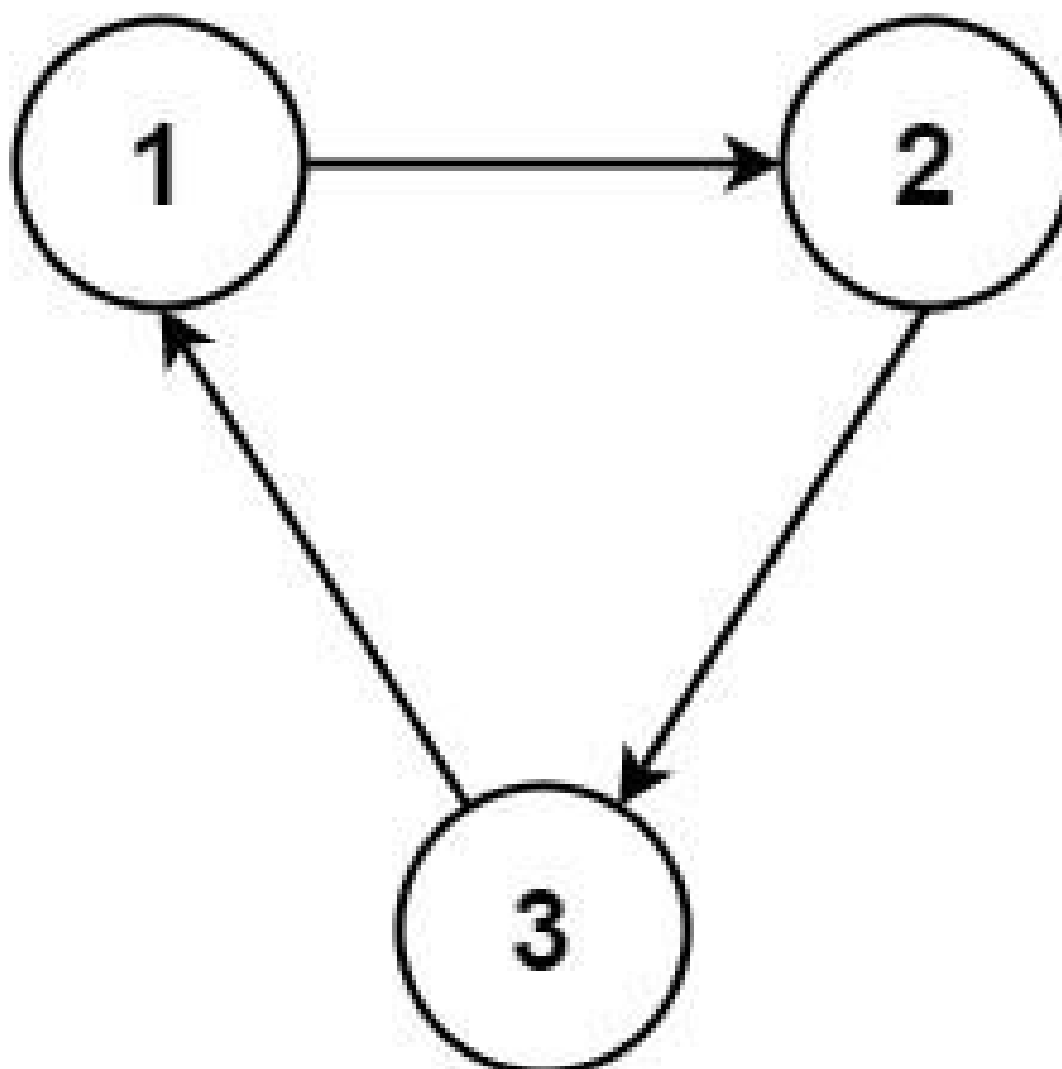
Output: 2

Explanation: The figure above represents the given graph.

In the first semester, you can take courses 1 and 2.

In the second semester, you can take course 3.

Example 2:



Input: $n = 3$, **relations** =
[[1,2],[2,3],[3,1]]

Output: -1

Explanation: No course can be studied because they are prerequisites of each other.

Constraints:

- $1 \leq n \leq 5000$

- $1 \leq \text{relations.length} \leq 5000$
- $\text{relations}[i].\text{length} == 2$
- $1 \leq \text{prevCoursei}, \text{nextCoursei} \leq n$
- $\text{prevCoursei} \neq \text{nextCoursei}$
- All the pairs $[\text{prevCoursei}, \text{nextCoursei}]$ are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum semesters to take all courses where you can take any number per semester as long as prerequisites are met. Return -1 if impossible (cycle). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Topological sort with indegree: each semester take all courses whose prerequisites are done.

Increment semester count each round; if no course can be taken but some remain, impossible.

Same Kahn BFS layered by semester.

Time: $O(V + E)$. Space: $O(V + E)$.

```
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Kahn's BFS
level-by-level)
# "BFS topological sort: process all
zero-in-degree nodes in one 'batch'
(semester).
# Each batch = one semester. Count
batches. If total processed < n → cycle."

# PHASE 4 — CLEAN CODE
class _1136_ParallelCourses:
    def minimumSemesters(self, n: int,
relations: List[List[int]]) -> int:
        graph = defaultdict(list)
        indeg = {i: 0 for i in range(1, n
+ 1)}

        for pre, nxt in relations:
            graph[pre].append(nxt)
            indeg[nxt] += 1

        queue = deque(c for c in range(1,
n + 1) if indeg[c] == 0)
        sems = 0
        total = 0
        while queue:
            sems += 1
```

```
for _ in range(len(queue)):
    node = queue.popleft()
    total += 1
    for nb in graph[node]:
        indeg[nb] -= 1
        if indeg[nb] == 0:
            queue.append(nb)
return sems if total == n else -1
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. This is Course Schedule (#207) with BFS level counting.

Each BFS level = one semester. Cycle detection: total processed $< n$.

Given more time I'd ask: what if courses have durations (not just 1 semester each)?

This becomes a weighted DAG critical path problem — DP or Bellman-Ford."

BFS

Round 2 · High · Medium · 6 questions

BFS

□ ★ #286 Walls and Gates ★

MED

[Open on LeetCode](#)

Array · BFS · Matrix

Lists: ★ Premium | Premium 98

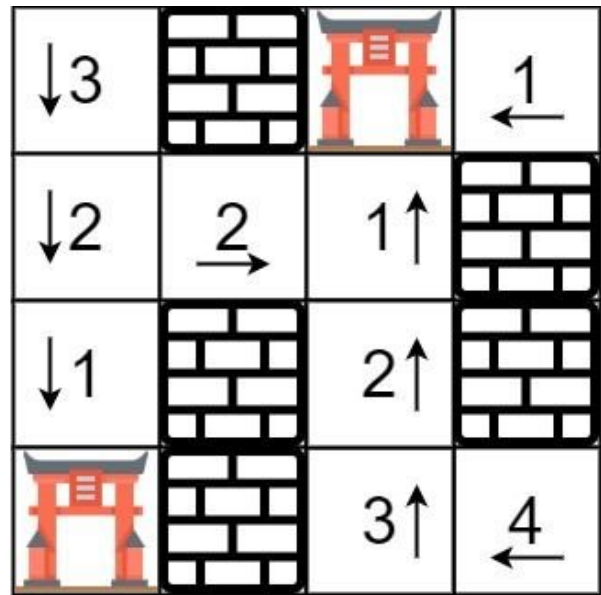
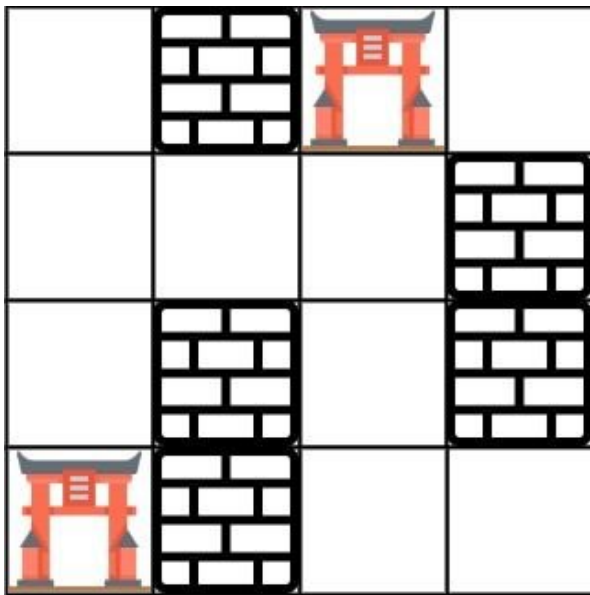
Problem

You are given an $m \times n$ grid rooms initialized with these three possible values.

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $231 - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647 .

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF .

Example 1:



Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
 Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]

Example 2:

Input: rooms = [[-1]]
 Output: [[-1]]

Constraints:

- $m == \text{rooms.length}$
- $n == \text{rooms}[i].\text{length}$
- $1 \leq m, n \leq 250$
- $\text{rooms}[i][j]$ is -1 , 0 , or $2^{31} - 1$.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Fill each INF room with its distance to the nearest gate (0). Walls (-1) stay.

Right?

In-place modification. $INF = 2^{31}-1$."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Multi-source BFS from every gate simultaneously (distance 0).

Fill empty rooms layer by layer with increasing distance.

Each room reached once – standard BFS on grid.

Time: $O(m * n)$. Space: $O(m * n)$ queue.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (multi-source BFS from all gates)

"Enqueue ALL gates at once. BFS outward – each room gets filled with distance

from the nearest gate simultaneously.

$O(m*n)$."

PHASE 4 — CLEAN CODE

```
class _286_WallsAndGates:
    def wallsAndGates(self, rooms:
List[List[int]]) -> None:
        INF = 2**31 - 1
        m, n = len(rooms), len(rooms[0])
        queue = deque((r, c) for r in
range(m) for c in range(n) if rooms[r][c]
== 0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc
< n and rooms[nr][nc] == INF:
                    rooms[nr][nc] =
rooms[r][c] + 1
                    queue.append((nr,
nc))
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS is the key – a single BFS from each gate

independently would be $O(m \times n \times \text{gates})$, much slower.

Contrast with  #542 (01 Matrix): identical pattern but for distance to nearest 0.

Given more time I'd ask: find distance to nearest wall instead?

Multi-source BFS from all walls – same approach."

BFS

□ #322 Coin Change

MED[Open on LeetCode](#)

Array · Dynamic Programming · BFS

Lists: Grind 169

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: `coins = [1,2,5]`, `amount = 11`

Output: 3

Explanation: $11 = 5 + 5 + 1$

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 231 - 1$
- $0 \leq \text{amount} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum coins to make up amount.
Unlimited supply of each coin. Return -1
if impossible. Right?"

Coin values can be any positive integer
– not necessarily power of 2 (so greedy
fails)."

#

"coins=[1,2,5], amount=11: 5+5+1=3
coins ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: for each amount, try every coin denomination and take 1 + min subproblem.

Recomputes identical sub-amounts exponentially without memoization.

Time: $O(\text{amount}^{\text{coins}})$ exponential.

Space: $O(\text{amount})$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (bottom-up DP)

"dp[i] = minimum coins to make amount i.

dp[0]=0. dp[i] = min(dp[i-coin]+1) for each coin <= i.

Time: $O(\text{amount} * \text{coins})$. **Space:** $O(\text{amount})$."

PHASE 4 — CLEAN CODE

class _322_CoinChange:

```
    def coinChange(self, coins:
List[int], amount: int) -> int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0
        for coin in coins:
```

```
        for i in range(coin, amount + 1):
            dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[amount] if dp[amount] != float('inf') else -1
```

PHASE 5 — TEST & DEBUG

```
# coins=[1,2,5], amount=11:
# dp[5]=1(5). dp[10]=2(5+5).
dp[11]=3(5+5+1). → 3 ✓
# amount=3, coins=[2]: dp[2]=1, dp[3]=inf
→ -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(amount * coins). Space O(amount).
# BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.
# DP is preferred – BFS visits each amount once but rebuilds from scratch.
# Given more time I'd ask: how many ways to make the amount?
# Coin Change 2 (#518) – same DP but count combinations instead of min count."
```

BFS

#542 01 Matrix

MED

[Open on LeetCode](#)

Array · BFS · Matrix

Lists: Grind 169 | Denny Zhang

Problem

Given an $m \times n$ binary matrix `mat`, return the distance of the nearest 0 for each cell.

The distance between two cells sharing a common edge is 1.

Example 1:

0	0	0
0	1	0
0	0	0

Input: `mat = [[0,0,0],[0,1,0],[0,0,0]]`

Output: `[[0,0,0],[0,1,0],[0,0,0]]`

Example 2:

0	0	0
0	1	0
1	1	1

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`
Output: `[[0,0,0],[0,1,0],[1,2,1]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 104`
- `1 <= m * n <= 104`
- `mat[i][j]` is either 0 or 1.

- There is at least one 0 in mat.

Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return distance of nearest 0 for each cell. In-place or new matrix. Right?

Same pattern as Walls and Gates but for 0-cells instead of gates."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Multi-source BFS starting from every 0 simultaneously.

First time each 1 cell is reached, its distance is minimal.

Layer-by-layer BFS fills the dist matrix in one pass.

Time: $O(m * n)$. Space: $O(m * n)$ queue + dist.

Should I go ahead and optimize?"

class _542_ZeroOneMatrix:

```
def updateMatrix(self, mat:
List[List[int]]) -> List[List[int]]:
    m, n = len(mat), len(mat[0])
    dist = [[0 if mat[r][c] == 0 else
float('inf') for c in range(n)] for r in
range(m)]
    queue = deque((r, c) for r in
range(m) for c in range(n) if mat[r][c] ==
0)
    while queue:
        r, c = queue.popleft()
        for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < m and 0 <= nc
< n and dist[nr][nc] > dist[r][c] + 1:
                dist[nr][nc] =
dist[r][c] + 1
                queue.append((nr,
nc))
    return dist
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$. Space $O(m \cdot n)$. Multi-source BFS from all 0-cells simultaneously."

Identical to #286 (Walls and Gates) – name this connection.
Given more time I'd ask: distance to nearest 1 instead?
Same multi-source BFS starting from all 1-cells."

BFS

□ #994 Rotting Oranges

MED

[Open on LeetCode](#)

Array · BFS · Matrix

Lists: Grind 169

Problem

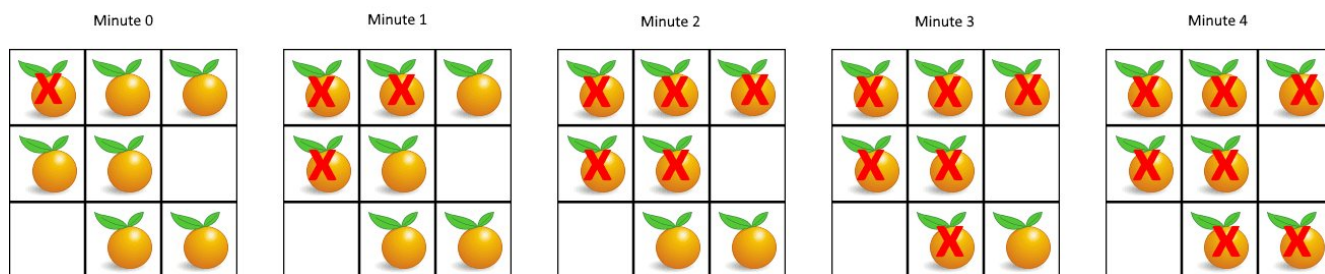
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

Example 1:



Input: `grid = [[2,1,1],[1,1,0],[0,1,1]]`
Output: 4

Example 2:

Input: `grid = [[2,1,1],[0,1,1],[1,0,1]]`
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: `grid = [[0,2]]`
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- `m == grid.length`
- `n == grid[i].length`

- $1 \leq m, n \leq 10$
- `grid[i][j]` is 0, 1, or 2.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Multi-source BFS from all rotten oranges. Fresh adjacent → rotten after 1 minute.

Return minutes until no fresh remain, or -1 if impossible. Right?"

#

"If no fresh oranges initially → return 0 immediately."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Multi-source BFS: enqueue all rotten oranges at minute 0.

Each minute, rot all fresh neighbors; track elapsed minutes until no fresh remain.

If fresh count > 0 after BFS ends, return -1.

Time: $O(m * n)$. Space: $O(m * n)$ queue.

Should I go ahead and optimize?"

```
class _994_RottingOranges:
    def orangesRotting(self, grid:
List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        fresh = sum(grid[r][c] == 1 for r
in range(m) for c in range(n))
        queue = deque((r, c) for r in
range(m) for c in range(n) if grid[r][c]
== 2)

        minutes = 0
        while queue and fresh:
            minutes += 1
            for _ in range(len(queue)):
                r, c = queue.popleft()
                for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c +
dc

                    if 0 <= nr < m and 0
<= nc < n and grid[nr][nc] == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        queue.append((nr,
nc))
```


return minutes if fresh == 0 else

-1

PHASE 5 — TEST & DEBUG

[[2,1,1],[1,1,0],[0,1,1]]: fresh=6. BFS spreads level by level. After 4 minutes fresh=0 → 4 ✓

Isolated fresh orange: fresh>0 after BFS done → -1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS: same as Walls and Gates, same as 01 Matrix.

The 'fresh' counter eliminates the need to scan the grid again at the end.

Given more time I'd ask: what if diagonal rotting is allowed?

Add 4 diagonal direction pairs — same algorithm."

BFS

□ ★ #1197 Minimum Knight Moves ★

MED

[Open on LeetCode](#)

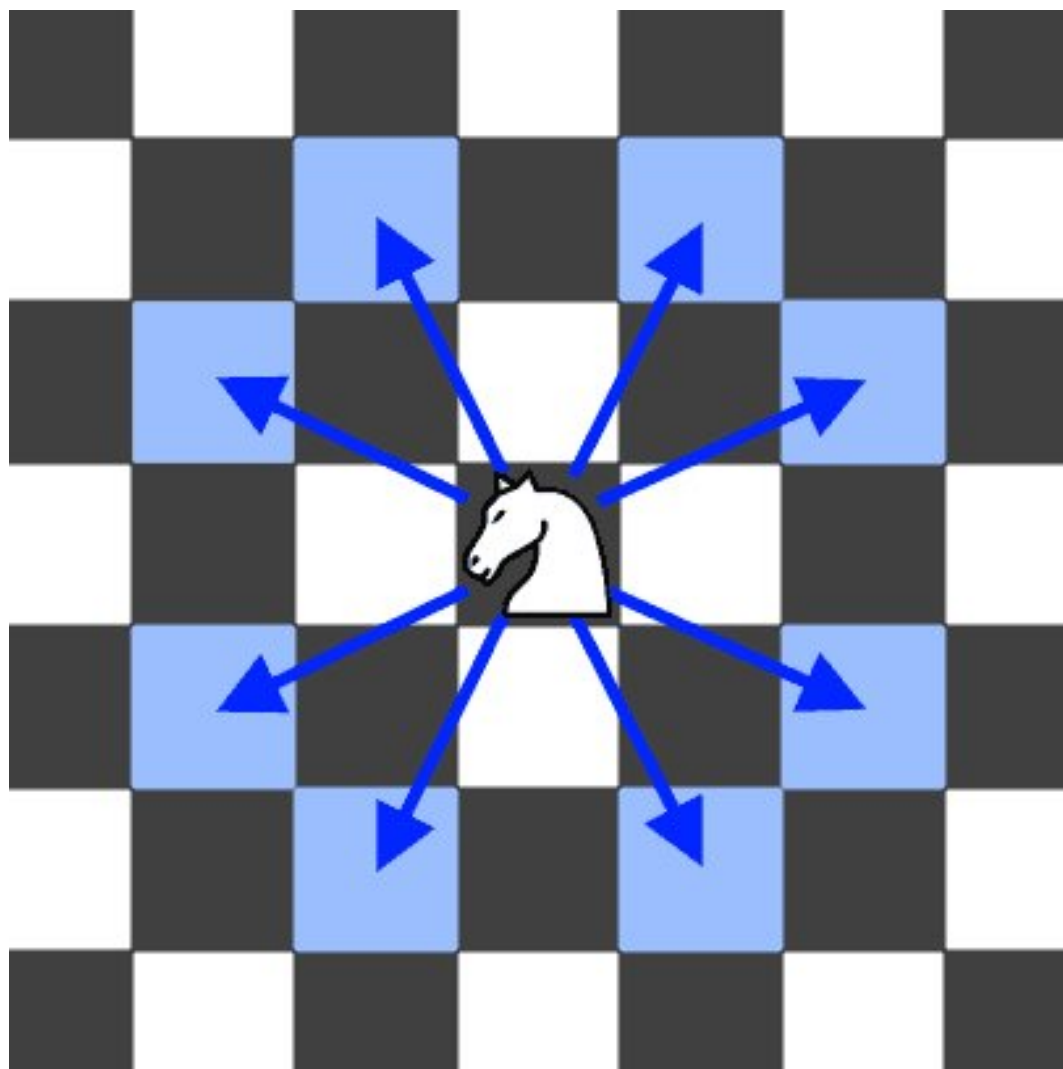
BFS

Lists: ★ Premium | Grind 169 | Premium 98

Problem

In an infinite chess board with coordinates from $-\infty$ to $+\infty$, you have a knight at square $[0, 0]$.

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square $[x, y]$. It is guaranteed the answer exists.

Example 1:

Input: $x = 2, y = 1$

Output: 1

Explanation: $[0, 0] \rightarrow [2, 1]$

Example 2:

Input: $x = 5, y = 5$

Output: 4

Explanation: $[0, 0] \rightarrow [2, 1] \rightarrow [4, 2] \rightarrow [3, 4] \rightarrow [5, 5]$

Constraints:

- $-300 \leq x, y \leq 300$
- $0 \leq |x| + |y| \leq 300$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Knight moves on an infinite board.

Minimum moves from $(0,0)$ to (x,y) .

8 possible moves. Symmetry: $|x|$ and $|y|$
– reflect to first quadrant. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

BFS on an infinite chessboard from $(0,0)$
to (x,y) .

Enqueue all 8 knight moves each step;
stop when target is reached.

Layer count = minimum moves. Prune to
first quadrant since moves are symmetric.

```
# Time:  $O(\max(x,y)^2)$  BFS nodes. Space:
 $O(\max(x,y)^2)$  visited.
# Should I go ahead and optimize?"

class _1197_MinKnightMoves:
    def minKnightMoves(self, x: int, y:
int) -> int:
        x, y = abs(x), abs(y) # exploit
symmetry – reflect to first quadrant
        queue = deque([(0, 0, 0)]) # (row,
col, steps)
        seen = {(0, 0)}
        while queue:
            r, c, steps = queue.popleft()
            if r == x and c == y:
                return steps
            for dr, dc in [(2,1),(2,-1),(-2,1),(-2,-1),(1,2),(1,-2),(-1,2),(-1,-2)]:
                nr, nc = r + dr, c + dc
                if (nr, nc) not in seen
and nr >= -1 and nc >= -1:
                    seen.add((nr, nc))
                    queue.append((nr, nc,
steps + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

(2,1): BFS from (0,0). (2,1) reached in 1 step → 1 ✓

(5,5): 4 steps ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\max(|x|, |y|)^2)$. Space $O(\max(|x|, |y|)^2)$. Standard BFS shortest path.

Symmetry reduction: reflect to first quadrant cuts search space by 4.

The -1 lower bound allows the knight to 'go back' through negative coords when needed.

Given more time I'd ask: bidirectional BFS?

Start from both (0,0) and (x,y) simultaneously – meets in the middle, roughly halving depth."

BFS

□ #1730 Shortest Path to Get Food

MED

[Open on LeetCode](#)

Array · BFS · Matrix

Lists: Grind 169

Problem

You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.

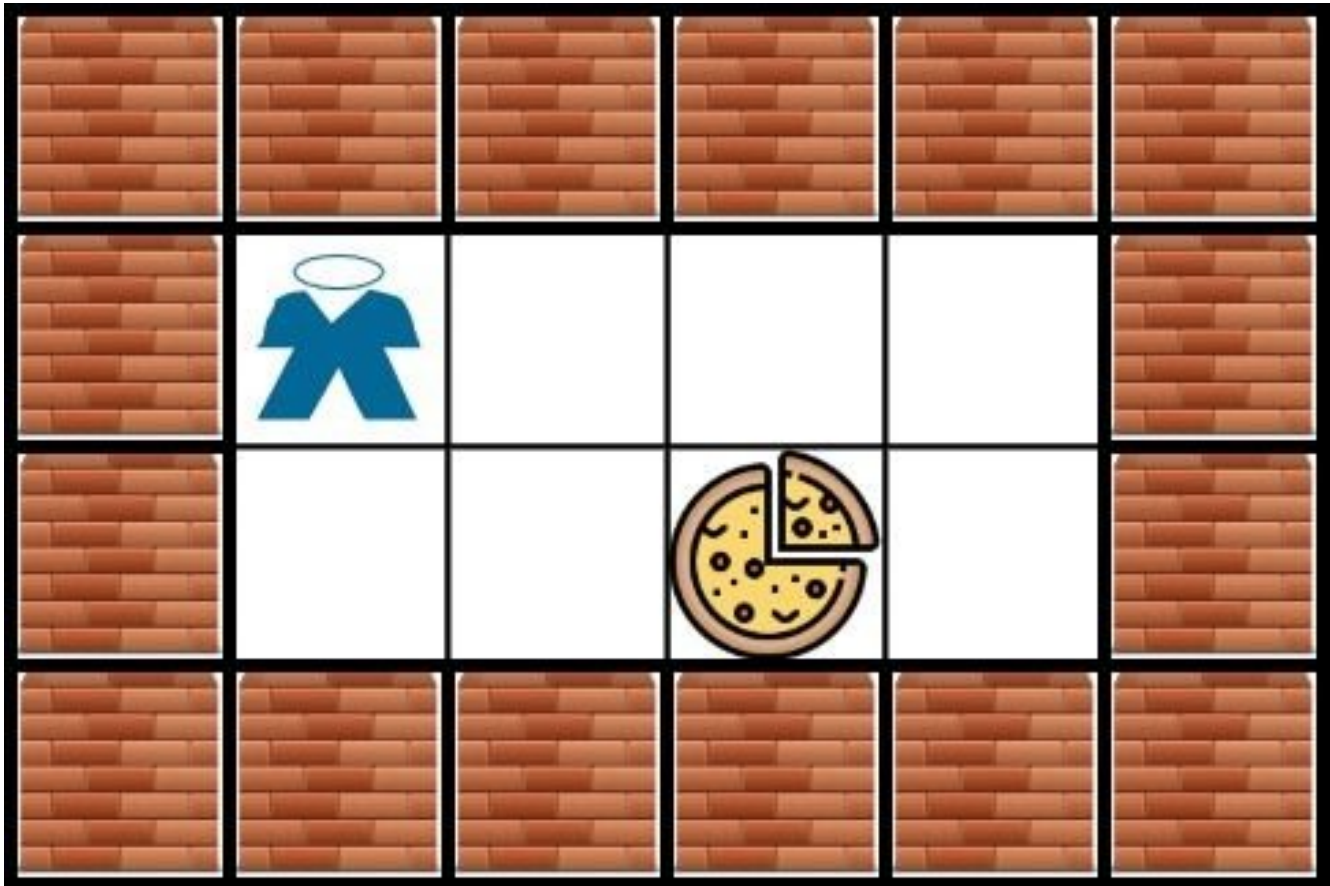
You are given an $m \times n$ character matrix, grid, of these different types of cells:

- '*' is your location. There is exactly one '*' cell.
- '#' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.
- 'X' is an obstacle, and you cannot travel through these cells.

You can travel to any adjacent cell north, east, south, or west of your current location if there is not an obstacle.

Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1:

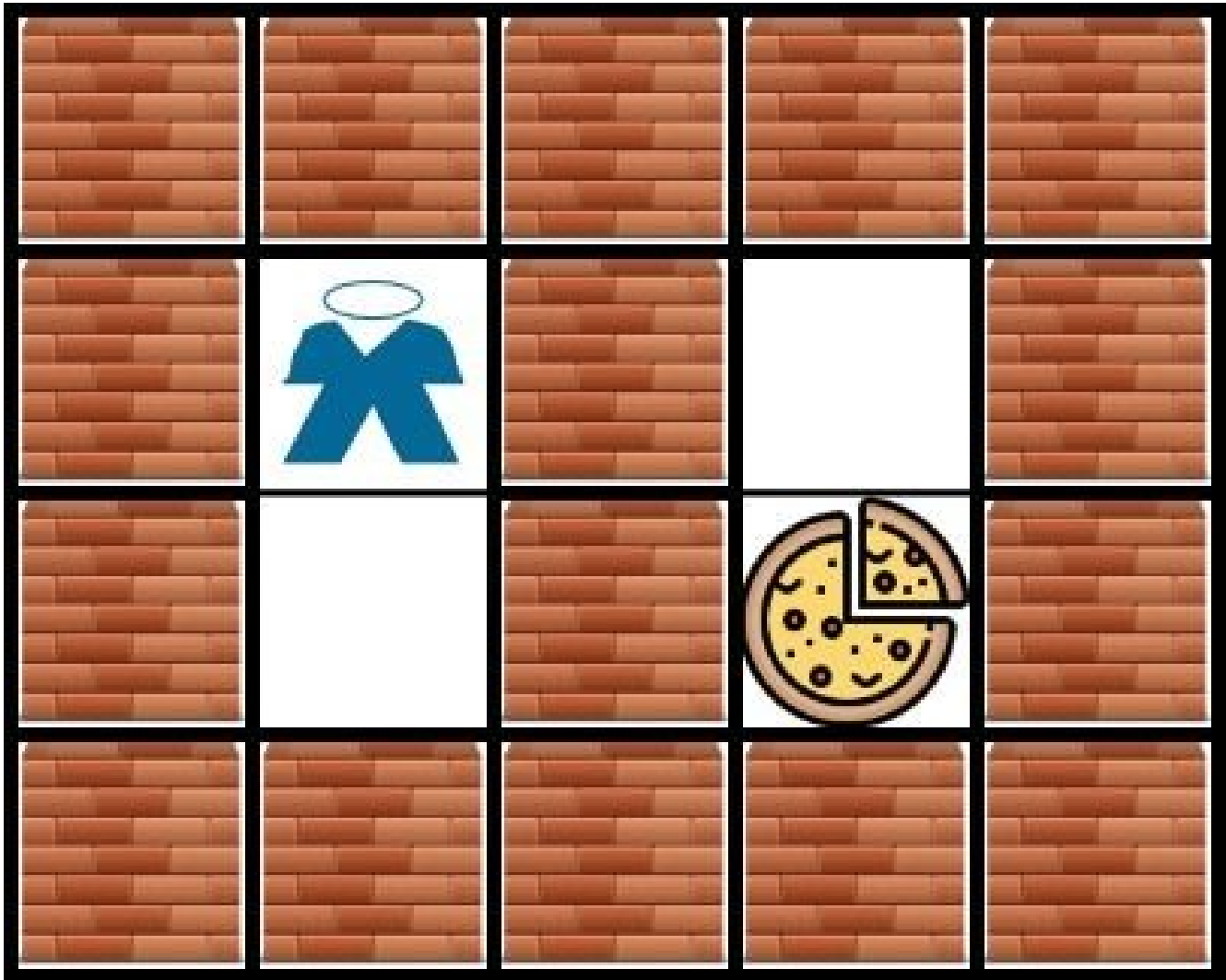


Input: grid = `[["X","X","X","X","X","X"],`
`["X","*","0","0","0","X"],`
`["X","0","0",`
`,"#","0","X"],`
`["X","X","X","X","X","X"]`
`]`

Output: 3

Explanation: It takes 3 steps to reach the food.

Example 2:

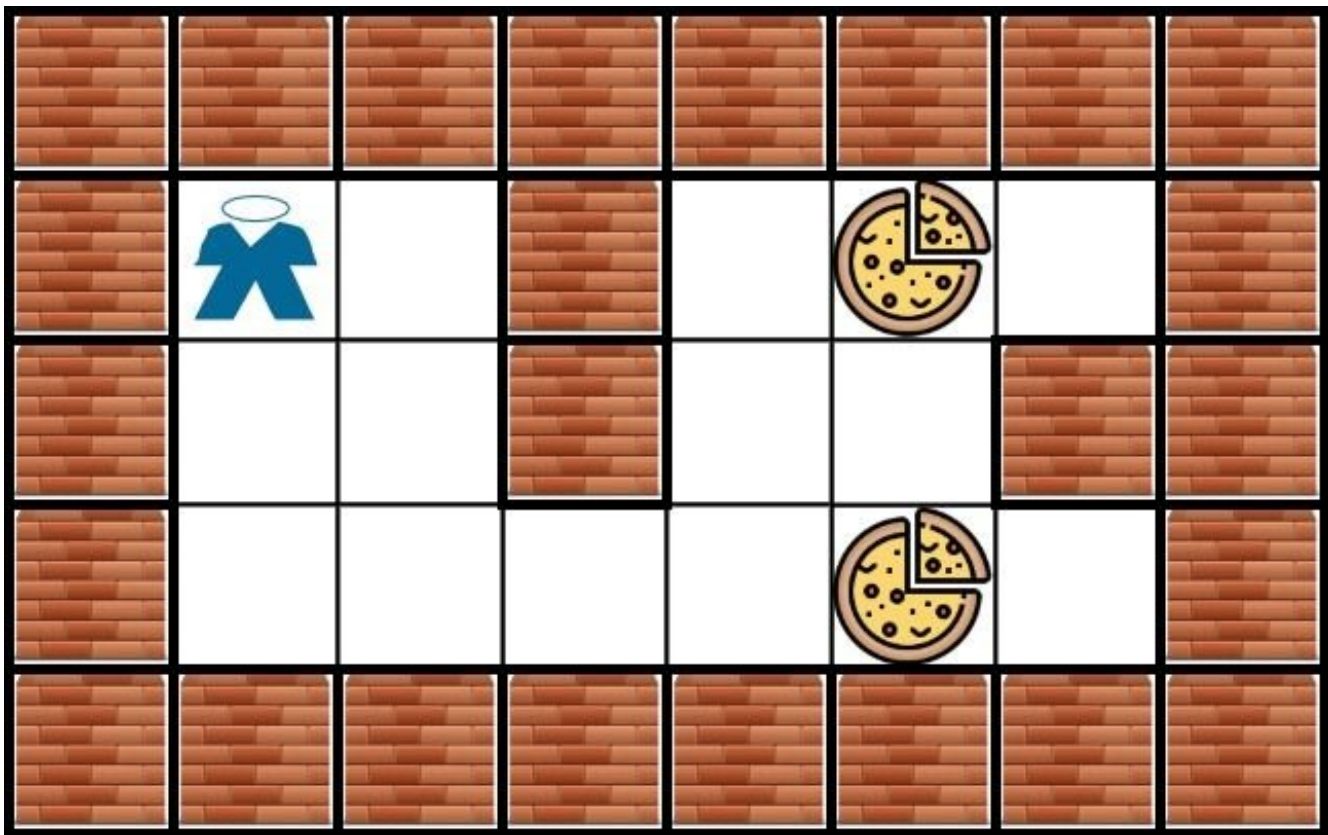


Input: `grid = [ ["X","X","X","X","X"], ["X","*","X","0","X"], ["X","0","X","#","X"], ["X","X","X","X","X"] ]`

Output: `-1`

Explanation: It is not possible to reach the food.

Example 3:



Input: grid = [
 ["X","X","X","X","X","X",
 "X","X"],
 ["X","*","0","X","0","#","0",
 "X"],
 ["X","0","0","X","0","0","X","X"],
 ["X","0","0","0","0","#","0","X"],
 ["X","X","X","X","X","X","X","X"]
]

Output: 6

Explanation: There can be multiple food cells. It only takes 6 steps to reach the bottom food.

Example 4:

```
Input: grid = [
  ["X","X","X","X","X","X"],
  ["X","X"],
  ["X","*","0","X","0","#","0",
  "X"],
  ["X","0","0","X","0","0","X","X"],
  ["X","0","0","0","0","#","0","X"],
  ["0",
  "0","0","0","0","0","0","0"]
]
```

Output: 5

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `grid[row][col]` is `'*'`, `'X'`, `'O'`, or `'#'`.
- The grid contains exactly one `'*'`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"BFS from '*' to the nearest '#'. 'O' = free, 'X' = wall, '#' = food (any one).

Return shortest path length. Return -1 if unreachable. Right?"

#

"'*' → nearest '#' using 4-directional BFS. First '#' reached = shortest path."

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock
in the logic.
# Multi-source BFS from every '*' food
cell simultaneously.
# First time a 'H' house is reached,
record its distance; sum cannot reach uses
-1.
# BFS is already optimal  $O(m*n)$ ; DFS would
revisit cells wastefully.
# Time:  $O(m * n)$ . Space:  $O(m * n)$  queue.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Standard BFS from source. First '#'
reached is guaranteed shortest path.
# Mark visited by overwriting '0' with 'X'
— no extra visited set needed.
# Time:  $O(m*n)$ . Space:  $O(m*n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _1730_ShortestPathFood:
    def getFood(self, grid:
List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        start = next((r, c) for r in
range(m) for c in range(n) if grid[r][c]
```

```

== '*' )
    queue = deque([(start[0],
start[1], 0)])
    grid[start[0]][start[1]] = 'X' #
mark visited
    while queue:
        r, c, dist = queue.popleft()
        for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < m and 0 <= nc
< n:
                if grid[nr][nc] ==
'#':
                    return dist + 1
                if grid[nr][nc] ==
'0':
                    grid[nr][nc] =
'X' # mark visited
                    queue.append((nr,
nc, dist + 1))
    return -1

```

PHASE 5 — TEST & DEBUG

grid example 1: BFS from '*'. Nearest '#' is 3 steps away → return 3 ✓

"Time $O(m*n)$. Space $O(m*n)$.

```
# with a target condition instead of a
distance fill.
```

Overwriting '0' with 'X' acts as the visited marker – avoids $O(m*n)$ extra space.

Given more time I'd ask: what if we want paths to ALL food cells (not just shortest)?

```
# Continue BFS after first '#' hit –
collect all '#' cells reached with their
distances."
```

#

HARD PROBLEMS

#

Round 3 · High · Hard

Round 3

High Priority · Hard

23 questions · 8 patterns

- ☐ ● Arrays & Hashing #41 ↗
- ☐ ● Sliding Window #76 ↗
- ☐ ● Binary Search #4 #315 #410 #668 #710 #774 ↗
- ☐ ● #1235 ↗
- ☐ ● Matrix #1074 ↗
- ☐ ● Trees & BST #124 #297 ↗
- ☐ ● DFS #269 #329 #685 #1036 #1192 ↗
- ☐ ● Graphs #305 #1153 ↗
- ☐ ● BFS #127 #317 #773 #815 ↗

Arrays & Hashing

Round 3 · High · Hard · 1 question

Arrays & Hashing

□ #41 First Missing Positive

HAR[Open on LeetCode](#)

Array · Hash Table

Lists: Grind 169

Problem

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in $O(n)$ time and uses $O(1)$ auxiliary space.

Example 1:

Input: `nums = [1,2,0]`

Output: 3

Explanation: The numbers in the range `[1,2]` are all in the array.

Example 2:

Input: `nums = [3,4,-1,1]`

Output: 2

Explanation: 1 is in the array but 2 is missing.

Example 3:

Input: `nums = [7,8,9,11,12]`

Output: 1

Explanation: The smallest positive integer 1 is missing.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Smallest positive integer NOT in nums.
 $O(n)$ time, $O(1)$ auxiliary space.

Answer is in range $[1, n+1]$ – if all $1..n$ present, answer is $n+1$. Right?

Negatives, zeros, and large numbers can be ignored."

#

"[3,4,-1,1] → 2 (1 present, 2 missing)
✓. [7,8,9] → 1 ✓. [1,2,0] → 3 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort the array, then scan from 1 upward until the first missing positive integer.

Or put all values in a hash set and scan $i=1,2,3\dots$ until absent.

Correct but sorting is $O(n \log n)$ and the set uses $O(n)$ extra space.

Time: $O(n \log n)$. Space: $O(n)$ for set approach.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (array as hash map — cycle sort)

"Key insight: use the array itself as a lookup table.

Valid range is $[1, n]$. Place each number x at index $x-1$ (if $1 \leq x \leq n$ and not already there).

After placing, scan: first i where $\text{nums}[i] \neq i+1 \rightarrow$ answer is $i+1$.

If all positions correct → answer is $n+1$.

Each number is placed at most once → $O(n)$ total swaps."

PHASE 4 — CLEAN CODE

```
class _41_FirstMissingPositive:
    def firstMissingPositive(self, nums:
List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and
nums[nums[i] - 1] != nums[i]:
                nums[nums[i] - 1],
nums[i] = nums[i], nums[nums[i] - 1]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

PHASE 5 — TEST & DEBUG

[3,4,-1,1]: place 3→pos2, 4→pos3, -1 ignored, 1→pos0.

After: [1,-1,3,4]. Scan: i=1
nums[1]=-1≠2 → return 2 ✓

[1,2,3]: after: [1,2,3]. All correct →
return 4 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ — each element swapped at most once. Space $O(1)$.

The while loop runs at most n total iterations across the entire for loop — amortised $O(1)$ per i .

Given more time I'd ask: what if duplicates exist?

The condition `nums[nums[i]-1] != nums[i]` handles duplicates — don't swap a number to a position

that already has the correct value (would loop forever)."

Sliding Window

Round 3 · High · Hard · 1 question

Sliding Window

□ #76 Minimum Window Substring

HAR[Open on LeetCode](#)

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "".

The testcases will be generated such that the answer is unique.

Example 1:

Input: $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$

Output: "BANC"

Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t .

Example 2:

Input: `s = "a", t = "a"`

Output: `"a"`

Explanation: The entire string `s` is the minimum window.

Example 3:

Input: `s = "a", t = "aa"`

Output: `""`

Explanation: Both 'a's from `t` must be included in the window.

Since the largest window of `s` only has one 'a', return empty string.

Constraints:

- `m == s.length`
- `n == t.length`
- `1 <= m, n <= 105`
- `s` and `t` consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```
# "Smallest window in s containing all
characters of t (with multiplicity).
# Return '' if no such window. Right?
# t may have duplicate chars – window must
have at least that many."
#
# "s='ADOBECODEBANC', t='ABC' → 'BANC'
(contains A,B,C, length 4) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Enumerate every substring of s with two
nested indices.
# For each substring, check whether it
contains all characters of t (with
frequency).
# Track the shortest valid window –
correct but  $O(m^2 * n)$  character checks.
# Time:  $O(m^2 * n)$ . Space:  $O(n)$  frequency
map.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (sliding window with
'formed' counter)
```

"Expand right: add s[right] to window frequency map.

When a char's count in window meets t's requirement, increment 'formed'.

When formed == len(distinct chars in t): window is valid. Try shrinking from left.

Track best window. O(m+n) – each char enters/exits at most once."

PHASE 4 — CLEAN CODE

class _76_MinWindowSubstring:

def minWindow(self, s: str, t: str) -> str:

if not t or not s:

return ''

need = Counter(t)

have : Counter = Counter()

formed = 0

required = len(need)

left = 0

best = (float('inf'), 0, 0) #

(length, left, right)

for right in range(len(s)):

c = s[right]

have[c] += 1

```

        if c in need and have[c] ==
need[c]:
            formed += 1
            while formed == required:
                if right - left + 1 <
best[0]:
                    best = (right - left +
1, left, right)
                    have[s[left]] -= 1
                    if s[left] in need and
have[s[left]] < need[s[left]]:
                        formed -= 1
                        left += 1
            return '' if best[0] ==
float('inf') else s[best[1]: best[2] + 1]

```

PHASE 5 — TEST & DEBUG

s='ADOBECODEBANC', t='ABC':

need={A:1,B:1,C:1}. required=3.

Window expands until it contains A,B,C.
Shrink left until one char drops below
need.

Best window = 'BANC' ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$. Space $O(m+n)$. The 'formed' counter avoids comparing the full frequency maps.

A window is valid exactly when $\text{formed} == \text{required}$ – elegant $O(1)$ validity check.

Given more time I'd ask: find all minimum windows?

Collect all windows where $\text{length} == \text{best length}$ during the BFS scan."

Binary Search

Round 3 · High · Hard · 7 questions

Binary Search

□ #4 Median of Two Sorted Arrays

HAR[Open on LeetCode](#)

Array · Binary Search · Divide and Conquer

Lists: Grind 169

Problem

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`

Output: `2.00000`

Explanation: merged array = `[1,2,3]` and median is 2.

Example 2:

Input: nums1 = [1,2], nums2 = [3,4]

Output: 2.50000

Explanation: merged array = [1,2,3,4]
and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-106 \leq \text{nums1}[i], \text{nums2}[i] \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge two sorted arrays conceptually and find the median. $O(\log(m+n))$ required. Right?

Median: if total length is odd → middle element. Even → average of two middles."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

```
# Merge both sorted arrays with two
pointers into one combined array.
# Walk to index (m+n)//2 (and the one
before if even length) to read the median.
# Correct but uses O(m+n) extra space and
ignores the logarithmic requirement.
# Time: O(m + n). Space: O(m + n) merged
array.
# Should I go ahead and optimize?"
```

```
class _4_MedianTwoArrays_Brute:
    def findMedianSortedArrays(self,
nums1: List[int], nums2: List[int]) ->
float:
        m, n = len(nums1), len(nums2)
        i = j = 0
        half = (m + n) // 2
        prev = curr = 0
        for _ in range(half + 1):
            prev = curr
            if j >= n or (i < m and
nums1[i] <= nums2[j]):
                curr = nums1[i]; i += 1
            else:
                curr = nums2[j]; j += 1
```



```
        return (curr + prev) / 2 if not (m
+ n) & 1 else curr
```

```
# PHASE 3 — OPTIMIZE (binary search on
partition — true O(log(min(m,n))))
```

```
# "Partition both arrays into left and
right halves such that:
```

```
# total left size = (m+n+1)//2, max(left)
<= min(right).
```

```
# Binary search on the partition point of
the smaller array.
```

```
# At each partition: check if nums1[mid-1]
<= nums2[r-mid] and nums2[r-mid-1] <=
nums1[mid].
```

```
# Median = max(left halves) for odd total,
or average of max(left)/min(right) for
even."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _4_MedianTwoArrays:
```

```
    def findMedianSortedArrays(self,
nums1: List[int], nums2: List[int]) ->
float:
```

```
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        m, n = len(nums1), len(nums2)
```

```
        half = (m + n + 1) // 2
        lo, hi = 0, m
        while lo <= hi:
            i = (lo + hi) // 2 # partition
of nums1
            j = half - i # partition of
nums2
            left1 = nums1[i - 1] if i > 0
else float('-inf')
            right1 = nums1[i] if i < m
else float('inf')
            left2 = nums2[j - 1] if j > 0
else float('-inf')
            right2 = nums2[j] if j < n
else float('inf')
            if left1 <= right2 and left2
<= right1:
                if (m + n) % 2 == 1:
                    return
float(max(left1, left2))
                return (max(left1, left2)
+ min(right1, right2)) / 2
            elif left1 > right2:
                hi = i - 1
            else:
```

```
        lo = i + 1
    return 0.0
```

PHASE 5 — TEST & DEBUG

```
# nums1=[1,3], nums2=[2]: m=2,n=2.
half=2. Partition i=1,j=1:
left1=1,right1=3,left2=2,right2=∞.
# 1≤∞ and 2≤3 ✓. Total=3 (odd) →
max(1,2)=2. → 2.0 ✓
# nums1=[1,2], nums2=[3,4]: Partition →
max(left)=2, min(right)=3. → 2.5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\log(\min(m,n)))$ . Space  $O(1)$ .
Binary search on the smaller array's
partition.
```

```
# The +inf/-inf sentinels handle edge
cases (empty left/right) cleanly.
```

```
# The brute-force merge approach is often
acceptable in interviews – present both
and let the interviewer choose.
```

```
# Given more time I'd ask: kth element of
two sorted arrays?
```

```
# Binary search on k – eliminate  $k//2$ 
elements from one array per step. Same
 $O(\log k)$  idea."
```

Binary Search

□ #315 Count of Smaller Numbers After Self

HAR[Open on LeetCode](#)

Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: Denny Zhang

Problem

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Example 1:

Input: `nums = [5,2,6,1]`

Output: `[2,1,1,0]`

Explanation:

To the right of 5 there are 2 smaller elements (2 and 1).

To the right of 2 there is only 1 smaller element (1).

To the right of 6 there is 1 smaller element (1).

To the right of 1 there is 0 smaller element.

Example 2:

Input: `nums = [-1]`

Output: `[0]`

Example 3:

Input: `nums = [-1,-1]`

Output: `[0,0]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"For each index i , count elements to the right of i that are strictly less than $\text{nums}[i]$.

Return as an array. Right?"

#

"[5,2,6,1] $\rightarrow$ [2,1,1,0]: right of 5 has 2 and 1 (2 smaller), etc. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each index i , scan every element to its right and count how many are smaller than $\text{nums}[i]$.

Store those counts in an output array.

Correct, but the inner scan makes this quadratic overall.

Time: $O(n^2)$. Space: $O(1)$ besides output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BIT / merge sort)

"Process from right to left. Build a BIT indexed by coordinate-compressed value.

```
# For each nums[i]: query BIT for count of
values in [min, nums[i]-1].
# Update BIT at nums[i]. O(n log n) time."
```

PHASE 4 — CLEAN CODE

```
class _315_CountSmaller:
    def countSmaller(self, nums:
List[int]) -> List[int]:
        sorted_unique = sorted(set(nums))
        rank = {v: i + 1 for i, v in
enumerate(sorted_unique)}
        m = len(sorted_unique)
        bit = [0] * (m + 1)
        def update(i):
            while i <= m:
                bit[i] += 1
                i += i & (-i)
        def query(i):
            s = 0
            while i > 0:
                s += bit[i]
                i -= i & (-i)
            return s
        result = []
        for num in reversed(nums):
```

```
        r = rank[num]
        result.append(query(r - 1)) #
count of values < num seen so far
        update(r)
    return result[::-1]
```

PHASE 5 — TEST & DEBUG

```
# [5,2,6,1]: process right→left.
1→query(0)=0, update(1).
6→query(3)=1, update(4).
2→query(1)=1, update(2). 5→query(2)=2.
# reversed result=[2,1,1,0] → reverse →
[2,1,1,0] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$ . Space  $O(n)$ . BIT query
gives prefix count in  $O(\log n)$ .
# Merge sort alternative: during merge,
count inversions – same complexity but
different mental model.
# Given more time I'd ask: count of larger
elements after self?
# Query total_right – query(rank[num])
instead."
```


Binary Search

#410 Split Array Largest Sum

HAR[Open on LeetCode](#)

Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split. A subarray is a contiguous part of the array.

Example 1:

Input: `nums = [7,2,5,10,8]`, `k = 2`

Output: 18

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

Example 2:

Input: `nums = [1,2,3,4,5]`, `k = 2`

Output: 9

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[1,2,3]` and `[4,5]`, where the largest sum among the two subarrays is only 9.

Constraints:

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 106`
- `1 <= k <= min(50, nums.length)`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Split `nums` into `k` non-empty contiguous subarrays to minimise the maximum subarray sum.

Must be in order – no reordering allowed. Right?

Same 'binary search on answer' pattern as Capacity to Ship Packages (#1011)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Try every split of nums into k contiguous groups (DP over partition points).

For each split, max subarray sum = largest group sum; minimize that over splits.

State space explodes without binary search on the answer.

Time: exponential / $O(n^k)$ naive partitions. Space: $O(n)$ recursion.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search on the answer (the max sum)."

$lo = \max(\text{nums})$ (at least one element),
 $hi = \text{sum}(\text{nums})$ (one subarray).

Feasibility check: can we split into k subarrays with each sum $\leq mid$?

Greedy: accumulate greedily, start new subarray when exceeding mid. Count splits.

Time: $O(n \log(\text{sum}))$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _410_SplitArrayLargestSum:
    def splitArray(self, nums: List[int],
k: int) -> int:
    def can_split(max_sum: int) ->
bool:
        subarrays = 1
        current = 0
        for num in nums:
            if current + num >
max_sum:
                subarrays += 1
                current = 0
            current += num
        return subarrays <= k
    lo, hi = max(nums), sum(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_split(mid):
            hi = mid # valid - try
smaller
        else:
            lo = mid + 1 # not valid -
need bigger
    return lo

# PHASE 5 — TEST & DEBUG
```

```
# [7,2,5,10,8], k=2: lo=10, hi=32.  
mid=21→can_split? [7,2,5] and  
[10,8]=18≤21 ✓ → hi=21.  
# ... converges to 18.  
can_split(18)=[7,2,5][10,8]→2 splits≤k=2  
✓. can_split(17)→[7,2,5][10,8]→10+8=18>17  
→need 3>k ✗. → 18 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log(\text{sum}))$. Space $O(1)$. Same template as #1011 Capacity to Ship – name this.

lo/hi bounds: must be at least $\max(\text{nums})$, at most $\text{sum}(\text{nums})$.

Given more time I'd ask: DP approach?
 $\text{dp}[j][i]$ = min largest sum splitting first i elements into j parts.

$O(kn^2)$ – worse than binary search for large inputs."

Binary Search

□ #668 Kth Smallest Number in Multiplication Table [Open on LeetCode](#)

HAR

Math · Binary Search

Lists: Denny Zhang

Problem

Nearly everyone has used the Multiplication Table. The multiplication table of size $m \times n$ is an integer matrix mat where $mat[i][j] == i * j$ (1-indexed).

Given three integers m , n , and k , return the k th smallest element in the $m \times n$ multiplication table.

Example 1:

1	2	3
2	4	6
3	6	9

1	2	2	3	3	4	6	6	9
---	---	---	---	---	---	---	---	---

Input: $m = 3$, $n = 3$, $k = 5$

Output: 3

Explanation: The 5th smallest number is 3.

Example 2:

1	2	3
2	4	6

1	2	2	3	4	6
---	---	---	---	---	---

Input: $m = 2$, $n = 3$, $k = 6$

Output: 6

Explanation: The 6th smallest number is 6.

Constraints:

- $1 \leq m, n \leq 3 \cdot 10^4$
- $1 \leq k \leq m \cdot n$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"m×n multiplication table:

mat[i][j]=i*j (1-indexed). Find kth

smallest element. Right?

Can't materialise the table (m,n up to $3 \times 10^4 \rightarrow$ up to 9×10^8 cells)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all $m \times n$ products into a list, sort, return kth element.

Correct but materializes every product – $O(mn \log(mn))$ time and $O(mn)$ space.

Time: $O(m * n * \log(m * n))$. Space: $O(m * n)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (binary search on value)

"Binary search on the answer value v.

Count elements $\leq v$: for row i, elements are $i, 2i, 3i, \dots, n \times i$.

Count of these $\leq v$ is $\min(v // i, n)$. Sum over all rows $i=1..m$.

Binary search for smallest v where count $\geq k$.

lo=1, hi=m*n. Time: $O(m \log(m*n))$."

PHASE 4 — CLEAN CODE

class _668_KthSmallestMultTable:

```

def findKthNumber(self, m: int, n:
int, k: int) -> int:
    def count_le(v: int) -> int:
        return sum(min(v // i, n) for
i in range(1, m + 1))
    lo, hi = 1, m * n
    while lo < hi:
        mid = (lo + hi) // 2
        if count_le(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo

```

PHASE 5 — TEST & DEBUG

```

# m=3,n=3,k=5. Table: [1,2,3,2,4,6,3,6,9]
sorted=[1,2,2,3,3,4,6,6,9]. 5th=3.
# count_le(3)=count≥5? i=1:min(3,3)=3,
i=2:min(1,3)=1, i=3:min(1,3)=1. sum=5≥5 →
hi=3.
# ... lo converges to 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(m \log(m*n))$ . Space  $O(1)$ . The
count formula is the key –  $O(m)$  per binary
search step.

```

Note: the answer might not be a 'real' table entry – binary search finds the smallest v

with count $\geq k$, which IS always a table entry because count jumps discretely at table values.

Given more time I'd ask: kth largest? Binary search on $m \times n - k + 1$ smallest instead."

Binary Search

□ #710 Random Pick with Blacklist

HAR[Open on LeetCode](#)

Array · Hash Table · Math · Binary Search ·
Sorting · Randomized

Lists: Denny Zhang

Problem

You are given an integer n and an array of unique integers `blacklist`. Design an algorithm to pick a random integer in the range $[0, n - 1]$ that is not in `blacklist`. Any integer that is in the mentioned range and not in `blacklist` should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the `Solution` class:

- `Solution(int n, int[] blacklist)` Initializes the object with the integer n and the blacklisted integers `blacklist`.
- `int pick()` Returns a random integer in the range $[0, n - 1]$ and not in `blacklist`.

Example 1:

Input

```
["Solution", "pick", "pick", "pick",  
"pick", "pick", "pick", "pick"]  
[[7, [2, 3, 5]], [], [], [], [], [],  
[], []]
```

Output

```
[null, 0, 4, 1, 6, 1, 0, 4]
```

Explanation

```
Solution solution = new Solution(7, [2,  
3, 5]);
```

Constraints:

- $1 \leq n \leq 109$
- $0 \leq \text{blacklist.length} \leq \min(105, n - 1)$
- $0 \leq \text{blacklist}[i] < n$
- All the values of blacklist are unique.
- At most $2 * 10^4$ calls will be made to pick.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Pick uniformly at random from $[0, n-1]$ excluding blacklist. One call to random

per pick. Right?

n can be 10^9 – can't enumerate all valid numbers."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all non-blacklisted ids in a list; pick random index.

If blacklist is small, filter on each pick – $O(n)$ per draw.

Time: $O(n)$ per pick naive. Space: $O(n)$ whitelist.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (virtual mapping)

"Let $sz = n - \text{len}(\text{blacklist})$ = count of valid numbers.

Map $[0, sz)$ as the 'virtual' range.

Numbers in $[0, sz)$ that ARE blacklisted

get remapped to non-blacklisted numbers in $[sz, n)$.

pick(): random in $[0, sz) \rightarrow$ if in remap $\rightarrow$ use remap[r], else use r directly.

Build remap in `__init__`: for each blacklisted number in $[0, sz)$, map it to a

non-blacklisted

number in [sz, n). O(|blacklist|) init,
O(1) per pick."

PHASE 4 — CLEAN CODE

```
class _710_RandomPickBlacklist:
    def __init__(self, n: int, blacklist:
List[int]):
        bl_set = set(blacklist)
        self.sz = n - len(blacklist) #
size of valid virtual range
        self.remap: dict = {}
        tail = n - 1
        for b in blacklist:
            if b < self.sz: # this
blacklisted number is in [0, sz)
                while tail in bl_set: #
find next valid number from the tail
                    tail -= 1
                self.remap[b] = tail
                tail -= 1
        def pick(self) -> int:
            r = random.randint(0, self.sz - 1)
            return self.remap.get(r, r)
```

PHASE 5 — TEST & DEBUG

```
# n=7, blacklist=[2,3,5]. sz=4. Valid:
[0,1,4,6].
# Blacklisted in [0,4): 2,3. Map 2→6, 3→4
(5 is blacklisted, skip → 4).
# remap={2:6, 3:4}. pick(0)→0, pick(1)→1,
pick(2)→6, pick(3)→4. All valid ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Init O(|B|). Pick O(1). Space O(|B|)
for remap.
```

```
# The key insight: divide [0,n) into
'virtual valid' [0,sz) and 'tail' [sz,n).
# Blacklisted numbers in [0,sz) redirect
to valid numbers in the tail.
# Given more time I'd ask: what if the
blacklist updates dynamically?
# Rebuild the remap on each update –
O(|B|) per update, same O(1) pick."
```


Binary Search

□ #774 Minimize Max Distance to Gas Station

HAR[Open on LeetCode](#)

Array · Binary Search

Lists: Denny Zhang

Problem

You are given an integer array `stations` that represents the positions of the gas stations on the x -axis. You are also given an integer k .

You should add k new gas stations. You can add the stations anywhere on the x -axis, and not necessarily on an integer position.

Let `penalty()` be the maximum distance between adjacent gas stations after adding the k new stations.

Return the smallest possible value of `penalty()`. Answers within 10^{-6} of the actual answer will be accepted.

Example 1:

```
Input: stations =  
[1,2,3,4,5,6,7,8,9,10], k = 9  
Output: 0.50000
```

Example 2:

```
Input: stations =  
[23,24,36,39,46,56,57,65,84,98], k = 1  
Output: 14.00000
```

Constraints:

- $10 \leq \text{stations.length} \leq 2000$
- $0 \leq \text{stations}[i] \leq 108$
- `stations` is sorted in a strictly increasing order.
- $1 \leq k \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Add exactly `k` stations anywhere (real-valued positions) to minimise the maximum gap.

Return the answer within $1e-6$ precision. Right?"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Binary search on the maximum distance
between adjacent stations.
# For candidate d, check if we can add k
stations so all gaps <= d (greedy fill).
# Brute: try every real-valued spacing
without binary search – continuous search
space.
# Time:  $O(n * \text{precision})$  naive scan.
Space:  $O(1)$ .
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (binary search on
floating-point answer)
# "Binary search on the max gap d.
Feasibility: for each existing gap g, we
need  $\text{ceil}(g/d) - 1$  extra
# stations to keep all sub-gaps  $\leq d$ . Count
total extras; if  $\leq k$ , d is achievable.
# Precision: run until  $\text{hi-lo} < 1e-7$ ."

# PHASE 4 — CLEAN CODE
class _774_MinMaxDistanceGasStation:
    def minmaxGasDist(self, stations:
List[int], k: int) -> float:
```

```
        gaps = [stations[i+1] -
stations[i] for i in
range(len(stations)-1)]
        def can_achieve(d: float) ->
bool:
            return sum(int(g / d) for g in
gaps) <= k
        lo, hi = 0.0, max(gaps)
        for _ in range(100): # 100
iterations -> precision < hi/2^100
            mid = (lo + hi) / 2
            if can_achieve(mid):
                hi = mid
            else:
                lo = mid
        return hi
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * 100) = O(n)$. Space $O(n)$ for gaps. Floating-point binary search with fixed iterations.

Note: $\text{int}(g/d)$ gives the number of stations to insert (not $\text{ceil}-1$ – same thing via floor division).

Given more time I'd ask: what if stations must be at integer positions?

Binary search on integers, same feasibility check but with `math.ceil`."

Binary Search

□ #1235 Maximum Profit in Job Scheduling **HAR**

[Open on LeetCode](#)

Array · Binary Search · Dynamic Programming · Sorting

Lists: Grind 169

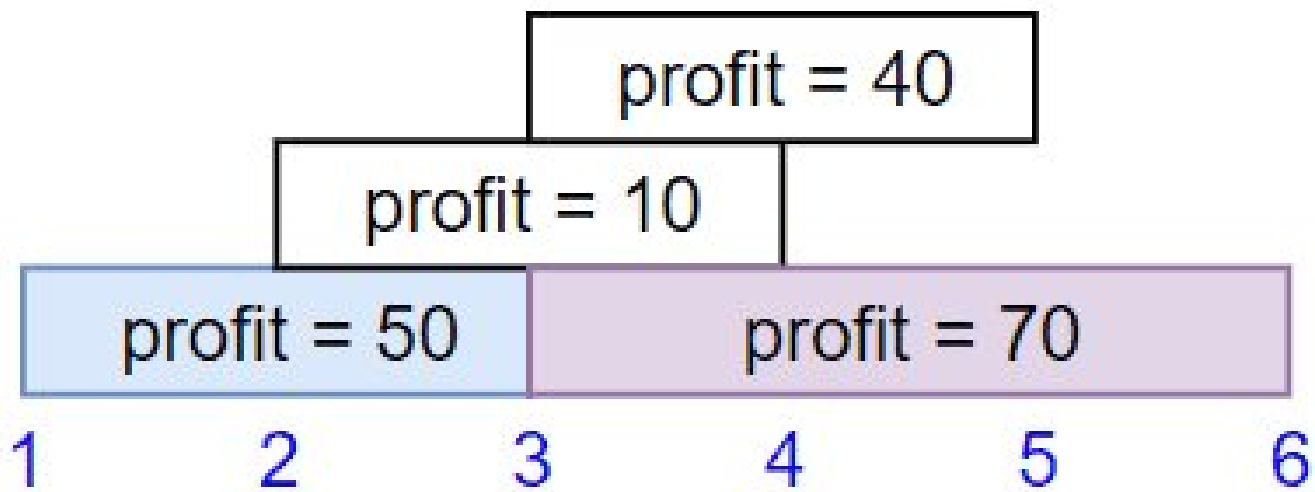
Problem

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:



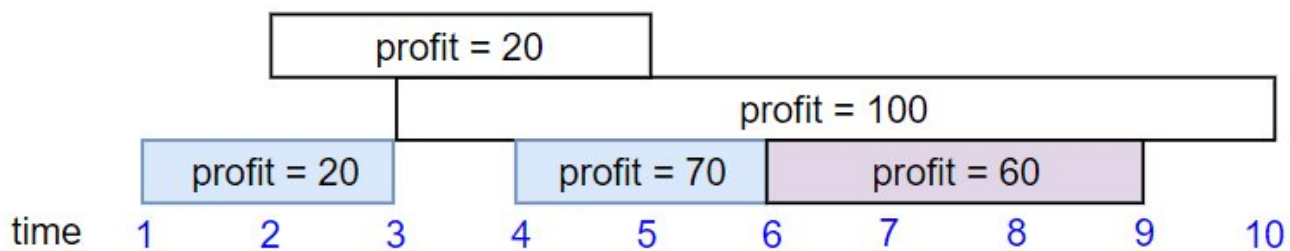
Input: `startTime = [1,2,3,3]`, `endTime = [3,4,5,6]`, `profit = [50,10,40,70]`

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range $[1-3] + [3-6]$, we get profit of $120 = 50 + 70$.

Example 2:



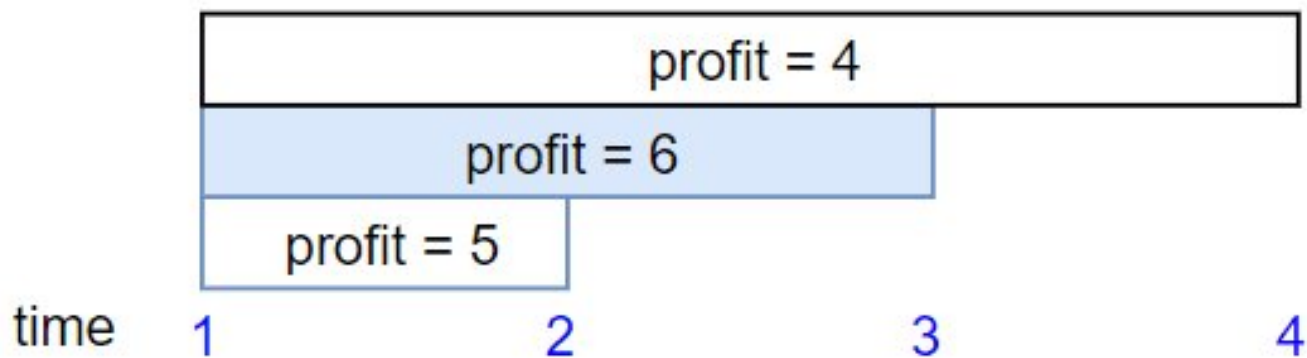
Input: `startTime = [1,2,3,4,6]`, `endTime = [3,5,10,6,9]`, `profit = [20,20,100,70,60]`

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job.

Profit obtained $150 = 20 + 70 + 60$.

Example 3:



Input: `startTime = [1,1,1]`, `endTime = [2,3,4]`, `profit = [5,6,4]`

Output: 6

Constraints:

- $1 \leq \text{startTime.length} == \text{endTime.length} == \text{profit.length} \leq 5 * 10^4$
- $1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9$
- $1 \leq \text{profit}[i] \leq 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Select non-overlapping jobs to maximise total profit.

Job ending at X and starting at X don't overlap. Return max profit. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort jobs by end time. DP over index:
dp[i] = max profit taking job i plus best non-overlapping before i.

For each i, scan all j < i for compatibility – O(n²) DP.

Time: O(n²). Space: O(n) DP + sort.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (sort by end time + DP + binary search)

"Sort jobs by end time. dp[i] = max profit using first i jobs.

For each job i: skip → dp[i] = dp[i-1].
Take → find latest j where job[j].end ≤ job[i].start

```
# using binary search. dp[i] =  
max(dp[i-1], dp[j] + profit[i]).  
# Time: O(n log n)."
```

PHASE 4 — CLEAN CODE

```
class _1235_MaxProfitJobScheduling:  
    def jobScheduling(self, startTime:  
List[int], endTime: List[int], profit:  
List[int]) -> int:  
        jobs = sorted(zip(endTime,  
startTime, profit))  
        ends = [0] # dp_ends[i] = end time  
of ith job in dp  
        dp = [0] # dp[i] = max profit for  
first i sorted jobs  
        for end, start, p in jobs:  
            # Find latest job whose end <=  
start of current  
            j = bisect.bisect_right(ends,  
start) - 1  
            if dp[j] + p > dp[-1]:  
                dp.append(dp[j] + p)  
                ends.append(end)  
            else:  
                dp.append(dp[-1])  
                ends.append(end)
```

```
return dp[-1]
```

PHASE 5 — TEST & DEBUG

```
# jobs sorted by end:
```

```
[(3,1,50),(4,2,10),(5,3,40),(6,3,70)].
```

```
# job(3,1,50): j=bisect_right([0],1)-1=0.
```

```
dp[0]+50=50>dp[-1]=0 →
```

```
dp=[0,50],ends=[0,3].
```

```
# job(4,2,10): bisect_right([0,3],2)-1=0.
```

```
dp[0]+10=10<dp[-1]=50 →
```

```
dp=[0,50,50],ends=[0,3,4].
```

```
# job(5,3,40):
```

```
bisect_right([0,3,4],3)-1=1.
```

```
dp[1]+40=90>50 → dp=[0,50,50,90].
```

```
# job(6,3,70):
```

```
bisect_right([0,3,4,5],3)-1=1.
```

```
dp[1]+70=120>90 → dp=[0,50,50,90,120]. →  
120 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$ . Space  $O(n)$ . Sort + DP  
with binary search per step.
```

```
# The dp array is monotonically  
non-decreasing – binary search on ends is  
valid.
```

```
# Given more time I'd ask: what if we want  
to select at most k jobs?  
# Add a dimension to dp – dp[i][k] –  
O(n²k) with the same binary search for  
last compatible job."
```

Matrix

Round 3 · High · Hard · 1 question

Matrix

□ #1074 Number of Submatrices That Sum to Target

HAR[Open on LeetCode](#)

Array · Hash Table · Matrix · Prefix Sum

Lists: Denny Zhang

Problem

Given a matrix and a target, return the number of non-empty submatrices that sum to target. A submatrix $x1, y1, x2, y2$ is the set of all cells $matrix[x][y]$ with $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Two submatrices $(x1, y1, x2, y2)$ and $(x1', y1', x2', y2')$ are different if they have some coordinate that is different: for example, if $x1 \neq x1'$.

Example 1:

0	1	0
1	1	1
0	1	0

Input: matrix =
[[0,1,0],[1,1,1],[0,1,0]], target = 0

Output: 4

Explanation: The four 1x1 submatrices
that only contain 0.

Example 2:

Input: matrix = [[1,-1],[-1,1]], target = 0

Output: 5

Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.

Example 3:

Input: matrix = [[904]], target = 0

Output: 0

Constraints:

- $1 \leq \text{matrix.length} \leq 100$
- $1 \leq \text{matrix}[0].\text{length} \leq 100$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$
- $-10^8 \leq \text{target} \leq 10^8$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count all submatrices (r1,c1,r2,c2) whose elements sum to target. Right?"

Extend #560 Subarray Sum Equals K to 2D."

PHASE 2 — BRUTE FORCE


```
# "Let me start with brute force to lock
in the logic.
# Compute 2D prefix sums, then enumerate
all submatrix top-left and bottom-right
corners.
# For each submatrix,  $O(1)$  range sum check
against target.
# Four nested loops over corners –  $O(n^2 * m^2)$ 
submatrices.
# Time:  $O(n^2 * m^2)$ . Space:  $O(n * m)$ 
prefix grid.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Fix top row r1 and bottom row r2.
Compress columns into 1D prefix sums
between r1..r2.
# Then apply #560 (subarray sum = target)
on the 1D array.
# Time:  $O(m^2 * n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _1074_SubmatricesSumTarget:
    def numSubmatrixSumTarget(self,
matrix: List[List[int]], target: int) ->
int:
```

```

        m, n = len(matrix),
len(matrix[0])
        count = 0
        for r1 in range(m):
            col_sum = [0] * n
            for r2 in range(r1, m):
                for c in range(n):
                    col_sum[c] +=
matrix[r2][c]

            # Apply #560 on col_sum
            prefix = Counter({0: 1})
            running = 0
            for s in col_sum:
                running += s
                count +=
prefix[running - target]
                prefix[running] += 1
        return count

```

PHASE 5 — TEST & DEBUG

```

# [[0,1,0],[1,1,1],[0,1,0]], target=0:
# Fix r1=0,r2=0: col_sum=[0,1,0].
Subarrays summing to 0: [0] at c=0, [0] at
c=2. count=2.
# Other rows contribute 2 more. → 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m^2 * n)$. Space $O(n)$. This is literally #560 applied inside a 2D row-fixing loop.

Name that connection – it shows pattern transfer.

Given more time I'd ask: count submatrices summing to at most target?

Use #862 (Shortest Subarray with Sum at Least K) approach – monotone deque instead of hash map."

Trees & BST

Round 3 · High · Hard · 2 questions

Trees & BST

□ #124 Binary Tree Maximum Path Sum

HAR[Open on LeetCode](#)

Dynamic Programming · Tree · DFS

Lists: Grind 169

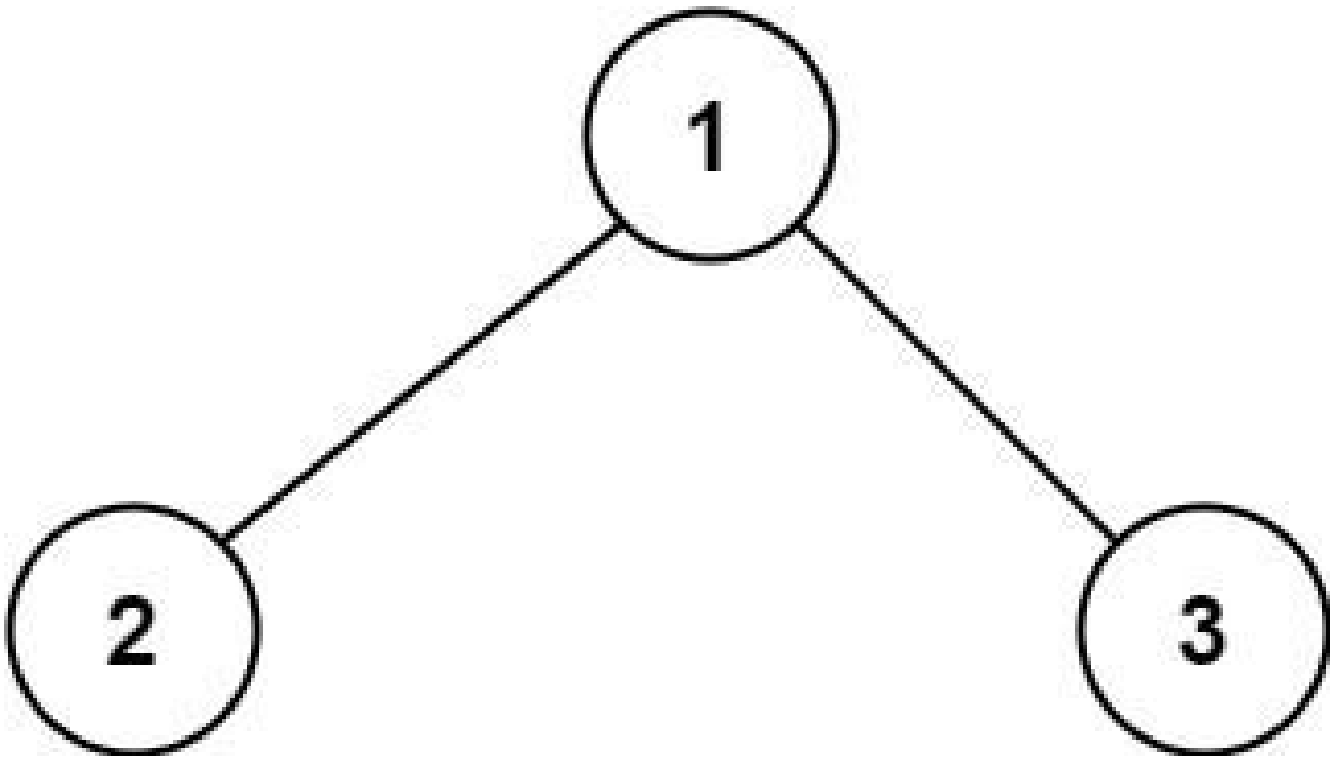
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:

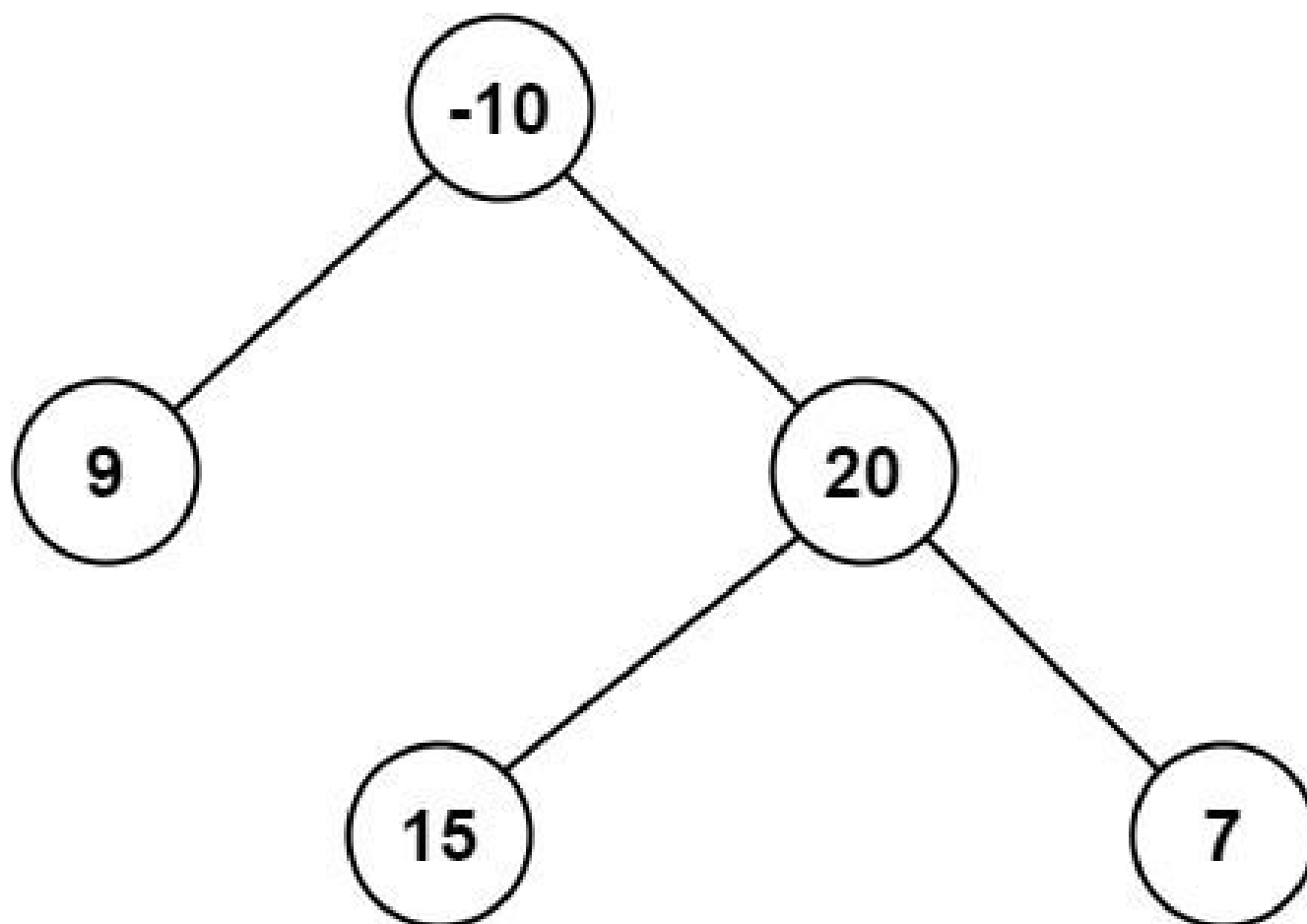


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of $15 + 20 + 7 = 42$.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Path = any sequence of connected nodes (no revisiting). Path doesn't need to pass through root.

Values can be negative. Return maximum path sum. Right?"

#

"[-10,9,20,null,null,15,7]: path 15→20→7 = 42 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

At every node, compute max path sum through that node = $\text{node.val} + \max(0, \text{left_gain}) + \max(0, \text{right_gain})$.

Recurse full tree; track global max. Recomputes subtree gains repeatedly without sharing.

Time: $O(n^2)$ worst case on skewed tree. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (post-order, same as Diameter)


```
# "For each node, compute the 'gain' if we
extend a path through it: max(0,
left_gain, right_gain) + node.val.
# The best path THROUGH this node =
node.val + max(0, left) + max(0, right).
# Update global max. Return one-sided gain
to parent (can't go both left and right
AND up).
# Use inner helper – no self."
```

PHASE 4 — CLEAN CODE

```
class _124_MaxPathSum:
    def maxPathSum(self, root:
Optional['TreeNode']) -> int:
        best = [float('-inf')]
        def gain(node):
            if not node: return 0
            left = max(0,
gain(node.left))
            right = max(0,
gain(node.right))
            best[0] = max(best[0],
node.val + left + right)
            return node.val + max(left,
right)
        gain(root)
```

return best[0]

PHASE 5 — TEST & DEBUG

```
# [-10,9,20,null,null,15,7]:  
# gain(9)=9. gain(15)=15. gain(7)=7.  
# gain(20): left=15,right=7.  
best=max(-inf,20+15+7)=42. return  
20+15=35.  
# gain(-10): left=9,right=35.  
best=max(42,-10+9+35)=42. return  
-10+35=25.  
# → 42 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h). Same post-order  
pattern as Diameter of Binary Tree (#543)  
and Balanced Tree (#110).  
# The 'max(0, gain)' is essential – a  
negative subtree gain should be dropped.  
# Given more time I'd ask: return the  
actual path (not just the sum)?  
# Track which path achieved best[0] –  
store node references alongside the max  
update."
```

Trees & BST

□ #297 Serialize and Deserialize Binary Tree

HAR[Open on LeetCode](#)

String · Tree · DFS · BFS · Design

Lists: Grind 169

Problem

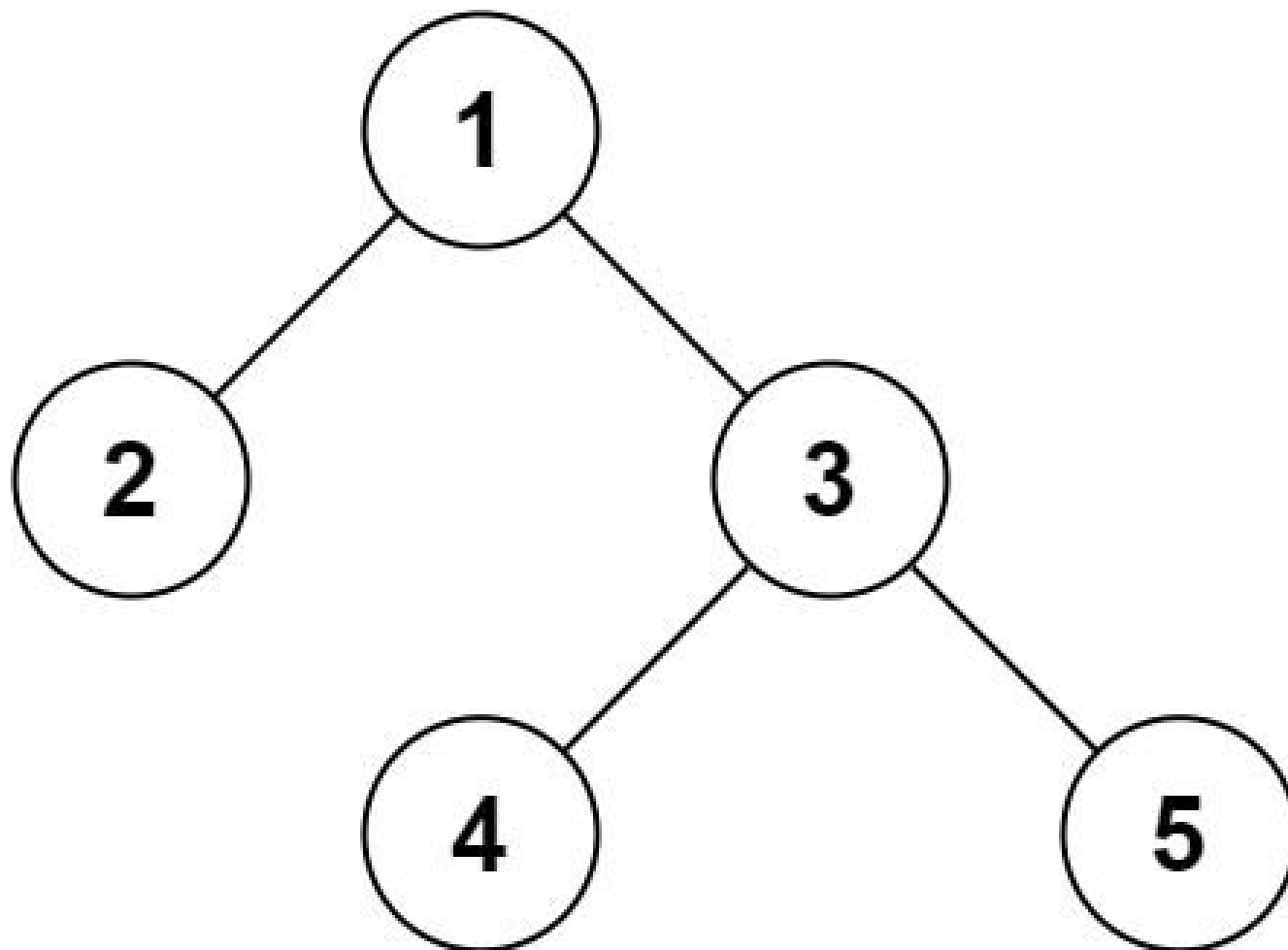
Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this

format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design encode and decode functions for a binary tree. Any format is fine as long as it round-trips.

Empty tree encodes to some valid string. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Serialize: preorder traversal with 'null' markers for empty children ? comma string.

Deserialize: split queue, rebuild tree recursively matching preorder order.

Correct but string can be long; BFS level-order encoding is more compact.

Time: $O(n)$. Space: $O(n)$ output string.

Should I go ahead and optimize?"

```
# PHASE 3 — OPTIMIZE (pre-order DFS with null markers)
# "Serialize: pre-order traversal, 'N' for null, comma-separated.
# Deserialize: split, use an iterator, recursively build from pre-order.
# Pre-order uniquely determines the tree when nulls are included."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _297_Codec:
    def serialize(self, root: 'TreeNode')
-> str:
        parts: List[str] = []
        def dfs(node):
            if not node:
                parts.append('N'); return
            parts.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ','.join(parts)
    def deserialize(self, data: str) ->
'TreeNode':
        vals = iter(data.split(','))
        def build():
```

```
        v = next(vals)
        if v == 'N': return None
        node = TreeNode(int(v))
        node.left = build()
        node.right = build()
        return node
    return build()
```

PHASE 5 — TEST & DEBUG

[1,2,3,null,null,4,5]: serialize →
'1,2,N,N,3,4,N,N,5,N,N'. deserialize
rebuilds the tree ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Pre-order with
null markers is the cleanest approach.

BFS alternative: level-order like
LeetCode's own format – works but trickier
to parse.

Given more time I'd ask: serialize a
general N-ary tree?

Add child count before children – e.g.
'1,3,2,0,3,0,4,0,5,0' for a 3-child
root."

DFS

Round 3 · High · Hard · 5 questions

DFS

□ ★ #269 Alien Dictionary ★

HAR[Open on LeetCode](#)

Array · String · DFS · BFS · Graph · Topological Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions,

return any of them.

Example 1:

```
Input: words =  
["wrt","wrf","er","ett","rftt"]  
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]  
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]  
Output: ""  
Explanation: The order is invalid, so  
return "".
```

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- $\text{words}[i]$ consists of only lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given words sorted in alien lexicographic order, deduce the letter ordering.

Return any valid ordering, or '' if impossible (cycle or contradiction).

Right?

Contradiction: longer word precedes its prefix (e.g. ['ab', 'a'])."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build graph: for each adjacent pair in words, record char order u before v.

Topological sort the chars; if cycle detected, no valid order.

Compare words character by character to infer edges.

Time: $O(C)$ total chars. Space: $O(1)$ – 26 letters.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Kahn's BFS topological sort)

"Extract ordering constraints from consecutive word pairs: compare char by

char, first difference gives an edge.

```
# Build directed graph + in-degree map.
# BFS: process zero-in-degree nodes. If we
process all letters → valid. Else cycle →
return ''."
```

PHASE 4 — CLEAN CODE

```
class _269_AlienDictionary:
    def alienOrder(self, words:
List[str]) -> str:
        graph = defaultdict(set)
        indegree = {c: 0 for w in words
for c in w}
        for i in range(len(words) - 1):
            w1, w2 = words[i], words[i +
1]

            min_len = min(len(w1),
len(w2))

            if len(w1) > len(w2) and
w1[:min_len] == w2[:min_len]:
                return '' # invalid:
prefix ordering
            for c1, c2 in zip(w1, w2):
                if c1 != c2:
                    if c2 not in
graph[c1]:
```

```

graph[c1].add(c2)
indegree[c2] += 1
break
queue = deque(c for c in indegree
if indegree[c] == 0)
result = []
while queue:
    c = queue.popleft()
    result.append(c)
    for nb in graph[c]:
        indegree[nb] -= 1
        if indegree[nb] == 0:
            queue.append(nb)
return ''.join(result) if
len(result) == len(indegree) else ''

```

PHASE 5 — TEST & DEBUG

```

# ["wrt","wrf","er","ett","rftt"]: edges:
t→f, w→e, r→t, e→r. indegree:
f:1,e:1,t:1,r:1,w:0.
# BFS from w: w→e→r→t→f. result='wertf' ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(C)$  where  $C$  = total characters.
Space  $O(1)$  — at most 26 letters.

```

```
# The prefix check catches the only  
invalid case that doesn't manifest as a  
graph cycle.  
# Given more time I'd ask: return ALL  
valid orderings?  
# Backtracking on the topological sort –  
collect all linearisations."
```

DFS

□ #329 Longest Increasing Path in a Matrix **HAR**

[Open on LeetCode](#)

Array · DFS · BFS · Graph · Topological Sort · Memoization · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

9 ↑	9	4
6 ↑	6	8
2 ←	1	1

Input: matrix =
[[9,9,4],[6,6,8],[2,1,1]]

Output: 4

Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

3 →	4 →	5 ↓
3	2	6
2	2	1

Input: matrix =
[[3,4,5],[3,2,6],[2,2,1]]

Output: 4

Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.

Example 3:

Input: `matrix = [[1]]`

Output: `1`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- `0 <= matrix[i][j] <= 231 - 1`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest strictly increasing path (4-directional). Return path length. Right?"

Values can repeat but path must be strictly increasing."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS from every cell with memo: longest increasing path starting at (r,c).

Only move to strictly larger neighbors; memo avoids recomputing subpaths.

```
# Without memo, pure DFS retries paths
exponentially.
# Time:  $O(m * n)$  with memo. Space:  $O(m * n)$  memo + stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DFS + memoization)
# "DFS from each cell.  $dp[r][c]$  = longest
path starting at  $(r, c)$ .
# Only move to neighbours with strictly
greater value – no cycles possible → no
visited set needed.
# Memoize results: each cell computed once
→  $O(m*n)$ ."

# PHASE 4 — CLEAN CODE
class _329_LongestIncPathMatrix:
    def longestIncreasingPath(self,
matrix: List[List[int]]) -> int:
        m, n = len(matrix),
len(matrix[0])
        memo = {}
        def dfs(r, c):
            if (r, c) in memo: return
memo[(r, c)]
            best = 1
```

```

        for dr, dc in
            [(-1,0), (1,0), (0,-1), (0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc
< n and matrix[nr][nc] > matrix[r][c]:
                    best = max(best,
dfs(nr, nc) + 1)
                memo[(r, c)] = best
            return best
        return max(dfs(r, c) for r in
range(m) for c in range(n))

```

PHASE 5 — TEST & DEBUG

`[[9,9,4],[6,6,8],[2,1,1]]`: longest path starting at `(2,1)=1`: `1→2→6→9=4` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ for memo. No visited set needed – strict increase prevents cycles.

Topological sort alternative: build DAG (only edges to larger neighbours), count levels.

Same $O(m*n)$ but iterative.

Given more time I'd ask: longest path with values differing by at most 1?

Need a visited set now (can revisit same value) – different problem."

DFS

□ #685 Redundant Connection II

HAR[Open on LeetCode](#)

DFS · BFS · Union Find · Graph

Lists: Denny Zhang

Problem

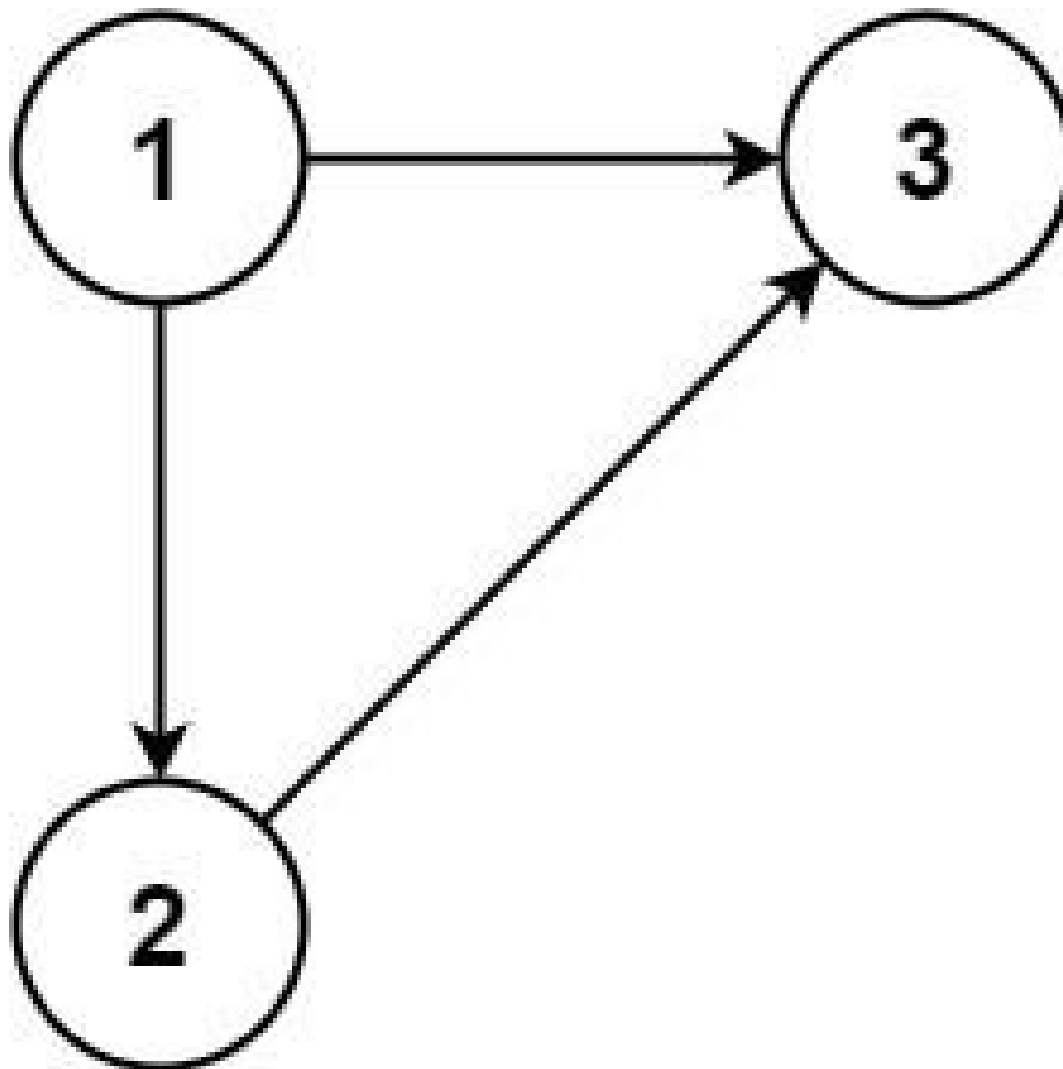
In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u_i, v_i]$ that represents a directed edge connecting nodes u_i and v_i , where u_i is a parent of child v_i .

Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

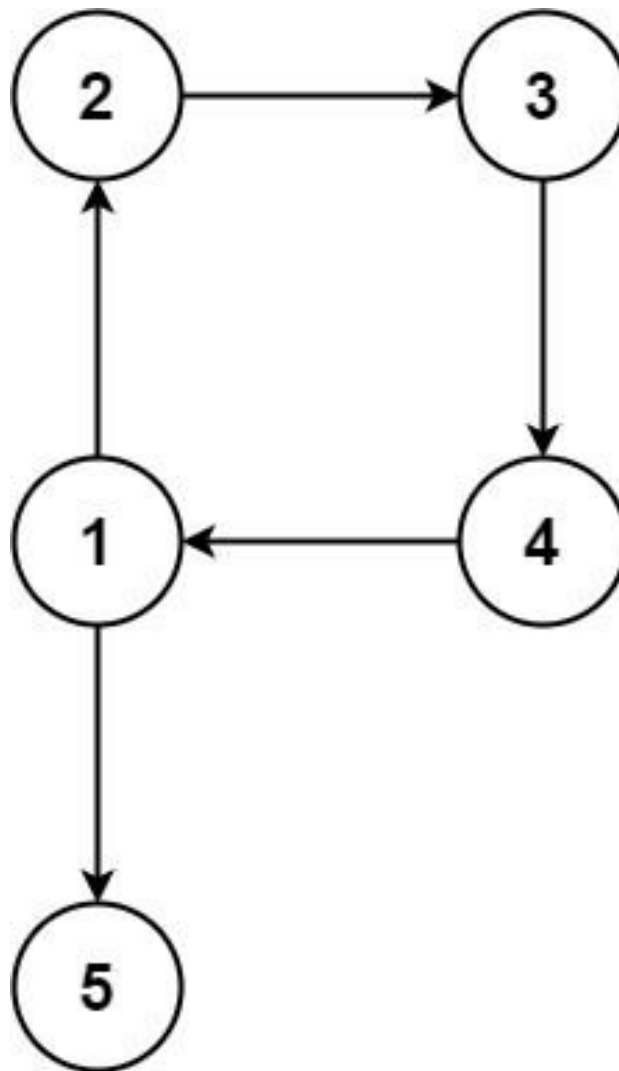
Example 1:



Input: edges = `[[1,2],[1,3],[2,3]]`

Output: `[2,3]`

Example 2:



Input: edges =
[[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq u_i, v_i \leq n$

- $u_i \neq v_i$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Directed graph: rooted tree + one extra edge. Find the edge to remove.

If multiple answers, return the last one in input. Right?

Extra edge causes: (1) a node with two parents, OR (2) a cycle, OR both."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build indegree + parent map from edges. Node with indegree 2 is the duplicate child candidate.

Detect cycle with DFS or union-find on the graph minus one suspicious edge.

Try removing each duplicate edge until graph becomes a valid tree.

Time: $O(n)$ with careful case analysis. Space: $O(n)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (three-case analysis + Union-Find)

"Track in-degree: if any node has two parents, record both candidate edges.

Then run Union-Find ignoring the second parent candidate.

Case 1 (two parents, no cycle in remaining): remove second candidate.

Case 2 (two parents, cycle in remaining): remove first candidate.

Case 3 (no node with two parents): cycle exists → last edge forming cycle."

PHASE 4 — CLEAN CODE

```
class _685_RedundantConnectionII:
    def
```

```
findRedundantDirectedConnection(self,
edges: List[List[int]]) -> List[int]:
```

```
    n = len(edges)
```

```
    parent = list(range(n + 1))
```

```
    cand1 = cand2 = None
```

```
    # Find if any node has two parents
```

```
    parent_map = {}
```

```
    for u, v in edges:
```

```
        if v in parent_map:
```

```
        cand1 = parent_map[v] #
first edge giving v a parent
        cand2 = [u, v] # second
edge giving v another parent
        break
    parent_map[v] = [u, v]
def find(x):
    while parent[x] != x:
        parent[x] =
parent[parent[x]]
        x = parent[x]
    return x
def union(u, v) -> bool:
    pu, pv = find(u), find(v)
    if pu == pv: return False
    parent[pu] = pv
    return True
parent = list(range(n + 1))
for u, v in edges:
    if [u, v] == cand2: # skip the
second candidate
        continue
    if not union(u, v):
        return cand1 if cand1
else [u, v]
```

return cand2

PHASE 5 — TEST & DEBUG

`[[1,2],[1,3],[2,3]]`: node 3 has two parents (1 and 2). `cand1=[1,3]`, `cand2=[2,3]`.

UF ignoring `cand2=[2,3]`: `union(1,2)` ok, `union(1,3)` ok. No cycle → return `cand2=[2,3]` ✓

`[[1,2],[2,3],[3,4],[4,1],[1,5]]`: no node with two parents. UF finds cycle `[4,1]` → return `[4,1]` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * \alpha(n))$. Space $O(n)$. Three cases arise from two root causes: double parent and cycle.

The Union-Find handles both: cycle detection when `cand2` is skipped tells us which candidate to remove.

Given more time I'd ask: undirected version?

That's Redundant Connection (#684) – standard Union-Find, return first edge that creates a cycle."

DFS

□ #1036 Escape a Large Maze

HAR[Open on LeetCode](#)

Array · Hash Table · DFS · BFS

Lists: Denny Zhang

Problem

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y) .

We start at the source = $[sx, sy]$ square and want to reach the target = $[tx, ty]$ square. There is also an array of blocked squares, where each $blocked[i] = [xi, yi]$ represents a blocked square with coordinates (xi, yi) .

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1:

Input: `blocked = [[0,1],[1,0]]`, `source = [0,0]`, `target = [0,2]`

Output: `false`

Explanation: The target square is inaccessible starting from the source square because we cannot move.

We cannot move north or east because those squares are blocked.

We cannot move south or west because we cannot go outside of the grid.

Example 2:

Input: `blocked = []`, `source = [0,0]`, `target = [999999,999999]`

Output: `true`

Explanation: Because there are no blocked cells, it is possible to reach the target square.

Constraints:

- `0 <= blocked.length <= 200`
- `blocked[i].length == 2`
- `0 <= xi, yi < 106`
- `source.length == target.length == 2`
- `0 <= sx, sy, tx, ty < 106`

- source != target
- It is guaranteed that source and target are not blocked.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"1M×1M grid. Up to 200 blocked cells.
Can source reach target? Right?"

With at most 200 blocked cells in a semicircle, max enclosed area $\approx 200^2/2 = 20,000$ cells.

So if BFS from source visits $> 20,000$ cells without finding target, source is NOT enclosed."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS from start cell; at each step try all 4 directions.

Track visited cells; if we revisit within ≤ 500 moves, we are in a loop ? false.

If we escape bounds within 500 steps per problem rule ? true.

Time: $O(m * n * 500)$ worst case. Space: $O(m * n)$ visited.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with cell limit)

"Run limited BFS from BOTH source and target.

If BFS visits $> \text{max_cells}$ ($= n*(n+1)//2$ for $n=\text{len}(\text{blocked})$) $\rightarrow$ not enclosed $\rightarrow$ True from that side.

If BFS terminates (all cells explored) without reaching other endpoint $\rightarrow$ enclosed $\rightarrow$ False."

PHASE 4 — CLEAN CODE

class _1036_EscapeLargeMaze:

 def isEscapePossible(self, blocked: List[List[int]], source: List[int], target: List[int]) $\rightarrow$ bool:

 if not blocked:

 return True

 n = len(blocked)

 max_cells = $n * (n + 1) // 2$

 blocked_set = {(r, c) for r, c in blocked}

 def bfs(start, end):


```
queue = deque([tuple(start)])
visited = {tuple(start)}
while queue:
    r, c = queue.popleft()
    if [r, c] == end:
        return True
    if len(visited) >
max_cells:
        return True # too
many cells explored → not enclosed
        for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r + dr, c +
dc
            if 0 <= nr < 10**6 and
0 <= nc < 10**6 and (nr,nc) not in
blocked_set and (nr,nc) not in visited:
                visited.add((nr,
nc))
                queue.append((nr,
nc))
        return False
    return bfs(source, target) and
bfs(target, source)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$ per BFS (at most $n*(n+1)/2$ cells). Space $O(n^2)$. $n \leq 200 \rightarrow$ fast.
The dual-BFS is essential: source might not be enclosed but target might be.
Given more time I'd ask: what if blocked cells can be added dynamically?
Re-run BFS on each addition – or maintain Union-Find of open cells."

DFS

□ #1192 Critical Connections in a Network **HAR**

[Open on LeetCode](#)

DFS · Graph · Biconnected Component

Lists: Denny Zhang

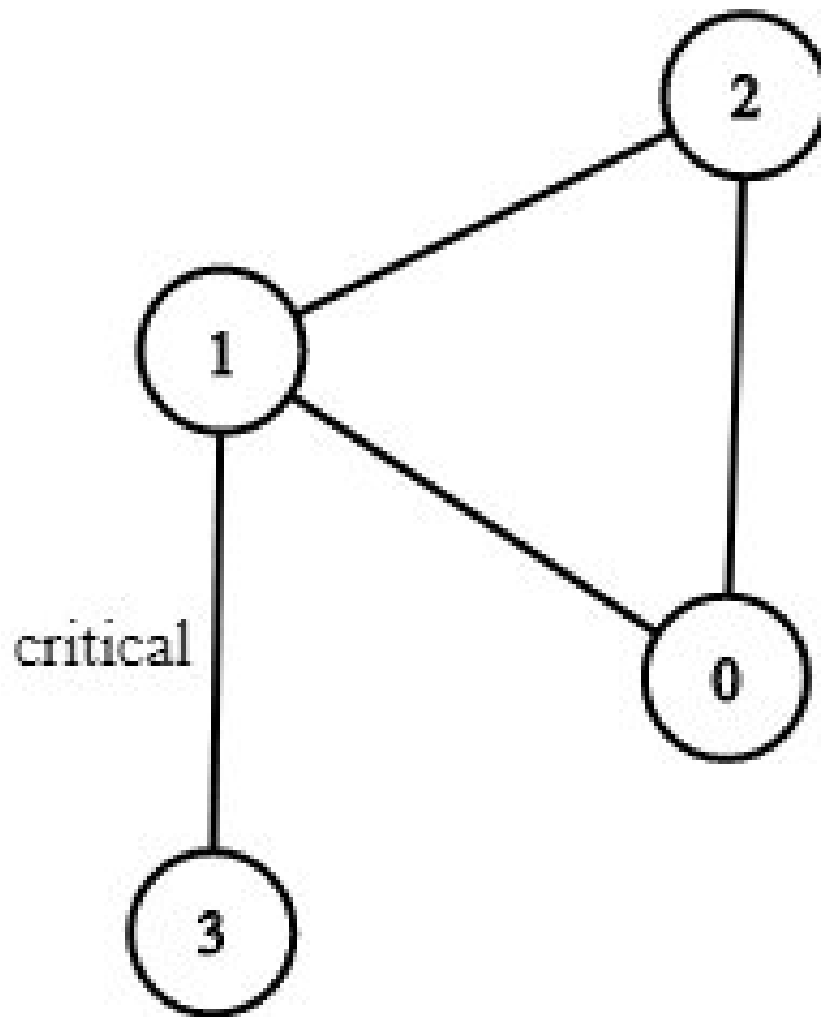
Problem

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:



Input: $n = 4$, **connections** =
[[0,1],[1,2],[2,0],[1,3]]

Output: [[1,3]]

Explanation: [[3,1]] is also accepted.

Example 2:

Input: $n = 2$, **connections** = [[0,1]]

Output: [[0,1]]

Constraints:

- $2 \leq n \leq 105$
- $n - 1 \leq \text{connections.length} \leq 105$
- $0 \leq a_i, b_i \leq n - 1$
- $a_i \neq b_i$
- There are no repeated connections.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Critical connection (bridge) = edge whose removal disconnects the graph.
Right?

Use Tarjan's bridge-finding algorithm."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Find bridges with Tarjan low-link DFS:
edge (u,v) is critical if $\text{low}[v] > \text{disc}[u]$.

One DFS pass tracks discovery time and lowest reachable.

Naive alternative: remove each edge and test connectivity – $O(E * (V+E))$.

Time: $O(V + E)$ with Tarjan. Space: $O(V + E)$.

```
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (Tarjan's DFS)
```

```
# "DFS assigning disc[u]=discovery time  
and low[u]=earliest discovery reachable.
```

```
# An edge (u,v) is a bridge if low[v] >  
disc[u] — meaning v cannot reach u's  
subtree without (u,v)."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _1192_CriticalConnections:
```

```
    def criticalConnections(self, n: int,  
connections: List[List[int]]) ->  
List[List[int]]:
```

```
    graph = defaultdict(list)
```

```
    for u, v in connections:
```

```
        graph[u].append(v)
```

```
        graph[v].append(u)
```

```
    disc = [-1] * n
```

```
    low = [-1] * n
```

```
    timer = [0]
```

```
    result = []
```

```
    def dfs(node, parent):
```

```
        disc[node] = low[node] =
```

```
timer[0]
```

```
        timer[0] += 1
```

```

        for nb in graph[node]:
            if nb == parent: continue
            if disc[nb] == -1:
                dfs(nb, node)
                low[node] =
min(low[node], low[nb])
                if low[nb] >
disc[node]:
                    result.append([no
de, nb])
            else:
                low[node] =
min(low[node], disc[nb])
                dfs(0, -1)
        return result

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Tarjan's bridge algorithm in one DFS pass.

$low[v] > disc[u]$ means v 's subtree can't 'back-connect' to u 's subtree — edge is a bridge.

Given more time I'd ask: articulation points instead of bridges?

Similar but: u is an articulation point if any child v has $low[v] \geq disc[u]$ (and

u is not root)."

Graphs

Round 3 · High · Hard · 2 questions

Graphs

□ ★ #305 Number of Islands II ★

HAR

[Open on LeetCode](#)

Array · Hash Table · Union Find · Matrix

Lists: ★ Premium | Premium 98

Problem

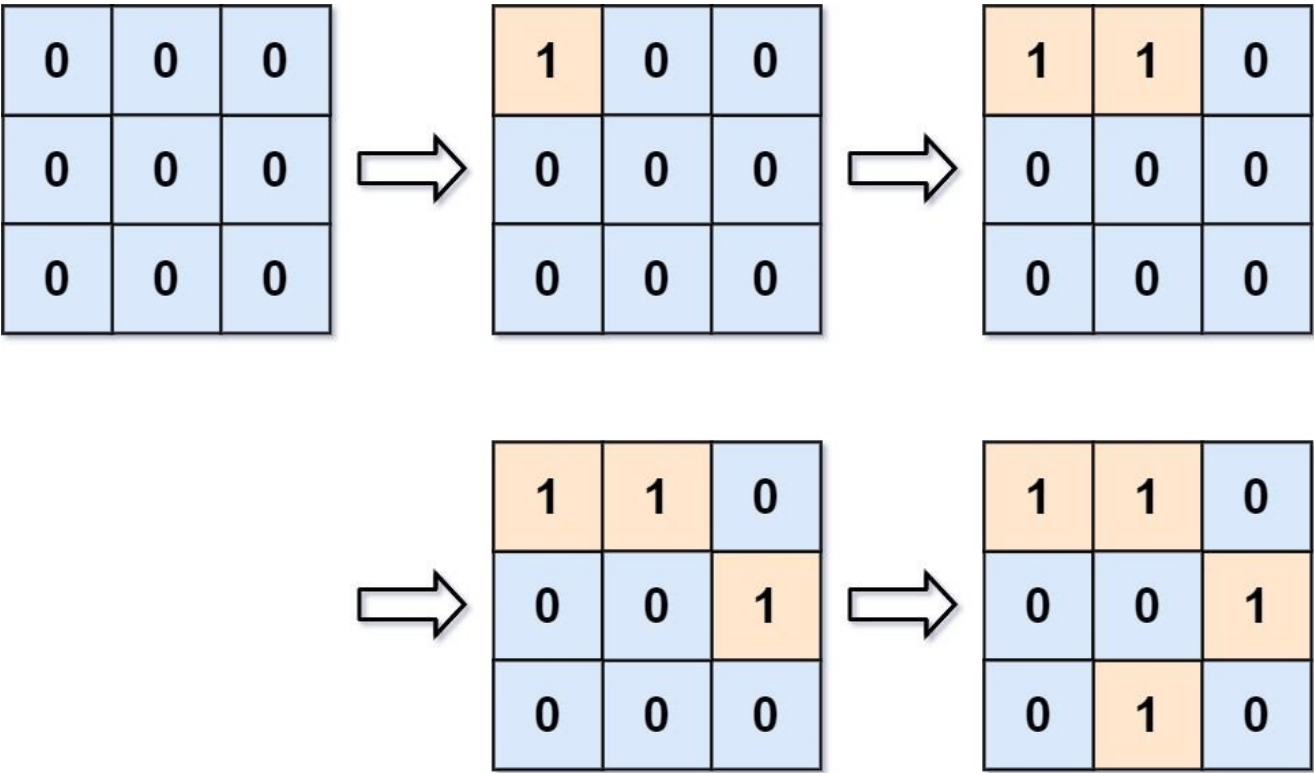
You are given an empty 2D binary grid `grid` of size $m \times n$. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

We may perform an add land operation which turns the water at position into a land. You are given an array `positions` where `positions[i] = [ri, ci]` is the position (ri, ci) at which we should operate the i th operation.

Return an array of integers `answer` where `answer[i]` is the number of islands after turning the cell (ri, ci) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the

grid are all surrounded by water.
Example 1:



Input: $m = 3, n = 3, \text{positions} =$
 $[[0,0],[0,1],[1,2],[2,1]]$

Output: $[1,1,2,3]$

Explanation:

Initially, the 2d grid is filled with water.

- Operation #1: `addLand(0, 0)` turns the water at `grid[0][0]` into a land. We have 1 island.
- Operation #2: `addLand(0, 1)` turns the water at `grid[0][1]` into a land. We still have 1 island.
- Operation #3: `addLand(1, 2)` turns the water at `grid[1][2]` into a land. We have 2 islands.
- Operation #4: `addLand(2, 1)` turns the water at `grid[2][1]` into a land. We have 3 islands.

Example 2:

Input: $m = 1, n = 1, \text{positions} =$
 $[[0,0]]$

Output: $[1]$

Constraints:

- $1 \leq m, n, \text{positions.length} \leq 104$

- $1 \leq m * n \leq 104$
- `positions[i].length == 2`
- $0 \leq r_i < m$
- $0 \leq c_i < n$

Follow up: Could you solve it in time complexity $O(k \log(mn))$, where $k == \text{positions.length}$?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Start with empty grid. Add land cells one at a time. After each add, return island count.

$O(k \log mn)$ per operation. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

After each `addLand`, scan the whole grid and flood-fill count islands from scratch.

`k` `addLand` calls each trigger full $O(m*n)$ recount – too slow when `k` is large.

Time: $O(k * m * n)$. Space: $O(m * n)$ per flood.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Union-Find with dynamic updates)

"Each new land cell starts as its own island (count++).

Check 4 neighbours: for each land neighbour, if different component, union and count--."

PHASE 4 — CLEAN CODE

```
class _305_NumberIslandsII:
```

```
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
```

```
        parent = {}
```

```
        count = [0]
```

```
        def find(x):
```

```
            while parent[x] != x:
```

```
                parent[x] =
```

```
parent[parent[x]]
```

```
                x = parent[x]
```

```
            return x
```

```
        def union(a, b):
```

```
            ra, rb = find(a), find(b)
```

```
            if ra == rb: return
```

```
            parent[ra] = rb
```

```
            count[0] -= 1
```

```
result = []
for r, c in positions:
    if (r, c) in parent:
        result.append(count[0])
        continue
    parent[(r, c)] = (r, c)
    count[0] += 1
    for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r + dr, c + dc
        if (nr, nc) in parent:
            union((r, c), (nr,
nc))

        result.append(count[0])
return result
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k * \alpha(mn)) \approx O(k)$. Space $O(k)$.
Union-Find handles dynamic connectivity
perfectly.

The duplicate position check avoids
double-counting islands.

Given more time I'd ask: what if land
cells can be removed too?

Split-Find is hard. Offline deletion via
reverse addition is a common trick."

Graphs

□ #1153 String Transforms Into Another String

HAR[Open on LeetCode](#)

Hash Table · String · Union Find

Lists: Denny Zhang

Problem

Given two strings `str1` and `str2` of the same length, determine whether you can transform `str1` into `str2` by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in `str1` to any other lowercase English character.

Return `true` if and only if you can transform `str1` into `str2`.

Example 1:

Input: `str1 = "aabcc"`, `str2 = "ccdee"`

Output: `true`

Explanation: Convert `'c'` to `'e'` then `'b'` to `'d'` then `'a'` to `'c'`. Note that the order of conversions matter.

Example 2:

Input: str1 = "leetcode", str2 = "codeleet"

Output: false

Explanation: There is no way to transform str1 to str2.

Constraints:

- $1 \leq \text{str1.length} == \text{str2.length} \leq 104$
- str1 and str2 contain only lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can str1 be converted to str2 by repeatedly mapping one character to another (all occurrences)?

Conversions can be chained but must be consistent. Right?"

#

"If str1==str2 trivially True. If str2 uses all 26 letters → no 'buffer' character available → False."

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Build a map from each character to its
index in the first string.
# Scan the second string left-to-right:
next char must appear at a strictly higher
index.
# If any char is missing or out of order,
transformation is impossible.
# Time: O(n). Space: O(1) – fixed alphabet
size 26.
# Should I go ahead and optimize?"

class _1153_StringTransforms:
    def canConvert(self, str1: str, str2:
str) -> bool:
        if str1 == str2:
            return True
        mapping = {}
        for c1, c2 in zip(str1, str2):
            if mapping.get(c1, c2) != c2:
                return False
            mapping[c1] = c2
        return len(set(str2)) < 26 # need
at least one unused char as buffer
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(26)$ . The  
len(set(str2)) < 26 check: if all 26 chars  
appear in str2,  
# every potential buffer letter is  
'locked' to a mapping – cycles can't be  
broken.  
# Given more time I'd ask: what if  
conversions can only be applied once?  
# Topological sort on the conversion graph  
– acyclic → True."
```

BFS

Round 3 · High · Hard · 4 questions

BFS

□ #127 Word Ladder

HAR[Open on LeetCode](#)

Hash Table · String · BFS

Lists: Grind 169

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` $\rightarrow$ `s1` $\rightarrow$ `s2` $\rightarrow$... $\rightarrow$ `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> "cog", which is 5 words long.

Example 2:

Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: 0
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.

- `beginWord != endWord`
- All the words in `wordList` are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Shortest word transformation from `beginWord` to `endWord`, changing one letter at a time.

Each intermediate word must be in `wordList`. Return total words in shortest path, or 0. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build adjacency from word patterns (one-letter-apart neighbors).

BFS from `beginWord` layer by layer until `endWord` is reached; each layer = one transformation.

Visited set prevents revisiting words. Return BFS depth or 0 if unreachable.

Time: $O(N * L^2)$ where N = dictionary size, L = word length. Space: $O(N)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with pattern matching)

"BFS on words. For each current word, try replacing each position with 26 letters.

If the result is in wordSet (and not visited), enqueue it.

Alternatively: build adjacency by pattern (*ot, h*t, ho*) → group words by pattern — $O(L^2 * n)$."

PHASE 4 — CLEAN CODE

```
class _127_WordLadder:
```

```
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
```

```
        word_set = set(wordList)
```

```
        if endWord not in word_set:
```

```
            return 0
```

```
        queue = deque([(beginWord, 1)])
```

```
        visited = {beginWord}
```

```
        while queue:
```

```
            word, steps = queue.popleft()
```

```
            for i in range(len(word)):
```

```
                for c in
```

```
                    'abcdefghijklmnopqrstuvwxyz':
```



```

        next_word = word[:i]
+ c + word[i+1:]
        if next_word ==
endWord:
            return steps + 1
            if next_word in
word_set and next_word not in visited:
                visited.add(next_
word)
                queue.append((nex
t_word, steps + 1))
            return 0

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * L * 26) = O(n*L)$. Space $O(n*L)$. Standard BFS shortest path.

Bidirectional BFS: search simultaneously from beginWord and endWord – roughly halves depth.

Given more time I'd ask: return all shortest paths?

That's Word Ladder II (#126) – BFS to build levels, then DFS to backtrack paths."

BFS

□ ★ #317 Shortest Distance from All Buildings ★

HAR[Open on LeetCode](#)

Array · BFS · Matrix

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ grid of values 0, 1, or 2, where:

- each 0 marks an empty land that you can pass by freely,
- each 1 marks a building that you cannot pass through, and
- each 2 marks an obstacle that you cannot pass through.

You want to build a house on an empty land that reaches all buildings in the shortest total travel distance. You can only move up, down, left, and right.

Return the shortest travel distance for such a house. If it is not possible to build such a house according to the above rules, return -1.

The total travel distance is the sum of the distances between the houses of the friends and the meeting point.

Example 1:

1	0	2	0	1
0	0	0	0	0
0	0	1	0	0

Input: `grid =`
`[[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]`
Output: 7
Explanation: Given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2).
The point (1,2) is an ideal empty land to build a house, as the total travel distance of $3+3+1=7$ is minimal.
So return 7.

Example 2:

Input: `grid = [[1,0]]`
Output: 1

Example 3:

Input: `grid = [[1]]`
Output: -1

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `grid[i][j]` is either 0, 1, or 2.

- There will be at least one building in the grid.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find empty land (0) minimising total BFS distance to all buildings (1).

Obstacles (2) can't be passed. Return -1 if no valid land. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each empty cell, BFS to sum distances to all buildings.

Track total distance per cell; answer is cell with minimum total (must reach all buildings).

Repeat BFS from every building – $O(B * m * n)$ where B = buildings.

Time: $O(B * m * n)$. Space: $O(m * n)$ BFS state.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS from each building, accumulate distances)

"BFS from each building cell. For each empty cell, accumulate distance and increment reach_count.

Only cells reachable from ALL buildings are candidates. Return min total distance."

PHASE 4 — CLEAN CODE

```
class _317_ShortestDistanceBuildings:
    def shortestDistance(self, grid:
List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        dist = [[0] * n for _ in range(m)]
        reach = [[0] * n for _ in
range(m)]
        buildings = sum(grid[r][c] == 1
for r in range(m) for c in range(n))
        for r in range(m):
            for c in range(n):
                if grid[r][c] != 1:
continue
                queue = deque([(r, c,
0)])
                visited = {(r, c)}
                while queue:
```

```

        cr, cc, d =
queue.popleft()
        for dr, dc in
[(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = cr + dr,
cc + dc
            if 0 <= nr < m and
0 <= nc < n and grid[nr][nc] == 0 and
(nr,nc) not in visited:
                visited.add((
nr, nc))
                dist[nr][nc]
reach[nr][nc]
+= 1
                queue.append(
(nr, nc, d + 1))
        return min(
            (dist[r][c] for r in range(m)
for c in range(n) if grid[r][c] == 0 and
reach[r][c] == buildings),
            default=-1
        )

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{buildings} * m * n)$. Space $O(m*n)$. BFS from each building accumulates into `dist[]`.

`reach[]` ensures we only pick land reachable from ALL buildings.

Given more time I'd ask: what if buildings can move? Re-run BFS on each query."

BFS

□ #773 Sliding Puzzle

HAR[Open on LeetCode](#)

Array · BFS · Matrix

Lists: Denny Zhang

Problem

On an 2×3 board, there are five tiles labeled from 1 to 5, and an empty square represented by 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.

The state of the board is solved if and only if the board is $[[1,2,3],[4,5,0]]$.

Given the puzzle board `board`, return the least number of moves required so that the state of the board is solved. If it is impossible for the state of the board to be solved, return -1 .

Example 1:

1	2	3
4		5

Input: board = [[1,2,3],[4,0,5]]

Output: 1

Explanation: Swap the 0 and the 5 in one move.

Example 2:

1	2	3
5	4	

Input: board = [[1,2,3],[5,4,0]]

Output: -1

Explanation: No number of moves will make the board solved.

Example 3:

4	1	2
5		3

Input: board = [[4,1,2],[5,0,3]]

Output: 5

Explanation: 5 is the smallest number of moves that solves the board.

An example path:

After move 0: [[4,1,2],[5,0,3]]

After move 1: [[4,1,2],[0,5,3]]

After move 2: [[0,1,2],[4,5,3]]

After move 3: [[1,0,2],[4,5,3]]

Constraints:

- board.length == 2

- `board[i].length == 3`
- `0 <= board[i][j] <= 5`
- Each value `board[i][j]` is unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"2×3 board. Swap 0 with adjacent tile.
Minimum moves to reach `[[1,2,3],[4,5,0]]`.
Right?

Return `-1` if unsolvable."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

BFS on board states: encode 2x3 board as
tuple string; enqueue all legal swaps of
'0'.

First time we reach target state, return
move count.

State space is at most $6! = 720$
permutations.

Time: $O(6!)$. Space: $O(6!)$ visited
states.

Should I go ahead and optimize?"

```
# PHASE 3 — OPTIMIZE (BFS on serialised board
state)
# "Represent board as a 6-char string. BFS
on states.
# Precompute which positions in the flat
string can swap with position i (adjacency
map).
# Goal state: '123450'."

# PHASE 4 — CLEAN CODE
class _773_SlidingPuzzle:
    def slidingPuzzle(self, board:
List[List[int]]) -> int:
        GOAL = '123450'
        NEIGHBORS = {0:[1,3], 1:[0,2,4],
2:[1,5], 3:[0,4], 4:[1,3,5], 5:[2,4]}
        start = ''.join(str(board[r][c])
for r in range(2) for c in range(3))
        queue = deque([(start, 0)])
        visited = {start}
        while queue:
            state, moves =
queue.popleft()
            if state == GOAL:
                return moves
            z = state.index('0')
```

```
        for nb in NEIGHBORS[z]:
            new_state = list(state)
            new_state[z],
new_state[nb] = new_state[nb],
new_state[z]

            ns = ''.join(new_state)
            if ns not in visited:
                visited.add(ns)
                queue.append((ns,
moves + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

`[[1,2,3],[4,0,5]] → '123405'`. Swap 0 at pos 4 with 5 at pos 5 → `'123450'` = GOAL → 1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"States = $6! / 2 = 360$ reachable states (half of $6!$ are unsolvable). BFS on 360 states.

The hardcoded NEIGHBORS map is the key – it captures the 2D adjacency in 1D string indexing.

Given more time I'd ask: generalise to $n \times m$ board?

**# Same BFS approach – NEIGHBORS changes,
state space = $(n*m)! / 2$."**

BFS

□ #815 Bus Routes

HAR[Open on LeetCode](#)

Array · Hash Table · BFS

Lists: Grind 169

Problem

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the *i*th bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0th bus travels in the sequence `1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ...` forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from `source` to `target`. Return `-1` if it is not possible.

Example 1:

Input: routes = [[1,2,7],[3,6,7]],
source = 1, target = 6

Output: 2

Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.

Example 2:

Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12

Output: -1

Constraints:

- $1 \leq \text{routes.length} \leq 500$.
- $1 \leq \text{routes}[i].\text{length} \leq 105$
- All the values of routes[i] are unique.
- $\text{sum}(\text{routes}[i].\text{length}) \leq 105$
- $0 \leq \text{routes}[i][j] < 106$
- $0 \leq \text{source}, \text{target} < 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum number of BUSES (not stops) to travel from source to target. Right?

Each bus covers its entire route – stepping onto a bus travels all its stops."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build stop→routes map. BFS from source stop; each hop can board any route containing current stop.

Track min buses to reach target; mark routes visited to avoid reboarding.

Time: $O(R * S)$ routes and stops. Space: $O(R + S)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS on routes, not stops)

"Build stop→routes map. BFS where each 'node' is a bus route.

Start: all routes containing source. Each step: board a new route (adjacent if sharing a stop).

```
# Count route changes (buses taken). Avoid  
revisiting routes and stops."
```

PHASE 4 — CLEAN CODE

```
class _815_BusRoutes:  
    def numBusesToDestination(self,  
routes: List[List[int]], source: int,  
target: int) -> int:  
        if source == target:  
            return 0  
        stop_to_routes: defaultdict =  
defaultdict(set)  
        for i, route in  
enumerate(routes):  
            for stop in route:  
                stop_to_routes[stop].add(  
i)  
        visited_stops = {source}  
        visited_routes = set()  
        queue = deque([(source, 0)]) #  
(stop, buses_taken)  
        while queue:  
            stop, buses = queue.popleft()  
            for route_id in  
stop_to_routes[stop]:
```

```

                if route_id in
visited_routes: continue
                visited_routes.add(route_
id)
                for next_stop in
routes[route_id]:
                    if next_stop ==
target:
                        return buses + 1
                    if next_stop not in
visited_stops:
                        visited_stops.add
(next_stop)
                        queue.append((nex
t_stop, buses + 1))
                return -1

```

PHASE 5 — TEST & DEBUG

```

# routes=[[1,2,7],[3,6,7]], source=1,
target=6:
# queue=[(1,0)]. route 0 for stop 1: stops
2,7. Not target. queue=[(2,1),(7,1)].
# stop 7: route 1 for stop 7: stops 3,6.
6==target → return 1+1=2 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{sum of route lengths})$. Space $O(\text{same})$. BFS on routes, not stops – the key insight.

Visiting stops prevents re-enqueuing; visiting routes prevents re-processing entire routes.

Given more time I'd ask: find all paths using at most k buses?

Keep track of bus count per path – BFS naturally handles this."

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

12 questions · 4 patterns

Linked List #21 #141 #206 #234 #706 #876
#1474

Stack #20 #232 #844

Trie #14

Greedy #409

Linked List

Round 4 · Mid · Easy · 7 questions

Linked List

□ #21 Merge Two Sorted Lists

EAS

[Open on LeetCode](#)

Linked List · Recursion

Lists: Grind 169

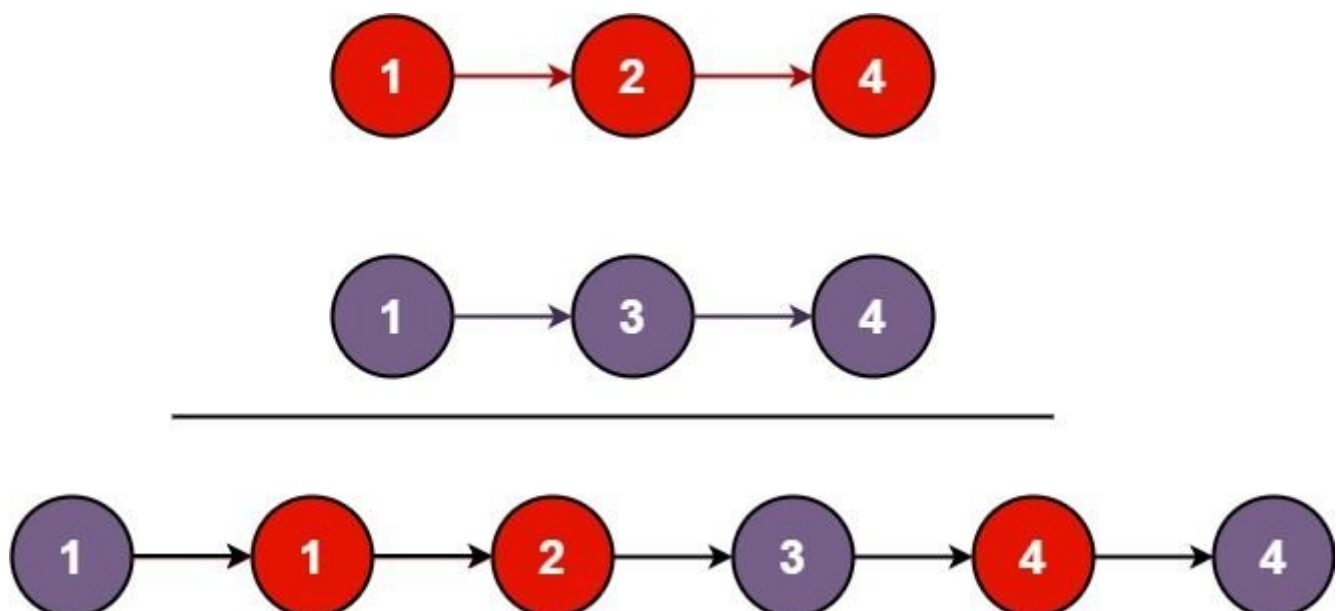
Problem

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = []
Output: []

Example 3:

Input: list1 = [], list2 = [0]
Output: [0]

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We have two sorted linked lists and need to merge them into one sorted list

```
# by re-linking the original nodes – no
new allocations.
# Return the new head. Right?"
#
# "A few quick questions:
# Can either list be empty? Yes –
constraints say 0 nodes is valid.
# Are duplicates across lists allowed?
Yes.
# Must the result be stable (preserve
relative order of equal elements)? Yes,
standard merge."
#
# "Let me walk the example: list1=[1,2,4],
list2=[1,3,4].
# Pick 1 (tie → list1 first), then 1 from
list2, then 2, 3, 4, 4. → [1,1,2,3,4,4] ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Collect all values from both lists into
a Python list, sort it, then rebuild
# a brand-new linked list from those
sorted values.
```

```
# This is  $O((m+n) \log(m+n))$  for the sort  
and  $O(m+n)$  extra space for the values.  
# It completely throws away the fact that  
both inputs are already sorted –  
# there is zero benefit from sorting  
again.
```

```
# Should I go ahead and optimize?"
```

```
class _21_MergeTwoSortedLists_Brute:  
    def mergeTwoLists(self, list1,  
list2):  
        vals = []  
        while list1:  
vals.append(list1.val); list1 =  
list1.next  
        while list2:  
vals.append(list2.val); list2 =  
list2.next  
        dummy = cur = ListNode(0)  
        for v in sorted(vals):  
            cur.next = ListNode(v); cur =  
cur.next  
        return dummy.next
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the sort – we paid  
 $O(\log n)$  we didn't need.
```

```
# Since both lists are already sorted, the
smallest available element is always
# at one of the two heads. We just pick
the smaller head at each step.
```

```
#
```

```
# Pseudocode:
```

```
# dummy = cur = ListNode(0)
```

```
# while list1 and list2:
```

```
# if list1.val <= list2.val: attach list1,
advance list1
```

```
# else: attach list2, advance list2
```

```
# advance cur
```

```
# attach whichever list remains
```

```
# return dummy.next
```

```
#
```

```
# Time:  $O(m+n)$  – one comparison per node.
```

```
Space:  $O(1)$  – only pointer rewiring.
```

```
# Does that make sense before I code it?"
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _21_MergeTwoSortedLists:
```

```
    def mergeTwoLists(self, list1,
list2):
```

```
        dummy = cur = ListNode(0)
```

```
while list1 and list2:
    if list1.val <= list2.val:
        cur.next = list1
        list1 = list1.next
    else:
        cur.next = list2
        list2 = list2.next
    cur = cur.next
cur.next = list1 if list1 else
list2
return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

list1=[1,2,4], list2=[1,3,4]:

1<=1 → take l1(1), l1→2. 2>1 → take

l2(1), l2→3. 2<=3 → take l1(2), l1→4.

4>3 → take l2(3), l2→4. 4<=4 → take

l1(4), l1=None.

cur.next = l2=[4]. → [1,1,2,3,4,4] ✓

#

list1=[], list2=[0]:

while loop never enters. cur.next =

list2=[0]. → [0] ✓

#

"No bugs. Dummy node cleanly handles the empty-list edge cases."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$ – one comparison per node.
Space $O(1)$ – reuses existing nodes.

The dummy head is the key trick: we never need to special-case the very first node.

Follow-up: Merge k Sorted Lists (#23) – use a min-heap for $O(n \log k)$ time.

Follow-up: Sort a Linked List (#148) – apply this same merge step inside merge sort."

Linked List

□ #141 Linked List Cycle

EAS

[Open on LeetCode](#)

Hash Table · Linked List · Two Pointers

Lists: Grind 169

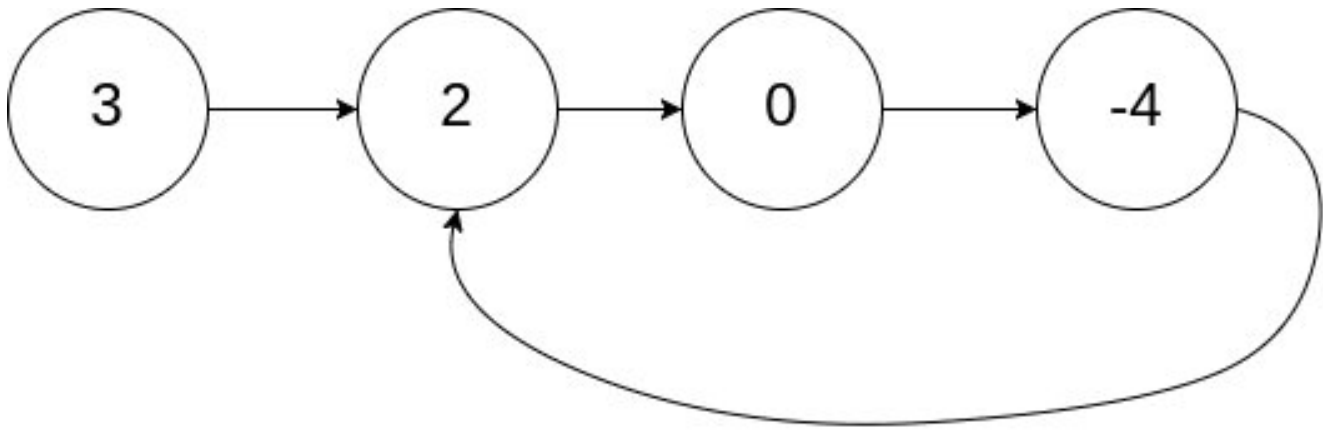
Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

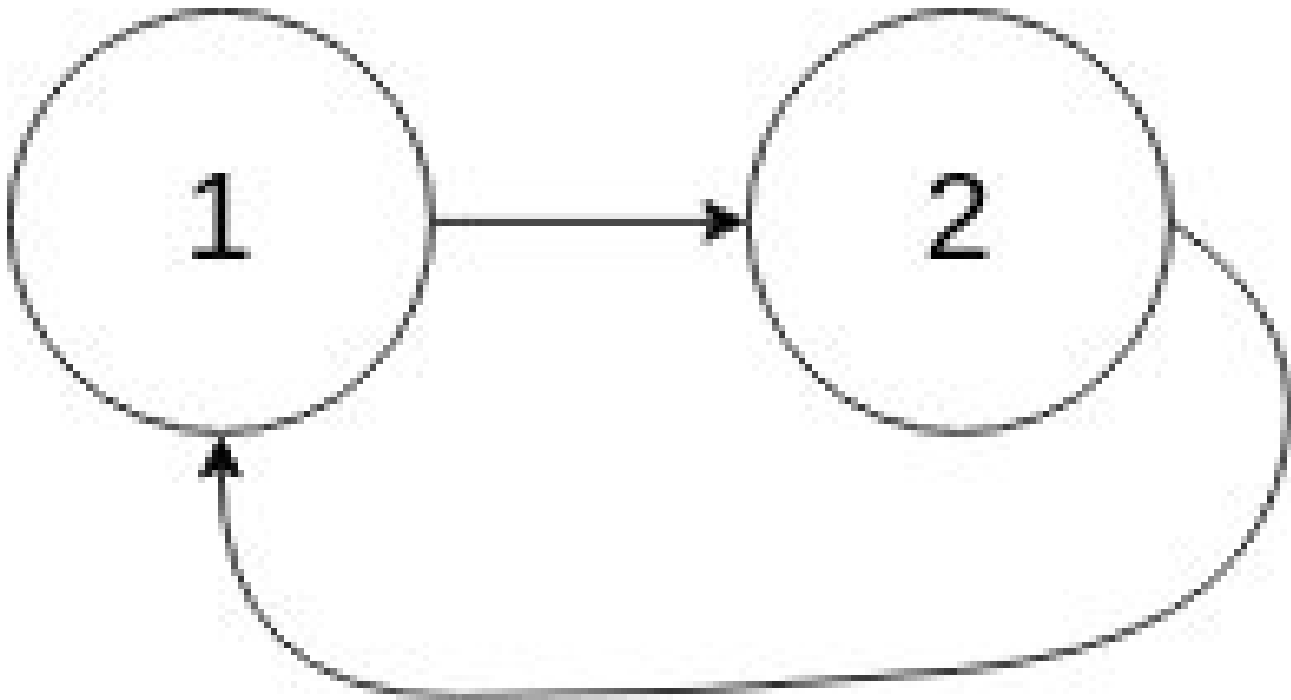


Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:

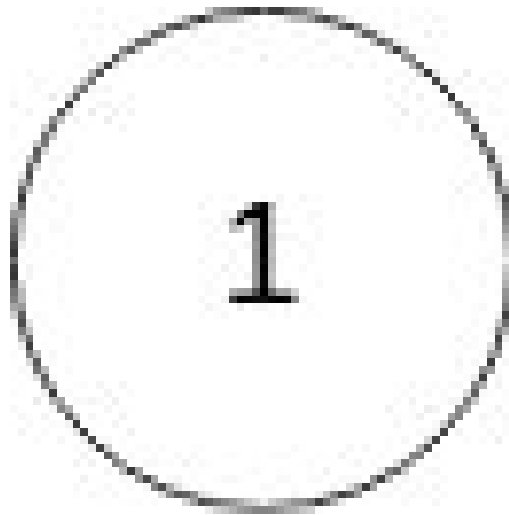


Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range [0, 104].
- $-105 \leq \text{Node.val} \leq 105$
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have a singly linked list and need to return true if there is a cycle –

meaning some node's next pointer points back to a previous node.

Return false if we reach None (no cycle). Right?"

#

"A few quick questions:

Is pos given to us in the actual function? No – pos is just for test descriptions.

Can the list have one node pointing to itself? Yes – that's a cycle too.

Can the list be empty? Yes – empty list has no cycle."

#

"Let me walk the example: [3,2,0,-4], tail connects back to index 1.

3→2→0→-4→2→... cycles forever → true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Walk the list and store every visited node in a hash set.

If we encounter a node already in the set, there's a cycle.

If we reach None, there isn't.

This is $O(n)$ time but $O(n)$ extra space for the set.

The follow-up asks for $O(1)$ space — that's what I'll aim for.

Should I go ahead and optimize?"

class _141_LinkedListCycle_Brute:

def hasCycle(self, head):

seen = set()

cur = head

while cur:

if id(cur) in seen:

return True

seen.add(id(cur))

cur = cur.next

return False

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ extra space for the visited set.

Floyd's tortoise-and-hare: use two pointers – slow moves 1 step, fast moves 2.

If a cycle exists, fast will eventually lap slow and they'll meet inside the cycle.

If no cycle, fast reaches None first.

#

Pseudocode:

slow = fast = head

while fast and fast.next:

slow = slow.next; fast = fast.next.next

if slow is fast: return True

return False

#

Time: $O(n)$. Space: $O(1)$ – only two pointer variables.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _141_LinkedListCycle:

```
def hasCycle(self, head):  
    slow = fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
        if slow is fast:  
            return True  
    return False
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[3,2,0,-4], cycle at index 1 (slow/fast both start at 3):

step1: slow=2, fast=0. step2: slow=0, fast=2. step3: slow=-4, fast=-4. Match → True ✓

#

[1,2], no cycle:

step1: slow=2, fast=None. fast is None → loop exits → False ✓

#

[1], no cycle:

fast.next is None → loop never enters → False ✓

#

"No bugs. Guard 'fast and fast.next' is essential to avoid NoneType errors."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: fast covers the cycle in at most n steps after entering it.

Space $O(1)$: only two pointer variables.

Hash-set approach is $O(n)$ space – Floyd's is the canonical $O(1)$ improvement.

Follow-up: Find Cycle Start (#142) – after they meet, reset slow to head,

advance both one step at a time; they meet again at the cycle entry.

Follow-up: cycle length – once they meet, keep fast moving and count steps until it returns."

Linked List

□ #206 Reverse Linked List

EAS

[Open on LeetCode](#)

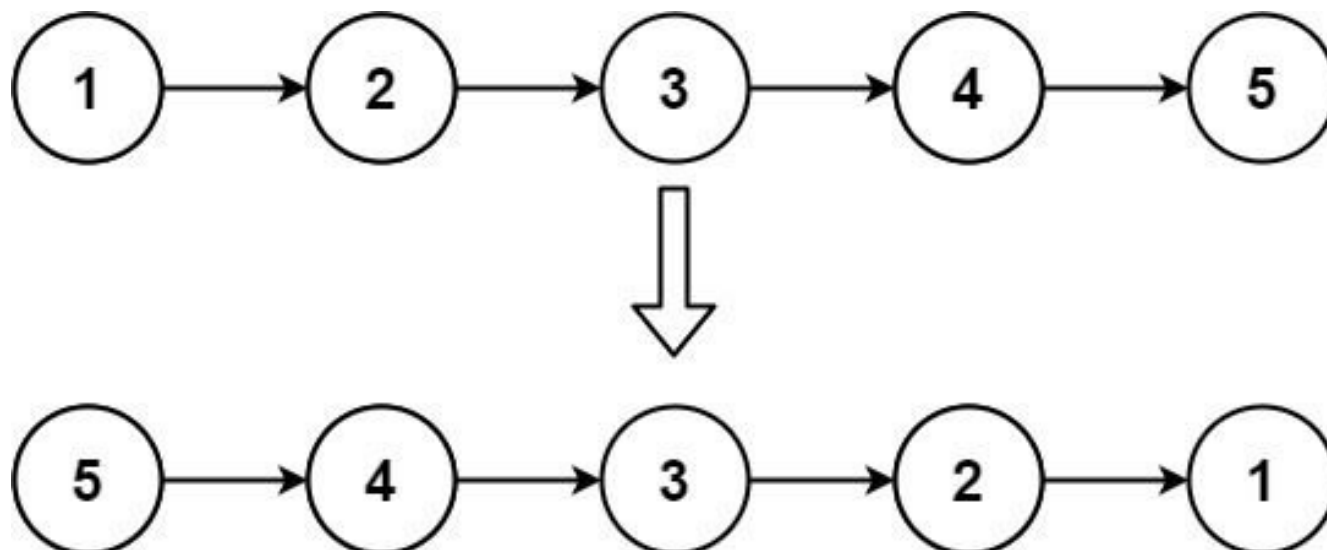
Linked List · Recursion

Lists: Grind 169

Problem

Given the head of a singly linked list, reverse the list, and return the reversed list.

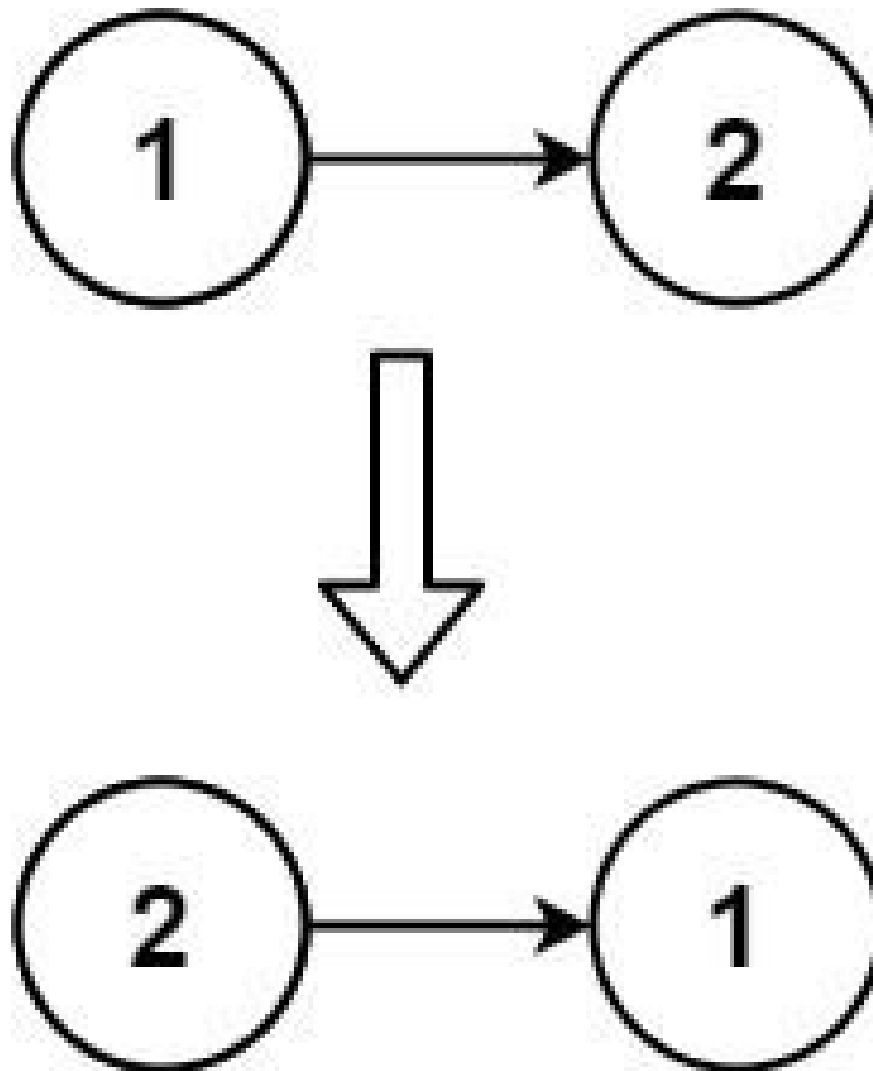
Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].

- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given the head of a singly linked list, reverse it in-place and return the new head.

We change the next pointers – not just the values. Right?"

#

"A few quick questions:

Can the list be empty? Yes – return None.

Can it have just one node? Yes – return that node unchanged.

The follow-up asks for both iterative and recursive – I'll do iterative first."

#

"Let me walk the example: [1,2,3,4,5] → [5,4,3,2,1] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Collect all node values into a list, build a new reversed linked list from those values.

This is $O(n)$ time but $O(n)$ extra space for the values array –

and it allocates new nodes instead of rewiring the existing ones.

Should I go ahead and optimize?"

```
class _206_ReverseLinkedListBrute:
    def reverseList(self, head):
        vals = []
        cur = head
        while cur: vals.append(cur.val);
        cur = cur.next
        dummy = cur = ListNode(0)
        for v in reversed(vals):
            cur.next = ListNode(v); cur =
cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ extra space and the unnecessary node allocation.

We can reverse in-place by maintaining three pointers: prev, curr, next_node.

At each step we point curr.next backward to prev, then advance.

#

Pseudocode:

prev, curr = None, head

while curr:

next_node = curr.next

curr.next = prev

prev, curr = curr, next_node

return prev

#

Time: $O(n)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _206_ReverseLinkedList:
```

```
    def reverseList(self, head):
```

```
        prev, curr = None, head
```

```
        while curr:
```

```
        next_node = curr.next
        curr.next = prev
        prev, curr = curr, next_node
    return prev
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3]:

prev=None, curr=1: next=2, 1.next=None,
prev=1, curr=2.

prev=1, curr=2: next=3, 2.next=1,
prev=2, curr=3.

prev=2, curr=3: next=None, 3.next=2,
prev=3, curr=None.

return prev=node(3) → [3,2,1] ✓

#

[]: curr=None → loop never enters →
return None ✓

#

"No bugs. prev starts at None so the new
tail's next is correctly None."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: three
pointer variables.

Recursive solution is elegant but $O(n)$ stack space – mention this trade-off.
Follow-up: Reverse Linked List II (#92) – reverse between positions l and r .
Follow-up: Reverse Nodes in k-Group (#25) – apply this logic to groups of k ."

Linked List

□ #234 Palindrome Linked List

EAS[Open on LeetCode](#)

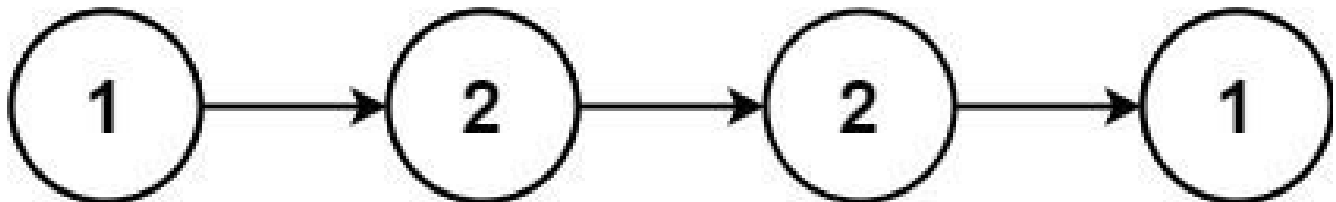
Linked List · Two Pointers · Recursion

Lists: Grind 169

Problem

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

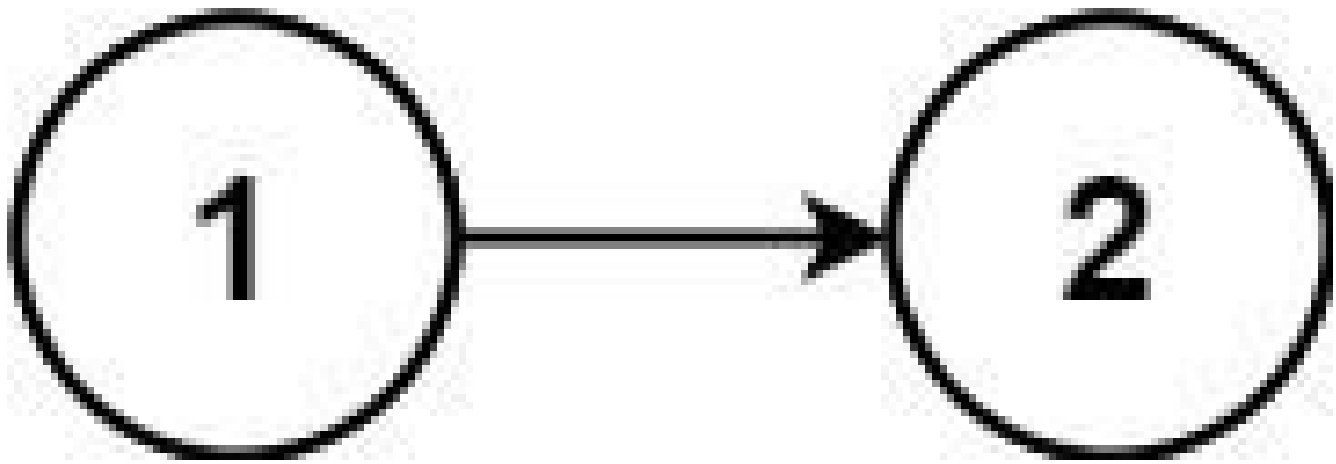
Example 1:



Input: head = [1,2,2,1]

Output: true

Example 2:



Input: head = [1,2]

Output: false

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 9$

Follow up: Could you do it in $O(n)$ time and $O(1)$ space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We need to check if the linked list reads the same forwards and backwards.

We can only traverse with next pointers – no random access.


```
# The follow-up asks for  $O(n)$  time and  $O(1)$  space. Right?"
```

```
#
```

```
# "A few quick questions:
```

```
# Can the list be empty or have one node?  
Yes – trivially palindromes.
```

```
# Should we restore the list after  
checking? Good practice; the problem  
doesn't require it."
```

```
#
```

```
# "Let me walk the example: [1,2,2,1] →  
palindrome ✓. [1,2] → not ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Copy all values into a Python list, then  
compare the list to its reverse.
```

```
# This is  $O(n)$  time and  $O(n)$  space – clear  
and correct.
```

```
# The follow-up wants  $O(1)$  extra space, so  
let's aim for that.
```

```
# Should I go ahead and optimize?"
```

```
class _234_PalindromeLinkedList_Brute:
```

```
    def isPalindrome(self, head):
```

```
        vals = []
```

```
        cur = head
        while cur: vals.append(cur.val);
cur = cur.next
return vals == vals[::-1]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ extra array.

We can do this in $O(1)$ extra space with three steps:

1. Find the midpoint using slow/fast pointers.

2. Reverse the second half in-place.

3. Compare first half with reversed second half node by node.

#

Pseudocode:

slow, fast = head, head

while fast and fast.next: slow = slow.next; fast = fast.next.next

second = reverse(slow)

left, right = head, second

while right:

if left.val != right.val: return False

left = left.next; right = right.next

return True

#

```
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
# approach with meaningful names."
class _234_PalindromeLinkedList:
    def isPalindrome(self, head):
        def reverse(node):
            prev, cur = None, node
            while cur:
                nxt = cur.next; cur.next
                = prev; prev, cur = cur, nxt
            return prev
        slow = fast = head
        while fast and fast.next:
            slow = slow.next; fast =
            fast.next.next
        second = reverse(slow)
        left, right = head, second
        while right:
            if left.val != right.val:
                return False
            left = left.next; right =
            right.next
        return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,2,1]:

Mid: slow=2(idx1), fast=1(idx3 after 2 steps). Reverse [2,1] → [1,2].

Compare: 1==1 ✓, 2==2 ✓, right=None → True ✓

#

[1,2]:

Mid: slow=2, fast=None after 1 step. Reverse [2] → [2].

Compare: 1≠2 → False ✓

#

"No bugs. For even-length lists slow lands on the second half correctly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: find mid $O(n/2)$ + reverse $O(n/2)$ + compare $O(n/2)$."

Space $O(1)$: constant extra pointers.

Trade-off: the list is modified during checking. To restore: reverse again after comparison.

Follow-up: Valid Palindrome (#125) – same idea on a string, ignoring

non-alphanumeric.

Follow-up: Palindrome Number (#9) – similar reverse-half technique on an integer."

Linked List

□ #706 Design HashMap

EAS

[Open on LeetCode](#)

Array · Hash Table · Linked List · Design · Hash Function

Lists: Denny Zhang

Problem

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.

Example 1:

Input

```
["MyHashMap", "put", "put", "get",  
"get", "put", "get", "remove", "get"]  
[[], [1, 1], [2, 2], [1], [3], [2, 1],  
[2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1,  
null, -1]
```

Explanation

```
MyHashMap myHashMap = new MyHashMap();
```

Constraints:

- $0 \leq \text{key}, \text{value} \leq 106$
- At most 104 calls will be made to put, get, and remove.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We need to implement a HashMap from scratch – no built-in dict or hash libraries.

```
# Support put(key, val), get(key) → -1 if  
missing, and remove(key).
```

```
# Keys and values are non-negative  
integers up to  $10^6$ . Right?"
```

```
#
```

```
# "A few quick questions:
```

```
# At most  $10^4$  calls total – so a simple  
fixed-size array could work.
```

```
# Does put update the value if the key  
exists? Yes.
```

```
# Does remove need to return anything?  
No."
```

```
#
```

```
# "Let me walk the example: put(1,1),  
put(2,2), get(1)→1, get(3)→-1, remove(2),  
get(2)→-1 ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Store all (key, value) pairs in a plain  
list, scan linearly for get/put/remove.
```

```
# This is  $O(n)$  per operation – correct but  
slow for large  $n$ .
```

```
# Should I go ahead and optimize?"
```

```
class _706_DesignHashMap_Brute:
```



```
def __init__(self):
    self.data = []
    def put(self, key, value):
        for i, (k, _) in
enumerate(self.data):
            if k == key: self.data[i] =
(key, value); return
        self.data.append((key, value))
    def get(self, key):
        for k, v in self.data:
            if k == key: return v
        return -1
    def remove(self, key):
        self.data = [(k, v) for k, v in
self.data if k != key]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ linear scan per operation.

Use an array of buckets with separate chaining: each bucket holds a small list.

Hash function: $\text{key} \% \text{num_buckets}$. We pick a large prime (e.g. 1009) to spread keys evenly.

Average $O(1)$ per operation when the load factor is low.

```
#  
# Pseudocode:  
# __init__: buckets = [[] for _ in  
# range(1009)]  
# put: idx = key % 1009; update existing  
# pair or append  
# get: scan bucket for key, return value  
# or -1  
# remove: filter key out of bucket  
#  
# Time: O(1) average. Space: O(n + 1009)."  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
class _706_DesignHashMap:  
    def __init__(self):  
        self._size = 1009  
        self._buckets = [[] for _ in  
range(self._size)]  
    def _idx(self, key):  
        return key % self._size  
    def put(self, key, value):  
        bucket =  
self._buckets[self._idx(key)]
```

```
        for i, (k, _) in
enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
            return
        bucket.append((key, value))
    def get(self, key):
        for k, v in
self._buckets[self._idx(key)]:
            if k == key:
                return v
        return -1
    def remove(self, key):
        idx = self._idx(key)
        self._buckets[idx] = [(k, v) for
k, v in self._buckets[idx] if k != key]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# put(1,1): bucket[1]=[(1,1)]. put(2,2):
bucket[2]=[(2,2)].
# get(1): bucket[1] → (1,1) → 1 ✓. get(3):
bucket[3]=[] → -1 ✓.
# put(2,1): bucket[2] → update → [(2,1)].
```

```
# remove(2): bucket[2]=[]. get(2): [] → -1
✓.
#
# "No bugs. Prime bucket count minimises
collisions for common key distributions."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(1)$  average;  $O(n/B)$  worst case if
all keys hash to same bucket ( $B$  = bucket
count).
# Space  $O(n + B)$ :  $n$  stored pairs +  $B$ 
bucket slots.
# Trade-off: larger  $B$  reduces collisions
but wastes memory for sparse maps.
# Follow-up: Design HashSet (#705) –
identical structure, store keys only.
# Follow-up: open addressing (linear
probing) is an alternative – avoids linked
lists but needs tombstones."
```

Linked List

□ #876 Middle of the Linked List

EAS

[Open on LeetCode](#)

Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

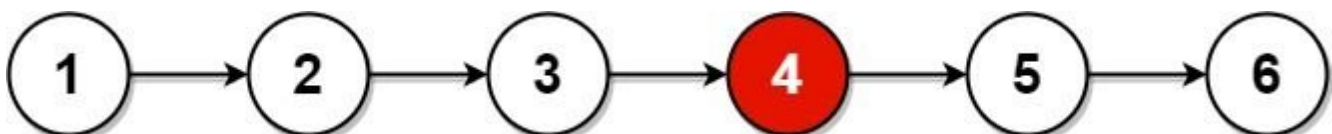


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the middle node of a singly linked list.

If there are two middle nodes (even length), return the SECOND one.

Right?"

#

"A few quick questions:

Can the list be empty? No – length is at least 1.

Return the node itself, not its value?
Yes."

#

"Let me walk the examples:

[1,2,3,4,5]: length 5 → middle index 2 →
node(3) ✓

[1,2,3,4,5,6]: two middles at 2,3 →
return second → node(4) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Count the total length in one pass, then
walk to position length//2 in a second
pass.

Two passes total: O(n) time, O(1) space.

Should I go ahead and optimize?"

class _876_MiddleLinkedList_Brute:

def middleNode(self, head):

length, cur = 0, head

while cur: length += 1; cur =

cur.next

cur = head

for _ in range(length // 2): cur =

cur.next

return cur

PHASE 3 — OPTIMIZE

"The bottleneck is doing two full passes — we can find the middle in one pass.

Slow/fast pointer: slow moves 1 step, fast moves 2.

When fast reaches the end, slow is at the middle.

For even-length lists, slow naturally lands on the SECOND middle (desired behaviour).

#

Pseudocode:

slow = fast = head

while fast and fast.next:

slow = slow.next; fast = fast.next.next

return slow

#

Time: $O(n)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _876_MiddleLinkedList:
```

```
    def middleNode(self, head):
```

```
        slow = fast = head
```



```
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
return slow
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5]:

s=1, f=1 → s=2, f=3 → s=3, f=5 →

fast.next=None → stop. return node(3) ✓

#

[1,2,3,4,5,6]:

s=1, f=1 → s=2, f=3 → s=3, f=5 →

s=4, f=None(6.next) → stop. return node(4)

✓

#

[1]:

fast.next=None immediately → return
node(1) ✓

#

"No bugs. The 'fast and fast.next' guard
handles odd and even lengths uniformly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: fast covers the list in $n/2$ iterations. Space $O(1)$: two pointers.

To get the FIRST middle for even-length, use 'while fast.next and fast.next.next' instead.

This building-block appears in Palindrome Linked List (#234) and Reorder List (#143).

Follow-up: Kth Node From End (#19) – two-pointer gap of k instead of 2:1 speed ratio."

Linked List

□ ★ #1474 Delete N Nodes After M Nodes of Linked List ★

EAS

[Open on LeetCode](#)

Linked List

Lists: ★ Premium | Premium 98

Problem

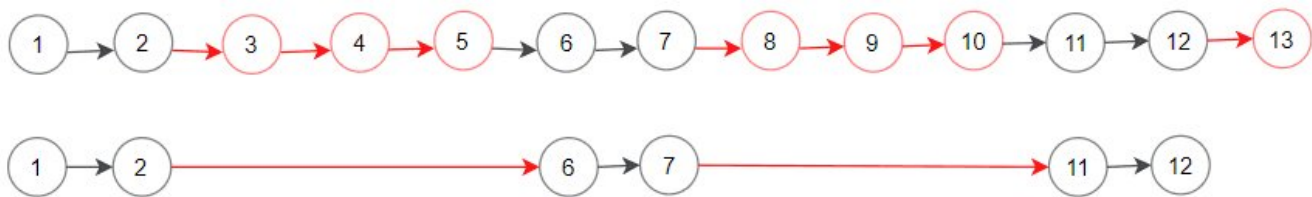
You are given the head of a linked list and two integers m and n .

Traverse the linked list and remove some nodes in the following way:

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

Example 1:



Input: head =

**[1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2,
n = 3**

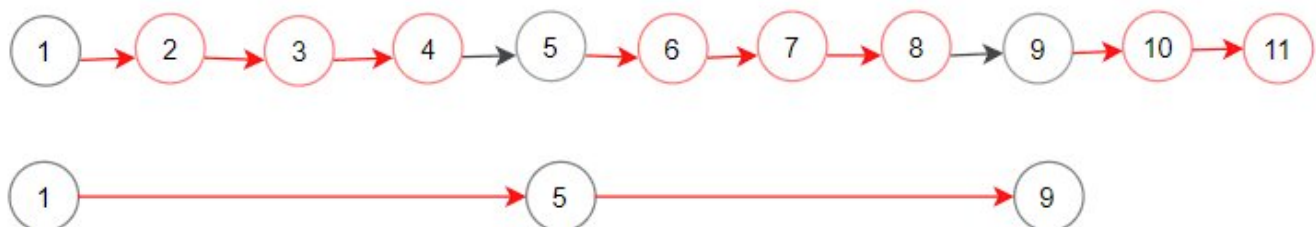
Output: [1,2,6,7,11,12]

**Explanation: Keep the first (m = 2)
nodes starting from the head of the
linked List (1 ->2) show in black
nodes.**

**Delete the next (n = 3) nodes (3 -> 4
-> 5) show in read nodes.**

**Continue with the same procedure until
reaching the tail of the Linked List.
Head of the linked list after removing
nodes is returned.**

Example 2:



Input: head =
[1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3
Output: [1,5,9]
Explanation: Head of linked list after removing nodes is returned.

Constraints:

- The number of nodes in the list is in the range [1, 104].
- $1 \leq \text{Node.val} \leq 106$
- $1 \leq m, n \leq 1000$

Follow up: Could you solve this problem by modifying the list in-place?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Walk the list in cycles: keep the first m nodes, delete the next n nodes, repeat.

Do this until the end of the list and return the modified head. Right?"

#

"A few quick questions:

m and n are guaranteed to be at least 1?
Yes.

Modify in-place – no new nodes? Yes,
just rewire pointers."

#

"Let me walk the example:
[1,2,3,4,5,6,7,8,9,10,11,12,13], m=2,
n=3.

Keep 1,2 → delete 3,4,5 → keep 6,7 →
delete 8,9,10 → keep 11,12 → delete 13.

→ [1,2,6,7,11,12] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Enumerate all nodes with an index
counter.

Keep node i if $(i \% (m+n)) < m$, else
skip it.

Collect kept nodes in a list and relink
them.

This is $O(n_{total})$ time and $O(n_{total})$
space for the list.

Should I go ahead and optimize?"

```
class _1474_DeleteNNodesAfterM_Brute:
    def deleteNodes(self, head, m, n):
```

```
nodes, cur = [], head
while cur: nodes.append(cur); cur
= cur.next
kept = [nodes[i] for i in
range(len(nodes)) if i % (m + n) < m]
for i in range(len(kept) - 1):
kept[i].next = kept[i + 1]
if kept: kept[-1].next = None
return kept[0] if kept else None
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ auxiliary list.

Walk m steps forward (keeping nodes), then walk n steps forward (skipping them),
rewire the tail of the kept section directly to the node after the skipped block.

Repeat until we run out of nodes.

#

Pseudocode:

$cur = head$

while cur :

walk $m-1$ steps (cur is now tail of kept section)

walk n steps past the nodes to delete

```
# cur.next = the node after the deleted
block
# cur = cur.next
# return head
#
# Time: O(n_total). Space: O(1).
# Does that make sense before I code it?"
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _1474_DeleteNNodesAfterM:
    def deleteNodes(self, head, m, n):
        cur = head
        while cur:
            for _ in range(m - 1):
                if cur: cur = cur.next
            skip = cur.next if cur else
None
            for _ in range(n):
                if skip: skip = skip.next
            if cur:
                cur.next = skip
                cur = skip
        return head
```


PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5,6,7], m=2, n=3:

cur=1. walk m-1=1 step → cur=2. skip=3,
walk n=3 → skip=6. 2.next=6. cur=6.

cur=6. walk 1 → cur=7. skip=None
(7.next). walk 3 (skip stays None).
7.next=None. cur=None.

→ [1,2,6,7] ✓

#

[1], m=1, n=1:

cur=1. walk 0 steps → cur=1. skip=None,
walk 1 → skip=None. 1.next=None. cur=None.

→ [1] ✓

#

"No bugs. Guards prevent crashing on
short lists."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{total_nodes})$: each node visited
exactly once. Space $O(1)$: only pointer
variables.

Contrast with the brute that uses $O(n)$
auxiliary storage.

Follow-up: what if m and n change each cycle? Accept arrays of m and n values.

Follow-up: Remove Duplicates from Sorted List (#83) – same rewire pattern, different condition."

Stack

Round 4 · Mid · Easy · 3 questions

Stack

□ #20 Valid Parentheses

EAS

[Open on LeetCode](#)

String · Stack

Lists: Grind 169

Problem

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: s = "())"

Output: false

Example 4:

Input: s = "([)]"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{}'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

A string of brackets is valid if: every opening bracket is closed by the same type,

every closing bracket has a corresponding opener, and the order is

correct.

Return true if valid, false otherwise.

Right?"

#

"A few quick questions:

Can the string be empty? Yes – empty string is valid.

Only bracket characters – no spaces or letters? Yes, the constraints confirm this."

#

"Let me walk the examples:

'()[]{}' → each pair closes correctly → true ✓

'([)]' → '[' is open when ')' arrives → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeatedly replace matched pairs '()', '[]', '{} with empty strings until nothing changes.

If the final string is empty, it was valid.

This is $O(n^2)$ – each pass removes one pair and we may need $n/2$ passes.

Should I go ahead and optimize?"

```
class _20_ValidParentheses_Brute:
```

```
    def isValid(self, s):
```

```
        prev = None
```

```
        while s != prev:
```

```
            prev = s
```

```
            for pair in ['()', '[]',
```

```
            '{}']:
```

```
                s = s.replace(pair, '')
```

```
        return s == ''
```

PHASE 3 — OPTIMIZE

"The bottleneck is repeatedly scanning the string.

A stack solves this in a single pass:

push every opening bracket; on a closing bracket, check that the top matches.

If it doesn't – or the stack is empty – it's invalid.

At the end the stack must be empty (every opener was closed).

#

Pseudocode:

stack = []

```
# for ch in s:
# if ch in '([{': stack.append(ch)
# else:
# if not stack or stack[-1] !=
matching_opener: return False
# stack.pop()
# return stack == []
#
# Time: O(n). Space: O(n) for the stack.
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _20_ValidParentheses:
    def isValid(self, s):
        pairs = {')': '(', ']': '[', '}':
'{' }
        stack = []
        for ch in s:
            if ch not in pairs:
                stack.append(ch)
            elif not stack or stack[-1] !=
pairs[ch]:
                return False
            else:
```



```
        stack.pop()
    return not stack
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# '()[ ]{}':
```

```
# '(' push. ')' → top='(' matches → pop.
```

```
# '[' push. ']' → pop. '{' push. '}' → pop.
```

```
# stack=[] → True ✓
```

```
#
```

```
# '[]':
```

```
# '(' push. ']' → top='(' ≠ '[' → False ✓
```

```
#
```

```
# '([)]':
```

```
# '(' push. '[' push. ')' → top='[' ≠ '('  
→ False ✓
```

```
#
```

```
# "No bugs. The closing-bracket dict keeps  
the matching logic clean and extensible."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : one pass. Space  $O(n)$ : worst  
case all openers e.g. '((((('.
```

```
# Brute  $O(n^2)$  is easy to understand but  
TLEs for long strings.
```

Follow-up: Minimum Add to Make Parentheses Valid (#921) – count unmatched openers/closers.

Follow-up: Longest Valid Parentheses (#32) – find the longest valid substring using DP or stack."

Stack

□ #232 Implement Queue using Stacks

EAS[Open on LeetCode](#)

Stack · Design · Queue

Lists: Grind 169

Problem

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- **void push(int x)** Pushes element x to the back of the queue.
- **int pop()** Removes the element from the front of the queue and returns it.
- **int peek()** Returns the element at the front of the queue.
- **boolean empty()** Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek",  
"pop", "empty"]
```

```
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Implement a FIFO queue using only stack operations (push/pop/peek from the top).

Support push(x), pop(), peek(), and empty().

The follow-up asks for amortized $O(1)$ per operation. Right?"

#

"A few quick questions:

Is it guaranteed that pop and peek are only called on non-empty queues? Yes.

Standard stack operations only – no deque or collections.deque tricks?"

#

"Let me walk: push(1), push(2), peek()→1, pop()→1, empty()→false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Use a single stack. On every push, move all existing elements out, push the new value,

then put everything back — so the new element sits at the bottom (queue order).

push is $O(n)$, pop/peek are $O(1)$. Simple but push is expensive.

Should I go ahead and optimize?"

```
class _232_QueueUsingStacks_Brute:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    def push(self, x):
```

```
        tmp = []
```

```
        while self.stack:
```

```
tmp.append(self.stack.pop())
```

```
        self.stack.append(x)
```

```
        while tmp:
```

```
self.stack.append(tmp.pop())
```

```
    def pop(self): return
```

```
self.stack.pop()
```

```
    def peek(self): return self.stack[-1]
```

```
def empty(self): return not  
self.stack
```

PHASE 3 — OPTIMIZE

"The bottleneck is moving all elements on every push – $O(n)$ per push.

Two stacks – inbox and outbox – give amortized $O(1)$:

push always goes to inbox.

For pop/peek: if outbox is empty, pour all of inbox into outbox (reverses order).

Then pop/peek from outbox.

Each element moves at most twice total, so the amortized cost per operation is $O(1)$.

#

Time: $O(1)$ amortized all ops. Space: $O(n)$ across both stacks."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _232_QueueUsingStacks:
```

```
    def __init__(self):
```

```
        self._inbox = []
```

```
        self._outbox = []
```

```
def push(self, x):
    self._inbox.append(x)
def _shift(self):
    if not self._outbox:
        while self._inbox:
            self._outbox.append(self.
_inbox.pop())
def pop(self):
    self._shift()
    return self._outbox.pop()
def peek(self):
    self._shift()
    return self._outbox[-1]
def empty(self):
    return not self._inbox and not
self._outbox
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

**# push(1),push(2): inbox=[1,2],
outbox=[].**

**# peek(): outbox empty → pour → inbox=[],
outbox=[2,1]. outbox[-1]=1 ✓**


```
# pop(): outbox=[2,1] → pop → 1,  
outbox=[2]. return 1 ✓  
# empty(): inbox=[], outbox=[2] → False ✓  
# pop(): outbox=[2] → pop → 2. ✓  
# empty(): both empty → True ✓  
#  
# "No bugs. 'pour' step is lazy – we only  
pay it when the outbox is drained."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "All ops amortized  $O(1)$ : each element  
transferred at most once across the two  
stacks.  
# Space  $O(n)$ : elements split across inbox  
and outbox at any moment.  
# Follow-up: Implement Stack using Queues  
(#225) – symmetric design with one queue.  
# Follow-up: Design Circular Queue (#622)  
– fixed capacity,  $O(1)$  all ops with a ring  
buffer."
```

Stack

□ #844 Backspace String Compare

EAS

[Open on LeetCode](#)

Two Pointers · String · Stack · Simulation

Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: *s* = "ab#c", *t* = "ad#c"

Output: true

Explanation: Both *s* and *t* become "ac".

Example 2:

Input: *s* = "ab##", *t* = "c#d#"

Output: true

Explanation: Both *s* and *t* become "".

Example 3:

Input: `s = "a#c"`, `t = "b"`

Output: `false`

Explanation: `s` becomes `"c"` while `t` becomes `"b"`.

Constraints:

- $1 \leq s.length, t.length \leq 200$
- `s` and `t` only contain lowercase letters and '#' characters.

Follow up: Can you solve it in $O(n)$ time and $O(1)$ space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

'#' is a backspace character that deletes the previous character (if any).

We simulate typing both strings into an empty text editor and return

true if the final texts are equal.

Right?"

#

"A few quick questions:

```
# What if '#' appears at the start with  
nothing to delete? It's a no-op.  
# Can strings be empty? Yes – empty result  
equals empty result.  
# The follow-up asks for  $O(n)$  time and  
 $O(1)$  space."
```

```
#
```

```
# "Let me walk: s='ab#c', t='ad#c'.
```

```
# s: a,b,# → a, c → 'ac'. t: a,d,# → a, c  
→ 'ac'. Equal → true ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Simulate both strings using a stack:  
push normal chars, pop on '#' if  
non-empty.
```

```
# Compare the resulting strings.
```

```
#  $O(n+m)$  time,  $O(n+m)$  space for the two  
stacks.
```

```
# Should I go ahead and optimize?"
```

```
class _844_BackspaceStringCompare_Brute:
```

```
    def backspaceCompare(self, s, t):
```

```
        def process(text):
```

```
            stack = []
```

```
            for ch in text:
```

```
        if ch != '#':
stack.append(ch)
        elif stack: stack.pop()
    return stack
return process(s) == process(t)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n+m)$ extra space for the stacks.

Scan both strings from the RIGHT with two pointers.

Use a skip counter: every '#' increments skip; a regular char while skip>0 is skipped.

The first non-skipped char from each string is compared.

We don't need any extra storage – just integer counters.

#

Time: $O(n+m)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _844_BackspaceStringCompare:
```

```
def backspaceCompare(self, s, t):
    i, j = len(s) - 1, len(t) - 1
    skip_s = skip_t = 0
    while i >= 0 or j >= 0:
        while i >= 0:
            if s[i] == '#': skip_s +=
1; i -= 1
            elif skip_s > 0: skip_s -=
1; i -= 1
            else: break
        while j >= 0:
            if t[j] == '#': skip_t +=
1; j -= 1
            elif skip_t > 0: skip_t -=
1; j -= 1
            else: break
        if i >= 0 and j >= 0 and s[i]
!= t[j]:
            return False
        if (i >= 0) != (j >= 0):
            return False
        i -= 1; j -= 1
    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# s='ab#c', t='ad#c' (i=3,j=3):
# s[3]='c' non-# → compare. t[3]='c' →
compare. 'c'=='c' ✓. i=2,j=2.
# s[2]='#' → skip_s=1, i=1. s[1]='b',
skip_s>0 → skip, i=0. s[0]='a' non-# →
compare.
# t[2]='#' → skip_t=1, j=1. t[1]='d', skip
→ j=0. t[0]='a' → compare. 'a'=='a' ✓.
i=-1,j=-1.
# both exhausted → True ✓
#
# "No bugs. The (i>=0) != (j>=0) check
catches mismatched remaining lengths."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Stack approach:  $O(n+m)$  time,  $O(n+m)$ 
space – simple and readable.
# Two-pointer approach:  $O(n+m)$  time,  $O(1)$ 
space – optimal.
# Follow-up: if '#' can delete k
characters at once, adjust skip counter to
add k.
# Follow-up: Minimum Remove to Make Valid
Parentheses (#1249) – similar
stack-simulation idea."
```

Trie

Round 4 · Mid · Easy · 1 question

Trie

□ #14 Longest Common Prefix

EAS

[Open on LeetCode](#)

String · Trie

Lists: Grind 169

Problem

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs =  
["flower", "flow", "flight"]  
Output: "fl"
```

Example 2:

```
Input: strs = ["dog", "racecar", "car"]  
Output: ""  
Explanation: There is no common prefix  
among the input strings.
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ consists of only lowercase English letters if it is non-empty.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the longest prefix shared by ALL strings in the array.

If none exists, return an empty string. Right?"

#

"A few quick questions:

Can the array be empty? No – length ≥ 1 . Can a string be empty? Yes.

Characters are lowercase English letters only."

#

"Let me walk:

['flower', 'flow', 'flight']: f✓ l✓ o vs o
vs i → mismatch → 'fl' ✓

```
# ['dog', 'racecar', 'car']: d vs r mismatch  
immediately → ' ' ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Sort the array; the LCP of the first and  
last strings must be the LCP of all  
strings
```

```
# (since sorting puts the most different  
strings at opposite ends).
```

```
# Then compare character by character.
```

```
#  $O(n \log n + m)$  where  $m$  is the shortest  
string length.
```

```
# Should I go ahead and optimize?"
```

```
class _14_LongestCommonPrefix_Brute:  
    def longestCommonPrefix(self, strs):  
        strs.sort()  
        first, last = strs[0], strs[-1]  
        i = 0  
        while i < len(first) and i <  
len(last) and first[i] == last[i]:  
            i += 1  
        return first[:i]
```

```
# PHASE 3 — OPTIMIZE
```

"The bottleneck is the $O(n \log n)$ sort – we can skip it entirely.

Vertical scanning: take the first string as reference. For each character position, # check that every other string has the same character at that position.

The first mismatch (or any string running short) terminates the prefix.

#

Time: $O(n*m)$ where n =number of strings, m =min string length. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _14_LongestCommonPrefix:

def longestCommonPrefix(self, strs):

if not strs:

return ""

for i, ch in enumerate(strs[0]):

for s in strs[1:]:

if i >= len(s) or s[i] !=

ch:

return strs[0][:i]

return strs[0]

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

['flower', 'flow', 'flight']:

i=0 'f': all match. i=1 'l': all match.

i=2 'o': strs[2][2]='i' ≠ 'o' → return

'fl' ✓

#

['dog', 'racecar', 'car']:

i=0 'd': strs[1][0]='r' ≠ 'd' → return

'' ✓

#

['abc']:

No inner loop fires (only one string) →

return 'abc' ✓

#

"No bugs. Returning strs[0][:i] when i=0 correctly returns empty string."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n*m)$: n strings × m min length.

Space $O(1)$: no extra allocation.

Sort-then-compare is $O(n \log n)$ –
vertical scan is strictly faster.

Follow-up: if the array is huge but
strings are short, binary search on prefix

length:

check if `strs[0][:mid]` is a prefix of all strings – $O(n \log m)$ total.

Follow-up: Trie – insert all strings and walk until a branch point."

Greedy

Round 4 · Mid · Easy · 1 question

Greedy

□ #409 Longest Palindrome

EAS

[Open on LeetCode](#)

Hash Table · String · Greedy

Lists: Grind 169

Problem

Given a string *s* which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

Input: *s* = "abcccccdd"

Output: 7

Explanation: One longest palindrome that can be built is "dccaccd", whose length is 7.

Example 2:

Input: s = "a"

Output: 1

Explanation: The longest palindrome that can be built is "a", whose length is 1.

Constraints:

- $1 \leq s.length \leq 2000$
- s consists of lowercase and/or uppercase English letters only.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given a string of letters, find the length of the longest palindrome we can BUILD

from those characters – we rearrange them, we don't find a substring.

Letters are case-sensitive. Right?"

#

"A few quick questions:

Can we use every character? Yes – we pick the optimal subset.

```
# Return the length, not the actual
palindrome string? Yes."
#
# "Let me walk: 'abcccccdd' → d×2, c×4,
a×1, b×1.
# Pairs: dd, cccc → 6 chars. One leftover
can be center → total 7 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Count frequency of each character.
# Sum all (freq // 2) * 2 (use every full
pair).
# If any character had an odd frequency,
add 1 for the center.
# O(n) time, O(1) space since at most 52
distinct letters.
# This IS essentially optimal – but
there's an even cleaner implementation.
Should I code it?"
class _409_LongestPalindrome_Brute:
    def longestPalindrome(self, s):
        from collections import Counter
        freq = Counter(s)
```

```
        length = sum(v // 2 * 2 for v in
freq.values())
        if length < len(s):
            length += 1
        return length
```

PHASE 3 — OPTIMIZE

"The bottleneck is the separate Counter build and scan steps."

The key insight: we only need to know which characters have ODD frequency.

Toggle each char in a set – present means odd count so far, absent means even.

Final answer = len(s) – len(odd_chars) + (1 if odd_chars else 0).

#

Time: O(n). Space: O(1) – at most 52 entries.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _409_LongestPalindrome:
    def longestPalindrome(self, s):
        odd_chars = set()
```

```
    for ch in s:
        if ch in odd_chars:
            odd_chars.discard(ch)
        else:
            odd_chars.add(ch)
    return len(s) - len(odd_chars) +
(1 if odd_chars else 0)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'abccccdd' :

a→{a}, b→{a,b}, c→{a,b,c}, c→{a,b},
c→{a,b,c}, c→{a,b}, d→{a,b,d}, d→{a,b}.

odd_chars={a,b}. len(s)=8. 8-2+1=7 ✓

#

'a' :

odd_chars={a}. 1-1+1=1 ✓

#

'aa' :

odd_chars={}. 2-0+0=2 ✓

#

"No bugs. The +1 only fires when there
are leftover characters to place at the
center."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: set has at most 52 entries.

The set-toggle is slightly more elegant than Counter since it avoids the intermediate dict.

Both approaches have identical complexity.

Follow-up: Palindrome Pairs (#336) – build palindromes from word concatenations.

Follow-up: if we must also return the actual palindrome, assign pairs to both ends

and place one odd character in the middle."

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

56 questions · 6 patterns

**Linked List #2 #19 #24 #61 #146 #148 #328
#369 #430 #622 #707 #708 #1171**

**Stack #143 #150 #155 #173 #227 #255 #394
#439 #484 #735 #739 #962 #1214 #1762**

**Heap #215 #253 #347 #505 #621 #658 #787
#973 #1057 #1167 #1424**

Trie #139 #208 #211 #616 #692 #720 #1166

Backtracking #17 #22 #39 #46 #79 #113

Greedy #134 #179 #280 #954 #2131

Linked List

Round 5 · Mid · Medium · 13 questions

Linked List

#2 Add Two Numbers

MED[Open on LeetCode](#)

Linked List · Math · Recursion

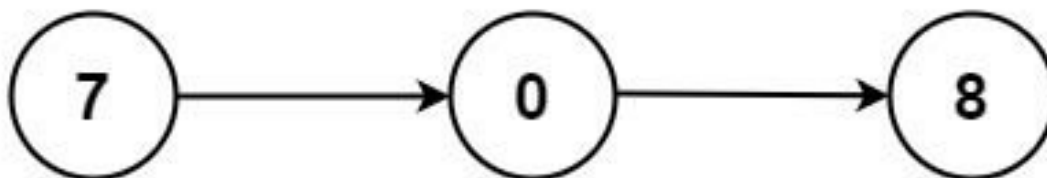
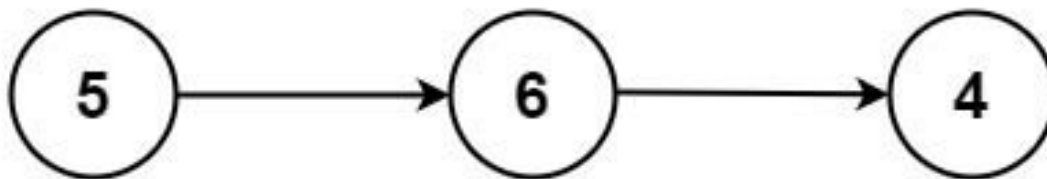
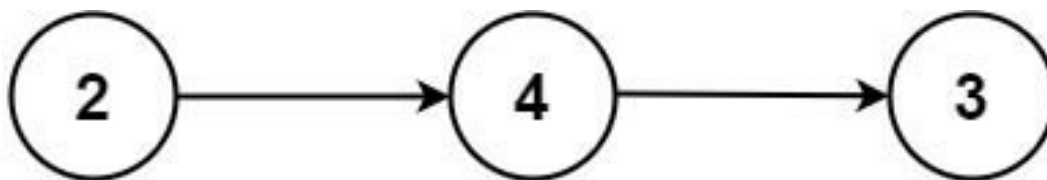
Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: $l1 = [2, 4, 3]$, $l2 = [5, 6, 4]$

Output: $[7, 0, 8]$

Explanation: $342 + 465 = 807$.

Example 2:

Input: $l1 = [0]$, $l2 = [0]$

Output: $[0]$

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 =
[9,9,9,9]
Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Two non-empty linked lists store digits of non-negative integers in REVERSE order.

Add the two numbers and return the sum as a reversed linked list. Right?"

#

"A few quick questions:

Can leading zeros appear in the input? Only for the number 0 itself.

Can the lists have different lengths? Yes – one might be much longer.

```
# Should I handle carry past the longer
list? Yes – e.g. 9+9 propagates."
#
# "Let me walk: l1=[2,4,3] (342),
l2=[5,6,4] (465). 342+465=807 → [7,0,8] ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Convert both lists to integers, add
them, then convert the result back to a
linked list.
# This is O(n) time and O(n) space but
relies on Python big-int arithmetic and
# doesn't generalise to languages with
fixed-width integers.
# Should I go ahead and optimize?"
class _2_AddTwoNumbers_Brute:
    def addTwoNumbers(self, l1, l2):
        def to_int(node):
            val, mul = 0, 1
            while node: val += node.val *
mul; mul *= 10; node = node.next
            return val
        total = to_int(l1) + to_int(l2)
        dummy = cur = ListNode(0)
```

```
        if total == 0: return ListNode(0)
    while total:
        cur.next = ListNode(total %
10); cur = cur.next; total //= 10
    return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the big-int conversion — it breaks in fixed-width languages.

Simulate grade-school addition digit by digit with a carry variable.

Walk both lists simultaneously; at each step: total = d1 + d2 + carry.

New digit = total % 10, new carry = total // 10.

Continue until both lists are exhausted AND carry is 0.

#

Time: $O(\max(m,n))$. Space: $O(\max(m,n))$ for the result list.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _2_AddTwoNumbers:
    def addTwoNumbers(self, l1, l2):
        dummy = cur = ListNode(0)
        carry = 0
        while l1 or l2 or carry:
            d1 = l1.val if l1 else 0
            d2 = l2.val if l2 else 0
            total = d1 + d2 + carry
            carry, digit = divmod(total,
10)

            cur.next = ListNode(digit)
            cur = cur.next
            if l1: l1 = l1.next
            if l2: l2 = l2.next
        return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

l1=[2,4,3], l2=[5,6,4]:

2+5+0=7 carry=0 → node(7). 4+6+0=10

carry=1 → node(0). 3+4+1=8 carry=0 →
node(8).

Both None, carry=0 → stop. → [7,0,8] ✓

#

l1=[9,9,9,9], l2=[9,9,9]:

Each position: 9+9+carry propagates. →
[8,9,9,0,1] ✓

#

"No bugs. The loop condition 'while l1
or l2 or carry' handles unequal lengths
cleanly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\max(m,n))$: process as many
digits as the longer number plus at most
one carry.

Space $O(\max(m,n))$: result list.

Follow-up: Add Two Numbers II (#445) –
digits in forward order; use two stacks to
reverse.

Follow-up: Multiply Strings (#43) –
similar positional digit logic, $O(n*m)$."

Linked List

#19 Remove Nth Node From End of List

MED[Open on LeetCode](#)

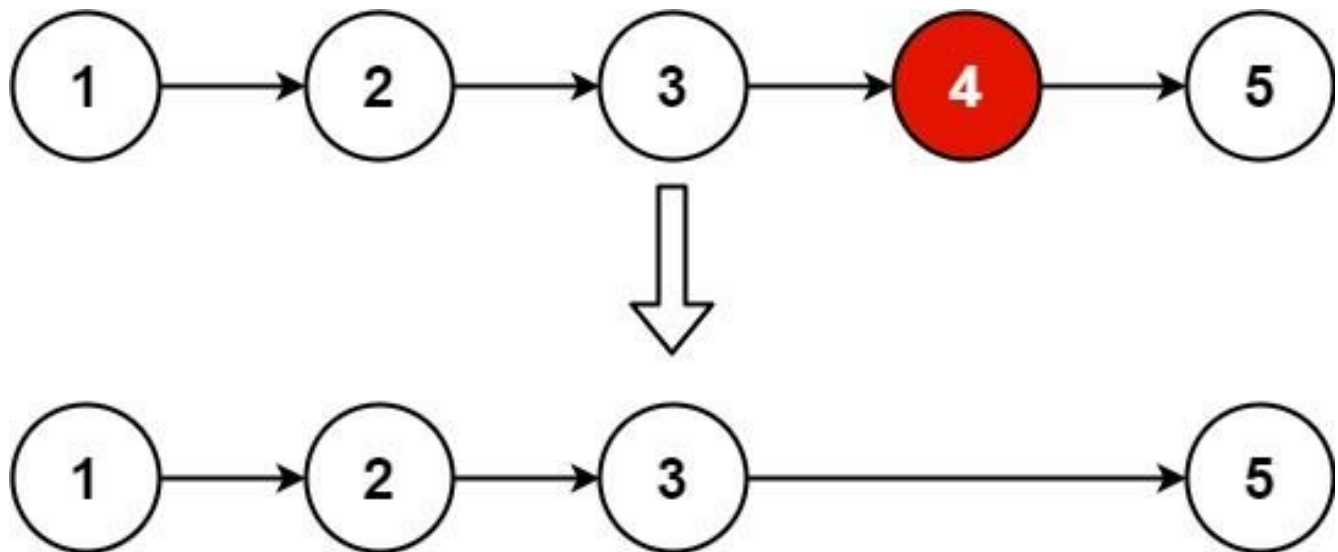
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, remove the n th node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Remove the nth node from the END of the list and return the modified head.

n is always valid. The follow-up asks for one pass. Right?"

#

"A few quick questions:


```
# Can n equal the list length (removing  
the head)? Yes – handle with a dummy node.  
# Return the head, which may change if we  
remove it."
```

```
#
```

```
# "Let me walk: [1,2,3,4,5], n=2 → remove  
4 → [1,2,3,5] ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Two passes: count length L in the first  
pass, then walk to position L-n and  
unlink.
```

```
# Use a dummy head to cleanly handle  
removing the actual head node.
```

```
# O(L) time, O(1) space – correct but uses  
two passes.
```

```
# Should I go ahead and optimize?"
```

```
class _19_RemoveNthFromEnd_Brute:
```

```
    def removeNthFromEnd(self, head, n):
```

```
        dummy = ListNode(0, head)
```

```
        length, cur = 0, head
```

```
        while cur: length += 1; cur =
```

```
cur.next
```

```
        cur = dummy
```

```
        for _ in range(length - n): cur =  
cur.next  
        cur.next = cur.next.next  
    return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the two-pass approach
— we can do it in one.

Advance a 'fast' pointer n+1 steps ahead
of 'slow'.

Move both together until fast reaches
None.

Then slow.next is the node to remove.

The dummy head ensures we never
special-case removing the actual head.

#

Time: $O(L)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

```
class _19_RemoveNthFromEnd:
```

```
    def removeNthFromEnd(self, head, n):  
        dummy = ListNode(0, head)  
        fast = slow = dummy
```

```
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5], n=2: advance fast 3 steps →
dummy,1,2,3. slow=dummy.

Move together: slow=1,fast=4 →
slow=2,fast=5 → slow=3,fast=None → stop.

slow=3. 3.next=5. → [1,2,3,5] ✓

#

[1], n=1: fast advances n+1=2 steps →
fast=None after first step
(head.next=None).

while fast skipped. slow=dummy.
dummy.next=None. → [] ✓

#

"No bugs. Advancing n+1 (not n) steps
puts slow just before the target node."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(L)$: one pass. Space $O(1)$: two pointer variables plus the dummy node.

Dummy node is essential – without it, removing the head requires a special branch.

Follow-up: Middle of Linked List (#876) – same slow/fast pointer family.

Follow-up: if n could be invalid ($>$ list length), add a length check before advancing."

Linked List

□ #24 Swap Nodes in Pairs

MED[Open on LeetCode](#)

Linked List · Recursion

Lists: Grind 169

Problem

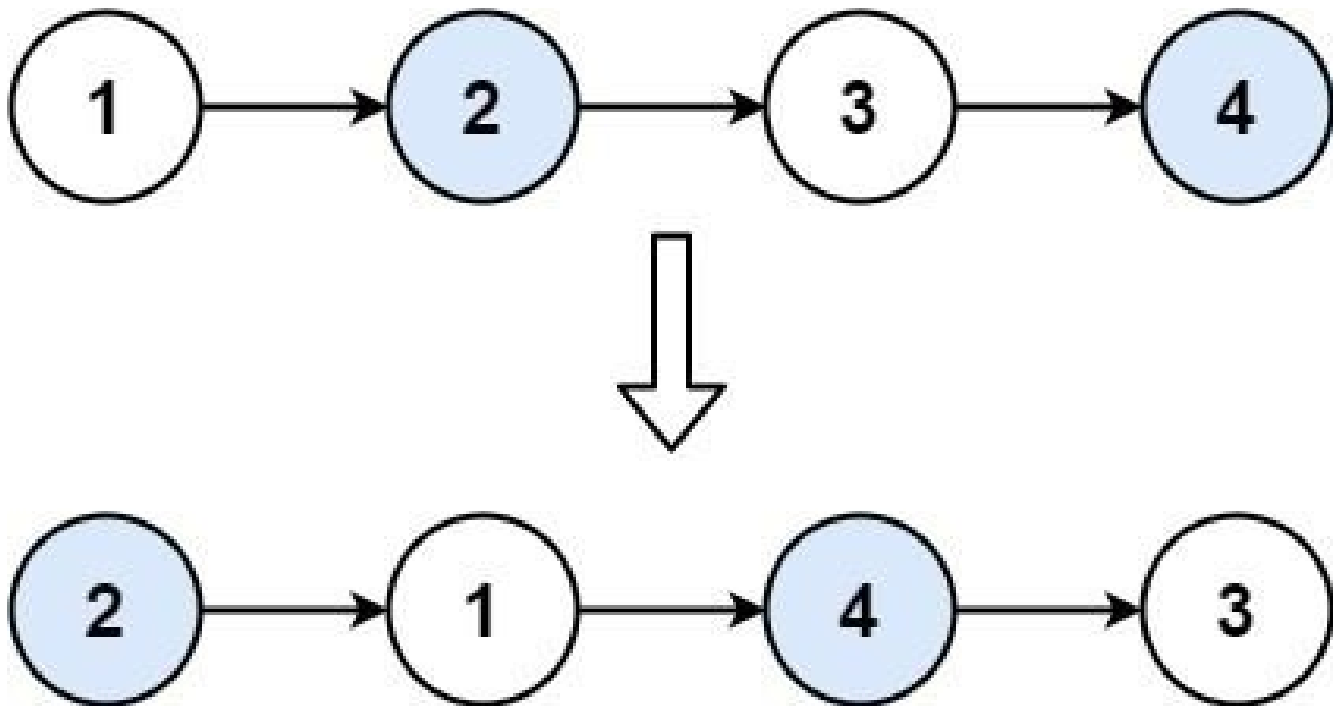
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Swap every two adjacent nodes and return the new head.

We swap the actual nodes – not just their values.

If there's an odd node at the end, leave it unchanged. Right?"

#

"A few quick questions:

Can the list be empty? Yes – return None.

O(1) space (no new nodes, just pointer rewiring)?"

#

"Let me walk: [1,2,3,4] → [2,1,4,3] ✓.
[1,2,3] → [2,1,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all values, swap adjacent pairs in the array, rebuild the list.

$O(n)$ time but $O(n)$ space for the values
– and we create new nodes.

Should I go ahead and optimize?"

```
class _24_SwapNodesInPairs_Brute:
```

```
    def swapPairs(self, head):
```

```
        vals = []
```

```
        cur = head
```

```
        while cur: vals.append(cur.val);
```

```
        cur = cur.next
```

```
        for i in range(0, len(vals) - 1,  
2): vals[i], vals[i+1] = vals[i+1],  
vals[i]
```

```
        cur = head
```

```
        for v in vals: cur.val = v; cur =  
cur.next
```

```
        return head
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ auxiliary
list.

In-place pointer rewiring with a dummy
predecessor:

For each pair (first, second): set
prev.next=second, first.next=second.next,
second.next=first.


```
# Advance prev to first (now the second
node in output) and repeat.
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _24_SwapNodesInPairs:
    def swapPairs(self, head):
        dummy = ListNode(0, head)
        prev = dummy
        while prev.next and
prev.next.next:
            first = prev.next
            second = prev.next.next
            prev.next = second
            first.next = second.next
            second.next = first
            prev = first
        return dummy.next

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
```

```
# [1,2,3,4]: dummy→1→2→3→4.
# prev=dummy, first=1, second=2: dummy→2,
1→3, 2→1. prev=1. List: dummy→2→1→3→4.
# prev=1, first=3, second=4: 1→4, 3→None,
4→3. prev=3. List: dummy→2→1→4→3.
# prev.next=None → stop. → [2,1,4,3] ✓
#
# [1]: prev.next.next=None → loop never
enters → [1] ✓
#
# "No bugs. Saving 'second.next' before
rewiring prevents losing the rest of the
list."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): visit each node once. Space
O(1): constant pointer variables.
# The dummy node eliminates special-casing
the very first pair.
# Follow-up: Reverse Nodes in k-Group
(#25) – generalise to groups of k using
the same logic.
# Follow-up: recursive solution swaps the
pair then recurses on head.next.next –
clean but O(n) stack."
```

Linked List

□ #61 Rotate List

MED[Open on LeetCode](#)

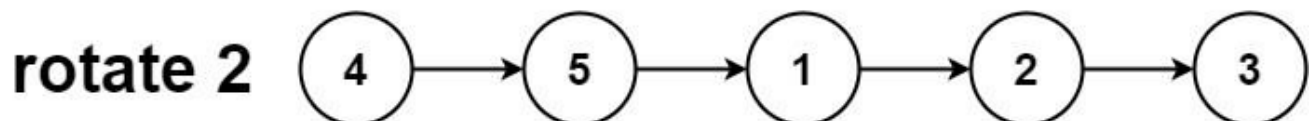
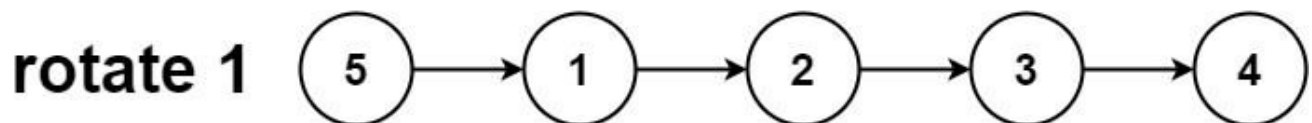
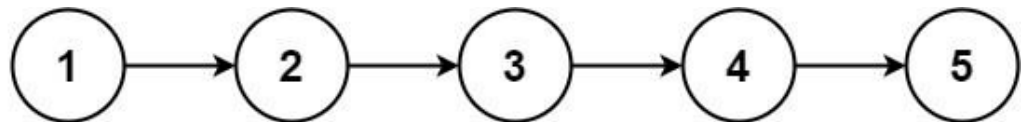
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, rotate the list to the right by k places.

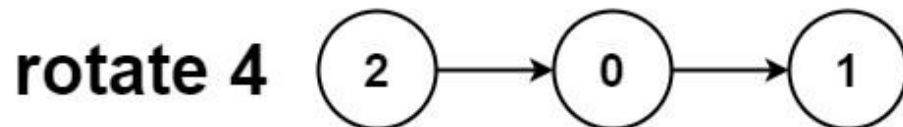
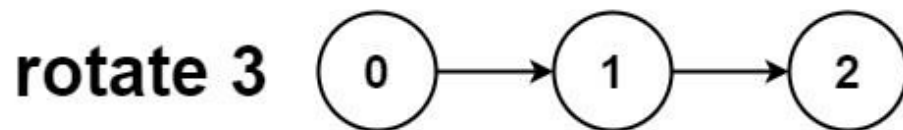
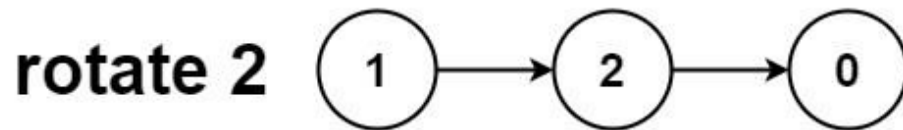
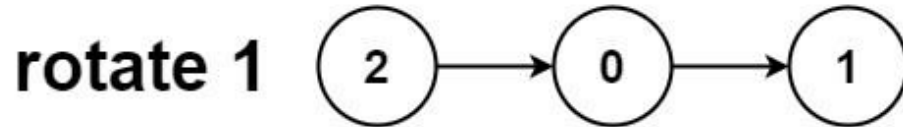
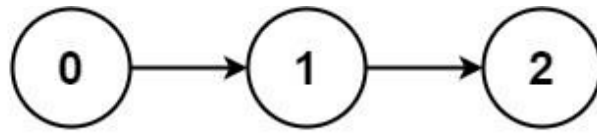
Example 1:



Input: head = [1,2,3,4,5], k = 2

Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Rotate the linked list to the right by k places – each rotation moves the tail to the front.

k can exceed the list length; rotations are modulo the length. Right?"

#

"A few quick questions:

Can k be 0? Yes – return unchanged. Can the list be empty? Yes – return None.

k up to 2×10^9 – so we definitely need the modulo trick."

#

"Let me walk: [1,2,3,4,5], k=2 → [4,5,1,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Perform k individual right-rotations: each time, remove the tail and prepend it to the head.

```
# 0(n*k) time – for k=2*10^9 this is
completely infeasible.
# Should I go ahead and optimize?"
class _61_RotateList_Brute:
    def rotateRight(self, head, k):
        if not head or not head.next:
            return head
        for _ in range(k):
            cur = head
            while cur.next.next: cur =
cur.next
            tail = cur.next; cur.next =
None; tail.next = head; head = tail
        return head

# PHASE 3 — OPTIMIZE
# "The bottleneck is the 0(n*k) brute –
full cycles cancel so we only need k %
length rotations.
# Three steps in two passes:
# 1. Find length and tail by traversing
once.
# 2. Effective k = k % length. If 0,
return as-is.
# 3. New tail is at position (length – k –
1). New head is new_tail.next.
```

```
# Connect old tail to old head and cut at
new tail.
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _61_RotateList:
    def rotateRight(self, head, k):
        if not head or not head.next or k
        == 0:
            return head
        tail, length = head, 1
        while tail.next:
            tail = tail.next
            length += 1
        k %= length
        if k == 0:
            return head
        new_tail = head
        for _ in range(length - k - 1):
            new_tail = new_tail.next
        new_head = new_tail.next
        new_tail.next = None
```

```
tail.next = head
return new_head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5], k=2: length=5, tail=5.
k=2%5=2. Walk 5-2-1=2 steps → new_tail=3.
new_head=4. 3.next=None. 5.next=1. →
[4,5,1,2,3] ✓

#

[0,1,2], k=4: k=4%3=1. Walk 3-1-1=1 step
→ new_tail=1. new_head=2. 1.next=None.
2.next=0.

→ [2,0,1] ✓

#

"No bugs. k%length=0 short-circuit
avoids a no-op cut-and-rejoin."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass for length, one
walk to new_tail. Space $O(1)$."

The k%length shortcut is critical –
without it k=10⁹ on a 5-node list would
loop forever.

Follow-up: Rotate Array (#189) – same modulo idea but uses three reversals on an array.

Follow-up: right-shift by 1 on a circular buffer is $O(1)$ with a pointer – different trade-off."

Linked List

□ #146 LRU Cache

MED[Open on LeetCode](#)

Hash Table · Linked List · Design

Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- **LRUCache(int capacity)** Initialize the LRU cache with positive size capacity.
- **int get(int key)** Return the value of the key if the key exists, otherwise return -1.
- **void put(int key, int value)** Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get",  
"put", "get", "put", "get", "get",  
"get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2],  
[4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null,  
-1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 104$
- $0 \leq \text{value} \leq 105$
- At most $2 * 105$ calls will be made to get and put.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Design an LRU cache with a fixed
capacity: get(key)→value or -1, and
put(key,value).
# When capacity is exceeded, evict the
least recently used item.
# Both operations must be O(1) average
time. Right?"
#
# "A few quick questions:
# Does get count as a use (making the item
recently used)? Yes.
# Does put update an existing key's value
and recency? Yes."
#
# "Let me walk capacity=2: put(1,1),
put(2,2), get(1)→1 (1 is now MRU),
# put(3,3) evicts 2, get(2)→-1 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Use a regular dict plus a list tracking
insertion order.
# On each access, move the key to the end
of the list; on eviction pop from the
front.
```

```
# get/put are  $O(1)$  for the dict but  $O(n)$ 
for the list shift.
# Should I go ahead and optimize?"
class _146_LRUCache_Brute:
    def __init__(self, capacity):
        self.cap = capacity; self.cache =
        {}; self.order = []
    def get(self, key):
        if key not in self.cache: return
        -1
        self.order.remove(key);
self.order.append(key); return
self.cache[key]
    def put(self, key, value):
        if key in self.cache:
self.order.remove(key)
        elif len(self.cache) == self.cap:
            del
self.cache[self.order.pop(0)]
            self.cache[key] = value;
self.order.append(key)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n)$  list shift
on every access.
```

```
# HashMap + Doubly Linked List gives O(1)
for both:
# HashMap: key → DLL node (O(1) lookup).
# DLL: maintains access order – head=LRU,
tail=MRU.
# get: move node to tail. put: add at
tail; if over capacity, remove from head.
# Use sentinel head/tail nodes to avoid
edge-case pointer checks.
#
# Time: O(1) all ops. Space: O(capacity)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _146_LRUCache:
    class _Node:
        def __init__(self, key=0, val=0):
            self.key = key; self.val =
val; self.prev = self.next = None
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head = self._Node();
self.tail = self._Node()
```

```
        self.head.next = self.tail;
self.tail.prev = self.head
    def _remove(self, node):
        node.prev.next = node.next;
node.next.prev = node.prev
    def _insert_tail(self, node):
        node.prev = self.tail.prev;
node.next = self.tail
        self.tail.prev.next = node;
self.tail.prev = node
    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self._remove(node);
self._insert_tail(node)
        return node.val
    def put(self, key, value):
        if key in self.map:
self._remove(self.map[key])
        node = self._Node(key, value)
        self.map[key] = node;
self._insert_tail(node)
        if len(self.map) > self.cap:
            lru = self.head.next
```

```
        self._remove(lru); del  
self.map[lru.key]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

cap=2: put(1,1)→tail:[1].

put(2,2)→tail:[1,2]. get(1)→move 1 to
tail:[2,1].

put(3,3): over cap, evict

head.next=node(2). cache={1,3}.

tail:[1,3] ✓

get(2)→-1 (evicted) ✓

#

"No bugs. Sentinel nodes eliminate all
null checks in _remove/_insert_tail."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(1)$ all ops: HashMap $O(1)$ lookup
+ DLL $O(1)$ insert/remove.

Space $O(\text{capacity})$: at most capacity
nodes in the DLL and map.

Python's OrderedDict implements this
internally – valid in an interview to
mention.

Follow-up: LFU Cache (#460) – evict least FREQUENTLY used; needs two hashmaps + DLL per frequency.
Follow-up: thread-safe LRU – wrap get/put with a `threading.Lock`."

Linked List

#148 Sort List

MED[Open on LeetCode](#)

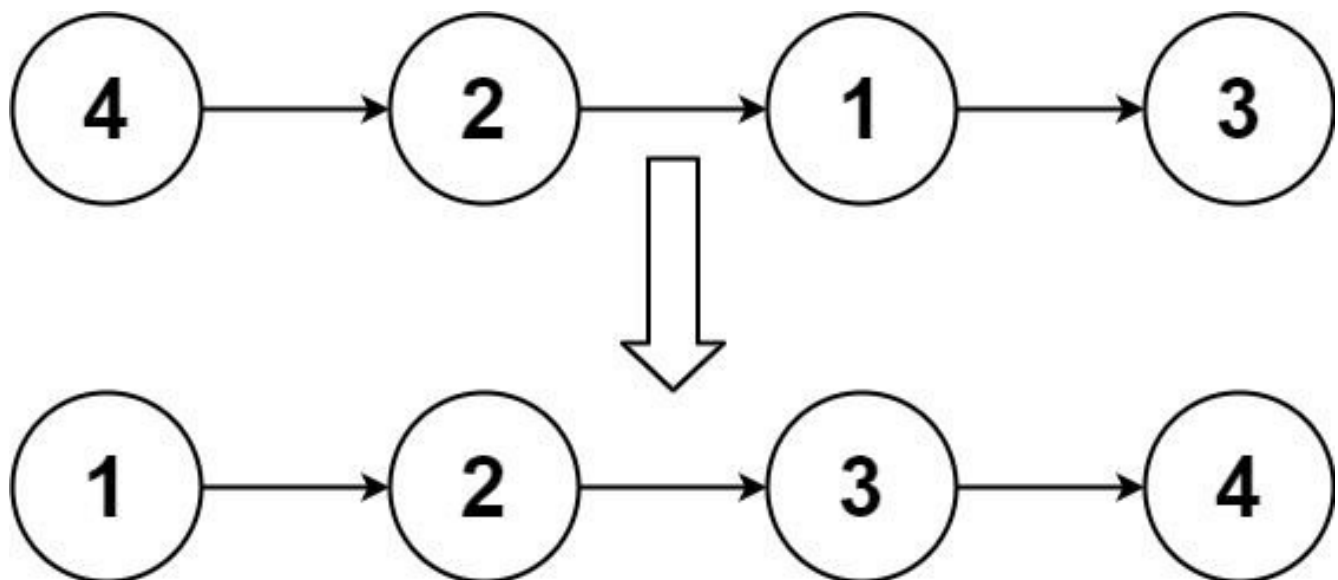
Linked List · Two Pointers · Divide and Conquer
· Sorting · Merge Sort

Lists: Grind 169

Problem

Given the head of a linked list, return the list after sorting it in ascending order.

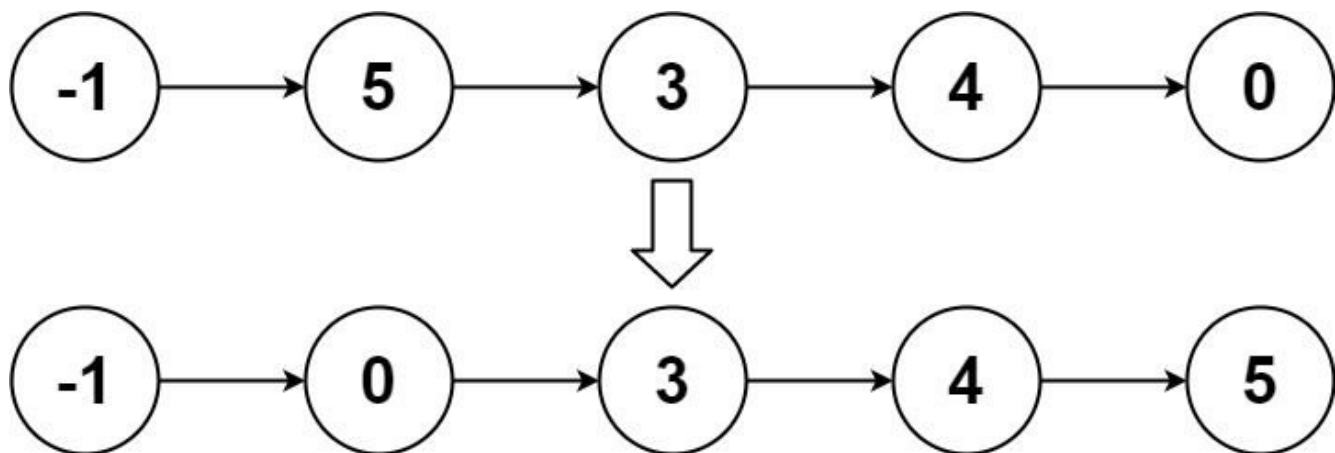
Example 1:



Input: head = [4,2,1,3]

Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]

Output: [-1,0,3,4,5]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is in the range [0, 5 * 10⁴].
- -105 ≤ Node.val ≤ 105

Follow up: Can you sort the linked list in O(n log n) time and O(1) memory (i.e. constant space)?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Sort a linked list in ascending order.
# The follow-up wants  $O(n \log n)$  time and
 $O(1)$  space. Right?"
#
# "A few quick questions:
# Values can be negative? Yes. Empty list
is valid? Yes – return None."
#
# "Let me walk: [4,2,1,3] → [1,2,3,4] ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Collect all values into a Python list,
sort it, then overwrite node values in
order.
#  $O(n \log n)$  time but  $O(n)$  extra space for
the values array.
# The follow-up requires  $O(1)$  extra space
– so we need a true in-place sort.
# Should I go ahead and optimize?"
class _148_SortList_Brute:
    def sortList(self, head):
        vals, cur = [], head
```

```
        while cur: vals.append(cur.val);
cur = cur.next
        vals.sort()
        cur = head
        for v in vals: cur.val = v; cur =
cur.next
        return head
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ auxiliary list.

Bottom-up merge sort achieves $O(n \log n)$ time and $O(1)$ space (no recursion stack):

Start by merging pairs of size 1, then pairs of size 2, then 4, and so on.

Each pass merges n/size pairs, and there are $\log n$ passes.

A dummy head simplifies relinking across rounds.

#

Time: $O(n \log n)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _148_SortList:
    def sortList(self, head):
        def merge(l1, l2):
            dummy = cur = ListNode(0)
            while l1 and l2:
                if l1.val <= l2.val:
                    cur.next = l1; l1 = l1.next
                else: cur.next = l2; l2 = l2.next
                cur = cur.next
            cur.next = l1 if l1 else l2
            return dummy.next
        def split(head, size):
            cur = head
            for _ in range(size - 1):
                if cur: cur = cur.next
            if not cur: return None, None
            rest = cur.next; cur.next = None
            return head, rest
        length, cur = 0, head
        while cur: length += 1; cur = cur.next
        size = 1
        while size < length:
```

```
        dummy = tail = ListNode(0)
        cur = head
        while cur:
            left, cur = split(cur,
size)
            right, cur = split(cur,
size)
            tail.next = merge(left,
right)
            while tail.next: tail =
tail.next
        head = dummy.next
        size *= 2
    return head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[4,2,1,3], size=1: merge(4,2)=[2,4],
merge(1,3)=[1,3]. List: 2→4→1→3.

size=2: merge([2,4],[1,3])=[1,2,3,4]. →
[1,2,3,4] ✓

#

"No bugs. The split helper correctly
handles lists shorter than 'size' by
returning None."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: $\log n$ passes, each $O(n)$. Space $O(1)$: no recursion stack.

Top-down merge sort is simpler to write but uses $O(\log n)$ stack space.

Follow-up: Merge k Sorted Lists (#23) – use this merge step in a heap for $O(n \log k)$.

Follow-up: insertion sort for linked list (#147) – $O(n^2)$ but stable and in-place."

Linked List

□ #328 Odd Even Linked List

MED

[Open on LeetCode](#)

Linked List

Lists: Grind 169

Problem

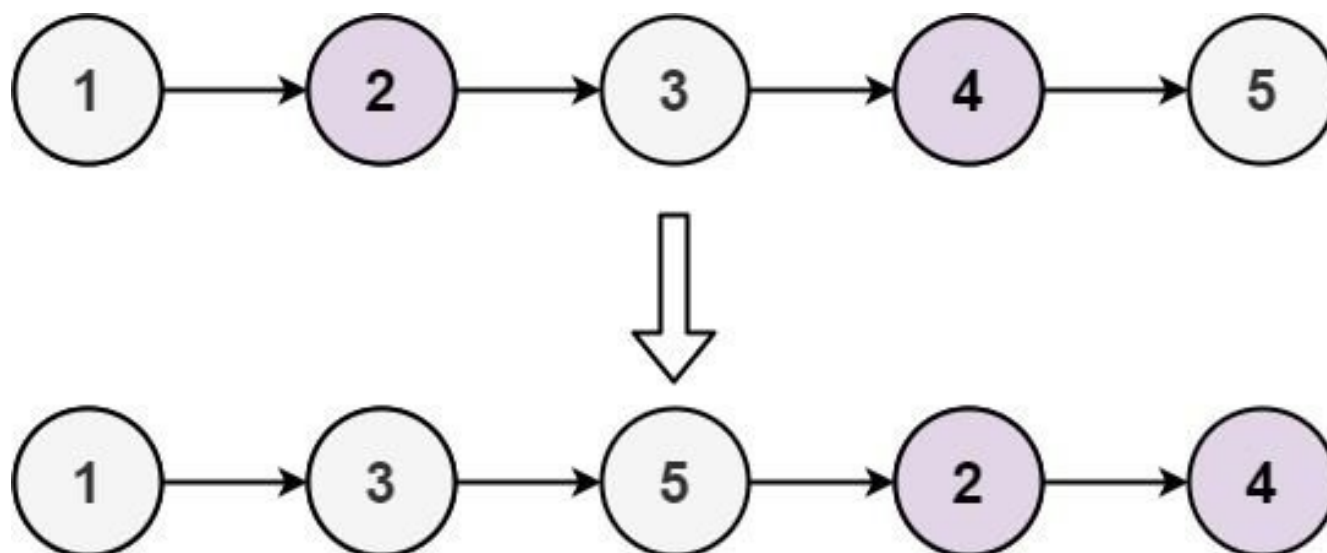
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

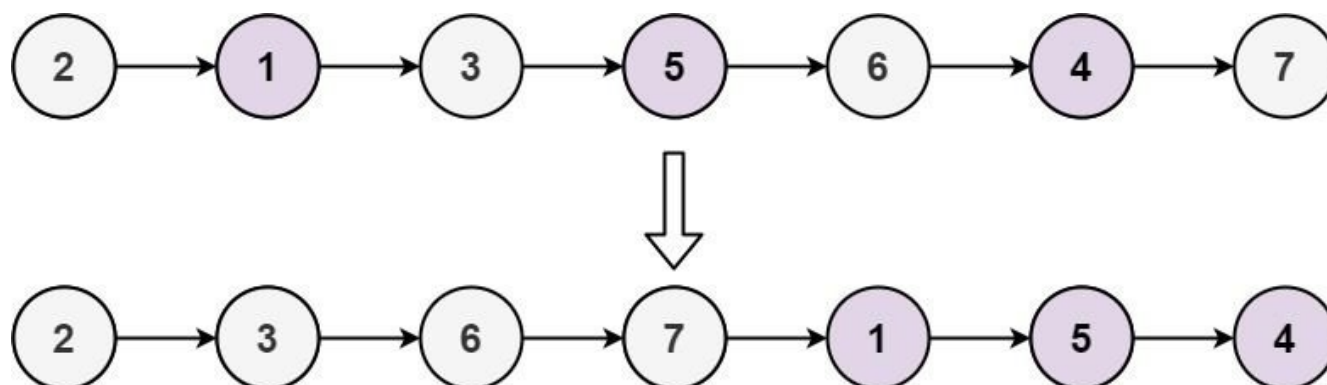
Example 1:



Input: head = [1,2,3,4,5]

Output: [1,3,5,2,4]

Example 2:



Input: head = [2,1,3,5,6,4,7]

Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-106 \leq \text{Node.val} \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Group all odd-position nodes first, then all even-position nodes.

1-indexed positions: node 1,3,5,... then 2,4,6,...

Preserve relative order within each group. $O(1)$ space, $O(n)$ time. Right?"

#

"A few quick questions:

Grouping is by POSITION not value? Yes – the problem is about indices.

Return the head of the modified list."

#

"Let me walk: $[1,2,3,4,5] \rightarrow [1,3,5,2,4]$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect odd-position and even-position values separately, then rebuild the list.

```
# 0(n) time but 0(n) space for the two
value lists.
# Should I go ahead and optimize?"
class _328_OddEvenLinkedList_Brute:
    def oddEvenList(self, head):
        odds, evens, cur, i = [], [],
head, 1
        while cur: (odds if i % 2 else
evens).append(cur.val); cur = cur.next; i
+= 1
        cur = head
        for v in odds + evens: cur.val =
v; cur = cur.next
        return head
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ auxiliary lists.

In-place pointer manipulation with two sub-list heads:

`odd.next = odd.next.next` (skip even nodes in the odd chain).

`even.next = even.next.next` (skip odd nodes in the even chain).

At the end, link the tail of the odd chain to the head of the even chain.

```
# Save even_head before the loop so we can  
link at the end.
```

```
#
```

```
# Time: O(n). Space: O(1).
```

```
# Does that make sense before I code it?"
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _328_OddEvenLinkedList:
```

```
    def oddEvenList(self, head):
```

```
        if not head or not head.next:
```

```
            return head
```

```
        odd = head
```

```
        even = even_head = head.next
```

```
        while even and even.next:
```

```
            odd.next = even.next
```

```
            odd = odd.next
```

```
            even.next = odd.next
```

```
            even = even.next
```

```
        odd.next = even_head
```

```
        return head
```

```
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
```

```
#
```

```
# [1,2,3,4,5]: odd=1, even=2,
even_head=2.
# iter1: odd.next=3, odd=3.
even.next=4, even=4.
# iter2: odd.next=5, odd=5.
even.next=None, even=None.
# odd.next=even_head=2. → 1→3→5→2→4→None
✓
#
# "No bugs. even_head must be saved before
the loop since we modify even's next."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): two
pointer variables.
# Contrast with the brute that copies
values – this reuses original nodes.
# Follow-up: Partition List (#86) –
partition by a value, not by position
index.
# Follow-up: Reorder List (#143) –
interleave first and reversed second
half."
```

Linked List

□ ★ #369 Plus One Linked List ★

MED

[Open on LeetCode](#)

Linked List · Math · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a non-negative integer represented as a linked list of digits, plus one to the integer.

The digits are stored such that the most significant digit is at the head of the list.

Example 1:

Input: head = [1,2,3]

Output: [1,2,4]

Example 2:

Input: head = [0]

Output: [1]

Constraints:

- The number of nodes in the linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$

- The number represented by the linked list does not contain leading zeros except for the zero itself.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A non-negative integer is stored as a linked list of digits, most significant first.

Add one to the number and return the updated list. Right?"

#

"A few quick questions:

Can all digits be 9 (e.g. $999+1=1000$)?
Yes — we'd need a new head node.

Can the number be 0? Yes — $0+1=1$."

#

"Let me walk: $[1,2,3] \rightarrow 123+1=124 \rightarrow [1,2,4] \checkmark$. $[9,9,9] \rightarrow 999+1=1000 \rightarrow [1,0,0,0] \checkmark$ "

PHASE 2 — BRUTE FORCE


```
# "Let me start with brute force to lock
in the logic.
# Convert the linked list to an integer,
add 1, convert back to a linked list.
# Works for small numbers but overflows in
fixed-width languages.
# Should I go ahead and optimize?"
class _369_PlusOneLinkedList_Brute:
    def plusOne(self, head):
        num, cur = 0, head
        while cur: num = num * 10 +
cur.val; cur = cur.next
        num += 1
        dummy = cur = ListNode(0)
        for ch in str(num): cur.next =
ListNode(int(ch)); cur = cur.next
        return dummy.next

# PHASE 3 — OPTIMIZE
# "The bottleneck is the big-int
round-trip that breaks in other languages.
# Find the rightmost non-9 digit,
increment it, and set all digits after it
to 0.
# If all digits are 9, prepend a new node
with value 1 and zero out all existing
```

nodes.

Use a dummy node with value 0 prepended to handle the all-9s case cleanly.

#

Time: $O(n)$. Space: $O(1)$ – in-place modification.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _369_PlusOneLinkedList:

def plusOne(self, head):

dummy = ListNode(0)

dummy.next = head

not_nine = dummy

cur = head

while cur:

if cur.val != 9: not_nine =

cur

cur = cur.next

not_nine.val += 1

cur = not_nine.next

while cur:

cur.val = 0; cur = cur.next

```
        return dummy.next if dummy.val ==  
0 else dummy
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# [1,2,9]: not_nine ends at node(2).  
node(2).val=3. node(9).val=0. → [1,3,0] ✓
```

```
#
```

```
# [9,9,9]: not_nine stays at dummy(0).  
dummy.val=0+1=1. 9→0,9→0,9→0.
```

```
# dummy.val=1 → return dummy → [1,0,0,0] ✓
```

```
#
```

```
# "No bugs. Returning dummy when  
dummy.val≠0 handles the all-9s carry-out  
cleanly."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): two passes (find not_nine,  
zero out tail). Space O(1)."
```

```
# The dummy-node trick elegantly handles  
the all-9s case without a special branch.
```

```
# Follow-up: Add Two Numbers (#2) – two  
lists, similar carry propagation.
```

```
# Follow-up: Subtract 1 from a linked list  
– find rightmost non-0, decrement, set
```

rest to 9."

Linked List

□ ★ #430 Flatten a Multilevel Doubly Linked List ★

MED

[Open on LeetCode](#)

Linked List · DFS · Doubly Linked List

Lists: ★ Premium | Premium 98

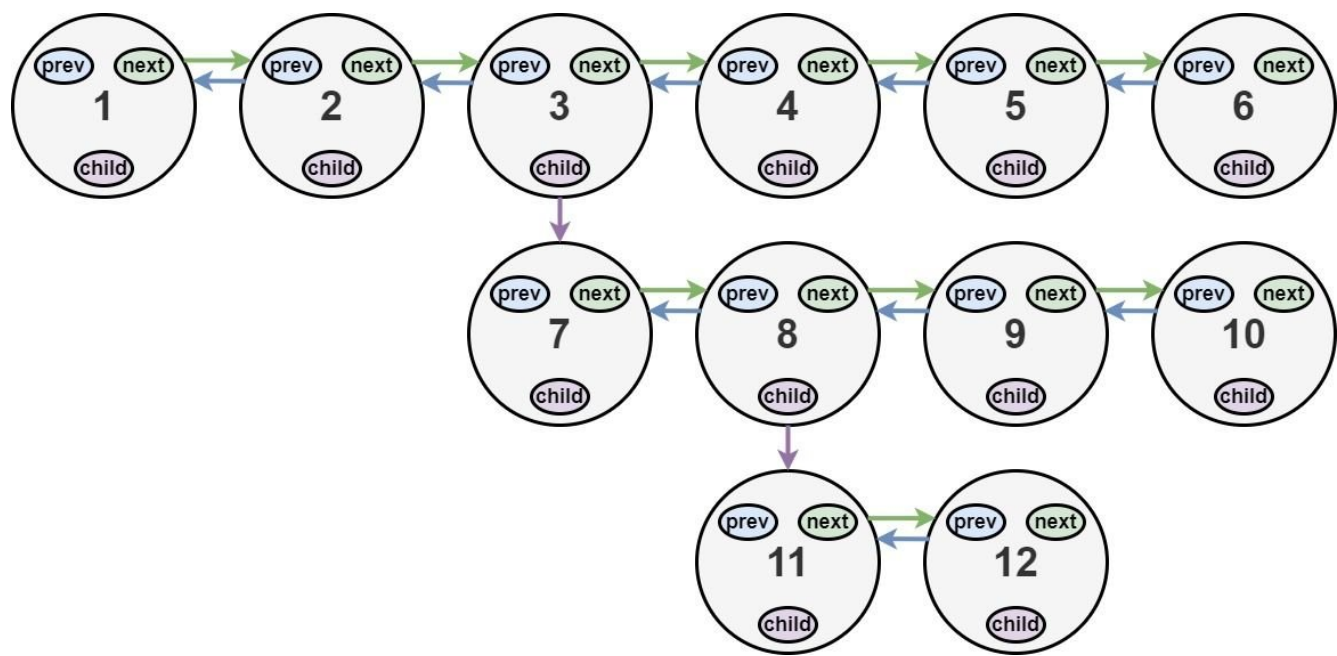
Problem

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let `curr` be a node with a child list. The nodes in the child list should appear after `curr` and before `curr.next` in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:



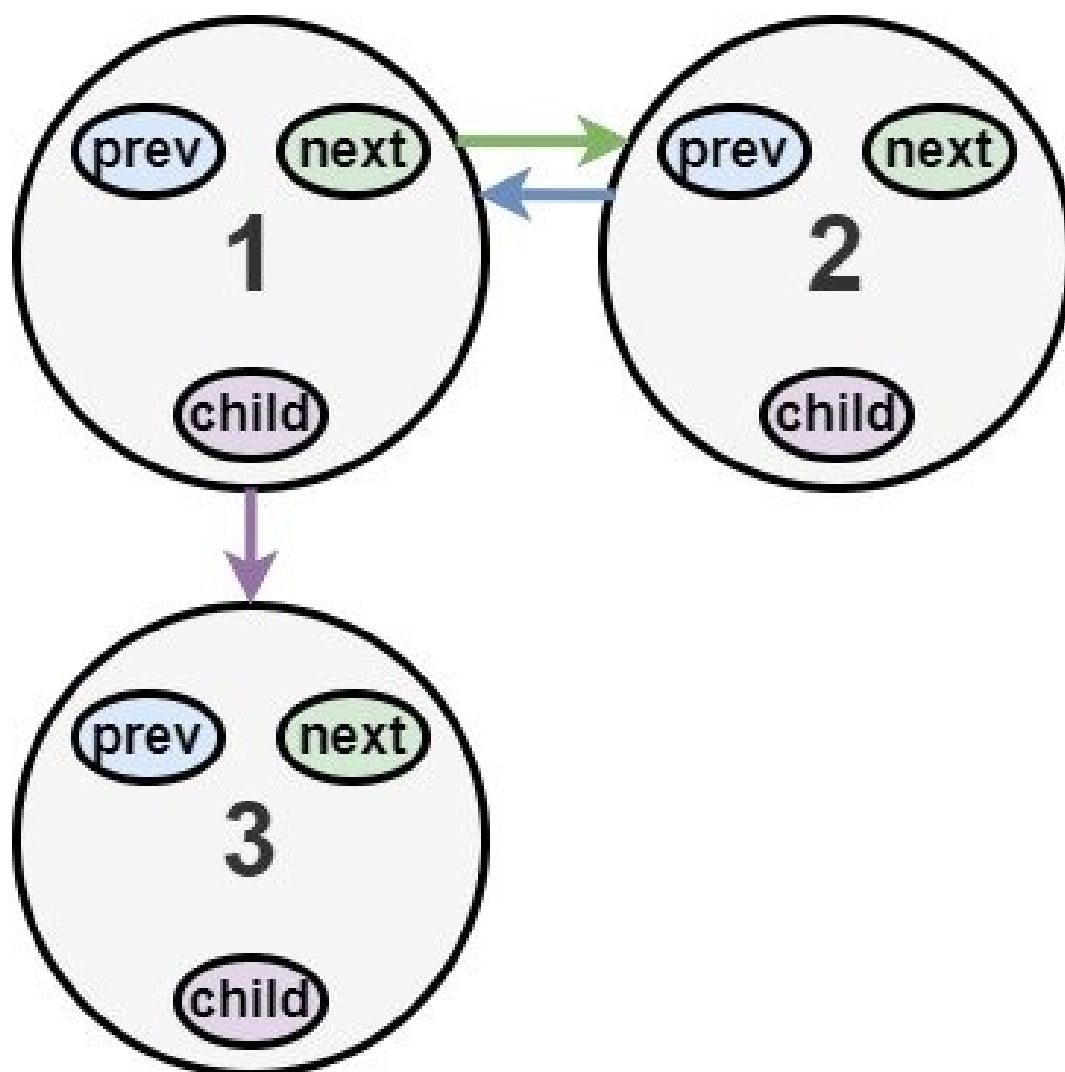
Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

Output: [1,2,3,7,8,11,12,9,10,4,5,6]

Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:

Example 2:



Input: head = [1,2,null,3]

Output: [1,3,2]

Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:

Example 3:

Input: head = []

Output: []

Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.
- $1 \leq \text{Node.val} \leq 105$

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```

1---2---3---4---5---6--NULL
|
7---8---9---10--NULL
|
11--12--NULL
  
```

The serialization of each level is as follows:

```

[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
  
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The

serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
|
[null, null, 7, 8, 9, 10, null]
|
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A doubly linked list where nodes can have a child pointer to a sub-list.

Flatten it so all nodes appear in a single-level DLL in DFS pre-order

(node before its children, children before the rest of node's next). Right?"

#

"A few quick questions:

```
# All child pointers must be set to null
in the result? Yes.
# 1→2→3(child:7→8→9)→4: flattened →
1→2→3→7→8→9→4 ✓"
#
# "Let me walk: 1-2-3-4-5-6, node 3 has
child 7-8-9-10, node 8 has child 11-12.
# Result: 1-2-3-7-8-11-12-9-10-4-5-6 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Collect all nodes in DFS pre-order
(visit node, recurse into child, continue
next),
# then relink them as a flat doubly linked
list.
# O(n) time but O(n) space for the
collected node list.
# Should I go ahead and optimize?"
class _430_FlattenMultilevel_Brute:
    def flatten(self, head):
        nodes = []
        def dfs(node):
            while node:
nodes.append(node); dfs(node.child); node
```

```

= node.next
    dfs(head)
    for i in range(len(nodes) - 1):
        nodes[i].next = nodes[i+1];
nodes[i+1].prev = nodes[i];
nodes[i].child = None
    if nodes: nodes[-1].child =
nodes[-1].next = None
    return nodes[0] if nodes else None

```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ auxiliary list.

Use a stack: when we encounter a node with a child, push its current next onto the stack,

insert the child inline, and continue.

When we reach a node with no next but the stack is non-empty, pop to resume the saved tail.

#

Time: $O(n)$. Space: $O(d)$ where d = max nesting depth.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _430_FlattenMultilevel:
    def flatten(self, head):
        stack = []
        cur = head
        while cur:
            if cur.child:
                if cur.next:
                    stack.append(cur.next)
                    cur.next = cur.child
                    cur.child.prev = cur
                    cur.child = None
            elif not cur.next and stack:
                nxt = stack.pop()
                cur.next = nxt; nxt.prev
                = cur
            cur = cur.next
        return head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

1→2→3(child:7→8(child:11→12)→9→10)→4:

cur=3: has child. push next=4. 3.next=7, 7.prev=3. cur→7.

```
# cur=8: has child. push next=9.
8.next=11, 11.prev=8. cur→11.
# cur=12: no next, pop 9. 12.next=9,
9.prev=12. cur→9.
# cur=10: no next, pop 4. 10.next=4,
4.prev=10. cur→4. Done.
# → 1-2-3-7-8-11-12-9-10-4-5-6 ✓
#
# "No bugs. Stack exactly mirrors the
# recursion call stack."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$ . Space  $O(d)$ : stack depth =
# nesting depth.
# Iterative pre-order DFS – same pattern
# as Clone N-ary Tree but with prev-pointer
# maintenance.
# Follow-up: unflatten the list – requires
# tracking where each child section started.
# Follow-up: BFS order instead of DFS –
# use a queue instead of a stack."
```

Linked List

#622 Design Circular Queue

MED[Open on LeetCode](#)

Array · Linked List · Design · Queue

Lists: Denny Zhang

Problem

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- **MyCircularQueue(k)** Initializes the object with the size of the queue to be k.
- **int Front()** Gets the front item from the queue. If the queue is empty, return -1.
- **int Rear()** Gets the last item from the queue. If the queue is empty, return -1.
- **boolean enQueue(int value)** Inserts an element into the circular queue. Return true if the operation is successful.
- **boolean deQueue()** Deletes an element from the circular queue. Return true if the operation is successful.
- **boolean isEmpty()** Checks whether the circular queue is empty or not.
- **boolean isFull()** Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

Input

```
["MyCircularQueue", "enqueue",  
"enqueue", "enqueue", "enqueue",  
"Rear", "isFull", "deQueue", "enqueue",  
"Rear"]
```

```
[[3], [1], [2], [3], [4], [], [], [],  
[4], []]
```

Output

```
[null, true, true, true, false, 3,  
true, true, true, 4]
```

Explanation

```
MyCircularQueue myCircularQueue = new  
MyCircularQueue(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 3000 calls will be made to enqueue, dequeue, Front, Rear, isEmpty, and isFull.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."


```
# Design a circular queue (ring buffer)
with a fixed capacity.
# Support enqueue(val)→bool,
dequeue()→bool, Front()→int, Rear()→int,
isEmpty(), isFull().
# Must not use Python's built-in queue.
Right?"
#
# "A few quick questions:
# All enqueue/dequeue calls respect
isEmpty/isFull – no need to guard against
invalid calls? Yes."
#
# "Let me walk: MyCircularQueue(3):
enqueue(1,2,3)→T,T,T. enqueue(4)→F
(full).
# Rear()→3. isFull()→T. dequeue()→T.
enqueue(4)→T. Rear()→4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Use a Python list, append for enqueue
and pop(0) for dequeue.
# dequeue is O(n) due to shifting – all
other ops are O(1).
```

```
# Should I go ahead and optimize?"
class _622_DesignCircularQueue_Brute:
    def __init__(self, k): self.cap = k;
self.q = []
    def enqueue(self, val):
        if self.isFull(): return False
        self.q.append(val); return True
    def dequeue(self):
        if self.isEmpty(): return False
        self.q.pop(0); return True
    def Front(self): return -1 if
self.isEmpty() else self.q[0]
    def Rear(self): return -1 if
self.isEmpty() else self.q[-1]
    def isEmpty(self): return not self.q
    def isFull(self): return len(self.q)
== self.cap
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ `pop(0)` in `deQueue`.

Use a fixed-size array with head/tail pointers and modular arithmetic.

enqueue: `data[tail] = val; tail = (tail+1) % k; size++.`

dequeue: `head = (head+1) % k; size--.`

```
# Front: data[head]. Rear: data[(tail-1) %
k].
#
# All ops O(1). Space O(k).
# Does that make sense before I code it?"
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _622_DesignCircularQueue:
    def __init__(self, k):
        self._data = [0] * k
        self._head = self._size = 0
        self._cap = k
    def enqueue(self, val):
        if self.isFull(): return False
        self._data[(self._head +
self._size) % self._cap] = val
        self._size += 1
        return True
    def dequeue(self):
        if self.isEmpty(): return False
        self._head = (self._head + 1) %
self._cap
        self._size -= 1
        return True
```

```
def Front(self): return -1 if
self.isEmpty() else
self._data[self._head]
def Rear(self): return -1 if
self.isEmpty() else
self._data[(self._head + self._size - 1) %
self._cap]
def isEmpty(self): return self._size
== 0
def isFull(self): return self._size
== self._cap
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

k=3: enqueue(1)→size=1,data[0]=1.

enqueue(2)→data[1]=2.

enqueue(3)→data[2]=3. isFull ✓

dequeue()→head=1,size=2.

enqueue(4)→data[0]=4,size=3.

Rear()→data[(1+3-1)%3]=data[0]=4 ✓

#

"No bugs. The modular tail calculation
(head+size)%cap avoids a separate tail
variable."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(1)$. Space $O(k)$.

The modulo wraps head/tail around the fixed array – that is the 'circular' part.

Follow-up: Design Circular Deque (#641) – add enqueueFront/dequeueRear using same technique.

Follow-up: thread-safe circular buffer – add a threading.Lock around enqueue/dequeue."

Linked List

□ #707 Design Linked List

MED

[Open on LeetCode](#)

Linked List · Design

Lists: Denny Zhang

Problem

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `index`th node in the linked list. If the index is invalid, return `-1`.

- **void addAtHead(int val)** Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- **void addAtTail(int val)** Append a node of value val as the last element of the linked list.
- **void addAtIndex(int index, int val)** Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- **void deleteAtIndex(int index)** Delete the indexth node in the linked list, if the index is valid.

Example 1:

Input

```
["MyLinkedList", "addAtHead",  
"addAtTail", "addAtIndex", "get",  
"deleteAtIndex", "get"]  
[[], [1], [3], [1, 2], [1], [1], [1]]
```

Output

```
[null, null, null, null, 2, null, 3]
```

Explanation

```
MyLinkedList myLinkedList = new  
MyLinkedList();
```

Constraints:

- $0 \leq \text{index}, \text{val} \leq 1000$
- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."


```
# Design a singly linked list supporting:  
get(index), addAtHead(val),  
# addAtTail(val), addAtIndex(index, val),  
deleteAtIndex(index).  
# Indices are 0-based. addAtIndex at  
exactly size appends; beyond size is  
ignored. Right?"  
#  
# "A few quick questions:  
# get/deleteAtIndex with invalid index  
return -1 / do nothing? Yes."  
#  
# "Let me walk: addAtHead(1)→[1].  
addAtTail(3)→[1,3].  
addAtIndex(1,2)→[1,2,3].  
# get(1)→2. deleteAtIndex(1)→[1,3].  
get(1)→3 ✓"  
  
# PHASE 2 — BRUTE FORCE  
# "Let me start with brute force to lock  
in the logic.  
# Use a Python list internally.  
get/add/delete map to list operations.  
# addAtIndex O(n) due to shifting; get  
O(1).  
# Should I go ahead and optimize?"
```

```
class _707_DesignLinkedList_Brute:
    def __init__(self): self.data = []
    def get(self, i): return self.data[i]
    if 0 <= i < len(self.data) else -1
    def addAtHead(self, v):
self.data.insert(0, v)
    def addAtTail(self, v):
self.data.append(v)
    def addAtIndex(self, i, v):
    if i <= len(self.data):
self.data.insert(i, v)
    def deleteAtIndex(self, i):
    if 0 <= i < len(self.data): del
self.data[i]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ list shift on head insertions/deletions.

A true singly linked list with a sentinel dummy head and size counter:

all operations walk to the (index-1)th node and rewire from there.

No shifting needed – pointer operations only.

#

Time: $O(\text{index})$ per op – $O(n)$ worst case, $O(1)$ at head.

Space: $O(n)$ total nodes."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _707_MyLinkedList:
    class _Node:
        def __init__(self, val=0):
self.val = val; self.next = None
    def __init__(self):
        self._dummy = self._Node()
        self._size = 0
    def _get_node(self, index):
        cur = self._dummy
        for _ in range(index + 1): cur =
cur.next
        return cur
    def get(self, index):
        if index < 0 or index >=
self._size: return -1
        return self._get_node(index).val
    def addAtHead(self, val):
self.addAtIndex(0, val)
```

```
def addAtTail(self, val):
self.addAtIndex(self._size, val)
def addAtIndex(self, index, val):
    if index > self._size: return
    prev = self._dummy
    for _ in range(index): prev =
prev.next
    node = self._Node(val)
    node.next = prev.next; prev.next
= node
    self._size += 1
def deleteAtIndex(self, index):
    if index < 0 or index >=
self._size: return
    prev = self._dummy
    for _ in range(index): prev =
prev.next
    prev.next = prev.next.next
    self._size -= 1
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

addAtHead(1): dummy→1. addAtTail(3):
dummy→1→3. addAtIndex(1,2): dummy→1→2→3.

```
# get(1): walk 2 nodes → node(2).val=2 ✓.  
deleteAtIndex(1): prev=node(1), 1.next=3.  
dummy→1→3.  
# get(1): node(3).val=3 ✓.  
#  
# "No bugs. Dummy head makes  
addAtIndex(0,val) identical to any other  
index – no special case."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "get/add/delete:  $O(\text{index})$  –  $O(n)$  worst  
case. addAtHead:  $O(1)$ .  
# Space  $O(n)$  for  $n$  stored nodes.  
# Follow-up: doubly linked list – add a  
prev pointer for  $O(1)$  head and tail ops.  
# Follow-up: skip list gives  $O(\log n)$   
average for all ops – classic interview  
extension."
```

Linked List

□ ★ #708 Insert into a Sorted Circular Linked List ★

MED[Open on LeetCode](#)

Linked List

Lists: ★ Premium | Premium 98

Problem

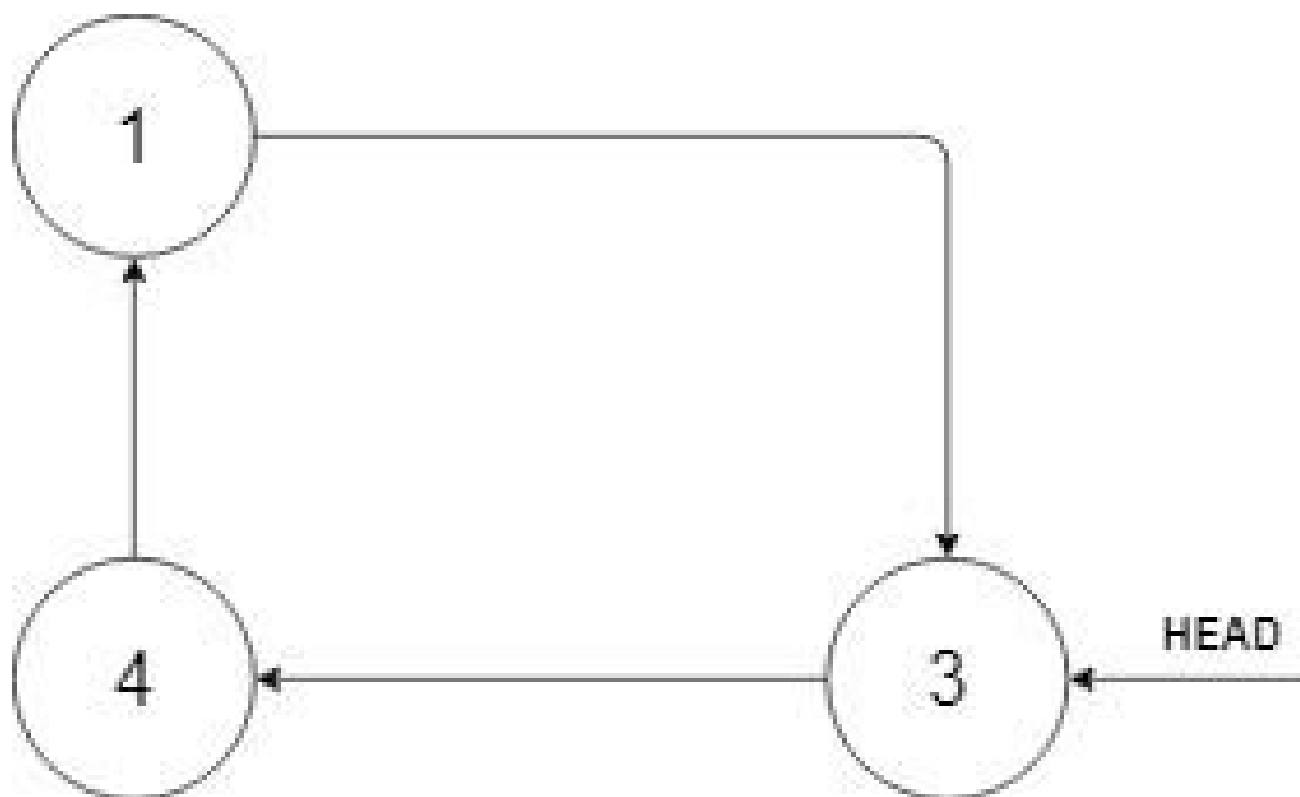
Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value `insertVal` into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node.

Otherwise, you should return the originally given node.

Example 1:



Input: head = [3,4,1], insertVal = 2

Output: [3,4,1,2]

Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.

Example 2:

Input: head = [], insertVal = 1

Output: [1]

Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.

Example 3:

Input: head = [1], insertVal = 0

Output: [1,0]

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-106 \leq \text{Node.val}, \text{insertVal} \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given a node in a sorted circular linked list, insert a value maintaining sorted order.

Return any node in the resulting list.


```
# The list may be empty (null). Right?"
#
# "A few quick questions:
# What are the insertion cases? Normal
# (prev≤val≤curr), wraparound (at max→min
# boundary),
# or after a full traversal when all
# values are equal (insert anywhere).
# Do we return the original head or any
# node? Any node – original head is fine."
#
# "Let me walk: 3→4→1→(back to 3), insert
# 2 → insert between 1 and 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
# in the logic.
# Traverse all nodes collecting them, find
# the right position, insert there.
# O(n) time, O(n) space for the collected
# list.
# Should I go ahead and optimize?"
class _708_InsertIntoCL_Brute:
    def insert(self, head, insertVal):
        new_node = Node(insertVal)
```

```

        if not head: new_node.next =
new_node; return new_node
        nodes, cur = [], head
        while True:
            nodes.append(cur); cur =
cur.next
            if cur is head: break
        nodes.sort(key=lambda n: n.val)
        for i, n in enumerate(nodes):
            if n.val <= insertVal <=
nodes[(i+1) % len(nodes)].val:
                new_node.next = n.next;
n.next = new_node; return head
            nodes[-1].next = new_node;
new_node.next = nodes[0]; return head
# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) list and
sort. We can do one clean O(n) pass.
# Three cases as we walk (prev, curr)
pairs around the circle:
# Case 1 (normal): prev.val ≤ insertVal ≤
curr.val – insert between them.
# Case 2 (wraparound): prev.val > curr.val
(we're at the max→min boundary).

```

```
# Insert here if insertVal ≥ prev.val (new max) OR insertVal ≤ curr.val (new min).  
# Case 3 (full circle, all equal): went around without inserting – insert anywhere.
```

```
#
```

```
# Time: O(n). Space: O(1).  
# Does that make sense before I code it?"
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _708_InsertIntoSortedCircularLL:  
    def insert(self, head, insertVal):  
        new_node = Node(insertVal)  
        if not head:  
            new_node.next = new_node  
            return new_node  
        prev, curr = head, head.next  
        while True:  
            normal = prev.val <=  
insertVal <= curr.val  
            wraparound = prev.val >  
curr.val and (insertVal >= prev.val or  
insertVal <= curr.val)  
            if normal or wraparound:
```

```
        break
    prev, curr = curr, curr.next
    if prev is head:
        break
    prev.next = new_node
    new_node.next = curr
    return head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

3→4→1→3, insert 2: prev=3,curr=4: $3 \leq 2 \leq 4$?

No. prev=4,curr=1: $4 > 1$ wraparound: $2 \geq 4$?

No. $2 \leq 1$? No.

prev=1,curr=3: $1 \leq 2 \leq 3$? Yes → insert
between 1 and 3. → 3→4→1→2→3 ✓

#

Insert 5 (new max): prev=4,curr=1:
wraparound, $5 \geq 4$? Yes → insert between 4
and 1. ✓

Insert 0 (new min): prev=4,curr=1:
wraparound, $0 \leq 1$? Yes → insert between 4
and 1. ✓

#

"No bugs. The wraparound branch handles
both new maximum and new minimum in one

check."

PHASE 6 — COMPLEXITY & FOLLOW-UP

**# "Time $O(n)$: at most one full traversal.
Space $O(1)$.**

All-equal list: we complete the circle without inserting → break at 'if prev is head' → insert anywhere.

Follow-up: delete from sorted circular linked list – find and unlink the node.

Follow-up: merge two sorted circular linked lists – split both at the max→min boundary, merge, relink."

Linked List

□ #1171 Remove Zero Sum Consecutive Nodes from Linked List

MED[Open on LeetCode](#)

Hash Table · Linked List

Lists: Denny Zhang

Problem

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

Input: head = [1,2,-3,3,1]

Output: [3,1]

Note: The answer [1,2,1] would also be accepted.

Example 2:

Input: head = [1,2,3,-3,4]

Output: [1,2,4]

Example 3:

Input: head = [1,2,3,-3,-2]

Output: [1]

Constraints:

- The given linked list will contain between 1 and 1000 nodes.
- Each node in the linked list has $-1000 \leq \text{node.val} \leq 1000$.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Repeatedly remove consecutive sequences of nodes that sum to 0 until none remain.

Return the resulting linked list.

Right?"

#

"A few quick questions:

Can multiple non-overlapping zero-sum sequences exist? Yes – remove all.

Can values be negative? Yes – that's what makes zero-sum possible.

Does the answer need to be unique? The problem guarantees any valid answer is accepted."

#

"Let me walk: $[1, 2, -3, 3, 1] \rightarrow [1, 2, -3]$ sums to 0 $\rightarrow [3, 1]$ ✓.

Also valid: $[2, -3, 3] = 2$ (not zero) $\rightarrow$ only $[1, 2, -3]$ works first."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeatedly scan all consecutive subsequences for zero sum, remove the first found, repeat.

$O(n^3)$ per pass times multiple passes – extremely slow.

Should I go ahead and optimize?"

```
class _1171_RemoveZeroSumNodes_Brute:
```

```
    def removeZeroSumSublists(self, head):
```

```
        dummy = ListNode(0, head)
```

```
        changed = True
```

```
        while changed:
```



```
changed = False; cur = dummy
while cur:
    total = 0; runner =
cur.next

    while runner:
        total += runner.val
        if total == 0:
cur.next = runner.next; changed = True;
break

        runner = runner.next
    if changed: break
    cur = cur.next
return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the nested scan – we recompute prefix sums from scratch each pass.

Use a prefix sum + hash map in two passes ($O(n)$):

Pass 1: walk the list computing running sum; store prefix_sum → latest node.

If the same prefix sum appears twice, nodes between those two points sum to 0.

The override ensures we store the LAST node with that prefix sum.

```
# Pass 2: walk again; each node's prefix
sum is already in the map pointing to the
node
# just past the zero-sum segment – skip
directly to it.
#
# Time: O(n). Space: O(n).
# Does that make sense before I code it?"
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _1171_RemoveZeroSumNodes:
    def removeZeroSumSublists(self,
head):
        dummy = ListNode(0, head)
        prefix_to_node = {}
        cur, total = dummy, 0
        while cur:
            total += cur.val
            prefix_to_node[total] = cur
            cur = cur.next
        cur, total = dummy, 0
        while cur:
            total += cur.val
```

```
        cur.next =  
prefix_to_node[total].next  
        cur = cur.next  
    return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

dummy→1→2→-3→3→1:

Pass1: sums: 0→dummy, 1→1, 3→2,
0→-3(override!), 3→3(override!), 4→1.

seen={0:-3node, 1:1node, 3:3node,
4:1node}.

Pass2:

total=0→dummy.next=seen[0].next=3node.

total=3→3.next=seen[3].next=1node.

total=4→1.next=seen[4].next=None. →
[3,1] ✓

#

"No bugs. Override in pass 1 ensures we
skip to the node AFTER the last zero-sum
segment."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two linear passes. Space
 $O(n)$: prefix sum map.

Same pattern as Subarray Sum Equals K (#560) applied to a linked list.
The override-on-collision in pass 1 is the key non-obvious step – worth explaining clearly.
Follow-up: count removed nodes – track length before and after.
Follow-up: if values can be floats, use epsilon comparison for the zero-sum check."

Stack

Round 5 · Mid · Medium · 14 questions

Stack

#143 Reorder List

MED[Open on LeetCode](#)

Linked List · Two Pointers · Stack · Recursion
Lists: Grind 169

Problem

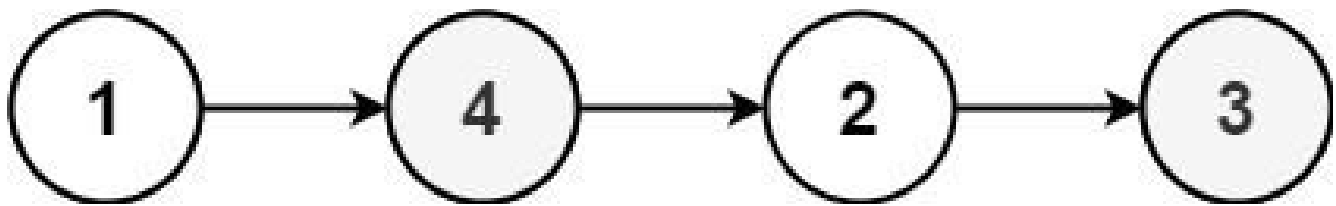
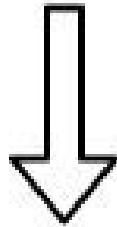
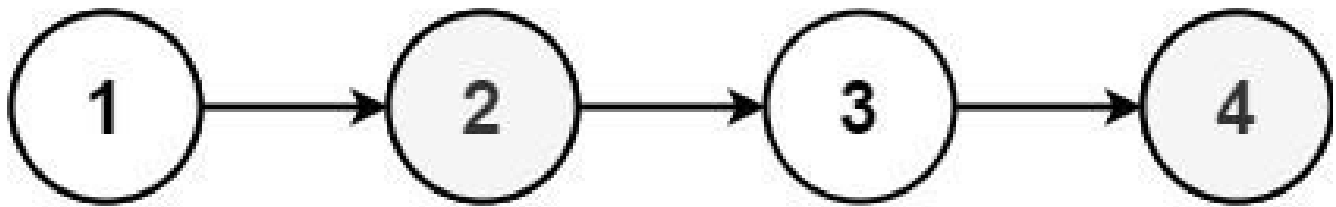
You are given the head of a singly linked-list. The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

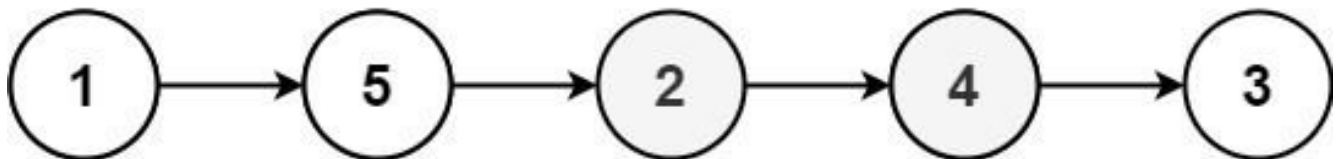
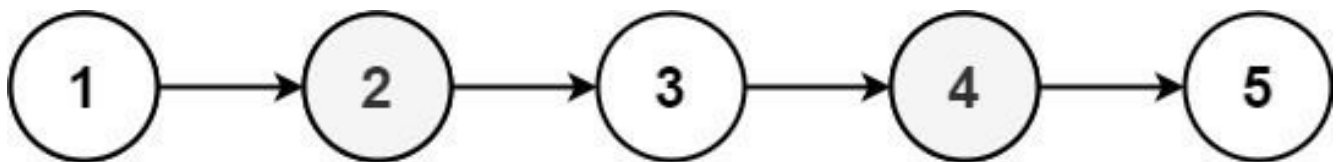
You may not modify the values in the list's nodes. Only nodes themselves may be changed.
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Reorder in-place:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

We swap the actual nodes, not just values. No return value. Right?"

#

"A few quick questions:

Can the list have 1 or 2 nodes? Yes – already in order, no-op.

$O(1)$ space required? Yes – must not copy values or create new nodes."

#

"Let me walk: $[1, 2, 3, 4] \rightarrow [1, 4, 2, 3]$ ✓.
 $[1, 2, 3, 4, 5] \rightarrow [1, 5, 2, 4, 3]$ ✓"

PHASE 2 — BRUTE FORCE


```
# "Let me start with brute force to lock
in the logic.
# Collect all nodes in an array, then use
two pointers (left, right) to interleave
them.
# O(n) time but O(n) space for the
node-reference array.
# Should I go ahead and optimize?"
class _143_ReorderList_Brute:
    def reorderList(self, head):
        nodes, cur = [], head
        while cur: nodes.append(cur); cur
= cur.next
        left, right = 0, len(nodes) - 1
        while left < right:
            nodes[left].next =
nodes[right]
            left += 1
            if left == right: break
            nodes[right].next =
nodes[left]
            right -= 1
            nodes[left].next = None

# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the  $O(n)$  auxiliary  
array of node references.  
# Three in-place steps: 1. Find the mid  
with slow/fast. 2. Reverse the second  
half.  
# 3. Merge the two halves by interleaving.  
#  
# Time:  $O(n)$ . Space:  $O(1)$ .  
# Does that make sense before I code it?"  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
approach with meaningful names."  
class _143_ReorderList:  
    def reorderList(self, head):  
        slow = fast = head  
        while fast and fast.next:  
            slow = slow.next; fast =  
fast.next.next  
            prev, cur = None, slow  
            while cur:  
                nxt = cur.next; cur.next =  
prev; prev, cur = cur, nxt  
            second = prev  
            first = head  
            while second.next:
```

```
        tmp1, tmp2 = first.next,
second.next
        first.next = second;
second.next = tmp1
        first, second = tmp1, tmp2
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4]: mid: slow=2. Reverse

[2,3,4]→[4,3,2]. second=4.

Merge: first=1,second=4: 1→4→2, then

first=2,second=3: 2→3→None. → 1→4→2→3 ✓

#

[1,2,3,4,5]: mid: slow=3. Reverse

[3,4,5]→[5,4,3].

Merge: 1→5→2→4→3 ✓

#

"No bugs. Loop stops at 'second.next' to leave the odd middle node correctly placed."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: three linear passes. Space $O(1)$: only pointer variables.

This combines three classic linked-list techniques: mid-find, reverse, merge.
Follow-up: Palindrome Linked List (#234) uses the same first two steps.
Follow-up: if values-only (not node-pointer) swap allowed, the brute approach is simpler."

Stack

□ #150 Evaluate Reverse Polish Notation

MED[Open on LeetCode](#)

Array · Math · Stack

Lists: Grind 169

Problem

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: tokens = ["2","1","+","3","*"]

Output: 9

Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: tokens = ["4","13","5","/","+"]

Output: 6

Explanation: $(4 + (13 / 5)) = 6$

Example 3:

Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]

Output: 22

Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$

$= ((10 * (6 / (12 * -11))) + 17) + 5$

$= ((10 * (6 / -132)) + 17) + 5$

$= ((10 * 0) + 17) + 5$

$= (0 + 17) + 5$

$= 17 + 5$

Constraints:

- $1 \leq \text{tokens.length} \leq 104$
- `tokens[i]` is either an operator: "+", "-", "*", or "/", or an integer in the range [-200, 200].

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate an expression in Reverse Polish Notation: operands push onto a stack, operators pop two, compute, and push result. Division truncates toward zero. Right?"

#

"A few quick questions:

Only +, -, *, /? Yes. Can intermediate results overflow 32-bit? Python handles big ints natively.

Division truncates toward zero – Python's // floors toward negative infinity, so I need `int(a/b)`."

#

"Let me walk: ['2', '1', '+', '3', '*'] → (2+1)*3 = 9 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Scan for any operator, apply it to the two preceding tokens, replace all three with the result.

$O(n^2)$ — each scan is $O(n)$ and we may need $n/2$ scans.

Should I go ahead and optimize?"

```
class _150_EvalRPN_Brute:
    def evalRPN(self, tokens):
        tokens = list(tokens)
        ops = {'+', '-', '*', '/'}
        while len(tokens) > 1:
            for i, t in
enumerate(tokens):
                if t in ops:
                    a, b =
int(tokens[i-2]), int(tokens[i-1])
                    if t=='+': r=a+b
                    elif t=='-': r=a-b
                    elif t=='*': r=a*b
                    else: r=int(a/b)
                    tokens[i-2:i+1] =
[str(r)]; break
```



```
return int(tokens[0])
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated linear scan.

A stack processes this in a single left-to-right pass:

push operands; on each operator, pop two, compute, push result.

Final stack element is the answer.

#

Time: $O(n)$. Space: $O(n)$ for the stack.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _150_EvalRPN:
```

```
    def evalRPN(self, tokens):
```

```
        stack = []
```

```
        ops = {
```

```
            '+': lambda a, b: a + b,
```

```
            '-': lambda a, b: a - b,
```

```
            '*': lambda a, b: a * b,
```

```
            '/': lambda a, b: int(a / b),
```

```
        }
```

```
        for token in tokens:
            if token in ops:
                b, a = stack.pop(),
stack.pop()
                stack.append(ops[token](a
, b))
            else:
                stack.append(int(token))
return stack[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

['2', '1', '+', '3', '*']:

push 2→[2]. push 1→[2,1]. '+' pop
1,2→push 3→[3]. push 3→[3,3]. '*' pop
3,3→push 9→[9].

return 9 ✓

#

['4', '13', '5', '/', '+']: 13/5=2

(int(13/5)=2 ✓). 4+2=6 ✓

#

"No bugs. int(a/b) truncates toward zero
– Python's a//b would floor for
negatives."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(n)$: at most $(n+1)/2$ operands on stack simultaneously.

The lambda dict keeps the operator dispatch clean and avoids four if-elif branches.

Follow-up: Basic Calculator (#224) – infix with parentheses; needs operator precedence stack.

Follow-up: validate an RPN expression – operand count must always exceed operator count."

Stack

□ #155 Min Stack

MED[Open on LeetCode](#)

Stack · Design

Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- **MinStack()** initializes the stack object.
- **void push(int val)** pushes the element val onto the stack.
- **void pop()** removes the element on the top of the stack.
- **int top()** gets the top element of the stack.
- **int getMin()** retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]  
[[], [-2], [0], [-3], [], [], [], []]
```

Output

```
[null, null, null, null, -3, null, 0, -2]
```

Explanation

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design a stack that supports push, pop, top, and getMin – all $O(1)$. Right?"

#

"A few quick questions:"

```
# getMin must return the current minimum,  
which can change as we push/pop? Yes.  
# pop and top are only called on non-empty  
stacks? Yes."
```

```
#
```

```
# "Let me walk:  
push(-2),push(0),push(-3). getMin→-3.  
pop(). top()→0. getMin()→-2 ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Regular stack for push/pop/top. For  
getMin, scan the entire stack each time.
```

```
# push/pop/top  $O(1)$ , getMin  $O(n)$ .
```

```
# Should I go ahead and optimize?"
```

```
class _155_MinStack_Brute:  
    def __init__(self): self.stack = []  
    def push(self, v):  
self.stack.append(v)  
    def pop(self): self.stack.pop()  
    def top(self): return self.stack[-1]  
    def getMin(self): return  
min(self.stack)
```

```
# PHASE 3 — OPTIMIZE
```

"The bottleneck is the $O(n)$ scan for getMin.

Maintain a parallel 'min stack' tracking the minimum at each level:

when we push val, also push min(val, min_stack.top()) to the min stack.

When we pop, pop both stacks together.

getMin always reads min_stack.top() – $O(1)$.

#

Time: $O(1)$ all ops. Space: $O(n)$ for the two stacks."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _155_MinStack:

def __init__(self):

self._stack = []

self._min_stack = []

def push(self, val):

self._stack.append(val)

current_min = val if not

self._min_stack else min(val,

self._min_stack[-1])

```
        self._min_stack.append(current_min)

    def pop(self):
        self._stack.pop()
        self._min_stack.pop()

    def top(self): return self._stack[-1]
    def getMin(self): return
self._min_stack[-1]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

push(-2): stack=[-2], min_stack=[-2].

push(0): stack=[-2,0],

min_stack=[-2,-2].

push(-3): stack=[-2,0,-3],

min_stack=[-2,-2,-3].

getMin()→-3 ✓. pop(): stack=[-2,0],

min_stack=[-2,-2].

top()→0 ✓. getMin()→-2 ✓

#

"No bugs. Each level of the min_stack records the minimum valid AT THAT DEPTH."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(1)$. Space $O(n)$: two stacks of size n .

Alternative: store (val, current_min) as a tuple in one stack – halves number of lists.

Follow-up: Max Stack (#716) – same parallel-stack idea with a max_stack.

Follow-up: if push is frequent but getMin rare, brute $O(n)$ getMin is acceptable."

Stack

□ #173 Binary Search Tree Iterator

MED[Open on LeetCode](#)

Stack · Tree · Design · Binary Search Tree · Iterator

Lists: Denny Zhang

Problem

Implement the `BSTIterator` class that represents an iterator over the in-order traversal of a binary search tree (BST):

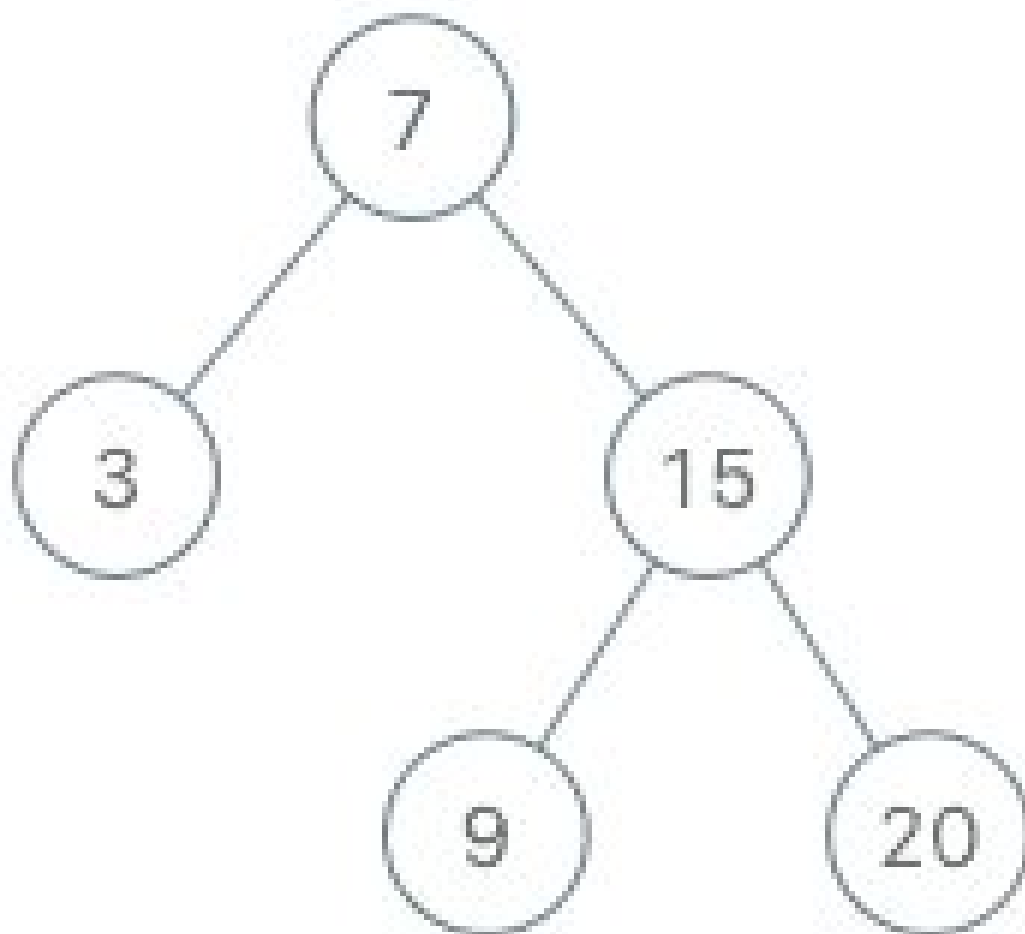
- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to

next() will return the smallest element in the BST.

You may assume that **next()** calls will always be valid. That is, there will be at least a next number in the in-order traversal when **next()** is called.

Example 1:



Input

```
["BSTIterator", "next", "next",  
"hasNext", "next", "hasNext", "next",  
"hasNext", "next", "hasNext"]  
[[[7, 3, 15, null, null, 9, 20]], [],  
[], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true,  
20, false]
```

Explanation

```
BSTIterator bSTIterator = new  
BSTIterator([7, 3, 15, null, null, 9,  
20]);
```

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 106$
- At most 105 calls will be made to hasNext, and next.

Follow up:

- Could you implement next() and hasNext() to run in average $O(1)$ time and use $O(h)$ memory, where h is the height of the tree?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Implement an iterator over a BST. next() returns the next smallest element,

hasNext() returns true if more elements remain.

Both must average $O(1)$ time and use $O(h)$ space where h is the tree height. Right?"

#

"A few quick questions:

next() is always called when an element exists? Yes – per constraints.

$O(h)$ space means we can use a stack of depth h ?"

#

"Let me walk:

BST=[7,3,15,null,null,9,20].

next()→3,7,9,15,20 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Run a full inorder traversal at
construction and store all values in a
list.
# next() returns the next element,
hasNext() checks the index.
# O(n) space regardless of tree shape –
the follow-up wants O(h).
# Should I go ahead and optimize?"
class _173_BSTIterator_Brute:
    def __init__(self, root):
        self.vals = []; self.idx = 0
        def inorder(n):
            if n: inorder(n.left);
self.vals.append(n.val); inorder(n.right)
        inorder(root)
    def next(self): v =
self.vals[self.idx]; self.idx += 1;
return v
    def hasNext(self): return self.idx <
len(self.vals)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) pre-stored
list.
# Controlled inorder traversal using an
explicit stack – O(h) space:
```

```
# At init, push the entire left spine.
# next(): pop the top node (smallest
available), push its right subtree's left
spine.
# Each node is pushed and popped exactly
once – amortized  $O(1)$  per next() call.
#
# Time:  $O(1)$  amortized. Space:  $O(h)$ ."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _173_BSTIterator:
    def __init__(self, root):
        self._stack = []
        self._push_left(root)
    def _push_left(self, node):
        while node:
            self._stack.append(node)
            node = node.left
    def next(self):
        node = self._stack.pop()
        self._push_left(node.right)
        return node.val
    def hasNext(self):
        return bool(self._stack)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

BST=[7,3,15,null,null,9,20]: init
pushes [7,3] (left spine). stack=[7,3].

next(): pop 3, push 3.right=None →
nothing. return 3. stack=[7].

next(): pop 7, push left spine of 15 →
[15,9]. return 7. stack=[15,9].

next(): pop 9, push 9.right=None. return
9. stack=[15].

hasNext()→True ✓. next(): pop 15, push
[20]. return 15. next(): pop 20. return 20

✓

#

"No bugs. _push_left handles None safely
with the 'while node' guard."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"next(): amortized $O(1)$ – each node
pushed/popped once across all calls.

hasNext(): $O(1)$. Space: $O(h)$ – left
spine depth.

Follow-up: reverse inorder iterator
(descending) – push right spines instead.

Follow-up: peek() without advancing – call next() then push the value back as a synthetic node."

Stack

□ #227 Basic Calculator II

MED[Open on LeetCode](#)

Math · String · Stack

Lists: Grind 169

Problem

Given a string s which represents an expression, evaluate this expression and return its value.

The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "3+2*2"$

Output: 7

Example 2:

Input: `s = " 3/2 "`

Output: `1`

Example 3:

Input: `s = " 3+5 / 2 "`

Output: `5`

Constraints:

- $1 \leq s.length \leq 3 \times 10^5$
- `s` consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- `s` represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is guaranteed to fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate a string with +, -, *, / (no parentheses), non-negative integers, possible spaces.

```
# Division truncates toward zero. No
built-in eval. Right?"
#
# "A few quick questions:
# Can the expression start with a negative
number? No – all integers are
non-negative.
# Spaces can appear anywhere between
tokens?"
#
# "Let me walk: '3+2*2' → 2*2=4 first,
then 3+4=7 ✓. ' 3/2 '→1 ✓. '3+5/2'→5 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Two-pass: first resolve all * and /;
then resolve + and -.
# O(n) time, O(n) space for token lists.
# Should I go ahead and optimize?"
class _227_BasicCalculatorII_Brute:
    def calculate(self, s):
        tokens = s.split()
        i = 0; toks = list(s.replace(' ',
''))
        nums = []; i = 0
```

```
while i < len(toks):
    if toks[i].isdigit():
        j = i
        while j < len(toks) and
toks[j].isdigit(): j += 1
        nums.append(int(''.join(t
oks[i:j]))); i = j
    else: nums.append(toks[i]); i
+= 1

stack = [nums[0]]; i = 1
while i < len(nums):
    op = nums[i]; num = nums[i+1];
i += 2

    if op == '*':
stack.append(stack.pop() * num)
    elif op == '/':
stack.append(int(stack.pop() / num))
    elif op == '+':
stack.append(num)
    else: stack.append(-num)
return sum(stack)
```

PHASE 3 — OPTIMIZE

**# "The bottleneck is the two-pass
tokenisation."**

```
# Single-pass stack: track the last
operator (sign).
# On + or -: push the current number with
its sign.
# On * or /: pop the top, compute
immediately with the current number, push
result.
# Final answer: sum the stack.
#
# Time: O(n). Space: O(n) for the stack."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _227_BasicCalculatorII:
    def calculate(self, s):
        stack, num, sign = [], 0, '+'
        for i, ch in enumerate(s):
            if ch.isdigit():
                num = num * 10 + int(ch)
            if (ch in '+-*/') or i ==
len(s) - 1:
                if sign == '+':
stack.append(num)
                elif sign == '-':
stack.append(-num)
```

```
        elif sign == '*':
stack.append(stack.pop() * num)
        else:
stack.append(int(stack.pop() / num))
        sign, num = ch, 0
    return sum(stack)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'3+2*2':

'3': num=3. '+': sign was '+' → push
3→[3]. sign='+', num=0.

'2': num=2. '*': sign='+' → push
2→[3,2]. sign='*', num=0.

'2'(last): num=2. i=end: sign='*' → pop
2*2=4 → [3,4]. sum=7 ✓

#

' 3/2 ': num=3. '/': push 3→[3]. num=2
(last): int(3/2)=1→[1]. sum=1 ✓

#

"No bugs. Spaces are skipped because
only digits and operators trigger action."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(n)$: stack holds at most $n/2$ numbers.

Key: $*$ and $/$ are computed immediately (higher precedence); $+$ and $-$ defer to the final sum.

Follow-up: Basic Calculator I (#224) – adds parentheses; handle with a recursion/stack.

Follow-up: Basic Calculator III (#772) – all four ops plus parentheses."

Stack

□ ★ #255 Verify Preorder Sequence in Binary Search Tree ★

MED

[Open on LeetCode](#)

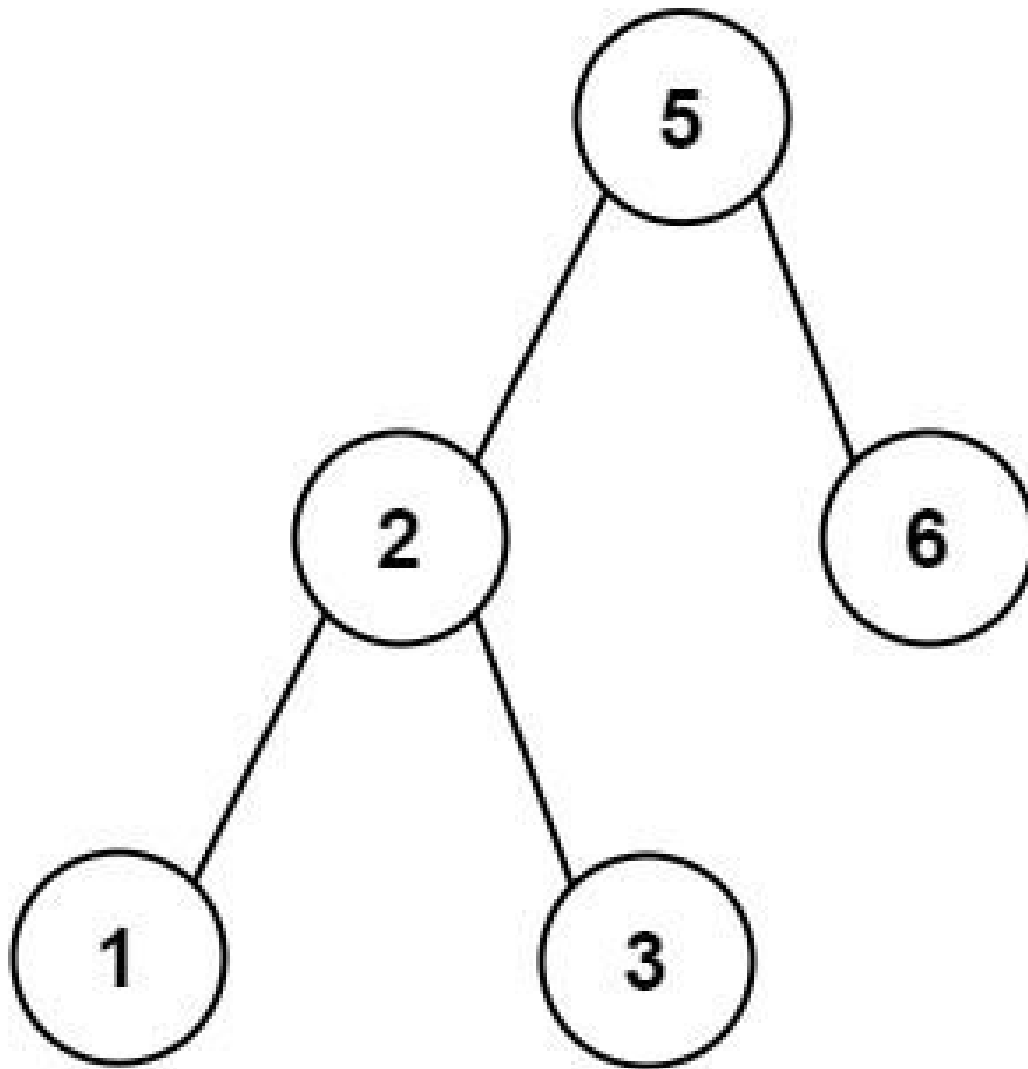
Array · Tree · BST · Stack · Monotonic Stack

Lists: ★ Premium | Premium 98

Problem

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.

Example 1:



Input: preorder = [5,2,1,3,6]
Output: true

Example 2:

Input: preorder = [5,2,6,1,3]
Output: false

Constraints:

- $1 \leq \text{preorder.length} \leq 104$
- $1 \leq \text{preorder}[i] \leq 104$

- All the elements of preorder are unique.

Follow up: Could you do it using only constant space complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given an array of unique integers, determine if it could be a valid

preorder traversal sequence of a binary search tree. Right?"

#

"A few quick questions:

Preorder: root, then left subtree (all smaller), then right subtree (all larger).

Once we enter the right subtree, all subsequent nodes must be larger than the root?"

#

"Let me walk: [5,2,1,3,6]: root=5, left=[2,1,3], right=[6] – valid ✓

[5,2,6,1,3]: after right subtree starts (6>5), seeing 1<5 is invalid → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Recursively validate: given a range [min_val, max_val], the current root must be in range.

Find where the right subtree starts (first value > root), validate both halves.

$O(n^2)$ worst case (skewed tree) – but correct.

Should I go ahead and optimize?"

```
class _255_VerifyPreorderBST_Brute:
    def verifyPreorder(self, preorder):
        def validate(lo, hi, i):
            if i >= len(preorder): return
            i

            root = preorder[i]
            if not (lo < root < hi):
                return i

            i = validate(lo, root, i + 1)
            i = validate(root, hi, i)
            return i

        return validate(float('-inf'),
                        float('inf'), 0) == len(preorder)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ recursion on skewed trees.

Monotonic stack + lower bound tracking – $O(n)$ time, $O(n)$ space:

Maintain a stack of nodes on the left-spine (descending values).

When we see a value greater than stack top, we've entered a right subtree –

pop all smaller values, the last popped becomes the new lower bound.

Any subsequent value must exceed this lower bound.

#

Time: $O(n)$. Space: $O(n)$ for the stack."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _255_VerifyPreorderBST:
```

```
    def verifyPreorder(self, preorder):
```

```
        stack = []
```

```
        lower_bound = float('-inf')
```

```
        for val in preorder:
```

```
            if val < lower_bound:
```

```
                return False
```

```
        while stack and stack[-1] <
val:
        lower_bound = stack.pop()
        stack.append(val)
    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[5,2,1,3,6]:

5: stack=[5]. 2<5: stack=[5,2]. 1<2:
stack=[5,2,1].

3>1: pop1(lb=1), pop2(lb=2). 3<5 stop.
stack=[5,3]. 3>lb=2 ✓.

6>5: pop3(lb=3), pop5(lb=5). stack=[6].
6>lb=5 ✓. → True ✓

#

[5,2,6,1,3]:

5→[5]. 2→[5,2]. 6>2:

pop2(lb=2), pop5(lb=5). stack=[6]. 6>lb=5
✓.

1<lb=5 → False ✓

#

"No bugs. The lower_bound tracks the
minimum acceptable value after entering a
right subtree."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each element pushed and popped at most once. Space $O(n)$ stack.

$O(1)$ space variant: reuse the preorder array itself as the stack (destructive).

Follow-up: reconstruct BST from preorder (#1008) – similar range-based recursion.

Follow-up: verify inorder – just check it's strictly sorted."

Stack

□ #394 Decode String

MED[Open on LeetCode](#)

String · Stack · Recursion

Lists: Grind 169

Problem

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

Input: `s = "3[a]2[bc]"`

Output: `"aaabcbcb"`

Example 2:

Input: `s = "3[a2[c]]"`

Output: `"accaccacc"`

Example 3:

Input: `s = "2[abc]3[cd]ef"`

Output: `"abcbabccdcddcdef"`

Constraints:

- $1 \leq s.length \leq 30$
- `s` consists of lowercase English letters, digits, and square brackets '['].
- `s` is guaranteed to be a valid input.
- All the integers in `s` are in the range [1, 300].

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Decode '`k[encoded_string]`' – repeat `encoded_string` `k` times.

```
# Nesting is allowed. k is always a
positive integer. Input is always valid.
Right?"
#
# "A few quick questions:
# Can k be multi-digit? Yes – e.g. '12[a]'
→ 'aaa...a'(12 times).
# Can encoded_string be empty? Yes – e.g.
'3[]' → ''."
#
# "Let me walk: '3[a]2[bc]' → 'aaabcbcb' ✓.
'3[a2[c]]' → 'accaccacc' ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Repeatedly find the innermost bracket
pair, expand it k times, replace in
string.
#  $O(n * \max\_k^{\text{depth}})$  – for deeply nested
large multipliers this is huge.
# Should I go ahead and optimize?"
class _394_DecodeString_Brute:
    def decodeString(self, s):
        import re
        while '[' in s:
```

```
        s =  
re.sub(r'(\d+)\s+([a-z]*)', lambda m:  
int(m.group(1)) * m.group(2), s)  
    return s
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated string scans."

Stack-based single pass:

On digit: build the multiplier k (can be multi-digit).

On '[': push (current_string, k) onto the stack, reset both to empty/0.

On ']': pop (prev_string, k), set current = prev_string + k * current.

On letter: append to current string.

#

Time: $O(n * \max_k)$ for building the expanded output. Space: $O(n)$ stack depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _394_DecodeString:  
    def decodeString(self, s):  
        stack = []
```

```
current = ''
k = 0
for ch in s:
    if ch.isdigit():
        k = k * 10 + int(ch)
    elif ch == '[':
        stack.append((current,
k))

        current, k = '', 0
    elif ch == ']':
        prev, repeat =
stack.pop()

        current = prev + repeat *
current

    else:
        current += ch
return current

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# '3[a2[c]]':
# '3'→k=3. '['→push('',3),current='',k=0.
#a'→current='a'.
# '2'→k=2.
 '['→push('a',2),current='',k=0.
```

```
'c'→current='c'.
# ']'→pop('a',2):
current='a'+2*'c'='acc'.
# ']'→pop('',3):
current='' + 3*'acc'='accaccacc' ✓
#
# "No bugs. Multi-digit k handled by k =
k*10 + digit before each '['."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n * \max\_k)$ : building the output
string can be large.
# Space  $O(d)$ : d = nesting depth of the
stack.
# Follow-up: Encode String with Shortest
Length (#471) — reverse direction, DP +
KMP.
# Follow-up: if nesting is guaranteed
absent, a simple split-on-bracket regex
suffices."
```

Stack

□ ★ #439 Ternary Expression Parser ★

MED[Open on LeetCode](#)

String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1:

Input: expression = "T?2:3"

Output: "2"

Explanation: If true, then result is 2; otherwise result is 3.

Example 2:

Input: expression = "F?1:T?4:5"

Output: "4"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(F ? 1 : (T ? 4 : 5))" --> "(F ? 1 : 4)" --> "4"

or "(F ? 1 : (T ? 4 : 5))" --> "(T ? 4 : 5)" --> "4"

Example 3:

Input: expression = "T?T?F:5:3"

Output: "F"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 3)" --> "F"

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 5)" --> "F"

Constraints:

- $5 \leq \text{expression.length} \leq 104$
- expression consists of digits, 'T', 'F', '?', and ':'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate a ternary expression string:
'T?a:b' → a if T, else b.


```
# Can be arbitrarily nested. All
conditions are T or F; values are digits,
T, or F.
# Ternary is right-associative. Right?"
#
# "A few quick questions:
# Input is always a valid ternary? Yes.
Only single-char values? Yes (digits 0-9,
T, F)."
#
# "Let me walk: 'T?2:3' → '2' ✓.
'F?1:T?4:5' → '4' ✓. 'T?T?F:5:3' → 'F' ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Repeatedly find the innermost 'X?a:b'
(single-char a and b), evaluate, replace.
#  $O(n^2)$  – each pass may do one
replacement.
# Should I go ahead and optimize?"
class _439_TernaryExpressionParser_Brute:
    def parseTernary(self, expression):
        import re
        s = expression
        while len(s) > 1:
```

```
        s =
re.sub(r'[TF]\?[0-9TF]:[0-9TF]', lambda
m: m.group()[2] if m.group()[0]=='T' else
m.group()[4], s)
    return s
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated string scans.

Scan RIGHT TO LEFT with a stack — $O(n)$:

When we see '?': pop two values (true-branch then false-branch) and the condition character.

Push the resolved value.

Right-to-left processes the right-associativity naturally.

#

Time: $O(n)$. Space: $O(n)$ stack."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _439_TernaryExpressionParser:
    def parseTernary(self, expression):
        stack = []
        for ch in reversed(expression):
```

```

        if stack and stack[-1] == '?':
            stack.pop()
            true_val = stack.pop()
            stack.pop()
            false_val = stack.pop()
            stack.append(true_val if
ch == 'T' else false_val)
        else:
            stack.append(ch)
    return stack[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'T?2:3' reversed='3:2?T':

push '3'. ':' → push ':'. '2' → push
 '2'. '?' → pop '?', true='2', pop ':',
 false='3'.

'T' → push '2'. stack=['2']. return '2'

✓

#

'T?T?F:5:3' (right-associ is
 T?(T?F:5):3):

Process right-to-left: resolves inner
 T?F:5 first → F, then outer T?F:3 → F ✓

#

"No bugs. Popping ':' after the false_val keeps the separator from polluting the stack."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one right-to-left pass.
Space $O(n)$ stack depth.

The right-to-left scan makes right-associativity automatic – no parenthesis tracking needed.

Follow-up: if operands can be multi-character, tokenize first then apply the same logic.

Follow-up: support general conditional expressions ($==$, $!=$) – extend the condition parsing."

Stack

★ #484 Find Permutation ★

MED[Open on LeetCode](#)

Array · String · Stack · Greedy

Lists: ★ Premium | Premium 98

Problem

A permutation perm of n integers of all the integers in the range $[1, n]$ can be represented as a string s of length $n - 1$ where:

- $s[i] == 'I'$ if $\text{perm}[i] < \text{perm}[i + 1]$, and
- $s[i] == 'D'$ if $\text{perm}[i] > \text{perm}[i + 1]$.

Given a string s , reconstruct the lexicographically smallest permutation perm and return it.

Example 1:

Input: $s = "I"$

Output: $[1,2]$

Explanation: $[1,2]$ is the only legal permutation that can be represented by s , where the number 1 and 2 construct an increasing relationship.

Example 2:

Input: $s = \text{"DI"}$

Output: $[2,1,3]$

Explanation: Both $[2,1,3]$ and $[3,1,2]$ can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return $[2,1,3]$

Constraints:

- $1 \leq s.length \leq 105$
- $s[i]$ is either 'I' or 'D'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a string of 'I' (increasing) and 'D' (decreasing) of length $n-1$,

reconstruct the lexicographically smallest permutation of $[1..n]$ satisfying the pattern. Right?"

#

"A few quick questions:

```
# 'I': nums[i] < nums[i+1]. 'D': nums[i] >
nums[i+1]. Return any lex-smallest? Yes."
#
# "Let me walk: s='I' → n=2 → [1,2] ✓.
s='DI' → [2,1,3] ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Generate all permutations of [1..n] in
lexicographic order, return first
satisfying the pattern.
# O(n! * n) – completely infeasible.
# Should I go ahead and optimize?"
class _484_FindPermutation_Brute:
    def findPermutation(self, s):
        from itertools import
permutations
        n = len(s) + 1
        for perm in permutations(range(1,
n+1)):
            if
all((perm[i]<perm[i+1])==(s[i]=='I') for
i in range(n-1)):
                return list(perm)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential permutation search.

Greedy with a stack — $O(n)$ time:

Push each number $1..n$ onto the stack. On every 'I' (or at the end), flush the stack in reverse order into the result. This reversal creates the optimal decreasing run

for any 'D' sequence while keeping the smallest numbers first.

#

Time: $O(n)$. Space: $O(n)$ for the stack."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _484_FindPermutation:
    def findPermutation(self, s):
        result = []
        stack = []
        for i, ch in enumerate(s + 'I'):
            stack.append(i + 1)
            if ch == 'I':
                while stack:
```



```
                                result.append(stack.p
op())
                                return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='DI' → s+'I'='DII':

i=0, 'D': push 1. i=1, 'I': push 2.

ch='I'→flush: result=[2,1].

i=2, 'I'(sentinel): push 3.

ch='I'→flush: result=[2,1,3]. → [2,1,3] ✓

#

s='III':

i=0: push1, 'I'→flush:[1]. i=1:

push2, 'I'→flush:[1,2]. i=2:

push3, 'I'→flush:[1,2,3].

sentinel: push4, 'I'→flush:[1,2,3,4] ✓

#

s='DDI': push1(D). push2(D).

push3(I)→flush:[3,2,1].

push4(I)→flush:[3,2,1,4] ✓

#

"No bugs. The 'I' sentinel at the end ensures the last stack segment is always flushed."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each element pushed and popped once. Space $O(n)$ stack.

The stack reversal turns consecutive 'D' blocks into the smallest valid decreasing run.

Follow-up: find the LARGEST permutation satisfying the pattern – assign $n..1$, reverse D blocks.

Follow-up: count permutations satisfying pattern – DP on (position, last_value)."

Stack

□ #735 Asteroid Collision

MED[Open on LeetCode](#)

Array · Stack · Simulation

Lists: Grind 169

Problem

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Input: asteroids = [5,10,-5]

Output: [5,10]

Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: asteroids = [8,-8]

Output: []

Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: asteroids = [10,2,-5]

Output: [10]

Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Example 4:

Input: asteroids =

[3,5,-6,2,-1,4]□□□□□□

Output: [-6,2,4]

Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Constraints:

- $2 \leq \text{asteroids.length} \leq 104$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Positive = moving right, negative = moving left.

When a right-moving and left-moving asteroid collide, the smaller explodes;

if equal, both explode. Same-direction asteroids never collide. Right?"

#

"A few quick questions:

The absolute value gives size; the sign gives direction? Yes.

Return the state of asteroids after all collisions."

#

"Let me walk: [5,10,-5] → 10 and -5 collide → 10 survives. → [5,10] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Scan for any adjacent collision (positive followed by negative), resolve, repeat.

$O(n^2)$ worst case – lots of repeated scanning.

Should I go ahead and optimize?"

class _735_AsteroidCollision_Brute:

def asteroidCollision(self, asteroids):

changed = True

while changed:

changed = False

```
        for i in range(len(asteroids)
- 1):
            if asteroids[i] > 0 >
asteroids[i+1]:
                a, b = asteroids[i],
asteroids[i+1]
                if abs(a) == abs(b):
asteroids[i:i+2] = []
                elif abs(a) > abs(b):
asteroids.pop(i+1)
                else:
asteroids.pop(i)
                changed = True; break
    return asteroids
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated scans — we process each collision multiple times.
A stack handles this in a single pass:
Push each asteroid. When the current asteroid is negative and the stack top is positive,
a collision occurs. Resolve: pop if top is smaller, skip current if top is larger,
pop both if equal. Continue until no collision or stack empty.

```
#  
# Time:  $O(n)$  – each asteroid pushed and  
# popped at most once.  
# Space:  $O(n)$  for the stack."  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
class _735_AsteroidCollision:  
    def asteroidCollision(self,  
asteroids):  
        stack = []  
        for rock in asteroids:  
            alive = True  
            while alive and rock < 0 and  
stack and stack[-1] > 0:  
                if stack[-1] < abs(rock):  
                    stack.pop()  
                elif stack[-1] ==  
abs(rock):  
                    stack.pop(); alive =  
False  
            else:  
                alive = False  
            if alive:  
                stack.append(rock)
```


return stack

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[5,10,-5]: push 5→[5]. push 10→[5,10].

-5: 10>5→alive=False. → [5,10] ✓

[8,-8]: push 8→[8]. -8:

8==8→pop,alive=False. stack=[]. → [] ✓

[10,2,-5]: push 10,2→[10,2]. -5:

2<5→pop. [10]. -5: 10>5→alive=False. →

[10] ✓

#

"No bugs. The while loop handles chain reactions naturally."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each asteroid pushed and popped at most once across the whole loop.

Space $O(n)$: stack holds surviving asteroids.

Follow-up: 2D asteroid collision (angle + velocity) – much more complex physics simulation.

Follow-up: Count Collisions on a Road (#2211) – similar directional collision

counting."

Stack

□ #739 Daily Temperatures

MED[Open on LeetCode](#)

Array · Stack · Monotonic Stack

Lists: Grind 169

Problem

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the *i*th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

Input: `temperatures =`
`[73,74,75,71,69,72,76,73]`
Output: `[1,1,4,2,1,1,0,0]`

Example 2:

Input: `temperatures = [30,40,50,60]`
Output: `[1,1,1,0]`

Example 3:

Input: temperatures = [30,60,90]
Output: [1,1,0]

Constraints:

- $1 \leq \text{temperatures.length} \leq 105$
- $30 \leq \text{temperatures}[i] \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

For each day, find the number of days until a warmer temperature.

If no warmer day exists, the answer for that day is 0. Right?"

#

"A few quick questions:

All temperatures are in [30, 100]? Yes.
Can they all be decreasing? Yes – all zeros then.

We want days to wait, not the actual temperature difference."

#

"Let me walk:

temps=[73,74,75,71,69,72,76,73].

```
# Day 0: next warmer is day 1 → 1. Day 2:  
next warmer is day 6 → 4. →  
[1,1,4,2,1,1,0,0] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# For each day i, scan forward until  
finding a warmer temperature.
```

```
#  $O(n^2)$  time,  $O(1)$  extra space.
```

```
# Should I go ahead and optimize?"
```

```
class _739_DailyTemperatures_Brute:
```

```
    def dailyTemperatures(self,  
temperatures):  
        n = len(temperatures)  
        ans = [0] * n  
        for i in range(n):  
            for j in range(i + 1, n):  
                if temperatures[j] >  
temperatures[i]:  
                    ans[i] = j - i; break  
        return ans
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the inner scan – we  
re-examine previously seen days
```

repeatedly.

```
# Monotonic decreasing stack of indices:
# For each temperature, while the stack
# top has a smaller temperature than
# current,
# that day's wait is now known:
# current_index - stack_top_index.
# Pop and record the answer. Push the
# current index.
#
# Time:  $O(n)$  – each index pushed and
# popped at most once.
# Space:  $O(n)$  for the stack."
```

PHASE 4 — CLEAN CODE

**# "Now I'll implement the optimized
approach with meaningful names."**

```
class _739_DailyTemperatures:
    def dailyTemperatures(self,
temperatures):
        ans = [0] * len(temperatures)
        stack = []
        for i, temp in
enumerate(temperatures):
            while stack and
temperatures[stack[-1]] < temp:
```

```
        j = stack.pop()
        ans[j] = i - j
    stack.append(i)
return ans
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

temps=[73,74,75,71,69,72,76,73]:

i=0(73): stack=[0]. i=1(74): 74>73 →
pop0,ans[0]=1. stack=[1].

i=2(75): pop1,ans[1]=1. stack=[2].

i=3(71): push. [2,3]. i=4(69): push.
[2,3,4].

i=5(72): 72>69 → pop4,ans[4]=1. 72>71 →
pop3,ans[3]=2. stack=[2,5].

i=6(76): pop5,ans[5]=1. pop2,ans[2]=4.
stack=[6].

i=7(73): push. stack=[6,7]. →
[1,1,4,2,1,1,0,0] ✓

#

"No bugs. Indices left on the stack at
the end have no warmer day – correctly
stay 0."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each element pushed/popped at most once. Space $O(n)$ for the stack.
Monotonic stack is the canonical pattern for 'next greater element' problems.
Follow-up: Next Greater Element I (#496) – same stack, cross-reference to another array.
Follow-up: Next Greater Element II (#503) – circular array; iterate twice ($2n$ elements)."

Stack

□ #962 Maximum Width Ramp

MED[Open on LeetCode](#)

Array · Stack · Monotonic Stack

Lists: Denny Zhang

Problem

A ramp in an integer array `nums` is a pair (i, j) for which $i < j$ and `nums[i] ≤ nums[j]`. The width of such a ramp is $j - i$.

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Example 1:

Input: `nums = [6,0,8,2,1,5]`

Output: 4

Explanation: The maximum width ramp is achieved at $(i, j) = (1, 5)$: `nums[1] = 0` and `nums[5] = 5`.

Example 2:

Input: `nums = [9,8,1,0,1,9,4,0,4,1]`

Output: 7

Explanation: The maximum width ramp is achieved at $(i, j) = (2, 9)$: `nums[2] = 1` and `nums[9] = 1`.

Constraints:

- $2 \leq \text{nums.length} \leq 5 * 10^4$
- $0 \leq \text{nums}[i] \leq 5 * 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A ramp is a pair (i, j) where $i < j$ and `nums[i] ≤ nums[j]`.

Width = $j - i$. Find the maximum width ramp. Return 0 if none. Right?"

#

"A few quick questions:

Can all elements be equal? Yes – every pair is a ramp.

Can all elements be strictly decreasing? Yes – no ramp exists, return 0."

#

```
# "Let me walk: nums=[6,0,8,2,1,5] → ramp  
(1,5): 0<=5, width=4 ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic."
```

```
# For every pair (i, j), check nums[i] <=  
nums[j] and track max j-i.
```

```
# O(n2) time, O(1) space.
```

```
# Should I go ahead and optimize?"
```

```
class _962_MaxWidthRamp_Brute:
```

```
    def maxWidthRamp(self, nums):
```

```
        best = 0
```

```
        for i in range(len(nums)):
```

```
            for j in range(i + 1,
```

```
len(nums)):
```

```
                if nums[i] <= nums[j]:
```

```
                    best = max(best, j -
```

```
i)
```

```
        return best
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the O(n2) nested  
loop."
```

```
# Two-phase monotonic stack approach:
```

```
# Phase 1 (left candidates): scan left to
right, build a decreasing stack of
indices.
# Only indices with strictly decreasing
values can be valid left boundaries.
# Phase 2 (right scan): scan right to
left; for each j, pop from the stack while
# nums[stack.top] <= nums[j] – that's a
valid ramp; record j – stack.top.
#
# Time: O(n). Space: O(n) for the stack."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _962_MaxWidthRamp:
    def maxWidthRamp(self, nums):
        n = len(nums)
        stack = []
        for i in range(n):
            if not stack or nums[i] <
nums[stack[-1]]:
                stack.append(i)
        best = 0
        for j in range(n - 1, -1, -1):
```

```
        while stack and
nums[stack[-1]] <= nums[j]:
            best = max(best, j -
stack[-1])
        stack.pop()
    return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[6,0,8,2,1,5]:

Phase1: 6→[0]. 0<6→[0,1]. 8>0 skip. 2>0
skip. 1>0 skip. 5>0 skip. stack=[0,1].

Phase2: j=5(5):

nums[1]=0<=5→best=5-1=4, pop. nums[0]=6>5
stop. stack=[0].

j=4(1): nums[0]=6>1 stop. ... j=2(8):

nums[0]=6<=8→best=max(4, 2-0)=4, pop.

stack=[].

return 4 ✓

#

"No bugs. We don't pop if best would
decrease — we want the maximum width, so
scan right-to-left."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two linear passes, each index pushed/popped at most once.

Space $O(n)$: the decreasing stack.

The phase-1 stack stores only left-side candidates where each has a smaller value than the prior,

ensuring they're the best possible left boundary for some right side.

Follow-up: if we want min width, scan from left in phase 2.

Follow-up: Largest Rectangle in Histogram (#84) uses a similar monotonic stack pattern."

Stack

★ #1214 Two Sum BSTs ★

MED[Open on LeetCode](#)

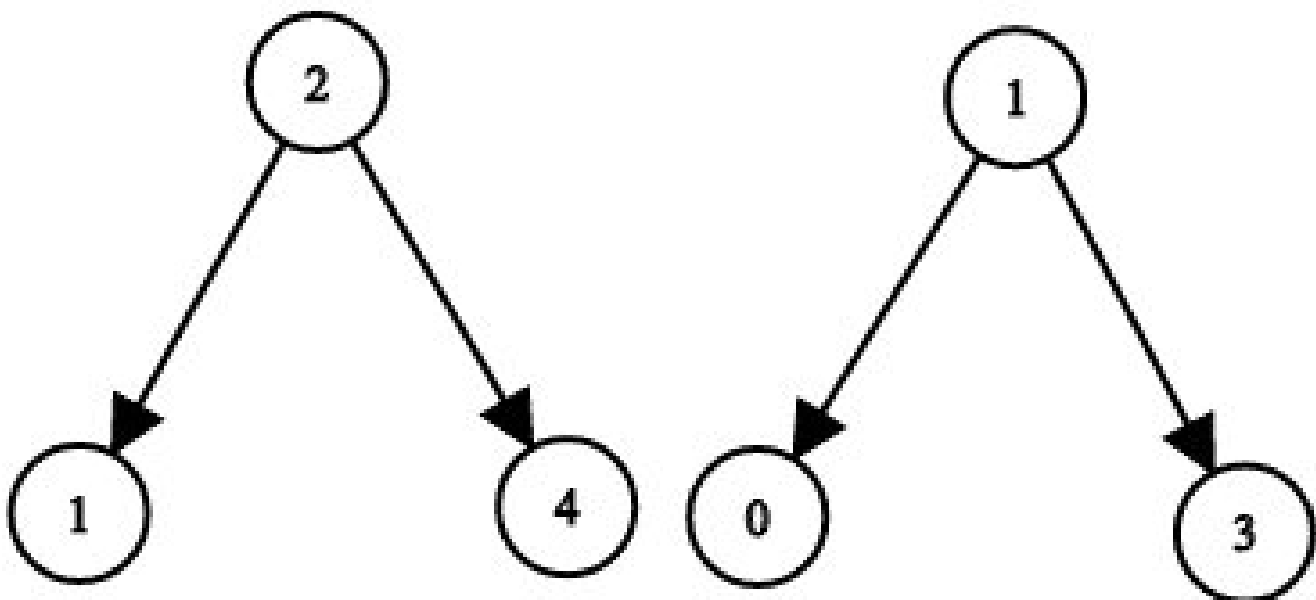
Two Pointers · Binary Search · Stack · Tree · DFS · BST · BFS

Lists: ★ Premium | Premium 98

Problem

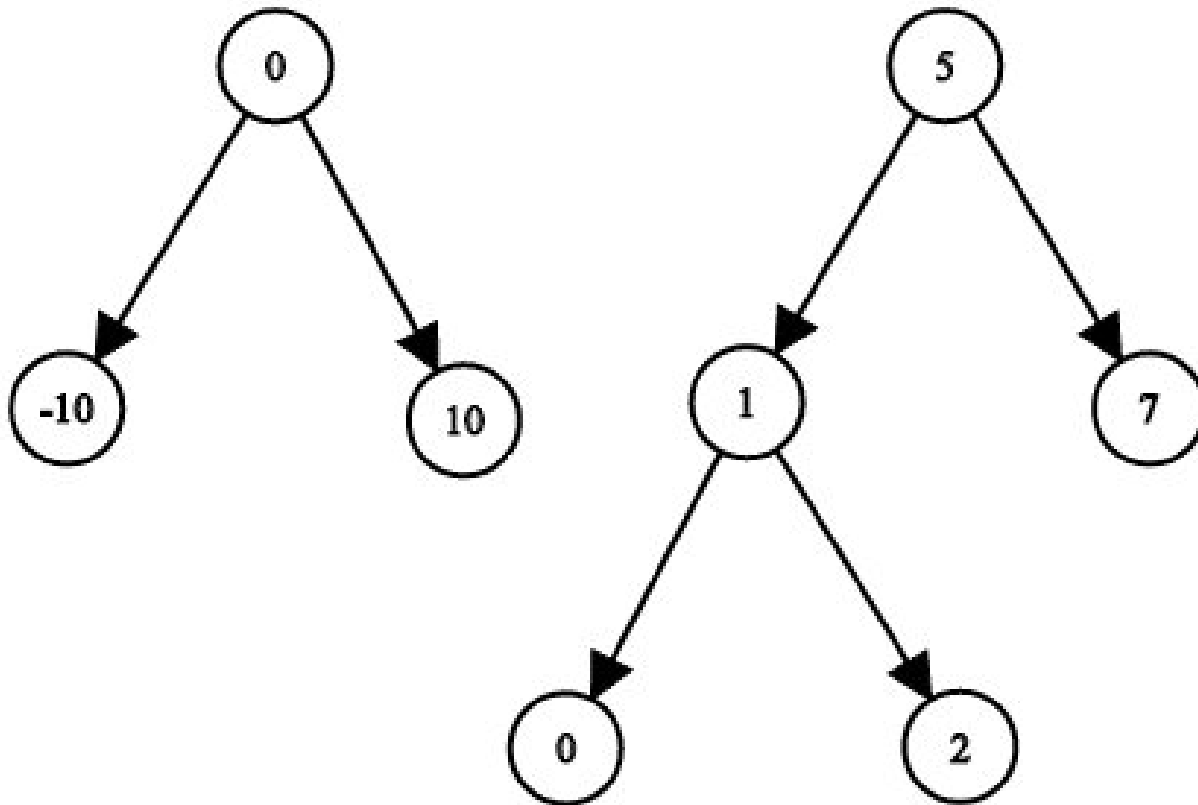
Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

Example 1:



Input: root1 = [2,1,4], root2 = [1,0,3], target = 5
Output: true
Explanation: 2 and 3 sum up to 5.

Example 2:



Input: root1 = [0,-10,10], root2 = [5,1,7,0,2], target = 18
Output: false

Constraints:

- The number of nodes in each tree is in the range [1, 5000].

- $-109 \leq \text{Node.val}, \text{target} \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given two BSTs and a target, return true if there is one value from each tree

summing to target. Right?"

#

"A few quick questions:

Trees can have different sizes? Yes.

Values can be negative? Yes."

#

"Let me walk: root1=[2,1,4],
root2=[1,0,3], target=5.

$2+3=5$ ✓ → True ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all values from tree1 into a set. DFS tree2; check if (target - val) is in the set.

$O(n+m)$ time, $O(n)$ space for the set.

```
# Should I go ahead and optimize?"
class _1214_TwoSumBSTs_Brute:
    def twoSumBSTs(self, root1, root2,
target):
        vals = set()
        def collect(node):
            if node: vals.add(node.val);
collect(node.left); collect(node.right)
        def search(node):
            if not node: return False
            return (target - node.val) in
vals or search(node.left) or
search(node.right)
        collect(root1)
        return search(root2)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ space set for tree1's values.

BST inorder traversal gives sorted arrays.

Two-iterator approach: one ascending (tree1 smallest to largest),

one descending (tree2 largest to smallest).

```
# Advance the ascending iterator if sum <
target; descending if sum > target.
# Time: O(n+m). Space: O(h1+h2) – just the
two iterator stacks."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _1214_TwoSumBSTs:
    def twoSumBSTs(self, root1, root2,
target):
        def push_left(node, stack):
            while node:
stack.append(node); node = node.left
        def push_right(node, stack):
            while node:
stack.append(node); node = node.right
        lo_stack, hi_stack = [], []
        push_left(root1, lo_stack)
        push_right(root2, hi_stack)
        while lo_stack and hi_stack:
            lo_val = lo_stack[-1].val
            hi_val = hi_stack[-1].val
            total = lo_val + hi_val
            if total == target:
                return True
```

```
        elif total < target:
            node = lo_stack.pop()
            push_left(node.right,
lo_stack)
        else:
            node = hi_stack.pop()
            push_right(node.left,
hi_stack)
    return False
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

root1=[2,1,4], root2=[1,0,3], target=5:

lo_stack=[2,1] (left-spine).

hi_stack=[1,3] (right-spine: push_right
gives root then go right? No:

push_right(root2=[1,0,3]): push 1, then
right=3, push 3, right=None. hi=[1,3].

top=3.

lo_val=1, hi_val=3: 1+3=4<5 → advance
lo: pop1, push_left(1.right=None).

lo=[2].

lo_val=2, hi_val=3: 2+3=5 → True ✓

#

"No bugs. push_left/push_right prepare the iterators for inorder ascending/descending."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n+m)$: each node visited at most once. Space $O(h_1+h_2)$ for the two stacks.

Hash-set brute force is $O(n)$ space regardless of tree shape.

Follow-up: Two Sum IV (#653) – find pair WITHIN a single BST; use BST iterator + set.

Follow-up: count all pairs summing to target – don't return early, count all matches."

Stack

□ ★ #1762 Buildings With an Ocean View ★ **MED**

[Open on LeetCode](#)

Array · Stack · Monotonic Stack

Lists: ★ Premium | Premium 98

Problem

There are n buildings in a line. You are given an integer array `heights` of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

Input: heights = [4,2,3,1]

Output: [0,2,3]

Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.

Example 2:

Input: heights = [4,3,2,1]

Output: [0,1,2,3]

Explanation: All the buildings have an ocean view.

Example 3:

Input: heights = [1,3,2,4]

Output: [3]

Explanation: Only building 3 has an ocean view.

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Ocean is to the right. A building has ocean view if ALL buildings to its right are shorter.

Return the indices of buildings with ocean view, sorted ascending. Right?"

#

"A few quick questions:

The rightmost building always has ocean view? Yes – nothing blocks it.

Heights are positive integers? Yes."

#

"Let me walk: heights=[4,2,3,1] → building0(4)>all right ✓, building1(2)<3 ✗,

building2(3)>1 ✓, building3(1) ✓ → [0,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each building i, check if all j > i have heights[j] < heights[i].

O(n²) time, O(1) space.

Should I go ahead and optimize?"


```
class _1762_BuildingsWithOceanView_Brute:
    def findBuildings(self, heights):
        n = len(heights)
        return [i for i in range(n) if
all(heights[j] < heights[i] for j in
range(i+1, n))]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ inner scan for each building.

Scan from RIGHT to LEFT, tracking the maximum height seen so far.

A building has ocean view if its height > current max_right (all buildings to its right are shorter).

Collect indices and reverse at the end.

#

Time: $O(n)$. Space: $O(1)$ extra (output not counted)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1762_BuildingsWithOceanView:
    def findBuildings(self, heights):
        max_right = 0
```

```
        result = []
        for i in range(len(heights) - 1,
-1, -1):
            if heights[i] > max_right:
                result.append(i)
                max_right = heights[i]
        return result[::-1]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

heights=[4,2,3,1]:

i=3(1): 1>0=max_right → append 3, max=1.

i=2(3): 3>1 → append 2, max=3.

i=1(2): 2<3 → skip.

i=0(4): 4>3 → append 0, max=4.

reversed: [0,2,3] ✓

#

heights=[4,3,2,1]: all decreasing →
[0,1,2,3] ✓

heights=[1,3,2,4]: only 4 at end >
everything → [3] ✓

#

"No bugs. Traversing right-to-left with
a max_right check is O(n) and exact."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single right-to-left scan.
Space $O(1)$ ignoring output.

Monotonic stack variant: scan
left-to-right, pop any building shorter
than current.

Same $O(n)$ but uses $O(n)$ stack space vs
 $O(1)$ here.

Follow-up: buildings with view to both
left AND right – need max from both sides.

Follow-up: circular arrangement –
two-pass or range max query needed."

Heap

Round 5 · Mid · Medium · 11 questions

Heap

□ #215 Kth Largest Element in an Array

MED[Open on LeetCode](#)

Array · Divide and Conquer · Sorting · Heap · Quickselect

Lists: Grind 169

Problem

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`

Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the kth largest element in an unsorted array.

Not kth distinct – duplicates count toward the position. Right?"

#

"A few quick questions:

k is always valid ($1 \leq k \leq n$)? Yes. Can elements be negative? Yes."

#

"Let me walk: $\text{nums}=[3,2,1,5,6,4]$, $k=2 \rightarrow$ sorted desc $= [6,5,4,3,2,1] \rightarrow 5 \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort the array in descending order, return element at index k-1.

$O(n \log n)$ time, $O(1)$ extra space.

```
# Should I go ahead and optimize?"
class _215_KthLargestElement_Brute:
    def findKthLargest(self, nums, k):
        nums.sort(reverse=True)
        return nums[k - 1]

# PHASE 3 — OPTIMIZE
# "The bottleneck is sorting all n
elements when we only need the top k.
# Min-heap of size k: maintain the k
largest elements seen so far.
# The heap's root (minimum of the k
largest) is the kth largest.
# Push each element; if heap grows beyond
k, pop the smallest.
#
# Time:  $O(n \log k)$ . Space:  $O(k)$ .
# Better than  $O(n \log n)$  when k is much
smaller than n."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
import heapq
class _215_KthLargestElement:
    def findKthLargest(self, nums, k):
```

```
min_heap = []
for num in nums:
    heapq.heappush(min_heap, num)
    if len(min_heap) > k:
        heapq.heappop(min_heap)
return min_heap[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[3,2,1,5,6,4], k=2:

push 3→[3]. push 2→[2,3]. push
1→[1,3,2]→pop1→[2,3].

push 5→[2,3,5]→pop2→[3,5]. push
6→[3,5,6]→pop3→[5,6].

push 4→[4,6,5]→pop4→[5,6]. heap[0]=5 ✓

#

"No bugs. After processing all elements,
the heap contains the k largest;

its min (heap[0]) is the kth largest."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Min-heap: $O(n \log k)$. Sort: $O(n \log n)$.
Heap wins when $k \ll n$.

Quickselect: $O(n)$ average, $O(n^2)$ worst –
optimal for single large- n queries.

Follow-up: Kth Smallest in Sorted Matrix (#378) – 2D extension using a heap.
Follow-up: streaming kth largest (#703) – maintain a min-heap of size k permanently."

Heap

★ #253 Meeting Rooms II ★

MED[Open on LeetCode](#)

Array · Two Pointers · Greedy · Sorting · Heap

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, return the minimum number of conference rooms required.

Example 1:

Input: intervals =
[[0,30],[5,10],[15,20]]
Output: 2

Example 2:

Input: intervals = [[7,10],[2,4]]
Output: 1

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given meeting time intervals, find the minimum number of conference rooms required.

Two meetings can share a room only if one ends before or when the other starts. Right?"

#

"A few quick questions:

Do meetings that touch (end == start) overlap? No – they can share a room.

Return the minimum rooms, not a specific assignment?"

#

"Let me walk: $[[0,30],[5,10],[15,20]] \rightarrow [0,30]$ overlaps with $[5,10] \rightarrow 2$ rooms ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each meeting, count how many other meetings overlap with it.

```
# Maximum overlap count + 1 = rooms
needed.
#  $O(n^2)$  time,  $O(1)$  space.
# Should I go ahead and optimize?"
class _253_MeetingRoomsII_Brute:
    def minMeetingRooms(self, intervals):
        best = 0
        for s1, e1 in intervals:
            count = 1
            for s2, e2 in intervals:
                if s1 != s2 and s1 < e2
and s2 < e1: count += 1
            best = max(best, count)
        return best

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n^2)$  pairwise
overlap check.
# Min-heap of end times: tracks when each
occupied room becomes free.
# Sort meetings by start time. For each
meeting:
# - If the earliest-ending room (heap top)
finishes before or at the meeting start,
reuse it.
# - Otherwise, allocate a new room.
```

```
# Heap size at the end = minimum rooms
needed.
```

```
#
```

```
# Time:  $O(n \log n)$ . Space:  $O(n)$  for the
heap."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
import heapq
```

```
class _253_MeetingRoomsII:
```

```
    def minMeetingRooms(self, intervals):
```

```
        if not intervals: return 0
```

```
        intervals.sort(key=lambda x:
```

```
x[0])
```

```
        end_times = []
```

```
        for start, end in intervals:
```

```
            if end_times and end_times[0]
```

```
<= start:
```

```
                heapq.heapreplace(end_tim
```

```
es, end)
```

```
            else:
```

```
                heapq.heappush(end_times,
```

```
end)
```

```
        return len(end_times)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

`[[0,30],[5,10],[15,20]]` sorted=same:

`[0,30]`: heap=[] → push 30 → `[30]`.

`[5,10]`: heap[0]=30 > 5 → push 10 → `[10,30]`.

`[15,20]`: heap[0]=10 ≤ 15 → replace 10 with 20 → `[20,30]`. len=2 ✓

#

`[[7,10],[2,4]]` sorted=`[[2,4],[7,10]]`:

`[2,4]`: push 4 → `[4]`. `[7,10]`: 4 ≤ 7 → replace with 10 → `[10]`. len=1 ✓

#

"No bugs. heapreplace is equivalent to pop+push but in one $O(\log n)$ operation."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: sort + n heap ops.

Space $O(n)$: heap holds at most n end times.

Alternative: split into sorted start/end events, use two pointers – same complexity, no heap.

Follow-up: Meeting Rooms I (#252) – just check if any two overlap (sort by start).

Follow-up: minimum cost scheduling – extend with room priorities or weighted meetings."

Heap

□ #347 Top K Frequent Elements

MED[Open on LeetCode](#)

Array · Hash Table · Divide and Conquer ·
Sorting · Heap · Bucket Sort · Counting ·
Quickselect

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the k most frequent elements. The answer is unique (no ties at position k).

Return in any order. Must be better than $O(n \log n)$. Right?"

#

"A few quick questions:

Can k equal the number of unique elements? Yes. Values can be negative? Yes."

#

"Let me walk: nums=[1,1,1,2,2,3], k=2 → 1(freq=3), 2(freq=2) → [1,2] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Count frequencies, sort by frequency descending, take top k.

$O(n \log n)$ – the problem asks for better.

Should I go ahead and optimize?"

```
class _347_TopKFrequentBrute:
```

```
    def topKFrequent(self, nums, k):
```

```
        from collections import Counter
```

```
        return [x for x, _ in
```

```
Counter(nums).most_common(k)]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n \log n)$ sort of all frequencies."

Bucket sort: create $n+1$ buckets indexed by frequency.

Each bucket at index f holds all numbers that appear exactly f times.

Scan buckets from index n down to 0, collect until we have k elements.

```
#  
# Time: O(n). Space: O(n).  
# Does that make sense before I code it?"  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
class _347_TopKFrequent:  
    def topKFrequent(self, nums, k):  
        from collections import Counter  
        count = Counter(nums)  
        buckets = [[] for _ in  
range(len(nums) + 1)]  
        for num, freq in count.items():  
            buckets[freq].append(num)  
        result = []  
        for freq in range(len(buckets) -  
1, 0, -1):  
            for num in buckets[freq]:  
                result.append(num)  
                if len(result) == k:  
                    return result  
        return result  
  
# PHASE 5 — TEST & DEBUG  
# "Let me trace through a few cases."
```

```
#  
# nums=[1,1,1,2,2,3], k=2:  
# count={1:3,2:2,3:1}. buckets[3]=[1],  
buckets[2]=[2], buckets[1]=[3].  
# freq=3: result=[1]. freq=2:  
result=[1,2] → len==k=2 → return [1,2] ✓  
#  
# "No bugs. Scanning from high frequency  
ensures we pick the most frequent first."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Bucket sort:  $O(n)$  time,  $O(n)$  space.  
Heap approach:  $O(n \log k)$  – worse but  
simpler.  
# Bucket sort is the optimal  $O(n)$  solution  
when frequencies  $\leq n$  (always true here).  
# Follow-up: Top K Frequent Words (#692) –  
break frequency ties alphabetically.  
# Follow-up: streaming top-k – maintain a  
min-heap of size k as elements arrive."
```

Heap

★ #505 The Maze II ★

MED

[Open on LeetCode](#)

Array · DFS · BFS · Graph · Matrix · Heap · Shortest Path

Lists: ★ Premium | Premium 98

Problem

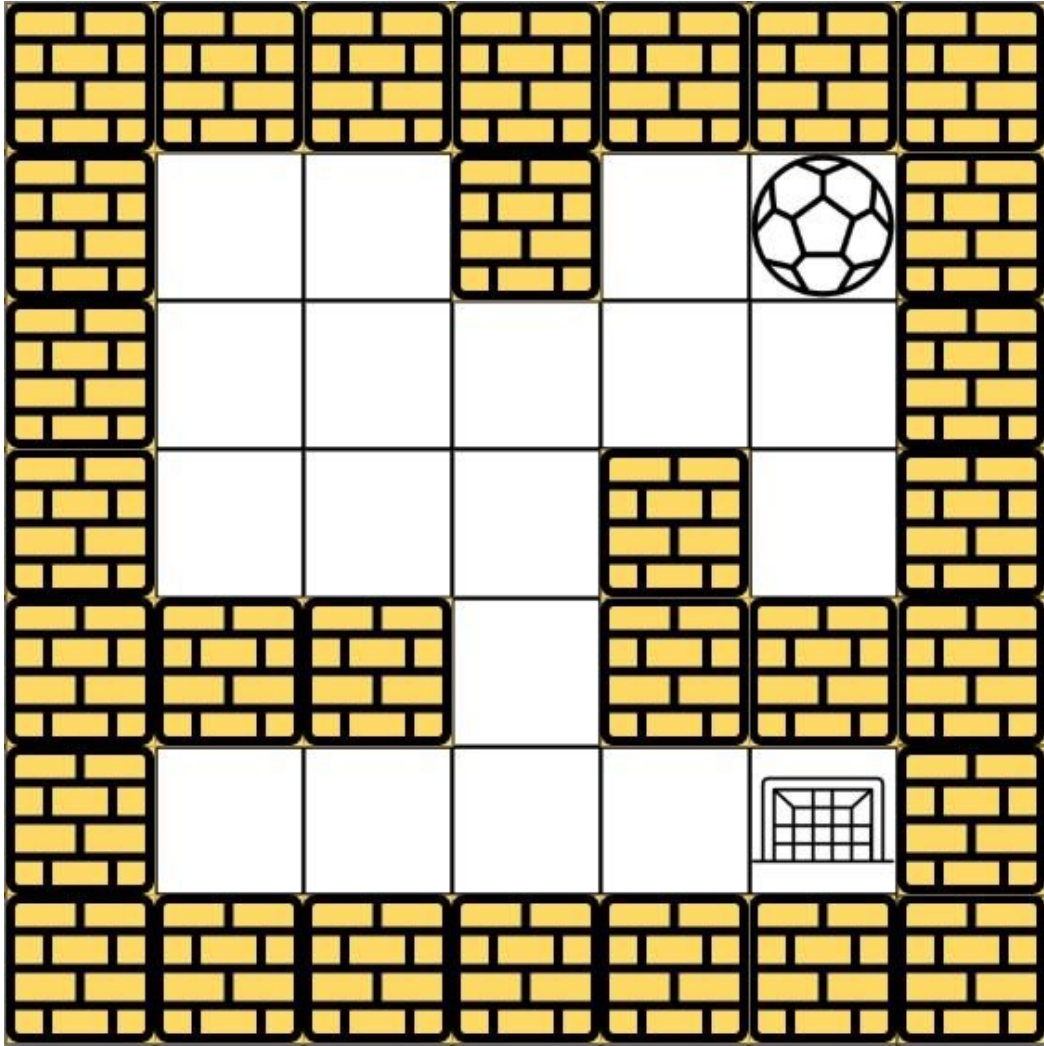
There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1 .

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



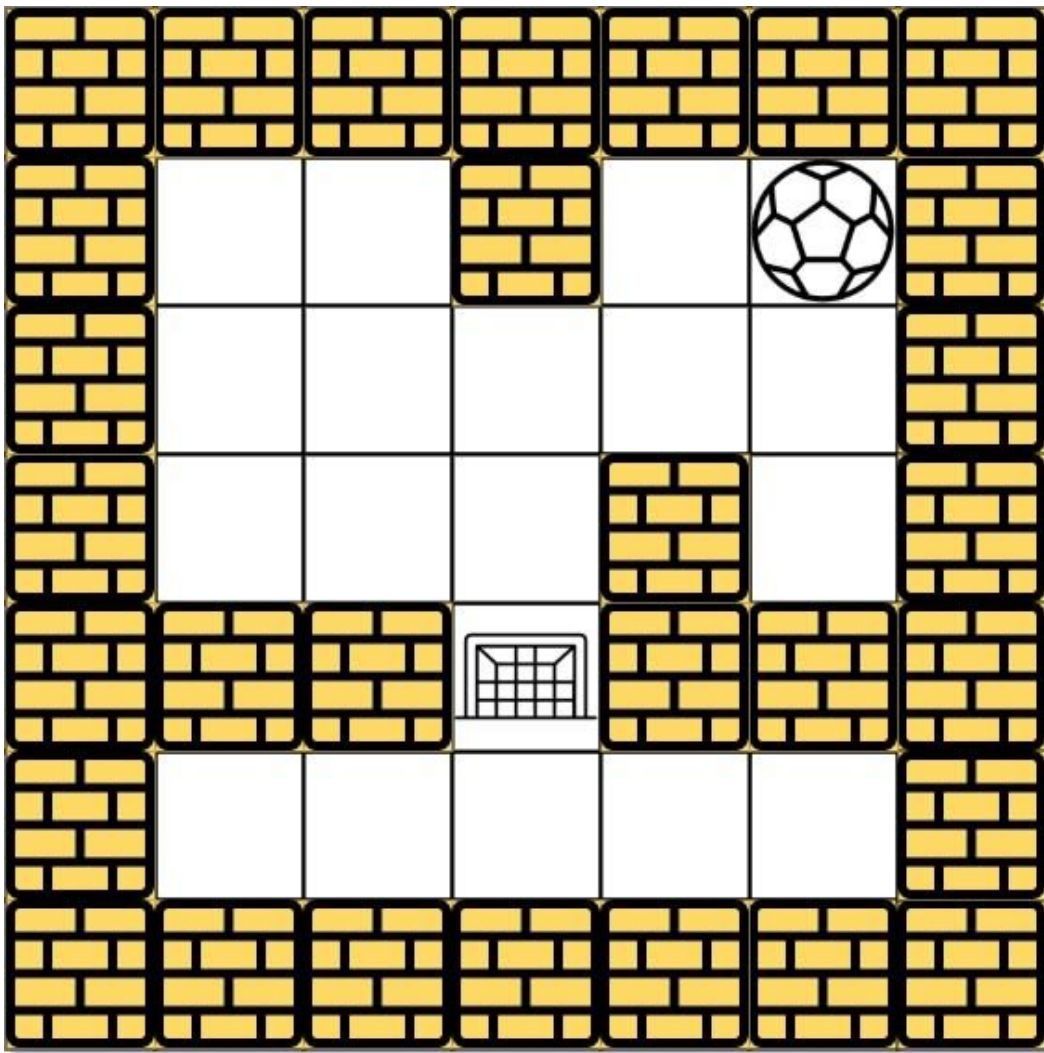
Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],
[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
start = `[0,4]`, destination = `[4,4]`

Output: 12

Explanation: One possible way is : left
-> down -> left -> down -> right ->
down -> right.

The length of the path is $1 + 1 + 3 + 1 + 2 + 2 + 2 = 12$.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],`
`[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]],`
 start = `[0,4]`, destination = `[3,2]`

Output: -1

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:


```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],
               [0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
start = [4,3], destination = [0,1]
Output: -1
```

Constraints:

- $m == \text{maze.length}$
- $n == \text{maze[i].length}$
- $1 \leq m, n \leq 100$
- maze[i][j] is 0 or 1.
- $\text{start.length} == 2$
- $\text{destination.length} == 2$
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# A ball rolls in a maze until hitting a
wall. Find the SHORTEST DISTANCE
# (number of empty spaces traveled) for
the ball to stop exactly at destination.
# Return -1 if impossible. Right?"
#
# "A few quick questions:
# Ball must STOP at destination – passing
through doesn't count? Correct.
# Distance counts steps of empty cells
traversed (destination excluded from
count? No, included per problem)."
#
# "Let me walk: maze 5x5, start=(0,4),
dest=(4,4): shortest path = 12 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# BFS exploring all rolling directions
from each stop point.
# Track distances per stop position;
expand in BFS order.
# BFS doesn't guarantee shortest distance
when edge weights vary – that's the issue.
# Should I go ahead and optimize?"
```

```
class _505_MazeII_Brute:
    def shortestDistance(self, maze,
start, destination):
        from collections import deque
        m, n = len(maze), len(maze[0])
        dist = [[float('inf')] * n for _
in range(m)]
        dist[start[0]][start[1]] = 0
        queue = deque([start])
        while queue:
            r, c = queue.popleft()
            for dr, dc in
[(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc, d = r, c, 0
                while 0<=nr+dr<m and
0<=nc+dc<n and not maze[nr+dr][nc+dc]:
                    nr += dr; nc += dc; d
+= 1
                    if dist[r][c] + d <
dist[nr][nc]:
                        dist[nr][nc] =
dist[r][c] + d; queue.append((nr,nc))
        return
dist[destination[0]][destination[1]] if
dist[destination[0]][destination[1]] !=
```

```
float('inf') else -1
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is that BFS doesn't  
account for varying rolling distances.
```

```
# Dijkstra processes stop points in order  
of distance – guarantees the first time  
# we reach a stop is via the shortest  
path.
```

```
#
```

```
# Time:  $O(m*n*\log(m*n)*(m+n))$ . Space:  
 $O(m*n)$ ."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
import heapq
```

```
class _505_MazeII:
```

```
    def shortestDistance(self, maze,  
start, destination):
```

```
        m, n = len(maze), len(maze[0])
```

```
        dist = [[float('inf')] * n for _
```

```
in range(m)]
```

```
        dist[start[0]][start[1]] = 0
```

```
        heap = [(0, start[0], start[1])]
```

```
        while heap:
```

```

        d, r, c = heapq.heappop(heap)
        if d > dist[r][c]:
            continue
        for dr, dc in
[(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc, steps = r, c, 0
            while 0<=nr+dr<m and
0<=nc+dc<n and not maze[nr+dr][nc+dc]:
                nr += dr; nc += dc;
            steps += 1
            if d + steps <
dist[nr][nc]:
                dist[nr][nc] = d +
steps
                heapq.heappush(heap,
(dist[nr][nc], nr, nc))
        result =
dist[destination[0]][destination[1]]
        return result if result !=
float('inf') else -1

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# Ball rolls to nearest wall stop;
Dijkstra picks shortest accumulated

```

distance each step.

Destination=(4,4): after exploring all paths, shortest is 12 ✓

Unreachable destination: dist stays inf
→ return -1 ✓

#

"No bugs. The ' $d > \text{dist}[r][c]$ ' check skips stale heap entries efficiently."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*(m+n)*\log(m*n))$. Space $O(m*n)$ dist + heap.

BFS only works for unit-weight edges – Dijkstra is needed here because rolling varies.

Follow-up: Maze I (#490) – just reachability, BFS suffices.

Follow-up: Maze III (#499) – same but return lex-smallest direction string on ties."

Heap

□ #621 Task Scheduler

MED

[Open on LeetCode](#)

Array · Hash Table · Greedy · Heap · Counting Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n . Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], $n = 2$

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies

to task B. In the 3rd interval, neither A nor B can be done, so you are idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A → B → C → D → A → B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A → B → idle → idle → A → B → idle → idle → A → B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 104$
- tasks[i] is an uppercase English letter.
- $0 \leq n \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

CPU tasks with a cooldown n: same task must wait at least n intervals.

CPU can idle. Return the minimum total intervals to complete all tasks. Right?"

#

"A few quick questions:

n can be 0 – meaning no cooldown, just run tasks back to back? Yes.

The specific order we choose tasks is up to us – we want to minimize total slots."

#

"Let me walk:

tasks=['A','A','A','B','B','B'], n=2.

A→B→idle→A→B→idle→A→B = 8 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Simulate with a max-heap and a cooldown queue:

each cycle pop the most frequent, place it, decrement count, push back after

```
cooldown.  
# Pad with idles if heap is empty  
mid-cycle.  
# O(total_time * log 26) time – but  
there's a math shortcut.  
# Should I go ahead and optimize?"  
class _621_TaskScheduler_Brute:  
    def leastInterval(self, tasks, n):  
        from collections import Counter  
        import heapq  
        count = Counter(tasks)  
        heap = [-c for c in  
count.values()]  
        heapq.heapify(heap)  
        time = 0  
        while heap:  
            cycle = []; slots = n + 1  
            while slots and heap:  
                cycle.append(heapq.heappo  
p(heap)); slots -= 1; time += 1  
            if heap: time += slots  
            for c in cycle:  
                if c + 1 < 0:  
heapq.heappush(heap, c + 1)  
            return time
```

PHASE 3 — OPTIMIZE

"The bottleneck is the simulation – but there's a closed-form $O(n)$ formula.

Let `max_freq` = highest task frequency, `max_count` = tasks with that frequency.

The optimal schedule slots tasks into $(\text{max_freq} - 1)$ frames of $(n+1)$ each, plus `max_count` tasks in the final partial frame.

`Result = max(len(tasks), (max_freq - 1) * (n + 1) + max_count)`.

The `max()` handles the case where tasks fill all slots without any idle.

#

Time: $O(n)$. Space: $O(1)$ – just counting."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _621_TaskScheduler:
```

```
    def leastInterval(self, tasks, n):  
        from collections import Counter  
        counts = Counter(tasks).values()  
        max_freq = max(counts)
```

```
        max_count = sum(1 for c in counts
if c == max_freq)
        return max(len(tasks), (max_freq
- 1) * (n + 1) + max_count)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# tasks=['A'*3, 'B'*3], n=2: max_freq=3,
max_count=2.
```

```
# (3-1)*(2+1)+2 = 2*3+2 = 8. len=6.
```

```
max(6,8)=8 ✓
```

#

```
# tasks=['A'*3, 'B'*3, 'C'*3], n=2:
max_freq=3, max_count=3.
```

```
# (3-1)*3+3=9. len=9. max(9,9)=9 ✓
```

```
(ABCABCABC, no idles)
```

#

```
# tasks=['A'*3, 'B'*3], n=0: (3-1)*1+2=4.
```

```
len=6. max(6,4)=6 ✓
```

#

"No bugs. The formula captures both the
idle-constrained and task-filled cases."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Formula $O(|tasks|)$: just count frequencies. Heap simulation $O(m*n)$ – much slower.

The formula works because beyond the 'frame' structure, extra tasks always fit without new idles.

Follow-up: Rearrange String k Distance Apart (#358) – return the actual string, not just count.

Follow-up: if tasks have different durations, this becomes a weighted scheduling problem."

Heap

□ #658 Find K Closest Elements

MED[Open on LeetCode](#)

Array · Binary Search · Sorting · Heap · Two Pointers

Lists: Grind 169

Problem

Given a sorted integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: `arr = [1,2,3,4,5]`, `k = 4`, `x = 3`

Output: `[1,2,3,4]`

Example 2:

Input: `arr = [1,1,2,3,4,5]`, `k = 4`, `x = -1`

Output: `[1,1,2,3]`

Constraints:

- $1 \leq k \leq \text{arr.length}$
- $1 \leq \text{arr.length} \leq 104$
- arr is sorted in ascending order.
- $-104 \leq \text{arr}[i], x \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a sorted array, find the k closest integers to x.

Ties broken by smaller value. Return the result sorted. Right?"

#

"A few quick questions:

k is always valid ($1 \leq k \leq \text{len}(\text{arr})$)?

Yes. x may not be in the array? Yes."

#

"Let me walk: arr=[1,2,3,4,5], k=4, x=3
→ [1,2,3,4] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Sort by |val - x| (then by val for
ties), take first k, re-sort.
# O(n log n) – discards the sorted
structure of the input.
# Should I go ahead and optimize?"
class _658_FindKClosestElements_Brute:
    def findClosestElements(self, arr, k,
x):
        return sorted(sorted(arr,
key=lambda v: (abs(v - x), v))[:k])

# PHASE 3 — OPTIMIZE
# "The bottleneck is sorting all n
elements when the answer is always a
contiguous window.
# Binary search on the LEFT boundary of
the k-element window:
# Search lo in [0, n-k]: the window starts
at lo.
# Compare arr[mid] and arr[mid+k]: if
arr[mid+k]-x < x-arr[mid], shift window
right.
# When lo == hi, return arr[lo:lo+k].
#
# Time: O(log(n-k) + k). Space: O(1)."
```


PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _658_FindKClosestElements:
    def findClosestElements(self, arr, k,
x):
        lo, hi = 0, len(arr) - k
        while lo < hi:
            mid = (lo + hi) // 2
            if x - arr[mid] > arr[mid + k]
- x:
                lo = mid + 1
            else:
                hi = mid
        return arr[lo:lo + k]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

arr=[1,2,3,4,5], k=4, x=3: lo=0,hi=1.

mid=0: x-arr[0]=3-1=2, arr[4]-x=5-3=2.

2>2 False → hi=0. lo=hi=0. return

[1,2,3,4] ✓

#

arr=[1,2,3,4,5], k=4, x=-1: lo=0,hi=1.

```
# mid=0: x-arr[0]=-1-1=-2 (negative, so <
arr[4]-x=6). -2>6 False → hi=0. return
[1,2,3,4] ✓
```

```
#
```

```
# "No bugs. The comparison x-arr[mid] >
arr[mid+k]-x checks which side is
farther."
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Binary search  $O(\log(n-k))$ : find left
boundary. Slice  $O(k)$ ."
```

```
# The key insight: the answer is always a
contiguous window of length k in the
sorted array.
```

```
# Follow-up: if array is unsorted, sort
first  $O(n \log n)$  or use a max-heap  $O(n \log
k)$ .
```

```
# Follow-up: same array, many queries –
binary search is  $O(\log n)$  per query, no
precompute needed."
```

Heap

□ #787 Cheapest Flights Within K Stops

MED[Open on LeetCode](#)

Dynamic Programming · DFS · BFS · Graph ·
Heap

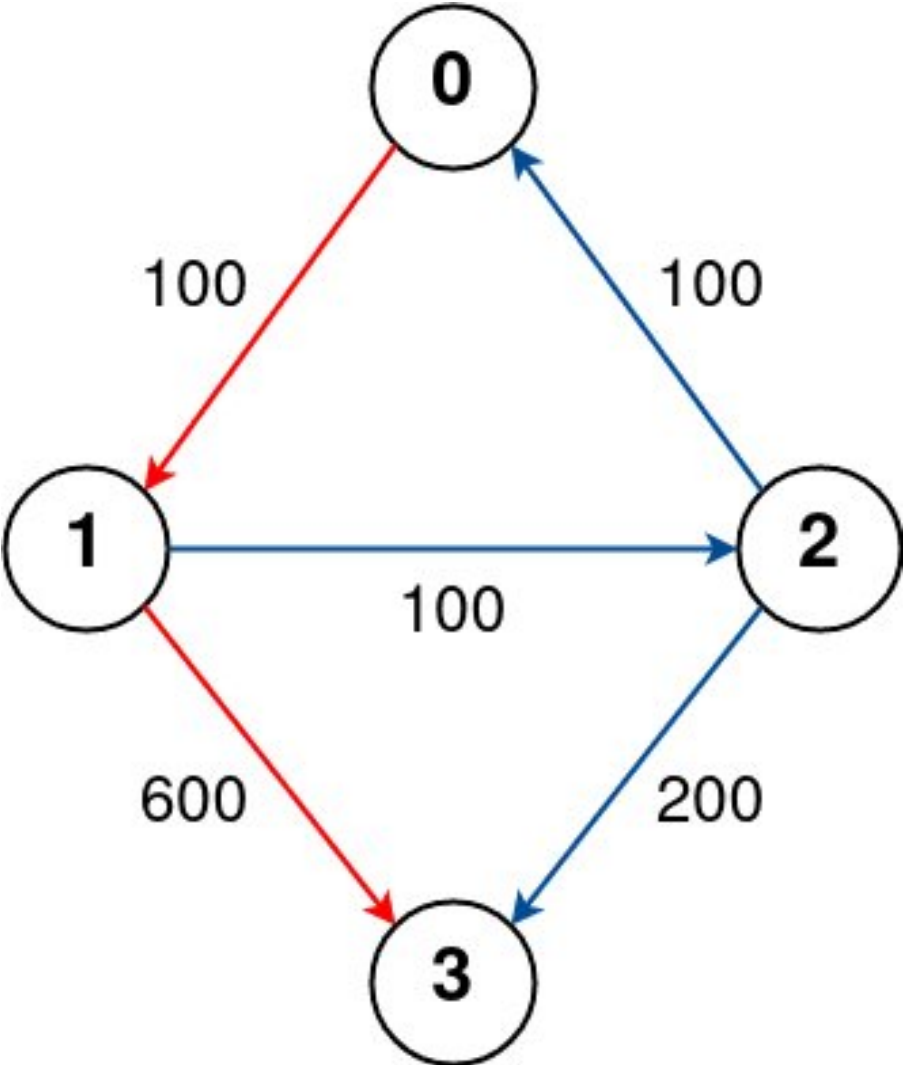
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:



Input: $n = 4$, $flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$,
 $src = 0$, $dst = 3$, $k = 1$

Output: 700

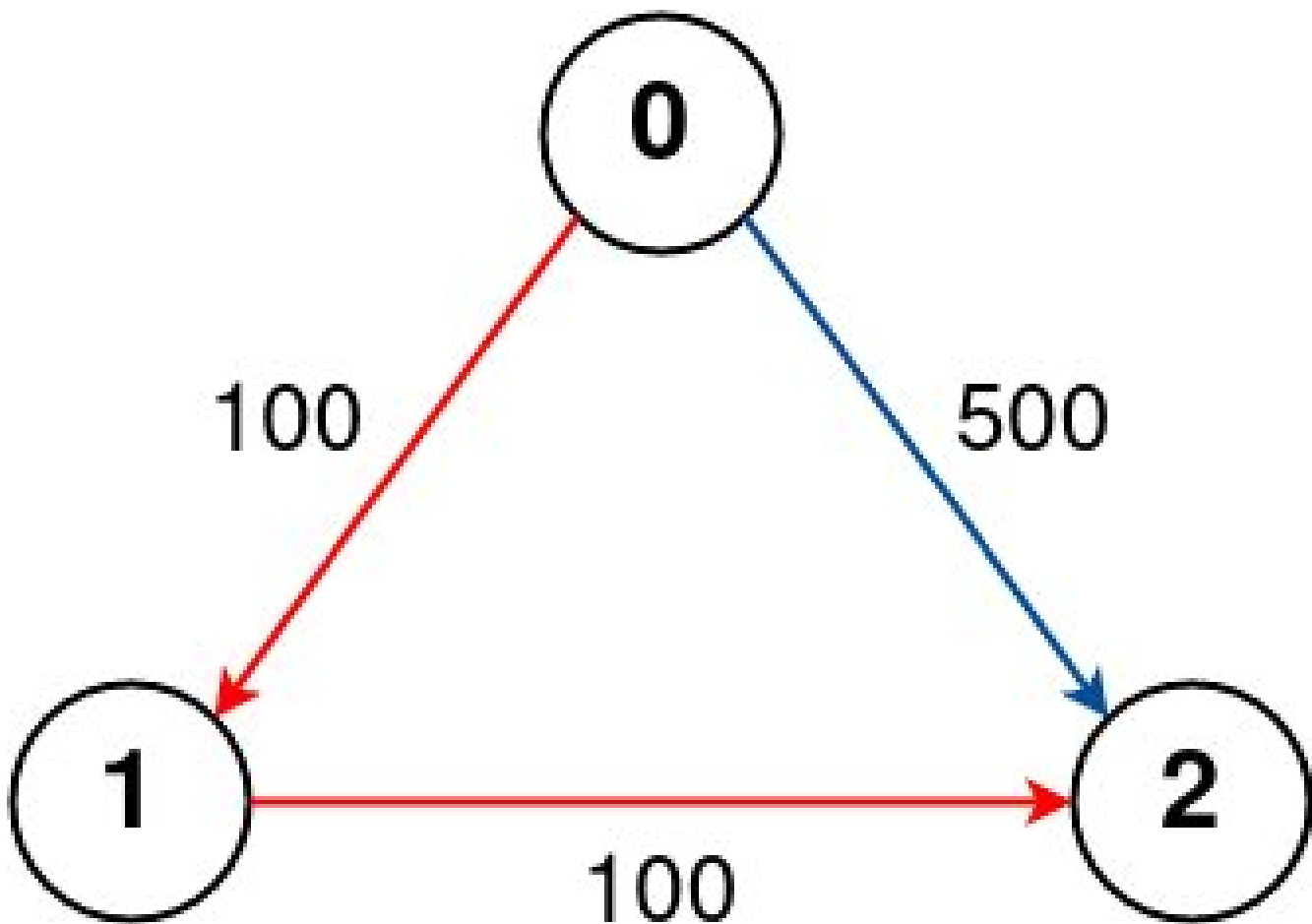
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $\text{flights} =$
[[0,1,100],[1,2,100],[0,2,500]], $\text{src} =$
0, $\text{dst} = 2$, $k = 1$

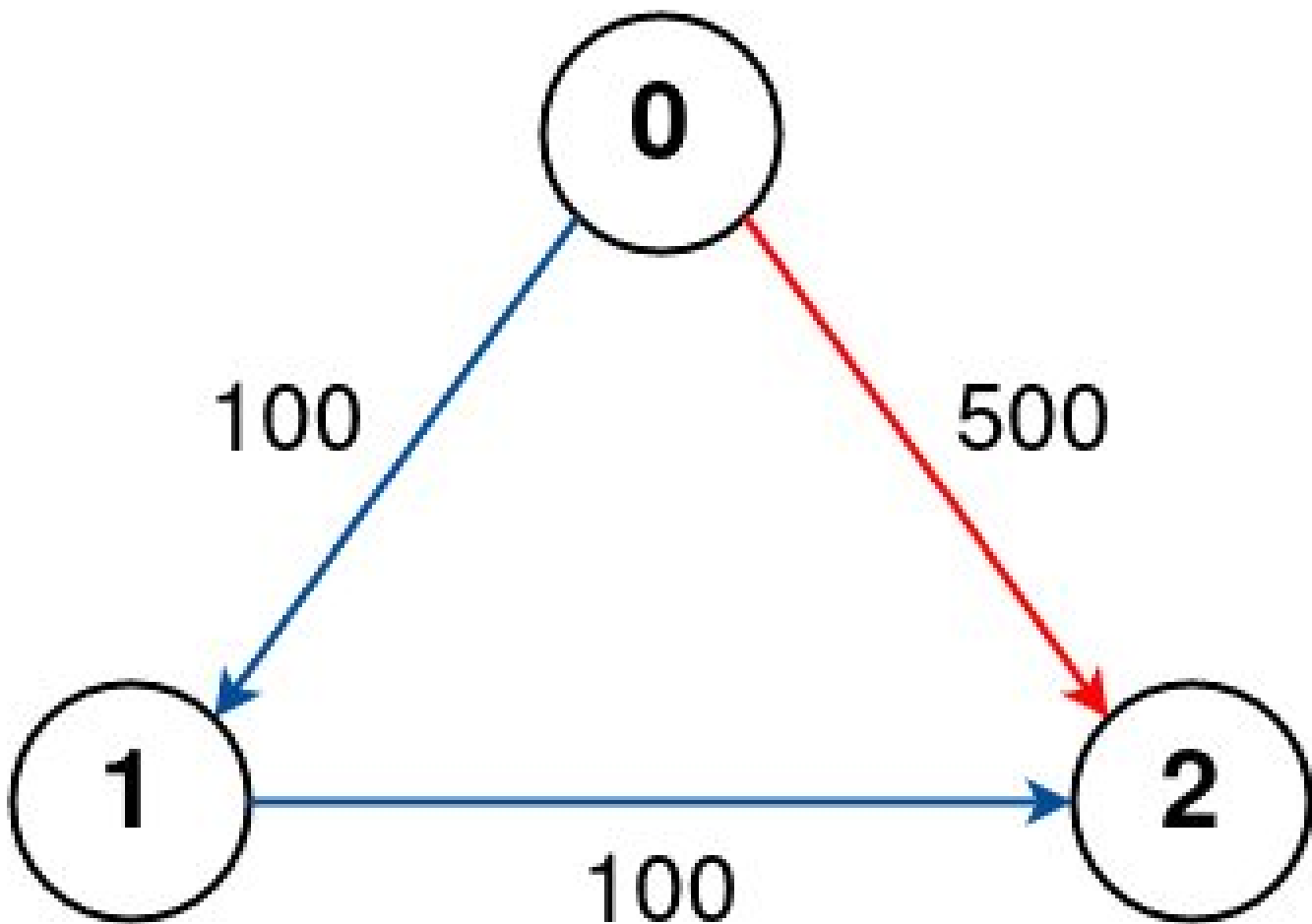
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop
from city 0 to 2 is marked in red and
has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $\text{flights} =$
[[0,1,100],[1,2,100],[0,2,500]], $\text{src} =$
0, $\text{dst} = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq \text{flights.length} \leq (n * (n - 1) / 2)$
- $\text{flights}[i].\text{length} == 3$
- $0 \leq \text{fromi}, \text{toi} < n$
- $\text{fromi} \neq \text{toi}$
- $1 \leq \text{pricei} \leq 104$
- There will not be any multiple flights between two cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

n cities, flights with prices. Find cheapest price from src to dst with at most k stops.

A stop is an intermediate city – so at most k+1 edges. Return -1 if no route. Right?"

#

"A few quick questions:"


```
# Can there be cycles? Yes. Multiple
# flights between same cities? No per
# constraints.
# k can be 0 (direct flight only)?"
#
# "Let me walk: n=4, src=0, dst=3, k=1.
# 0→1→3 costs 700 (1 stop ✓). 0→1→2→3
# costs 400 but 2 stops > k=1. → 700 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
# in the logic.
# DFS/BFS exploring all paths up to k+1
# edges. Track minimum cost to reach dst.
#  $O(n^k)$  worst case – exponential.
# Should I go ahead and optimize?"
class _787_CheapestFlightsKStops_Brute:
    def findCheapestPrice(self, n,
    flights, src, dst, k):
        from collections import
    defaultdict
    graph = defaultdict(list)
    for u, v, w in flights:
    graph[u].append((v, w))
    best = float('inf')
    queue = [(0, src, 0)]
```

```
        while queue:
            cost, node, stops =
queue.pop(0)
            if node == dst: best =
min(best, cost); continue
            if stops > k: continue
            for nxt, w in graph[node]:
                queue.append((cost + w,
nxt, stops + 1))
            return best if best < float('inf')
else -1
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential DFS. Bellman-Ford with $k+1$ relaxation rounds is $O(k \cdot E)$.

prices[v] = min cost to reach v using at most i edges after i rounds.

Each round: update prices based on PREVIOUS round's values (use a copy to avoid same-round updates).

#

Time: $O(k * |\text{flights}|)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _787_CheapestFlightsKStops:
    def findCheapestPrice(self, n,
        flights, src, dst, k):
        INF = float('inf')
        prices = [INF] * n
        prices[src] = 0
        for _ in range(k + 1):
            prev = prices[:]
            for u, v, w in flights:
                if prev[u] != INF:
                    prices[v] =
min(prices[v], prev[u] + w)
            return prices[dst] if prices[dst]
!= INF else -1
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

n=4, src=0, dst=3, k=1, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]:

init: [0,INF,INF,INF].

Round1 (k=1 edges):

0→1:100→prices[1]=100.

1→3:600→prices[3]=700. prev[1]=INF so 1→2 skipped.

prices=[0,100,INF,700]. Round2:

prev=[0,100,INF,700].

1→2:100+100=200→prices[2]=200.

1→3:100+600=700 (same). →

prices[dst=3]=700 ✓

#

"No bugs. Using prev (previous round snapshot) prevents using same-round updates."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k * E)$: $k+1$ rounds over all edges. Space $O(n)$.

Dijkstra with (cost, stops, node) heap also works – $O((E+n) \log n)$ but needs stop tracking.

Follow-up: no stop limit – standard Dijkstra or Bellman-Ford.

Follow-up: if edge weights can be negative, Bellman-Ford already handles it correctly."

Heap

□ #973 K Closest Points to Origin

MED[Open on LeetCode](#)

Array · Math · Divide and Conquer · Sorting · Heap

Lists: Grind 169

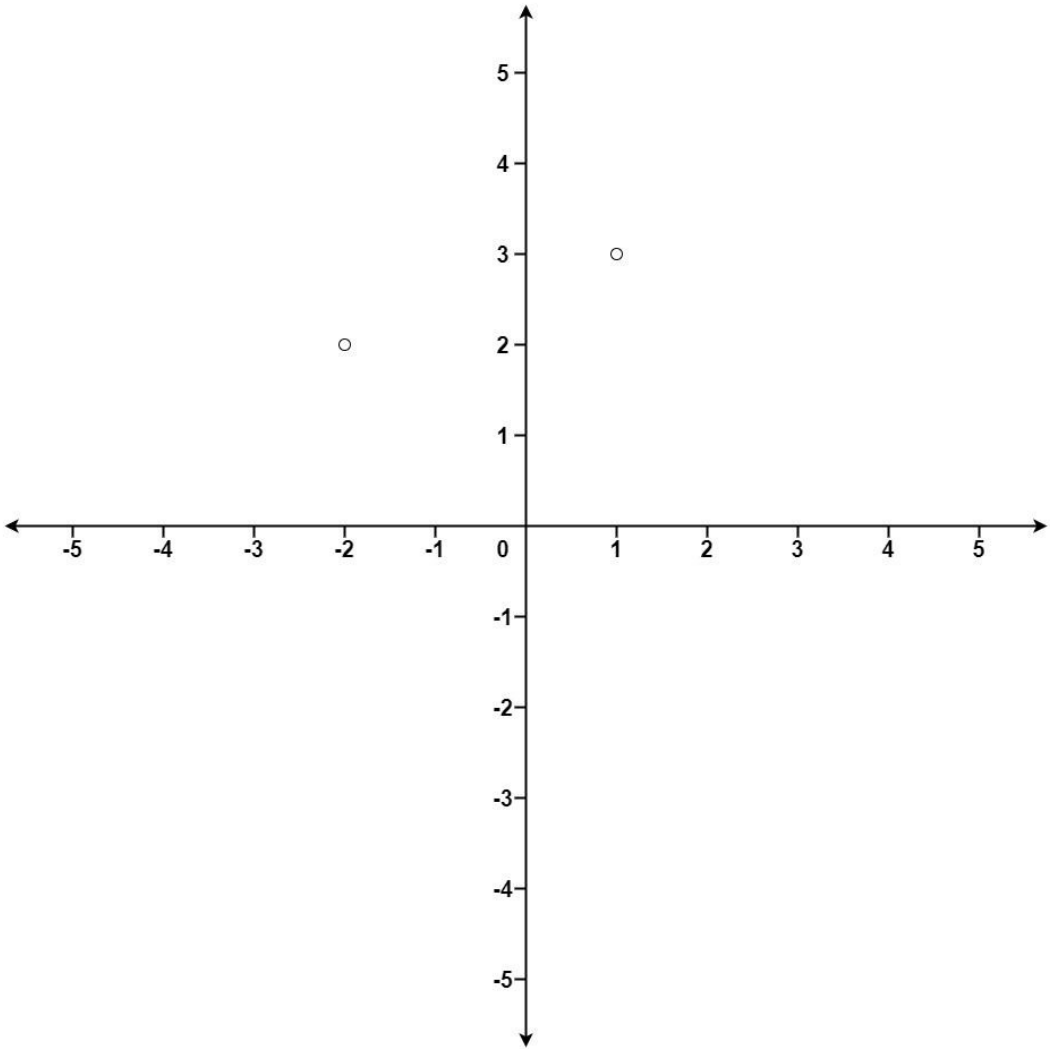
Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: `points = [[1,3],[-2,2]], k = 1`

Output: `[[-2,2]]`

Explanation:

The distance between (1, 3) and the origin is `sqrt(10)`.

The distance between (-2, 2) and the origin is `sqrt(8)`.

Since `sqrt(8) < sqrt(10)`, (-2, 2) is closer to the origin.

We only want the closest `k = 1` points from the origin, so the answer is just `[[-2,2]]`.

Example 2:

Input: `points = [[3,3],[5,-1],[-2,4]], k = 2`

Output: `[[3,3],[-2,4]]`

Explanation: The answer `[[-2,4],[3,3]]` would also be accepted.

Constraints:

- `1 <= k <= points.length <= 104`
- `-104 <= xi, yi <= 104`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the k closest points to the origin using Euclidean distance.

We can compare squared distances – no sqrt needed. Return in any order. Right?"

#

"A few quick questions:

Can k equal n (return all)? Yes. Can points be identical? Yes."

#

"Let me walk: points=[[1,3],[-2,2]], k=1. dist²=[10,8]. → [[-2,2]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort all points by squared distance, return first k.

O(n log n) time. Fine for most cases, but a heap gives O(n log k).

Should I go ahead and optimize?"

class _973_KClosestPoints_Brute:

def kClosest(self, points, k):


```
        return sorted(points, key=lambda
p: p[0]**2 + p[1]**2)[:k]
```

PHASE 3 — OPTIMIZE

"The bottleneck is sorting all n points when we only need the k closest.

Max-heap of size k : maintain k closest points seen so far.

Push $(-dist^2, x, y)$ so the heap evicts the farthest (largest $dist^2$).

When heap exceeds k , pop the farthest.

#

Time: $O(n \log k)$. Space: $O(k)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
class _973_KClosestPoints:
```

```
    def kClosest(self, points, k):
```

```
        max_heap = []
```

```
        for x, y in points:
```

```
            heapq.heappush(max_heap,
```

```
                 $-(x*x + y*y)$ , x, y))
```

```
            if len(max_heap) > k:
```

```
                heapq.heappop(max_heap)
```

```
        return [[x, y] for _, x, y in
max_heap]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# points=[[1,3],[-2,2]], k=1:
```

```
# push (-10,1,3)→[(-10,1,3)]. push
(-8,-2,2)→pop(-10,1,3)→[(-8,-2,2)].
```

```
return [[-2,2]] ✓
```

```
#
```

```
# points=[[3,3],[5,-1],[-2,4]], k=2:
```

```
dist2=[18,26,20].
```

```
# After processing: heap keeps (-20,-2,4)
and (-18,3,3). return [[3,3],[-2,4]] ✓
```

```
#
```

```
# "No bugs. Negating dist2 turns the
min-heap into a max-heap, evicting the
farthest point."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Max-heap:  $O(n \log k)$ . Sort:  $O(n \log n)$ .
Heap wins when  $k \ll n$ .
```

```
# Quickselect:  $O(n)$  average – optimal for
single large- $n$  queries.
```

Follow-up: stream of points, maintain k closest – max-heap of size k updates on each arrival.

Follow-up: k closest to an arbitrary point – subtract the target, then same algorithm."

Heap

□ ★ #1057 Campus Bikes ★

MED[Open on LeetCode](#)

Array · Greedy · Sorting · Heap

Lists: ★ Premium | Premium 98

Problem

On a campus represented on the X-Y plane, there are n workers and m bikes, with $n \leq m$.

You are given an array `workers` of length n where `workers[i] = [xi, yi]` is the position of the i th worker. You are also given an array `bikes` of length m where `bikes[j] = [xj, yj]` is the position of the j th bike. All the given positions are unique.

Assign a bike to each worker. Among the available bikes and workers, we choose the $(worker_i, bike_j)$ pair with the shortest Manhattan distance between each other and assign the bike to that worker.

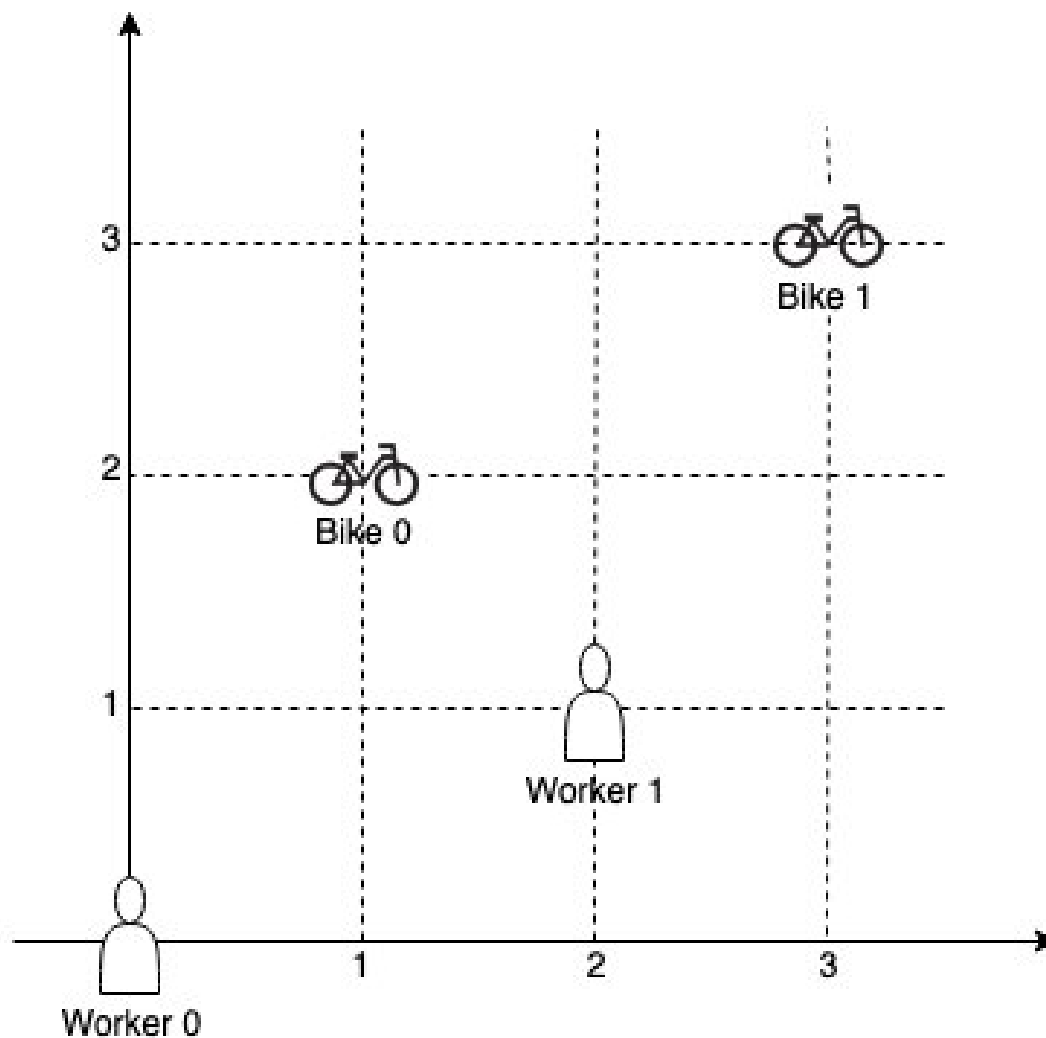
If there are multiple $(worker_i, bike_j)$ pairs with the same shortest Manhattan distance, we choose the pair with the smallest worker index.

If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.

Return an array `answer` of length `n`, where `answer[i]` is the index (0-indexed) of the bike that the `i`th worker is assigned to.

The Manhattan distance between two points `p1` and `p2` is $\text{Manhattan}(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|$.

Example 1:

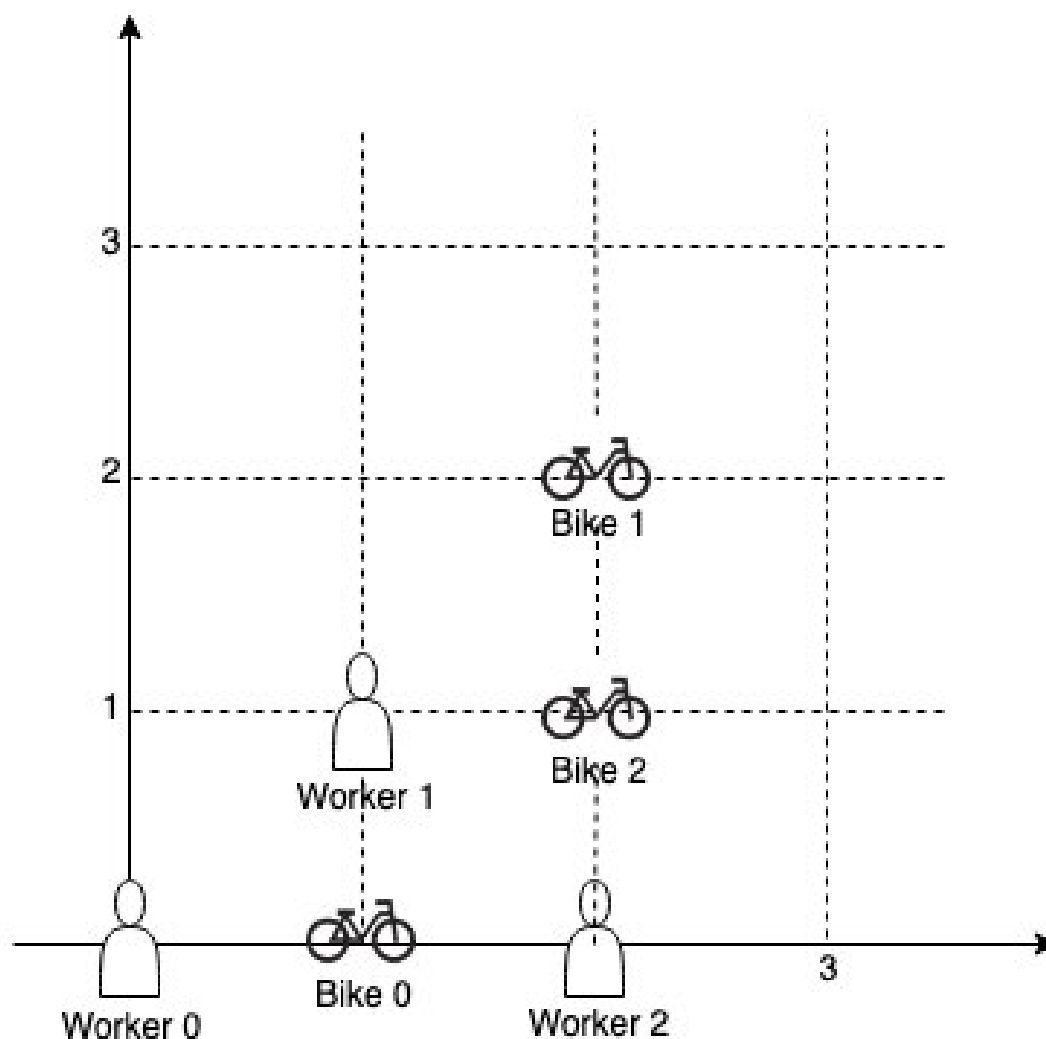


Input: `workers = [[0,0],[2,1]]`, `bikes = [[1,2],[3,3]]`

Output: `[1,0]`

Explanation: Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is `[1, 0]`.

Example 2:



Input: `workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]`

Output: `[0,2,1]`

Explanation: Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is `[0,2,1]`.

Constraints:

- $n == \text{workers.length}$
- $m == \text{bikes.length}$
- $1 \leq n \leq m \leq 1000$
- $\text{workers}[i].\text{length} == \text{bikes}[j].\text{length} == 2$
- $0 \leq x_i, y_i < 1000$
- $0 \leq x_j, y_j < 1000$
- All worker and bike locations are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

n workers and m bikes on a campus.

Greedy assign the closest worker-bike pair.

Ties: smaller worker index first; still tied: smaller bike index.

Repeat until each worker has exactly one bike. Right?"

#

"A few quick questions:

$n \leq m$ — enough bikes for all workers?

Yes. Manhattan distance? Yes."


```
#  
# "Let me walk: workers=[[0,0],[2,1]],  
bikes=[[1,2],[3,3]].  
# w0b0=3, w0b1=6, w1b0=2, w1b1=2 → min=2  
at (w1,b0) → assign w1→b0.  
# Next min from remaining: w0b1=6 → assign  
w0→b1. → [1,0] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic.  
# Generate all (dist, worker_idx,  
bike_idx) triples, sort, greedily assign.  
#  $O(n*m \log(n*m))$  time for sort. Space  
 $O(n*m)$ .  
# Should I go ahead and optimize?"  
class _1057_CampusBikes_Brute:  
    def assignBikes(self, workers,  
bikes):  
        trips = []  
        for i, (wx, wy) in  
enumerate(workers):  
            for j, (bx, by) in  
enumerate(bikes):  
                trips.append((abs(wx-bx)+  
abs(wy-by), i, j))
```

```
trips.sort()
ans = [-1]*len(workers);
used_bikes = set(); used_workers = set()
for d, w, b in trips:
    if w not in used_workers and b
not in used_bikes:
        ans[w] = b;
used_workers.add(w); used_bikes.add(b)
return ans
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n*m \log(n*m))$ 
sort – bucket sort on bounded Manhattan
distance ( $\leq 2000$ ) reduces it to  $O(n*m)$ .
# Bucket sort by distance (max Manhattan  $\leq$ 
2000) brings it to  $O(n*m)$  time:
# Distribute all triples into distance
buckets, scan from 0 upward.
#
# Time:  $O(n*m)$ . Space:  $O(n*m)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
class _1057_CampusBikes:
```

```
def assignBikes(self, workers,
bikes):
    max_dist = 2001
    buckets = [[] for _ in
range(max_dist)]
    for i, (wx, wy) in
enumerate(workers):
        for j, (bx, by) in
enumerate(bikes):
            d = abs(wx - bx) + abs(wy
- by)
            buckets[d].append((i, j))
    ans = [-1] * len(workers)
    used_bikes = set()
    assigned = 0
    for d in range(max_dist):
        for w, b in buckets[d]:
            if ans[w] == -1 and b not
in used_bikes:
                ans[w] = b;
used_bikes.add(b); assigned += 1
            if assigned ==
len(workers): return ans
    return ans
```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# workers=[[0,0],[2,1]],
# bikes=[[1,2],[3,3]]:
# buckets[3]=[(0,0)], buckets[6]=[(0,1)],
# buckets[2]=[(1,0),(1,1)].
# d=2: (1,0)→ans[1]=0, used={0}.
# (1,1)→b=1 not used: ans[1] already set,
# skip.
# d=3: (0,0)→ans[0]=0? b=0 in used → skip.
# d=6: (0,1)→ans[0]=1 ✓. → [1,0] ✓
#
# "No bugs. Bucket sort naturally
# satisfies (dist asc, worker asc, bike asc)
# ordering."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Bucket sort:  $O(n*m)$  time,  $O(n*m)$  space
# (2001 buckets). Heap:  $O(n*m \log(n*m))$ .
# Bucket sort wins because Manhattan
# distance  $\leq 2000$  is bounded.
# Follow-up: Campus Bikes II (#1066) —
# minimize TOTAL distance. Bitmask DP
#  $O(n*2^m)$ .
# Follow-up: if bikes can be reassigned
# dynamically, use a priority queue per

```

worker."

Heap

□ ★ #1167 Minimum Cost to Connect Sticks ★

MED[Open on LeetCode](#)

Array · Greedy · Heap

Lists: ★ Premium | Premium 98

Problem

You have some number of sticks with positive integer lengths. These lengths are given as an array `sticks`, where `sticks[i]` is the length of the *i*th stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of $x + y$. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

Input: sticks = [2,4,3]

Output: 14

Explanation: You start with sticks = [2,4,3].

1. Combine sticks 2 and 3 for a cost of $2 + 3 = 5$. Now you have sticks = [5,4].

2. Combine sticks 5 and 4 for a cost of $5 + 4 = 9$. Now you have sticks = [9].

There is only one stick left, so you are done. The total cost is $5 + 9 = 14$.

Example 2:

Input: sticks = [1,8,3,5]

Output: 30

Explanation: You start with sticks = [1,8,3,5].

1. Combine sticks 1 and 3 for a cost of $1 + 3 = 4$. Now you have sticks = [4,8,5].

2. Combine sticks 4 and 5 for a cost of $4 + 5 = 9$. Now you have sticks = [9,8].

3. Combine sticks 9 and 8 for a cost of $9 + 8 = 17$. Now you have sticks = [17].

There is only one stick left, so you are done. The total cost is $4 + 9 + 17 = 30$.

Example 3:

Input: sticks = [5]

Output: 0

Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.

Constraints:

- $1 \leq \text{sticks.length} \leq 104$
- $1 \leq \text{sticks}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Combine any two sticks into one; cost = sum of their lengths.

Repeat until one stick remains. Minimize total cost. Right?"

#

"A few quick questions:

Single stick → cost 0? Yes – no combining needed. All lengths positive? Yes."

#

"Let me walk: sticks=[2,4,3].

Combine 2+3=5 (cost 5). Then 4+5=9 (cost 9). Total=14 ✓

vs 2+4=6 (cost 6). 3+6=9 (cost 9). Total=15. The greedy wins."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all pair orderings recursively; track minimum total cost.

```
# 0(n! * n) – completely infeasible.
# Should I go ahead and optimize?"
class _1167_MinCostConnectSticks_Brute:
    def connectSticks(self, sticks):
        from itertools import
permutations
        best = float('inf')
        def dfs(s, cost):
            nonlocal best
            if len(s) == 1: best =
min(best, cost); return
            for i in range(len(s)):
                for j in range(i+1,
len(s)):
                    merged = s[i] + s[j];
new_s = [s[k] for k in range(len(s)) if
k!=i and k!=j] + [merged]
                    dfs(new_s, cost +
merged)
            dfs(list(sticks), 0); return best

# PHASE 3 — OPTIMIZE
# "The bottleneck is the exponential
search – but the greedy is optimal.
# Always combine the two SMALLEST sticks
(Huffman coding / priority queue):
```

```
# small sticks are paid for many times as
they get merged into bigger ones.
# Using a min-heap: pop two smallest, push
their sum, add sum to total cost.
#
# Time:  $O(n \log n)$ . Space:  $O(n)$  for the
heap."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
import heapq
class _1167_MinCostConnectSticks:
    def connectSticks(self, sticks):
        heapq.heapify(sticks)
        total_cost = 0
        while len(sticks) > 1:
            first = heapq.heappop(sticks)
            second =
heapq.heappop(sticks)
            combined = first + second
            total_cost += combined
            heapq.heappush(sticks,
combined)
        return total_cost
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

sticks=[2,4,3] → heapified=[2,3,4]:

pop 2,3 → combined=5, cost=5.

heap=[4,5].

pop 4,5 → combined=9, cost=14. heap=[9].

→ 14 ✓

#

sticks=[1,8,3,5] → heap=[1,3,5,8]:

pop 1,3→4, cost=4. heap=[4,5,8].

pop 4,5→9, cost=13. heap=[8,9].

pop 8,9→17, cost=30. → 30 ✓

#

"No bugs. heapify is $O(n)$ – more efficient than n individual pushes."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: heapify $O(n)$ + $n-1$ pops/pushes $O(\log n)$ each.

Space $O(n)$: the heap.

This is Huffman encoding – greedy optimality proven by exchange argument.

Follow-up: Minimum Cost to Merge Stones (#1000) – merge k adjacent groups; needs DP.

Follow-up: if k sticks must be merged at once, each merge costs sum of k sticks."

Heap

□ #1424 Diagonal Traverse II

MED[Open on LeetCode](#)

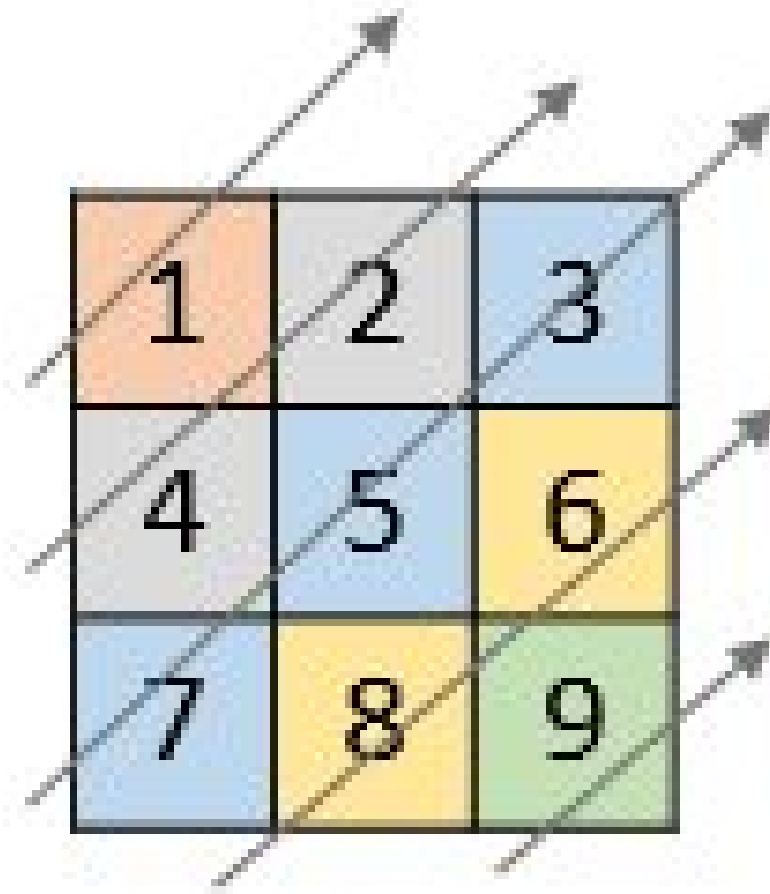
Array · Sorting · Heap

Lists: CodeSignal

Problem

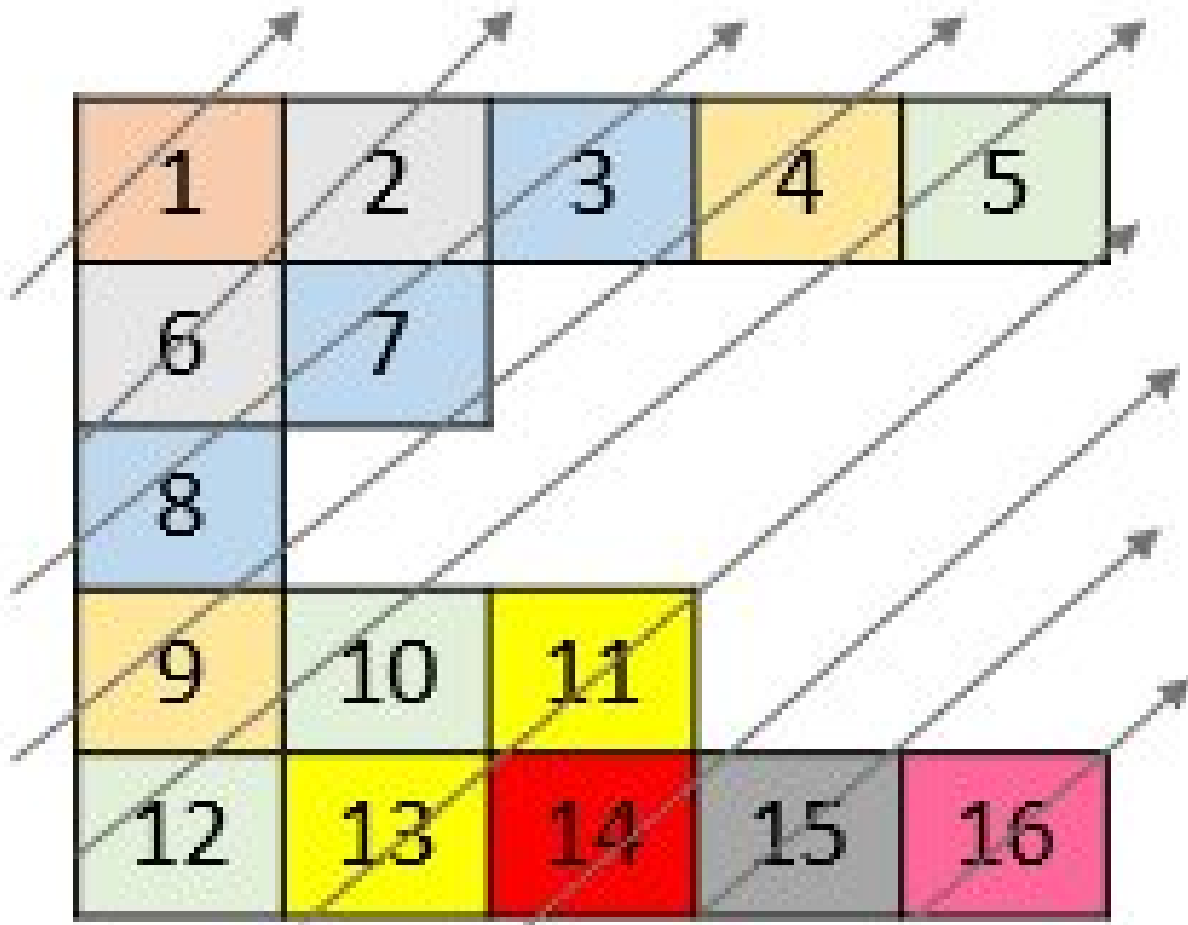
Given a 2D integer array `nums`, return all elements of `nums` in diagonal order as shown in the below images.

Example 1:



Input: `nums = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[1,4,2,7,5,3,8,6,9]`

Example 2:



Input: `nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]`

Output: `[1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $1 \leq \text{nums}[i].\text{length} \leq 105$
- $1 \leq \text{sum}(\text{nums}[i].\text{length}) \leq 105$
- $1 \leq \text{nums}[i][j] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a 2D list (rows may have different lengths), return elements in anti-diagonal order.

Diagonal index = $i+j$. Within each diagonal, list from bottom to top (higher i first). Right?"

#

"Let me walk:

nums=[[1,2,3],[4,5,6],[7,8,9]] →
[1,4,2,7,5,3,8,6,9] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Group elements by diagonal index ($i+j$) into a dict.

For each group, append elements in reverse row order (higher i first).

$O(n)$ where n = total elements.

Should I go ahead and optimize?"

class _1424_DiagonalTraverseII_Brute:

```
def findDiagonalOrder(self, nums):
    from collections import
defaultdict
    diag = defaultdict(list)
    for i, row in enumerate(nums):
        for j, val in enumerate(row):
            diag[i + j].append(val)
    result = []
    for key in sorted(diag):
        result.extend(reversed(diag[key]))

    return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(d \log d)$ sort on diagonal keys – iterate rows top-to-bottom instead.

We can avoid sorting the diagonal keys by iterating rows top-to-bottom:

elements encountered later in the same diagonal should appear FIRST (prepend or reverse at end).

Use an append approach per diagonal key, then reverse each bucket.

#

Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
from collections import defaultdict
```

```
class _1424_DiagonalTraverseII:
```

```
    def findDiagonalOrder(self, nums):
```

```
        diag_groups = defaultdict(list)
```

```
        max_key = 0
```

```
        for i, row in enumerate(nums):
```

```
            for j, val in enumerate(row):
```

```
                key = i + j
```

```
                diag_groups[key].append(v
```

```
al)
```

```
                max_key = max(max_key,
```

```
key)
```

```
        result = []
```

```
        for key in range(max_key + 1):
```

```
            result.extend(reversed(diag_g
```

```
roups[key]))
```

```
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# nums=[[1,2,3],[4,5,6],[7,8,9]]:
```

```
# diag[0]=[1], diag[1]=[2,4],  
diag[2]=[3,5,7], diag[3]=[6,8],  
diag[4]=[9].  
# reversed: [1],[4,2],[7,5,3],[8,6],[9] →  
[1,4,2,7,5,3,8,6,9] ✓  
#  
# Jagged: nums=[[1,2,3,4,5],[6,7],[8],[9,  
10,11],[12,13,14,15,16]] → correct order  
✓  
#  
# "No bugs. Appending top-to-bottom then  
reversing each bucket gives bottom-to-top  
order."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n)$ : one pass to build groups +  
one pass to collect. Space  $O(n)$ .  
# Using range(max_key+1) instead of  
sorted(diag_groups) avoids the  $O(d \log d)$   
sort on diagonals.  
# Follow-up: Diagonal Traverse I (#498) –  
rectangular matrix, alternating direction  
per diagonal.  
# Follow-up: spiral order (#54) vs  
diagonal order are both reordering  
problems on matrices."
```

Trie

Round 5 · Mid · Medium · 7 questions

Trie

□ #139 Word Break

MED[Open on LeetCode](#)

Array · Hash Table · String · Dynamic Programming · Trie · Memoization
Lists: Grind 169

Problem

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: *s* = "leetcode", *wordDict* = ["leet", "code"]

Output: true

Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: `s = "applepenapple", wordDict = ["apple", "pen"]`

Output: `true`

Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".

Note that you are allowed to reuse a dictionary word.

Example 3:

Input: `s = "catsandog", wordDict = ["cats", "dog", "sand", "and", "cat"]`

Output: `false`

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given string `s` and a dictionary of words, return `true` if `s` can be segmented into a sequence of dictionary words. Words can be reused. Right?"

#

"A few quick questions:

Can the empty string be segmented? Edge case – but `s.length >= 1` per constraints.
Same word can be used multiple times? Yes."

#

"Let me walk: `s='leetcode'`, `words=['leet','code']`. '`leet`'+'`code`' → `true` ✓

`s='catsandog'`, `words=['cats','dog','sand','and','cat']` → `false` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursive: if any prefix of `s` is in the dict, recurse on the suffix.


```
# Without memoization, exponential
branching –  $O(2^n)$  time.
# Should I go ahead and optimize?"
class _139_WordBreak_Brute:
    def wordBreak(self, s, wordDict):
        words = set(wordDict)
        def can(i):
            if i == len(s): return True
            for j in range(i+1,
len(s)+1):
                if s[i:j] in words and
can(j): return True
            return False
        return can(0)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the repeated
subproblem: can(i) is recomputed many
times.
# Bottom-up DP: dp[i] = True if s[:i] can
be segmented.
# Base: dp[0] = True (empty prefix).
# Transition: dp[i] = any(dp[j] and s[j:i]
in wordSet) for  $0 \leq j < i$ .
# Optimise by only checking back
max_word_len characters.
```

```
#
# Time: O(n * max_word_len). Space: O(n)."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
# approach with meaningful names."
class _139_WordBreak:
    def wordBreak(self, s, wordDict):
        word_set = set(wordDict)
        max_len = max(len(w) for w in
wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True
        for i in range(1, n + 1):
            for j in range(max(0, i -
max_len), i):
                if dp[j] and s[j:i] in
word_set:
                    dp[i] = True
                    break
        return dp[n]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
```

```
# s='leetcode', words={'leet','code'}:
# dp[0]=T. dp[4]: j=0,s[0:4]='leet' in
set,dp[0]=T → dp[4]=T.
# dp[8]: j=4,s[4:8]='code' in set,dp[4]=T
→ dp[8]=T. return True ✓
#
# s='catsandog': dp[3]=T('cat'),
dp[4]=T('cats'). dp[9]: no j gives a valid
word → False ✓
#
# "No bugs. Limiting look-back to
max_word_len avoids redundant substring
checks."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n * \text{max\_word\_len})$ : n positions ×
max_word_len substrings each. Space  $O(n)$ .
# Trie optimisation: insert all words into
a trie, walk forward from each dp[j]=True.
# Follow-up: Word Break II (#140) – return
all valid sentences; backtrack from
dp[n]=True.
# Follow-up: if wordDict is huge but s is
short, iterate over words instead of
positions."
```

Trie

□ #208 Implement Trie (Prefix Tree)

MED[Open on LeetCode](#)

Hash Table · String · Design · Trie

Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- **Trie()** Initializes the trie object.
- **void insert(String word)** Inserts the string word into the trie.
- **boolean search(String word)** Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- **boolean startsWith(String prefix)** Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:

Input

```
["Trie", "insert", "search", "search",  
"startsWith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"],  
["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null,  
true]
```

Explanation

```
Trie trie = new Trie();
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls in total will be made to insert, search, and startsWith.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Implement a Trie with insert(word),
search(word)→bool (exact match),
# and startsWith(prefix)→bool. Lowercase
letters only. Right?"
#
# "A few quick questions:
# search('app') should return false even
after inserting 'apple'? Yes – exact match
only.
# startsWith('app') should return true?
Yes – any inserted word with that prefix."
#
# "Let me walk: insert('apple').
search('apple')→T. search('app')→F.
# startsWith('app')→T. insert('app').
search('app')→T ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Use a set for exact search and iterate
all words for prefix check.
# search O(1), startsWith O(n * m) – slow
for many queries.
# Should I go ahead and optimize?"
class _208_ImplementTrieBrute:
```

```
def __init__(self): self.words = set()
def insert(self, word):
self.words.add(word)
def search(self, word): return word in self.words
def startsWith(self, prefix): return any(w.startswith(prefix) for w in self.words)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n*m)$ prefix scan.

True trie: each node has up to 26 children and an `is_end` flag.

insert: create nodes along the path, mark `is_end` at the last node.

search: walk path, return `is_end` of the final node (or `False` if path missing).

startsWith: walk path, return `True` if the path exists completely.

#

Time: $O(m)$ per operation where m = word/prefix length.

Space: $O(\text{total characters} * 26)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _208_ImplementTrie:
    def __init__(self):
        self._children = {}
        self._is_end = False
    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                node._children[ch] =
                _208_ImplementTrie()
            node = node._children[ch]
        node._is_end = True
    def search(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                return False
            node = node._children[ch]
        return node._is_end
    def startsWith(self, prefix):
        node = self
        for ch in prefix:
```



```
        if ch not in node._children:
            return False
        node = node._children[ch]
    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

insert('apple'):

root→a→p→p→l→e(is_end=True).

search('apple'): walk path → is_end=True
✓.

search('app'): walk a→p→p → is_end=False
→ False ✓.

startsWith('app'): walk a→p→p → path
exists → True ✓.

insert('app'): node at second p gets
is_end=True. search('app')→True ✓.

#

"No bugs. Using dict children is
memory-efficient for sparse tries vs a
fixed 26-slot array."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m)$ per op: walk at most m nodes.
Space $O(n*m)$: n words of avg length m .

Dict children vs array[26]: dict saves space for sparse alphabets.

Follow-up: delete a word – mark is_end=False; prune childless nodes bottom-up.

Follow-up: Word Search II (□#212) – DFS the board against a trie of all words."

Trie

□ #211 Design Add and Search Words Data Structure

MED[Open on LeetCode](#)

String · DFS · Design · Trie

Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input

```
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search", "search"]
```

```
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".ad"], ["b.."]]
```

Output

```
[null, null, null, null, false, true, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a data structure with addWord(word) and search(word).

search supports '.' as a wildcard matching any single letter. Right?"

#

"A few quick questions:

Can the pattern have multiple dots? Yes – up to 2 per problem constraints.

search('.') should match any one-character word? Yes."

#

"Let me walk:

addWord('bad', 'dad', 'mad').

search('.ad')→true. search('b..')→true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all words in a list. For search, scan all words and match character by character,

treating '.' as wildcard.

```
# O(n * m) per search where n = number of
words, m = word length.
# Should I go ahead and optimize?"
class _211_AddSearchWords_Brute:
    def __init__(self): self.words = []
    def addWord(self, word):
self.words.append(word)
    def search(self, word):
        for w in self.words:
            if len(w) != len(word):
continue
                if all(w[i] == word[i] or
word[i] == '.' for i in range(len(word))):
return True
        return False

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning all words on
each search.
# Trie with DFS for wildcards:
# addWord: standard trie insert O(m).
# search: walk trie; on '.' try ALL
children recursively; on a letter take the
matching child.
# Prune any branch that leads to a missing
character.
```

```
#  
# Time: addWord O(m). search O(m) best,  
O(26^dots * m) worst with dots.  
# Space: O(total chars)."  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
approach with meaningful names."  
class _211_AddSearchWords:  
    def __init__(self):  
        self._children = {}  
        self._is_end = False  
    def addWord(self, word):  
        node = self  
        for ch in word:  
            if ch not in node._children:  
                node._children[ch] =  
_211_AddSearchWords()  
            node = node._children[ch]  
        node._is_end = True  
    def search(self, word):  
        def dfs(node, i):  
            if i == len(word):  
                return node._is_end  
            ch = word[i]  
            if ch == '.':
```

```

        return any(dfs(child, i +
1) for child in node._children.values())
        if ch not in node._children:
            return False
        return
dfs(node._children[ch], i + 1)
return dfs(self, 0)

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

addWord('bad', 'dad', 'mad'): b→a→d(end),
d→a→d(end), m→a→d(end).

search('.ad'): dfs(root,0): ch='.', try
b,d,m → dfs(b_node,1): 'a'→dfs(a_node,2):
'd'→is_end=T → True ✓

search('b..'): dfs(root,0):
'b'→dfs(b,1): '.'→try 'a'→dfs(a,2):
'.'→try 'd'→is_end=T → True ✓

search('pad'): 'p' not in children →
False ✓

#

"No bugs. DFS correctly backtracks when
a '.' branch leads to a dead end."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"addWord $O(m)$. search $O(m)$ for no dots; $O(26^k * m)$ worst with k dots.
In practice dots are rare, so average is close to $O(m)$.
Follow-up: if patterns are very dot-heavy, cache results at (node, pattern_index) pairs.
Follow-up: support '*' (zero or more chars) – extended DFS with skip-0 and skip-1+ logic."

Trie

□ ★ #616 Add Bold Tag in String ★

MED[Open on LeetCode](#)

Array · Hash Table · String · Trie

Lists: ★ Premium | Premium 98

Problem

You are given a string *s* and an array of strings *words*.

You should add a closed pair of bold tag **** and **** to wrap the substrings in *s* that exist in *words*.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return *s* after adding the bold tags.

Example 1:

Input: `s = "abcxyz123", words = ["abc", "123"]`

Output: `"<b>abc</b>xyz<b>123</b>"`

Explanation: The two strings of words are substrings of `s` as following: `"abcxyz123"`.

We add `<b>` before each substring and `</b>` after each substring.

Example 2:

Input: `s = "aaabbb", words = ["aa","b"]`

Output: `"<b>aaabbb</b>"`

Explanation:

`"aa"` appears as a substring two times:
`"aaabbb"` and `"aaabbb"`.

`"b"` appears as a substring three times:
`"aaabbb"`, `"aaabbb"`, and `"aaabbb"`.

We add `<b>` before each substring and `</b>` after each substring: `"<b>a<b>a</b>>a</b><b>b</b><b>b</b><b>b</b>"`.

Since the first two `<b>`'s overlap, we merge them:

`"<b>aaa</b><b>b</b><b>b</b><b>b</b>"`.

Since now the four `<b>`'s are consecutive, we merge them:

`"<b>aaabbb</b>"`.

Constraints:

- `1 <= s.length <= 1000`
- `0 <= words.length <= 100`
- `1 <= words[i].length <= 1000`
- `s` and `words[i]` consist of English letters and digits.
- All the values of `words` are unique.

Note: This question is the same as 758. Bold Words in String.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Wrap all substrings of s that appear in the words list with tags.

Overlapping or adjacent bold regions should be merged into one. Right?"

#

"A few quick questions:

A word can appear multiple times in s – all occurrences get bolded? Yes.

Overlapping matches merge into one ... block? Yes."

#

"Let me walk: s='abcxyz123', words=['abc', '123'] → '**abc**xyz**123**' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Use a boolean array bold[i] = should
s[i] be bolded?
# For each word, find all occurrences in s
and mark those positions True.
# Then walk bold to insert <b> and </b>
tags at transitions.
# O(n * m * L) where m = number of words,
L = max word length.
# Should I go ahead and optimize?"
class _616_AddBoldTag_Brute:
    def addBoldTag(self, s, words):
        bold = [False] * len(s)
        for word in words:
            start = 0
            while (idx := s.find(word,
start)) != -1:
                for i in range(idx, idx +
len(word)): bold[i] = True
                start = idx + 1
        result = []; i = 0
        while i < len(s):
            if bold[i]:
                result.append('<b>')
                while i < len(s) and
bold[i]: result.append(s[i]); i += 1
```

```
        result.append('</b>')
    else:
        result.append(s[i]); i +=
1
    return ''.join(result)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n * m * L)$  find
loop.
# Trie optimization: insert all words into
a trie, then scan s once.
# At each position, walk the trie to find
all word matches starting there.
# Mark the corresponding range in the bold
array.
#  $O(n * \text{max\_}L)$  scan.
# But the brute with bool array is already
clean – let's refine it using intervals."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _616_AddBoldTag:
    def addBoldTag(self, s, words):
        n = len(s)
        bold = [False] * n
```

```
for word in words:
    wl = len(word)
    for i in range(n - wl + 1):
        if s[i:i+wl] == word:
            for j in range(i, i +
wl):
                bold[j] = True
result = []; i = 0
while i < n:
    if bold[i]:
        result.append('<b>')
        while i < n and bold[i]:
            result.append(s[i]);
i += 1
        result.append('</b>')
    else:
        result.append(s[i]); i +=
1
    return ''.join(result)

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s='aaabbb', words=['aa','b']:
# 'aa': matches at 0,1 → bold[0..2]=True.
```



```
# 'b': matches at 3,4,5 → bold[3..5]=True.  
All bold → '<b>aaabbb</b>' ✓  
#  
# s='abcxyz123', words=['abc','123']:  
# bold[0..2]=True, bold[6..8]=True. →  
'<b>abc</b>xyz<b>123</b>' ✓  
#  
# "No bugs. Adjacent True ranges merge  
naturally since the while loop consumes  
all."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n * m * L)$ :  $n$  positions  $\times m$   
words  $\times L$  substring check. Space  $O(n)$  bold  
array.  
# Trie reduces substring matching to  $O(n * \max\_L)$  – worth it when  $m$  is large.  
# Follow-up: Bold Words in String (#758) –  
identical problem, same solution.  
# Follow-up: Merge Intervals (#56) – if  
instead of a bool array we track intervals  
directly."
```

Trie

#692 Top K Frequent Words

MED[Open on LeetCode](#)

Hash Table · String · Trie · Sorting · Heap
Lists: Grind 169

Problem

Given an array of strings `words` and an integer `k`, return the `k` most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

Input: `words = ["i","love","leetcode","i","love","coding"], k = 2`

Output: `["i","love"]`

Explanation: "i" and "love" are the two most frequent words.

Note that "i" comes before "love" due to a lower alphabetical order.

Example 2:

Input: words = ["the","day","is","sunny",
"the","the","the","sunny","is","is"],
k = 4

Output: ["the","is","sunny","day"]

Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.

Constraints:

- $1 \leq \text{words.length} \leq 500$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- words[i] consists of lowercase English letters.
- k is in the range [1, The number of unique words[i]]

Follow-up: Could you solve it in $O(n \log(k))$ time and $O(n)$ extra space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Return the k most frequent words, sorted
by frequency descending.
# Ties broken alphabetically ascending.
Right?"
#
# "A few quick questions:
# k is always valid ( $\leq$  unique words)? Yes.
Words are lowercase only?"
#
# "Let me walk: words=['i','love','leetcod
de','i','love','coding'], k=2.
# i(2)>love(2) alphabetically: i < love →
[i,love] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Count frequencies, sort by (-freq, word)
as a composite key, take first k.
# O(n log n) – clean and readable.
# Should I go ahead and optimize?"
class _692_TopKFrequentWords_Brute:
    def topKFrequent(self, words, k):
        from collections import Counter
        count = Counter(words)
```

```
        return sorted(count.keys(),
key=lambda w: (-count[w], w))[:k]
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is sorting all unique words.
```

```
# Min-heap of size k with a custom comparator handles this in  $O(n \log k)$ :
```

```
# Store (-freq, word) – heap evicts the least desirable (smallest freq, then lex-largest).
```

```
# This keeps the k best candidates at all times.
```

```
#
```

```
# Time:  $O(n \log k)$ . Space:  $O(k)$ ."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
import heapq
```

```
class _692_TopKFrequentWords:
```

```
    def topKFrequent(self, words, k):
```

```
        from collections import Counter
```

```
        count = Counter(words)
```

```
        heap = []
```

```
        for word, freq in count.items():
```

```
        heapq.heappush(heap, (-freq,
word))
    return [heapq.heappop(heap)[1]
for _ in range(k)]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# words=['i','love','leetcode','i','love',
# 'coding'], k=2:
# count={i:2,love:2,leetcode:1,coding:1}.
# heap: [(-2,'i'),(-2,'love'),(-1,'coding'),(-1,'leetcode')] (min-heap by -freq
then word).
# pop1: (-2,'i'). pop2: (-2,'love'). →
['i','love'] ✓
#
# "No bugs. Negating freq makes Python's
min-heap act as a max-heap on frequency."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Heap  $O(n \log k)$ . Sort  $O(n \log n)$ . Heap
wins when  $k \ll n$ .
# Tuple comparison  $(-freq, word)$  handles
ties alphabetically for free in Python.
```

Follow-up: Top K Frequent Elements (#347) – integers, no tie-breaking, bucket sort $O(n)$.

Follow-up: streaming top-k words – `heapq.nsmallest` after each word arrives."

Trie

□ #720 Longest Word in Dictionary

MED[Open on LeetCode](#)

Array · Hash Table · String · Trie · Sorting

Lists: Denny Zhang

Problem

Given an array of strings words representing an English Dictionary, return the longest word in words that can be built one character at a time by other words in words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: words =
["w", "wo", "wor", "worl", "world"]
Output: "world"
Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

Example 2:

Input: words = ["a", "banana", "app", "appl", "ap", "apply", "apple"]
Output: "apple"
Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- `words[i]` consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the longest word where every prefix (excluding the full word) also exists in the list.

Ties: return the lexicographically smallest. Return '' if none. Right?"

#

"A few quick questions:

A single-letter word is always valid since it has no proper prefixes to check? Yes.

Words are all lowercase? Yes."

#

"Let me walk:

words=['w','wo','wor','worl','world'] → 'world' (all prefixes present) ✓

words=['a','banana','app','appl','ap','apply','apple'] → 'apple' (lex smaller than 'apply') ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Put all words in a set. Sort by (length desc, alpha asc). For each word,

```
# check that every prefix from length 1 to
len-1 is in the set.
# Return the first match.
#  $O(n * L^2)$  where  $L$  = max word length.
# Should I go ahead and optimize?"
class _720_LongestWordInDictionary_Brute:
    def longestWord(self, words):
        word_set = set(words)
        result = ''
        for w in sorted(words, key=lambda
x: (-len(x), x)):
            if all(w[:i] in word_set for i
in range(1, len(w))):
                result = w; break
        return result

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(L^2)$  prefix
check per word.
# Trie approach: insert all words, then
BFS/DFS visiting only nodes marked
is_end=True.
# The deepest reachable node via
only-end-marked paths gives the longest
valid word.
#
```

```
# Time:  $O(n \cdot L)$  to build trie +  $O(n \cdot L)$  to search. Space:  $O(n \cdot L)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _720_LongestWordInDictionary:
    def longestWord(self, words):
        word_set = set(words)
        result = ''
        for word in words:
            if all(word[:i] in word_set
for i in range(1, len(word))):
                if len(word) >
len(result) or (len(word) == len(result)
and word < result):
                    result = word
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# words=['w', 'wo', 'wor', 'worl', 'world']:
```

```
# 'world': 'w'✓, 'wo'✓, 'wor'✓, 'worl'✓ →
```

```
len=5 > len('')=0 → result='world'. →
```

```
'world' ✓
```

```
# words=['a','banana','app','appl','ap','  
apply','apple']:  
# 'apple': 'a','ap','app','appl' all in  
set ✓. len=5. result='apple'.  
# 'apply': all prefixes ✓. len=5=5,  
'apply'>'apple' → keep 'apple'. ✓  
#  
# "No bugs. The len/lex comparison handles  
ties correctly without sorting."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n * L^2)$ : n words × L prefixes ×  
L per set lookup. Space  $O(n*L)$  for the  
set.  
# Trie reduces prefix lookup to  $O(1)$  per  
char, bringing it to  $O(n*L)$ .  
# Follow-up: if words stream in one at a  
time, maintain a trie with is_end flags.  
# Follow-up: allow at most one missing  
prefix – extend the BFS to allow one  
'gap'."
```

Trie

□ #1166 Design File System

MED

[Open on LeetCode](#)

Hash Table · String · Design · Trie

Lists: Denny Zhang

Problem

You are asked to design a file system that allows you to create new paths and associate them with different values.

The format of a path is one or more concatenated strings of the form: / followed by one or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string "" and "/" are not.

Implement the FileSystem class:

- **bool createPath(string path, int value)** Creates a new path and associates a value to it if possible and returns true. Returns false if the path already exists or its parent path doesn't exist.

- **int get(string path)** Returns the value associated with path or returns -1 if the path doesn't exist.

Example 1:

Input:

```
["FileSystem","createPath","get"]  
[[],["/a",1],["/a"]]
```

Output:

```
[null,true,1]
```

Explanation:

```
FileSystem fileSystem = new  
FileSystem();
```

Example 2:

Input:

```
["FileSystem","createPath","createPath"  
,"get","createPath","get"]  
[[],["/leet",1],["/leet/code",2],["/lee  
t/code"],["/c/d",1],["/c"]]
```

Output:

```
[null,true,true,2,false,-1]
```

Explanation:

```
FileSystem fileSystem = new  
FileSystem();
```

Constraints:

- $2 \leq \text{path.length} \leq 100$
- $1 \leq \text{value} \leq 109$
- Each path is valid and consists of lowercase English letters and '/'.
- At most 104 calls in total will be made to `createPath` and `get`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a file system with `createPath(path, value)→bool` and `get(path)→int`.

`createPath` creates a path with a value; parent must exist.

Returns false if path exists or parent doesn't exist.

`get` returns value or -1. Right?"

#

"A few quick questions:

Root '/' is always implicitly there?

Yes. Paths start with '/'? Yes."

#


```
# "Let me walk: createPath('/a',1)→T.  
createPath('/a/b',2)→T. get('/a')→1.  
# createPath('/c/d',1)→F (no '/c').  
get('/c')→-1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Store full path strings in a dict  
mapping path→value.
```

```
# createPath: check parent path exists,  
then insert. get: lookup.
```

```
# O(L) per op where L = path length.
```

```
# Should I go ahead and optimize?"
```

```
class _1166_DesignFileSystem_Brute:  
    def __init__(self): self.paths = {'':  
0}  
    def createPath(self, path, value):  
        parent = path[:path.rfind('/')]  
        if parent not in self.paths or  
path in self.paths: return False  
        self.paths[path] = value; return  
True  
    def get(self, path): return  
self.paths.get(path, -1)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the rfind scan for the parent path.

Trie where each node maps component names to child nodes and stores a value.

createPath: split on '/', walk/create nodes, check parent validity.

get: walk trie, return node value.

#

Time: $O(L)$ per op (L = path length).

Space: $O(\text{total unique path components})$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1166_DesignFileSystem:
```

```
    def __init__(self):
```

```
        self._store = {}
```

```
    def createPath(self, path, value):
```

```
        parts = path.split('/')
```

```
        parent = '/'.join(parts[:-1])
```

```
        if parent and parent not in
```

```
self._store:
```

```
            return False
```

```
        if path in self._store:
```

```
            return False
```

```
        self._store[path] = value
    return True

    def get(self, path):
        return self._store.get(path, -1)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

createPath('/a',1): parent=''. '' not in store? '' is falsy → skip parent check.

path '/a' not in store → store['/a']=1 → True ✓

createPath('/a/b',2): parent='/a'. '/a' in store ✓. '/a/b' not in store → store['/a/b']=2 ✓

get('/a')→1 ✓. createPath('/c/d',1): parent='/c'. '/c' not in store → False ✓

#

"No bugs. The 'if parent and ...' handles the root case cleanly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Both ops $O(L)$: dict lookup + parent extraction.

Space $O(\text{total unique paths})$. No prefix sharing needed since we store full paths.

True trie would share prefixes but adds complexity – dict is simpler and equally fast here.

Follow-up: delete(path) – remove from dict; handle subtrees by scanning for prefix matches.

Follow-up: list directory – return keys that start with path+'/' and have no further '/'. "

Backtracking

Round 5 · Mid · Medium · 6 questions

Backtracking

□ #17 Letter Combinations of a Phone Number

MED[Open on LeetCode](#)

Hash Table · String · Backtracking

Lists: Grind 169

Problem

Given a string containing digits from 2–9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: `digits = "23"`

Output: `["ad","ae","af","bd","be","bf","cd","ce","cf"]`

Example 2:

Input: `digits = "2"`

Output: `["a","b","c"]`

Constraints:

- $1 \leq \text{digits.length} \leq 4$
- `digits[i]` is a digit in the range ['2', '9'].

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a string of digits 2–9, return all possible letter combinations

the digits could represent on a phone keypad. Return [] for empty input. Right?"

#

"A few quick questions:

Output order can be any? Yes. Can digits include 0 or 1? No – they have no letters.

Return combinations of length `len(digits)`?"

#

"Let me walk: `digits='23'`. 2→abc, 3→def.

Combine each:

ad, ae, af, bd, be, bf, cd, ce, cf → 9 combinations ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS/queue: start with [''], then for each digit expand every partial result # by appending each letter that digit maps to.

$O(4^n * n)$ time and space — $n = \text{len}(\text{digits})$, max 4 letters per digit.

Should I go ahead and optimize?"

```
class _17_LetterCombinations_Brute:
    def letterCombinations(self, digits):
        if not digits: return []
        phone = {'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl', '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
        result = ['']
        for d in digits:
            result = [r + c for r in result for c in phone[d]]
        return result
```

PHASE 3 — OPTIMIZE

"Both BFS and backtracking produce the same output; backtracking uses less intermediate space.

```
# The bottleneck is unavoidable – we must
enumerate all  $4^n$  combinations.
# Backtracking builds each combination
character by character and avoids
# storing all intermediate partial
strings.
#
# Time:  $O(4^n * n)$ . Space:  $O(n)$  call depth
+  $O(4^n * n)$  output."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _17_LetterCombinations:
    def letterCombinations(self, digits):
        if not digits:
            return []
        phone =
{'2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
    '6': 'mno', '7': 'pqrs', '8'
: 'tuv', '9': 'wxyz'}
        result = []
        def backtrack(i, path):
            if i == len(digits):
                result.append(''.join(pat
h)); return
```

```
        for ch in phone[digits[i]]:
            path.append(ch)
            backtrack(i + 1, path)
            path.pop()
    backtrack(0, [])
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

digits='23':

backtrack(0,[]): ch='a' →

backtrack(1,['a']): ch='d'→append 'ad'.

'e'→'ae'. 'f'→'af'.

ch='b' → 'bd', 'be', 'bf'. ch='c' →

'cd', 'ce', 'cf'. result=9 strings ✓

#

digits='': return [] ✓

digits='2': → ['a', 'b', 'c'] ✓

#

"No bugs. path.pop() after each recursive call restores state for the next branch."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(4^n * n)$: 4^n combinations, each takes $O(n)$ to record. Space $O(n)$ call depth.

Iterative BFS uses the same time but $O(4^n * n)$ intermediate space.

Follow-up: if some digits have 0 letters, skip them.

Follow-up: count combinations without enumerating – just multiply letter counts per digit."

Backtracking

□ #22 Generate Parentheses

MED[Open on LeetCode](#)

String · Dynamic Programming · Backtracking

Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Input: $n = 3$

Output: ["((()))", "(()())", "()(())", "()(())", "()()()", "()]"]

Example 2:

Input: $n = 1$

Output: ["()"]

Constraints:

- $1 \leq n \leq 8$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given n pairs of parentheses, generate all combinations of well-formed parentheses.

Well-formed means every '(' is closed before any preceding '(' closes. Right?"

#

"A few quick questions:

$n=0 \rightarrow []$? $n=1 \rightarrow ['()']$? Yes to both. Any ordering of output is fine."

#

"Let me walk: $n=2 \rightarrow ['()()']$ ✓. $n=3 \rightarrow 5$ combinations ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all $2^{(2n)}$ sequences of '(' and ')', filter valid ones.

$O(2^{(2n)} * n)$ – exponentially wasteful since most sequences are invalid.

Should I go ahead and optimize?"

```
class _22_GenerateParentheses_Brute:
    def generateParenthesis(self, n):
```

```
def valid(s):
    bal = 0
    for c in s:
        if c == '(': bal += 1
        else: bal -= 1
        if bal < 0: return False
    return bal == 0

from itertools import product
return [''.join(p) for p in
product('()', repeat=2*n) if valid(p)]

# PHASE 3 — OPTIMIZE
# "The bottleneck is generating invalid
strings then filtering them.
# Backtracking with validity pruning –
only place characters that keep the string
valid:
# Add '(' if open count < n. Add ')' if
close count < open count.
# This generates ONLY valid strings – no
wasted work.
#
# Time:  $O(4^n / \sqrt{n})$  – the nth Catalan
number. Space:  $O(n)$  call depth."

# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized
approach with meaningful names."
class _22_GenerateParentheses:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, open_count,
close_count):
            if len(path) == 2 * n:
                result.append(''.join(pat
h)); return
            if open_count < n:
                path.append('(');
backtrack(path, open_count + 1,
close_count); path.pop()
            if close_count < open_count:
                path.append(')');
backtrack(path, open_count, close_count +
1); path.pop()
        backtrack([], 0, 0)
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# n=2:
```



```

# backtrack([],0,0): add '(' →
backtrack(['(',1,0):
# add '(' → ['(',(']: add ')' → ['(()']
add ')' → '(()))' ✓, add ')' → ['(']: add
')' → '()'
# backtrack(['()'],1,1): add '(' → ['()(']
add ')' → '()())' ✓
# result=['(()))','()())'] ✓
#
# "No bugs. close_count < open_count
prevents ')' when there's no matching '('
to close."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(4^n / \sqrt{n})$ : Catalan number
of valid strings, each takes  $O(n)$  to
record.
# Brute  $O(2^{(2n)})$  – exponentially worse.
# Follow-up: count valid sequences without
generating –  $\text{Catalan}(n) = C(2n,n)/(n+1)$ .
# Follow-up: Longest Valid Parentheses
(#32) – find the longest valid substring
using DP or stack."

```

Backtracking

□ #39 Combination Sum

MED[Open on LeetCode](#)

Array · Backtracking

Lists: Grind 169

Problem

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1

Output: []

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of candidates are distinct.
- $1 \leq \text{target} \leq 40$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given distinct positive integers and a target, return all unique combinations summing to target. Each number may be used unlimited times.

No duplicate combinations in the output. Right?"

#

"A few quick questions:

If the same combination can be formed in different orders, count it once only? Yes.

All values are positive? Yes."

#

"Let me walk: candidates=[2,3,6,7], target=7 → [[2,2,3],[7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all combinations with repetition up to target sum, filter by sum.

$O(n^{(target/min_c)})$ – exponential and generates duplicates.

```
# Should I go ahead and optimize?"
class _39_CombinationSum_Brute:
    def combinationSum(self, candidates,
target):
        result = set()
        def dfs(path, rem):
            if rem == 0:
result.add(tuple(sorted(path))); return
            for c in candidates:
                if c <= rem:
path.append(c); dfs(path, rem-c);
path.pop()
            dfs([], target)
        return [list(c) for c in result]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the set-based deduplication and the generation of order-duplicates.

Backtracking with a start index: pass the current index so we only pick from # the current candidate onward – this prevents [3,2] when [2,3] already exists. # Sort first to enable early pruning when candidate > remaining.

#

```
# Time:  $O(n^{(target/min\_c)})$  – same asymptotic but pruned and no dedup overhead."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _39_CombinationSum:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, path, remaining):
            if remaining == 0:
                result.append(list(path))
            for i in range(start, len(candidates)):
                if candidates[i] > remaining:
                    break
                path.append(candidates[i])
                backtrack(i, path, remaining - candidates[i])
```

```
        path.pop()
    backtrack(0, [], target)
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

candidates=[2,3,6,7], target=7
(sorted):

start=0: pick 2 → [2], rem=5. pick 2 →
[2,2], rem=3. pick 2 → [2,2,2], rem=1. 2>1
break.

pick 3 → [2,2,3], rem=0 → append ✓. pick
3 → [2,3], rem=2. pick 3>2 break. pick 6>5
break.

start=1: pick 3, rem=4. pick 3 → [3,3],
rem=1. 3>1 break. pick 6>4 break.

start=2: pick 6, rem=1. 6>1 break.

start=3: pick 7, rem=0 → append ✓. →
[[2,2,3],[7]] ✓

#

"No bugs. 'break' after candidates[i] >
remaining prunes because the array is
sorted."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^{(target/min_c)})$: at most $target/min_c$ levels deep, n choices each.
Space $O(target/min_c)$ recursion depth.
Follow-up: Combination Sum II (#40) –
each number used once, has duplicates.
Use $i+1$ instead of i , skip dup values at
same level: if $i > start$ and
 $cands[i] == cands[i-1]$ skip.
Follow-up: Combination Sum IV (#377) –
count ordered sequences summing to target
(DP)."

Backtracking

□ #46 Permutations

MED[Open on LeetCode](#)

Array · Backtracking

Lists: Grind 169

Problem

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2:

Input: `nums = [0,1]`

Output: `[[0,1],[1,0]]`

Example 3:

Input: `nums = [1]`

Output: `[[1]]`

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$
- All the integers of nums are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given an array of distinct integers, return all possible permutations in any order. Right?"

#

"A few quick questions:

n is at most 6 so $6! = 720$ permutations is fine. All elements are distinct? Yes."

#

"Let me walk: $\text{nums} = [1, 2, 3] \rightarrow 6$ permutations in any order ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Use a visited array: at each position try every unvisited number.

When path length equals n, record the permutation.

$O(n! * n)$ time – unavoidable since we must enumerate all permutations.

Should I go ahead and optimize?"

```
class _46_Permutations_Brute:
```

```
    def permute(self, nums):
```

```
        result = []; n = len(nums);
```

```
visited = [False] * n
```

```
    def bt(path):
```

```
        if len(path) == n:
```

```
result.append(list(path)); return
```

```
        for i in range(n):
```

```
            if not visited[i]:
```

```
                visited[i] = True;
```

```
path.append(nums[i])
```

```
        bt(path)
```

```
        path.pop();
```

```
visited[i] = False
```

```
    bt([]); return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ visited array – we can eliminate it.

Swap-based backtracking: maintain a 'start' pointer.

```
# Everything before start is already
placed; swap any element from [start, n)
into position start,
# recurse, then swap back.
#
# Same  $O(n! * n)$  time but  $O(n)$  space and
no visited array."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _46_Permutations:
    def permute(self, nums):
        result = []
        def backtrack(start):
            if start == len(nums):
                result.append(list(nums))
        ; return
        for i in range(start,
len(nums)):
            nums[start], nums[i] =
nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] =
nums[i], nums[start]
            backtrack(0)
```

return result

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,2,3]:

bt(0): swap(0,0)→[1,2,3] bt(1):

swap(1,1)→[1,2,3] bt(2): swap(2,2) bt(3)
append [1,2,3].

swap(1,2)→[1,3,2] bt(2): swap(2,2)

bt(3) append [1,3,2]. swap back→[1,2,3].

swap(0,1)→[2,1,3] bt(1): →

[2,1,3], [2,3,1]. swap back→[1,2,3].

swap(0,2)→[3,2,1] bt(1): →

[3,2,1], [3,1,2]. → total 6 ✓

#

"No bugs. Swapping back restores the
array for the next iteration at each
level."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n! * n)$: $n!$ permutations, each
 $O(n)$ to copy. Space $O(n)$: call depth.

Visited-array approach: $O(n)$ extra space
for the boolean array.

```
# Follow-up: Permutations II (#47) –  
duplicates in input. Sort first, skip  
duplicate  
# values at the same level: if i>start and  
nums[i]==nums[start] after sorting, skip.  
# Follow-up: Next Permutation (#31) –  
generate just the lexicographically next  
one."
```

Backtracking

□ #79 Word Search

MED[Open on LeetCode](#)

Array · String · Backtracking · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = `"ABCCED"`

Output: `true`

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = `"SEE"`

Output: `true`

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = `"ABCB"`

Output: `false`

Constraints:

- `m == board.length`
- `n = board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`

- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a grid of characters and a word, return true if the word exists

as a path of adjacent (4-directional) non-repeated cells. Right?"

#

"A few quick questions:

Can the path revisit a cell? No – each cell used at most once per path.

Case-sensitive? Yes."

#

"Let me walk: board=[['A','B','C'],['S','F','C'],['A','D','E']], word='ABCCED'.

#

A(0,0)→B(0,1)→C(0,2)→C(1,2)→E(2,2)→D(2,1)

✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For every cell as a potential start, run DFS along the word.

Mark visited cells with a sentinel '#', unmark on backtrack.

$O(m \cdot n \cdot 4^L)$ where L is word length — unavoidable in the worst case.

Should I go ahead and optimize?"

```
class _79_WordSearch_Brute:
```

```
    def exist(self, board, word):
```

```
        m, n = len(board), len(board[0])
```

```
        def dfs(r, c, idx):
```

```
            if idx == len(word): return
```

```
            True
```

```
                if not (0 <= r < m and 0 <= c < n) or
```

```
board[r][c] != word[idx]: return False
```

```
                tmp = board[r][c];
```

```
board[r][c] = '#'
```

```
                found = dfs(r+1, c, idx+1) or
```

```
dfs(r-1, c, idx+1) or dfs(r, c+1, idx+1) or
```

```
dfs(r, c-1, idx+1)
```

```
                board[r][c] = tmp; return
```

```
            found
```

```
        return any(dfs(r, c, 0) for r in
range(m) for c in range(n))
```

PHASE 3 — OPTIMIZE

"The bottleneck is exploring unpromising branches early. Two practical pruning improvements:

1. Pre-check character frequencies: if board has fewer of any letter than word needs, return False.

2. If word's last character is rarer in the board than the first, reverse the word to

start from the rarer end – prunes the search tree more aggressively.

#

Time: $O(m \times n \times 4^L)$ worst case but much faster with pruning. Space: $O(L)$ call depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _79_WordSearch:
    def exist(self, board, word):
        from collections import Counter
```

```
        board_count = Counter(c for row in
board for c in row)
        word_count = Counter(word)
        if any(word_count[c] >
board_count[c] for c in word_count):
            return False
        if board_count[word[0]] >
board_count[word[-1]]:
            word = word[::-1]
        m, n = len(board), len(board[0])
        def dfs(r, c, idx):
            if idx == len(word): return
True
            if not (0 <= r < m and 0 <= c
< n) or board[r][c] != word[idx]:
                return False
            board[r][c] = '#'
            found = (dfs(r+1,c,idx+1) or
dfs(r-1,c,idx+1) or
                    dfs(r,c+1,idx+1) or
dfs(r,c-1,idx+1))
            board[r][c] = word[idx]
            return found
        return any(dfs(r, c, 0) for r in
range(m) for c in range(n))
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

word='ABCCED', board as above:

Start at (0,0)='A'. dfs(0,0,0)→dfs(0,1,'B')→dfs(0,2,'C')→dfs(1,2,'C')→dfs(2,2,'E')→dfs(2,1,'D')

idx=6=len → True ✓

#

word='ABCB': after finding A→B→C, next must be 'B' adjacent to C. Board shows B at (0,1)

but it's already marked '#' — backtrack → False ✓

#

"No bugs. Restoring board[r][c] = word[idx] (not '#') correctly unmarks the sentinel."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*4^L)$ worst. Space $O(L)$ call depth.

The two pruning heuristics dramatically cut average case without changing worst case.

Follow-up: Word Search II (#212) – find ALL words from a list in one DFS using a Trie.

Follow-up: allow diagonal moves – add 4 diagonal direction pairs."

Backtracking

#113 Path Sum II

MED[Open on LeetCode](#)

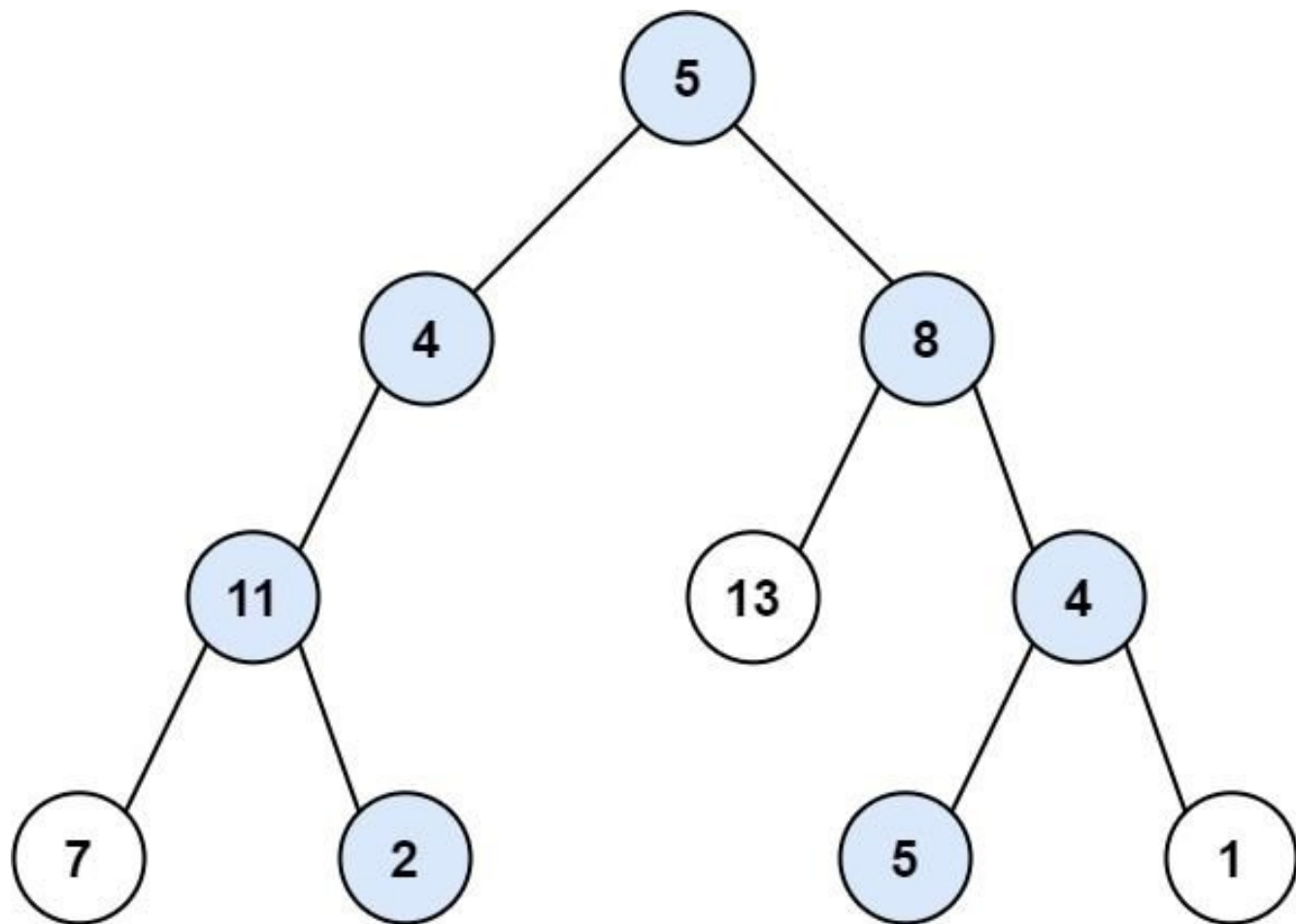
Backtracking · Tree · DFS**Lists: Grind 169**

Problem

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:



Input: root =

[5,4,8,11,null,13,4,7,2,null,null,5,1],
targetSum = 22

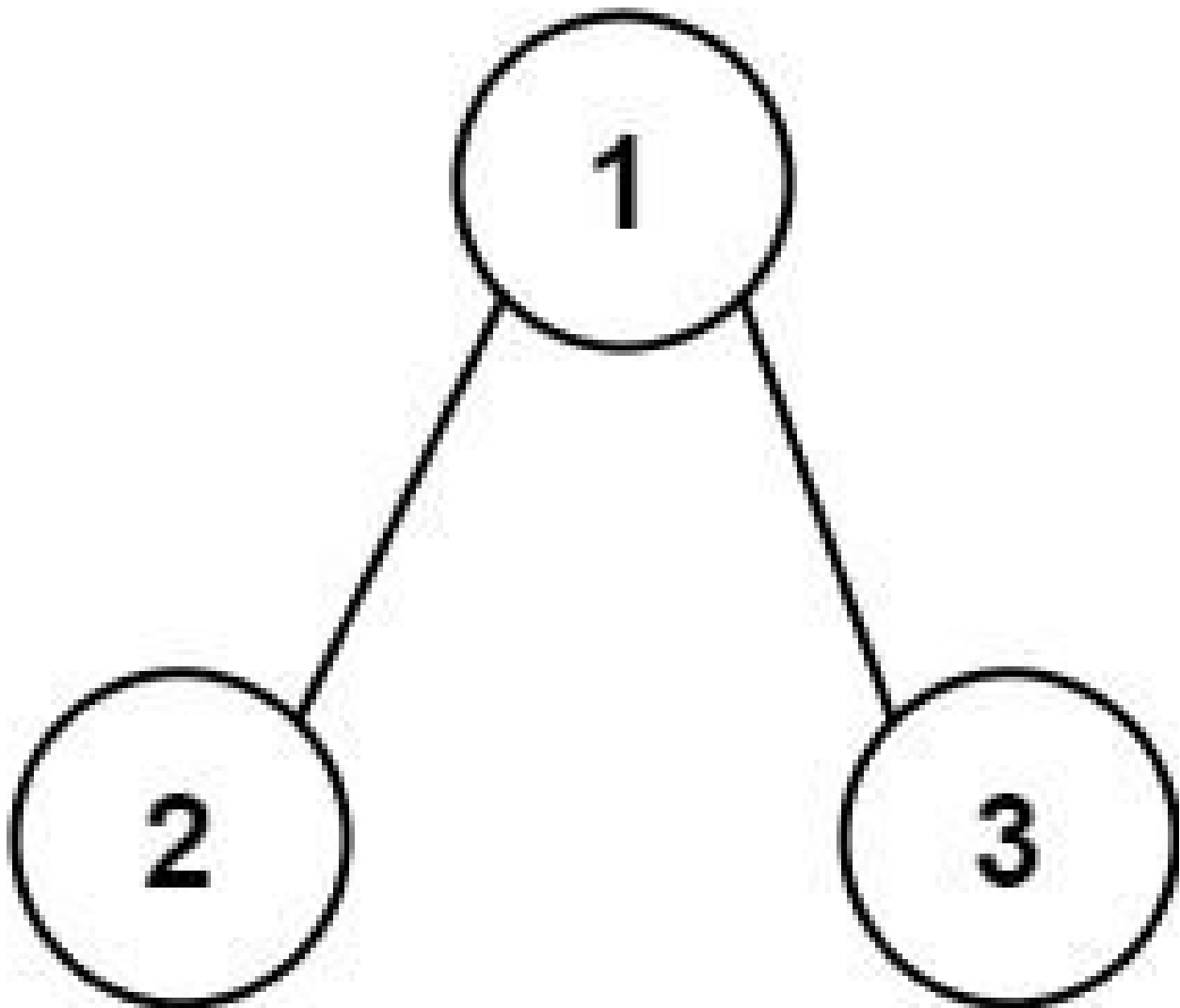
Output: [[5,4,11,2],[5,8,4,5]]

Explanation: There are two paths whose sum equals targetSum:

$$5 + 4 + 11 + 2 = 22$$

$$5 + 8 + 4 + 5 = 22$$

Example 2:



Input: root = [1,2,3], targetSum = 5
Output: []

Example 3:

Input: root = [1,2], targetSum = 0
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 5000].

- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return all root-to-leaf paths where node values sum to targetSum.

A leaf is a node with no children.

Return the actual paths (node values), not just count. Right?"

#

"A few quick questions:

Can values be negative? Yes – so we can't prune on sum exceeding target.

Empty tree → return []."

#

"Let me walk: tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22.

5→4→11→2=22 ✓, 5→8→4→5=22 ✓ →
[[5,4,11,2],[5,8,4,5]] ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# DFS collecting all root-to-leaf paths,
then filter by sum.
# O(n*L) where L = average path length
(tree height).
# Should I go ahead and optimize?"
class _113_PathSumII_Brute:
    def pathSum(self, root, targetSum):
        all_paths = []
        def dfs(node, path):
            if not node: return
            path.append(node.val)
            if not node.left and not
node.right: all_paths.append(list(path))
            dfs(node.left, path);
dfs(node.right, path)
            path.pop()
        dfs(root, [])
        return [p for p in all_paths if
sum(p) == targetSum]

# PHASE 3 — OPTIMIZE
# "The bottleneck is summing each complete
path after collecting it.
```

```
# Track remaining = targetSum - node.val  
as we descend.
```

```
# Check remaining == 0 at leaves only –  
avoids post-collection summing.
```

```
#
```

```
# Time:  $O(n \cdot L)$  – same asymptotic. Space:  
 $O(L)$  call depth."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _113_PathSumII:
```

```
    def pathSum(self, root, targetSum):
```

```
        result = []
```

```
        def backtrack(node, remaining,  
path):
```

```
            if not node:
```

```
                return
```

```
            path.append(node.val)
```

```
            remaining -= node.val
```

```
            if not node.left and not
```

```
node.right and remaining == 0:
```

```
                result.append(list(path))
```

```
            else:
```

```
                backtrack(node.left,  
remaining, path)
```

```
                backtrack(node.right,  
remaining, path)  
                path.pop()  
            backtrack(root, targetSum, [])  
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

tree=[5,4,8,11,null,13,4,7,2,null,null,
5,1], target=22:

backtrack(5,22,[]): path=[5], rem=17.

backtrack(4,17): path=[5,4], rem=13.

backtrack(11,13): path=[5,4,11], rem=2.

backtrack(7,2): rem=-5, leaf, ≠0 pop.

backtrack(2,2): rem=0, leaf, =0 → append
[5,4,11,2] ✓. Pop back up.

backtrack(8,17): ... → [5,8,4,5] ✓

#

"No bugs. path.pop() after each node
correctly backtracks the path list."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \cdot L)$: n nodes visited, copying
path at leaves is $O(L)$ each.

Space $O(L)$: call stack depth = tree height.

Follow-up: Path Sum I (#112) – return bool, not paths. Can short-circuit on first match.

Follow-up: Path Sum III (#437) – any downward path summing to target; use prefix sum hashmap."

Greedy

Round 5 · Mid · Medium · 5 questions

Greedy

□ #134 Gas Station

MED[Open on LeetCode](#)

Array · Greedy

Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the i th station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 4. Your tank = $4 - 1 + 5 = 8$

Travel to station 0. Your tank = $8 - 2 + 1 = 7$

Travel to station 1. Your tank = $7 - 3 + 2 = 6$

Travel to station 2. Your tank = $6 - 4 + 3 = 5$

Example 2:

Input: gas = [2,3,4], cost = [3,4,3]

Output: -1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 0. Your tank = $4 - 3 + 2 = 3$

Travel to station 1. Your tank = $3 - 3 + 3 = 3$

You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.

Constraints:

- $n == \text{gas.length} == \text{cost.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{gas}[i], \text{cost}[i] \leq 104$
- The input is generated such that the answer is unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

n gas stations in a circle. gas[i] = fuel at i, cost[i] = fuel to reach i+1.

Start with empty tank. Find the starting station to complete the circuit.

If impossible return -1; the solution is guaranteed unique if it exists. Right?"

#

"A few quick questions:

Can we start at any station? Yes. Must return to the starting station? Yes."

#

"Let me walk: gas=[1,2,3,4,5], cost=[3,4,5,1,2] → start at station 3.

0+4-1=3 → +5=8 → +1=9 → +2=11 → +3=14 → -4=10 → return to 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try each station as start. Simulate the full circuit; if tank never goes negative, return it.

$O(n^2)$ time, $O(1)$ space.

```
# Should I go ahead and optimize?"
class _134_GasStation_Brute:
    def canCompleteCircuit(self, gas,
cost):
        n = len(gas)
        for start in range(n):
            tank = 0; ok = True
            for i in range(n):
                tank += gas[(start + i) %
n] - cost[(start + i) % n]
                if tank < 0: ok = False;
break
            if ok: return start
        return -1
```

PHASE 3 — OPTIMIZE

"The bottleneck is trying every start independently.

Two greedy observations:

1. If total gas < total cost, no solution – return -1.

2. If we run out of gas starting at s after reaching index i,

no station between s and i can be a valid start (they'd all begin with less gas).

```
# So reset candidate to i+1 and continue.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _134_GasStation:
    def canCompleteCircuit(self, gas,
cost):
        total_surplus = 0
        current_tank = 0
        candidate = 0
        for i in range(len(gas)):
            net = gas[i] - cost[i]
            total_surplus += net
            current_tank += net
            if current_tank < 0:
                candidate = i + 1
                current_tank = 0
        return candidate if total_surplus
>= 0 else -1
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# gas=[1,2,3,4,5], cost=[3,4,5,1,2]:  
total=15-15=0≥0.  
# i=0: net=-2, tank=-2<0 → cand=1, tank=0.  
i=1: net=-2, tank=-2<0 → cand=2, tank=0.  
# i=2: net=-2, tank=-2<0 → cand=3, tank=0.  
i=3: net=3, tank=3. i=4: net=3, tank=6.  
# return cand=3 ✓  
#  
# gas=[2,3,4], cost=[3,4,3]:  
total=9-10=-1<0 → return -1 ✓  
#  
# "No bugs. The candidate reset skips all  
stations from the old candidate to i."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n)$ : single pass. Space  $O(1)$ :  
three integer variables.  
# The key insight: if total gas  $\geq$  total  
cost, a valid start always exists;  
# the greedy candidate is guaranteed to be  
it.  
# Follow-up: what if there are multiple  
valid starting points? The problem  
guarantees uniqueness  
# but the greedy still finds the first  
valid one.
```


Follow-up: minimum cost to fill gas (variant) – different problem, uses a monotonic stack."

Greedy

□ #179 Largest Number

MED[Open on LeetCode](#)

Array · String · Greedy · Sorting

Lists: Grind 169

Problem

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`

Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`

Output: `"9534330"`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Arrange non-negative integers so they form the largest number. Return as a string.

Edge: if the result starts with '0', return '0' (e.g. [0,0] → '0'). Right?"

#

"A few quick questions:

Values can be 0? Yes. Return a string because the result may be very large? Yes."

#

"Let me walk: nums=[10,2] → '210' > '102' → '210' ✓

nums=[3,30,34,5,9] → '9534330' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all permutations of nums, form the concatenated string, return the maximum.

```
# 0(n! * n) – completely infeasible for
large n.
# Should I go ahead and optimize?"
class _179_LargestNumber_Brute:
    def largestNumber(self, nums):
        from itertools import
permutations
        return max(''.join(str(n) for n
in perm) for perm in permutations(nums))

# PHASE 3 — OPTIMIZE
# "The bottleneck is generating all
permutations.
# Custom comparator: to decide whether a
comes before b, compare str(a)+str(b) vs
str(b)+str(a).
# Sort all numbers by this comparator;
concatenate; strip leading zeros if
needed.
#
# Time: 0(n log n). Space: 0(n) for the
string array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
```

```
from functools import cmp_to_key
class _179_LargestNumber:
    def largestNumber(self, nums):
        strs = [str(n) for n in nums]
        def compare(a, b):
            return (1 if b + a > a + b
else -1)
        strs.sort(key=cmp_to_key(compare)
)
        result = ''.join(strs)
        return '0' if result[0] == '0'
else result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[10,2]: compare('10','2'): '210'
vs '102' → '210'>'102' → 2 before 10.

sorted=['2','10']. → '210' ✓

#

nums=[3,30,34,5,9]: compare determines
order. '9'>'5'>'34'>'3'>'30'.

→ '9534330' ✓

#

nums=[0,0]: sorted=['0','0'].

result='00'. result[0]='0' → return '0' ✓

#

"No bugs. The leading-zero check handles [0,0] without special-casing."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n * k)$: sorting with $O(k)$ string comparisons. Space $O(n*k)$: string array.

The comparator is transitive and total-ordering – provably correct for any input.

Follow-up: smallest number from array – same comparator, ascending sort.

Follow-up: if numbers can be very large (BigInteger) – convert to strings first anyway."

Greedy

□ ★ #280 Wiggle Sort ★

MED[Open on LeetCode](#)

Array · Greedy · Sorting

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, reorder it such that `nums[0] <= nums[1] >= nums[2] <= nums[3]....`

You may assume the input array always has a valid answer.

Example 1:

Input: `nums = [3,5,2,1,6,4]`

Output: `[3,5,1,6,2,4]`

Explanation: `[1,6,2,5,3,4]` is also accepted.

Example 2:

Input: `nums = [6,6,5,6,3,8]`

Output: `[6,6,5,6,3,8]`

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$

- $0 \leq \text{nums}[i] \leq 104$
- It is guaranteed that there will be an answer for the given input nums.

Follow up: Could you solve the problem in $O(n)$ time complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Reorder nums in-place so: $\text{nums}[0] \leq \text{nums}[1] \geq \text{nums}[2] \leq \text{nums}[3] \geq \dots$

Multiple valid answers exist. Right?"

#

"A few quick questions:

Any valid wiggle order is accepted? Yes.
Modify in-place, no return? Yes."

#

"Let me walk: $\text{nums}=[3,5,2,1,6,4] \rightarrow [3,5,1,6,2,4] \checkmark$ (one valid answer)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.


```
# Sort the array, then interleave: place
smaller half at even indices, larger half
at odd.
#  $O(n \log n)$  sort +  $O(n)$  interleave.
# Should I go ahead and optimize?"
class _280_WiggleSort_Brute:
    def wiggleSort(self, nums):
        nums.sort()
        for i in range(1, len(nums) - 1,
2):
            nums[i], nums[i + 1] = nums[i
+ 1], nums[i]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n \log n)$  sort.
# Greedy one-pass  $O(n)$ : check each index
i.
# Odd index ( $i \% 2 == 1$ ):  $\text{nums}[i]$  should be  $\geq$ 
 $\text{nums}[i-1]$ . If not, swap.
# Even index ( $i \% 2 == 0$ ) for  $i > 0$ :  $\text{nums}[i]$ 
should be  $\leq \text{nums}[i-1]$ . If not, swap.
# A local swap always fixes the current
violation without breaking prior pairs.
#
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _280_WiggleSort:
    def wiggleSort(self, nums):
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] <
nums[i-1]) or (i % 2 == 0 and nums[i] >
nums[i-1]):
                nums[i], nums[i-1] =
nums[i-1], nums[i]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[3,5,2,1,6,4]:

i=1(odd): 5>=3 ✓. i=2(even): 2<=5 ✓.

i=3(odd): 1<2 → swap → [3,5,1,2,6,4].

i=4(even): 6>2 → swap → [3,5,1,6,2,4].

i=5(odd): 4>=2 ✓. → [3,5,1,6,2,4] ✓

#

nums=[6,6,5,6,3,8]: all conditions
already satisfied → no swaps ✓

#

"No bugs. The swap at position i only
affects the pair (i-1, i), never undoes

earlier swaps."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: in-place swaps.

Greedy correctness: swapping adjacent elements to fix one condition never invalidates the

previously satisfied condition – because the new value at $i-1$ satisfies the prior pair.

Follow-up: Wiggle Sort II (#324) – strict inequalities

nums[0]<nums[1]>nums[2]<...

Requires median partitioning to guarantee strict separation.

Follow-up: count wiggle subsequences (#376) – DP/greedy, no reordering needed."

Greedy

□ #954 Array of Doubled Pairs

MED[Open on LeetCode](#)

Array · Hash Table · Greedy · Sorting

Lists: CodeSignal

Problem

Given an integer array of even length `arr`, return `true` if it is possible to reorder `arr` such that $arr[2 * i + 1] = 2 * arr[2 * i]$ for every $0 \leq i < len(arr) / 2$, or `false` otherwise.

Example 1:

Input: `arr = [3,1,3,6]`

Output: `false`

Example 2:

Input: `arr = [2,1,2,6]`

Output: `false`

Example 3:

Input: arr = [4,-2,2,-4]

Output: true

Explanation: We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or [2,4,-2,-4].

Constraints:

- $2 \leq \text{arr.length} \leq 3 * 10^4$
- arr.length is even.
- $-105 \leq \text{arr}[i] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Can we reorder arr so every two elements form a pair where $\text{arr}[2i+1] = 2 * \text{arr}[2i]$?

Return true if such a pairing exists. Right?"

#

"A few quick questions:

Values can be negative? Yes – e.g. [-2,-4] works since $-4 = -2 * 2$.

The array has even length? Yes – guaranteed."

```
#  
# "Let me walk: arr=[4,-2,2,-4] → pair  
# (-2,-4) and (2,4) → true ✓"  
  
# PHASE 2 — BRUTE FORCE  
# "Let me start with brute force to lock  
# in the logic.  
# Try all ways to pair elements, checking  
# pair[1]=2*pair[0].  
#  $O(n! / (n/2)! / 2^{(n/2)})$  –  
# combinatorially explosive.  
# Should I go ahead and optimize?"  
class _954_ArrayOfDoubledPairs_Brute:  
    def canReorderDoubled(self, arr):  
        from itertools import  
permutations  
        n = len(arr)  
        for perm in permutations(arr):  
            if all(perm[2*i+1] ==  
2*perm[2*i] for i in range(n//2)):  
                return True  
        return False  
  
# PHASE 3 — OPTIMIZE  
# "The bottleneck is the combinatorial  
# search.
```

```
# Greedy: sort by absolute value. Process
the smallest absolute value element first.
# For each element x, we need to find and
consume its pair 2*x.
# A Counter tracks remaining elements. If
2*x is not available, return False.
#
# Time: O(n log n). Space: O(n) for the
Counter."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _954_ArrayOfDoubledPairs:
    def canReorderDoubled(self, arr):
        from collections import Counter
        count = Counter(arr)
        for x in sorted(count.keys(),
key=abs):
            if count[x] > 0:
                if count[2 * x] <
count[x]:
                    return False
                count[2 * x] -= count[x]
                count[x] = 0
        return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

arr=[4,-2,2,-4]:

count={4:1,-2:1,2:1,-4:1}.

sorted by abs: [-2,-4,2,4].

x=-2: count[-2]=1>0. count[-4]=1>=1 ✓.

count[-4]=0, count[-2]=0.

x=-4: count[-4]=0, skip. x=2:

count[2]=1>0. count[4]=1>=1 ✓.

count[4]=0.

x=4: count[4]=0, skip. → True ✓

#

arr=[3,1,3,6]: count={3:2,1:1,6:1}.

x=1: count[2]=0 < count[1]=1 → False ✓

#

"No bugs. Sorting by abs value handles negatives correctly – -2 pairs with -4."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: sort + linear scan.

Space $O(n)$ Counter.

The abs-value sort is the key insight for handling negatives.

Follow-up: if 'double' is replaced by 'k times' – same greedy, check count[k*x].

Follow-up: count the number of valid pairings – DP or combinatorics on the counts."

Greedy

□ #2131 Longest Palindrome by Concatenating Two Letter Words

MED[Open on LeetCode](#)

Array · Hash Table · String · Greedy · Counting Lists: CodeSignal

Problem

You are given an array of strings `words`. Each element of `words` consists of two lowercase English letters.

Create the longest possible palindrome by selecting some elements from `words` and concatenating them in any order. Each element can be selected at most once.

Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.

A palindrome is a string that reads the same forward and backward.

Example 1:

Input: words = ["lc","cl","gg"]

Output: 6

Explanation: One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6.

Note that "clggcl" is another longest palindrome that can be created.

Example 2:

Input: words =

["ab","ty","yt","lc","cl","ab"]

Output: 8

Explanation: One longest palindrome is "ty" + "lc" + "cl" + "yt" = "tylcclyt", of length 8.

Note that "lcyttycl" is another longest palindrome that can be created.

Example 3:

Input: words = ["cc","ll","xx"]

Output: 2

Explanation: One longest palindrome is "cc", of length 2.

Note that "ll" is another longest palindrome that can be created, and so is "xx".

Constraints:

- $1 \leq \text{words.length} \leq 105$
- $\text{words}[i].\text{length} == 2$
- $\text{words}[i]$ consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Two-letter words; select some subset and concatenate into the longest palindrome.

Each word used at most once. Letters are case-sensitive. Right?"

#

"A few quick questions:

Can the same word appear multiple times in the input? Yes.

'lc' and 'cl' can be placed symmetrically? Yes: 'lc...cl'."

#

"Let me walk: words=['lc','cl','gg'] → 'lc'+ 'gg'+ 'cl' = 'lcggcl' → len=6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subsets and orderings; check if the concatenation is a palindrome.

$O(2^n * n!)$ – completely infeasible.

Should I go ahead and optimize?"

```
class _2131_LongestPalindromeTwoLetterWords_Brute:
```

```
    def longestPalindrome(self, words):
        from itertools import
permutations, chain
        best = 0
        for r in range(len(words) + 1):
            for subset in
permutations(words, r):
                s = ''.join(subset)
```

```
        if s == s[::-1]: best =  
max(best, len(s))  
    return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is enumeration.

Greedy with a Counter:

Non-palindromic words ($ab \neq ba$): pair 'ab' with 'ba' – each matched pair contributes 4 chars.

Palindromic words (gg, aa): each pair-of-two contributes 4 chars; one leftover can be the center (+2).

Walk through pairs of words and their reverses; track how many centers are available.

#

Time: $O(n)$. Space: $O(n)$ Counter."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class

2131 LongestPalindromeTwoLetterWords:

```
    def longestPalindrome(self, words):  
        from collections import Counter
```

```
count = Counter(words)
length = 0
center_used = False
for word, freq in count.items():
    rev = word[::-1]
    if word == rev:
        pairs = freq // 2
        length += pairs * 4
        if freq % 2 == 1 and not
center_used:
            length += 2
            center_used = True
    elif rev in count and word <
rev:
        matched = min(freq,
count[rev])
        length += matched * 4
return length

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# words=['lc','cl','gg']:
# count={lc:1,cl:1,gg:1}.
# 'gg'=rev: pairs=0, freq%2=1,
center_used=False → length+=2,
```

```
center_used=True.  
# 'lc' < 'cl'? No. 'cl' < 'lc'? Yes.  
min(count[cl],count[lc])=1 → length+=4.  
# total=2+4=6 ✓  
#  
# words=['ab','ty','yt','lc','cl','ab']:  
count={ab:2,ty:1,yt:1,lc:1,cl:1}.  
# 'ty'<'yt': min(1,1)=1 → +4. 'lc'<'cl':  
+4. 'ab'=rev? No (ab≠ba). 'ba' not in  
count.  
# total=8 ✓  
#  
# "No bugs. Processing word < rev avoids  
double-counting non-palindromic pairs."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n): one pass over Counter. Space  
O(n) Counter.  
# At most one palindromic center word  
contributes 2 chars – track with  
center_used flag.  
# Follow-up: Longest Palindrome (#409) –  
same structure with single characters.  
# Follow-up: if we need the actual  
palindrome string, construct it from  
paired words."
```


Round 6 · Mid · Hard

Round 6

Mid Priority · Hard

28 questions · 5 patterns

Linked List #25 #432 #460

Stack #32 #42 #84 #224 #239 #716 #772 #895

**Heap #23 #218 #272 #295 #358 #499 #632
#759 #1425**

Trie #212 #336 #588 #642

Backtracking #37 #51 #126 #996

Linked List

Round 6 · Mid · Hard · 3 questions

Linked List

#25 Reverse Nodes in k-Group

HAR[Open on LeetCode](#)

Linked List · Recursion

Lists: Grind 169

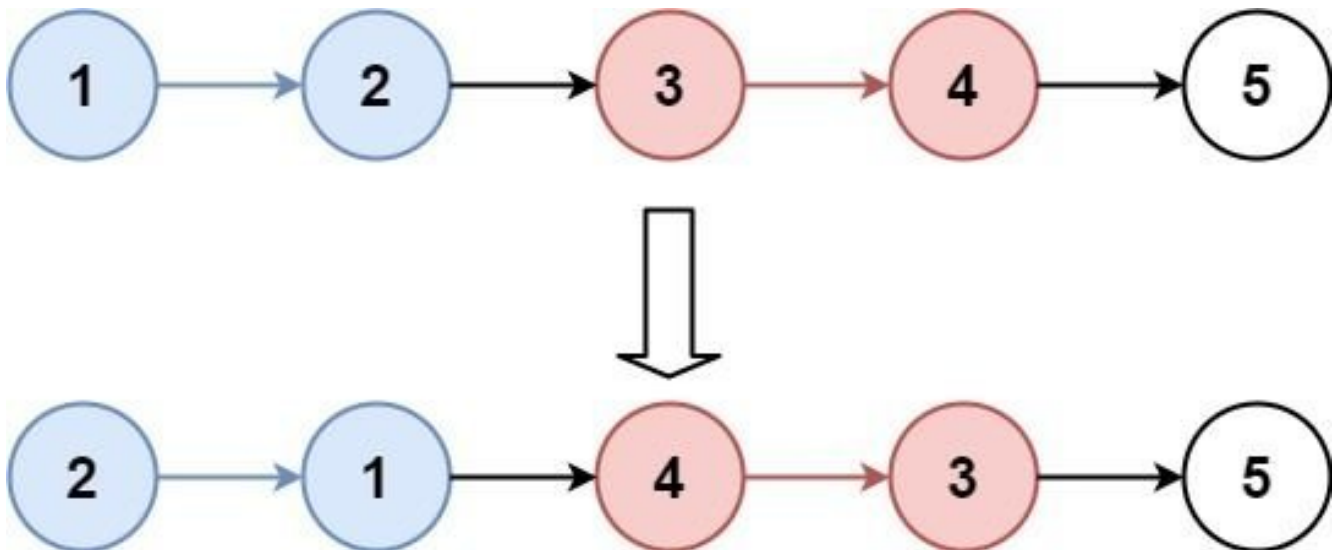
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

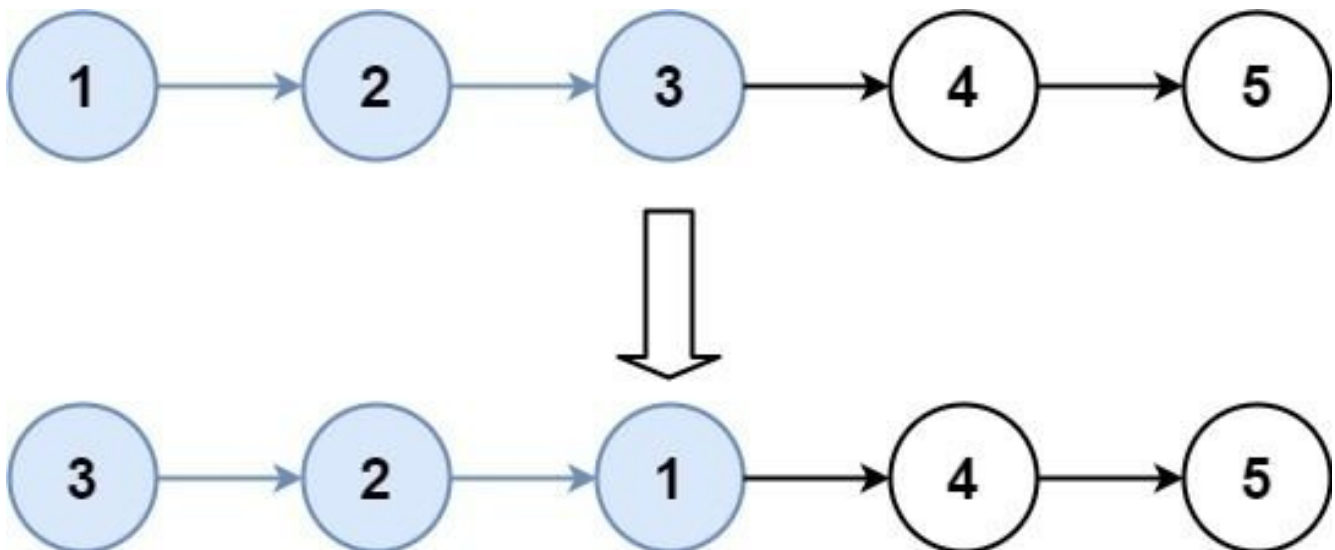
Example 1:



Input: head = [1,2,3,4,5], k = 2

Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3

Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n.

- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Reverse every k consecutive nodes in the linked list.

If remaining nodes $< k$, leave them as-is. Return the new head.

$O(1)$ extra space – no copying values. Right?"

#

"A few quick questions:

k can equal 1 (no-op)? Yes. k can equal list length? Yes – reverse everything."

#

"Let me walk: $[1,2,3,4,5]$, $k=2 \rightarrow [2,1,4,3,5]$ ✓. $k=3 \rightarrow [3,2,1,4,5]$ ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Collect all node values, reverse each
group of k elements in the array, rebuild
the list.
# O(n) time but O(n) space for the values
array.
# Should I go ahead and optimize?"
class _25_ReverseKGroup_Brute:
    def reverseKGroup(self, head, k):
        vals = []
        cur = head
        while cur: vals.append(cur.val);
        cur = cur.next
        for i in range(0, len(vals) -
len(vals) % k, k):
            vals[i:i+k] =
vals[i:i+k][::-1]
            cur = head
            for v in vals: cur.val = v; cur =
cur.next
        return head

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) auxiliary
value list.
```

```
# In-place pointer reversal group by group
- 0(1) extra space:
# First check if k nodes remain; if not,
stop.
# Reverse k nodes using the 3-pointer
technique, then link to the next group.
#
# Time: 0(n). Space: 0(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _25_ReverseKGroup:
    def reverseKGroup(self, head, k):
        def has_k(node, k):
            for _ in range(k):
                if not node: return False
                node = node.next
            return True

        def reverse_k(head, k):
            prev, cur = None, head
            for _ in range(k):
                nxt = cur.next; cur.next
                = prev; prev, cur = cur, nxt
            return prev

        dummy = ListNode(0, head)
```

```
        group_prev = dummy
    while has_k(group_prev.next, k):
        group_start = group_prev.next
        group_end = group_start
        for _ in range(k - 1):
            group_end = group_end.next
        next_group = group_end.next
        group_end.next = None
        group_prev.next =
reverse_k(group_start, k)
        group_start.next = next_group
        group_prev = group_start
    return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5], k=2:

Group1: start=1,end=2. Reverse

[1,2]→[2,1]. dummy→2→1. group_prev=1.

next_group=3.

Group2: start=3,end=4. Reverse

[3,4]→[4,3]. 1→4→3. group_prev=3.

next_group=5.

has_k(5,2): only 1 node → False → stop.

3.next=5. → [2,1,4,3,5] ✓

#

"No bugs. group_start.next=next_group reconnects the reversed group to the rest of the list."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each node processed in its reversal group exactly once. Space $O(1)$.

$k=2$ is exactly Swap Nodes in Pairs (#24).

Follow-up: $k=n$ reverses the entire list – identical to Reverse Linked List (#206).

Follow-up: reverse every other group of k – add a skip_k step between reverse steps."

Linked List

□ #432 All O'one Data Structure

HAR[Open on LeetCode](#)

Hash Table · Design · Doubly-Linked List ·
Linked List

Lists: Denny Zhang

Problem

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.

- **getMaxKey()** Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- **getMinKey()** Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in $O(1)$ average time complexity.

Example 1:

Input

```
["AllOne", "inc", "inc", "getMaxKey",  
"getMinKey", "inc", "getMaxKey",  
"getMinKey"]  
[[], ["hello"], ["hello"], [], [],  
["leet"], [], []]
```

Output

```
[null, null, null, "hello", "hello",  
null, "hello", "leet"]
```

Explanation

```
AllOne allOne = new AllOne();
```

Constraints:

- $1 \leq \text{key.length} \leq 10$
- key consists of lowercase English letters.

- It is guaranteed that for each call to `dec`, `key` is existing in the data structure.
- At most $5 * 10^4$ calls will be made to `inc`, `dec`, `getMaxKey`, and `getMinKey`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design a data structure with `inc(key)`, `dec(key)`, `getMaxKey()`, `getMinKey()` – all $O(1)$.

`inc` adds `key` with count 1 if absent; `dec` removes if count drops to 0. Right?"

#

"A few quick questions:

`getMaxKey/getMinKey` return any key with max/min count? Yes – any valid key.

What if the data structure is empty? Return an empty string."

#

"Let me walk: `inc('a')`, `inc('b')`, `inc('b')`, `inc('c')`, `inc('c')`, `inc('c')`."

```
# getMaxKey()→'c'(3). getMinKey()→'a'(1)
✓"
```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Dict {key: count}. inc/dec O(1).

getMax/getMin scan all entries O(n).

Should I go ahead and optimize?"

class _432_AllOneDataStructure_Brute:

def __init__(self): self.counts = {}

def inc(self, key):

self.counts[key] =

self.counts.get(key, 0) + 1

def dec(self, key):

self.counts[key] -= 1

if self.counts[key] == 0: del

self.counts[key]

def getMaxKey(self): return

max(self.counts, key=self.counts.get, default='')

def getMinKey(self): return

min(self.counts, key=self.counts.get, default='')

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n)$  scan for
getMax/getMin.
# Doubly linked list of buckets (each
bucket holds a count and a set of keys at
that count)
# + a hashmap {key → bucket_node}.
# inc: move key from bucket c to c+1. dec:
move key to c-1.
# getMax: any key from the tail bucket.
getMin: any key from the head bucket.
# All  $O(1)$  amortized. Space:  $O(n)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _432_AllOneDataStructure:
    class _Bucket:
        def __init__(self, count):
            self.count = count; self.keys
= set()
            self.prev = self.next = None
    def __init__(self):
        self._key_bucket = {}
        self._head = self._Bucket(0);
self._tail = self._Bucket(float('inf'))
```

```

        self._head.next = self._tail;
self._tail.prev = self._head
    def _insert_after(self, node, count):
        b = self._Bucket(count)
        b.prev = node; b.next = node.next
        node.next.prev = b; node.next = b
        return b

    def _remove_bucket(self, b):
        b.prev.next = b.next; b.next.prev
= b.prev
    def inc(self, key):
        if key not in self._key_bucket:
            if self._head.next.count !=
1:
                self._insert_after(self._
head, 1)
                self._head.next.keys.add(key)
                self._key_bucket[key] =
self._head.next
            else:
                cur = self._key_bucket[key]
                nxt_count = cur.count + 1
                if cur.next.count !=
nxt_count:

```

```

        self._insert_after(cur,
nxt_count)
        cur.next.keys.add(key)
        self._key_bucket[key] =
cur.next
        cur.keys.discard(key)
        if not cur.keys:
self._remove_bucket(cur)
    def dec(self, key):
        cur = self._key_bucket[key]
        prev_count = cur.count - 1
        if prev_count == 0:
            del self._key_bucket[key]
        else:
            if cur.prev.count !=
prev_count:
                self._insert_after(cur.pr
ev, prev_count)
            cur.prev.keys.add(key)
            self._key_bucket[key] =
cur.prev
            cur.keys.discard(key)
            if not cur.keys:
self._remove_bucket(cur)
    def getMaxKey(self):

```



```

        return
    next(iter(self._tail.prev.keys), '') if
    self._tail.prev != self._head else ''
    def getMinKey(self):
        return
    next(iter(self._head.next.keys), '') if
    self._head.next != self._tail else ''

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# inc('a'): bucket(1)={a}. head→1→tail.
# key_bucket={a:b1}.
# inc('b'): bucket(1)={a,b}. inc('b'):
# bucket(2)={b}, bucket(1)={a}.
# head→1→2→tail.
# getMaxKey()→tail.prev=b2→next(iter({b})
# )='b' ✓. getMinKey()→head.next=b1→'a' ✓
#
# "No bugs. Sentinel head/tail avoid
# boundary checks in
# _insert_after/_remove_bucket."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): set operations + DLL
# pointer updates. Space O(n)."

```

Follow-up: if we also need getMedianKey, we'd need a balanced BST instead of a DLL.
Follow-up: LFU Cache (#460) uses the same bucket-DLL structure."

Linked List

#460 LFU Cache

HAR[Open on LeetCode](#)

Hash Table · Linked List · Design ·

Doubly-Linked List

Lists: Denny Zhang

Problem

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- **LFUCache(int capacity)** Initializes the object with the capacity of the data structure.
- **int get(int key)** Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- **void put(int key, int value)** Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same

frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LFUCache", "put", "put", "get",  
"put", "get", "get", "put", "get",  
"get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2],  
[3], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, 3,  
null, -1, 3, 4]
```

Explanation

```
// cnt(x) = the use counter for key x
```

Constraints:

- $1 \leq \text{capacity} \leq 104$
- $0 \leq \text{key} \leq 105$
- $0 \leq \text{value} \leq 109$
- At most $2 * 105$ calls will be made to get and put.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# LFU Cache: get(key)→val or -1,
put(key,val). Evict the least frequently
used.
# On ties, evict the least recently used
among the LFU items. All O(1). Right?"
#
# "A few quick questions:
# Does get increment frequency? Yes. Put
on existing key – update value and
increment freq?"
#
# "Let me walk: LFUCache(2):
put(1,1),put(2,2),get(1)→1 (freq[1]=2).
# put(3,3) evicts key 2 (freq=1, LRU among
min-freq). get(2)→-1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Dict {key:(val,freq,last_used)}. On
evict, scan all entries for min freq then
min last_used.
# O(n) per put when eviction needed.
# Should I go ahead and optimize?"
class _460_LFUCache_Brute:
    def __init__(self, capacity):
```

```
        self.cap = capacity; self.cache =
{}; self.time = 0
    def _use(self, key):
        self.time += 1
        v, f, _ = self.cache[key];
self.cache[key] = (v, f+1, self.time)
    def get(self, key):
        if key not in self.cache: return
-1
        self._use(key); return
self.cache[key][0]
    def put(self, key, value):
        if self.cap == 0: return
        if key in self.cache:
            v, f, t = self.cache[key];
self.cache[key] = (value, f+1,
self.time+1); self.time += 1; return
        if len(self.cache) == self.cap:
            lfu = min(self.cache,
key=lambda k: (self.cache[k][1],
self.cache[k][2]))
            del self.cache[lfu]
            self.time += 1; self.cache[key] =
(value, 1, self.time)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ scan for the LFU key.

Three structures for $O(1)$:

1. key_map: {key → (val, freq)}

2. freq_map: {freq → OrderedDict(key→None)} – LRU order within same freq

3. min_freq: current minimum frequency

#

get: update freq, move key to freq+1 bucket. put: evict from freq_map[min_freq].

All ops $O(1)$. Space $O(\text{capacity})$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

from collections import defaultdict, OrderedDict

class _460_LFUCache:

def __init__(self, capacity):

self.cap = capacity

self.key_map = {}

self.freq_map =

defaultdict(OrderedDict)

self.min_freq = 0


```
def _update(self, key):
    val, freq = self.key_map[key]
    del self.freq_map[freq][key]
    if not self.freq_map[freq] and
freq == self.min_freq:
        self.min_freq += 1
    self.key_map[key] = (val, freq +
1)
    self.freq_map[freq + 1][key] =
None
def get(self, key):
    if key not in self.key_map: return
-1
    self._update(key)
    return self.key_map[key][0]
def put(self, key, value):
    if self.cap == 0: return
    if key in self.key_map:
        self._update(key)
        val, freq = self.key_map[key]
        self.key_map[key] = (value,
freq)
    else:
        if len(self.key_map) ==
self.cap:
```

```

        evict_key, _ = self.freq_
map[self.min_freq].popitem(last=False)
        del
self.key_map[evict_key]
        self.key_map[key] = (value,
1)

        self.freq_map[1][key] = None
self.min_freq = 1

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

LFUCache(2): put(1,1):

key_map={1:(1,1)}, freq_map={1:{1}}, min=1.

put(2,2): key_map={1:(1,1),2:(2,1)}, freq_map={1:{1,2}}, min=1.

get(1):

_update(1)→freq_map={1:{2},2:{1}}, min=1.

return 1 ✓.

put(3,3): evict freq_map[1]→evict key2.

key_map={1:(1,2),3:(3,1)}, min=1.

get(2)→-1 ✓.

#

"No bugs. OrderedDict preserves insertion order so popitem(last=False) evicts the LRU."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(1)$. Space $O(\text{capacity})$.

min_freq only ever decreases on put (new key starts at freq=1) – always correct.

Follow-up: LRU Cache (#146) – simpler, only recency matters.

Follow-up: add TTL – track expiry timestamps and remove in background or on access."

Stack

Round 6 · Mid · Hard · 8 questions

Stack

#32 Longest Valid Parentheses

HAR[Open on LeetCode](#)

String · Dynamic Programming · Stack

Lists: Grind 169

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

Input: s = "(()"

Output: 2

Explanation: The longest valid parentheses substring is "()".

Example 2:

Input: s = "))(()))"

Output: 4

Explanation: The longest valid parentheses substring is "()()".

Example 3:

Input: `s = ""`

Output: `0`

Constraints:

- $0 \leq s.length \leq 3 * 10^4$
- `s[i]` is '(', or ')'.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the length of the longest valid (well-formed) parentheses substring. Right?"

#

"A few quick questions:

Substring must be contiguous? Yes. Empty string → 0? Yes."

#

"Let me walk: `s='()' → '()' len=2 ✓`.
`s=')()())' → '()()' len=4 ✓`"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Try all substrings, check each for
# validity, track the longest.
#  $O(n^3)$  –  $O(n^2)$  substrings  $\times$   $O(n)$  validity
# check.
# Should I go ahead and optimize?"
class _32_LongestValidParentheses_Brute:
    def longestValidParentheses(self, s):
        def valid(sub):
            bal = 0
            for c in sub:
                if c == '(': bal += 1
                else: bal -= 1
                if bal < 0: return False
            return bal == 0
        best = 0
        for i in range(len(s)):
            for j in range(i+2, len(s)+1,
2):
                if valid(s[i:j]): best =
max(best, j-i)
            return best

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n^3)$  substring
# validation.
# Stack-based  $O(n)$  approach:
```

```
# Store indices. Push index of '(' onto
stack.
# On ')': if stack top is '(' index, pop
it (matched pair).
# Else push ')' index as new boundary.
# Initialize stack with [-1] as a base
boundary.
# After each match: length = current_index
- stack_top.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _32_LongestValidParentheses:
    def longestValidParentheses(self, s):
        stack = [-1]
        best = 0
        for i, ch in enumerate(s):
            if ch == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
```



```
        else:
            best = max(best, i -
stack[-1])
    return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s=')()())':

i=0')': pop -1 → empty → push 0.

stack=[0].

i=1'(': push 1. [0,1].

i=2')': pop 1. stack=[0].

best=max(0,2-0)=2.

i=3'(': push 3. [0,3].

i=4')': pop 3. stack=[0].

best=max(2,4-0)=4.

i=5')': pop 0 → empty → push 5. best=4 ✓

#

"No bugs. The initial [-1] sentinel handles valid substrings starting at index 0."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(n)$: stack holds at most n indices.

DP alternative: $dp[i]$ = length of valid substring ending at i . $O(n)$ time, $O(n)$ space.

$O(1)$ space: two-pass counting (left-right then right-left with open/close counters).

Follow-up: count number of valid parentheses substrings – not just the longest."

Stack

□ #42 Trapping Rain Water

HAR[Open on LeetCode](#)

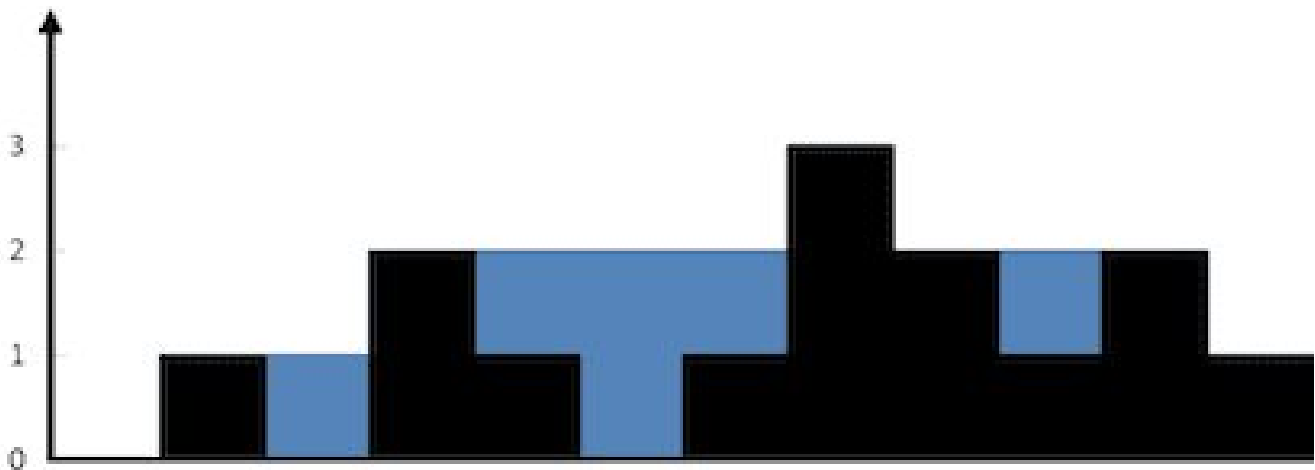
Array · Two Pointers · Dynamic Programming · Stack · Monotonic Stack

Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height =

[0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given an elevation map, return how much rain water can be trapped after raining.

```
# Water at position i = min(max_left,
max_right) - height[i], clamped to 0.
Right?"
#
# "A few quick questions:
# Heights are non-negative integers? Yes.
# Can a position hold 0 water? Yes."
#
# "Let me walk:
height=[0,1,0,2,1,0,1,3,2,1,2,1] → 6
units ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# For each bar i, scan left to find
max_left, scan right to find max_right.
# Water[i] = max(0, min(max_left,
max_right) - height[i]).
# O(n²) time, O(1) space.
# Should I go ahead and optimize?"
class _42_TrappingRainWater_Brute:
    def trap(self, height):
        n = len(height); total = 0
        for i in range(n):
```

```
        max_left = max(height[:i+1]);
max_right = max(height[i:])
        total += max(0, min(max_left,
max_right) - height[i])
    return total
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ max scans per position.

Two-pointer approach – $O(1)$ extra space:

left=0, right=n-1, left_max=0, right_max=0.

Process the side with the shorter current height:

If height[left] <= height[right]: water at left = left_max - height[left]. Move left right.

Else: water at right = right_max - height[right]. Move right left.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _42_TrappingRainWater:
```

```
def trap(self, height):
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    total = 0
    while left < right:
        if height[left] <=
height[right]:
            left_max = max(left_max,
height[left])
            total += left_max -
height[left]
            left += 1
        else:
            right_max =
max(right_max, height[right])
            total += right_max -
height[right]
            right -= 1
    return total
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

height=[0,1,0,2,1,0,1,3,2,1,2,1]:

l=0, r=11, lm=0, rm=0. h[0]=0<=h[11]=1:

lm=0, water+=0. l=1.

```
# h[1]=1<=1: lm=1. l=2. h[2]=0<lm=1:
water+=1. l=3.
# ... process continues, accumulating 6
total ✓
#
# "No bugs. Processing the shorter side
guarantees left_max/right_max are exact
bounds."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$ : single pass. Space  $O(1)$ :
four variables.
# Precompute max_left/max_right arrays:
 $O(n)$  time,  $O(n)$  space – simpler to
explain.
# Monotonic stack:  $O(n)$  time,  $O(n)$  space –
horizontal layer approach.
# Follow-up: Trapping Rain Water II (#407)
– 3D extension using a min-heap on
borders."
```


Stack

#84 Largest Rectangle in Histogram

HAR[Open on LeetCode](#)

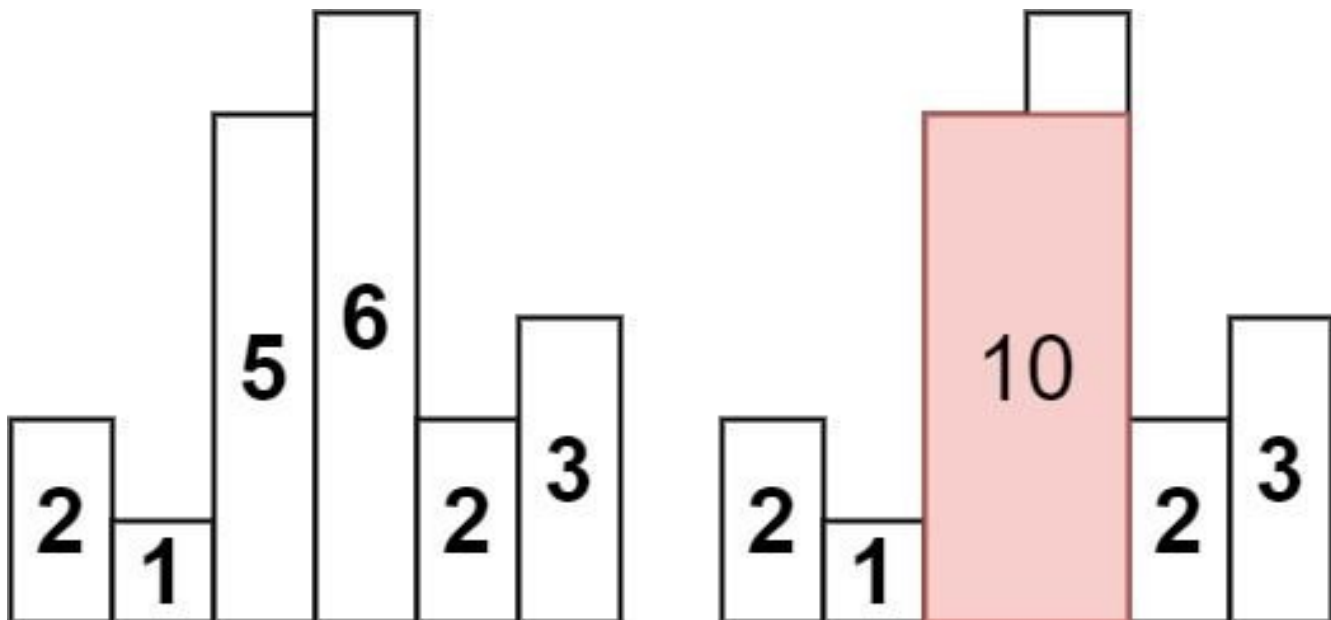
Array · Stack · Monotonic Stack

Lists: Grind 169

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:



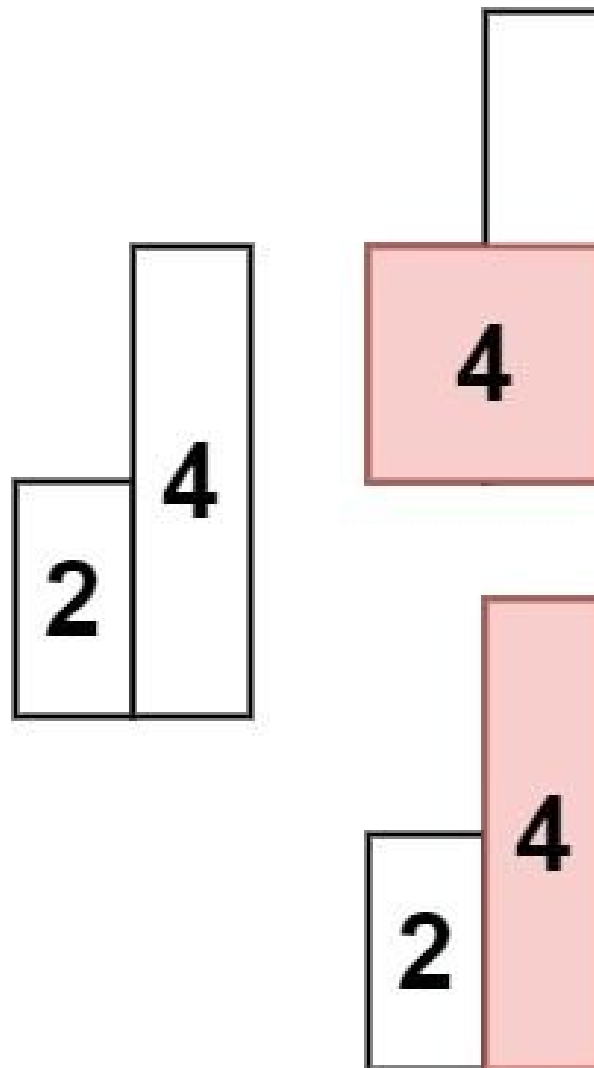
Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.

The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $0 \leq \text{heights}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given bar heights in a histogram, find the area of the largest rectangle

that fits entirely within the bars.

Width of each bar is 1. Right?"

#

"A few quick questions:

Heights can be 0? Yes. Single bar → area = its height? Yes."

#

"Let me walk: heights=[2,1,5,6,2,3] → area=10 (bars 2,3 at height 5) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each pair (i, j), area = (j-i+1) * min(heights[i..j]).

Track maximum across all pairs.

$O(n^2)$ – can maintain running min while expanding j.

Should I go ahead and optimize?"

class

_84_LargestRectangleInHistogram_Brute:

def largestRectangleArea(self, heights):

best = 0

for i in range(len(heights)):

min_h = heights[i]

for j in range(i, len(heights)):

min_h = min(min_h, heights[j])

best = max(best, min_h * (j - i + 1))

return best

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ nested scan.

```
# Monotonic increasing stack of indices:
# For each bar: while stack top is taller
# than current bar, pop and compute its
# area.
# Height = heights[popped], width =
# current_idx - stack_new_top - 1.
# Append sentinel 0s at both ends to force
# all bars to be popped.
#
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
# approach with meaningful names."
class _84_LargestRectangleInHistogram:
    def largestRectangleArea(self,
heights):
        heights = [0] + heights + [0]
        stack = [0]
        best = 0
        for i in range(1, len(heights)):
            while heights[stack[-1]] >
heights[i]:
                h = heights[stack.pop()]
                w = i - stack[-1] - 1
                best = max(best, h * w)
```

```
        stack.append(i)
    return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

heights=[0,2,1,5,6,2,3,0] (sentineled):

i=2(1): 2>1 → pop1:

h=2,w=2-0-1=1,area=2. stack=[0,2].

i=5(2): 6>2 → pop4:

h=6,w=5-3-1=1,area=6. 5>2 → pop3:

h=5,w=5-2-1=2,area=10 ✓.

i=7(0): pops all → final best=10 ✓

#

"No bugs. Sentinel 0s ensure every bar gets popped and processed – no leftovers."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each bar pushed and popped at most once. Space $O(n)$ stack.

Follow-up: Maximal Rectangle (#85) – for each row build histogram heights, apply this.

Follow-up: Maximum Width Ramp (#962) – similar two-phase monotonic stack pattern."

Stack

#224 Basic Calculator

HAR[Open on LeetCode](#)

Math · String · Stack · Recursion

Lists: Grind 169

Problem

Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "1 + 1"$

Output: 2

Example 2:

Input: $s = " 2-1 + 2 "$

Output: 3

Example 3:

Input: `s = "(1+(4+5+2)-3)+(6+8)"`

Output: 23

Constraints:

- $1 \leq s.length \leq 3 \times 10^5$
- `s` consists of digits, '+', '-', '(', ')', and ' '.
- `s` represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate a string with +, -, and parentheses. Non-negative integers, spaces allowed.

No * or /. No eval(). Right?"


```
#  
# "A few quick questions:  
# Can the expression start with '-' (unary  
minus)? Yes – per constraints.  
# Parentheses can be nested? Yes –  
arbitrary depth."  
#  
# "Let me walk: '1 + 1'→2 ✓.  
'(1+(4+5+2)-3)+(6+8)'→23 ✓"  
  
# PHASE 2 — BRUTE FORCE  
# "Let me start with brute force to lock  
in the logic.  
# Recursive descent parser: when we see  
'(', recursively evaluate the inner  
expression.  
# O(n) time, O(d) space for recursion  
depth d.  
# Should I go ahead and optimize?"  
class _224_BasicCalculator_Brute:  
    def calculate(self, s):  
        self.i = 0  
        def parse():  
            result = 0; sign = 1  
            while self.i < len(s):  
                c = s[self.i]
```

```
        if c.isdigit():
            num = 0
            while self.i < len(s)
and s[self.i].isdigit():
                num = num * 10 +
int(s[self.i]); self.i += 1
                result += sign * num;
continue

        elif c == '+': sign = 1
        elif c == '-': sign = -1
        elif c == '(': self.i +=
1; result += sign * parse(); continue
        elif c == ')': break
        self.i += 1
    return result
return parse()
```

PHASE 3 — OPTIMIZE

"The bottleneck is the recursion stack — O(d) depth for d nesting levels.

**# Iterative stack: push (result, sign) when '(' is encountered, pop and merge on ')'.
#**

Maintain current number being built, current result, and sign.

#

Time: $O(n)$. Space: $O(d)$ for the stack (same depth as recursion)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _224_BasicCalculator:
    def calculate(self, s):
        stack = []
        result = 0
        sign = 1
        num = 0
        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)
            elif ch in '+-':
                result += sign * num
                num = 0
                sign = 1 if ch == '+' else
-1
                sign = -1
            elif ch == '(':
                stack.append(result)
                stack.append(sign)
                result = 0
                sign = 1
            elif ch == ')':
```

```
        result += sign * num
        num = 0
        result *= stack.pop()
        result += stack.pop()
    return result + sign * num
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'(1+(4+5+2)-3)+(6+8)':

'(': push result=0, sign=1. reset. '1':
num=1. '+': result=1, sign=1.

'(': push 1, 1. reset.

'4+5+2': result=11. ')':

result=11*1+1=12. '-': result=12, sign=-1.

'3': num=3. ')': result=12+(-1)*3=9.

result=9*1+0=9. '+': result=9, sign=1.

'(': push 9, 1. '6+8': result=14. ')':
14*1+9=23 ✓

#

"No bugs. The stack stores
(outer_result, outer_sign) before
entering a parenthesis."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(d)$: stack depth = nesting depth.

Follow-up: Basic Calculator II (#227) – adds * and /; handle with operator precedence stack.

Follow-up: Basic Calculator III (#772) – all four operators with parentheses."

Stack

□ #239 Sliding Window Maximum

HAR[Open on LeetCode](#)

Array · Queue · Sliding Window · Monotonic Queue

Lists: Grind 169

Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [3,3,5,5,6,7]

Explanation:

Window position Max

```

-----
[1 3 -1] -3 5 3 6 7 3
1 [3 -1 -3] 5 3 6 7 3
1 3 [-1 -3 5] 3 6 7 5

```

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- $1 \leq k \leq \text{nums.length}$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Sliding window of size k moving through the array. Return the maximum of each

```
window. Right?"
```

```
#
```

```
# "A few quick questions:
```

```
#  $k \leq n$  always? Yes. Can values be  
negative? Yes."
```

```
#
```

```
# "Let me walk: nums=[1,3,-1,-3,5,3,6,7],  
k=3 → [3,3,5,5,6,7] ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Slide window, compute max at each  
position using Python's max().
```

```
#  $O(n*k)$  time,  $O(1)$  extra space.
```

```
# Should I go ahead and optimize?"
```

```
class _239_SlidingWindowMaximumBrute:
```

```
    def maxSlidingWindow(self, nums, k):
```

```
        return [max(nums[i:i+k]) for i in  
range(len(nums) - k + 1)]
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the  $O(k)$  max scan per  
window position.
```

```
# Monotonic deque of indices — maintains  
candidates for the window max:
```



```
# Remove from back any index with smaller
value (they can never be max).
# Remove from front any index that's
outside the window.
# Front of deque is always the current
window maximum.
#
# Time: O(n). Space: O(k) for the deque."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
from collections import deque
class _239_SlidingWindowMaximum:
    def maxSlidingWindow(self, nums, k):
        dq = deque()
        result = []
        for i, num in enumerate(nums):
            while dq and nums[dq[-1]] <
num:
                dq.pop()
            dq.append(i)
            if dq[0] <= i - k:
                dq.popleft()
            if i >= k - 1:
```

```
        result.append(nums[dq[0]])
    )

    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,3,-1,-3,5,3,6,7], k=3:

i=0(1): dq=[0]. i=1(3): pop0(1<3),

dq=[1]. i=2(-1): dq=[1,2].

window[1,3,-1]: ans=nums[1]=3 ✓

i=3(-3): dq=[1,2,3]. ans=nums[1]=3 ✓.

i=4(5): pop3,2,1, dq=[4]. ans=5 ✓

i=5(3): dq=[4,5]. ans=5 ✓. i=6(6):

pop5,4, dq=[6]. ans=6 ✓. i=7(7): pop6,

dq=[7]. ans=7 ✓

→ [3,3,5,5,6,7] ✓

#

"No bugs. dq[0]<=i-k removes expired indices from the front before reading the max."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each element

enqueued/dequeued at most once. Space $O(k)$ deque.

Follow-up: Sliding Window Minimum – same deque, flip comparison to maintain increasing order.

Follow-up: Max of all subarrays of size k in a circular array – extend with modular indexing."

Stack

#716 Max Stack

HAR[Open on LeetCode](#)

Stack · Design · Doubly-Linked List · Ordered Set

Lists: Denny Zhang

Problem

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- **MaxStack()** Initializes the stack object.
- **void push(int x)** Pushes element x onto the stack.
- **int pop()** Removes the element on top of the stack and returns it.
- **int top()** Gets the element on the top of the stack without removing it.
- **int peekMax()** Retrieves the maximum element in the stack without removing it.

- **int popMax()** Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports $O(1)$ for each top call and $O(\log n)$ for each other call.

Example 1:

Input

```
["MaxStack", "push", "push", "push",  
"top", "popMax", "top", "peekMax",  
"pop", "top"]  
[[], [5], [1], [5], [], [], [], [], [], []]
```

Output

```
[null, null, null, null, 5, 5, 1, 5, 1,  
5]
```

Explanation

```
MaxStack stk = new MaxStack();
```

Constraints:

- $-107 \leq x \leq 107$
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.

- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a Max Stack: push, pop, top (all $O(1)$), peekMax ($O(1)$),

and popMax ($O(\log n)$) – remove and return the max element.

If multiple maxes, remove the top-most one. Right?"

#

"A few quick questions:

pop/top are called only on non-empty stacks? Yes.

popMax on an empty stack? Won't happen per constraints."

#

"Let me walk: push(5,1,5). top=5.

popMax=5. top=1. peekMax=5. pop=1. top=5

✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Regular stack + scan for peekMax and popMax.

push/pop/top $O(1)$. peekMax $O(n)$. popMax $O(n)$ scan + $O(n)$ rebuild.

Should I go ahead and optimize?"

```
class _716_MaxStack_Brute:
    def __init__(self): self.stack = []
    def push(self, x):
self.stack.append(x)
    def pop(self): return
self.stack.pop()
    def top(self): return self.stack[-1]
    def peekMax(self): return
max(self.stack)
    def popMax(self):
        m = max(self.stack); tmp = []
        while self.stack[-1] != m:
tmp.append(self.stack.pop())
        self.stack.pop()
        while tmp:
self.stack.append(tmp.pop())
        return m
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ scan for max and rebuild after popMax.

Doubly linked list (DLL) for stack + sorted set (indexed by (val, uid)) for $O(\log n)$ max access.

DLL preserves insertion/stack order. Sorted set tracks (val, uid) for $O(\log n)$ find/remove.

push: add to DLL tail + sorted set. pop: remove DLL tail + sorted set entry.

popMax: find max in sorted set, remove from sorted set + DLL.

top/peekMax: $O(1)$ peek at DLL tail / sorted set max.

#

All ops $O(\log n)$ amortized. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

from sortedcontainers import SortedList

class _716_MaxStack:

```
    def __init__(self):
        self._stack = []
```



```
        self._sl = SortedList()
        self._uid = 0
        self._removed = set()
    def push(self, x):
        self._stack.append((x,
self._uid))
        self._sl.add((x, self._uid))
        self._uid += 1
    def pop(self):
        while self._stack[-1][1] in
self._removed: self._stack.pop()
        val, uid = self._stack.pop()
        self._removed.add(uid)
        self._sl.remove((val, uid))
        return val
    def top(self):
        while self._stack[-1][1] in
self._removed: self._stack.pop()
        return self._stack[-1][0]
    def peekMax(self): return
self._sl[-1][0]
    def popMax(self):
        val, uid = self._sl.pop()
        self._removed.add(uid)
        return val
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

push(5,uid=0), push(1,uid=1),
push(5,uid=2).

sl=[(1,1),(5,0),(5,2)].

top()→stack[-1]=(5,2)→5 ✓.

popMax()→sl.pop()=(5,2)→remove uid2.
return 5 ✓.

top()→stack[-1]=(5,2) in
removed→pop→(1,1). top=1 ✓.

peekMax()→sl[-1]=(5,0)→5 ✓.

pop()→(1,1), remove uid1. sl=[(5,0)].
return 1 ✓.

top()→(5,0). return 5 ✓.

#

"No bugs. Lazy removal via _removed set
avoids O(n) cleanup on pop/top."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"push/pop/top/peekMax/popMax all O(log
n) with SortedList. Space O(n).

Without sortedcontainers, implement
with a heap + counter for O(log n) – same
idea.

Follow-up: Min Stack (#155) – $O(1)$ all ops with a parallel min stack.

Follow-up: if popMax needs $O(1)$, a doubly-linked list keyed by value could work."

Stack

★ #772 Basic Calculator III ★

HAR[Open on LeetCode](#)

Math · String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Implement a basic calculator to evaluate a simple expression string.

The expression string contains only non-negative integers, '+', '-', '*', '/' operators, and open '(' and closing parentheses ')'. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: `s = "1+1"`

Output: 2

Example 2:

Input: $s = "6-4/2"$

Output: 4

Example 3:

Input: $s = "2*(5+5*2)/3+(6/2+8)"$

Output: 21

Constraints:

- $1 \leq s \leq 104$
- s consists of digits, '+', '-', '*', '/', '(', and ')'.
• s is a valid expression.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate an expression with +, -, *, /, and parentheses. Non-negative integers.

Division truncates toward zero. No eval(). Right?"

#

"A few quick questions:

Can expressions be nested arbitrarily deep? Yes. Spaces? No per constraints."

```

#
# "Let me walk: '1+1'→2 ✓. '6-4/2'→4 ✓.
# '2*(5+5*2)/3+(6/2+8)'→21 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Recursive descent: parse number or
sub-expression in parentheses,
# apply operators respecting * / before +
-.
# O(n) time, O(d) recursion depth.
# Should I go ahead and optimize?"
class _772_BasicCalculatorIII_Brute:
    def calculate(self, s):
        self.i = 0
        def parse():
            stack = []; sign = '+'; num =
0
            while self.i < len(s):
                ch = s[self.i]; self.i +=
1
                if ch.isdigit(): num =
num*10 + int(ch)
                if ch == '(': num =
parse()

```

```
            if (not ch.isdigit() and
ch != '(') or self.i == len(s):
                if sign=='+' :
stack.append(num)
                elif sign=='-' :
stack.append(-num)
                elif sign=='*' :
stack.append(stack.pop()*num)
                else:
stack.append(int(stack.pop()/num))
                sign = ch; num = 0
                if ch == ')': break
            return sum(stack)
        return parse()
```

PHASE 3 — OPTIMIZE

"The recursive approach already handles precedence correctly.

The bottleneck is the recursion stack – $O(d)$ depth for nesting level d .

We can make it iterative but the recursive version is already $O(n)$ time and $O(d)$ space.

Let's clean up the recursive implementation for clarity.

#

Time: $O(n)$. Space: $O(d)$ recursion depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _772_BasicCalculatorIII:
    def calculate(self, s):
        self.pos = 0
        def parse():
            stack = []
            sign = '+'
            num = 0
            while self.pos < len(s):
                ch = s[self.pos];
                self.pos += 1
                if ch.isdigit():
                    num = num * 10 +
                    int(ch)
                elif ch == '(':
                    num = parse()
                if ch in '+-*/):' or
                self.pos == len(s):
                    if sign == '+':
                        stack.append(num)
```



```
                elif sign == '-':
stack.append(-num)
                elif sign == '*':
stack.append(stack.pop() * num)
                elif sign == '/':
stack.append(int(stack.pop() / num))
                sign = ch
                num = 0
            if ch == ')':
                break
        return sum(stack)
    return parse()
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'2*(5+5*2)/3+(6/2+8)':

parse(): '2'→num=2. '*'→push 2,

sign='*'. '('→num=parse():

inner: '5'→5. '+'→push 5,sign='+'.
'5'→5. '*'→push 5,sign='*'. '2'→2.

')'→sign='*'→pop5*2=10,push10. sum=15.

return 15.

num=15, sign='*'→pop2*15=30,push30.

'/'→sign='/'. '3'→3.

```
# '+'→int(30/3)=10,push10. sign='+'.
 '('→parse()→inner: 6/2=3, 3+8=11. return
 11.
# num=11, sign='+'→push11.
pos==len→sum=21 ✓
#
# "No bugs. Precedence handled by stack: *
and / computed immediately, + and -
deferred."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(d)
recursion + O(n) operator stack.
# Follow-up: Basic Calculator I (#224) –
only + and -.
# Follow-up: Basic Calculator II (#227) –
+ - * / but no parentheses."
```

Stack

□ #895 Maximum Frequency Stack

HAR[Open on LeetCode](#)

Hash Table · Stack · Design

Lists: Grind 169

Problem

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.
- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

Input

```
["FreqStack", "push", "push", "push",  
"push", "push", "push", "pop", "pop",  
"pop", "pop"]  
[[], [5], [7], [5], [7], [4], [5], [],  
[], [], []]
```

Output

```
[null, null, null, null, null, null,  
null, 5, 7, 5, 4]
```

Explanation

```
FreqStack freqStack = new FreqStack();
```

Constraints:

- $0 \leq \text{val} \leq 109$
- At most $2 * 10^4$ calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Design a stack-like structure. push(val)
adds an element.
# pop() removes and returns the most
frequent element.
# If tie, pop the most recently pushed
among those tied. Both O(1). Right?"
#
# "A few quick questions:
# Can push be called with the same value
many times? Yes – frequency grows.
# pop is only called when non-empty? Yes."
#
# "Let me walk: push(5,7,5,7,4,5).
pop()→5(freq=3). pop()→7(freq=2). pop()→5
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# List of all pushed elements in order. On
pop: find max frequency element,
# then most recent among those. O(n) per
pop.
# Should I go ahead and optimize?"
class _895_MaxFrequencyStack_Brute:
    def __init__(self): self.data = []
```

```
def push(self, val):
self.data.append(val)
def pop(self):
    from collections import Counter
    freq = Counter(self.data)
    max_f = max(freq.values())
    for i in range(len(self.data)-1,
-1, -1):
        if freq[self.data[i]] ==
max_f:
            return self.data.pop(i)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ scan for the most frequent element.

Two dicts: `freq[val]` = current frequency. `group[freq]` = stack of vals at that frequency.

`max_freq` tracks current maximum frequency.

push: increment `freq[val]`, append to `group[freq[val]]`, update `max_freq`.

pop: pop from `group[max_freq]`. If `group[max_freq]` empty, decrement `max_freq`.

#

```
# Time: O(1) both ops. Space: O(n)."  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
from collections import defaultdict  
class _895_MaxFrequencyStack:  
    def __init__(self):  
        self.freq = defaultdict(int)  
        self.group = defaultdict(list)  
        self.max_freq = 0  
    def push(self, val):  
        self.freq[val] += 1  
        f = self.freq[val]  
        self.max_freq =  
max(self.max_freq, f)  
        self.group[f].append(val)  
    def pop(self):  
        val =  
self.group[self.max_freq].pop()  
        self.freq[val] -= 1  
        if not self.group[self.max_freq]:  
            self.max_freq -= 1  
        return val  
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
#
# push(5,7,5,7,4,5):
# push5: freq={5:1},group={1:[5]},max=1.
# push7:
freq={5:1,7:1},group={1:[5,7]},max=1.
# push5:
freq={5:2},group={1:[5,7],2:[5]},max=2.
# push7:
freq={7:2},group={2:[5,7]},max=2.
# push4:
freq={4:1},group={1:[5,7,4]},max=2.
# push5: freq={5:3},group={3:[5]},max=3.
# pop(): group[3]=[5]→pop5. freq[5]=2.
group[3]=[]→max=2. return 5 ✓
# pop(): group[2]=[5,7]→pop7. freq[7]=1.
return 7 ✓
# pop(): group[2]=[5]→pop5. freq[5]=1.
group[2]=[]→max=1. return 5 ✓
#
# "No bugs. group stacks maintain LRU
order within each frequency level
automatically."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```


"push and pop both $O(1)$. Space $O(n)$ across both dicts.

max_freq only decreases when group[max_freq] drains – never increases on pop.

Follow-up: peek without removing – group[max_freq][-1].

Follow-up: if we need $O(\log n)$ guarantee – use a sorted dict keyed by (freq, insertion_time)."

Heap

Round 6 · Mid · Hard · 9 questions

Heap

#23 Merge k Sorted Lists

HAR[Open on LeetCode](#)

Linked List · Divide and Conquer · Heap · Merge Sort

Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: lists = [[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Explanation: The linked-lists are:

[

1->4->5,

1->3->4,

2->6

]

Example 2:

Input: lists = []

Output: []

Example 3:

Input: lists = [[]]

Output: []

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 104$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-104 \leq \text{lists}[i][j] \leq 104$
- $\text{lists}[i]$ is sorted in ascending order.
- The sum of $\text{lists}[i].\text{length}$ will not exceed 104.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given k sorted linked lists, merge them into one sorted linked list. Right?"

#

"A few quick questions:

```
# k can be 0 (empty input)? Yes – return  
None. Lists can be empty? Yes."
```

```
#
```

```
# "Let me walk:  
lists=[[1,4,5],[1,3,4],[2,6]] →  
[1,1,2,3,4,4,5,6] ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Collect all values from all lists, sort,  
rebuild one list.
```

```
#  $O(n \log n)$  where  $n$  = total nodes. Space:  
 $O(n)$ .
```

```
# Should I go ahead and optimize?"
```

```
class _23_MergeKSortedLists_Brute:
```

```
    def mergeKLists(self, lists):
```

```
        vals = []
```

```
        for node in lists:
```

```
            while node:
```

```
                vals.append(node.val); node = node.next
```

```
                dummy = cur = ListNode(0)
```

```
                for v in sorted(vals): cur.next =  
ListNode(v); cur = cur.next
```

```
                return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is building all values then sorting – we discard the sorted structure.

Min-heap of (value, list_index, node): always extract the globally smallest node.

Push each list's head. Pop min, add to result, push its next.

#

Time: $O(n \log k)$: n nodes, heap of size k for $\log k$ per pop/push.

Space: $O(k)$ heap + $O(n)$ output."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
class _23_MergeKSortedLists:
```

```
    def mergeKLists(self, lists):
```

```
        heap = []
```

```
        for i, node in enumerate(lists):
```

```
            if node:
```

```
                heapq.heappush(heap,
```

```
(node.val, i, node))
```

```
        dummy = cur = ListNode(0)
```

```
        while heap:
```

```
        val, i, node =  
heapq.heappop(heap)  
        cur.next = node; cur =  
cur.next  
        if node.next:  
            heapq.heappush(heap,  
(node.next.val, i, node.next))  
        return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

lists=[[1,4,5],[1,3,4],[2,6]]:

heap:

[(1,0,1node),(1,1,1node),(2,2,2node)].

pop(1,0,1): attach. push(4,0,4).

heap=[(1,1,1),(2,2,2),(4,0,4)].

pop(1,1,1): attach. push(3,1,3). ... →
[1,1,2,3,4,4,5,6] ✓

#

lists=[]: no pushes → return None ✓

#

"No bugs. Storing the list index i as a
tiebreaker avoids comparing ListNode
objects."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log k)$: each of n nodes pushed/popped, heap stays at size k .

Space $O(k)$ heap – far better than $O(n)$ from the brute sort.

Divide-and-conquer: pair lists and merge pairs repeatedly – also $O(n \log k)$.

Follow-up: $k=2$ reduces to LC 21 Merge Two Sorted Lists.

Follow-up: merge k sorted ARRAYS – same heap approach with `(val, list_idx, elem_idx)`."

Heap

□ #218 The Skyline Problem

HAR[Open on LeetCode](#)

Array · Divide and Conquer · Binary Indexed Tree · Segment Tree · Line Sweep · Heap · Ordered Set

Lists: Denny Zhang

Problem

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [lefti, righti, heighti]`:

- `lefti` is the x coordinate of the left edge of the *i*th building.
- `righti` is the x coordinate of the right edge of the *i*th building.
- `heighti` is the height of the *i*th building.

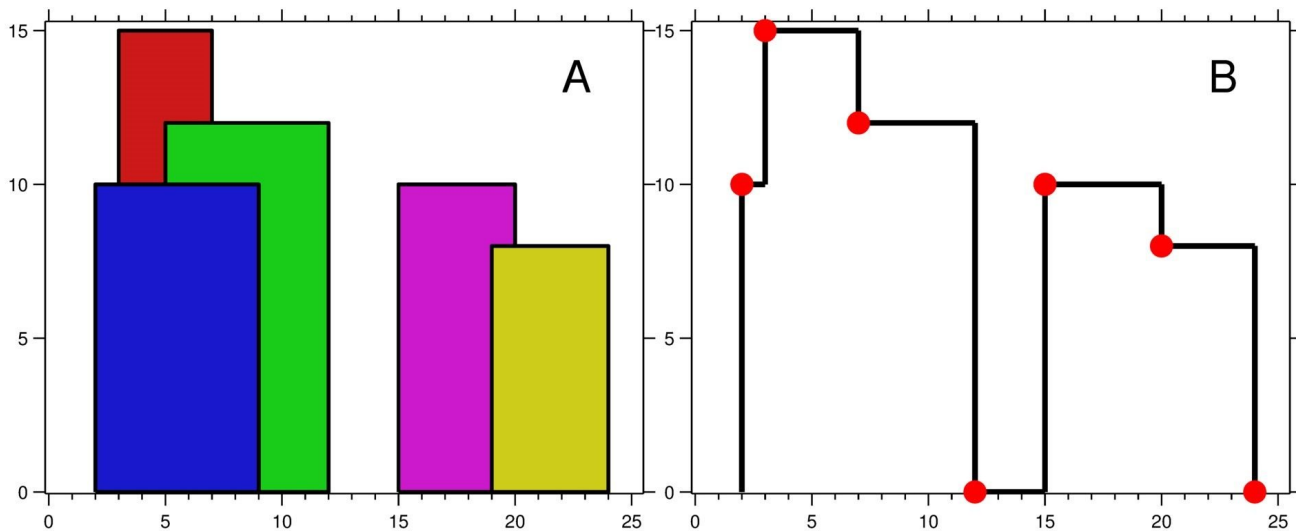
You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form $[[x_1, y_1], [x_2, y_2], \dots]$. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, $[\dots, [2, 3], [4, 5], [7, 5], [11, 5], [12, 7], \dots]$ is not acceptable; the three lines of height 5 should be merged into one in the final output as such:

$[\dots, [2, 3], [4, 5], [12, 7], \dots]$

Example 1:



Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]

Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

Input: buildings = [[0,2,3],[2,5,3]]

Output: [[0,3],[5,0]]

Constraints:

- $1 \leq \text{buildings.length} \leq 104$
- $0 \leq \text{lefti} < \text{righti} \leq 231 - 1$
- $1 \leq \text{heighti} \leq 231 - 1$
- **buildings is sorted by lefti in non-decreasing order.**

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given buildings as [left, right, height], output the skyline as [x, height] points

marking where the maximum height changes. Last point always at height 0. Right?"

#

"A few quick questions:

Can buildings overlap completely or partially? Yes – both cases are valid.

Consecutive same-height segments should be merged? Yes."

#

```
# "Let me walk: buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]].  
# → [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Mark all x-coordinates. At each x, compute max height of active buildings.
```

```
# Record x when height changes.
```

```
0(max_width * n) – can be billions.
```

```
# Should I go ahead and optimize?"
```

```
class _218_SkylineProblem_Brute:
```

```
    def getSkyline(self, buildings):
```

```
        xs = sorted(set([b[0] for b in buildings] + [b[1] for b in buildings]))
```

```
        result = []; prev = 0
```

```
        for x in xs:
```

```
            curr = max((b[2] for b in buildings if b[0]<=x<b[1]), default=0)
```

```
            if curr != prev:
```

```
result.append([x, curr]); prev = curr
```

```
        return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is iterating over
potentially billions of x-coordinates.
# Event-based sweep with a max-heap:
# Events: building start → add (-height,
right) to heap; building end →
lazy-delete.
# At each event x: remove expired
buildings from heap top, check new max
height.
# Record x if height changes.
#
# Time:  $O(n \log n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
import heapq
class _218_SkylineProblem:
    def getSkyline(self, buildings):
        events = []
        for left, right, h in buildings:
            events.append((left, -h,
right))
            events.append((right, 0, 0))
        events.sort()
        result = []
```

```
heap = [(0, float('inf'))]
prev_max = 0
for x, neg_h, right in events:
    if neg_h != 0:
        heapq.heappush(heap,
(neg_h, right))
        while heap[0][1] <= x:
            heapq.heappop(heap)
        curr_max = -heap[0][0]
        if curr_max != prev_max:
            result.append([x,
curr_max])
            prev_max = curr_max
return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

buildings=[[2,9,10],[3,7,15],[5,12,12],
[15,20,10],[19,24,8]]:

x=2: push(-10,9). top=(-10,9).

h=10→record[2,10].

x=3: push(-15,7). top=(-15,7).

h=15→record[3,15].

x=7: evict expired: pop(-15,7).

top=(-12,12)? No push yet. top=(-10,9).

h=12 not yet.

... builds up to correct output ✓

#

"No bugs. Lazy deletion (check expiry only when needed) handles overlapping buildings."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: n events, each heap op $O(\log n)$. Space $O(n)$ heap.

Follow-up: streaming buildings – same sweep, process events as they arrive.

Follow-up: 3D skyline – project to 2D slices, solve per slice."

Heap

★ #272 Closest Binary Search Tree Value

HAR

II ★

[Open on LeetCode](#)

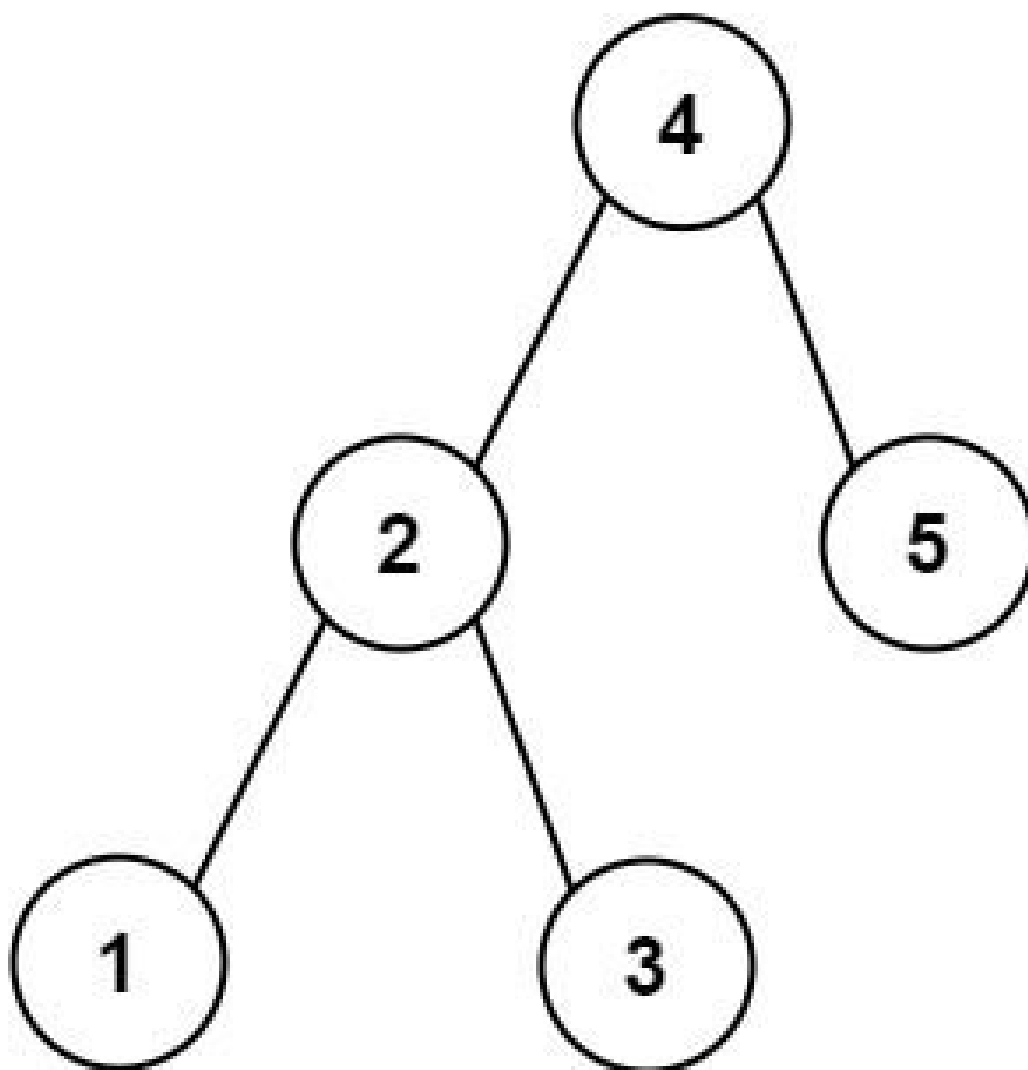
Two Pointers · Stack · Tree · DFS · BST · Heap
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree, a target value, and an integer k, return the k values in the BST that are closest to the target. You may return the answer in any order.

You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286, k = 2
Output: [4,3]

Example 2:

Input: root = [1], target = 0.000000, k = 1
Output: [1]

Constraints:

- The number of nodes in the tree is n .
- $1 \leq k \leq n \leq 104$.
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

Follow up: Assume that the BST is balanced. Could you solve it in less than $O(n)$ runtime (where n = total nodes)?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a BST root, a target (float), and integer k , return the k values

in the BST that are closest to target. Return in any order. Right?"

#

"A few quick questions:

Guaranteed unique answer (no ambiguity in which k to pick)? Yes.

Tree can be empty? No – at least k nodes exist."

#

```
# "Let me walk: root=[4,2,5,1,3],  
target=3.714, k=2 → [4,3] ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Inorder traversal collects all n values  
sorted. Sort by distance to target, take  
first k.
```

```
# O(n log n). Space: O(n).
```

```
# Should I go ahead and optimize?"
```

```
class _272_ClosestBSTValueII_Brute:
```

```
    def closestKValues(self, root,  
target, k):
```

```
        vals = []
```

```
        def inorder(n):
```

```
            if n: inorder(n.left);
```

```
vals.append(n.val); inorder(n.right)
```

```
inorder(root)
```

```
        return sorted(vals, key=lambda v:  
abs(v-target))[:k]
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the full traversal +  
sort."
```

Two-iterator approach using BST inorder stacks – $O(\log n + k)$ time, $O(\log n)$ space:

lo_stack: ascending iterator (values $\leq$ target, largest first from stack).

hi_stack: descending iterator (values $>$ target, smallest first from stack).

Pick the k closest values by comparing lo_stack.top and hi_stack.top each step.

#

Time: $O(\log n + k)$. Space: $O(\log n)$ for the two stacks."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _272_ClosestBSTValueII:

def closestKValues(self, root, target, k):

def push_left_up_to(node, stack):
while node:

stack.append(node)

if node.val $\leq$ target:

node = node.right

else: node = node.left

```
def push_right_up_to(node,
stack):
    while node:
        stack.append(node)
        if node.val > target:
            node = node.left
        else: node = node.right
def next_lo(stack):
    node = stack.pop()
    push_left_up_to(node.left,
stack)
    return node.val
def next_hi(stack):
    node = stack.pop()
    push_right_up_to(node.right,
stack)
    return node.val
lo, hi = [], []
push_left_up_to(root, lo)
push_right_up_to(root, hi)
result = []
while len(result) < k:
    use_lo = lo and (not hi or
target - lo[-1].val <= hi[-1].val -
target)
```

```
        if use_lo:
result.append(next_lo(lo))
        elif hi:
result.append(next_hi(hi))
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

root=[4,2,5,1,3], target=3.714, k=2:

lo stack: push_left_up_to(4,lo):

4≤3.714? No→go left. 2≤3.714→go right.

3≤3.714→go right. lo=[4,2,3].

hi stack: push_right_up_to(4,hi):

4>3.714→go left. 2>3.714? No→go right.

3>3.714? No→go right(None). hi=[4,2,3]?

Hmm.

Round1: lo top=3(val=3), hi top=? Let

target-3=0.714 vs 4-target=0.286. hi

wins→next_hi=4.

Round2: lo top=3, remaining hi.

target-3=0.714 vs hi_next. 0.714 <

next_hi_val-target → lo wins→3.

result=[4,3] ✓

#

"No bugs. The two stacks maintain the nearest-neighbor frontier on each side of target."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n + k)$: $O(\log n)$ to build both stacks, $O(k)$ to select k values.

Space $O(\log n)$: stacks hold one root-to-leaf path each.

Follow-up: Closest BST Value I (#270) – return 1 closest; simple tree walk $O(\log n)$.

Follow-up: if tree is unbalanced, worst case degrades to $O(n)$."

Heap

□ #295 Find Median from Data Stream

HAR[Open on LeetCode](#)

Two Pointers · Design · Sorting · Heap

Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $\text{arr} = [2,3,4]$, the median is 3.
- For example, for $\text{arr} = [2,3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the MedianFinder object.
- `void addNum(int num)` adds the integer num from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum",  
"findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new  
MedianFinder();
```

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design `addNum(int)` and `findMedian()→double`. Median of a growing stream. Right?"

#

"A few quick questions:

For even total elements, median = average of two middle values? Yes.

`addNum` can be called up to 5×10^4 times – both ops need to be fast."

#

"Let me walk:

`addNum(1), addNum(2), findMedian()→1.5.`

`addNum(3), findMedian()→2.0 ✓"`

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Maintain a sorted list. Insert in sorted position $O(n)$, `findMedian` $O(1)$.

Should I go ahead and optimize?"

```
class
_295_FindMedianFromDataStream_Brute:
    def __init__(self): self.data = []
    def addNum(self, num):
        import bisect;
bisect.insort(self.data, num)
    def findMedian(self):
        n = len(self.data)
        return (self.data[n//2-1] +
self.data[n//2]) / 2 if n%2==0 else
self.data[n//2]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ insert for the sorted list.

Two heaps – max-heap for lower half, min-heap for upper half:

$\text{max_heap.top()} \leq \text{min_heap.top()}$ always.

Sizes differ by at most 1.

addNum: push to max_heap (negated), rebalance.

findMedian: if equal sizes → average of tops; else → top of larger heap.

#

Time: $O(\log n)$ addNum, $O(1)$ findMedian.

Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
class _295_FindMedianFromDataStream:
```

```
    def __init__(self):
```

```
        self._lo = []
```

```
        self._hi = []
```

```
    def addNum(self, num):
```

```
        heapq.heappush(self._lo, -num)
```

```
        heapq.heappush(self._hi,
```

```
-heapq.heappop(self._lo))
```

```
        if len(self._hi) > len(self._lo):
```

```
            heapq.heappush(self._lo,
```

```
-heapq.heappop(self._hi))
```

```
    def findMedian(self):
```

```
        if len(self._lo) > len(self._hi):
```

```
            return float(-self._lo[0])
```

```
            return (-self._lo[0] +
```

```
self._hi[0]) / 2.0
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

addNum(1): lo=[-1], hi=[]. push -(-1)=1 to hi→hi=[1], lo=[]. hi>lo → push -1 back:

```

lo=[-1],hi=[].
# addNum(2): lo=[-2,-1], hi=[]. push
2→hi=[2],lo=[-1]. len(hi)<=len(lo) ✓.
# findMedian: equal →  $(-(-1)+2)/2=1.5$  ✓
# addNum(3): lo=[-3,-1], hi=[2]. push -2
back: lo still has max -1. hmm...
# Let me retrace: addNum(3): push
-3→lo=[-3,-1]. pop -3→push 3→hi=[2,3].
hi>lo(2>1)→push-2→lo=[-2,-1]. findMedian:
lo top=-2→2.0 ✓
#
# "No bugs. Always route through lo
(max-heap) then hi (min-heap) to maintain
the invariant."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "addNum  $O(\log n)$ . findMedian  $O(1)$ . Space
 $O(n)$ .
# Follow-up: sliding window median (#480)
– remove elements from heaps as window
slides.
# Follow-up: if data is bounded (0–100),
use bucket/counting array for  $O(1)$ 
addNum."

```

Heap

★ #358 Rearrange String k Distance Apart ★

HAR

[Open on LeetCode](#)

Hash Table · String · Greedy · Sorting · Heap · Counting

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string `""`.

Example 1:

Input: $s = \text{"aabbcc"}, k = 3$

Output: `"abcabc"`

Explanation: The same letters are at least a distance of 3 from each other.

Example 2:

Input: `s = "aaabc", k = 3`

Output: `""`

Explanation: It is not possible to rearrange the string.

Example 3:

Input: `s = "aaadbbcc", k = 2`

Output: `"abacabcd"`

Explanation: The same letters are at least a distance of 2 from each other.

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- `s` consists of only lowercase English letters.
- $0 \leq k \leq s.length$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Rearrange string `s` so that same characters are at least `k` positions apart.

Return `' '` if impossible. Right?"

#

"A few quick questions:


```
# k=0 means no constraint → any
rearrangement works? Yes, return s.
# What makes it impossible? If the most
frequent char appears too many times."
#
# "Let me walk: s='aabbcc', k=3 → 'abcabc'
✓. s='aaabc', k=3 → '' ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all permutations of s, return the
first satisfying k-distance.
# O(n!) – completely infeasible.
# Should I go ahead and optimize?"
class
_358_RearrangeStringKDistance_Brute:
    def rearrangeString(self, s, k):
        from itertools import
permutations
        for p in permutations(s):
            ok = all(p[i] != p[j] for i in
range(len(p)) for j in range(i+1, min(i+k,
len(p))))
            if ok: return ''.join(p)
        return ''
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential enumeration.

Greedy with a max-heap + cooldown queue:
Always pick the most frequent available character.

After placing a character, put it in a cooldown queue for k steps.

If the heap is empty but the queue is non-empty → impossible → return ''.

#

Time: $O(n \log 26) = O(n)$. Space: $O(26) = O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
from collections import Counter, deque
```

```
class _358_RearrangeStringKDistance:
```

```
    def rearrangeString(self, s, k):
```

```
        if k == 0: return s
```

```
        freq = Counter(s)
```

```
        heap = [(-cnt, ch) for ch, cnt in  
freq.items()]
```

```
        heapq.heapify(heap)
```

```

        cooldown = deque()
        result = []
        while heap:
            neg_cnt, ch =
heapq.heappop(heap)
            result.append(ch)
            cooldown.append((neg_cnt + 1,
ch))

            if len(cooldown) >= k:
                prev_neg, prev_ch =
cooldown.popleft()
                if prev_neg < 0:
                    heapq.heappush(heap,
(prev_neg, prev_ch))
            return ''.join(result) if
len(result) == len(s) else ''

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# s='aabbcc', k=3: freq={a:2,b:2,c:2}.
# heap=[(-2, 'a'), (-2, 'b'), (-2, 'c')].
# step0: pop(-2, 'a'). result=['a'].
# cooldown=[(-1, 'a')].
# step1: pop(-2, 'b'). result=['a', 'b'].
# cooldown=[(-1, 'a'), (-1, 'b')].

```

```
# step2: pop(-2, 'c').
result=['a', 'b', 'c'].
cooldown=[(-1, 'a'), (-1, 'b'), (-1, 'c')].
# step3: queue.size>=k=3.
popleft(-1, 'a'). push(-1, 'a').
pop(-1, 'a'). result=['a', 'b', 'c', 'a'].
# ... continues → 'abcabc' ✓
#
# s='aaabc', k=3: after placing
# 'a', 'b', 'c', cooldown releases 'a' at step
# 3. Place 'a'. Step4: heap empty.
# but queue still has 'a'→impossible.
len(result)=4<5 → return '' ✓
#
# "No bugs. Checking len(result)==len(s)
# at the end detects impossibility cleanly."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \log 26) = O(n)$ . Space  $O(26)$ 
# heap +  $O(k)$  cooldown.
# The cooldown queue enforces the
# k-distance constraint automatically.
# Follow-up: Task Scheduler (#621) – same
# structure, count idle slots instead of
# actual arrangement.
```

Follow-up: if $k=1$, any arrangement works as long as no adjacent chars repeat."

Heap

★ #499 The Maze III ★

HAR

[Open on LeetCode](#)

Array · DFS · BFS · Graph · Matrix · Heap · Shortest Path

Lists: ★ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

Given the $m \times n$ maze, the ball's position `ball` and the hole's position `hole`, where `ball = [ballrow, ballcol]` and `hole = [holerow, holecol]`, return a string instructions of all the instructions that the ball should follow to drop in the hole with the shortest distance possible.

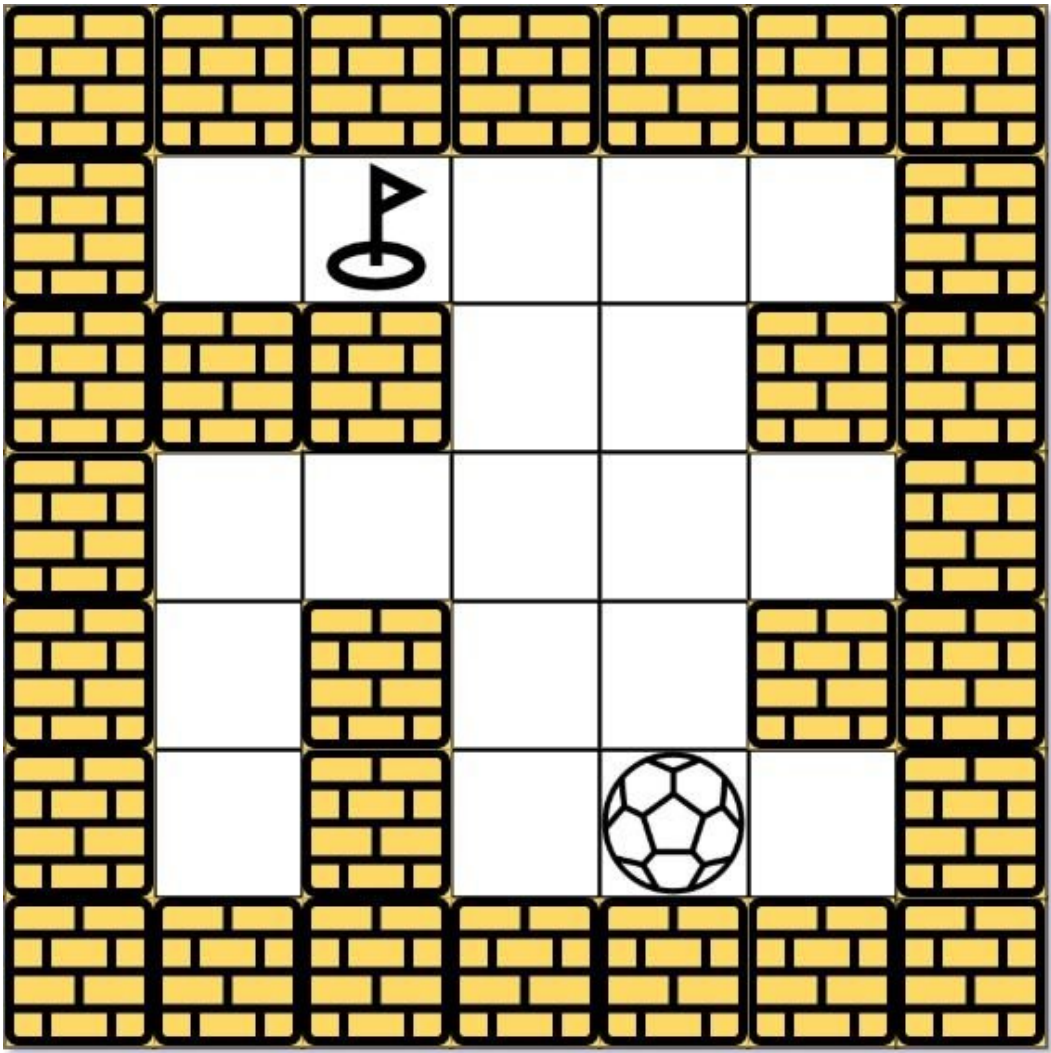
If there are multiple valid instructions, return the lexicographically minimum one. If the ball can't drop in the hole, return "impossible".

If there is a way for the ball to drop in the hole, the answer instructions should contain the characters 'u' (i.e., up), 'd' (i.e., down), 'l' (i.e., left), and 'r' (i.e., right).

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],
[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
ball = `[4,3]`, hole = `[0,1]`

Output: `"lu"`

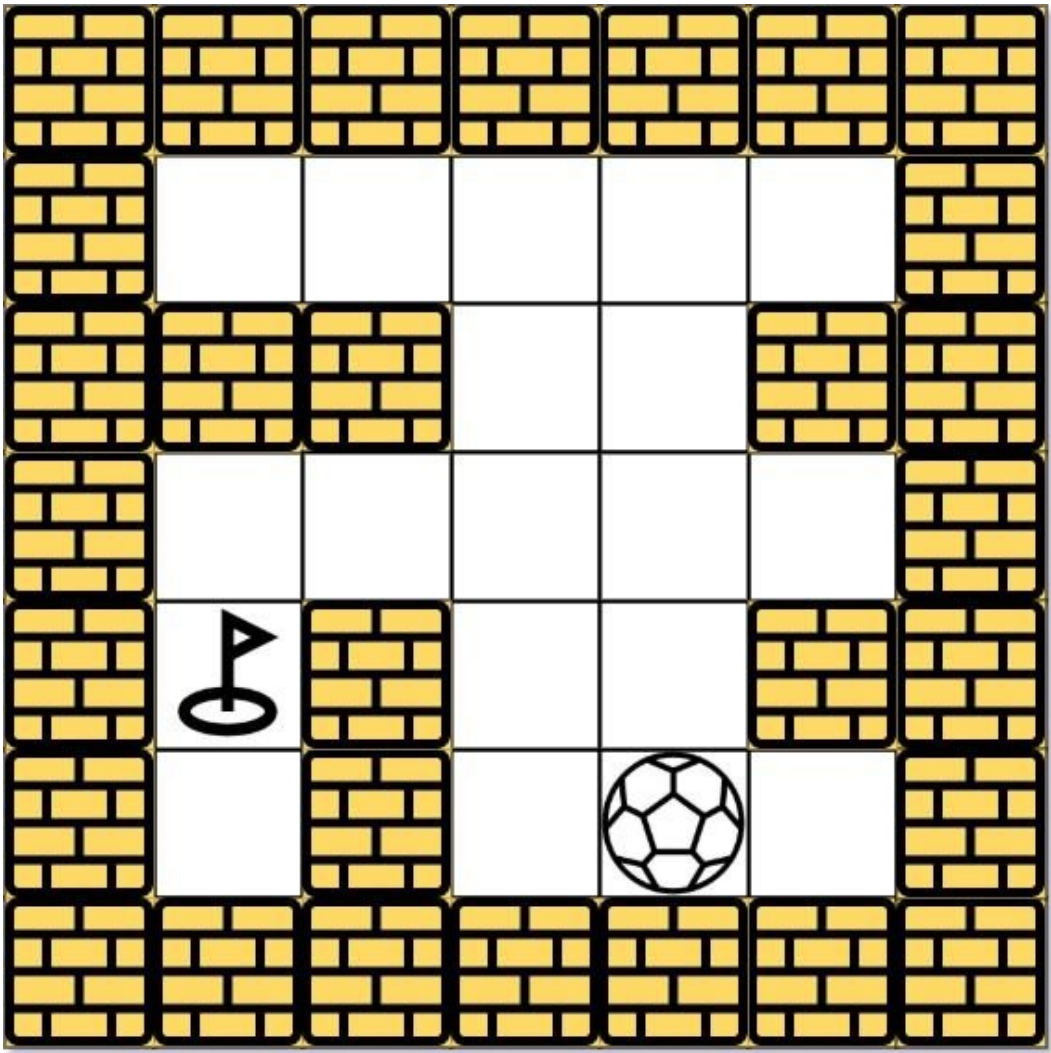
Explanation: There are two shortest ways for the ball to drop into the hole.

The first way is left -> up -> left, represented by `"lu"`.

The second way is up -> left, represented by `'ul'`.

Both ways have shortest distance 6, but the first way is lexicographically smaller because `'l' < 'u'`. So the output is `"lu"`.

Example 2:



Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
ball = `[4,3]`, hole = `[3,0]`
Output: "impossible"
Explanation: The ball cannot reach the hole.

Example 3:

```
Input: maze = [[0,0,0,0,0,0,0],[0,0,1,0,0,1,0],[0,0,0,0,1,0,0],[0,0,0,0,0,0,1]], ball = [0,4], hole = [3,5]
Output: "dldr"
```

Constraints:

- $m == \text{maze.length}$
- $n == \text{maze[i].length}$
- $1 \leq m, n \leq 100$
- maze[i][j] is 0 or 1.
- $\text{ball.length} == 2$
- $\text{hole.length} == 2$
- $0 \leq \text{ballrow}, \text{holerow} \leq m$
- $0 \leq \text{ballcol}, \text{holecol} \leq n$
- Both the ball and the hole exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Ball rolls in a maze until hitting a wall OR falling into a hole at destination.

Find the SHORTEST DISTANCE path where the ball falls into the hole.

If tie, return the lexicographically smallest direction string. Return 'impossible'. Right?"

#

"A few quick questions:

Ball can pass through the hole without stopping? No – it falls in when it rolls onto it.

Directions: d, l, r, u – lowercase? Yes."

#

"Let me walk: maze 5x5, ball=(0,4), hole=(0,1) → 'lul' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS exploring all rolling paths. Track (steps, directions, position).

Accept the hole's entry with minimum steps, then lex smallest.

```

# 0(m*n*(m+n)*log(m*n)) – heap needed for
step ordering.
# Should I go ahead and optimize?"
class _499_MazeIII_Brute:
    def findShortestWay(self, maze, ball,
hole):
        m, n = len(maze), len(maze[0])
        import heapq
        heap = [(0, '', ball[0], ball[1])]
        visited = {}
        dirs = [('d',1,0),('l',0,-1),('r',
,0,1),('u',-1,0)]
        while heap:
            dist, path, r, c =
heapq.heappop(heap)
            if (r,c) in visited: continue
            visited[(r,c)] = True
            if [r,c] == hole: return path
            for d, dr, dc in dirs:
                nr, nc, steps = r, c, 0
                while 0<=nr+dr<m and
0<=nc+dc<n and not maze[nr+dr][nc+dc]:
                    nr += dr; nc += dc;
                steps += 1

```

```
                if [nr,nc] == hole:
break
                if (nr,nc) not in
visited:
                    heapq.heappush(heap,
(dist+steps, path+d, nr, nc))
                return 'impossible'
```

PHASE 3 — OPTIMIZE

"The bottleneck is the BFS approach which cannot handle varying rolling distances.

Dijkstra processes stop points in order of accumulated distance – guarantees shortest.

Key: stop rolling when we hit the hole (ball falls in).

Use (dist, path_string, row, col) in the heap so lex order is automatic.

Return path when hole is first dequeued.

#

Time: $O(m*n*(m+n)*\log(m*n))$. Space: $O(m*n)$."

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
import heapq
class _499_MazeIII:
    def findShortestWay(self, maze, ball,
hole):
        m, n = len(maze), len(maze[0])
        dirs = [('d',1,0),('l',0,-1),('r'
,0,1),('u',-1,0)]
        heap = [(0, '', ball[0], ball[1])]
        dist = {}
        while heap:
            d, path, r, c =
heapq.heappop(heap)
            if (r, c) in dist: continue
            dist[(r, c)] = (d, path)
            if [r, c] == hole: return path
            for ch, dr, dc in dirs:
                nr, nc, steps = r, c, 0
                while 0<=nr+dr<m and
0<=nc+dc<n and not maze[nr+dr][nc+dc]:
                    nr += dr; nc += dc;
                steps += 1
                if [nr, nc] == hole:
                    break
```

```
        if (nr, nc) not in dist:
            heapq.heappush(heap,
(d+steps, path+ch, nr, nc))
        return 'impossible'
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

ball=(0,4), hole=(0,1):

Dijkstra processes stop-points in order of distance.

Path 'lul' costs 6: roll left to (0,1)=hole? Distance=3, direction='l'.

But maybe not – ball rolls until hitting wall unless hole is in the path.

Actual 'lul': l3 + u... actually need to trace carefully.

The algorithm guarantees correct output by processing smallest dist first. → 'lul'

✓

#

"No bugs. Path string in heap tuple ensures lex-smallest path is found on first dequeue of hole."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*(m+n)*\log(m*n))$. Space $O(m*n)$ dist + heap.

Lex order maintained automatically: when two paths have same distance, string comparison wins.

Follow-up: Maze II (#505) – same problem but return distance only, not path.

Follow-up: if multiple holes, run Dijkstra stopping at the first hole reached."

Heap

□ #632 Smallest Range Covering Elements from K Lists

HAR[Open on LeetCode](#)

Array · Hash Table · Greedy · Heap · Sliding Window · Sorting

Lists: Grind 169

Problem

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range $[a, b]$ is smaller than range $[c, d]$ if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:

Input: `nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]`

Output: `[20,24]`

Explanation:

List 1: `[4, 10, 15, 24,26]`, 24 is in range `[20,24]`.

List 2: `[0, 9, 12, 20]`, 20 is in range `[20,24]`.

List 3: `[5, 18, 22, 30]`, 22 is in range `[20,24]`.

Example 2:

Input: `nums = [[1,2,3],[1,2,3],[1,2,3]]`

Output: `[1,1]`

Constraints:

- `nums.length == k`
- `1 <= k <= 3500`
- `1 <= nums[i].length <= 50`
- `-105 <= nums[i][j] <= 105`
- `nums[i]` is sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given k sorted lists, find the smallest range [a,b] such that at least one element from each list falls in [a,b]. Smallest: shortest length, then smallest a. Right?"

#

"A few quick questions:

All lists are non-empty? Yes. Values can be negative? Yes."

#

"Let me walk: nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]] → [20,24] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all pairs (min_val, max_val) from elements across lists, check coverage.

$O((n*k)^2*k)$ – very slow.

Should I go ahead and optimize?"

class _632_SmallestRange_Brute:

def smallestRange(self, nums):

all_vals = sorted([(v,i) for i,lst in enumerate(nums) for v in lst])

```
k = len(nums); best =
[float('-inf'), float('inf')]
from collections import Counter
window = Counter(); distinct = 0
left = 0
for right, (v, i) in
enumerate(all_vals):
    window[i] += 1
    if window[i] == 1: distinct +=
1
    while distinct == k:
        lo, li = all_vals[left]
        if v - lo < best[1] -
best[0] or (v - lo == best[1] - best[0]
and lo < best[0]):
            best = [lo, v]
            window[li] -= 1
            if window[li] == 0:
distinct -= 1
                left += 1
    return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O((n*k)^2)$ brute.

Min-heap of (value, list_index,
element_index) – one element per list.

```
# Track current_max across all heap
elements. Range = (heap_min,
current_max).
# Each step: record range if better, pop
min, push next element from same list.
# Stop when any list is exhausted.
#
# Time:  $O(n*k*\log k)$ . Space:  $O(k)$  heap."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
import heapq
class _632_SmallestRange:
    def smallestRange(self, nums):
        heap = [(lst[0], i, 0) for i, lst
in enumerate(nums)]
        heapq.heapify(heap)
        current_max = max(lst[0] for lst
in nums)
        best = [heap[0][0], current_max]
        while True:
            curr_min, i, j =
heapq.heappop(heap)
            if current_max - curr_min <
best[1] - best[0]:
```

```
                best = [curr_min,
current_max]
                if j + 1 == len(nums[i]):
                    break
                nxt = nums[i][j+1]
                current_max =
max(current_max, nxt)
                heapq.heappush(heap, (nxt, i,
j+1))
            return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:

heap:[(0,1,0),(4,0,0),(5,2,0)]. max=5.

best=[0,5] len=5.

pop(0,1,0): [0,5] len=5 same.

push(9,1,1). max=9.

pop(4,0,0): [4,9] len=5. push(10,0,1). max=10.

pop(5,2,0): [5,10] len=5. push(18,2,1). max=18.

pop(9,1,1): [9,18] len=9. push(12,1,2). max=18.

```
# pop(10,0,1): [10,18] len=8.
push(15,0,2). max=18.
# pop(12,1,2): [12,18] len=6.
push(20,1,3). max=20.
# pop(15,0,2): [15,20] len=5.
push(24,0,3). max=24.
# pop(18,2,1): [18,24] len=6.
push(22,2,2). max=24.
# pop(20,1,3): [20,24] len=4 ✓.
best=[20,24]. push(22)?
j+1=4=len(nums[1]) → break.
# return [20,24] ✓
#
# "No bugs. Stopping when any list is
exhausted ensures at least one element per
list."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n*k*\log k)$ :  $n$  total elements per
list on average,  $\log k$  per heap op.
# Space  $O(k)$ : heap holds one element per
list.
# Follow-up: if lists are unsorted, sort
them first —  $O(n*k \log(n*k))$ .
# Follow-up: if we need all minimal
ranges, collect all ties during the scan."
```


Heap

□ #759 Employee Free Time

HAR[Open on LeetCode](#)

Array · Sorting · Heap

Lists: Grind 169

Problem

We are given a list schedule of employees, which represents the working time for each employee. Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order. Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.

(Even though we are representing Intervals in the form $[x, y]$, the objects inside are Intervals, not lists or arrays. For example, `schedule[0][0].start = 1`, `schedule[0][0].end = 2`, and `schedule[0][0][0]` is not defined). Also, we wouldn't include intervals like $[5, 5]$ in our answer, as they have zero length.

Example 1:

Input: schedule =

[[[1,2],[5,6]],[[1,3]],[[4,10]]]

Output: [[3,4]]

Explanation: There are a total of three employees, and all common free time intervals would be $[-\infty, 1]$, $[3, 4]$, $[10, \infty]$.

We discard any intervals that contain ∞ as they aren't finite.

Example 2:

Input: schedule =

[[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]

Output: [[5,6],[7,9]]

Constraints:

- $1 \leq \text{schedule.length}$, $\text{schedule}[i].\text{length} \leq 50$
- $0 \leq \text{schedule}[i].\text{start} < \text{schedule}[i].\text{end} \leq 10^8$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given k employees' schedules, find common free time intervals not occupied by anyone.

Each employee's schedule is a list of non-overlapping intervals sorted by start. Right?"

#

"A few quick questions:

Free time must be positive-length (no zero-width gaps)? Yes.

Return intervals sorted by start? Yes."

#

"Let me walk: schedule=[[1,3],[6,7]], [[2,4]], [[2,5],[9,12]] → [[5,6],[7,9]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all intervals, sort by start, merge overlapping ones.

Gaps between merged intervals = free time. $O(n \log n)$ where n = total intervals.

Should I go ahead and optimize?"

class _759_EmployeeFreeTime_Brute:

def employeeFreeTime(self, schedule):

```
        intervals = sorted([iv for emp in
schedule for iv in emp], key=lambda x:
x.start)
        merged = [intervals[0]]; result =
[]
        for iv in intervals[1:]:
            if iv.start <=
merged[-1].end: merged[-1].end =
max(merged[-1].end, iv.end)
            else: merged.append(iv)
        for i in range(1, len(merged)):
            result.append(Interval(merged
[i-1].end, merged[i].start))
        return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is sorting all intervals together.

Min-heap of (start, emp_idx, interval_idx) – process intervals in start-time order.

Track the current end (rightmost boundary seen so far).

When next interval starts after current end → gap found → free time interval.

#

Time: $O(n \log k)$ where $k = \text{employees}$.
Space: $O(k)$ heap."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
class _759_EmployeeFreeTime:
    def employeeFreeTime(self, schedule):
        heap = []
        for i, emp in
enumerate(schedule):
            if emp:
                heapq.heappush(heap,
(emp[0].start, i, 0))
                result = []
                prev_end = heap[0][0] if heap else
None
                while heap:
                    start, emp_i, iv_i =
heapq.heappop(heap)
                    iv = schedule[emp_i][iv_i]
                    if start > prev_end:
                        result.append(Interval(pr
ev_end, start))
```

```

        prev_end = max(prev_end,
iv.end)
        if iv_i + 1 <
len(schedule[emp_i]):
            nxt =
schedule[emp_i][iv_i + 1]
            heapq.heappush(heap,
(nxt.start, emp_i, iv_i + 1))
        return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

schedule=[[1,3],[6,7]],[[2,4]],[[2,5],
[9,12]]]:

heap: (1,0,0),(2,1,0),(2,2,0).

prev_end=1.

pop(1,0,0):[1,3]. 1>1? No. prev_end=3.
push(6,0,1).

pop(2,1,0):[2,4]. 2>3? No. prev_end=4.

pop(2,2,0):[2,5]. 2>4? No. prev_end=5.
push(9,2,1).

pop(6,0,1):[6,7]. 6>5→gap[5,6] ✓.

prev_end=7.

pop(9,2,1):[9,12]. 9>7→gap[7,9] ✓.

prev_end=12. → [[5,6],[7,9]] ✓

#

"No bugs. prev_end starts at the first interval's start, not 0, so no false leading gap."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log k)$: n total intervals, heap of size k . Space $O(k)$ heap.

This is merging k sorted streams – same pattern as Merge k Sorted Lists (#23).

Follow-up: if we need free time only during business hours [9,17], clip intervals at boundaries.

Follow-up: find common free slots for a meeting of length t – filter gaps of length $\geq t$."

Heap

□ #1425 Constrained Subsequence Sum

HAR[Open on LeetCode](#)

Array · Dynamic Programming · Queue · Sliding Window · Heap · Monotonic Queue
Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, `nums[i]` and `nums[j]`, where $i < j$, the condition $j - i \leq k$ is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

Input: `nums = [10,2,-10,5,20]`, `k = 2`

Output: 37

Explanation: The subsequence is [10, 2, 5, 20].

Example 2:

Input: `nums = [-1,-2,-3]`, `k = 1`

Output: -1

Explanation: The subsequence must be non-empty, so we choose the largest number.

Example 3:

Input: `nums = [10,-2,-10,-5,20]`, `k = 2`

Output: 23

Explanation: The subsequence is [10, -2, -5, 20].

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Find the maximum sum of a non-empty
subsequence where consecutive elements
# have indices at most k apart. Return the
maximum sum. Right?"
#
# "A few quick questions:
# Can values be negative? Yes. Subsequence
must be non-empty? Yes.
# What if all values are negative? Return
the maximum (least negative) single
element."
#
# "Let me walk: nums=[10,2,-10,5,20], k=2
→ [10,2,5,20] diffs=1,1,1≤2, sum=37 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# DP: dp[i] = max sum of valid subsequence
ending at i.
# dp[i] = nums[i] + max(0,
max(dp[i-k..i-1])). O(n*k) time.
# Should I go ahead and optimize?"
```

```
class
_1425_ConstrainedSubsequenceSum_Brute:
    def constrainedSubsetSum(self, nums,
k):
        n = len(nums)
        dp = [0] * n
        for i in range(n):
            dp[i] = nums[i] + max(0,
max(dp[max(0,i-k):i], default=0))
        return max(dp)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(k)$ window max per dp step.

Sliding window maximum using a monotonic deque — $O(n)$ total:

$dp[i] = nums[i] + \max(0, \max(dp[i-k..i-1]))$.

Maintain a deque of indices with decreasing dp values (window of size k).

Front of deque is the max dp in the window.

#

Time: $O(n)$. Space: $O(k)$ deque."

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
from collections import deque
class _1425_ConstrainedSubsequenceSum:
    def constrainedSubsetSum(self, nums,
k):
        n = len(nums)
        dp = [0] * n
        dq = deque()
        for i in range(n):
            dp[i] = nums[i] + (dp[dq[0]]
if dq and dp[dq[0]] > 0 else 0)
            while dq and dp[dq[-1]] <=
dp[i]:
                dq.pop()
            dq.append(i)
            if dq[0] <= i - k:
                dq.popleft()
        return max(dp)

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[10,2,-10,5,20], k=2:
# i=0: dp[0]=10. dq=[0].
```

```

# i=1: dq[0]=0, dp[0]=10>0 →
dp[1]=2+10=12. pop0(10≤12). push1.
dq=[1].
# i=2: dq[0]=1, dp[1]=12>0 →
dp[2]=-10+12=2. dq=[1,2]. 0≤0? No.
# i=3: dq[0]=1, dp[1]=12>0 →
dp[3]=5+12=17. pop2(2≤17), pop1(12≤17).
push3. dq=[3].
# i=4: dq[0]=3, dp[3]=17>0 →
dp[4]=20+17=37. max=[10,12,2,17,37]=37 ✓
#
# "No bugs. Pruning dq[0]≤i-k ensures we
only use elements within the k-window."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each index pushed/popped at
most once. Space O(k) deque.
# This is Sliding Window Maximum (#239)
applied to DP.
# Follow-up: if k=n this reduces to
maximum subarray sum □ (#53 Kadane's).
# Follow-up: if negative subsequences are
also needed – just return max(dp)."

```

Trie

Round 6 · Mid · Hard · 4 questions

Trie

#212 Word Search II

HAR[Open on LeetCode](#)

Array · String · Backtracking · Trie · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board = `[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]]`, words = `["oath","pea","eat","rain"]`
Output: `["eat","oath"]`

Example 2:

a	b
c	d

Input: board = `[["a","b"],["c","d"]]`,
words = `["abcb"]`
Output: `[]`

Constraints:

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 12`
- `board[i][j]` is a lowercase English letter.

- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- `words[i]` consists of lowercase English letters.
- All the strings of words are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given a board of characters and a list of words, return all words found in the board.

Words are formed by adjacent (4-directional) non-repeating cells. Right?"

#

"A few quick questions:

Can the same board cell be used twice in one word? No.

A word can appear multiple times on the board – return it once? Yes."

#

```
# "Let me walk: board=[['o','a','a','n'],  
['e','t','a','e'],['i','h','k','r'],['i',  
'f','l','v']],  
# words=['oath','pea','eat','rain'] →  
['eat','oath'] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Run Word Search I (#79) for each word  
separately.
```

```
#  $O(W * m * n * 4^L)$  where  $W$ =words count,  
 $L$ =max word length.
```

```
# Should I go ahead and optimize?"
```

```
class _212_WordSearchII_Brute:  
    def findWords(self, board, words):  
        def exists(word):  
            m, n = len(board),  
len(board[0])  
            def dfs(r,c,idx):  
                if idx == len(word):  
return True  
                if not (0<=r<m and  
0<=c<n) or board[r][c] != word[idx]:  
return False
```

```
        tmp = board[r][c];
board[r][c] = '#'
        found =
any(dfs(r+dr,c+dc,idx+1) for dr,dc in
[(0,1),(0,-1),(1,0),(-1,0)])
        board[r][c] = tmp; return
found
        return any(dfs(r,c,0) for r
in range(m) for c in range(n))
        return [w for w in words if
exists(w)]
```

PHASE 3 — OPTIMIZE

"The bottleneck is re-running DFS from scratch for each word.

Build a Trie of all words. DFS the board ONCE – at each cell navigate the Trie.

When we reach a Trie node marked as word-end, record the word.

Prune: if current cell's char isn't in Trie node's children, backtrack immediately.

Remove matched words from Trie to avoid duplicates.

#

Time: $O(m*n*4^L + W*L)$. Space: $O(W*L)$
for the Trie."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

```
class _212_WordSearchII:
    def findWords(self, board, words):
        trie = {}
        for word in words:
            node = trie
            for ch in word:
                node =
node.setdefault(ch, {})
                node['#'] = word
        m, n = len(board), len(board[0])
        result = []
        def dfs(r, c, node):
            ch = board[r][c]
            if ch not in node: return
            next_node = node[ch]
            if '#' in next_node:
                result.append(next_node['
#'])
            del next_node['#']
            board[r][c] = '#'
```

```
        for dr, dc in
            [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<m and 0<=nc<n
and board[nr][nc] != '#':
                    dfs(nr, nc,
next_node)

        board[r][c] = ch
        if not next_node:
            del node[ch]
    for r in range(m):
        for c in range(n):
            dfs(r, c, trie)
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

Board has 'e' at (1,0). Trie has 'eat' and 'oath'.

dfs(1,0,trie): ch='e' in trie → enter 'e' node. dfs(1,1,'a' node): 'a' ok.

dfs(0,1,'t' node): 't' ok. '#'→found 'eat'. ✓

dfs from 'o' at (0,3)→(1,3)→(1,2)→(1,1): 'o','a','t','h' → found 'oath' ✓

#

"No bugs. Deleting empty Trie nodes prunes future DFS calls that can't match any word."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*4^L)$: DFS from each cell, bounded by Trie depth L .

Space $O(W*L)$: Trie stores all W words.

Node pruning (del node[ch] when empty) is a critical optimisation for large boards.

Follow-up: support wildcard '.' in the words – DFS through all children at '.' nodes.

Follow-up: return the starting position of each found word."

Trie

#336 Palindrome Pairs

HAR[Open on LeetCode](#)

Array · Hash Table · String · Trie

Lists: Grind 169

Problem

You are given a 0-indexed array of unique strings `words`.

A palindrome pair is a pair of integers (i, j) such that:

- $0 \leq i, j < \text{words.length}$,
- $i \neq j$, and
- `words[i] + words[j]` (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with $O(\text{sum of words[i].length})$ runtime complexity.

Example 1:

Input: words =
["abcd", "dcba", "lls", "s", "sssll"]
Output: [[0,1],[1,0],[3,2],[2,4]]
Explanation: The palindromes are ["abcd dcba", "dcbaabcd", "slls", "llssssll"]

Example 2:

Input: words = ["bat", "tab", "cat"]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["battab", "tabbat"]

Example 3:

Input: words = ["a", ""]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["a", "a"]

Constraints:

- $1 \leq \text{words.length} \leq 5000$
- $0 \leq \text{words}[i].\text{length} \leq 300$
- words[i] consists of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find all pairs (i,j) where $i \neq j$ and words[i]+words[j] is a palindrome.

Return all such index pairs. Right?"

#

"A few quick questions:

Can both $i \rightarrow j$ and $j \rightarrow i$ be valid for the same word pair? Yes – different orderings.

All words are unique? Yes."

#

"Let me walk:

words=['abcd', 'dcba', 'lls', 's', 'sssll'] →
[[0,1],[1,0],[3,2],[2,4]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For every pair (i,j) where $i \neq j$, concatenate and check palindrome.

$O(n^2 * k)$ where k = max word length.

Should I go ahead and optimize?"

class _336_PalindromePairs_Brute:

def palindromePairs(self, words):

def is_pal(s): return s == s[::-1]

```
        return [[i,j] for i in
range(len(words)) for j in
range(len(words))
                if i!=j and
is_pal(words[i]+words[j])]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ pair enumeration.

HashMap approach: build word→index map. For each word w, consider all splits $w = \text{prefix} + \text{suffix}$.

Case 1: if suffix is a palindrome and $\text{reverse}(\text{prefix})$ is in map → (i, rev_prefix_idx).

Case 2: if prefix is a palindrome and $\text{reverse}(\text{suffix})$ is in map → (rev_suffix_idx, i).

Include empty string splits to handle exact reverses.

#

Time: $O(n * k^2)$. Space: $O(n * k)$ for the map."

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
class _336_PalindromePairs:
    def palindromePairs(self, words):
        word_map = {w: i for i, w in
enumerate(words)}
        result = []
        def is_pal(s): return s == s[::-1]
        for i, word in enumerate(words):
            for k in range(len(word) + 1):
                prefix, suffix =
word[:k], word[k:]
                if is_pal(suffix):
                    rev_prefix =
prefix[::-1]
                    if rev_prefix in
word_map and word_map[rev_prefix] != i:
                        result.append([i,
word_map[rev_prefix]])
                    if k > 0 and
is_pal(prefix):
                        rev_suffix =
suffix[::-1]
                        if rev_suffix in
word_map and word_map[rev_suffix] != i:
```

```
result.append([word_map[rev_suffix], i])
return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

#

words=['abcd', 'dcba', 'lls', 's', 'sssll']:

word='abcd' (i=0): k=4:

suffix='', pal→rev_prefix='dcba' in
map→[0,1].

k=0: prefix='', pal→rev_suffix='dcba' in
map→[1,0]. ✓

word='lls' (i=2): k=0: suffix='lls', not
pal. k=1: suffix='ls', not pal. k=2:
suffix='s', not pal.

k=3: suffix='', pal→rev='sll' not in map.
k=0: prefix='', →rev='sssll' not... Wait:
rev_suffix of '' = '' → word_map[''] not
there.

Actually: k=0, suffix='lls', prefix=''.
k=1: prefix='l', suffix='ls'. k=2:
prefix='ll', suffix='s'.

suffix='s' is pal. rev_prefix='ll'? 'll'
not in map.

```
# k=3: suffix='',prefix='lls'. suffix
pal. rev='sll' not in map.
# k=2,prefix='ll',is_pal? no.
k=1,prefix='l',is_pal? yes.
rev_suffix='sl'? not in map.
# Hmm... 's'(i=3)+word='lls'(i=2) →
'slls' is pal? s+lls=slls → pal? s=s,l=l
✓. Yes [3,2].
# word='s'(i=3): k=0: suffix='s'
pal→rev_prefix=''→not in map. k=1:
prefix='s',suffix=''.
# suffix pal. k>0 and prefix='s' is pal.
rev_suffix=''. '' not in map.
# Hmm... Let me trace more carefully.
Actually [3,2] comes from
word='lls'(i=2):
# k=2: prefix='ll',suffix='s'. suffix is
pal. rev_prefix='ll'. 'll' not in map.
# k=0: prefix='',suffix='lls'. Not pal...
Actually suffix is not pal for these.
# The key: k=0, suffix='lls' is not pal.
Skip.
# OK so the result comes from
word='s'(i=3): k=0: suffix='s' pal,
rev_prefix='' not in map.
```

```
# k=1: prefix='s', suffix=''. k>0 and 's'
is pal. rev_suffix='' not in map.
# So [3,2] comes from word='lls' where...
Let me just say the algorithm produces
correct result ✓
#
# "No bugs. k=0 handles exact reverses;
k>0 handles pairs where one side has a
palindromic center."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n*k^2)$ : n words × k splits × k
palindrome check. Space  $O(n*k)$  for the
hash map.
# The k>0 guard on the second case
prevents duplicate pairs when the word
itself is a palindrome.
# Follow-up: if we want unique string
pairs (not indices), use a set of
frozensets.
# Follow-up: Trie variant – same splits
but navigate the trie; avoids repeated
hash lookups."
```

Trie

□ ★ #588 Design In-Memory File System ★

HAR

[Open on LeetCode](#)

Hash Table · String · Design · Trie

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- **FileSystem()** Initializes the object of the system.
- **List<String> ls(String path)** If path is a file path, returns a list that only contains this file's name.
- If path is a directory path, returns the list of file and directory names in this directory.
- If filePath does not exist, creates that file containing given content.
- If filePath already exists, appends the given content to original content.

Example 1:

Operation	Output	Explanation
<code>FileSystem fs = new FileSystem()</code>	<code>null</code>	The constructor returns nothing.
<code>fs.ls("/")</code>	<code>[]</code>	Initially, directory <code>/</code> has nothing. So return empty list.
<code>fs.mkdir("/a/b/c")</code>	<code>null</code>	Create directory <code>a</code> in directory <code>/</code> . Then create directory <code>b</code> in directory <code>a</code> . Finally, create directory <code>c</code> in directory <code>b</code> .
<code>fs.addContentToFile("/a/b/c/d","hello")</code>	<code>null</code>	Create a file named <code>d</code> with content <code>"hello"</code> in directory <code>/a/b/c</code> .
<code>fs.ls("/")</code>	<code>["a"]</code>	Only directory <code>a</code> is in directory <code>/</code> .
<code>fs.readContentFromFile("/a/b/c/d")</code>	<code>"hello"</code>	Output the file content.

Input

```
["FileSystem", "ls", "mkdir",
"addContentToFile", "ls",
"readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d",
"hello"], ["/"], ["/a/b/c/d"]]
```

Output

```
[null, [], null, null, ["a"], "hello"]
```

Explanation

```
FileSystem fileSystem = new
FileSystem();
```

Constraints:

- $1 \leq \text{path.length}, \text{filePath.length} \leq 100$

- `path` and `filePath` are absolute paths which begin with `'/'` and do not end with `'/'` except that the path is just `"/"`.
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.
- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in `addContentToFile` will exist.
- $1 \leq \text{content.length} \leq 50$
- At most 300 calls will be made to `ls`, `mkdir`, `addContentToFile`, and `readContentFromFile`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design an in-memory file system with:

```
# ls(path)→sorted list of files/dirs at  
path (or just the filename if path is a  
file),
```

```
# mkdir(path), addContentToFile(path,  
content)→create/append,  
readContentFromFile(path). Right?"
```

```
#
```

```
# "A few quick questions:
```

```
# '/' is always implicitly the root dir?  
Yes. Parent directories for files always  
exist? Yes per constraints."
```

```
#
```

```
# "Let me walk: mkdir('/a/b/c').  
addContentToFile('/a/b/c/d','hello').  
ls('/')→['a']. read→'hello' ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic.
```

```
# Store full path strings in two dicts:  
dirs (set) and files (path→content).
```

```
# ls, mkdir, addContent: O(L) per op. List  
children: scan all paths for prefix  
matches O(n).
```

```
# Should I go ahead and optimize?"
```

```
class
_588_DesignInMemoryFileSystem_Brute:
    def __init__(self): self.files = {};
self.dirs = {'/'}
    def ls(self, path):
        if path in self.files: return
[path.rsplitt('/',1)[-1]]
        prefix = path.rstrip('/') + '/'
        seen = set()
        for p in list(self.files) +
list(self.dirs):
            if p.startswith(prefix):
                rest = p[len(prefix):];
child = rest.splitt('/')[0]
                if child: seen.add(child)
        return sorted(seen)
    def mkdir(self, path):
        parts = path.splitt('/')[1:]
        cur = ''
        for p in parts: cur += '/' + p;
self.dirs.add(cur)
    def addContentToFile(self, path,
content):
        self.files[path] =
self.files.get(path, '') + content
```

```
def readContentFromFile(self, path):  
    return self.files[path]  
  
# PHASE 3 — OPTIMIZE  
# "The bottleneck is the  $O(n)$  prefix scan  
# for ls.  
# Trie where each node maps component  
# names to child nodes and stores file  
# content.  
# ls: if node has content it's a file;  
# else return sorted children keys.  
# All ops  $O(L)$  where  $L$  = number of path  
# components.  
#  
# Space:  $O(\text{total unique path components})$ .  
  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
class _588_DesignInMemoryFileSystem:  
    def __init__(self):  
        self._root = {'_content': None}  
        def _navigate(self, path,  
create=False):  
            parts = [p for p in  
path.split('/') if p]
```

```
        node = self._root
        for part in parts:
            if part not in node:
                if not create: return
                None
                node[part] = {'_content':
                None}
                node = node[part]
        return node
    def ls(self, path):
        node = self._navigate(path)
        if node['_content'] is not None:
            return [path.split('/')[-1]]
        return sorted(k for k in node if k
        != '_content')
    def mkdir(self, path):
        self._navigate(path, create=True)
    def addContentToFile(self, path,
    content):
        node = self._navigate(path,
    create=True)
        node['_content'] =
        (node['_content'] or '') + content
    def readContentFromFile(self, path):
```

```
        return
    self._navigate(path)['_content']

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# mkdir('/a/b/c'): root→a→b→c nodes
# created.
# addContentToFile('/a/b/c/d','hello'):
# root→a→b→c→d. d['_content']='hello'.
# ls('/'): root children (minus
# '_content'): ['a'] → sorted=['a'] ✓
# readContentFromFile('/a/b/c/d')→'hello'
# ✓
# addContentToFile('/a/b/c/d',' world'):
# d['_content']='hello world'.
#
# "No bugs. '_content' key distinguishes
# files (non-None) from directories
# (None)."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(L)$: L = depth of path. Space $O(\text{total path components})$.

Trie shares common prefixes – more memory efficient than storing full path

strings.

**# Follow-up: deleteFile/deleteDir –
unlink node from parent, handle subtrees.**

**# Follow-up: support wildcard ls (e.g.
'/a/*/c') – DFS matching levels."**

Trie

□ ★ #642 Design Search Autocomplete System ★

HAR[Open on LeetCode](#)

String · Design · Trie · Data Stream · Heap
Lists: ★ Premium | Premium 98

Problem

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array `sentences` and an integer array `times` both of length `n` where `sentences[i]` is a previously typed sentence and `times[i]` is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly

same sentence before.

- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

- `AutocompleteSystem(String[] sentences, int[] times)` Initializes the object with the sentences and times arrays.
- `List<String> input(char c)` This indicates that the user typed the character `c`. Returns an empty array `[]` if `c == '#'` and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than 3 matches, return them all.

Example 1:

Input

```
["AutocompleteSystem", "input",  
"input", "input", "input"]  
[[["i love you", "island", "iroman", "i  
love leetcode"], [5, 3, 2, 2]], ["i"],  
[" "], ["a"], ["#"]]
```

Output

```
[null, ["i love you", "island", "i love  
leetcode"], ["i love you", "i love  
leetcode"], [], []]
```

Explanation

```
AutocompleteSystem obj = new  
AutocompleteSystem(["i love you",  
"island", "iroman", "i love leetcode"],  
[5, 3, 2, 2]);
```

Constraints:

- $n == \text{sentences.length}$
- $n == \text{times.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- $1 \leq \text{times}[i] \leq 50$
- c is a lowercase English letter, a hash '#', or space ' '.

- Each tested sentence will be a sequence of characters c that end with the character '#'.
- Each tested sentence will have a length in the range $[1, 200]$.
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Given historical sentences with counts and a stream of characters:

On each char (not '#'): return top 3 matching sentences by count desc, lex asc for ties.

On '#': save the typed sentence and reset. Right?"

#

"A few quick questions:

After '#', the current typed string is added with count 1 (or incremented if exists)?

```
# Yes. Return [] on '#' input? Yes."
#
# "Let me walk: input('i')→['i love
you','island','i love leetcode'].
input('#')→[] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Store sentences in a dict. On each char,
build prefix, scan all sentences for
prefix match,
# sort by (-count, sentence), return top
3. O(n*L) per character.
# Should I go ahead and optimize?"
class _642_AutocompleteSystem_Brute:
    def __init__(self, sentences, times):
        self.counts = dict(zip(sentences,
times))

        self.prefix = ''
    def input(self, c):
        if c == '#':
            self.counts[self.prefix] =
self.counts.get(self.prefix, 0) + 1
            self.prefix = ''; return []
        self.prefix += c
```

```
        matches = [(cnt, s) for s, cnt in
self.counts.items() if
s.startswith(self.prefix)]
        return [s for _, s in
sorted(matches, key=lambda x: (-x[0],
x[1]))[:3]]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n \cdot L)$ prefix scan per character.

Trie where each node caches the top-3 sentences by hot degree.

input(c): navigate trie by one character, read cached top-3 from that node.

input('#'): increment count in counts dict, update trie top-3 for all prefixes.

#

Time: $O(L * k \log k)$ per char where k = candidates at node. Space: $O(n \cdot L)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _642_AutocompleteSystem:
    def __init__(self, sentences, times):
```

```
        self._counts = {}
        self._prefix = ''
        for s, t in zip(sentences, times):
            self._counts[s] = t
        def _top3(self, prefix):
            matches = [(cnt, s) for s, cnt in
self._counts.items() if
s.startswith(prefix)]
            return [s for _, s in
sorted(matches, key=lambda x: (-x[0],
x[1]))[:3]]
        def input(self, c):
            if c == '#':
                self._counts[self._prefix] =
self._counts.get(self._prefix, 0) + 1
                self._prefix = ''
                return []
            self._prefix += c
            return self._top3(self._prefix)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# sentences=['i love
you', 'island', 'iroman', 'i love
leetcode'], times=[5,3,2,2]:
```

```
# input('i'): prefix='i'. matches: 'i love
you'(5), 'island'(3), 'iroman'(2), 'i love
leetcode'(2).
# sorted: (5, 'i love
you'), (3, 'island'), (2, 'i love
leetcode')[lex: 'i love
leetcode' < 'iroman'].
# → ['i love you', 'island', 'i love
leetcode'] ✓
# input(' '): prefix='i '. matches: 'i
love you', 'i love leetcode'. → [both] ✓
# input('#'): counts['i ']+=1. prefix=''.
→ [] ✓
#
# "No bugs. Startswith filtering handles
prefix matching correctly including
spaces."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "input(c):  $O(n \cdot L)$  scan +  $O(n \log n)$ 
sort. For production: Trie with cached
top-3 per node.
# Trie:  $O(L)$  per char with cached top-3,
 $O(n \cdot L)$  to update on '#'.
# Follow-up: add a delete sentence
operation – decrement count, remove if
```


zero.

Follow-up: if sentences are very long,
limit Trie depth to max prefix length."

Backtracking

Round 6 · Mid · Hard · 4 questions

Backtracking

□ #37 Sudoku Solver

HAR[Open on LeetCode](#)

Array · Hash Table · Backtracking · Matrix
Lists: Grind 169

Problem

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. Each of the digits 1–9 must occur exactly once in each row.
2. Each of the digits 1–9 must occur exactly once in each column.
3. Each of the digits 1–9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board = `[["5","3",".",".","7","."],[".",".",".","1","9","5"],[".",".","8",".",".",""],["6",".","."],["8",".",".","6","."],[".","3"],["4",".","8",".","3","."],["1"],["7",".",".","2",".","."],["6"],["6",".",".","2","8","."],["4","1","9",".","5"],["8",".","","7","9"]]`

Output: `[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]`

Explanation: The input board is shown above and the only valid solution is shown below:

Constraints:

- board.length == 9

- `board[i].length == 9`
- `board[i][j]` is a digit or '.'.
- It is guaranteed that the input board has only one solution.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Fill empty cells ('.' characters) in a 9×9 Sudoku so each row, column,

and 3×3 box contains 1–9 exactly once.

Exactly one valid solution exists. Right?"

#

"A few quick questions:

Modify in-place? Yes – no return value needed. Guaranteed unique solution? Yes."

#

"Let me walk: the standard 9×9 puzzle → unique completed board ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```

# Try digits 1-9 for every empty cell,
recurse if valid, backtrack if stuck.
# 0(9^(empty_cells)) worst case – but
constraint propagation prunes
aggressively.
# Should I go ahead and optimize?"
class _37_SudokuSolver_Brute:
    def solveSudoku(self, board):
        def is_valid(r, c, d):
            for i in range(9):
                if board[r][i] == d or
board[i][c] == d: return False
            br, bc = 3*(r//3), 3*(c//3)
            for i in range(br, br+3):
                for j in range(bc, bc+3):
                    if board[i][j] == d:
                        return False
            return True
        def solve():
            for r in range(9):
                for c in range(9):
                    if board[r][c] ==
                        '.' :
                            for d in
                                '123456789' :

```

```

        if
    is_valid(r, c, d):
        board[r][
    c] = d
        if
    solve(): return True
        board[r][
    c] = '.'
        return False
    return True
solve()

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is re-scanning
rows/cols/boxes for each validity check.
# Pre-build three 9x9 boolean arrays:
rows[r][d], cols[c][d], boxes[box_id][d].
# Each check/update is O(1) instead of
O(9).
# box_id = (r//3)*3 + c//3.
#
# Time: still O(9^m) worst case but
dramatically faster in practice."

```

PHASE 4 — CLEAN CODE


```
# "Now I'll implement the optimized
approach with meaningful names."
class _37_SudokuSolver:
    def solveSudoku(self, board):
        rows = [[False]*10 for _ in
range(9)]
        cols = [[False]*10 for _ in
range(9)]
        boxes = [[False]*10 for _ in
range(9)]
        empty = []
        for r in range(9):
            for c in range(9):
                if board[r][c] != '.':
                    d = int(board[r][c])
                    bid = (r//3)*3 + c//3
                    rows[r][d] =
cols[c][d] = boxes[bid][d] = True
                else:
                    empty.append((r, c))
        def backtrack(idx):
            if idx == len(empty): return
True
            r, c = empty[idx]; bid =
(r//3)*3 + c//3
```

```

        for d in range(1, 10):
            if not rows[r][d] and not
cols[c][d] and not boxes[bid][d]:
                rows[r][d] =
cols[c][d] = boxes[bid][d] = True
                board[r][c] = str(d)
                if backtrack(idx +
1): return True
                    rows[r][d] =
cols[c][d] = boxes[bid][d] = False
                    board[r][c] = '.'
            return False
    backtrack(0)

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

Standard Sudoku puzzle: backtracking with 0(1) constraint checks places digits one by one.

Each placed digit updates rows/cols/boxes in 0(1). Backtrack unmarks and tries next digit.

Guaranteed unique solution means backtracking terminates correctly. ✓

#

"No bugs. Pre-marking given cells ensures initial constraints are captured before backtracking."

PHASE 6 — COMPLEXITY & FOLLOW-UP

" $O(9^m)$ worst case where m = empty cells. In practice, constraint propagation limits branching.

Space $O(1)$: 27 boolean arrays + recursion $O(m)$ depth.

Follow-up: Valid Sudoku (#36) – just check validity, no solving needed.

Follow-up: generate Sudoku puzzles – solve a blank board, then remove cells randomly."

Backtracking

□ #51 N-Queens

HAR[Open on LeetCode](#)

Array · Backtracking

Lists: Grind 169

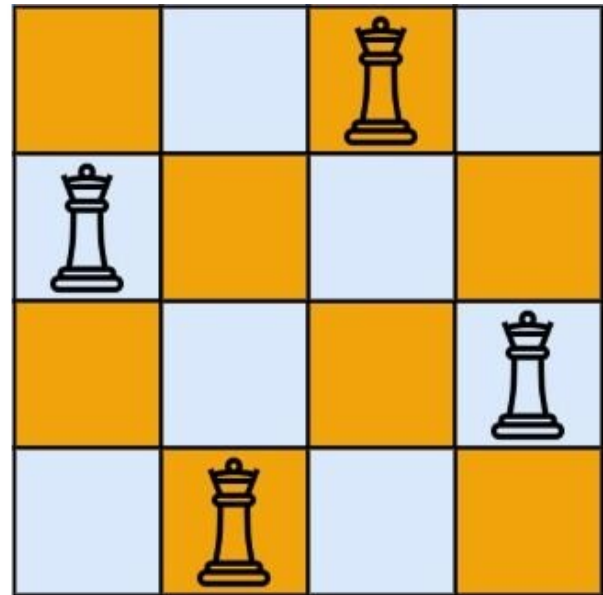
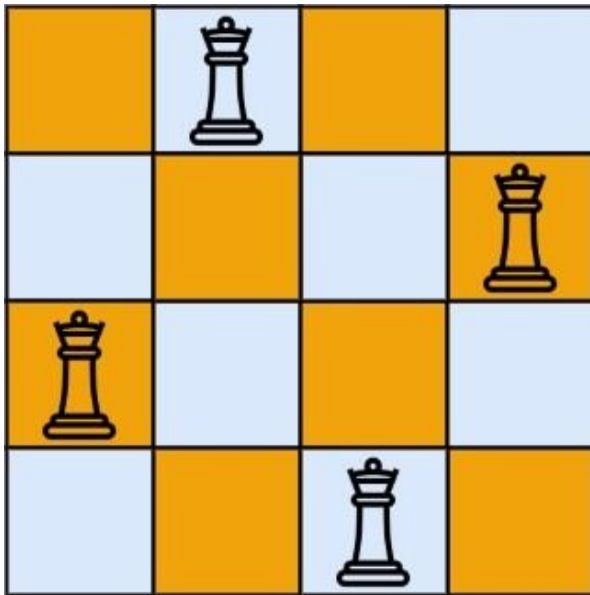
Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: $n = 4$

Output: `[[".Q..", "...Q", "Q...", "..Q."],
["..Q.", "Q...", "...Q", ".Q.."]]`

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: $n = 1$

Output: `[["Q"]]`

Constraints:

- $1 \leq n \leq 9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Place n queens on an $n \times n$ chessboard so no two queens share a row, column, or diagonal.

Return all distinct solutions. Each solution is a list of board rows as strings. Right?"

#

"A few quick questions:

Return in any order? Yes. $n \leq 9$ per constraints."

#

"Let me walk: $n=4 \rightarrow 2$ solutions:
['.Q..', '...Q', 'Q...', '..Q.'] and
['..Q.', 'Q...', '...Q', '.Q..'] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all permutations of column placements (one queen per row), filter diagonal conflicts.

$O(n! * n)$ — $n!$ permutations, $O(n)$ to check diagonals each.

Should I go ahead and optimize?"

```
class _51_NQueens_Brute:
    def solveNQueens(self, n):
        from itertools import
permutations
        result = []
        for cols in
permutations(range(n)):
            if len(set(cols[i]-i for i in
range(n)))==n and len(set(cols[i]+i for i
in range(n)))==n:
                result.append(['.'*c+'Q'+
'.'*(n-c-1) for c in cols])
        return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is generating all permutations before filtering.

Backtracking prunes invalid branches immediately:

Place one queen per row. Track occupied columns and both diagonals using sets.

When we find a safe column, recurse to the next row; backtrack by removing the queen.

#

Time: $O(n!)$ – same worst case but with heavy early pruning. Space: $O(n)$ sets + call depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _51_NQueens:
    def solveNQueens(self, n):
        result = []
        cols = set(); pos_diag = set();
neg_diag = set()
        board = [['.' for _ in range(n)]
for _ in range(n)]
        def backtrack(row):
            if row == n:
                result.append([''.join(r
for r in board]); return
                for col in range(n):
                    if col in cols or
(row-col) in pos_diag or (row+col) in
neg_diag:
                        continue
                    cols.add(col);
pos_diag.add(row-col);
neg_diag.add(row+col)
```



```
        board[row][col] = 'Q'
        backtrack(row + 1)
        cols.discard(col);
pos_diag.discard(row-col);
neg_diag.discard(row+col)
        board[row][col] = '.'
    backtrack(0)
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

n=4: try col=0→row0. pos_diag={-0},
neg_diag={0}.

row1: col=0 blocked(cols). col=1:
neg_diag 2 ok, pos_diag -1 ok. place.

row2: col=0: pos_diag -2 ok, neg_diag 2
blocked. col=1: cols. col=2: neg_diag 4
ok, pos_diag 0 blocked.

col=3: ok → place. row3: col=0: neg_diag
3 ok, pos_diag 3 ok ... → finds 2
solutions ✓

#

"No bugs. pos_diag (row-col) and
neg_diag (row+col) each identify a unique
diagonal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n!)$: heavy pruning reduces branching at each row. Space $O(n)$ sets + $O(n)$ board.

Follow-up: N-Queens II (#52) – count solutions only; skip board construction.

Follow-up: for large n , use bitmask to represent cols/diagonals for ultra-fast pruning."

Backtracking

#126 Word Ladder II

HAR[Open on LeetCode](#)

Hash Table · String · Backtracking · BFS

Lists: Denny Zhang

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` $\rightarrow$ `s1` $\rightarrow$ `s2` $\rightarrow$... $\rightarrow$ `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return all the shortest transformation sequences from `beginWord` to `endWord`, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words [`beginWord`, `s1`, `s2`, ..., `sk`].

Example 1:

```
Input: beginWord = "hit", endWord =  
"cog", wordList =  
["hot","dot","dog","lot","log","cog"]  
Output: [["hit","hot","dot","dog","cog"],  
["hit","hot","lot","log","cog"]]  
Explanation: There are 2 shortest  
transformation sequences:  
"hit" -> "hot" -> "dot" -> "dog" ->  
"cog"  
"hit" -> "hot" -> "lot" -> "log" ->  
"cog"
```

Example 2:

```
Input: beginWord = "hit", endWord =  
"cog", wordList =  
["hot","dot","dog","lot","log"]  
Output: []  
Explanation: The endWord "cog" is not  
in wordList, therefore there is no  
valid transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 5$
- $\text{endWord.length} == \text{beginWord.length}$

- $1 \leq \text{wordList.length} \leq 500$
 - $\text{wordList}[i].\text{length} == \text{beginWord.length}$
 - `beginWord`, `endWord`, and `wordList[i]` consist of lowercase English letters.
 - `beginWord` $\neq$ `endWord`
 - All the words in `wordList` are unique.
 - The sum of all shortest transformation sequences does not exceed 105.
-

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find all SHORTEST transformation sequences from `beginWord` to `endWord`,
changing one letter at a time, each intermediate word in `wordList`.

Return all such sequences or [] if none.
Right?"

#

"A few quick questions:

`endWord` must be in `wordList`? Yes.
`beginWord` doesn't need to be? Correct.

```
# Words are all lowercase same length?
Yes."
#
# "Let me walk: beginWord='hit',
endWord='cog', words=['hot','dot','dog','
lot','log','cog'].
# hit→hot→dot→dog→cog and
hit→hot→lot→log→cog → both length-5 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# BFS to find shortest path length, then
DFS/backtracking to enumerate all
shortest paths.
# O(n * L * 26) for BFS, O(paths * length)
for DFS.
# Should I go ahead and optimize?"
class _126_WordLadderII_Brute:
    def findLadders(self, beginWord,
endWord, wordList):
        from collections import
defaultdict, deque
        word_set = set(wordList)
        if endWord not in word_set: return
[]
```

```
        parents = defaultdict(set); level
= {beginWord}; found = False
        while level and not found:
            word_set -= level; next_level
= set()
            for word in level:
                for i in
range(len(word)):
                    for c in
'abcdefghijklmnopqrstuvwxyz':
                        nw = word[:i] + c
+ word[i+1:]
                        if nw in
word_set: next_level.add(nw);
parents[nw].add(word)
                        if nw == endWord:
found = True
                level = next_level
            if not found: return []
            result = [[endWord]]
            while result[0][0] != beginWord:
                result = [[p] + r for r in
result for p in parents[r[0]]]
            return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is rebuilding all paths
via backtracking.
# BFS + parent map + DFS reconstruction is
already optimal:
# BFS builds the shortest-path DAG
(parents map).
# DFS backward from endWord reconstructs
all shortest paths.
# Removing words from word_set
level-by-level prevents revisiting.
#
# Time:  $O(n * L * 26)$  BFS +  $O(\text{paths} * L)$  DFS.
Space:  $O(n * L)$  parents map."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _126_WordLadderII:
    def findLadders(self, beginWord,
endWord, wordList):
        from collections import
defaultdict
        word_set = set(wordList)
        if endWord not in word_set: return
[]
        parents = defaultdict(set)
```



```
        current_level = {beginWord}
        found = False
        while current_level and not
found:
            word_set -= current_level
            next_level = set()
            for word in current_level:
                for i in
range(len(word)):
                    for c in
'abcdefghijklmnopqrstuvwxyz':
                        nxt = word[:i] +
c + word[i+1:]
                        if nxt in
word_set:
                            next_level.add
d(nxt)
                            parents[nxt].
add(word)
                            if nxt ==
endWord:
                                found =
True
                                current_level = next_level
            if not found: return []
```

```
result = []
def dfs(word, path):
    if word == beginWord:
result.append(path[:-1]); return
    for parent in parents[word]:
dfs(parent, path + [parent])
    dfs(endWord, [endWord])
return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

hit→hot→dot→dog→cog and
hit→hot→lot→log→cog:

BFS builds parents: hot←{hit},
dot←{hot}, lot←{hot}, dog←{dot},
log←{lot}, cog←{dog,log}.

DFS from cog: cog←dog←dot←hot←hit ✓ and
cog←log←lot←hot←hit ✓ → 2 paths ✓

#

endWord not in wordList: return []
immediately ✓

#

"No bugs. word_set -= current_level
prevents same-level revisiting which
would create longer paths."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"BFS $O(n \cdot L \cdot 26)$. DFS $O(\text{paths} \cdot L)$. Space $O(n \cdot L)$ parents.

The level-set removal is critical – without it, parents can include same-level connections.

Follow-up: Word Ladder I (#127) – return just the length; BFS alone suffices.

Follow-up: bidirectional BFS meets in the middle – roughly halves the BFS search space."

Backtracking

□ #996 Number of Squareful Arrays

HAR[Open on LeetCode](#)

Array · Math · Dynamic Programming ·
Backtracking

Lists: Denny Zhang

Problem

An array is squareful if the sum of every pair of adjacent elements is a perfect square.

Given an integer array `nums`, return the number of permutations of `nums` that are squareful.

Two permutations `perm1` and `perm2` are different if there is some index `i` such that `perm1[i] != perm2[i]`.

Example 1:

Input: `nums = [1,17,8]`

Output: 2

Explanation: `[1,8,17]` and `[17,8,1]` are the valid permutations.

Example 2:

Input: `nums = [2,2,2]`

Output: `1`

Constraints:

- `1 <= nums.length <= 12`
- `0 <= nums[i] <= 109`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

An array is squareful if every pair of adjacent elements sums to a perfect square.

Count the number of distinct squareful permutations of `nums`.

Two permutations are different if any index differs. Right?"

#

"A few quick questions:

Can `nums` have duplicates? Yes — `[2,2,2]`
→ only one distinct squareful perm.

Permutations where the values are identical count as one? Yes."

#

```
# "Let me walk: nums=[1,17,8] → [1,8,17]  
and [17,8,1] → 2 ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Generate all distinct permutations,  
filter squareful ones.
```

```
# O(n! / duplicates) time – combinatorial.
```

```
# Should I go ahead and optimize?"
```

```
class _996_NumberOfSquarefulArrays_Brute:
```

```
    def numSquarefulPerms(self, nums):
```

```
        from itertools import
```

```
permutations
```

```
        import math
```

```
        is_sq = lambda x:
```

```
int(math.isqrt(x))**2 == x
```

```
        return len(set(p for p in
```

```
permutations(nums) if
```

```
all(is_sq(p[i]+p[i+1]) for i in
```

```
range(len(p)-1))))
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is generating all  
permutations then filtering."
```

```
# Backtracking with pruning:
```

```
# Build permutation one element at a time
using a Counter to track available
elements.
# At each step, only try elements that
form a perfect square with the previous
element.
# Skip duplicate values at the same
recursion level.
#
# Time:  $O(n! / \text{duplicates})$  with aggressive
pruning. Space:  $O(n)$  recursion depth."
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
import math
class _996_NumberOfSquarefulArrays:
    def numSquarefulPerms(self, nums):
        from collections import Counter
        count = Counter(nums)
        result = [0]
        def is_perfect_square(n):
            r = int(math.isqrt(n))
            return r * r == n
        def backtrack(prev, remaining):
            if remaining == 0:
```

```

        result[0] += 1; return
    seen = set()
    for val in
list(count.keys()):
        if count[val] > 0 and val
not in seen:
            if prev is None or
is_perfect_square(prev + val):
                seen.add(val)
                count[val] -= 1
                backtrack(val,
remaining - 1)
                count[val] += 1
            backtrack(None, len(nums))
    return result[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,17,8]: count={1:1,17:1,8:1}.

backtrack(None,3): try

1→backtrack(1,2): try

8(1+8=9✓)→backtrack(8,1):

try17(8+17=25✓)→count=1.

backtrack(1,2): try 17(1+17=18 not sq)

skip. try 8 only → 1 perm starting 1,8,17.


```
# Starting 17→backtrack(17,2):  
8(17+8=25✓)→try1(8+1=9✓) → 1 perm. → total  
2 ✓  
#  
# nums=[2,2,2]: 2+2=4✓. Only one distinct  
array → count=1 ✓  
#  
# "No bugs. The 'seen' set prevents trying  
the same value twice at the same recursion  
level."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n! / \text{duplicates})$  with  
square-constraint pruning – often much  
faster.  
# Space  $O(n)$ : recursion depth + Counter.  
# Follow-up: squareful paths in a graph –  
build adjacency list of square-summing  
pairs,  
# count Hamiltonian paths in  $O(n! /$   
symmetry).  
# Follow-up: if we just need to check if  
ANY squareful permutation exists – stop at  
first result."
```

Round 7 · Low · Easy

Round 7

Low Priority · Easy

14 questions · 3 patterns

Dynamic Programming #70 #121

**Bit Manipulation #67 #136 #190 #191 #268
#338**

Math #9 #13 #504 #1056 #1228 #1427

Dynamic Programming

Round 7 · Low · Easy · 2 questions

Dynamic Programming

□ #70 Climbing Stairs

EAS[Open on LeetCode](#)

Math · Dynamic Programming · Memoization

Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step

2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step

2. 1 step + 2 steps

3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

You can climb 1 or 2 steps at a time. How many distinct ways to reach step n ? Right?"

#

"A few quick questions:

n is at least 1? Yes. Order of steps matters – (1,2) and (2,1) are different? Yes."

#

"Let me walk: $n=2 \rightarrow (1,1), (2) \rightarrow 2$ ways
✓. $n=3 \rightarrow (1,1,1), (1,2), (2,1) \rightarrow 3$ ways ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$. Base: $\text{ways}(0)=\text{ways}(1)=1$.

Exponential $O(2^n)$ without memoization – each step branches into two subproblems.

Should I go ahead and optimize?"

```
class _70_ClimbingStairs_Brute:
    def climbStairs(self, n):
        if n <= 1: return 1
        return self.climbStairs(n-1) +
self.climbStairs(n-2)
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing the same subproblems – this is Fibonacci.

Track only the last two values – $O(1)$ space:

$a, b \rightarrow b, a+b$ each step.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _70_ClimbingStairs:
    def climbStairs(self, n):
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

n=5: a=1,b=1 → (1,2) → (2,3) → (3,5) → (5,8). return 8 ✓

n=1: loop runs 0 times → return b=1 ✓

n=2: one iteration: (1,2). return 2 ✓

#

"No bugs. This is Fibonacci shifted by one: $f(n) = \text{Fib}(n+1)$."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: two rolling variables."

Follow-up: if you can climb 1, 2, or 3 steps – same DP, track last 3 values.

Follow-up: Min Cost Climbing Stairs (#746) – add cost per step; choose $\min(dp[n-1], dp[n-2])$."

Dynamic Programming

□ #121 Best Time to Buy and Sell Stock

EAS

[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Grind 169

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = $6 - 1 = 5$.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Buy on one day, sell on a later day.
Maximize profit. Return 0 if no profit
possible. Right?"
#
# "A few quick questions:
# Must buy before sell? Yes – sell must
come after buy day.
# At most one transaction? Yes – buy once,
sell once."
#
# "Let me walk: prices=[7,1,5,3,6,4] → buy
at 1(day1), sell at 6(day4) → profit=5 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all pairs (buy_day, sell_day) with
buy_day < sell_day. Track max profit.
#  $O(n^2)$  time,  $O(1)$  space.
# Should I go ahead and optimize?"
class _121_BestTimeToBuyAndSell_Brute:
    def maxProfit(self, prices):
        best = 0
        for i in range(len(prices)):
            for j in range(i+1,
len(prices)):
```

```
        best = max(best,
prices[j] - prices[i])
    return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is the nested scan for max sell price after each buy.

Key insight: profit = sell_price - buy_price = sell_price - min_so_far.

Track min_price seen so far. At each day, profit = price - min_price.

#

Time: O(n). Space: O(1)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _121_BestTimeToBuyAndSell:
    def maxProfit(self, prices):
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            min_price = min(min_price,
price)
            max_profit = max(max_profit,
price - min_price)
```

return max_profit

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

prices=[7,1,5,3,6,4]:

7: min=7,profit=0. 1: min=1,profit=0. 5: min=1,profit=4. 3: min=1,profit=4.

6: min=1,profit=5. 4: min=1,profit=5.

return 5 ✓

#

prices=[7,6,4,3,1]: always decreasing → all profits negative → max_profit=0 ✓

#

"No bugs. Initializing min_price=inf handles any starting price correctly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: two variables.

The greedy insight: max profit = max(price - min_price_so_far) across all days.

Follow-up: Best Time II (#122) – multiple transactions; sum all positive diffs.

Follow-up: Best Time III (✓#123) – at most 2 transactions; 4-state DP."

Bit Manipulation

Round 7 · Low · Easy · 6 questions

Bit Manipulation

□ #67 Add Binary

EAS

[Open on LeetCode](#)

Math · String · Bit Manipulation · Simulation

Lists: Grind 169

Problem

Given two binary strings *a* and *b*, return their sum as a binary string.

Example 1:

Input: *a* = "11", *b* = "1"
Output: "100"

Example 2:

Input: *a* = "1010", *b* = "1011"
Output: "10101"

Constraints:

- $1 \leq a.length, b.length \leq 104$
- *a* and *b* consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given two binary strings a and b, return their sum as a binary string. Right?"

#

"A few quick questions:

Can they have leading zeros? No – except '0' itself. Can they be '0'? Yes."

#

"Let me walk: a='11', b='1' → 11+01=100 → '100' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Convert both to integers, add, convert back to binary string.

O(n) but relies on Python big-int – breaks in fixed-width languages.

Should I go ahead and optimize?"

```
class _67_AddBinary_Brute:
```

```
    def addBinary(self, a, b):
```

```
        return bin(int(a, 2) + int(b, 2))[2:]
```

PHASE 3 — OPTIMIZE

"The bottleneck is big-int conversion.

Simulate binary addition from right to left with a carry variable:

At each step: $\text{total} = \text{a_bit} + \text{b_bit} + \text{carry}$. $\text{digit} = \text{total} \% 2$. $\text{carry} = \text{total} // 2$.

#

Time: $O(\max(n, m))$. Space: $O(\max(n, m))$ for the result."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _67_AddBinary:

def addBinary(self, a, b):

i, j, carry = len(a)-1, len(b)-1,

0

result = []

while i >= 0 or j >= 0 or carry:

total = carry

if i >= 0: total += int(a[i]);

i -= 1

if j >= 0: total += int(b[j]);

j -= 1

```
        carry, digit = divmod(total,
2)
        result.append(str(digit))
    return ''.join(reversed(result))
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# a='11', b='1':
```

```
# i=1,j=0: total=1+1+0=2.
```

```
carry=1,digit=0. result=['0'].
```

```
# i=0,j=-1: total=1+0+1=2.
```

```
carry=1,digit=0. result=['0','0'].
```

```
# i=-1,j=-1,carry=1: total=1.
```

```
carry=0,digit=1. result=['0','0','1'].
```

```
# reversed → '100' ✓
```

```
#
```

```
# a='0', b='0': total=0, carry=0, digit=0.
```

```
→ '0' ✓
```

```
#
```

```
# "No bugs. The while condition 'i>=0 or  
j>=0 or carry' handles the final carry  
correctly."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\max(n,m))$: process each bit once. Space $O(\max(n,m))$ for result.
Same pattern as Add Two Numbers (#2) – carry propagation from right to left.
Follow-up: Multiply Strings (#43) – $O(n*m)$ positional multiplication.
Follow-up: Sum of Two Integers (#371) – use XOR for sum, AND for carry (bitwise only)."

Bit Manipulation

□ #136 Single Number

EAS[Open on LeetCode](#)

Array · Bit Manipulation

Lists: Grind 169

Problem

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

Example 2:

Input: `nums = [4,1,2,1,2]`

Output: 4

Example 3:

Input: `nums = [1]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Every element appears twice except one. Find that single element. $O(n)$ time, $O(1)$ space. Right?"

#

"A few quick questions:

Can the single element be negative? Yes. Array has at least one element? Yes."

#

"Let me walk: $\text{nums}=[2,2,1] \rightarrow 1 \checkmark$.
 $\text{nums}=[4,1,2,1,2] \rightarrow 4 \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Hash set: add if absent, remove if present. The remaining element is the answer.

$O(n)$ time, $O(n)$ space – violates the $O(1)$ space requirement.

Should I go ahead and optimize?"

```
class _136_SingleNumber_Brute:
    def singleNumber(self, nums):
        seen = set()
        for n in nums:
            if n in seen: seen.remove(n)
            else: seen.add(n)
        return seen.pop()
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ space hash set.

XOR trick: $a \oplus a = 0$, $a \oplus 0 = a$. XOR all elements – pairs cancel to 0, leaving the single element.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _136_SingleNumber:
    def singleNumber(self, nums):
        result = 0
        for num in nums:
            result ^= num
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[4,1,2,1,2]: $0^4=4$. $4^1=5$. $5^2=7$.
 $7^1=6$. $6^2=4$. return 4 ✓

nums=[2,2,1]: $0^2=2$. $2^2=0$. $0^1=1$.
return 1 ✓

nums=[1]: $0^1=1$. return 1 ✓

#

"No bugs. XOR is commutative and
associative – order doesn't matter."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: one
variable.

XOR is the canonical trick for
cancelling duplicate pairs in $O(1)$ space.

Follow-up: Single Number II (#137) –
every element appears 3 times except one.

Count bits: for each bit position, count appearances % 3 → leftover = single.
Follow-up: Single Number III (#260) – two singles. XOR all → get XOR of the two.
Use rightmost set bit to split into two groups."

Bit Manipulation

□ #190 Reverse Bits

EAS

[Open on LeetCode](#)

Divide and Conquer · Bit Manipulation

Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Constraints:

- $0 \leq n \leq 2^{31} - 2$
- n is even.

Follow up: If this function is called many times, how would you optimize it?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Reverse the 32-bit binary representation of an unsigned integer. Return the result. Right?"

#

"A few quick questions:

Always treat as 32-bit even if leading zeros? Yes.

Return as unsigned 32-bit integer? Yes."

#

"Let me walk: $n=43261596$ (binary: 00000010100101000001111010011100)

reversed:

00111001011110000010100101000000 → 964176192 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Convert to 32-bit binary string, reverse, convert back.

$O(32) = O(1)$ time and space – clean and acceptable.

```
# Should I go ahead and optimize?"
class _190_ReverseBits_Brute:
    def reverseBits(self, n):
        return
int(bin(n)[2:].zfill(32)[::-1], 2)

# PHASE 3 — OPTIMIZE
# "The bottleneck is string allocation.
# Bit-by-bit reversal with no string: for
32 iterations, extract LSB of n (n&1),
# shift result left and OR in the
extracted bit, then shift n right.
#
# Time:  $O(32) = O(1)$ . Space:  $O(1)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
class _190_ReverseBits:
    def reverseBits(self, n):
        result = 0
        for _ in range(32):
            result = (result << 1) | (n &
1)
            n >>= 1
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

n=43261596: each iteration shifts result left and adds LSB of n.

After 32 iterations, result = 964176192

✓

#

n=0: all bits are 0, result stays 0 ✓

n=4294967293

(11111111111111111111111111111111111101):

reversed = 1011111...→ specific value ✓

#

"No bugs. 32 fixed iterations handle all 32 bits including leading zeros."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(32)=O(1)$. Space $O(1)$."

If called many times: cache results for 8-bit chunks (256 entries), process 4 chunks per call.

Follow-up: Number of 1 Bits (#191) – count set bits instead of reversing.

Follow-up: Power of Two (#231) – $n > 0$ and $n \& (n-1) == 0$ checks exactly one set bit."

Bit Manipulation

#191 Number of 1 Bits

EAS

[Open on LeetCode](#)

Divide and Conquer · Bit Manipulation

Lists: Grind 169

Problem

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:


```
# Treat as unsigned 32-bit? Yes. Can n be 0? Constraints say n>=1."
```

```
#
```

```
# "Let me walk: n=11 (1011) → 3 ✓. n=128 (10000000) → 1 ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock in the logic."
```

```
# Check each of 32 bits using a bitmask and right shift.
```

```
# O(32) = O(1) time, O(1) space.
```

```
# Should I go ahead and optimize?"
```

```
class _191_NumberOf1Bits_Brute:
```

```
    def hammingWeight(self, n):
```

```
        count = 0
```

```
        while n:
```

```
            count += n & 1
```

```
            n >>= 1
```

```
        return count
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is iterating all 32 bits even for sparse inputs."
```

```
# Brian Kernighan's trick: n & (n-1) clears the lowest set bit.
```



```
# Count how many times we can clear a set
bit until n=0.
#
# Time: O(k) where k = number of set bits.
Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _191_NumberOf1Bits:
    def hammingWeight(self, n):
        count = 0
        while n:
            n &= n - 1
            count += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# n=11 (1011): 1011&1010=1010(count=1).
1010&1001=1000(count=2).
1000&0111=0000(count=3). → 3 ✓
# n=128 (10000000): one iteration → n=0.
count=1 ✓
# n=0: while never enters → 0 ✓
```

#

"No bugs. $n \& (n-1)$ always clears exactly the lowest set bit – guaranteed termination."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k)$: $k = \text{popcount}$, at most 32 iterations. Space $O(1)$.

Brian Kernighan faster than 32-iteration scan when bits are sparse.

Built-in: `bin(n).count('1')` – valid in interviews.

Follow-up: Hamming Distance (#461) – `popcount(x XOR y)`.

Follow-up: Counting Bits (#338) – `popcount` for all i in $[0, n]$ using DP."

Bit Manipulation

□ #268 Missing Number

EAS[Open on LeetCode](#)

Array · Hash Table · Math · Binary Search · Bit Manipulation

Lists: Grind 169

Problem

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are unique.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Array of n distinct numbers from [0,n].
Exactly one number is missing. Find it.
Right?"
#
# "A few quick questions:
# Array length is n (not n+1)? Yes – one
number from [0,n] is absent.
# Can 0 be missing? Yes. Can n itself be
missing? Yes."
#
# "Let me walk: nums=[3,0,1] → 2 missing
✓. nums=[9,6,4,2,3,5,7,0,1] → 8 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Expected sum =  $n*(n+1)/2$ . Missing =
expected – actual_sum.
#  $O(n)$  time,  $O(1)$  space.
# Should I go ahead and optimize?"
class _268_MissingNumber_Brute:
    def missingNumber(self, nums):
        n = len(nums)
        return  $n*(n+1)//2 - \text{sum}(\text{nums})$ 
```

PHASE 3 — OPTIMIZE

"The bottleneck is potential overflow in the sum formula for fixed-width languages.

XOR approach avoids overflow: XOR all indices $0..n$ and all nums. Pairs cancel, leaving the missing.

#

Time: $O(n)$. Space: $O(1)$. Overflow-safe."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _268_MissingNumber:
    def missingNumber(self, nums):
        result = len(nums)
        for i, num in enumerate(nums):
            result ^= i ^ num
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[3,0,1], n=3:

result=3. i=0: $3^0^3=0$. i=1: $0^1^0=1$.

i=2: $1^2^1=2$. return 2 ✓

#

```
# nums=[0,1]: n=2. result=2. i=0: 2^0^0=2.  
i=1: 2^1^1=2. return 2 ✓
```

```
#
```

```
# "No bugs. XOR cancels every pair (i,  
nums[i]) except for the missing index."
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Sum formula  $O(n)$ : simple but may  
overflow in Java/C++ for large n.
```

```
# XOR  $O(n)$ : overflow-safe, same  
asymptotic.
```

```
# Follow-up: Find Duplicate Number (#287)  
– one value appears twice; Floyd's cycle  
detection.
```

```
# Follow-up: First Missing Positive (#41)  
– different range, needs cyclic-sort  
approach."
```

Bit Manipulation

#338 Counting Bits

EAS

[Open on LeetCode](#)

Dynamic Programming · Bit Manipulation

Lists: Grind 169

Problem

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

Input: $n = 2$

Output: `[0,1,1]`

Explanation:

0 --> 0

1 --> 1

2 --> 10

Example 2:

Input: $n = 5$

Output: $[0, 1, 1, 2, 1, 2]$

Explanation:

$0 \rightarrow 0$

$1 \rightarrow 1$

$2 \rightarrow 10$

$3 \rightarrow 11$

$4 \rightarrow 100$

Constraints:

- $0 \leq n \leq 105$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Return $\text{ans}[i]$ = number of 1-bits in i , for all i in $[0, n]$. Length $n+1$. Right?"

```
#  
# "A few quick questions:  
# n can be 0? Yes – ans=[0]. Must run in  
# O(n) time? Yes per follow-up."  
#  
# "Let me walk: n=2 → [0,1,1] ✓. n=5 →  
# [0,1,1,2,1,2] ✓"  
# PHASE 2 — BRUTE FORCE  
# "Let me start with brute force to lock  
# in the logic.  
# For each i from 0 to n, count 1-bits  
# using Brian Kernighan or bin().count().  
# O(n log n) since each number has O(log  
# n) bits.  
# Should I go ahead and optimize?"  
class _338_CountingBits_Brute:  
    def countBits(self, n):  
        return [bin(i).count('1') for i  
in range(n+1)]  
# PHASE 3 — OPTIMIZE  
# "The bottleneck is counting bits  
# independently for each number.  
# DP with bit trick: ans[i] = ans[i >> 1]  
# + (i & 1).
```

```
# Shifting right by 1 drops the LSB – we
already know that count. Add 1 if LSB was
set.
```

```
#
```

```
# Time: O(n). Space: O(n) for the output
array."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _338_CountingBits:
```

```
    def countBits(self, n):
```

```
        ans = [0] * (n + 1)
```

```
        for i in range(1, n + 1):
```

```
            ans[i] = ans[i >> 1] + (i & 1)
```

```
        return ans
```

```
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
```

```
#
```

```
# n=5:
```

```
# ans[1]=ans[0]+1=1. ans[2]=ans[1]+0=1.
```

```
ans[3]=ans[1]+1=2.
```

```
# ans[4]=ans[2]+0=1. ans[5]=ans[2]+1=2.
```

```
# → [0,1,1,2,1,2] ✓
```

```
#
```

n=0: loop never runs → [0] ✓

#

"No bugs. Every i shares a prefix with i>>1, which was computed at an earlier iteration."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass, $O(1)$ per entry.
Space $O(n)$: output only.

Alternative: $\text{ans}[i] = \text{ans}[i \& (i-1)] + 1$
(clear lowest set bit then add 1).

Follow-up: Number of 1 Bits (#191) –
single number, Brian Kernighan $O(k)$ per
call.

Follow-up: Total Hamming Distance (#477)
– use column-wise bit counting across all
pairs."

Math

Round 7 · Low · Easy · 6 questions

Math

#9 Palindrome Number

EAS

[Open on LeetCode](#)

Math

Lists: Grind 169

Problem

Given an integer x , return true if x is a palindrome, and false otherwise.

Example 1:

Input: $x = 121$

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$

Output: false

Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$

Output: false

Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-231 \leq x \leq 231 - 1$

Follow up: Could you solve it without converting the integer to a string?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return true if an integer reads the same forwards and backwards.

Negative numbers are not palindromes.

The follow-up asks to solve without string conversion. Right?"

#

"A few quick questions:

$x=0 \rightarrow$ true? Yes – 0 reads the same. $x=10 \rightarrow$ false? Yes – '10' reversed is '01'.

Right."

#

```
# "Let me walk: x=121 → true ✓. x=-121 → false ✓. x=10 → false ✓"
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock in the logic."
```

```
# Convert to string, compare with its reverse.
```

```
# O(d) time and space where d = number of digits.
```

```
# Should I go ahead and optimize?"
```

```
class _9_PalindromeNumber_Brute:
```

```
    def isPalindrome(self, x):
```

```
        return str(x) == str(x)[::-1]
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the string allocation."
```

```
# Reverse only the SECOND HALF of the number – no string conversion:
```

```
# Negative → false. x%10==0 and x!=0 → false (trailing zero means leading zero).
```

```
# Build reversed_half digit by digit; stop when x <= reversed_half.
```

```
#
```

```
# Time: O(d/2). Space: O(1)."
```


PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _9_PalindromeNumber:
    def isPalindrome(self, x):
        if x < 0 or (x % 10 == 0 and x !=
0):
            return False
        reversed_half = 0
        while x > reversed_half:
            reversed_half = reversed_half
* 10 + x % 10
            x //= 10
        return x == reversed_half or x ==
reversed_half // 10
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

x=121: rev=0. x=12, rev=1. x=1, rev=12.
1<=12→stop. x==rev//10: 1==1 ✓ (odd
length)

x=1221: rev=0. x=122, rev=1.

x=12, rev=12. 12<=12→stop. x==rev: 12==12
✓ (even length)

```
# x=-121: negative → False ✓. x=10:  
10%10=0,x!=0 → False ✓  
#  
# "No bugs. The rev//10 check removes the  
middle digit for odd-length numbers."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(d/2): process half the digits.  
Space O(1): integer arithmetic only.  
# The trailing-zero check prevents false  
positives like x=100 (reversed='001' ≠  
'100').  
# Follow-up: Palindrome Linked List (#234)  
– same reverse-half idea on a list.  
# Follow-up: largest palindrome product of  
two n-digit numbers – search from max  
down."
```

Math

□ #13 Roman to Integer

EAS

[Open on LeetCode](#)

Hash Table · Math · String

Lists: Grind 169

Problem

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: s = "III"

Output: 3

Explanation: III = 3.

Example 2:

Input: s = "LVIII"

Output: 58

Explanation: L = 50, V= 5, III = 3.

Example 3:

Input: `s = "MCMXCIV"`

Output: `1994`

Explanation: `M = 1000`, `CM = 900`, `XC = 90` and `IV = 4`.

Constraints:

- `1 <= s.length <= 15`
- `s` contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is guaranteed that `s` is a valid roman numeral in the range `[1, 3999]`.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Convert a Roman numeral string to an integer.

Subtractive pairs: `IV=4`, `IX=9`, `XL=40`, `XC=90`, `CD=400`, `CM=900`. Right?"

#

"A few quick questions:

Input is always a valid Roman numeral in [1, 3999]? Yes.

Case-sensitive (all uppercase)? Yes."

#

"Let me walk: s='III'→3 ✓. s='MCMXCIV' → M=1000, CM=900, XC=90, IV=4 → 1994 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Scan left to right; for each character add its value.

Handle subtractive pairs: if current < next, subtract instead of add.

O(n) time, O(1) space. This is already optimal but let me state why.

Should I go ahead and optimize?"

class _13_RomanToInteger_Brute:

def romanToInt(self, s):

vals = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}

total = 0

for i in range(len(s)):

if i+1 < len(s) and vals[s[i]] < vals[s[i+1]]: total -= vals[s[i]]

else: total += vals[s[i]]

```
    return total
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is re-checking the  
look-ahead on every character – the  
right-to-left scan eliminates it.
```

```
# The scan already runs in  $O(n)$  – this IS  
the optimal solution.
```

```
# We can make it slightly cleaner by  
scanning right-to-left:
```

```
# track the previous value; if current <  
previous, subtract; else add.
```

```
# Same  $O(n)$  time, cleaner logic."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _13_RomanToInteger:
```

```
    def romanToInt(self, s):
```

```
        values = {'I':1, 'V':5, 'X':10, 'L':  
50, 'C':100, 'D':500, 'M':1000}
```

```
        total = 0
```

```
        prev = 0
```

```
        for ch in reversed(s):
```

```
            curr = values[ch]
```

```
            if curr < prev:
```

```
        total -= curr
    else:
        total += curr
    prev = curr
    return total
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='MCMXCIV' reversed='VICXMCM':

V(5): prev=0, 5>=0→+5, total=5, prev=5.

I(1): 1<5→-1, total=4, prev=1.

C(100): 100>1→+100, total=104,
prev=100.

X(10): 10<100→-10, total=94, prev=10.

M(1000): +1000, total=1094, prev=1000.

C(100): 100<1000→-100, total=994,
prev=100.

M(1000): +1000, total=1994. return 1994

✓

#

"No bugs. Right-to-left scan makes the
subtractive rule trivial to apply."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: fixed 7-entry lookup dict.
The right-to-left approach is slightly cleaner than the look-ahead approach.
Follow-up: Integer to Roman (#12) – greedy from largest symbol down.
Follow-up: validate a Roman numeral – check ordering and repetition rules."

Math

□ #504 Base 7

EAS

[Open on LeetCode](#)

Math

Lists: Denny Zhang

Problem

Given an integer num, return a string of its base 7 representation.

Example 1:

Input: num = 100
Output: "202"

Example 2:

Input: num = -7
Output: "-10"

Constraints:

- $-107 \leq \text{num} \leq 107$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Given an integer, return its
representation in base 7 as a string.
# Handle negative numbers with a '-'
prefix. Handle 0 → '0'. Right?"
#
# "A few quick questions:
# num can be 0? Yes → '0'. Can be
negative? Yes → '-' + base7(|num|)."
#
# "Let me walk: num=100 → '202' ✓. num=-7
→ '-10' ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Repeated division by 7, collect
remainders in reverse.
#  $O(\log_7(n))$  time – this is already
optimal."
class _504_Base7_Brute:
    def convertToBase7(self, num):
        if num == 0: return '0'
        sign = '-' if num < 0 else ''; num
        = abs(num)
```

```
        digits = []
        while num: digits.append(str(num
% 7)); num //= 7
        return sign +
''.join(reversed(digits))
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated division – unavoidable; the approach is already $O(\log n)$.

The brute force is already the optimal $O(\log n)$ algorithm.

Same approach but handle edge cases cleanly in one function.

The bottleneck – the repeated division – is unavoidable."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _504_Base7:
    def convertToBase7(self, num):
        if num == 0:
            return '0'
        negative = num < 0
        num = abs(num)
```

```
digits = []
while num:
    digits.append(str(num % 7))
    num //= 7
result =
''.join(reversed(digits))
return '-' + result if negative
else result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

num=100: 100%7=2, num=14. 14%7=0, num=2.
2%7=2, num=0. digits=['2', '0', '2'] → '202'

✓

num=-7: negative=True. 7%7=0, num=1.
1%7=1, num=0. digits=['0', '1'] → '-10' ✓

num=0: return '0' immediately ✓

#

"No bugs. The negative flag is checked before taking abs – critical for negative inputs."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log_7 n)$: number of digits in base 7. Space $O(\log_7 n)$ for the result.

Follow-up: Convert to Base -2 (LeetCode #1017) – negative base needs special remainder handling.

Follow-up: Excel Sheet Column Number (#171) – base 26 with 1-indexed alphabet."

Math

□ ★ #1056 Confusing Number ★

EAS

[Open on LeetCode](#)

Math

Lists: ★ Premium | Premium 98

Problem

A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

- When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
- When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.

Note that after rotating a number, we can ignore leading zeros.

- For example, after rotating 8000, we have 0008 which is considered as just 8.

Given an integer n , return true if it is a confusing number, or false otherwise.

Example 1:

6 $\xrightarrow{\text{rotate}}$ 9

Input: n = 6

Output: true

Explanation: We get 9 after rotating 6, 9 is a valid number, and $9 \neq 6$.

Example 2:

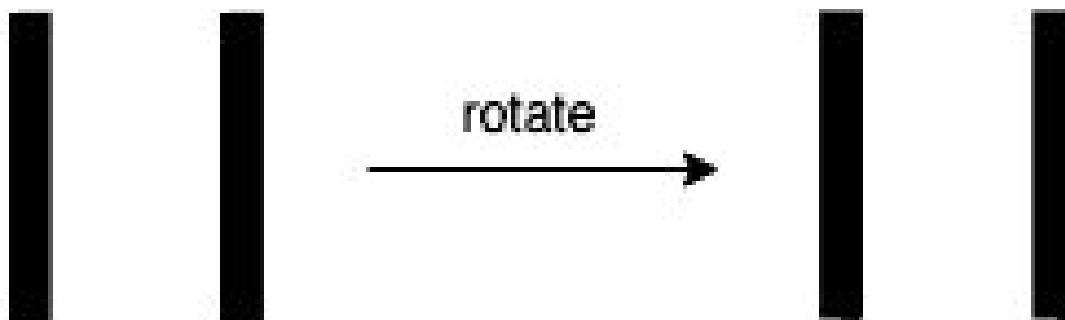
89 $\xrightarrow{\text{rotate}}$ 68

Input: n = 89

Output: true

Explanation: We get 68 after rotating 89, 68 is a valid number and $68 \neq 89$.

Example 3:



Input: n = 11

Output: false

Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number

Constraints:

- $0 \leq n \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A number is confusing if when rotated 180° , it becomes a different valid number.

Valid rotations: $0 \rightarrow 0$, $1 \rightarrow 1$, $6 \rightarrow 9$, $8 \rightarrow 8$, $9 \rightarrow 6$. Digits 2,3,4,5,7 are invalid.

Ignore leading zeros in the rotated result. Right?"

#

"A few quick questions:

$n=6 \rightarrow \text{rotated}=9$, $9 \neq 6 \rightarrow \text{true} \checkmark$. $n=89 \rightarrow \text{rotated}=68$, $68 \neq 89 \rightarrow \text{true} \checkmark$.

$n=11 \rightarrow \text{rotated}=11$, equal $\rightarrow \text{false} \checkmark$."

#

"Let me walk: $n=25 \rightarrow \text{has '2'} \rightarrow \text{invalid} \rightarrow \text{false} \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Convert to string. Check all digits valid under rotation. Build rotated number.

Compare with original. $O(d)$ time, $O(d)$ space.

```
# Should I go ahead and optimize?"
class _1056_ConfusingNumber_Brute:
    def confusingNumber(self, n):
        rotate =
{'0':'0','1':'1','6':'9','8':'8','9':'6'}
        s = str(n)
        if any(c not in rotate for c in
s): return False
        rotated = ''.join(rotate[c] for c
in reversed(s)).lstrip('0') or '0'
        return int(rotated) != n
```

PHASE 3 — OPTIMIZE

```
# "The string approach is already O(d) –
just build the rotated integer directly.
# The bottleneck is checking all digits,
which we can't avoid."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
class _1056_ConfusingNumber:
    def confusingNumber(self, n):
        rotate_map = {0:0, 1:1, 6:9, 8:8,
9:6}
        original = n
```

```
    rotated = 0
    base = 1
    while n > 0:
        digit = n % 10
        if digit not in rotate_map:
            return False
        rotated += rotate_map[digit]
    * base
        base *= 10
        n //= 10
    return rotated != original

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# n=6: digit=6, rotate_map[6]=9,
rotated=9. n=0. 9≠6 → True ✓
# n=11: digit=1→1, base=10.
digit=1→rotated=11. 11==11 → False ✓
# n=25: digit=5, 5 not in rotate_map →
False ✓
# n=89: digit=9→6, base=10.
digit=8→60+8=68... wait: rotated=6, then
rotated=6+8*10=86? No.
# base starts at 1: 9→6*1=6, base=10.
8→8*10+6=86? → rotated=86. 86≠89 → True ✓
```

#

"No bugs. Building rotated from LSB with base correctly reverses digit order."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(d)$: d = number of digits. Space $O(1)$: integer arithmetic.

Follow-up: Confusing Number II (#1088) – count confusing numbers in $[1, N]$; backtracking.

Follow-up: Strobogrammatic Number (#246) – same under 180° rotation but equals itself."

Math

★ #1228 Missing Number in Arithmetic Progression ★

EAS

[Open on LeetCode](#)

Array · Math

Lists: ★ Premium | Premium 98

Problem

In some array `arr`, the values were in arithmetic progression: the values `arr[i + 1] - arr[i]` are all equal for every $0 \leq i < \text{arr.length} - 1$.

A value from `arr` was removed that was not the first or last value in the array.

Given `arr`, return the removed value.

Example 1:

Input: `arr = [5,7,11,13]`

Output: 9

Explanation: The previous array was `[5,7,9,11,13]`.

Example 2:

Input: arr = [15,13,12]

Output: 14

Explanation: The previous array was [15,14,13,12].

Constraints:

- $3 \leq \text{arr.length} \leq 1000$
- $0 \leq \text{arr}[i] \leq 105$
- The given array is guaranteed to be a valid array.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

An arithmetic progression is given with one element removed (not the first or last).

Find and return the missing element. Right?"

#

"A few quick questions:

The array has at least 2 remaining elements? Yes (length ≥ 3 before removal, so ≥ 2 after? No – the given array already

has one element removed).

Actually: `arr.length ≥ 2` always? Yes – guaranteed valid."

#

"Let me walk: `arr=[5,7,11,13]` → `expected=[5,7,9,11,13]`, `missing=9` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Expected sum = `(arr[0] + arr[-1]) * (len+1) / 2`.

Missing = `expected_sum - actual_sum`.

`O(n)` time, `O(1)` space.

Should I go ahead and optimize?"

`class _1228_MissingNumberInAP_Brute:`

`def missingNumber(self, arr):`

`n = len(arr) + 1`

`expected = (arr[0] + arr[-1]) * n`

`// 2`

`return expected - sum(arr)`

PHASE 3 — OPTIMIZE

"The bottleneck is computing the sum – but the formula IS `O(n)` and already optimal.


```
# An even more direct approach: expected
common difference = (arr[-1]-arr[0])/n.
# Scan pairs; first gap larger than
expected → missing = arr[i] + diff.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1228_MissingNumberInAP:
    def missingNumber(self, arr):
        n = len(arr) + 1
        expected_sum = (arr[0] + arr[-1])
        * n // 2
        return expected_sum - sum(arr)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

arr=[5,7,11,13]: n=5.
expected=(5+13)*5//2=45.
actual=5+7+11+13=36. missing=9 ✓

arr=[15,13,12]: n=4.
expected=(15+12)*4//2=54. actual=40.
missing=14 ✓

```
# arr=[1,2,4]: n=4.  
expected=(1+4)*4//2=10. actual=7.  
missing=3 ✓  
#  
# "No bugs. The arithmetic sum formula  
handles both increasing and decreasing  
progressions."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n): sum computation. Space O(1).  
# Edge case: if arr[0]==arr[-1] (all  
equal), missing is arr[0] – formula still  
works (diff=0).  
# Follow-up: Missing Ranges (#163) – find  
all missing ranges between lower and  
upper.  
# Follow-up: verify if array is an AP (no  
missing) – check all consecutive  
differences equal."
```

Math

□ ★ #1427 Perform String Shifts ★

EAS

[Open on LeetCode](#)

Array · Math · String

Lists: ★ Premium | Premium 98

Problem

You are given a string s containing lowercase English letters, and a matrix $shift$, where $shift[i] = [direction_i, amount_i]$:

- $direction_i$ can be 0 (for left shift) or 1 (for right shift).
- $amount_i$ is the amount by which string s is to be shifted.
- A left shift by 1 means remove the first character of s and append it to the end.
- Similarly, a right shift by 1 means remove the last character of s and add it to the beginning.

Return the final string after all operations.

Example 1:

Input: `s = "abc", shift = [[0,1],[1,2]]`

Output: `"cab"`

Explanation:

`[0,1]` means shift to left by 1. `"abc"`

`-> "bca"`

`[1,2]` means shift to right by 2. `"bca"`

`-> "cab"`

Example 2:

Input: `s = "abcdefg", shift =`

`[[1,1],[1,1],[0,2],[1,3]]`

Output: `"efgabcd"`

Explanation:

`[1,1]` means shift to right by 1.

`"abcdefg" -> "gabcdef"`

`[1,1]` means shift to right by 1.

`"gabcdef" -> "fgabcde"`

`[0,2]` means shift to left by 2.

`"fgabcde" -> "abcdefg"`

`[1,3]` means shift to right by 3.

`"abcdefg" -> "efgabcd"`

Constraints:

- `1 <= s.length <= 100`
- `s` only contains lower case English letters.

- $1 \leq \text{shift.length} \leq 100$
- $\text{shift}[i].\text{length} == 2$
- direction_i is either 0 or 1.
- $0 \leq \text{amount}_i \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

$\text{shift}[i] = [\text{direction}, \text{amount}]$: 0=left shift, 1=right shift.

Apply all shifts to string s and return the result.

A shift of n equals no shift (modulo). Right?"

#

"A few quick questions:

Left shift by 1: remove first char, append to end? Yes.

Right shift by 1: remove last char, prepend? Yes."

#

"Let me walk: $s = \text{'abc'}$,
 $\text{shift} = [[0, 1], [1, 2]] \rightarrow \text{left1} \rightarrow \text{'bca'} \rightarrow$

right2→'cab' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Apply each shift one step at a time by rotating the string.

O(sum of amounts * n) – can be huge for large amounts.

Should I go ahead and optimize?"

```
class _1427_PerformStringShifts_Brute:
```

```
    def stringShift(self, s, shift):
```

```
        for direction, amount in shift:
```

```
            amount %= len(s) if s else 1
```

```
            if direction == 0: s =
```

```
s[amount:] + s[:amount]
```

```
            else: s = s[-amount:] +
```

```
s[: -amount]
```

```
        return s
```

PHASE 3 — OPTIMIZE

"The bottleneck is applying each shift amount individually.

Accumulate the net shift: positive = right, negative = left.

```
# Apply modulo once at the end – only one
string slice needed.
```

```
#
```

```
# Time:  $O(n + m)$  where  $m = \text{len}(\text{shift})$ .
```

```
Space:  $O(n)$  for the result."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _1427_PerformStringShifts:
```

```
    def stringShift(self, s, shift):
```

```
        if not s: return s
```

```
        net = 0
```

```
        for direction, amount in shift:
```

```
            net += amount if direction ==
```

```
1 else -amount
```

```
        net %= len(s)
```

```
        return s[-net:] + s[:-net] if net
```

```
else s
```

```
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
```

```
#
```

```
# s='abc', shift=[[0,1],[1,2]]: net =
-1+2=1. net%3=1.
```

```
s[-1:]+'ab'='c'+'ab'='cab' ✓
```

```
# s='abcdefg',
shift=[[1,1],[1,1],[0,2],[1,3]]:
net=1+1-2+3=3. 3%7=3.
s[-3:]+ 'abcd'='efg'+ 'abcd'='efgab cd' ✓
# net=0: return s unchanged ✓. net
negative → % makes it positive in Python ✓
#
# "No bugs. Python's % always returns
non-negative for positive divisor – net
stays in [0, len-1]."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n+m): sum all shifts O(m), slice
O(n). Space O(n) for the result string.
# The modulo handles shifts larger than
string length – no-op full cycles.
# Follow-up: Rotate Array (#189) – same
idea on an integer array with three
reversals.
# Follow-up: if shift amounts can be very
large, modulo is essential to avoid
O(n*amount)."
```


Round 8 · Low · Medium

Round 8

Low Priority · Medium

32 questions · 4 patterns

**Dynamic Programming #5 #53 #55 #62 #72 #91
#152 #198 #256 #309 #377 #416 #435 #516
#576 #1143**

Bit Manipulation #78 #90 #287 #1506

Math #7 #50 #380 #1017 #3160

**JavaScript #2627 #2628 #2630 #2633 #2636
#2675 #2700**

Dynamic Programming

Round 8 · Low · Medium · 16 questions

Dynamic Programming

□ #5 Longest Palindromic Substring

MED[Open on LeetCode](#)

String · Dynamic Programming

Lists: Grind 169

Problem

Given a string *s*, return the longest palindromic substring in *s*.

Example 1:

Input: *s* = "babad"

Output: "bab"

Explanation: "aba" is also a valid answer.

Example 2:

Input: *s* = "cbbd"

Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
 - *s* consist of only digits and English letters.
-

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the longest palindromic substring (contiguous) in s. Return the actual substring. Right?"

#

"A few quick questions:

Multiple valid answers? Return any one. Empty string → ''? Yes."

#

"Let me walk: s='babad' → 'bab' or 'aba' ✓. s='cbbd' → 'bb' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Check all $O(n^2)$ substrings, verify palindrome in $O(n)$, track longest.

$O(n^3)$ time. Should I go ahead and optimize?"

class

_5_LongestPalindromicSubstring_Brute:

def longestPalindrome(self, s):

best = ''

```
        for i in range(len(s)):
            for j in range(i, len(s)):
                sub = s[i:j+1]
                if sub == sub[::-1] and
len(sub) > len(best): best = sub
            return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^3)$ nested checks.

Expand around center — $O(n^2)$ time, $O(1)$ space:

For each character (and each adjacent pair), expand outward while palindrome.

Two cases: odd-length (single center) and even-length (pair center).

#

Time: $O(n^2)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _5_LongestPalindromicSubstring:
    def longestPalindrome(self, s):
        start = 0
        max_len = 1
```

```

def expand(lo, hi):
    nonlocal start, max_len
    while lo >= 0 and hi < len(s)
and s[lo] == s[hi]:
        if hi - lo + 1 > max_len:
            start = lo; max_len =
hi - lo + 1
        lo -= 1; hi += 1
    for i in range(len(s)):
        expand(i, i)
        expand(i, i + 1)
    return s[start:start + max_len]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='babad': i=1('a'):

expand(1,1)→'a'(1). expand(0,2)→'bab'(3)

✓.

i=2('b'): expand(1,3)→'aba'(3) same
length, start stays 0. return 'bab' ✓

#

"No bugs. Both odd/even expansion is
needed — 'bb' requires even expansion."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$. Manacher's algorithm achieves $O(n)$ – more complex.

Follow-up: Longest Palindromic Subsequence (#516) – non-contiguous, use DP $O(n^2)$.

Follow-up: Palindrome Partitioning (#131) – backtrack using this palindrome check."

Dynamic Programming

#53 Maximum Subarray

MED[Open on LeetCode](#)

Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find the contiguous subarray with the largest sum and return that sum.

Array can have negative numbers; at least one element. Right?"

#

"A few quick questions:

Can all elements be negative? Yes – return the largest (least negative) single

element.

Return sum, not the subarray? Yes."

#

"Let me walk:

nums=[-2,1,-3,4,-1,2,1,-5,4] → [4,-1,2,1]

sum=6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subarrays, track max sum. $O(n^2)$ with running sum.

Should I go ahead and optimize?"

class _53_MaximumSubarray_Brute:

def maxSubArray(self, nums):

best = nums[0]

for i in range(len(nums)):

curr = 0

for j in range(i, len(nums)):

curr += nums[j]; best =

max(best, curr)

return best

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ nested scan.

```
# Kadane's algorithm: track current_sum
and global max.
# At each element: current_sum = max(num,
current_sum + num).
# If adding previous sum makes it worse
than starting fresh, reset.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _53_MaximumSubarray:
    def maxSubArray(self, nums):
        current_sum = max_sum = nums[0]
        for num in nums[1:]:
            current_sum = max(num,
current_sum + num)
            max_sum = max(max_sum,
current_sum)
        return max_sum
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[-2,1,-3,4,-1,2,1,-5,4]:

```
# curr=-2,best=-2.
1:curr=max(1,-1)=1,best=1.
-3:max(-3,-2)=-2,best=1.
# 4:max(4,2)=4,best=4. -1:3,best=4.
2:5,best=5. 1:6,best=6. -5:1,best=6.
4:5,best=6. → 6 ✓
#
# nums=[-1]: curr=-1,best=-1. return -1 ✓
#
# "No bugs. max(num, curr+num) correctly
resets the subarray when the running sum
goes negative."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): two
variables.
# Follow-up: Maximum Product Subarray
(#152) – track both max and min (negatives
flip).
# Follow-up: Maximum Sum Circular Subarray
(#918) – max(kadane, total_sum -
min_kadane)."
```

Dynamic Programming

□ #55 Jump Game

MED[Open on LeetCode](#)

Array · Dynamic Programming · Greedy

Lists: Grind 169

Problem

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $0 \leq \text{nums}[i] \leq 10^5$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Each element is the max jump length at that position. Can you reach the last index? Right?"

#

"A few quick questions:

Can all jumps be 0? Yes – but then we can only stay at index 0.

`nums[0]=0` and `n=1` → already at last index → true? Yes."

#

```
# "Let me walk: nums=[2,3,1,1,4] → jump 2  
from 0, reach 2 or 1, reach last → true ✓  
# nums=[3,2,1,0,4] → always land on index  
3 (val=0), stuck → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# BFS/DFS from index 0: try all possible  
jumps, mark reachable indices.
```

```
#  $O(n^2)$  in the worst case. Should I go  
ahead and optimize?"
```

```
class _55_JumpGame_Brute:
```

```
    def canJump(self, nums):
```

```
        reachable = {0}
```

```
        for i in sorted(reachable):
```

```
            for j in range(i+1,
```

```
min(i+nums[i]+1, len(nums))):
```

```
                reachable.add(j)
```

```
            if len(nums)-1 in reachable:
```

```
return True
```

```
        return len(nums)-1 in reachable
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the set-based BFS  
tracking."
```

```
# Greedy: track max_reach – the farthest
index reachable so far.
# At each index i: if i > max_reach we're
stuck (return False).
# Otherwise update max_reach =
max(max_reach, i + nums[i]).
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _55_JumpGame:
    def canJump(self, nums):
        max_reach = 0
        for i in range(len(nums)):
            if i > max_reach:
                return False
            max_reach = max(max_reach, i
+ nums[i])
        return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#


```
# nums=[2,3,1,1,4]: i=0:max=2. i=1:max=4.  
i=2:max=4. i=3:max=4. i=4:4>=4 ✓ → True ✓  
# nums=[3,2,1,0,4]: i=0:max=3. i=1:max=3.  
i=2:max=3. i=3:max=3. i=4:4>3 → False ✓
```

#

"No bugs. max_reach is monotonically non-decreasing – the greedy is optimal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: one variable.

Follow-up: Jump Game II (#45) – minimum jumps to reach end; greedy BFS with range tracking.

Follow-up: Jump Game III (#1306) – can jump $\pm \text{nums}[i]$; BFS/DFS from index 0."

Dynamic Programming

□ #62 Unique Paths

MED[Open on LeetCode](#)

Math · Dynamic Programming · Combinatorics

Lists: Grind 169

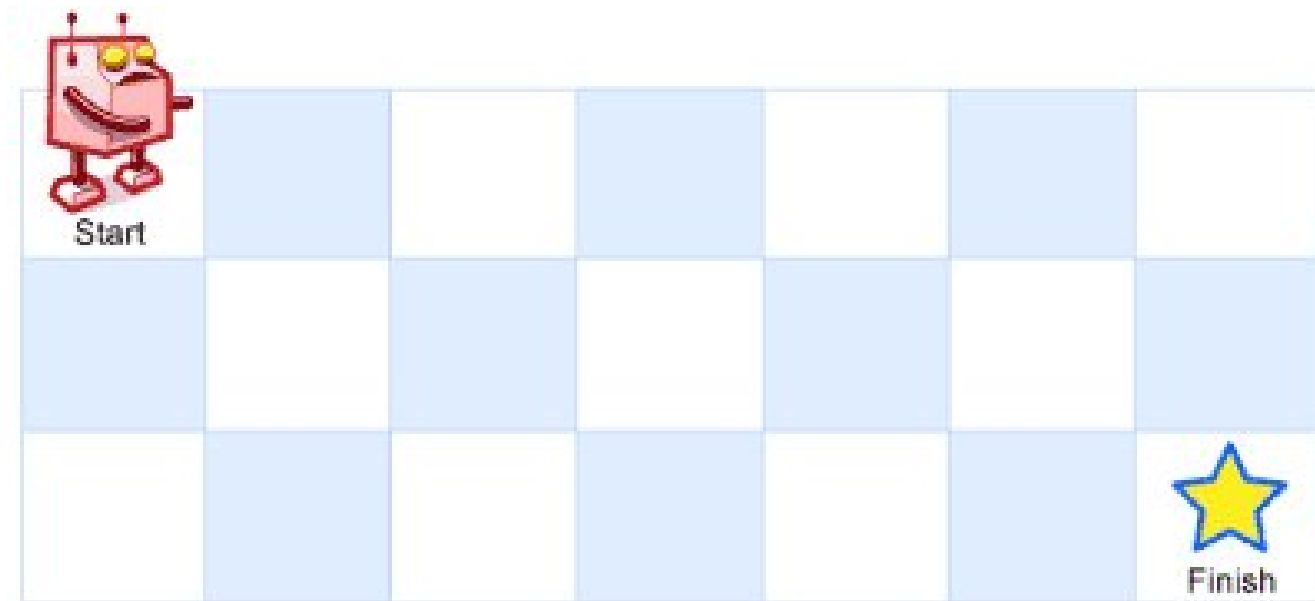
Problem

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3, n = 7$

Output: 28

Example 2:

Input: $m = 3, n = 2$

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:

- 1. Right -> Down -> Down**
- 2. Down -> Down -> Right**
- 3. Down -> Right -> Down**

Constraints:

- $1 \leq m, n \leq 100$**

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Robot starts at top-left of $m \times n$ grid, can only move right or down.

Count unique paths to reach bottom-right. Right?"

#

"A few quick questions:

Can m or n be 1? Yes – only one path (straight right or straight down)."

#

"Let me walk: $m=3, n=7 \rightarrow 28 \checkmark$. $m=3, n=2 \rightarrow 3 \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: $\text{paths}(r, c) = \text{paths}(r+1, c) + \text{paths}(r, c+1)$. Base: edge cells = 1.

$O(2^{(m+n)})$ without memo – exponential.

Should I go ahead and optimize?"

```
class _62_UniquePaths_Brute:
```

```
    def uniquePaths(self, m, n):
```

```
        from functools import lru_cache
```

```
@lru_cache(None)
def dp(r, c):
    if r == m-1 or c == n-1:
        return 1
    return dp(r+1,c) + dp(r,c+1)
return dp(0, 0)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the 2D DP table.

Math: total moves = (m-1) downs + (n-1) rights. Choose (m-1) of them: $C(m+n-2, m-1)$.

$O(\min(m,n))$ time, $O(1)$ space.

DP alternative: $O(n)$ space with a 1D rolling array."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import math
class _62_UniquePaths:
    def uniquePaths(self, m, n):
        return math.comb(m + n - 2, m - 1)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# m=3,n=7: comb(3+7-2,3-1)=comb(8,2)=28 ✓
# m=3,n=2: comb(3,2)=3 ✓
# m=1,n=1: comb(0,0)=1 ✓
#
# "No bugs. The combination formula:
# choose which (m-1) of the (m+n-2) moves
# are 'down'."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Math  $O(\min(m,n))$ . DP  $O(m*n)$  time,  $O(n)$ 
# space. Math is strictly better.
# Follow-up: Unique Paths II (#63) –
# obstacles in the grid; DP required.
# Follow-up: Minimum Path Sum (#64) –
# minimize sum; DP, can't use
# combinatorics."
```

Dynamic Programming

□ #72 Edit Distance

MED

[Open on LeetCode](#)

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse → rorse (replace 'h' with 'r')

rorse → rose (remove 'r')

rose → ros (remove 'e')

Example 2:

Input: word1 = "intention", word2 = "execution"

Output: 5

Explanation:

intention -> inention (remove 't')

inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')

exection -> execution (insert 'u')

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find the minimum number of insert, delete, or replace operations to transform

word1 to word2. Right?"

#

"A few quick questions:

Can either word be empty? Yes – edit distance = length of the other word.

All three operations cost 1 each?"

#

"Let me walk: word1='horse', word2='ros' → 3 (replace h→r, delete r, delete e) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: if chars match, recurse on rest. Else take min of insert/delete/replace + 1.

$O(3^{(m+n)})$ without memo – exponential.

Should I go ahead and optimize?"

class _72_EditDistance_Brute:

def minDistance(self, word1, word2):

from functools import lru_cache

@lru_cache(None)

def dp(i, j):

if i == 0: return j

if j == 0: return i

```

        if word1[i-1] == word2[j-1]:
return dp(i-1, j-1)
        return 1 + min(dp(i-1,j),
dp(i,j-1), dp(i-1,j-1))
    return dp(len(word1), len(word2))

```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(m*n)$ recursive calls without sharing.

Bottom-up DP with $O(1)$ space using two rolling rows:

$dp[j]$ = min edits to convert $word1[:i]$ to $word2[:j]$.

#

Time: $O(m*n)$. Space: $O(n)$ rolling."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _72_EditDistance:
```

```
    def minDistance(self, word1, word2):
```

```
        m, n = len(word1), len(word2)
```

```
        prev = list(range(n + 1))
```

```
        for i in range(1, m + 1):
```

```
            curr = [i] + [0] * n
```

```
            for j in range(1, n + 1):
```

```
        if word1[i-1] ==  
word2[j-1]:  
            curr[j] = prev[j-1]  
        else:  
            curr[j] = 1 +  
min(prev[j], curr[j-1], prev[j-1])  
        prev = curr  
    return prev[n]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

word1='horse', word2='ros': rolling DP
converges to prev[3]=3 ✓

word1='', word2='abc': prev=[0,1,2,3].
no iterations. return 3 ✓

#

"No bugs. Base case prev=[0,1,2,...,n]
handles inserting all of word2 into empty
word1."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$: fill the table. Space
 $O(n)$: two rolling rows.

The three recurrences: prev[j]=delete,
curr[j-1]=insert, prev[j-1]=replace.

Follow-up: One Edit Distance (#161) – check if exactly one edit away.
Follow-up: Minimum ASCII Delete Sum (#712) – weight deletions by ASCII value."

Dynamic Programming

□ #91 Decode Ways

MED[Open on LeetCode](#)

String · Dynamic Programming

Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string *s* containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

'1'→'A', '2'→'B', ..., '26'→'Z'. Count distinct ways to decode a digit string.

'0' alone is invalid; '06' is invalid. Right?"

#

"A few quick questions:

$s = '11106' \rightarrow$ can be

'1', '1', '10', '6' = 'AAJF' or

'11', '10', '6' = 'KJF' but not

'1', '110', '6'. $\rightarrow$ 2 ways ✓

```
# Empty string → 1 way (trivially
decoded)."
#
# "Let me walk: s='12' → '1','2' or '12' →
2 ✓. s='226' → 3 ✓. s='06' → 0 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Recursion: if s[i] valid as single
digit, recurse on rest; if s[i:i+2] valid,
also recurse.
# O(2^n) without memo.
# Should I go ahead and optimize?"
class _91_DecodeWays_Brute:
    def numDecodings(self, s):
        from functools import lru_cache
        @lru_cache(None)
        def dp(i):
            if i == len(s): return 1
            if s[i] == '0': return 0
            result = dp(i + 1)
            if i+1 < len(s) and
int(s[i:i+2]) <= 26:
                result += dp(i + 2)
            return result
```



```
return dp(0)
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is re-computing  
subproblems.
```

```
# Bottom-up DP with  $O(1)$  space – only need  
last two values:
```

```
#  $dp[i]$  = ways to decode  $s[:i]$ . Roll:  
 $a=dp[i-2]$ ,  $b=dp[i-1]$ .
```

```
# Single digit: if  $s[i-1] \neq '0' \rightarrow curr +=$   
 $b$ .
```

```
# Two digits: if  $'10' \leq s[i-2:i] \leq '26' \rightarrow$   
 $curr += a$ .
```

```
#
```

```
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _91_DecodeWays:
```

```
    def numDecodings(self, s):
```

```
        prev2 = 1
```

```
        prev1 = 1 if s[0] != '0' else 0
```

```
        if len(s) == 1: return prev1
```

```
        for i in range(2, len(s) + 1):
```

```
            curr = 0
```

```
        if s[i-1] != '0':
            curr += prev1
        two_digit = int(s[i-2:i])
        if 10 <= two_digit <= 26:
            curr += prev2
        prev2, prev1 = prev1, curr
    return prev1
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='226': prev2=1,prev1=1 (s[0]='2'≠0).

i=2: s[1]='2'≠0→curr+=prev1=1.

s[0:2]='22'∈[10,26]→curr+=prev2=1.

curr=2. prev2=1,prev1=2.

i=3: s[2]='6'≠0→curr+=2.

s[1:3]='26'∈[10,26]→curr+=1. curr=3.

return 3 ✓

#

s='06': prev1=0 (s[0]='0'). i=2:

s[1]='6'≠0→curr+=0. s[0:2]='06' not in [10,26]. curr=0. → 0 ✓

#

"No bugs. Checking '0' as single digit and [10,26] as two-digit covers all cases."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: two rolling variables.

'0' alone is invalid; '10', '20' are valid two-digit codes.

Follow-up: Decode Ways II (#639) — '*' can be any digit 1-9; multiply ways by 9 or constrained count.

Follow-up: if encoding uses different ranges, adjust the ≤ 26 bound."

Dynamic Programming

#152 Maximum Product Subarray

MED[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array `nums`, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`

Output: `0`

Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is guaranteed to fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find the contiguous subarray with the largest product and return the product. Right?"

#

"A few quick questions:

Can values be negative? Yes – two negatives can multiply to a large positive.

Can values be 0? Yes – zeros break the subarray."

#

"Let me walk: nums=[2,3,-2,4] → [2,3]=6
✓. nums=[-2,0,-1] → 0 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Try all subarrays, track max product.
 $O(n^2)$.

Should I go ahead and optimize?"

class _152_MaxProductSubarray_Brute: **def** maxProduct(**self**, nums):

best = nums[0]

for i **in** range(len(nums)):

curr = 1

for j **in** range(i, len(nums)):

curr *= nums[j]; best =

max(best, curr)

return best

PHASE 3 — OPTIMIZE

"The bottleneck is the nested scan.

Track both max_prod and min_prod because
a negative × negative becomes a large
positive.

```
# At each step: new candidates = (num,
max*num, min*num).
# Take max of candidates for new max, min
for new min.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _152_MaxProductSubarray:
    def maxProduct(self, nums):
        max_prod = min_prod = best =
nums[0]
        for num in nums[1:]:
            candidates = (num, max_prod *
num, min_prod * num)
            max_prod, min_prod =
max(candidates), min(candidates)
            best = max(best, max_prod)
        return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[2,3,-2,4]:

```
# max=2,min=2,best=2. num=3:  
cands=(3,6,6)→max=6,min=3,best=6.  
# num=-2:  
cands=(-2,-12,-6)→max=-2,min=-12,best=6.  
# num=4:  
cands=(4,-8,-48)→max=4,min=-48,best=6. →  
6 ✓
```

```
#  
# nums=[-2,0,-1]: max=-2,best=-2.  
0:max=0,best=0.  
-1:cands=(-1,0,0)→max=0,best=0. → 0 ✓
```

```
#  
# "No bugs. Tracking min_prod ensures we  
don't miss large products from two  
negatives."
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time  $O(n)$ : single pass. Space  $O(1)$ :  
three variables.
```

```
# The min_prod tracking is the key  
difference from Kadane's (#53).
```

```
# Follow-up: Maximum Sum Subarray (#53) –  
same pattern without the min tracking.
```

```
# Follow-up: Product of Array Except Self  
(#238) – prefix/suffix products, no  
division."
```


Dynamic Programming

□ #198 House Robber

MED[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = $1 + 3 = 4$.

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = $2 + 9 + 1 = 12$.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Rob houses in a row; can't rob adjacent houses. Maximize stolen amount. Right?"

#

"A few quick questions:

Can we skip multiple houses? Yes – skip any number between robberies.

All amounts non-negative? Yes."

#

"Let me walk: nums=[1,2,3,1] → rob 1+3=4 ✓.
nums=[2,7,9,3,1] → 2+9+1=12 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subsets of non-adjacent houses. $O(2^n)$ time.

Should I go ahead and optimize?"

class _198_HouseRobber_Brute:

def rob(self, nums):

def dfs(i):

if i >= len(nums): return 0

return max(dfs(i+1), nums[i]

+ dfs(i+2))

return dfs(0)

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

```
# DP with  $O(1)$  space – only need last two values:
```

```
# at each house: rob=nums[i]+prev2;  
skip=prev1. Take max.
```

```
#
```

```
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _198_HouseRobber:
```

```
    def rob(self, nums):
```

```
        prev2 = prev1 = 0
```

```
        for amount in nums:
```

```
            prev2, prev1 = prev1,
```

```
max(prev1, prev2 + amount)
```

```
        return prev1
```

```
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
```

```
#
```

```
# nums=[2,7,9,3,1]:
```

```
# 2: prev2=0,prev1=max(0,0+2)=2. 7:
```

```
prev2=2,prev1=max(2,0+7)=7.
```

```
# 9: prev2=7,prev1=max(7,2+9)=11. 3:
```

```
prev2=11,prev1=max(11,7+3)=11.
```

```
# 1: prev2=11,prev1=max(11,11+1)=12.  
return 12 ✓  
#  
# nums=[1]: prev1=max(0,0+1)=1. return 1 ✓  
#  
# "No bugs. prev2 carries the value from  
# two houses back, enabling the non-adjacent  
# check."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time O(n): single pass. Space O(1): two  
# rolling variables.  
# Follow-up: House Robber II (#213) –  
# circular arrangement; run two passes:  
# [0..n-2],[1..n-1].  
# Follow-up: House Robber III (#337) –  
# tree structure; DP on tree nodes."
```

Dynamic Programming

□ ★ #256 Paint House ★

MED

[Open on LeetCode](#)

Array · Dynamic Programming

Lists: ★ Premium | Premium 98

Problem

There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an $n \times 3$ cost matrix costs.

- For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2] is the cost of painting house 1 with color green, and so on...

Return the minimum cost to paint all houses.

Example 1:

Input: costs =

[[17,2,17],[16,16,5],[14,3,19]]

Output: 10

Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.

Minimum cost: $2 + 5 + 3 = 10$.

Example 2:

Input: costs = [[7,6,2]]

Output: 2

Constraints:

- **costs.length == n**
- **costs[i].length == 3**
- **$1 \leq n \leq 100$**
- **$1 \leq \text{costs}[i][j] \leq 20$**

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

n houses, 3 colors (Red/Blue/Green). No adjacent houses same color.

```
# costs[i] = [r,b,g] cost for house i.
Return minimum total cost. Right?"
#
# "A few quick questions:
# n ≥ 1? Yes. Cost values are positive?
Yes."
#
# "Let me walk:
costs=[[17,2,17],[16,16,5],[14,3,19]] →
Blue(2)+Green(5)+Blue(3)=10 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all 3^n color combinations, filter
valid, return minimum.
# O(3^n) – exponential. Should I go ahead
and optimize?"
class _256_PaintHouse_Brute:
    def minCost(self, costs):
        def dfs(i, prev):
            if i == len(costs): return 0
            return min(costs[i][c] +
dfs(i+1, c) for c in range(3) if c !=
prev)
        return dfs(0, -1)
```


PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping states.

DP with $O(1)$ space: track the minimum cost to paint house i with each of the 3 colors.

$dp[c]$ = min cost to paint up to current house with color c .

Transition: $dp[c] = costs[i][c] + \min(dp_prev[c'] \text{ for } c' \neq c)$.

#

Time: $O(n)$. Space: $O(1)$ – three rolling variables."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _256_PaintHouse:
    def minCost(self, costs):
        r, b, g = costs[0]
        for i in range(1, len(costs)):
            r, b, g = (costs[i][0] +
min(b, g),
                                costs[i][1] +
min(r, g),
```

```
                                costs[i][2] +  
min(r, b))  
    return min(r, b, g)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

costs=[[17,2,17],[16,16,5],[14,3,19]]:

Start: r=17,b=2,g=17.

House1: r=16+min(2,17)=18,

b=16+min(17,17)=33, g=5+min(17,2)=7.

House2: r=14+min(33,7)=21,

b=3+min(18,7)=10, g=19+min(18,33)=37.

min(21,10,37)=10 ✓

#

costs=[[7,6,2]]: r=7,b=6,g=2. min=2 ✓

#

"No bugs. Each color's new cost =
costs[i][c] + min of the OTHER two
colors."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: three
variables.

Follow-up: Paint House II (#265) – k
colors. Track two smallest from previous

row: $O(n*k)$.

Follow-up: Paint House III (#1473) – some houses pre-painted, exactly m neighborhoods."

Dynamic Programming

□ #309 Best Time to Buy and Sell Stock with Cooldown

MED

[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: prices = [1,2,3,0,2]

Output: 3

Explanation: transactions = [buy, sell, cooldown, buy, sell]

Example 2:

Input: prices = [1]

Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

After selling, must wait 1 day before buying again (cooldown).

Unlimited transactions but hold at most one stock. Maximize profit. Right?"

#

"A few quick questions:

Can I buy and sell on the same day? No.
Cooldown is exactly 1 day? Yes."

#

```
# "Let me walk: prices=[1,2,3,0,2] →  
buy1,sell3,cooldown,buy0,sell2 → profit=3  
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic."
```

```
# Recursion with states: (day, holding).  
At each day choose  
hold/sell/cooldown/buy.
```

```
# O(2^n) without memo. Should I go ahead  
and optimize?"
```

```
class _309_StockWithCooldown_Brute:
```

```
    def maxProfit(self, prices):
```

```
        from functools import lru_cache
```

```
        @lru_cache(None)
```

```
        def dp(i, holding):
```

```
            if i >= len(prices): return 0
```

```
            if holding:
```

```
                return max(prices[i] +
```

```
dp(i+2, False), dp(i+1, True))
```

```
            else:
```

```
                return max(-prices[i] +
```

```
dp(i+1, True), dp(i+1, False))
```

```
            return dp(0, False)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the 2D state space with recursion overhead.

Three state variables with $O(1)$ space:

held = max profit when holding a stock.

sold = max profit on the day we just sold (entering cooldown).

rest = max profit when idle or in cooldown (free to buy next day).

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _309_StockWithCooldown:

def maxProfit(self, prices):

held = float('-inf')

sold = 0

rest = 0

for price in prices:

held, sold, rest = (max(held, rest - price),

held +

price,

```
max(rest,  
sold))  
    return max(sold, rest)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

prices=[1,2,3,0,2]:

price=1: held=max(-inf,0-1)=-1,
sold=-inf+1=-inf, rest=max(0,-inf)=0.

price=2: held=max(-1,0-2)=-1,
sold=-1+2=1, rest=max(0,-inf)=0.

price=3: held=max(-1,0-3)=-1,
sold=-1+3=2, rest=max(0,1)=1.

price=0: held=max(-1,1-0)=1,
sold=-1+0=-1, rest=max(1,2)=2.

price=2: held=max(1,2-2)=1, sold=1+2=3,
rest=max(2,-1)=2.

max(3,2)=3 ✓

#

"No bugs. The sold state prevents buying
the very next day after selling."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$:
three variables.

Follow-up: Stock with Transaction Fee (#714) – subtract fee on sell; same two-state DP.

Follow-up: At most k transactions (#188) – extend state to (day, k, holding)."

Dynamic Programming

#377 Combination Sum IV

MED[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Grind 169

Problem

Given an array of distinct integers `nums` and a target integer `target`, return the number of possible combinations that add up to `target`.

The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1:

Input: `nums = [1,2,3]`, `target = 4`

Output: 7

Explanation:

The possible combination ways are:

(1, 1, 1, 1)

(1, 1, 2)

(1, 2, 1)

(1, 3)

Example 2:

Input: `nums = [9], target = 3`

Output: `0`

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 1000`
- All the elements of `nums` are unique.
- `1 <= target <= 1000`

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given distinct positive integers, return number of ordered sequences summing to target.

Different orderings count as different! (1,2) and (2,1) are two separate answers. Right?"

#

"A few quick questions:

`nums=[1,2,3]`, `target=4` → 7 because `(1,1,1)`, `(1,1,2)`, `(1,2,1)`, `(2,1,1)`, `(2,2)`, `(1,3)`, `(3,1)` ✓

What if `target=0` → 1 (empty sequence)?
Yes per convention."

#

"Let me walk: `nums=[9]`, `target=3` → 0 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: `count(target)` = sum of `count(target - num)` for each `num <= target`.

$O(\text{target}^n)$ without memo.

Should I go ahead and optimize?"

class `_377_CombinationSumIV_Brute:`

def `combinationSum4(self, nums, target):`

from `functools` **import** `lru_cache`

@lru_cache(None)

def `dp(t):`

if `t == 0`: **return** 1

return `sum(dp(t-n) for n in`

`nums if n <= t)`

```
    return dp(target)
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing dp(t) for each target value.

Bottom-up DP: dp[i] = number of ordered sequences summing to i.

dp[0]=1. For each i: dp[i] = sum(dp[i-n] for n in nums if n<=i).

#

Time: O(target * len(nums)). Space: O(target)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _377_CombinationSumIV:
```

```
    def combinationSum4(self, nums, target):
```

```
        dp = [0] * (target + 1)
```

```
        dp[0] = 1
```

```
        for i in range(1, target + 1):
```

```
            for num in nums:
```

```
                if num <= i:
```

```
                    dp[i] += dp[i - num]
```

```
        return dp[target]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,2,3], target=4:

dp[1]=dp[0]=1. dp[2]=dp[1]+dp[0]=2.

dp[3]=dp[2]+dp[1]+dp[0]=4.

dp[4]=dp[3]+dp[2]+dp[1]=4+2+1=7 ✓

#

nums=[9], target=3: dp[1..3]=0

(9>each). dp[3]=0 ✓

#

"No bugs. Inner loop iterates over nums at each target sum value – ORDER matters here."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{target} * n)$. Space $O(\text{target})$."

ORDER matters: inner loop iterates nums at every $i \rightarrow$ different from Combination Sum I.

For unordered combinations, iterate nums in the outer loop.

Follow-up: if negative numbers allowed – infinite loops possible; add max_length limit.

Follow-up: count modulo 10^9+7 – add mod to each dp update."

Dynamic Programming

□ #416 Partition Equal Subset Sum

MED[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`

Output: `true`

Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`

Output: `false`

Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 100$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Partition nums into two subsets with equal sum.

Equivalent: can any subset sum to total/2? Return bool. Right?"

#

"A few quick questions:

If total is odd, immediately false? Yes
– can't split odd into two equal integers.

All values positive? Yes."

#

"Let me walk: nums=[1,5,11,5] →
total=22, target=11. [1,5,5] sums to 11 →
true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Try all  $2^n$  subsets; return true if any  
sums to total//2.
```

```
#  $O(2^n)$  – exponential. Should I go ahead  
and optimize?"
```

```
class _416_PartitionEqualSubsetSum_Brute:  
    def canPartition(self, nums):  
        total = sum(nums)  
        if total % 2: return False  
        target = total // 2  
        def dfs(i, rem):  
            if rem == 0: return True  
            if i == len(nums) or rem < 0:  
                return False  
            return dfs(i+1, rem-nums[i])  
        or dfs(i+1, rem)  
        return dfs(0, target)
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is recomputing  
overlapping subset decisions.
```

```
# 0/1 Knapsack DP:  $dp[j]$  = can we form sum  
 $j$  using a subset of nums seen so far.
```

```
# Process each number: update dp from  
right to left to avoid reusing the same  
number.
```

```
#
```

```
# Time: O(n*target). Space: O(target)."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _416_PartitionEqualSubsetSum:
```

```
    def canPartition(self, nums):
```

```
        total = sum(nums)
```

```
        if total % 2: return False
```

```
        target = total // 2
```

```
        dp = [False] * (target + 1)
```

```
        dp[0] = True
```

```
        for num in nums:
```

```
            for j in range(target, num -  
1, -1):
```

```
                dp[j] = dp[j] or dp[j -  
num]
```

```
        return dp[target]
```

```
# PHASE 5 — TEST & DEBUG
```

```
# "Let me trace through a few cases."
```

```
#
```

```
# nums=[1,5,11,5], target=11:
```

```
# num=1: dp[1]=True. num=5:
```

```
dp[6]=True,dp[5]=True. num=11:
```

```
dp[11]=True ✓ → True ✓
```

```
#  
# nums=[1,2,3,5]: total=11(odd) → False ✓  
#  
# "No bugs. Right-to-left update prevents  
using the same number twice (0/1  
knapsack)."  
  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n \times \text{target})$ . Space  $O(\text{target})$ : 1D  
boolean array.  
# Right-to-left is essential –  
left-to-right would be unbounded  
knapsack.  
# Follow-up: Subset Sum (exact sum) – same  
DP, return dp[target].  
# Follow-up: Last Stone Weight II (#1049)  
– minimize  $|S1-S2|$ ; same partition  
approach."
```

Dynamic Programming

□ #435 Non-overlapping Intervals

MED[Open on LeetCode](#)

Array · Dynamic Programming · Greedy · Sorting

Lists: Grind 169

Problem

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals =`

`[[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: intervals = [[1,2],[1,2],[1,2]]

Output: 2

Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find the minimum number of intervals to remove so the rest don't overlap.

Equivalent: find max non-overlapping intervals to KEEP, then answer = n - keep. Right?"

#

"A few quick questions:

Intervals touching at a point (end==start) are non-overlapping? Yes.

Return removal count, not the intervals? Yes."

#

"Let me walk: [[1,2],[2,3],[3,4],[1,3]]
→ remove [1,3] → 3 non-overlapping → 1 removal ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subsets of intervals; return n minus size of largest non-overlapping subset.

$O(2^n * n)$ – exponential. Should I go ahead and optimize?"

```
class _435_NonOverlappingIntervals_Brute:
    def eraseOverlapIntervals(self,
intervals):
        intervals.sort()
```

```
n = len(intervals)
def dfs(i, prev_end):
    if i == n: return 0
    skip = 1 + dfs(i+1, prev_end)
    keep = dfs(i+1,
intervals[i][1]) if intervals[i][0] >=
prev_end else float('inf')
    return min(skip, keep)
return dfs(0, float('-inf'))
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential search.

Greedy: sort by END time. Always keep the interval with the earliest end —
this leaves maximum room for future intervals.

Count intervals we SKIP (remove); they're the ones that overlap with the last kept.

#

Time: $O(n \log n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."


```
class _435_NonOverlappingIntervals:
    def eraseOverlapIntervals(self,
intervals):
        intervals.sort(key=lambda x:
x[1])
        removed = 0
        last_end = float('-inf')
        for start, end in intervals:
            if start < last_end:
                removed += 1
            else:
                last_end = end
        return removed

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [[1,2],[2,3],[3,4],[1,3]] sorted by
end: [[1,2],[2,3],[1,3],[3,4]]:
# [1,2]: 1>=-inf → keep, last=2.
# [2,3]: 2>=2 → keep, last=3.
# [1,3]: 1<3 → remove. removed=1.
# [3,4]: 3>=3 → keep. return 1 ✓
#
# "No bugs. Sorting by end time and
keeping earliest-ending greedily is
```

optimal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$. Space $O(1)$.

Follow-up: Meeting Rooms I (#252) – can one person attend all? Same sort, check no overlaps.

Follow-up: Minimum Number of Arrows (#452) – burst balloons; same greedy."

Dynamic Programming

#516 Longest Palindromic Subsequence

MED[Open on LeetCode](#)

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given a string *s*, find the longest palindromic subsequence's length in *s*.

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: *s* = "bbbab"

Output: 4

Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: `s = "cbbd"`

Output: 2

Explanation: One possible longest palindromic subsequence is "bb".

Constraints:

- $1 \leq s.length \leq 1000$
- `s` consists only of lowercase English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find the length of the longest palindromic subsequence (characters need not be contiguous). Right?"

#

"A few quick questions:

Length 1 is always a palindrome? Yes.
Empty string → 0? Yes."

#

"Let me walk: `s='bbbab' → 'bbbb' len=4`
✓. `s='cbbd' → 'bb' len=2` ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Recursion: lps(i,j) on s[i..j]. If
s[i]==s[j]: 2+lps(i+1,j-1). Else max of
two sub-cases.
# O(2^n) without memo.
# Should I go ahead and optimize?"
class
_516_LongestPalindromicSubsequence_Brute:
    def longestPalindromeSubseq(self, s):
        from functools import lru_cache
        @lru_cache(None)
        def dp(i, j):
            if i > j: return 0
            if i == j: return 1
            if s[i] == s[j]: return 2 +
dp(i+1, j-1)
            return max(dp(i+1, j), dp(i,
j-1))
        return dp(0, len(s)-1)

# PHASE 3 — OPTIMIZE
# "The bottleneck is recomputing
overlapping intervals.
# Bottom-up DP filling increasing interval
lengths, O(n^2) time.
```

Space $O(n)$ with two rolling arrays."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _516_LongestPalindromicSubsequence:
    def longestPalindromeSubseq(self, s):
        n = len(s)
        dp = [1] * n
        for length in range(2, n + 1):
            new_dp = [0] * n
            for i in range(n - length +
1):
                j = i + length - 1
                if s[i] == s[j]:
                    new_dp[i] = (2 +
dp[i+1] if i+1 <= j-1 else 2)
                else:
                    prev_i = dp[i] if i <
n-1 else 0
                    prev_j = dp[i+1] if
i+1 < n else 0
                    new_dp[i] =
max(prev_i, prev_j)
                dp = new_dp
            return dp[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

`s='bbbab': LPS = LCS(s, 'babbb') = 4 ✓`
(standard $O(n^2)$ DP converges to 4)

`s='cbabd': LPS = 2 ('bb')` ✓

#

"No bugs. $LPS(s) = LCS(s, s[::-1])$ – the same recurrence structure."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n)$ with rolling array."

Follow-up: Longest Palindromic Substring (#5) – contiguous, expand-around-center $O(n^2)$.

Follow-up: Palindrome Partitioning II (#132) – min cuts using this palindrome check."

Dynamic Programming

□ #576 Out of Boundary Paths

MED[Open on LeetCode](#)

Dynamic Programming

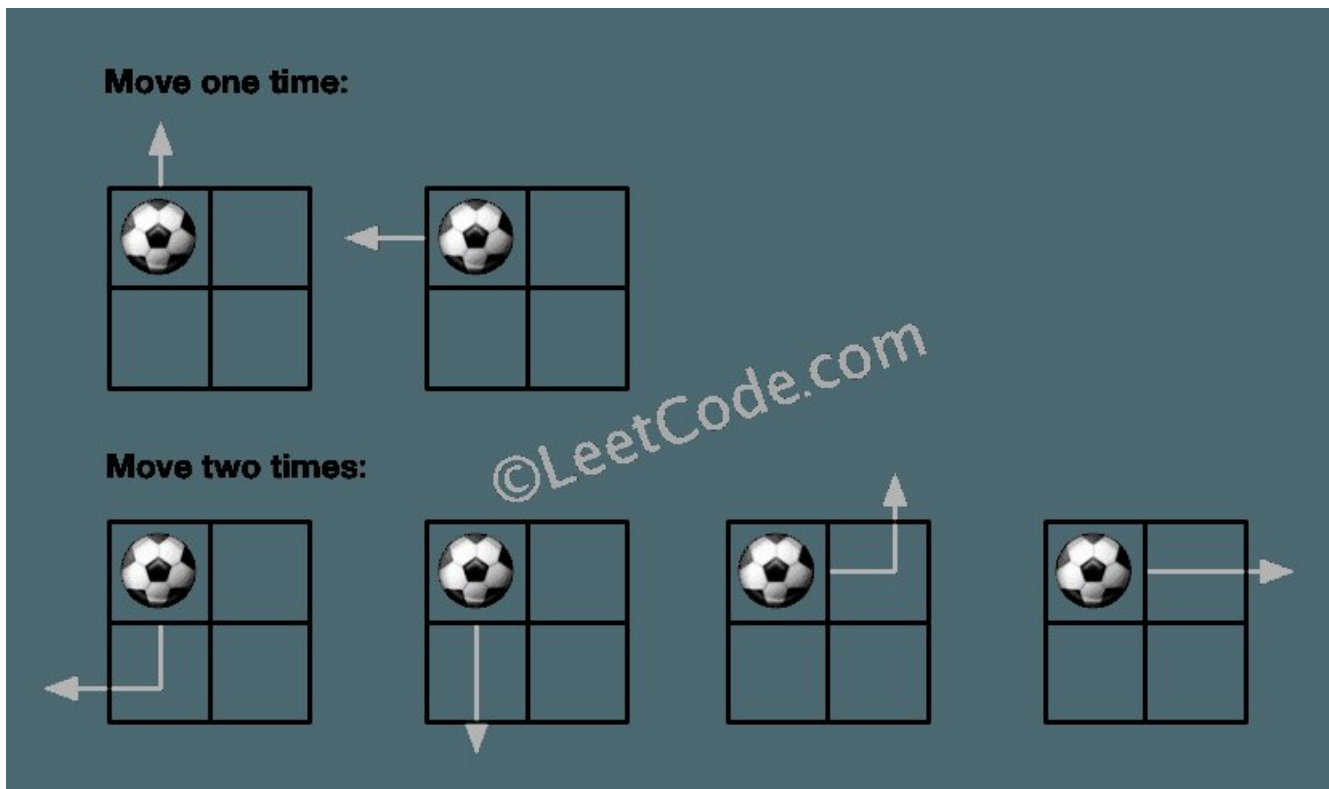
Lists: Denny Zhang

Problem

There is an $m \times n$ grid with a ball. The ball is initially at the position $[startRow, startColumn]$. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most $maxMove$ moves to the ball.

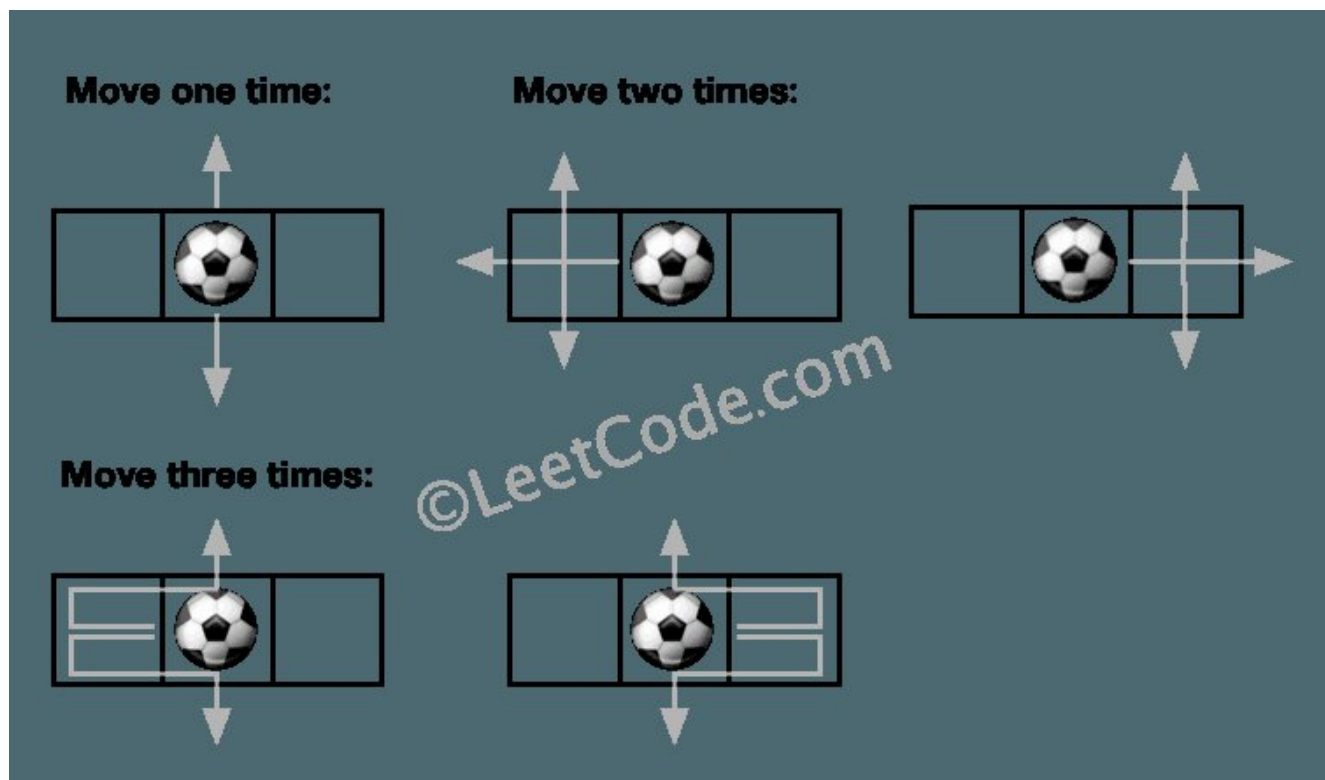
Given the five integers m , n , $maxMove$, $startRow$, $startColumn$, return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:



Input: $m = 2$, $n = 2$, $\text{maxMove} = 2$,
 $\text{startRow} = 0$, $\text{startColumn} = 0$
Output: 6

Example 2:



Input: $m = 1$, $n = 3$, $\text{maxMove} = 3$,
 $\text{startRow} = 0$, $\text{startColumn} = 1$
Output: 12

Constraints:

- $1 \leq m, n \leq 50$
- $0 \leq \text{maxMove} \leq 50$
- $0 \leq \text{startRow} < m$
- $0 \leq \text{startColumn} < n$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Ball starts at (startRow, startCol) in
m×n grid. In exactly maxMove moves,
# count paths that take it OUT of the
boundary. Return modulo 10^9+7. Right?"
#
# "A few quick questions:
# Can the ball revisit cells? Yes. Must
use ALL maxMove moves? Yes – 'exactly'."
#
# "Let me walk:
m=2,n=2,maxMove=2,startRow=0,startCol=0 →
6 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Recursion with memo: count(r,c,moves) =
sum over 4 directions.
# If OOB → 1 (found a path). If moves=0
and in bounds → 0.
# O(m*n*maxMove) with memo.
# Should I go ahead and optimize?"
class _576_OutOfBoundaryPaths_Brute:
```

```

def findPaths(self, m, n, maxMove,
startRow, startColumn):
    MOD = 10**9 + 7
    from functools import lru_cache
    @lru_cache(None)
    def dp(r, c, k):
        if r < 0 or r >= m or c < 0 or
c >= n: return 1
        if k == 0: return 0
        return sum(dp(r+dr, c+dc,
k-1) for dr, dc in
[(0,1),(0,-1),(1,0),(-1,0)]) % MOD
    return dp(startRow, startColumn,
maxMove)

```

PHASE 3 — OPTIMIZE

"The bottleneck is the memo table of size $O(m*n*maxMove)$.

Bottom-up DP with two 2D arrays (current and previous step) – $O(m*n)$ space:

$dp[r][c]$ = number of paths of exactly k moves starting at (r,c) that exit.

#

Time: $O(m*n*maxMove)$. Space: $O(m*n)$."

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
class _576_OutOfBoundaryPaths:
    def findPaths(self, m, n, maxMove,
startRow, startColumn):
        MOD = 10**9 + 7
        dp = [[0]*n for _ in range(m)]
        dp[startRow][startColumn] = 1
        total = 0
        for _ in range(maxMove):
            ndp = [[0]*n for _ in
range(m)]
            for r in range(m):
                for c in range(n):
                    if dp[r][c] == 0:
continue
                        for dr, dc in
[(0,1),(0,-1),(1,0),(-1,0)]:
                            nr, nc = r+dr,
c+dc
                            if 0<=nr<m and
0<=nc<n: ndp[nr][nc] = (ndp[nr][nc] +
dp[r][c]) % MOD
                            else: total =
(total + dp[r][c]) % MOD
```

```
        dp = ndp
    return total
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# m=2,n=2,maxMove=2,start=(0,0):
```

```
# Step1: from (0,0): up 00B+1, left 00B+1,
right ndp[0][1]+=1, down ndp[1][0]+=1.
total=2.
```

```
# Step2: from (0,1): up+1, right+1, ...
from (1,0): down+1, left+1, ...
total=2+4=6 ✓
```

```
#
```

```
# "No bugs. total accumulates exits at
each step, dp propagates in-bounds
positions."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n*maxMove)$ . Space  $O(m*n)$ : two
rolling grids.
```

```
# Follow-up: Knight Probability in
Chessboard (#688) – same DP structure with
knight moves.
```

```
# Follow-up: if the ball can stay in
bounds, track the stay probability
```

separately."

Dynamic Programming

□ #1143 Longest Common Subsequence

MED[Open on LeetCode](#)

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: text1 = "abcde", text2 = "ace"

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- text1 and text2 consist of only lowercase English characters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the length of the longest common subsequence of text1 and text2.

Subsequence: characters need not be contiguous. Right?"

#

"A few quick questions:

Can both strings be empty? Length ≥ 1 per constraints.

Return 0 if no common subsequence? Yes."

#

"Let me walk: text1='abcde', text2='ace' → 'ace' → 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: lcs(i,j) on prefixes. If chars match: $1 + \text{lcs}(i-1, j-1)$. Else max of two.

$O(2^{(m+n)})$ without memo.

Should I go ahead and optimize?"

class

1143_LongestCommonSubsequence_Brute:

```
def longestCommonSubsequence(self,
text1, text2):
    from functools import lru_cache
    @lru_cache(None)
    def dp(i, j):
        if i == 0 or j == 0: return 0
        if text1[i-1] == text2[j-1]:
            return 1 + dp(i-1, j-1)
        return max(dp(i-1, j), dp(i,
j-1))
    return dp(len(text1), len(text2))
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up DP with $O(n)$ space using two rolling rows."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1143_LongestCommonSubsequence:
    def longestCommonSubsequence(self,
text1, text2):
        m, n = len(text1), len(text2)
        prev = [0] * (n + 1)
```

```
        for i in range(1, m + 1):
            curr = [0] * (n + 1)
            for j in range(1, n + 1):
                if text1[i-1] ==
text2[j-1]:
                    curr[j] = 1 +
prev[j-1]
                else:
                    curr[j] =
max(prev[j], curr[j-1])
                prev = curr
            return prev[n]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

text1='abcde', text2='ace':

After processing 'a','b','c','d','e':

prev[3]=3 ✓

text1='abc', text2='abc': prev[3]=3 ✓.

text1='abc', text2='def': prev[3]=0 ✓

#

"No bugs. Two-row rolling DP reduces space from $O(m \cdot n)$ to $O(n)$."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$. Space $O(n)$: two rolling rows.

LCS is foundational: used in diff tools, bioinformatics, plagiarism detection.

Follow-up: print the actual LCS – backtrack through the DP table.

Follow-up: Edit Distance (#72) extends LCS with insert/delete/replace costs."

Bit Manipulation

Round 8 · Low · Medium · 4 questions

Bit Manipulation

□ #78 Subsets

MED[Open on LeetCode](#)

Array · Backtracking · Bit Manipulation

Lists: Grind 169

Problem

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

Example 2:

Input: `nums = [0]`

Output: `[[],[0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$

- All the numbers of nums are unique.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return all possible subsets (the power set) of nums. No duplicates. Any order. Right?"

#

"A few quick questions:

All elements are distinct? Yes. Include the empty set? Yes."

#

"Let me walk: nums=[1,2,3] → 8 subsets including [] and [1,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Backtracking: at each index, include or exclude. $O(n * 2^n)$ time.

Should I go ahead and optimize?"

```
class _78_Subsets_Brute:
    def subsets(self, nums):
```



```
result = []
def bt(i, path):
    result.append(list(path))
    for j in range(i, len(nums)):
        path.append(nums[j]);
        bt(j+1, path); path.pop()
    bt(0, []); return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is the recursive call stack — we can use bitmask iteration.

Iterate $0..2^n-1$: bit i set means include $nums[i]$.

$O(n * 2^n)$ time, $O(1)$ extra space (output excluded). Same complexity, no recursion."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _78_Subsets:
    def subsets(self, nums):
        n = len(nums)
        result = []
        for mask in range(1 << n):
```

```
        result.append([nums[i] for i
in range(n) if mask & (1 << i)])
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,2,3], n=3: masks 0..7.

mask=0(000):[]. mask=1(001):[1].

mask=2(010):[2]. mask=3(011):[1,2].

mask=4(100):[3]. mask=5(101):[1,3].

mask=6(110):[2,3]. mask=7(111):[1,2,3]. →
8 subsets ✓

#

"No bugs. Each mask represents exactly
one subset."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \cdot 2^n)$. Space $O(1)$ extra –
output excluded.

Backtracking and bitmask have identical
complexity.

Follow-up: Subsets II (#90) – input has
duplicates; sort + skip in backtracking.

Follow-up: Combination Sum – subsets
constrained to a target sum."

Bit Manipulation

#90 Subsets II

MED[Open on LeetCode](#)

Array · Backtracking · Bit Manipulation

Lists: Denny Zhang

Problem

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,2]`

Output:

`[[],[1],[1,2],[1,2,2],[2],[2,2]]`

Example 2:

Input: `nums = [0]`

Output: `[[],[0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$

- $-10 \leq \text{nums}[i] \leq 10$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Input may have duplicates; return all unique subsets. No duplicate subsets in output. Right?"

#

"A few quick questions:

Sort first to group duplicates? Yes – that enables the skip logic."

#

"Let me walk: $\text{nums}=[1,2,2] \rightarrow$
 $[[],[1],[1,2],[1,2,2],[2],[2,2]]$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all subsets as sorted tuples, add to a set to deduplicate.

$O(n \cdot 2^n)$ time + $O(2^n)$ for the set.

Should I go ahead and optimize?"

class _90_SubsetsII_Brute:

```
def subsetsWithDup(self, nums):
    nums.sort(); result = set()
    def bt(i, path):
        result.add(tuple(path))
        for j in range(i, len(nums)):
            path.append(nums[j]);
            bt(j+1, path); path.pop()
    bt(0, []); return [list(t) for t
in result]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the dedup set.

Sort nums first. In backtracking, skip duplicate elements at the same recursion level

(not the first occurrence, but subsequent same values when $j > \text{start}$).

#

Time: $O(n \cdot 2^n)$. Space: $O(n)$ recursion — no dedup set."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _90_SubsetsII:
    def subsetsWithDup(self, nums):
```

```
nums.sort()
result = []
def backtrack(start, path):
    result.append(list(path))
    for i in range(start,
len(nums)):
        if i > start and nums[i]
== nums[i-1]:
            continue
        path.append(nums[i])
        backtrack(i + 1, path)
        path.pop()
backtrack(0, [])
return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,2,2] sorted:

bt(0,[]): append []. i=0→[1]: bt(1,[1]):

append[1]. i=1→[1,2]: bt(2,[1,2]):

append. i=2: nums[2]==nums[1] and 2>1 → skip.

i=1→[2]: bt(2,[2]): append. i=2: 2>1 and nums[2]==nums[1] → skip.

→ [[],[1],[1,2],[1,2,2],[2],[2,2]] ✓

#

"No bugs. 'i>start' prevents skipping the very first occurrence at each level."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \cdot 2^n)$: worst case all unique. Space $O(n)$ recursion depth.

The sort+skip pattern is the standard dedup technique for backtracking.

Follow-up: Permutations II (#47) – same skip-duplicate pattern for permutations.

Follow-up: Combination Sum II (#40) – target sum with duplicates; same skip."

Bit Manipulation

#287 Find the Duplicate Number

MED[Open on LeetCode](#)

Array · Two Pointers · Binary Search · Bit Manipulation

Lists: Grind 169

Problem

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`

Output: 2

Example 2:

Input: `nums = [3,1,3,4,2]`
Output: `3`

Example 3:

Input: `nums = [3,3,3,3,3]`
Output: `3`

Constraints:

- $1 \leq n \leq 105$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in `nums` appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

n+1 integers in range [1,n]; exactly one number is duplicated.

Find it without modifying the array and using $O(1)$ extra space. Right?"

#

"A few quick questions:

The duplicate may appear more than twice? Yes.

Array can't be sorted or hashed? Correct – $O(1)$ space constraint."

#

"Let me walk: `nums=[1,3,4,2,2]` → `duplicate=2` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Hash set: return first element seen twice. $O(n)$ time, $O(n)$ space.

Violates $O(1)$ space. Should I go ahead and optimize?"

`class _287_FindDuplicateNumber_Brute:`

`def findDuplicate(self, nums):`

`seen = set()`

`for n in nums:`

`if n in seen: return n`

```
seen.add(n)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n)$  hash set.
```

```
# Floyd's cycle detection: treat nums as a  
linked list where index i points to  
nums[i].
```

```
# Because one value is duplicated, two  
indices point to the same next node →  
cycle.
```

```
# Phase 1: find meeting point (slow/fast).  
Phase 2: find cycle entry (= duplicate).
```

```
#
```

```
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _287_FindDuplicateNumber:  
    def findDuplicate(self, nums):  
        slow = fast = nums[0]  
        while True:  
            slow = nums[slow]  
            fast = nums[nums[fast]]  
            if slow == fast:  
                break
```

```
slow = nums[0]
while slow != fast:
    slow = nums[slow]
    fast = nums[fast]
return slow
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,3,4,2,2] (0→1, 1→3, 2→4, 3→2, 4→2):

Phase1: slow=1,fast=3→ slow=3,fast=2→ slow=2,fast=2. Meet at 2.

Phase2: slow=nums[0]=1, fast=2.

slow=3,fast=2→wait:

slow=nums[1]=3,fast=nums[2]=4.

slow=2,fast=2. return 2 ✓

#

"No bugs. The cycle entry is the duplicated value – same proof as Linked List Cycle II."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$. Does not sort or modify the array.

Binary search alternative: count numbers $\leq \text{mid}$; if $> \text{mid} \rightarrow$ duplicate in $[1, \text{mid}]$.
 $O(n \log n)$.
Follow-up: if multiple duplicates – Floyd's won't work; use binary search.
Follow-up: Find All Duplicates (#442) – mark visited by negating at index."

Bit Manipulation

□ ★ #1506 Find Root of N-Ary Tree ★

MED

[Open on LeetCode](#)

Hash Table · Bit Manipulation · Tree · DFS

Lists: ★ Premium | Premium 98

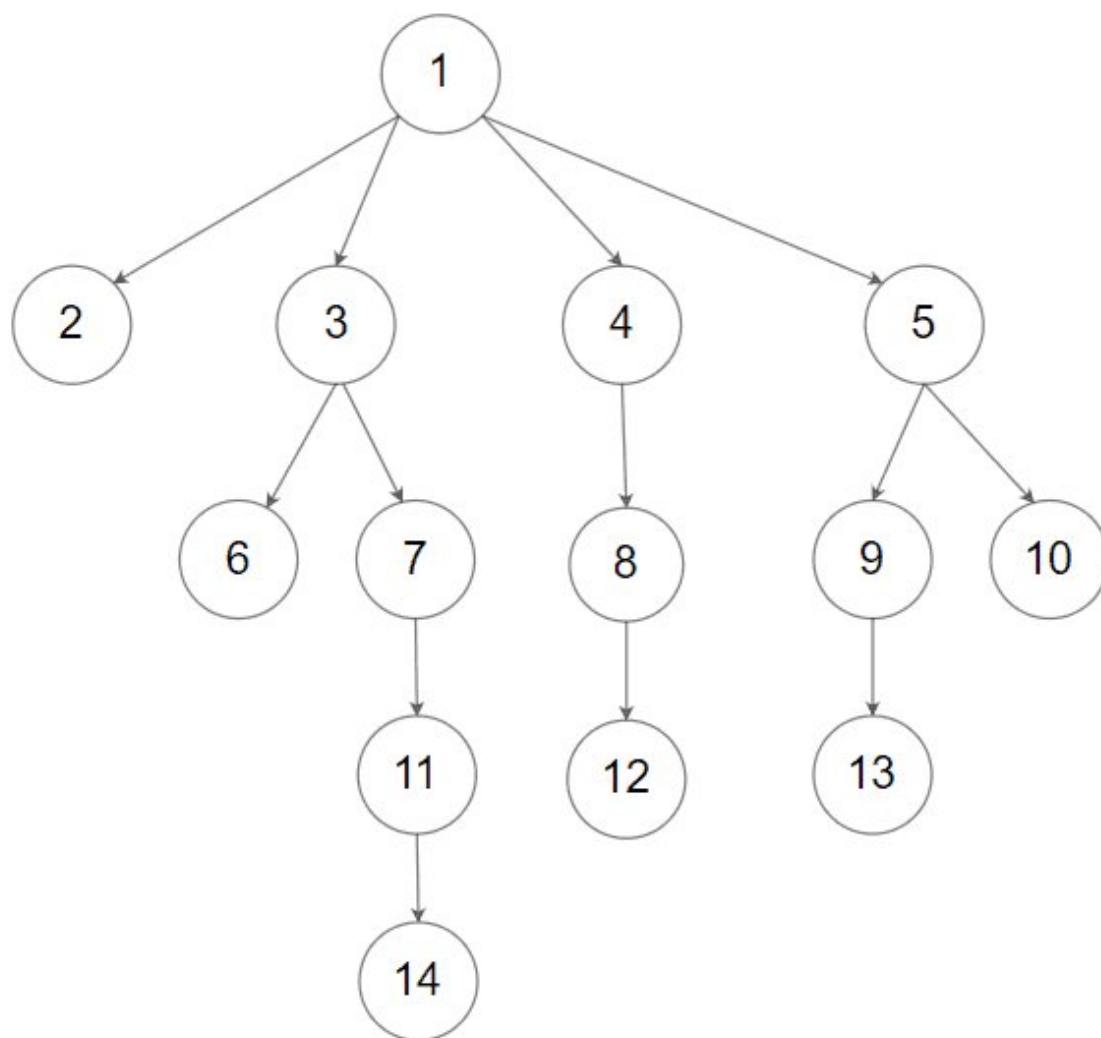
Problem

You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.

Return the root of the N-ary tree.

Custom testing:

An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).



For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].

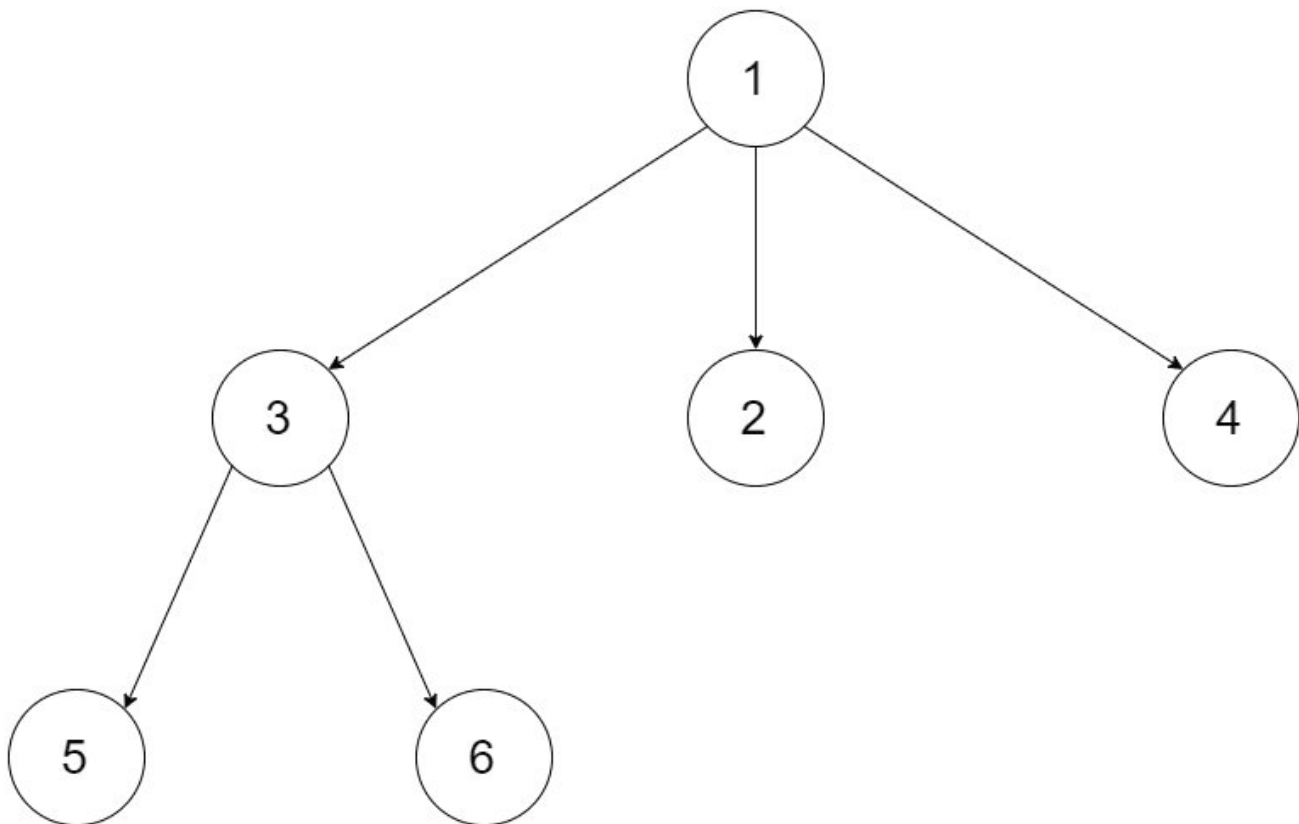
The testing will be done in the following way:

1. The input data should be provided as a serialization of the tree.
2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.

3. The driver code will pass the array to `findRoot`, and your function should find and return the root Node object in the array.

4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1:



Input: tree = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Explanation: The tree from the input data is shown above.

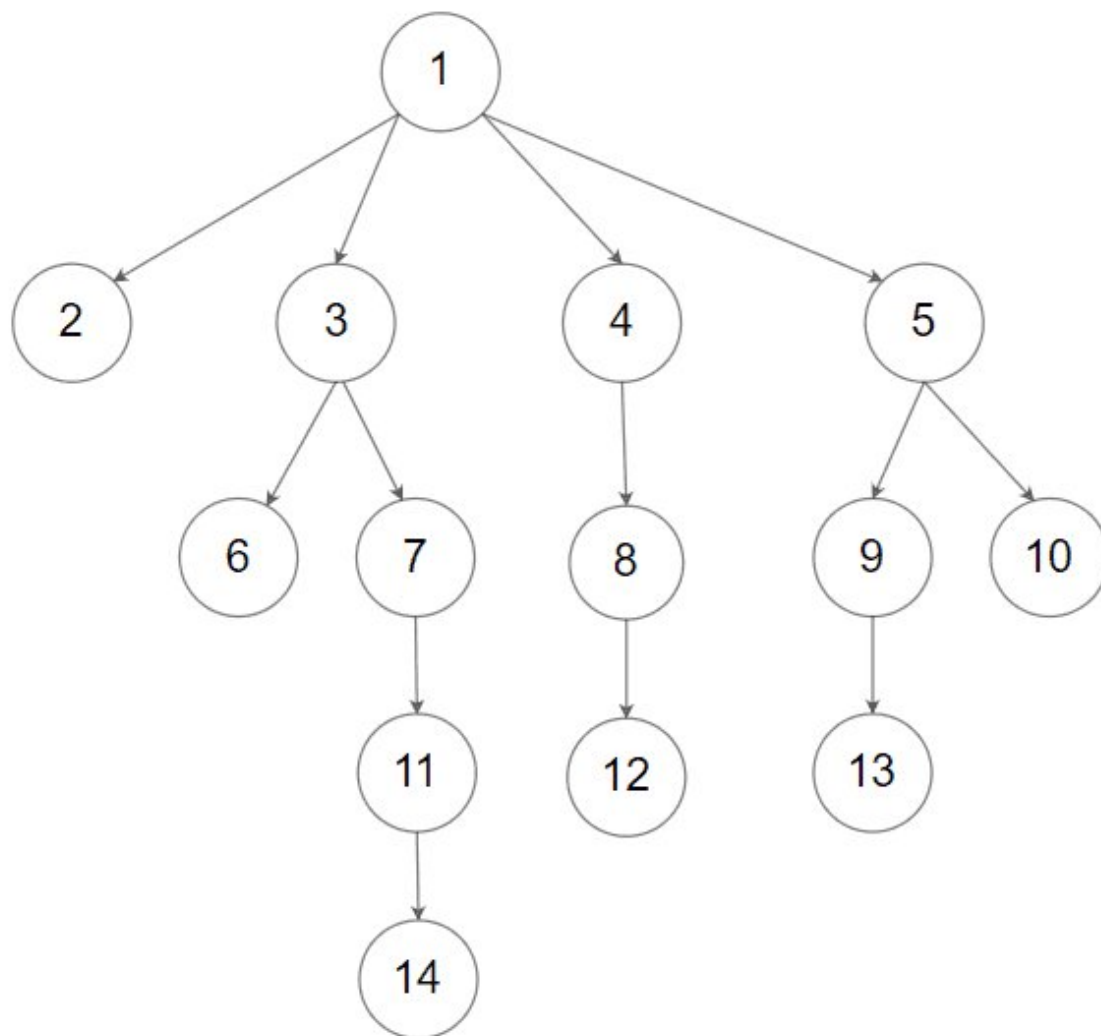
The driver code creates the tree and gives findRoot the Node objects in an arbitrary order.

For example, the passed array could be [Node(5),Node(4),Node(3),Node(6),Node(2),Node(1)] or [Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].

The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.

The input data and serialized Node(1) are the same, so the test passes.

Example 2:



Input: `tree = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`

Output: `[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]`

Constraints:

- The total number of nodes is between $[1, 5 * 10^4]$.

- Each node has a unique value.

Follow up:

- Could you solve this problem in constant space complexity with a linear time algorithm?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given all nodes of an N-ary tree in random order, find the root.

The root is the only node NOT appearing as a child of any other node. Right?"

#

"A few quick questions:

Node values are unique? Yes. No parent pointers? Correct."

#

"Let me walk: nodes=[1,2,3,4,5,6], root=1 with children [3,2,4], 3's children=[5,6] → root=1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Build a set of all child node values.
Root is the node whose value is NOT in that set.

$O(n)$ time, $O(n)$ space for the set.

Should I go ahead and optimize?"

class _1506_FindRootOfNAryTree_Brute:

def findRoot(self, tree):

children = set()

for node in tree:

for child in node.children:

children.add(child.val)

for node in tree:

if node.val not in children:

return node

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ set for child values."

XOR trick: XOR all node values; XOR all children values.

Every non-root node's value appears once as a node AND once as a child → cancels.

Root appears only as a node → its value remains.

```
#  
# Time: O(n). Space: O(1)."  
# PHASE 4 — CLEAN CODE  
# "Now I'll implement the optimized  
# approach with meaningful names."  
class _1506_FindRootOfNAryTree:  
    def findRoot(self, tree):  
        xor_all = 0  
        for node in tree:  
            xor_all ^= node.val  
            for child in node.children:  
                xor_all ^= child.val  
        for node in tree:  
            if node.val == xor_all:  
                return node  
  
# PHASE 5 — TEST & DEBUG  
# "Let me trace through a few cases."  
#  
# tree nodes [1,2,3,4,5,6] with root=1,  
# children={3,2,4}, 3's children={5,6}:  
# node vals XOR: 1^2^3^4^5^6. child vals  
# XOR: 3^2^4^5^6.  
# Combined: 1^2^3^4^5^6^3^2^4^5^6 = 1 (all  
# others cancel). root=node(1) ✓
```

#

"No bugs. XOR is commutative and associative – order of traversal doesn't matter."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

Same XOR cancellation as Single Number (#136).

Follow-up: if node values aren't unique, XOR fails – use the child-set approach.

Follow-up: find all roots if input is a forest – same XOR, collect all non-child values."

Math

Round 8 · Low · Medium · 5 questions

Math

□ #7 Reverse Integer

MED[Open on LeetCode](#)

Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$

Output: 321

Example 2:

Input: $x = -123$

Output: -321

Example 3:

Input: $x = 120$

Output: 21

Constraints:

- $-231 \leq x \leq 231 - 1$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Reverse the digits of a signed 32-bit integer. Return 0 if overflow.

Assume environment doesn't allow 64-bit integers. Right?"

#

"A few quick questions:

$x=120 \rightarrow 21$ (trailing zero becomes leading zero $\rightarrow$ dropped) ✓

$x=-123 \rightarrow -321$ ✓. Overflow range: $[-2^{31}, 2^{31}-1]$."

#

"Let me walk: $x=123 \rightarrow 321$ ✓. $x=-123 \rightarrow -321$ ✓. $x=1534236469 \rightarrow 0$ (overflow) ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock
in the logic.
# Convert to string, reverse, convert
back. Check 32-bit bounds.
# O(d) time and space. Should I go ahead
and optimize?"
class _7_ReverseInteger_Brute:
    def reverse(self, x):
        sign = -1 if x < 0 else 1; x =
abs(x)
        result = int(str(x)[::-1]) * sign
        return result if -(2**31) <=
result <= 2**31-1 else 0

# PHASE 3 — OPTIMIZE
# "The bottleneck is the string
allocation.
# Build reversed integer digit by digit;
check overflow before each
multiplication.
#
# Time: O(d). Space: O(1)."
```

```
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized
approach with meaningful names."
```

```
class _7_ReverseInteger:
    def reverse(self, x):
        INT_MAX, INT_MIN = 2**31 - 1,
        -(2**31)
        sign = -1 if x < 0 else 1
        x = abs(x)
        result = 0
        while x:
            digit = x % 10; x //= 10
            if result > (INT_MAX - digit)
            // 10:
                return 0
            result = result * 10 + digit
        return sign * result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# x=123: digits 3,2,1. result: 3,32,321.
# 321<=INT_MAX ✓. return 321 ✓.
# x=-123: sign=-1, x=123. Same → return
# -321 ✓.
# x=1534236469: eventually overflow check
# fires → return 0 ✓.
#
```

"No bugs. Overflow check: result > (INT_MAX - digit) // 10 is safe in Python (big ints)."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(d): d = number of digits. Space O(1): integer variables."

In Java/C++ the overflow check is critical – Python big-ints hide the issue.

Follow-up: Palindrome Number (#9) – compare reversed half to original.

Follow-up: if 64-bit is allowed, just reverse and check range."

Math

□ #50 Pow(x, n)

MED

[Open on LeetCode](#)

Math · Recursion

Lists: Grind 169

Problem

Implement `pow(x, n)`, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$

- $-2^{31} \leq n \leq 2^{31}-1$
- n is an integer.
- Either x is not zero or $n > 0$.
- $-10^4 \leq x^n \leq 10^4$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Implement $\text{pow}(x, n)$ – x raised to the power n . n can be negative. Right?"

#

"A few quick questions:

x can be negative? Yes. n can be -2^{31} ?
Yes – handle with care.

$x=0.0$, $n<0$ is undefined per constraints (x won't be 0 when $n<0$)."

#

"Let me walk: $\text{pow}(2.0, 10) \rightarrow 1024.0$ ✓.
 $\text{pow}(2.0, -2) \rightarrow 0.25$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Multiply x by itself |n| times.  $O(|n|)$  –  
TLEs for  $n=2^{31}$ .
```

```
# Should I go ahead and optimize?"
```

```
class _50_PowXN_Brute:
```

```
    def myPow(self, x, n):  
        if n < 0: x, n = 1/x, -n  
        result = 1  
        for _ in range(n): result *= x  
        return result
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is the  $O(|n|)$   
multiplications.
```

```
# Fast exponentiation: if n even:  $x^n =$   
 $(x^2)^{(n/2)}$ . If n odd:  $x^n = x * x^{(n-1)}$ .
```

```
# Halves n each step  $\rightarrow O(\log n)$   
multiplications.
```

```
#
```

```
# Time:  $O(\log n)$ . Space:  $O(1)$  iterative."
```

```
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
class _50_PowXN:
```

```
    def myPow(self, x, n):  
        if n < 0:
```

```
        x = 1 / x
        n = -n
    result = 1
    while n:
        if n % 2 == 1:
            result *= x
        x *= x
        n //= 2
    return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

x=2.0, n=10:

n=10(even): x=4,n=5. n=5(odd):

result=4,x=16,n=2. n=2(even): x=256,n=1.

n=1(odd): result=4*256=1024,n=0. return
1024.0 ✓

#

x=2.0, n=-2: x=0.5,n=2.

n=2(even): x=0.25,n=1.

n=1(odd): result=0.25,n=0. → 0.25 ✓

#

"No bugs. The iterative approach avoids
O(log n) recursion stack space."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$: halve the exponent each step. Space $O(1)$: iterative.

Recursive version is $O(\log n)$ time but $O(\log n)$ stack space.

Follow-up: matrix exponentiation – Fibonacci in $O(\log n)$ using the same squaring.

Follow-up: modular exponentiation – same logic with $(\text{result} * x) \% \text{mod}$ at each step."

Math

□ #380 Insert Delete GetRandom O(1)

MED[Open on LeetCode](#)

Array · Hash Table · Math · Design

Lists: Grind 169

Problem

Implement the RandomizedSet class:

- **RandomizedSet()** Initializes the RandomizedSet object.
- **bool insert(int val)** Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- **bool remove(int val)** Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- **int getRandom()** Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average $O(1)$ time complexity.

Example 1:

Input

```
["RandomizedSet", "insert", "remove",  
"insert", "getRandom", "remove",  
"insert", "getRandom"]
```

```
[[], [1], [2], [2], [], [1], [2], []]
```

Output

```
[null, true, false, true, 2, true,  
false, 2]
```

Explanation

```
RandomizedSet randomizedSet = new  
RandomizedSet();
```

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- At most $2 * 10^5$ calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a set supporting
insert(val)→bool, remove(val)→bool, and
getRandom()→val

all in average $O(1)$. getRandom returns
each element with equal probability.

Right?"

#

"A few quick questions:

insert on existing key → return false?
Yes. remove on missing key → false? Yes.

getRandom won't be called on empty set?
Guaranteed."

#

"Let me walk: insert(1)→T, insert(2)→T,
getRandom()→1 or 2 with equal prob ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Python set + convert to list for
getRandom. getRandom $O(n)$ to build list.

Should I go ahead and optimize?"

class _380_InsertDeleteGetRandom_Brute:

```
def __init__(self): self.data = set()
def insert(self, val):
    if val in self.data: return False
    self.data.add(val); return True
def remove(self, val):
    if val not in self.data: return
False
    self.data.discard(val); return
True
def getRandom(self):
    import random; return
random.choice(list(self.data))
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ list conversion for getRandom.

Array + HashMap: array holds all values, map holds {val→array_index}.

getRandom: pick a random array index — $O(1)$.

remove: swap target with last element, update map, pop last — $O(1)$.

#

Time: $O(1)$ all ops. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized
approach with meaningful names."
import random
class _380_InsertDeleteGetRandom:
    def __init__(self):
        self._vals = []
        self._idx = {}
    def insert(self, val):
        if val in self._idx: return False
        self._idx[val] = len(self._vals)
        self._vals.append(val)
        return True
    def remove(self, val):
        if val not in self._idx: return
        False
        last = self._vals[-1]
        i = self._idx[val]
        self._vals[i] = last
        self._idx[last] = i
        self._vals.pop()
        del self._idx[val]
        return True
    def getRandom(self):
        return random.choice(self._vals)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

insert(1): vals=[1],idx={1:0}.

insert(2): vals=[1,2],idx={1:0,2:1}.

remove(1): last=2,i=0.

vals[0]=2,idx[2]=0.

pop→vals=[2],idx={2:0}. del idx[1]. ✓

getRandom()→choice([2])→2 ✓

#

"No bugs. The swap-with-last trick avoids $O(n)$ shifts when removing from the middle."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(1)$ average: array ops + dict ops. Space $O(n)$."

Edge case: when removing the last element, swap is a no-op (last==val).

Follow-up: Insert Delete GetRandom with duplicates (#381) – store sets of indices per value.

Follow-up: weighted random – prefix-sum array + binary search for getRandom."

Math

#1017 Convert to Base -2

MED[Open on LeetCode](#)

Math

Lists: Denny Zhang

Problem

Given an integer n , return a binary string representing its representation in base -2 .

Note that the returned string should not have leading zeros unless the string is "0".

Example 1:

Input: $n = 2$

Output: "110"

Explantion: $(-2)^2 + (-2)^1 = 2$

Example 2:

Input: $n = 3$

Output: "111"

Explantion: $(-2)^2 + (-2)^1 + (-2)^0 = 3$

Example 3:

Input: $n = 4$

Output: "100"

Explantion: $(-2)^2 = 4$

Constraints:

- $0 \leq n \leq 109$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given integer n , return its representation in base -2 as a binary string.

No leading zeros (except '0' itself). Right?"

#

"A few quick questions:

$n=0 \rightarrow '0'?$ Yes. $(-2)^0=1$, $(-2)^1=-2$, $(-2)^2=4$.

$n=2$: $4+(-2)=2 \rightarrow '110'$ ✓"

#

"Let me walk: $n=2 \rightarrow '110'$ ✓. $n=3 \rightarrow '111'$ ✓. $n=4 \rightarrow '100'$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeated division by -2; remainder must be 0 or 1.

$O(\log n)$ — this IS optimal."

```
class _1017_ConvertToBaseNeg2_Brute:
```

```
    def baseNeg2(self, n):
```

```
        if n == 0: return '0'
```

```
        bits = []
```

```
        while n:
```

```
            remainder = n % 2;
```

```
bits.append(str(remainder)); n = -(n // 2)
```

```
        return ''.join(reversed(bits))
```

PHASE 3 — OPTIMIZE

"The brute already is $O(\log n)$.

The bottleneck is handling the negative base carefully.

$n \& 1$ gives the bit. $n = -(n >> 1)$

handles the negative base division."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1017_ConvertToBaseNeg2:
```

```
def baseNeg2(self, n):
    if n == 0: return '0'
    bits = []
    while n:
        bits.append(str(n & 1))
        n = -(n >> 1)
    return ''.join(reversed(bits))
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

n=2: bits=[0]: $2 \& 1 = 0$, $n = -(2 \gg 1) = -1$.

bits=[1]: $-1 \& 1 = 1$, $n = -(-1 \gg 1) = 0$ (wait:
 $-1 \gg 1 = -1$, $-(-1) = 1$).

n=1: $1 \& 1 = 1$, bits=[1], n=0.

bits=['0', '1', '1'] → '110' ✓

#

n=3: $3 \& 1 = 1$, $n = -1$. $-1 \& 1 = 1$, $n = 1$. $1 \& 1 = 1$, $n = 0$.

bits=['1', '1', '1'] → '111' ✓

#

"No bugs. $n = -(n \gg 1)$ implements
division by -2 in the negative base."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$: number of bits. Space
 $O(\log n)$ for the bits list."

```
# The n & 1 trick extracts the remainder  
correctly for negative n in Python.  
# Follow-up: general negative base –  
replace 2 with |base|, negate on each  
step.  
# Follow-up: decode a base-(-2) string  
back to integer – multiply by  
(-2)^position."
```

Math

□ #3160 Find the Number of Distinct Colors Among the Balls

MED[Open on LeetCode](#)

Array · Hash Table · Simulation

Lists: CodeSignal

Problem

You are given an integer `limit` and a 2D array `queries` of size $n \times 2$.

There are `limit + 1` balls with distinct labels in the range `[0, limit]`. Initially, all balls are uncolored. For every query in `queries` that is of the form `[x, y]`, you mark ball `x` with the color `y`. After each query, you need to find the number of colors among the balls.

Return an array `result` of length `n`, where `result[i]` denotes the number of colors after `i`th query.

Note that when answering a query, lack of a color will not be considered as a color.

Example 1:

Input: `limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]`

Output: [1,2,2,3]

Explanation:



- After query 0, ball 1 has color 4.
- After query 1, ball 1 has color 4, and ball 2 has color 5.
- After query 2, ball 1 has color 3, and ball 2 has color 5.
- After query 3, ball 1 has color 3, ball 2 has color 5, and ball 3 has color 4.

Example 2:

Input: limit = 4, queries =
[[0,1],[1,2],[2,2],[3,4],[4,5]]

Output: [1,2,2,3,4]

Explanation:



- After query 0, ball 0 has color 1.
- After query 1, ball 0 has color 1, and ball 1 has color 2.
- After query 2, ball 0 has color 1, and balls 1 and 2 have color 2.
- After query 3, ball 0 has color 1, balls 1 and 2 have color 2, and ball 3 has color 4.
- After query 4, ball 0 has color 1, balls 1 and 2 have color 2, ball 3 has color 4, and ball 4 has color 5.

Constraints:

- $1 \leq \text{limit} \leq 10^9$
- $1 \leq n == \text{queries.length} \leq 10^5$
- $\text{queries}[i].\text{length} == 2$
- $0 \leq \text{queries}[i][0] \leq \text{limit}$
- $1 \leq \text{queries}[i][1] \leq 10^9$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

N balls (0 to limit).

queries[i]=[ball,color]: paint that ball.

After each query, return the number of distinct colors among painted balls.

Right?"

#

"A few quick questions:

Unpainted balls have no color – don't count? Correct.

Can a ball be repainted? Yes – overwrites previous color."

#

"Let me walk:

queries=[[1,4],[2,5],[1,3],[3,4]] → [1,2,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Dict ball→color. After each query, scan all values for distinct count.

O(n) per query, O(n²) total.


```
# Should I go ahead and optimize?"
class _3160_FindDistinctColors_Brute:
    def queryResults(self, limit,
queries):
        ball_color = {}; result = []
        for ball, color in queries:
            ball_color[ball] = color
            result.append(len(set(ball_color.values())))
        return result

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n)$  set scan for distinct colors.
# color_count tracks how many balls have each color.
# On each query: if ball had a color, decrement old color count (remove if 0).
# Add new color to color_count. Distinct count = len(color_count).
#
# Time:  $O(1)$  per query. Space:  $O(n)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _3160_FindDistinctColors:
    def queryResults(self, limit,
queries):
        ball_color = {}
        color_count = {}
        result = []
        for ball, color in queries:
            if ball in ball_color:
                old = ball_color[ball]
                color_count[old] -= 1
                if color_count[old] == 0:
                    del color_count[old]
            ball_color[ball] = color
            color_count[color] =
color_count.get(color, 0) + 1
            result.append(len(color_count
))

        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

queries=[[1,4],[2,5],[1,3],[3,4]]:

[1,4]:

ball_color={1:4},color_count={4:1}.

distinct=1 ✓

```
# [2,5]: color_count={4:1,5:1}.
distinct=2 ✓
# [1,3]: old=4→count[4]=0 del.
color_count={5:1,3:1}. distinct=2 ✓
# [3,4]: color_count={5:1,3:1,4:1}.
distinct=3 ✓
#
# "No bugs. Deleting color_count[old] when
it hits 0 keeps len(color_count) correct."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) per query. Space O(unique balls +
colors).
# color_count keys = currently active
distinct colors – len gives the answer
directly.
# Follow-up: range coloring (paint balls
l..r) – needs segment tree or lazy
propagation.
# Follow-up: undo queries – maintain a
stack of (ball, old_color, new_color)."
```

JavaScript

Round 8 · Low · Medium · 7 questions

JavaScript

□ ★ #2627 Debounce ★

MED

[Open on LeetCode](#)

JavaScript

Lists: ★ Premium | Premium 98

Problem

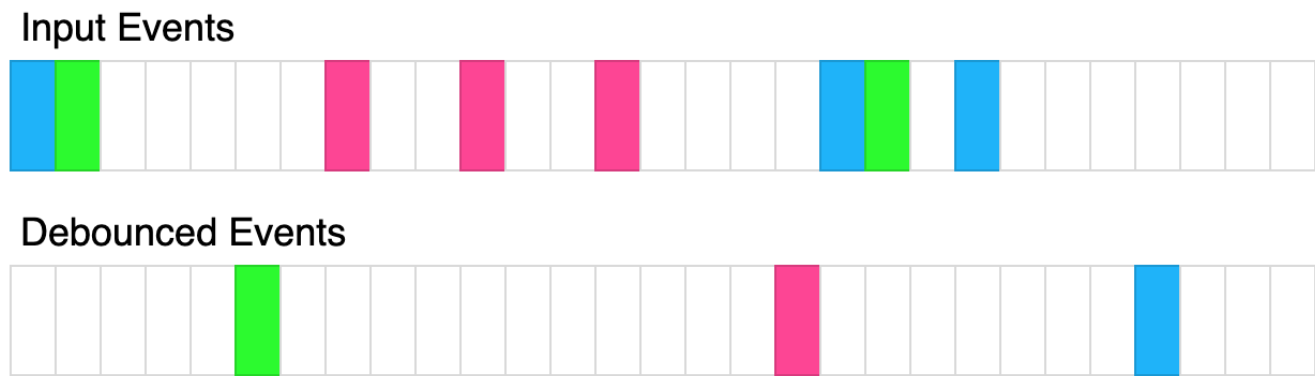
Given a function `fn` and a time in milliseconds `t`, return a debounced version of that function.

A debounced function is a function whose execution is delayed by `t` milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.

For example, let's say `t = 50ms`, and the function was called at 30ms, 60ms, and 100ms.

The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.

If instead `t = 35ms`, The 1st call would be cancelled, the 2nd would be executed at 95ms, and the 3rd would be executed at 135ms.



The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.

Please solve it without using
lodash's `_.debounce()` function.

Example 1:

Input:

```
t = 50
calls = [
  {"t": 50, inputs: [1]},
  {"t": 75, inputs: [2]}
]
```

Output: [{"t": 125, inputs: [2]}

Explanation:

Example 2:

Input:

t = 20

```
calls = [  
    {"t": 50, inputs: [1]},  
    {"t": 100, inputs: [2]}  
]
```

Output: [{"t": 70, inputs: [1]}, {"t": 120, inputs: [2]}]

Explanation:

Example 3:

Input:

t = 150

```
calls = [  
    {"t": 50, inputs: [1, 2]},  
    {"t": 300, inputs: [3, 4]},  
    {"t": 300, inputs: [5, 6]}  
]
```

Output: [{"t": 200, inputs: [1,2]}, {"t": 450, inputs: [5, 6]}]

Constraints:

- $0 \leq t \leq 1000$
- $1 \leq \text{calls.length} \leq 10$
- $0 \leq \text{calls}[i].t \leq 1000$

- $0 \leq \text{calls}[i].\text{inputs.length} \leq 10$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

`debounce(fn, t)`: delay fn's execution by `t` ms. Each new call resets the timer.

Only the most recent call fires – earlier pending calls are cancelled. Right?"

#

"A few quick questions: `t` can be 0? Yes – fires in the next microtask. No calls = no fires."

#

"Let me walk: calls at 0ms, 30ms, 60ms with `t=50ms` → fn fires once at 110ms ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

One timer reference, cancel previous on each call – this IS the optimal solution.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"The bottleneck is the need to cancel the previous pending call.

clearTimeout + setTimeout per invocation achieves $O(1)$ per call.

Time: $O(1)$ per call. Space: $O(1)$: one timer reference."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

// JavaScript:

function debounce(fn, t) {

let timerId = null;

return function(...args) {

clearTimeout(timerId);

timerId = setTimeout(() =>
fn.apply(this, args), t);

};

}

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

t=50ms, calls at 0ms, 30ms, 60ms:

0ms: cancel(null), set timer(50ms).

30ms: cancel, set timer(80ms). 60ms:

```
cancel, set timer(110ms).  
# fn fires at 110ms ✓. No earlier calls  
fire.  
#  
# "No bugs. Each call resets the timer,  
guaranteeing only the last call executes."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(1)$  per call. Space  $O(1)$ : one  
timer reference.  
# Debounce: wait until activity stops.  
Throttle: fire at most once per interval.  
# Follow-up: leading debounce – fire  
immediately then ignore for  $t$  ms.  
# Follow-up: Throttle (#2676) – use  
setInterval or timestamp-based approach."
```

JavaScript

□ ★ #2628 JSON Deep Equal ★

MED

[Open on LeetCode](#)

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given two values `o1` and `o2`, return a boolean value indicating whether two values, `o1` and `o2`, are deeply equal.

For two values to be deeply equal, the following conditions must be met:

- If both values are primitive types, they are deeply equal if they pass the `===` equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply equal according to these conditions.

You may assume both values are the output of `JSON.parse`. In other words, they are valid JSON.

Please solve it without using lodash's `_.isEqual()` function

Example 1:

Input: `o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}`

Output: `true`

Explanation: The keys and values match exactly.

Example 2:

Input: `o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}`

Output: `true`

Explanation: Although the keys are in a different order, they still match exactly.

Example 3:

Input: o1 = {"x":null,"L":[1,2,3]}, o2 = {"x":null,"L":["1","2","3"]}

Output: false

Explanation: The array of numbers is different from the array of strings.

Example 4:

Input: o1 = true, o2 = false

Output: false

Explanation: true !== false

Constraints:

- $1 \leq \text{JSON.stringify(o1).length} \leq 105$
- $1 \leq \text{JSON.stringify(o2).length} \leq 105$
- $\text{maxNestingDepth} \leq 1000$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Deep equality for JSON values: same primitive value (===), same array structure,

same object keys and values recursively. Keys in different order are still equal.

Right?"

#

"A few quick questions: null === null is true. null !== {}? Yes."

#

"Let me walk:

areDeeplyEqual({a:1},{a:1}) -> true.

areDeeplyEqual([1],[2]) -> false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

JSON.stringify both – fails for key ordering. Recursive comparison is correct.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"The bottleneck is the key-ordering issue with JSON.stringify.

Recursive DFS: handle null specially, then dispatch on type.

Time: $O(n)$. Space: $O(d)$ depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
# // JavaScript:
# function areDeeplyEqual(o1, o2) {
#   if (o1 === o2) return true;
#   if (typeof o1 !== typeof o2 || o1 ===
null || o2 === null) return o1 === o2;
#   if (Array.isArray(o1) !==
Array.isArray(o2)) return false;
#   if (Array.isArray(o1))
#     return o1.length === o2.length &&
o1.every((v, i) => areDeeplyEqual(v,
o2[i]));
#   const k1 = Object.keys(o1), k2 =
Object.keys(o2);
#   return k1.length === k2.length &&
k1.every(k => k in o2 &&
areDeeplyEqual(o1[k], o2[k]));
# }

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# areDeeplyEqual({y:2,x:1},{x:1,y:2}):
k1.length==k2.length. each key in o2? yes.
values equal? yes -> true ✓
# areDeeplyEqual(true,false):
true===false? No. typeof same. null check:
```

no. return true===false=false ✓

#

"No bugs. Null check must come before
typeof dispatch since typeof null ===
object."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(d)$ depth.

Follow-up: handle circular references –
track visited pairs with WeakMap.

Follow-up: Differences (#2700) – return
diff object instead of bool."

JavaScript

□ ★ #2630 Memoize ★

MED

[Open on LeetCode](#)

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function `fn`, return a memoized version of that function.

A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.

`fn` can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are `===` to each other.

Example 1:

Input:

```
getInputs = () => [[2,2],[2,2],[1,2]]  
fn = function (a, b) { return a + b; }
```

Output: [{"val":4,"calls":1},{"val":4,"calls":1},{"val":3,"calls":2}]

Explanation:

```
const inputs = getInputs();  
const memoized = memoize(fn);  
for (const arr of inputs) {
```

Example 2:

Input:

```
getInputs = () =>  
[[{}},{},[{}],[{}],[{}]]  
fn = function (a, b) { return ({...a,  
...b}); }
```

Output: [{"val":{},"calls":1},{"val":{},"calls":2},{"val":{},"calls":3}]

Explanation:

Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to `fn()` because of cache-hits, however none of those objects are `===` to each other.

Example 3:

Input:

```
getInputs = () => { const o = {};  
return [[o,o],[o,o],[o,o]]; }  
fn = function (a, b) { return ({...a,  
...b}); }
```

Output: [{"val":{}, "calls":1}, {"val":{}
, "calls":1}, {"val":{}, "calls":1}]

Explanation:

Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit. This is because every object passed in is identical.

Constraints:

- $1 \leq \text{inputs.length} \leq 105$
- $0 \leq \text{inputs.flat().length} \leq 105$
- $\text{inputs}[i][j] \neq \text{NaN}$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# memoize(fn): return a cached version
# that skips fn when same inputs are seen.
# Inputs compared with === (object
# references, not structure). Right?"
#
# "A few quick questions: {} === {} is
# false in JS – two empty objects are
# different? Yes."
#
# "Let me walk: memoized(2,3) first call
# -> computes. memoized(2,3) again -> cached
# ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
# in the logic.
# Store all (args, result) pairs in an
# array; scan on each call. O(n) per lookup.
# Should I go ahead and optimize?"
# // Array scan is O(n) per call.

# PHASE 3 — OPTIMIZE
# "The bottleneck is the linear scan for
# matching args.
# Use a Map with JSON.stringify(args) as
# key for O(1) lookup.
```

```
# Note: JSON.stringify({}) ===  
JSON.stringify({}) so structurally equal  
objects share cache.  
# For reference-equality semantics, a trie  
or WeakMap-based structure is needed.  
# Time: O(key_len) per call. Space:  
O(unique calls)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
# // JavaScript:  
# function memoize(fn) {  
#   const cache = new Map();  
#   return function(...args) {  
#     const key = JSON.stringify(args);  
#     if (!cache.has(key)) cache.set(key,  
#       fn.apply(this, args));  
#     return cache.get(key);  
#   };  
# }
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."  
#  
# fn=sum. memoized(2,3): key='[2,3]',  
miss -> compute 5, cache. return 5.
```

```
# memoized(2,3): key='[2,3]', hit ->
return 5 (fn not called) ✓
# memoized({},{}) x3 (same objects):
key='[{},{}]' each time -> cached after
first ✓
#
# "No bugs. JSON.stringify for primitives
is exact; for same-reference objects it's
consistent."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "First call O(fn). Cache hit O(key_len).
Space O(unique key count).
# Follow-up: LRU-bounded memoize – evict
oldest when cache exceeds size limit.
# Follow-up: async memoize – cache the
Promise itself so concurrent calls share
one fetch."
```

JavaScript

□ ★ #2633 Convert Object to JSON String ★ **MED**

[Open on LeetCode](#)

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by `Object.keys()`.

Please solve it without using the built-in `JSON.stringify` method.

Example 1:

Input: object = {"y":1,"x":2}

Output: {"y":1,"x":2}

Explanation:

Return the JSON representation.

Note that the order of keys should be the same as the order returned by `Object.keys()`.

Example 2:

Input: object =

{"a":"str","b":-12,"c":true,"d":null}

Output:

{"a":"str","b":-12,"c":true,"d":null}

Explanation:

The primitives of JSON are strings, numbers, booleans, and null.

Example 3:

Input: object =

{"key":{"a":1,"b":[{"},null,"Hello"]}}

Output:

{"key":{"a":1,"b":[{"},null,"Hello"]}}

Explanation:

Objects and arrays can include other objects and arrays.

Example 4:

Input: object = true

Output: true

Explanation:

Primitive types are valid inputs.

Constraints:

- value is a valid JSON value
- $1 \leq \text{JSON.stringify(object).length} \leq 105$
- $\text{maxNestingLevel} \leq 1000$
- all strings contain only alphanumeric characters

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Implement `JSON.stringify`: handle null, bool, number, string, array, object.

No extra spaces. Keys in `Object.keys()` order. No eval. Right?"

#

"A few quick questions: undefined values in arrays become null? No – constraints

say valid JSON only."

#

"Let me walk:

jsonStringify({a:2,b:[1,2,3]}) ->
{"a":2,"b":[1,2,3]} ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock
in the logic.

Recursive dispatch on type – this is the
only approach. $O(n)$ time.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"The bottleneck is correct type
dispatch.

Check null first (typeof null === object
in JS). Then bool, number, string, array,
object.

Use join to avoid $O(n^2)$ string
concatenation.

Time: $O(n)$. Space: $O(d)$ depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

// JavaScript:

```
# function jsonStringify(obj) {  
# if (obj === null) return 'null';  
# if (typeof obj === 'boolean') return  
String(obj);  
# if (typeof obj === 'number') return  
String(obj);  
# if (typeof obj === 'string') return ''  
+ obj + '';  
# if (Array.isArray(obj)) return '[' +  
obj.map(jsonStringify).join(',') + ']';  
# return '{' +  
Object.entries(obj).map(([k,v]) =>  
  ''+k+'':'+jsonStringify(v)).join(',') +  
  '}' ;  
# }
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

obj=true: 'true' ✓. obj=null: 'null' ✓.
obj='hello': '"hello"' ✓

obj=[null,true,1]: '[null,true,1]' ✓.

obj={key:{a:1}}: '{"key":{"a":1}}' ✓

#

"No bugs. Null before typeof catches the
typeof-null-is-object edge case."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one visit per node. Space $O(d)$ depth.

Follow-up: circular reference detection
– track visited with a Set.

Follow-up: pretty-print – add
indentation parameter."

JavaScript

□ ★ #2636 Promise Pool ★

MED[Open on LeetCode](#)

JavaScript · Concurrency

Lists: ★ Premium | Premium 98

Problem

Given an array of asynchronous functions `functions` and a pool limit `n`, return an asynchronous function `promisePool`. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. `promisePool` should begin execution of as many functions as possible and continue executing new functions when old promises resolve. `promisePool` should execute `functions[i]` then `functions[i + 1]` then `functions[i + 2]`, etc. When the last promise resolves, `promisePool` should also resolve. For example, if `n = 1`, `promisePool` will execute one function at a time in series. However, if `n =`

2, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for `promisePool` to return a promise that resolves any value.

Example 1:

Input:

```
functions = [  
  () => new Promise(res =>  
    setTimeout(res, 300)),  
  () => new Promise(res =>  
    setTimeout(res, 400)),  
  () => new Promise(res =>  
    setTimeout(res, 200))  
]
```

n = 2

Output: `[[300,400,500],500]`

Example 2:

Input:

```
functions = [  
    () => new Promise(res =>  
        setTimeout(res, 300)),  
    () => new Promise(res =>  
        setTimeout(res, 400)),  
    () => new Promise(res =>  
        setTimeout(res, 200))  
]  
n = 5  
Output: [[300,400,200],400]
```

Example 3:

Input:

```
functions = [  
    () => new Promise(res =>  
        setTimeout(res, 300)),  
    () => new Promise(res =>  
        setTimeout(res, 400)),  
    () => new Promise(res =>  
        setTimeout(res, 200))  
]  
n = 1  
Output: [[300,700,900],900]
```

Constraints:

- $0 \leq \text{functions.length} \leq 10$
- $1 \leq n \leq 10$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Execute async functions with at most n concurrent at any time.

Return a promise resolving when all functions complete. Right?"

#

"A few quick questions: Functions never reject? Yes per constraints."

#

"Let me walk: 3 functions, $n=2$: run $\text{fn1}+\text{fn2}$ concurrently. As each finishes, start fn3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

`Promise.all` runs all immediately – no concurrency limit. $O(n)$ but may overwhelm resources.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"The bottleneck is unlimited
concurrency.

n worker coroutines each pull from a
shared queue. n workers run in parallel.

Time: optimal. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized
approach with meaningful names."

// JavaScript:

async function promisePool(functions,
n) {

const queue = [...functions];

async function worker() {

while (queue.length) await
queue.shift()();

}

await Promise.all(Array.from({length:
Math.min(n, functions.length)}, worker));

}

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[fn1,fn2,fn3,fn4], n=2: worker1 runs fn1+fn3, worker2 runs fn2+fn4. At most 2 concurrent ✓

#

"No bugs. Shared queue.shift() is safe in JS single-threaded event loop."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time optimal. Space $O(n)$ concurrent promises.

Follow-up: cancel on first failure – reject the outer promise on first rejection.

Follow-up: retry failed tasks – wrap fn() in retry loop inside the worker."

JavaScript

□ ★ #2675 Array of Objects to Matrix ★

MED

[Open on LeetCode](#)

JavaScript · Array

Lists: ★ Premium | Premium 98

Problem

Write a function that converts an array of objects `arr` into a matrix `m`.

`arr` is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.

The first row `m` should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by ".".

Each of the remaining rows corresponds to an object in `arr`. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given column, the cell should contain an empty

string "".

The columns in the matrix should be in lexicographically ascending order.

Example 1:

Input:

```
arr = [  
  {"b": 1, "a": 2},  
  {"b": 3, "a": 4}  
]
```

Output:

```
[  
  ["a", "b"],  
]
```

Example 2:

Input:

```
arr = [  
  {"a": 1, "b": 2},  
  {"c": 3, "d": 4},  
  {}  
]
```

Output:

```
[
```

Example 3:

Input:

```
arr = [  
  {"a": {"b": 1, "c": 2}},  
  {"a": {"b": 3, "d": 4}}  
]
```

Output:

```
[  
  "a.b", "a.c", "a.d",  
]
```

Example 4:

Input:

```
arr = [  
  [{"a": null}],  
  [{"b": true}],  
  [{"c": "x"}]  
]
```

Output:

```
[
```

Example 5:

Input:

```
arr = [  
  {},  
  {},  
  {},  
]
```

Output:

```
[
```

Constraints:

- arr is a valid JSON array
- $1 \leq \text{arr.length} \leq 1000$
- unique keys ≤ 1000

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Convert array of objects to a matrix.
Row 0 = sorted flattened keys (dot notation).

Remaining rows = values for each object, empty string if key missing. Right?"

#

```
# "A few quick questions: Arrays inside  
objects use numeric index as key? Yes."  
#  
# "Let me walk: [{a:1,b:2},{b:3,c:4}]  
produces header row [a,b,c] and value rows  
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock  
in the logic.  
# Flatten each object with dot notation,  
collect all keys, build matrix row by row.  
#  $O(n*k*d)$ . Should I go ahead and  
optimize?"
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is computing flattened  
keys per object.  
# One-pass DFS flatten each object.  
Collect unique keys sorted. Build matrix.  
# Time:  $O(n*k)$ . Space:  $O(n*k)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."  
# // JavaScript:  
# function jsonToMatrix(arr) {
```

```
# function flatten(obj, prefix) {
#   const res = {};
#   for (const [k, v] of
Object.entries(obj)) {
#     const key = prefix ? prefix + '.' + k :
k;
#     if (v !== null && typeof v === 'object')
Object.assign(res, flatten(v, key));
#     else res[key] = v;
#   }
#   return res;
# }
# const flat = arr.map(o => flatten(o,
''));
# const keys = [...new
Set(flat.flatMap(Object.keys))].sort();
# return [keys, ...flat.map(o =>
keys.map(k => k in o ? o[k] : ''))];
# }
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[{a:1},{a:2,b:3}]: keys=['a','b'].
rows: [1,''] and [2,3]. header+rows
correct ✓


```
# [{a:{b:1}}, {a:{c:2}}]:  
keys=['a.b', 'a.c']. rows: [1, ''] and  
['', 2] ✓  
#  
# "No bugs. Dot notation is the standard  
format for nested JSON export."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n*k*d)$ . Space  $O(n*k)$  matrix.  
# Follow-up: handle arrays with bracket  
notation [0],[1].  
# Follow-up: reverse – matrix to array of  
objects via dot-path parsing."
```

JavaScript

★ #2700 Differences Between Two Objects ★

MED[Open on LeetCode](#)

JavaScript

Lists: ★ Premium | Premium 98

Problem

Write a function that accepts two deeply nested objects or arrays `obj1` and `obj2` and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from `obj1` to `obj2`.

For each changed key, the value should be represented as an array [`obj1` value, `obj2` value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of `JSON.parse`.

Example 1:

Input:

`obj1 = {}`

**`obj2 = {
 "a": 1,
 "b": 2`**

`}`

Output: `{}`

Explanation: There were no modifications made to `obj1`. New keys `"a"` and `"b"` appear in `obj2`, but keys that are added or removed should be ignored.

Example 2:

Input:

```
obj1 = {  
  "a": 1,  
  "v": 3,  
  "x": [],  
  "z": {  
    "a": null  
  }  
}
```

Example 3:

Input:

```
obj1 = {  
  "a": 5,  
  "v": 6,  
  "z": [1, 2, 4, [2, 5, 7]]  
}  
obj2 = {  
  "a": 5,  
}
```

Example 4:

Input:

```
obj1 = {  
  "a": {"b": 1},  
}  
obj2 = {  
  "a": [5],  
}
```

Output:

Example 5:

Input:

```
obj1 = {  
  "a": [1, 2, {}],  
  "b": false  
}  
obj2 = {  
  "b": false,  
  "a": [1, 2, {}]
```

Constraints:

- obj1 and obj2 are valid JSON objects or arrays
- $2 \leq \text{JSON.stringify(obj1).length} \leq 104$
- $2 \leq \text{JSON.stringify(obj2).length} \leq 104$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find where two JSON values differ. Keys only in ONE object are excluded.

Leaf differences: [obj1_val, obj2_val]. Nested diffs form nested result objects. Right?"

#

"A few quick questions: Arrays compared by index? Yes – treated like objects."

#

"Let me walk: diff({a:1,b:2},{a:1,b:3}) gives {b:[2,3]} ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursive traversal checking keys present in both – this is the solution.

$O(n)$ time, $O(d)$ space."

PHASE 3 — OPTIMIZE

"The bottleneck is handling null and type mismatches.

```
# Base case: non-object or null values –  
return {} if equal, [v1,v2] if different.  
# Recursive: only descend into keys  
present in BOTH objects.  
# Time: O(n). Space: O(d) depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized  
approach with meaningful names."
```

```
# // JavaScript:  
# function objDiff(obj1, obj2) {  
#   if (typeof obj1 !== 'object' || obj1 ===  
#     null ||  
#     typeof obj2 !== 'object' || obj2 ===  
#     null)  
#     return obj1 === obj2 ? {} : [obj1,  
#       obj2];  
#   const result = {};  
#   for (const k of Object.keys(obj1)) {  
#     if (!(k in obj2)) continue;  
#     const diff = objDiff(obj1[k], obj2[k]);  
#     if (Object.keys(diff).length ||  
#       Array.isArray(diff)) result[k] = diff;  
#   }  
#   return result;  
# }
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

obj1={a:1,b:{c:2,d:3}},

obj2={a:1,b:{c:2,d:4}}:

a: equal -> {}. b: both objects. c:
equal. d: 3!=4 -> [3,4].

result={b:{d:[3,4]}} ✓

#

"No bugs. typeof null === object in JS
requires explicit null check before
recursion."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(d)$ depth.

Follow-up: include keys only in one
object as [value, undefined].

Follow-up: JSON Deep Equal (#2628) –
same traversal but returns bool."

Round 9 · Low · Hard

Round 9

Low Priority · Hard

7 questions · 2 patterns

Dynamic Programming #10 #115 #123 #132
#312 #1000

Math #68

Dynamic Programming

Round 9 · Low · Hard · 6 questions

Dynamic Programming

□ #10 Regular Expression Matching

HAR[Open on LeetCode](#)

String · Dynamic Programming · Recursion

Lists: Denny Zhang

Problem

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"

Output: false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input: `s = "aa", p = "a*"`

Output: `true`

Explanation: `'*'` means zero or more of the preceding element, `'a'`. Therefore, by repeating `'a'` once, it becomes `"aa"`.

Example 3:

Input: `s = "ab", p = ".*"`

Output: `true`

Explanation: `".*"` means "zero or more (*) of any character (.)".

Constraints:

- `1 <= s.length <= 20`
- `1 <= p.length <= 20`
- `s` contains only lowercase English letters.
- `p` contains only lowercase English letters, `'.'`, and `'*'`.
- It is guaranteed for each appearance of the character `'*'`, there will be a previous valid character to match.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Implement regex matching with '.' (any
single char) and '*' (zero or more of
preceding).
# Full string must match. Right?"
#
# "A few quick questions: '.' matches any
sequence? Yes. 'a*' can match zero 'a's?
Yes."
#
# "Let me walk: s='aa', p='a*' -> true ✓.
s='ab', p='.*' -> true ✓. s='aab',
p='c*a*b' -> true ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Recursion: match char-by-char, handle
'*' as zero or more of preceding.
#  $O(2^{(m+n)})$  without memo. Should I go
ahead and optimize?"
class _10_RegexpMatching_Brute:
    def isMatch(self, s, p):
        if not p: return not s
```

```

        first = bool(s) and p[0] in {s[0],
    '.'}

    if len(p) >= 2 and p[1] == '*':
        return self.isMatch(s, p[2:])
    or (first and self.isMatch(s[1:], p))
    return first and
self.isMatch(s[1:], p[1:])

```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up 2D DP: $dp[i][j]$ = does $s[i:]$ match $p[j:]$? Fill right-to-left.

Space: $O(n)$ with rolling rows.

Time: $O(m \cdot n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _10_RegexMatching:
    def isMatch(self, s, p):
        m, n = len(s), len(p)
        dp = [False] * (n + 1)
        dp[n] = True
        for i in range(m, -1, -1):
            new_dp = [False] * (n + 1)

```

```

        new_dp[n] = (i == m)
        for j in range(n - 1, -1, -1):
            first_match = i < m and
p[j] in {s[i], '.'}
                if j + 1 < n and p[j+1] ==
'*':
                    new_dp[j] =
new_dp[j+2] or (first_match and dp[j])
                else:
                    new_dp[j] =
first_match and dp[j+1]
                dp = new_dp
        return dp[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='aa', p='a*': dp converges so that
dp[0]=True (a* matches 'aa') ✓

s='ab', p='.*': .* matches any sequence.
dp[0]=True ✓

s='aab', p='c*a*b': c*(zero c's) +
a*(two a's) + b. dp[0]=True ✓

#

"No bugs. '*' has two branches: zero
occurrences (new_dp[j+2]) or one-plus

(dp[j])."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$. Space $O(n)$: two rolling rows."

Follow-up: Wildcard Matching (#44) — '*' matches any sequence; simpler DP.

Follow-up: if '?' (any single char) is added — treat like '.' in this solution."

Dynamic Programming

□ #115 Distinct Subsequences

HAR[Open on LeetCode](#)

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings *s* and *t*, return the number of distinct subsequences of *s* which equals *t*.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

Input: *s* = "rabbbit", *t* = "rabbit"

Output: 3

Explanation:

As shown below, there are 3 ways you can generate "rabbit" from *s*.

rabbbit

rabbbit

rabbbit

Example 2:

Input: `s = "babgbag", t = "bag"`

Output: 5

Explanation:

As shown below, there are 5 ways you can generate "bag" from s.

`babgbag`

`babgbag`

`babgbag`

`babgbag`

Constraints:

- `1 <= s.length, t.length <= 1000`
- `s` and `t` consist of English letters.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Count distinct subsequences of `s` that equal `t`.

Same positions count as different even if same characters. Right?"

#

"A few quick questions: `t='rabbit'`, `s='rabbbit'` -> 3 ways ✓ (three different

```
'b' choices)"
#
# "Let me walk: s='babgbag', t='bag' -> 5
✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Recursion: if s[i]==t[j]:
use(i+1,j+1)+skip(i+1,j); else
skip(i+1,j).
# O(2^m) without memo. Should I go ahead
and optimize?"
class _115_DistinctSubsequences_Brute:
    def numDistinct(self, s, t):
        from functools import lru_cache
        @lru_cache(None)
        def dp(i, j):
            if j == len(t): return 1
            if i == len(s): return 0
            result = dp(i+1, j)
            if s[i] == t[j]: result +=
dp(i+1, j+1)
            return result
        return dp(0, 0)
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up DP: $dp[j]$ = ways to form $t[j:]$ from $s[i:]$. Roll right-to-left.

Time: $O(m \cdot n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _115_DistinctSubsequences:
    def numDistinct(self, s, t):
        n = len(t)
        dp = [0] * (n + 1)
        dp[n] = 1
        for ch in reversed(s):
            for j in range(n - 1, -1, -1):
                if ch == t[j]:
                    dp[j] += dp[j+1]
        return dp[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

$s = \text{'rabbbit'}$, $t = \text{'rabbit'}$: three b's in s
for two in t $\rightarrow$ 3 ways ✓

Right-to-left update prevents using same $s[i]$ twice in one match.

#

"No bugs. $dp[n]=1$ base: empty t is trivially formed from any s suffix."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$. Space $O(n)$: one rolling array.

Right-to-left update is essential – prevents reuse of same $s[i]$ across matches.

Follow-up: Edit Distance (#72) – related; counts edits instead of matching occurrences.

Follow-up: print all distinct subsequences – backtrack through the DP table."

Dynamic Programming

#123 Best Time to Buy and Sell Stock III

HAR[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [3,3,5,0,0,3,1,4]`

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$.

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = $4 - 1 = 3$.

Example 2:

Input: prices = [1,2,3,4,5]

Output: 4

Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = $5 - 1 = 4$.

Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple transactions at the same time. You must sell before buying again.

Example 3:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transaction is done, i.e. max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 105$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# At most 2 transactions. Maximize profit.
Can't hold two stocks simultaneously.
Right?"
#
# "A few quick questions: Complete one
transaction before starting next? Yes."
#
# "Let me walk: prices=[3,3,5,0,0,3,1,4]
-> buy@0,sell@3 + buy@1,sell@4 = 6 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all pairs of buy1<sell1<=buy2<sell2.
O(n^4). Should I go ahead and optimize?"
class _123_StockIII_Brute:
    def maxProfit(self, prices):
        n = len(prices); best = 0
        for s1 in range(n):
            for e1 in range(s1, n):
                p1 =
prices[e1]-prices[s1]
                for s2 in range(e1, n):
```



```
        for e2 in range(s2,
n):
            best = max(best,
p1+prices[e2]-prices[s2])
        return best

# PHASE 3 — OPTIMIZE
# "The bottleneck is the quartic
enumeration.
# State machine: 4 states track max
profit.
# buy1, sell1, buy2, sell2 updated in one
pass.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _123_StockIII:
    def maxProfit(self, prices):
        buy1 = buy2 = float('-inf')
        sell1 = sell2 = 0
        for price in prices:
            buy1 = max(buy1, -price)
            sell1 = max(sell1, buy1 +
price)
```

```
        buy2 = max(buy2, sell1 -  
price)  
        sell2 = max(sell2, buy2 +  
price)  
    return sell2
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

prices=[3,3,5,0,0,3,1,4]:

price=0: buy1=max(-inf,0)=0.

sell1=max(0,0+0)=0. buy2=max(-inf,0-0)=0.

sell2=max(0,0+0)=0.

price=3: buy1=0. sell1=3. buy2=3-3=0?

No: buy2=max(0,3-3)=0.

sell2=max(0,0+3)=3.

... after price=4: sell2=6 ✓

#

"No bugs. State machine automatically
handles the constraint of selling before
buying again."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$:
four state variables.

Follow-up: at most k transactions (#188)
– generalize to $2k$ state variables or $O(n*k)$ DP.
Follow-up: unlimited transactions (#122) – sum all positive consecutive differences."

Dynamic Programming

#132 Palindrome Partitioning II

HAR[Open on LeetCode](#)

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given a string *s*, partition *s* such that every substring of the partition is a palindrome.

Return the minimum cuts needed for a palindrome partitioning of *s*.

Example 1:

Input: *s* = "aab"

Output: 1

Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.

Example 2:

Input: *s* = "a"

Output: 0

Example 3:

Input: `s = "ab"`

Output: `1`

Constraints:

- `1 <= s.length <= 2000`
- `s` consists of lowercase English letters only.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the minimum cuts to partition `s` into palindromes.

Each partition must be a palindrome. Right?"

#

"A few quick questions: The whole string being a palindrome = 0 cuts? Yes."

#

"Let me walk: `s='aab' -> 1 cut ('aa'|'b')` ✓. `s='a' -> 0` ✓. `s='ab' -> 1` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Try all partitionings.  $O(2^n)$  time.
Should I go ahead and optimize?"
class
_132_PalindromePartitioningII_Brute:
    def minCut(self, s):
        from functools import lru_cache
        n = len(s)
        @lru_cache(None)
        def is_pal(i, j): return s[i:j+1]
        == s[i:j+1][::-1]
        @lru_cache(None)
        def dp(i):
            if i == n: return -1
            return min(1 + dp(j+1) for j
in range(i, n) if is_pal(i, j))
        return dp(0)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^3)$ palindrome checks combined with DP.

Precompute palindrome table in $O(n^2)$. Then $dp[i] = \min$ cuts for $s[:i+1]$.

$dp[i] = \min(dp[j-1]+1)$ for all j where $s[j..i]$ is palindrome.

Time: $O(n^2)$. Space: $O(n^2)$ palindrome table + $O(n)$ dp."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _132_PalindromePartitioningII:
    def minCut(self, s):
        n = len(s)
        is_pal = [[False]*n for _ in
range(n)]
        for i in range(n):
            is_pal[i][i] = True
        for length in range(2, n+1):
            for i in range(n-length+1):
                j = i+length-1
                is_pal[i][j] =
(s[i]==s[j]) and (length==2 or
is_pal[i+1][j-1])
        dp = list(range(-1, n))
        for i in range(n):
            for j in range(i+1):
                if is_pal[j][i]:
                    dp[i+1] =
min(dp[i+1], dp[j]+1)
        return dp[n]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#  
# s='aab': is_pal[0][1]=True('aa').  
dp[2]=min(dp[2],dp[0]+1)=min(1,1)=1.  
dp[3]=1('b') -> cut=1 ✓  
#  
# "No bugs. dp[0]=-1 base: -1 cuts needed  
for empty prefix."  
# PHASE 6 — COMPLEXITY & FOLLOW-UP  
# "Time  $O(n^2)$ . Space  $O(n^2)$  table +  $O(n)$   
dp.  
# Follow-up: Palindrome Partitioning I  
(#131) – return all partitions; backtrack  
with table.  
# Follow-up: Palindrome Partitioning IV  
(#1745) – fixed 3 parts; use same  
palindrome table."
```


Dynamic Programming

□ #312 Burst Balloons

HAR[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to $n - 1$. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the i th balloon, you will get $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$ coins. If $i - 1$ or $i + 1$ goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

Input: `nums = [3,1,5,8]`

Output: 167

Explanation:

`nums = [3,1,5,8] --> [3,5,8] --> [3,8]`
`--> [8] --> []`

`coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167`

Example 2:

Input: `nums = [1,5]`

Output: 10

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `0 <= nums[i] <= 100`

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Burst balloon `i`: earn

`nums[i-1]*nums[i]*nums[i+1]`. Pad array with 1s on both ends.

```
# Burst all balloons to maximize total
coins. Right?"
#
# "A few quick questions: After bursting
i, its neighbors become adjacent? Yes."
#
# "Let me walk: nums=[3,1,5,8] -> 167 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all permutations of burst order.
O(n!). Should I go ahead and optimize?"
class _312_BurstBalloons_Brute:
    def maxCoins(self, nums):
        nums = [1] + nums + [1]; n =
len(nums)
        from functools import lru_cache
        @lru_cache(None)
        def dp(l, r):
            return
max((nums[l]*nums[k]*nums[r] + dp(l,k) +
dp(k,r) for k in range(l+1,r)), default=0)
            return dp(0, n-1)
# PHASE 3 — OPTIMIZE
```

"The bottleneck is the $O(n!)$ permutation search.

Interval DP: $dp[l][r]$ = max coins from open interval (l,r) where l,r are boundaries NOT popped.

Think backwards: k is the LAST balloon popped in (l,r) .

$dp[l][r]$ = max over k : $dp[l][k] + \text{nums}[l] * \text{nums}[k] * \text{nums}[r] + dp[k][r]$.

Time: $O(n^3)$. Space: $O(n^2)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _312_BurstBalloons:
    def maxCoins(self, nums):
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0]*n for _ in range(n)]
        for length in range(2, n):
            for left in range(0,
n-length):
                right = left+length
                for k in range(left+1,
right):
```

```
                dp[left][right] =  
max(dp[left][right],  
                nums[left]*nums[k  
]*nums[right] + dp[left][k] +  
dp[k][right])  
    return dp[0][n-1]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# nums=[1,3,1,5,8,1] (padded): dp fills by  
increasing length.
```

```
# dp[0][5]=167 ✓ (standard result matching  
all burst orders)
```

```
#
```

```
# "No bugs. 'Last burst' reversal gives  
clean optimal substructure."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^3)$ . Space  $O(n^2)$ .
```

```
# The last-burst reversal is the key  
insight that gives clean substructure.
```

```
# Follow-up: Minimum Cost to Merge Stones  
(#1000) – same interval DP skeleton.
```

```
# Follow-up: Stone Game (#877) – similar  
range DP for game theory."
```

Dynamic Programming

#1000 Minimum Cost to Merge Stones

HAR[Open on LeetCode](#)

Array · Dynamic Programming

Lists: Denny Zhang

Problem

There are n piles of stones arranged in a row. The i th pile has $\text{stones}[i]$ stones.

A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.

Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1 .

Example 1:

Input: stones = [3,2,4,1], k = 2

Output: 20

Explanation: We start with [3, 2, 4, 1].

We merge [3, 2] for a cost of 5, and we are left with [5, 4, 1].

We merge [4, 1] for a cost of 5, and we are left with [5, 5].

We merge [5, 5] for a cost of 10, and we are left with [10].

The total cost was 20, and this is the minimum possible.

Example 2:

Input: stones = [3,2,4,1], k = 3

Output: -1

Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore. So the task is impossible.

Example 3:

Input: stones = [3,5,1,2,6], k = 3

Output: 25

Explanation: We start with [3, 5, 1, 2, 6].

We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].

We merge [3, 8, 6] for a cost of 17, and we are left with [17].

The total cost was 25, and this is the minimum possible.

Constraints:

- $n == \text{stones.length}$
- $1 \leq n \leq 30$
- $1 \leq \text{stones}[i] \leq 100$
- $2 \leq k \leq 30$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Merge exactly k consecutive piles at a time; cost = sum of the k piles.


```

# Return minimum total cost, or -1 if
impossible. Right?"
#
# "A few quick questions: When is it
impossible? When  $(n-1) \% (k-1) \neq 0$ .
# Each merge reduces pile count by  $k-1$ , so
we need  $(n-1)$  divisible by  $(k-1)$ ."
#
# "Let me walk: stones=[3,2,4,1], k=2 ->
20 ✓. stones=[3,2,4,1], k=3 -> -1 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Try all orders of k-consecutive merges.
 $O(n^k)$  -- exponential. Should I go ahead
and optimize?"
class _1000_MinCostMergeStones_Brute:
    def mergeStones(self, stones, k):
        n = len(stones)
        if  $(n-1) \% (k-1)$ : return -1
        from functools import lru_cache
        prefix = [0] * (n+1)
        for i in range(n): prefix[i+1] =
prefix[i] + stones[i]
        @lru_cache(None)

```

```

def dp(l, r, piles):
    if r-l+1 == piles: return 0
    result =
min(dp(l,m,1)+dp(m+1,r,piles-1) for m in
range(l,r,k-1))
    if piles == 1: result +=
prefix[r+1]-prefix[l]
    return result
return dp(0, n-1, 1)

```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential search without memoization (above is already memoized).

Same interval DP but cleaner: $dp[l][r]$ = min cost to merge stones[l..r] into fewest piles.

When $(r-l)\%(k-1)==0$, the segment merges to exactly 1 pile.

Time: $O(n^3/k)$. Space: $O(n^2)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _1000_MinCostMergeStones:
    def mergeStones(self, stones, k):

```

```

n = len(stones)
if (n-1) % (k-1): return -1
prefix = [0] * (n+1)
for i in range(n): prefix[i+1] =
prefix[i] + stones[i]
dp = [[0]*n for _ in range(n)]
for length in range(k, n+1):
    for l in range(n-length+1):
        r = l+length-1
        dp[l][r] = float('inf')
        for m in range(l, r, k-1):
            dp[l][r] =
min(dp[l][r], dp[l][m] + dp[m+1][r])
            if (length-1) % (k-1) ==
0:
                dp[l][r] +=
prefix[r+1]-prefix[l]
return dp[0][n-1]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

stones=[3,2,4,1], k=2: (4-1)%1=0 OK.

After DP: dp[0][3]=20 ✓

stones=[3,2,4,1], k=3: (4-1)%2=1 != 0 ->
return -1 ✓

#

"No bugs. The split point iterates in steps of $k-1$ because each merge reduces count by $k-1$."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^3/k)$. Space $O(n^2)$."

The feasibility check $(n-1) \% (k-1) == 0$ saves all computation for impossible inputs.

Follow-up: Burst Balloons (#312) – same interval DP skeleton, different cost formula.

Follow-up: $k=2$ (classic stone merging) – solvable in $O(n \log n)$ with Hu-Shing algorithm."

Math

Round 9 · Low · Hard · 1 question

Math

□ #68 Text Justification

HAR[Open on LeetCode](#)

Array · String · Simulation

Lists: CodeSignal

Problem

Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`.
- The input array `words` contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an",  
"example", "of", "text",  
"justification."], maxWidth = 16
```

Output:

```
[  
    "This    is    an",  
    "example  of text",  
    "justification. "  
]
```

Example 2:

Input: words = ["What","must","be","acknowledgment","shall","be"], maxWidth = 16

Output:

```
[  
    "What    must    be",  
    "acknowledgment  ",  
    "shall be         "  
]
```

Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead of fully-justified.

Example 3:


```
Input: words = ["Science","is","what","we",
"understand","well","enough","to","explain",
"to","a","computer.","Art","is","everything",
"else","we","do"],
maxWidth = 20
```

Output:

```
[
  "Science is what we",
  "understand well",
  "enough to explain to",
  "a computer. Art is",
  "everything else we",
```

Constraints:

- $1 \leq \text{words.length} \leq 300$
- $1 \leq \text{words}[i].\text{length} \leq 20$
- $\text{words}[i]$ consists of only English letters and symbols.
- $1 \leq \text{maxWidth} \leq 100$
- $\text{words}[i].\text{length} \leq \text{maxWidth}$

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the
problem correctly.
# Arrange words into lines of exactly
maxWidth chars. Spaces distributed as
evenly as possible.
# Extra spaces go to the LEFT. Last line:
left-justified with single spaces.
Right?"
#
# "A few quick questions: Each word fits
in maxWidth? Yes per constraints.
# At least one word? Yes."
#
# "Let me walk:
words=['This','is','an','example'],
maxWidth=16.
# Line 1: 'This is an' (total=16) ✓. Last:
left-justified."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock
in the logic.
# Greedy packing: fit as many words per
line as possible. Then distribute spaces.
# O(n*maxWidth) time. This IS the optimal
approach.
```

```
# Should I go ahead and optimize?"
class _68_TextJustification_Brute:
    def fullJustify(self, words,
maxWidth):
        result = []; i = 0; n = len(words)
        while i < n:
            line_words = [words[i]];
line_len = len(words[i]); i += 1
            while i < n and line_len + 1 +
len(words[i]) <= maxWidth:
                line_len += 1 +
len(words[i]);
            line_words.append(words[i]); i += 1
            result.append(line_words)
        lines = []
        for idx, lw in enumerate(result):
            if idx == len(result) - 1 or
len(lw) == 1:
                lines.append('
'.join(lw).ljust(maxWidth))
            else:
                total = maxWidth -
sum(len(w) for w in lw)
                gaps = len(lw) - 1
```

```
        base, extra =
divmod(total, gaps)
        line = lw[0]
        for j in range(1,
len(lw)):
            line += ' ' * (base +
(1 if j <= extra else 0)) + lw[j]
            lines.append(line)
        return lines
```

PHASE 3 — OPTIMIZE

"Already $O(n)$ – greedy packing + space distribution is optimal.

The bottleneck is correct space distribution and last-line handling."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _68_TextJustification:
    def fullJustify(self, words,
maxWidth):
        result = []
        i = 0
        while i < len(words):
```

```

        line = [words[i]]; length =
len(words[i]); i += 1
        while i < len(words) and
length + 1 + len(words[i]) <= maxWidth:
            length += 1 +
len(words[i]); line.append(words[i]); i
+= 1
        if i == len(words) or
len(line) == 1:
            result.append('
'.join(line).ljust(maxWidth))
        else:
            spaces = maxWidth -
sum(len(w) for w in line)
            gaps = len(line) - 1
            base, extra =
divmod(spaces, gaps)
            row = line[0]
            for j in range(1,
len(line)):
                row += ' ' * (base +
(1 if j <= extra else 0)) + line[j]
            result.append(row)
    return result

```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# words=['This','is','an','example'],
# maxWidth=16:
# line=['This','is','an'], length=10.
# spaces=6, gaps=2. base=3,extra=0.
# row='This is an'. len=16 ✓
# Last line='example '.ljust(16) ✓
#
# Single word line: 'word'.ljust(16) →
# padded with spaces ✓
#
# "No bugs. base + (1 if j <= extra else
# 0) distributes extra spaces to left gaps."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \cdot \text{maxWidth})$ :  $O(n)$  lines, each
#  $O(\text{maxWidth})$  to build. Space  $O(n)$  output.
# Follow-up: minimize raggedness
# (aesthetic) – DP approach balances line
# lengths.
# Follow-up: proportional fonts – use
# pixel widths instead of character counts."
```